AuxiRuta

Código explicado, archivo por archivo

Guía de lectura técnica: Java, SQL, seguridad, integración, configuración, despliegue, pruebas y empaquetado.

Edición: 13 de noviembre de 2026. Fuente: C:/Users/Enzo/BACK-TPI-G1. Describe la implementación encontrada; no sustituye una auditoría ni supone centralización completa de reglas.

Cada archivo incluye ruta, responsabilidad concreta y explicación del código. Se reproduce cada línea no vacía del código propio con su numeración actual. Las expresiones que abarcan varias líneas se explican como una unidad, indicando sus líneas y el significado de sus componentes; las líneas vacías son separadores. Los tests se resumen por clase/casos. Maven Wrapper es herramienta de terceros y se explica por función. Los JAR se inspeccionan como ZIP de bytecode, no texto fuente.

Exclusiones: .git, .gitignore, .dockerignore, documentación/diagramas previos, .env real, temporales y clases/reportes/cachés de target. Excepción: JAR presentes y sus originales. No se incluyeron secretos de la instalación.

Ruta de lectura: construcción -> Commons/seguridad -> persistencia -> flota/solicitudes -> evaluación -> propuestas -> ejecución/automatización -> pruebas/JAR. El índice y los marcadores del PDF permiten saltar directamente a cada archivo.

Índice de temas y archivos

1. Construcción, despliegue y entorno 7

.env.example . . . 7
api-gateway/Dockerfile . . . 8
api-gateway/pom.xml . . . 9
assistance-service/Dockerfile . . . 14
assistance-service/pom.xml . . . 15
auxiruta-commons/pom.xml . . . 21
docker-compose.yml . . . 24
fleet-service/Dockerfile . . . 33
fleet-service/pom.xml . . . 34
infra/postgres/01-databases.sh . . . 39
scripts/Format-Java.ps1 . . . 41
scripts/Start-AuxiRuta.ps1 . . . 43
scripts/Verify-AuxiRuta.ps1 . . . 46

2. Biblioteca común 49

GeoMath (auxiruta-commons) . . . 49
Problems (auxiruta-commons) . . . 50
TokenSupport (auxiruta-commons) . . . 51
UtcClock (auxiruta-commons) . . . 55

3. Arranque, seguridad, Gateway y errores 57

GatewayApplication (api-gateway) . . . 57
GatewayRoutes (api-gateway) . . . 58
RequestLoggingFilter (api-gateway) . . . 60
SecurityConfiguration (api-gateway) . . . 61
SecurityErrors (api-gateway) . . . 67
application-docker (api-gateway) . . . 71
application (api-gateway) . . . 71
AssistanceApplication (assistance-service) . . . 72
ClockConfiguration (assistance-service) . . . 73
OpenApiConfiguration (assistance-service) . . . 74
RequestLoggingFilter (assistance-service) . . . 75
SecurityConfiguration (assistance-service) . . . 77
SecurityErrors (assistance-service) . . . 82
ApiExceptionHandler (assistance-service) . . . 85
application-docker (assistance-service) . . . 88
application (assistance-service) . . . 91
FleetApplication (fleet-service) . . . 92
ClockConfiguration (fleet-service) . . . 93
OpenApiConfiguration (fleet-service) . . . 94
RequestLoggingFilter (fleet-service) . . . 95

SecurityConfiguration (fleet-service) . . . 97
SecurityErrors (fleet-service) . . . 102
ApiExceptionHandler (fleet-service) . . . 105
application-docker (fleet-service) . . . 108
application (fleet-service) . . . 110
auxiruta-realm (infra) . . . 111

4. Persistencia: entidades, repositorios y SQL 128

AssistanceRequest (assistance-service) . . . 128
CandidateEvaluation (assistance-service) . . . 134
CandidateEvaluationRepository (assistance-service) . . . 140
Evaluation (assistance-service) . . . 140
EvaluationRepository (assistance-service) . . . 146
HistoryRepository (assistance-service) . . . 146
RequestRepository (assistance-service) . . . 147
RequestStatusHistory (assistance-service) . . . 149
Vehicle (assistance-service) . . . 152
VehicleRepository (assistance-service) . . . 156
001-assistance (assistance-service) . . . 157
002-evaluation (assistance-service) . . . 162
003-assignments (assistance-service) . . . 167
004-execution (assistance-service) . . . 170
005-automation (assistance-service) . . . 170
Operator (fleet-service) . . . 172
OperatorRepository (fleet-service) . . . 175
ReservationRepository (fleet-service) . . . 177
ResourceReservation (fleet-service) . . . 178
TowTruck (fleet-service) . . . 181
TowTruckRepository (fleet-service) . . . 184
TruckStatusHistory (fleet-service) . . . 186
TruckStatusHistoryRepository (fleet-service) . . . 189
001-fleet-catalog (fleet-service) . . . 189
002-reservations (fleet-service) . . . 194
003-automation (fleet-service) . . . 196

5. Flota: administración, candidatos y reservas 197

CandidateController (fleet-service) . . . 197
CandidateService (fleet-service) . . . 199
FleetController (fleet-service) . . . 204
FleetDtos (fleet-service) . . . 215
FleetService (fleet-service) . . . 221
FleetTypes (fleet-service) . . . 245
ReservationController (fleet-service) . . . 246
ReservationJobs (fleet-service) . . . 250
ReservationService (fleet-service) . . . 253

6. Vehículos, solicitudes y reglas 278

DomainTypes (assistance-service) . . . 278
RequestController (assistance-service) . . . 279
RequestDtos (assistance-service) . . . 288
RequestService (assistance-service) . . . 294
RulesConfiguration (assistance-service) . . . 319
mvp (assistance-service) . . . 323

7. Evaluación, rutas e integración externa 329

DispatchValidator (assistance-service) . . . 329
EvaluationReadService (assistance-service) . . . 338
EvaluationService (assistance-service) . . . 341
EvaluationStore (assistance-service) . . . 360
FleetClient (assistance-service) . . . 368
OpenRouteServiceClient (assistance-service) . . . 378
RouteProvider (assistance-service) . . . 391
RoutingBudget (assistance-service) . . . 392
RoutingFailure (assistance-service) . . . 395

8. Propuestas, aceptación y recuperación durable 398

AssignmentController (assistance-service) . . . 398
AssignmentCoordinator (assistance-service) . . . 402
AssignmentStore (assistance-service) . . . 409

9. Ejecución y automatización 436

AutomationJobs (assistance-service) . . . 436
AutomationStore (assistance-service) . . . 442
ExecutionController (assistance-service) . . . 459
ExecutionService (assistance-service) . . . 465
ExecutionStore (assistance-service) . . . 467

10. Pruebas y utilidades de demostración 482

BackendClient (api-gateway) . . . 482
BackendIntegrationTest (api-gateway) . . . 482
DeliveryArtifacts (api-gateway) . . . 482
DeliveryArtifactsTest (api-gateway) . . . 482
DeliveryIntegrationTest (api-gateway) . . . 482
DemoDataSetupTest (api-gateway) . . . 483
KeycloakScopeSetupTest (api-gateway) . . . 483
Phase7IntegrationTest (api-gateway) . . . 483
Phase8IntegrationTest (api-gateway) . . . 483
Phase9IntegrationTest (api-gateway) . . . 484
PrivacyDemoSetupTest (api-gateway) . . . 484
SecurityConfigurationTest (assistance-service) . . . 484
AssignmentCoordinatorTest (assistance-service) . . . 485
AutomationConfigurationTest (assistance-service) . . . 485

AutomationStoreTest (assistance-service) . . . 485
DispatchValidatorTest (assistance-service) . . . 485
EvaluationServiceTest (assistance-service) . . . 485
EvaluationStoreTest (assistance-service) . . . 486
ExecutionRulesTest (assistance-service) . . . 486
ExecutionServiceTest (assistance-service) . . . 486
RequestRulesTest (assistance-service) . . . 486
OpenRouteServiceClientTest (assistance-service) . . . 487
RoutingBudgetTest (assistance-service) . . . 487
TokenSupportTest (auxiruta-commons) . . . 487
CandidateServiceTest (fleet-service) . . . 487
FleetServiceTest (fleet-service) . . . 488
ReservationServiceTest (fleet-service) . . . 488
SecurityConfigurationTest (fleet-service) . . . 488

11. Maven Wrapper: herramientas distribuidas 489

api-gateway/.mvn/wrapper/maven-wrapper.properties . . . 489
api-gateway/mvnw . . . 489
api-gateway/mvnw.cmd . . . 489
assistance-service/.mvn/wrapper/maven-wrapper.properties . . . 489
assistance-service/mvnw . . . 490
assistance-service/mvnw.cmd . . . 490
auxiruta-commons/.mvn/wrapper/maven-wrapper.properties . . . 490
auxiruta-commons/mvnw . . . 491
auxiruta-commons/mvnw.cmd . . . 491
fleet-service/.mvn/wrapper/maven-wrapper.properties . . . 491
fleet-service/mvnw . . . 491
fleet-service/mvnw.cmd . . . 492

12. JAR: binarios y contenido empaquetado 493

api-gateway/target/app.jar . . . 493
api-gateway/target/app.jar.original . . . 494
assistance-service/target/app.jar . . . 494
assistance-service/target/app.jar.original . . . 497
auxiruta-commons/target/auxiruta-commons-1.0.0.jar . . . 499
fleet-service/target/app.jar . . . 500
fleet-service/target/app.jar.original . . . 502

13. Glosario de lectura 504

Claves para leer el código

YAML: clave, mapa y lista

La sangría establece jerarquía. clave: valor fija un escalar; clave: sin valor abre un mapa de hijos. Un guion introduce un elemento de lista. El consumidor determina el significado: logging en Spring y logging en Compose no son la misma propiedad.

YAML: ancla, alias y mezcla

&nombre pone un nombre a un nodo. *nombre lo referencia. <<: *nombre mezcla claves de un mapa en otro. No son variables de entorno ni servicios. Una extensión x- de Compose guarda fragmentos que por sí solos no crean contenedores.

Variables de entorno

Compose usa expansiones con :- para un valor por defecto y :? para exigir un valor. Spring usa : dentro de su placeholder para un valor por defecto. Cada motor interpreta su propio archivo en momentos distintos.

Java: referencia y objeto

new construye un objeto; una variable contiene la referencia. final evita reasignar esa referencia, pero no vuelve inmutable cualquier objeto. Un record genera campos finales, constructor y accesores; si contiene una colección mutable, eso no la vuelve profundamente inmutable.

Java: llamadas y lambdas

obj.metodo(...) delega una operación a un objeto. -> define una lambda usada por streams, seguridad o callbacks. :: referencia un método sin llamarlo inmediatamente. El contexto determina cuándo se ejecuta.

Persistencia y efectos

Asignar un campo de una entidad no equivale a confirmar una transacción. JPA sincroniza al hacer flush/commit. JDBC ejecuta SQL, pero la transacción local decide confirmación. Un HTTP remoto no se revierte automáticamente al hacer rollback de la base local.

Cómo leer las líneas

Cada bloque muestra los números reales del archivo. Una llamada distribuida en varias líneas se explica completa para no describir sus argumentos aislados de su finalidad. Las continuaciones relevantes se detallan por número; llaves y separadores no tienen un efecto de negocio independiente.

1. Construcción, despliegue y entorno

Commons se instala antes de compilar dependientes. Docker construye y arranca cinco contenedores; no hace falta frontend para ejecutar el proceso.

.env.example

Ruta: .env.example

Objetivo y funcionalidad

Indica a quien clona el proyecto qué contraseñas y claves debe completar en su .env local para iniciar las bases, Keycloak, usuarios demo y OpenRouteService. Los valores CAMBIAR son marcadores; no contiene los secretos del equipo ni se usa como token de Postman.

L1

```
1 | # Copiar a .env y reemplazar cada valor antes del primer arranque.
```

Comentario o directiva del intérprete; no es regla de negocio.

L2

```
2 | POSTGRES_ADMIN_PASSWORD=CAMBIAR_POSTGRES_ADMIN_PASSWORD
```

Variable POSTGRES_ADMIN_PASSWORD. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L3

```
3 | ASSISTANCE_DB_PASSWORD=CAMBIAR_ASSISTANCE_DB_PASSWORD
```

Variable ASSISTANCE_DB_PASSWORD. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L4

```
4 | FLEET_DB_PASSWORD=CAMBIAR_FLEET_DB_PASSWORD
```

Variable FLEET_DB_PASSWORD. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L5

```
5 | KEYCLOAK_DB_PASSWORD=CAMBIAR_KEYCLOAK_DB_PASSWORD
```

Variable KEYCLOAK_DB_PASSWORD. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L6

```
6 | KC_BOOTSTRAP_ADMIN_PASSWORD=CAMBIAR_KC_BOOTSTRAP_ADMIN_PASSWORD
```

Variable KC_BOOTSTRAP_ADMIN_PASSWORD. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L7

```
7 | KEYCLOAK_CLIENT_SECRET=CAMBIAR_KEYCLOAK_CLIENT_SECRET
```

Variable KEYCLOAK_CLIENT_SECRET. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L8

```
8 | DEMO_DRIVER_PASSWORD=CAMBIAR_DEMO_DRIVER_PASSWORD
```

Variable DEMO_DRIVER_PASSWORD. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L9

```
9 | DEMO_OPERATOR_PASSWORD=CAMBIAR_DEMO_OPERATOR_PASSWORD
```

Variable DEMO_OPERATOR_PASSWORD. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L10

```
10 | DEMO_ADMIN_PASSWORD=CAMBIAR_DEMO_ADMIN_PASSWORD
```

Variable DEMO_ADMIN_PASSWORD. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L12

```
12 | # OpenRouteService (local secret; never commit a real key)
```

Comentario o directiva del intérprete; no es regla de negocio.

L13

```
13 | ORS_API_KEY=
```

Variable ORS_API_KEY. Clave externa obtenida de ORS.

L14

```
14 | ORS_BASE_URL=https://api.heigit.org/openrouteservice/v2
```

Variable ORS_BASE_URL. Configuración pública del servicio; conservar el default salvo cambio de entorno.

L16

```
16 | # Normal: docker. Demo: docker,demo. Temporal quotas remain unchanged.
```

Comentario o directiva del intérprete; no es regla de negocio.

L17

```
17 | SPRING_PROFILES_ACTIVE=docker
```

Variable SPRING_PROFILES_ACTIVE. Configuración pública del servicio; conservar el default salvo cambio de entorno.

L18

```
18 | AUXIRUTA_AUTOMATION_ENABLED=true
```

Variable AUXIRUTA_AUTOMATION_ENABLED. Configuración pública del servicio; conservar el default salvo cambio de entorno.

Líneas vacías 11, 15: separadores sin instrucciones.

api-gateway/Dockerfile

`Ruta: api-gateway/Dockerfile`

Objetivo y funcionalidad

Construye api-gateway en dos etapas: JDK 21 instala Commons y verifica el módulo; la imagen final usa JRE, app.jar y un usuario sin privilegios. Evita transportar herramientas de compilación al servidor.

L1

```
 1 | FROM eclipse-temurin:21-jdk-jammy AS build
```

Etapa build: imagen Temurin con JDK 21 y base Ubuntu Jammy. JDK incluye compilador; AS build permite recuperar luego el artefacto mediante COPY --from=build.

L2

```
 2 | WORKDIR /workspace
```

Crea/selecciona /workspace como directorio de trabajo para instrucciones siguientes. Rutas relativas de RUN y destinos COPY se resuelven aquí.

L3

```
 3 | COPY auxiruta-commons/ auxiruta-commons/
```

Introduce la carpeta auxiruta-commons/ del contexto en la etapa de construcción. Commons debe estar disponible para compilar el módulo dependiente; no se copia el .env real.

L4

```
 4 | COPY api-gateway/ api-gateway/
```

Introduce la carpeta api-gateway/ del contexto en la etapa de construcción. Commons debe estar disponible para compilar el módulo dependiente; no se copia el .env real.

L5

```
5 | COPY docs/api/ docs/api/
```

Introduce la carpeta docs/api/ del contexto en la etapa de construcción. Commons debe estar disponible para compilar el módulo dependiente; no se copia el .env real.

L6

```
6 | RUN chmod +x auxiruta-commons/mvnw api-gateway/mvnw && ./auxiruta-commons/mvnw -B -ntp -f
    auxiruta-commons/pom.xml install && ./api-gateway/mvnw -B -ntp -f api-gateway/pom.xml verify
```

Durante docker build da permiso de ejecución a los Wrapper, instala Commons en el repositorio Maven de esa etapa y ejecuta verify del módulo. && detiene la cadena si algo falla; -B evita interacción, -ntp suprime progreso y -f selecciona el POM.

L7

```
7 | FROM eclipse-temurin:21-jre-jammy
```

Comienza imagen final independiente con JRE 21. Conserva runtime Java sin el compilador ni el repositorio de fuentes de la etapa anterior.

L8

```
8 | RUN groupadd --system auxiruta && useradd --system --gid auxiruta --home-dir /app auxiruta
```

Crea grupo y usuario de sistema auxiruta dentro de la imagen. --system marca cuenta de servicio; --gid asocia grupo y --home-dir fija /app como directorio personal.

L9

```
9 | WORKDIR /app
```

Crea/selecciona /app como directorio de trabajo para instrucciones siguientes. Rutas relativas de RUN y destinos COPY se resuelven aquí.

L10

```
10 | COPY --from=build /workspace/api-gateway/target/app.jar app.jar
```

Copia únicamente app.jar desde la etapa build a la imagen final. Este JAR ya incluye launcher y dependencias Boot; los tests y herramientas de compilación no se trasladan.

L11

```
11 | USER auxiruta
```

Las instrucciones de ejecución siguientes y el proceso final usan auxiruta en vez de root, reduciendo privilegios del backend.

L12

```
12 | ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Forma exec JSON: Docker inicia java directamente con -jar /app/app.jar, sin un shell intermedio. El launcher Boot arranca la aplicación del módulo.

api-gateway/pom.xml

`Ruta: api-gateway/pom.xml`

Objetivo y funcionalidad

Define la construcción de el Gateway reactivo y su seguridad JWT. Maven lee coordenadas, versiones, dependencias y plugins para compilar, probar y empaquetar el módulo.

L1

```
1 | <?xml version="1.0" ?>
```

Declara sintaxis XML.

L2

```
2 | <project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

Abre bloque del modelo Maven y agrupa sus hijos.

L3

```
3 |     <modelVersion>4.0.0</modelVersion>
```

Versión del modelo POM, no de Maven. Valor: 4.0.0.

L4

```
4 |     <parent>
```

Herencia del padre que administra plugins y versiones.

L5

```
5 |         <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L6

```
6 |         <artifactId>spring-boot-starter-parent</artifactId>
```

Padre que administra versiones coherentes y valores por defecto de construcción Spring Boot.

L7

```
7 |         <version>3.5.16</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 3.5.16.

L8

```
8 |         <relativePath/>
```

La ruta vacía impide buscar el padre en ../pom.xml; Maven resuelve spring-boot-starter-parent desde repositorios.

L9

```
9 |     </parent>
```

Termina la sección Maven parent. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L10

```
10 |     <groupId>com.auxiruta</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: com.auxiruta.

L11

```
11 |     <artifactId>api-gateway</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: api-gateway.

L12

```
12 |     <version>1.0.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 1.0.0.

L13

```
13 |     <properties>
```

Variables de la construcción.

L14

```
14 |         <java.version>21</java.version>
```

Versión de Java para compilar. Valor: 21.

L15

```
15 |         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

Codificación del fuente. Valor: UTF-8.

L16

```
16 |     </properties>
```

Termina la sección Maven properties. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L17

```
17 |     <dependencyManagement>
```

Administra versiones mediante BOM sin añadir por sí mismo bibliotecas al classpath.

L18

```
18 |         <dependencies>
```

Lista de dependencias.

L19

```
19 |             <dependency>
```

Declara una dependencia Maven.

L20

```
20 |                 <groupId>org.springframework.cloud</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.cloud.

L21

```
21 |                 <artifactId>spring-cloud-dependencies</artifactId>
```

BOM que coordina versiones Spring Cloud; no añade por sí mismo librerías ejecutables.

L22

```
22 |                 <version>2025.0.3</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 2025.0.3.

L23

```
23 |                 <type>pom</type>
```

Indica que el artefacto es un POM, no una biblioteca JAR. Con scope import dentro de dependencyManagement incorpora el BOM de versiones.

L24

```
24 |                 <scope>import</scope>
```

Importa versiones de un BOM dentro de dependencyManagement; no carga código ni agrega todas sus dependencias.

L25

```
25 |             </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L26

```
26 |         </dependencies>
```

Termina la sección Maven dependencies. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L27

```
27 |     </dependencyManagement>
```

Termina la sección Maven dependencyManagement. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L28

```
28 |     <dependencies>
```

Lista de dependencias.

L29

```
29 |         <dependency>
```

Declara una dependencia Maven.

L30

```
30 |             <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L31

```
31 |             <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
```

Aporta recepción de Bearer JWT y validación de tokens.

L32

```
32 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L33

```
33 |         <dependency>
```

Declara una dependencia Maven.

L34

```
34 |             <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L35

```
35 |             <artifactId>spring-boot-starter-test</artifactId>
```

Incluye JUnit y herramientas de prueba; scope test lo excluye de ejecución productiva.

L36

```
36 |             <scope>test</scope>
```

Solo disponible en compilación/ejecución de tests. No se incorpora como dependencia productiva al JAR Boot.

L37

```
37 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L38

```
38 |         <dependency>
```

Declara una dependencia Maven.

L39

```
39 |             <groupId>org.springframework.security</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.security.

L40

```
40 |             <artifactId>spring-security-test</artifactId>
```

Permite simular identidades y verificar permisos en pruebas aisladas.

L41

```
41 |             <scope>test</scope>
```

Solo disponible en compilación/ejecución de tests. No se incorpora como dependencia productiva al JAR Boot.

L42

```
42 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L43

```
43 |         <dependency>
```

Declara una dependencia Maven.

L44

```
44 |             <groupId>org.springframework.cloud</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.cloud.

L45

```
45 |             <artifactId>spring-cloud-starter-gateway-server-webflux</artifactId>
```

Implementa el Gateway reactivo de enrutamiento HTTP.

L46

```
46 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L47

```
47 |         <dependency>
```

Declara una dependencia Maven.

L48

```
48 |             <groupId>com.auxiruta</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: com.auxiruta.

L49

```
49 |             <artifactId>auxiruta-commons</artifactId>
```

Biblioteca local de seguridad, errores, reloj y geografía; debe construirse antes de los dependientes.

L50

```
50 |             <version>1.0.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 1.0.0.

L51

```
51 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L52

```
52 |     </dependencies>
```

Termina la sección Maven dependencies. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L53

```
53 |     <build>
```

Abre bloque del modelo Maven y agrupa sus hijos.

L54

```
54 |         <finalName>app</finalName>
```

Fija app como nombre de artefacto bajo target. Los Dockerfiles esperan target/app.jar, por lo que el nombre debe coincidir.

L55

```
55 |         <plugins>
```

Abre bloque del modelo Maven y agrupa sus hijos.

L56

```
56 |             <plugin>
```

Plugin de construcción; el de Spring Boot reempaqueta el ejecutable.

L57

```
57 |                 <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L58

```
58 |                 <artifactId>spring-boot-maven-plugin</artifactId>
```

Reempaqueta el JAR con launcher/dependencias para ejecutarlo con java -jar.

L59

```
59 |             </plugin>
```

Termina la sección Maven plugin. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L60

```
60 |         </plugins>
```

Termina la sección Maven plugins. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L61

```
61 |     </build>
```

Termina la sección Maven build. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L62

```
62 | </project>
```

Termina la sección Maven project. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

assistance-service/Dockerfile

`Ruta: assistance-service/Dockerfile`

Objetivo y funcionalidad

Construye assistance-service en dos etapas: JDK 21 instala Commons y verifica el módulo; la imagen final usa JRE, app.jar y un usuario sin privilegios. Evita transportar herramientas de compilación al servidor.

L1

```
 1 | FROM eclipse-temurin:21-jdk-jammy AS build
```

Etapa build: imagen Temurin con JDK 21 y base Ubuntu Jammy. JDK incluye compilador; AS build permite recuperar luego el artefacto mediante COPY --from=build.

L2

```
 2 | WORKDIR /workspace
```

Crea/selecciona /workspace como directorio de trabajo para instrucciones siguientes. Rutas relativas de RUN y destinos COPY se resuelven aquí.

L3

```
 3 | COPY auxiruta-commons/ auxiruta-commons/
```

Introduce la carpeta auxiruta-commons/ del contexto en la etapa de construcción. Commons debe estar disponible para compilar el módulo dependiente; no se copia el .env real.

L4

```
4 | COPY assistance-service/ assistance-service/
```

Introduce la carpeta assistance-service/ del contexto en la etapa de construcción. Commons debe estar disponible para compilar el módulo dependiente; no se copia el .env real.

L5

```
5 | RUN chmod +x auxiruta-commons/mvnw assistance-service/mvnw && ./auxiruta-commons/mvnw -B -ntp -f
    auxiruta-commons/pom.xml install && ./assistance-service/mvnw -B -ntp -f
    assistance-service/pom.xml verify
```

Durante docker build da permiso de ejecución a los Wrapper, instala Commons en el repositorio Maven de esa etapa y ejecuta verify del módulo. && detiene la cadena si algo falla; -B evita interacción, -ntp suprime progreso y -f selecciona el POM.

L6

```
6 | FROM eclipse-temurin:21-jre-jammy
```

Comienza imagen final independiente con JRE 21. Conserva runtime Java sin el compilador ni el repositorio de fuentes de la etapa anterior.

L7

```
7 | RUN groupadd --system auxiruta && useradd --system --gid auxiruta --home-dir /app auxiruta
```

Crea grupo y usuario de sistema auxiruta dentro de la imagen. --system marca cuenta de servicio; --gid asocia grupo y --home-dir fija /app como directorio personal.

L8

```
8 | WORKDIR /app
```

Crea/selecciona /app como directorio de trabajo para instrucciones siguientes. Rutas relativas de RUN y destinos COPY se resuelven aquí.

L9

```
9 | COPY --from=build /workspace/assistance-service/target/app.jar app.jar
```

Copia únicamente app.jar desde la etapa build a la imagen final. Este JAR ya incluye launcher y dependencias Boot; los tests y herramientas de compilación no se trasladan.

L10

```
10 | USER auxiruta
```

Las instrucciones de ejecución siguientes y el proceso final usan auxiruta en vez de root, reduciendo privilegios del backend.

L11

```
11 | ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Forma exec JSON: Docker inicia java directamente con -jar /app/app.jar, sin un shell intermedio. El launcher Boot arranca la aplicación del módulo.

assistance-service/pom.xml

`Ruta: assistance-service/pom.xml`

Objetivo y funcionalidad

Define la construcción de solicitudes, propuestas y ejecución con persistencia y cliente OAuth. Maven lee coordenadas, versiones, dependencias y plugins para compilar, probar y empaquetar el módulo.

L1

```
1 | <?xml version="1.0" ?>
```

Declara sintaxis XML.

L2

```
2 | <project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

Abre bloque del modelo Maven y agrupa sus hijos.

L3

```
3 |     <modelVersion>4.0.0</modelVersion>
```

Versión del modelo POM, no de Maven. Valor: 4.0.0.

L4

```
4 |     <parent>
```

Herencia del padre que administra plugins y versiones.

L5

```
5 |         <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L6

```
6 |         <artifactId>spring-boot-starter-parent</artifactId>
```

Padre que administra versiones coherentes y valores por defecto de construcción Spring Boot.

L7

```
7 |         <version>3.5.16</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 3.5.16.

L8

```
8 |         <relativePath/>
```

La ruta vacía impide buscar el padre en ../pom.xml; Maven resuelve spring-boot-starter-parent desde repositorios.

L9

```
9 |     </parent>
```

Termina la sección Maven parent. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L10

```
10 |     <groupId>com.auxiruta</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: com.auxiruta.

L11

```
11 |     <artifactId>assistance-service</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: assistance-service.

L12

```
12 |     <version>1.0.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 1.0.0.

L13

```
13 |     <properties>
```

Variables de la construcción.

L14

```
14 |         <java.version>21</java.version>
```

Versión de Java para compilar. Valor: 21.

L15

```
15 |         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

Codificación del fuente. Valor: UTF-8.

L16

```
16 |     </properties>
```

Termina la sección Maven properties. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L17

```
17 |     <dependencies>
```

Lista de dependencias.

L18

```
18 |         <dependency>
```

Declara una dependencia Maven.

L19

```
19 |             <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L20

```
20 |             <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
```

Aporta recepción de Bearer JWT y validación de tokens.

L21

```
21 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L22

```
22 |         <dependency>
```

Declara una dependencia Maven.

L23

```
23 |             <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L24

```
24 |             <artifactId>spring-boot-starter-web</artifactId>
```

Incluye Spring MVC, JSON y servidor Servlet embebido para REST.

L25

```
25 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L26

```
26 |         <dependency>
```

Declara una dependencia Maven.

L27

```
27 |             <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L28

```
28 |             <artifactId>spring-boot-starter-data-jpa</artifactId>
```

Incluye repositorios Spring Data, Hibernate y transacciones de persistencia.

L29

```
29 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L30

```
30 |         <dependency>
```

Declara una dependencia Maven.

L31

```
31 |             <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L32

```
32 |             <artifactId>spring-boot-starter-validation</artifactId>
```

Activa restricciones Bean Validation de los DTO, como NotBlank o Size.

L33

```
33 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L34

```
34 |         <dependency>
```

Declara una dependencia Maven.

L35

```
35 |             <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L36

```
36 |             <artifactId>spring-boot-starter-oauth2-client</artifactId>
```

Aporta infraestructura de cliente OAuth para identidades de servicio.

L37

```
37 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L38

```
38 |         <dependency>
```

Declara una dependencia Maven.

L39

```
39 |             <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L40

```
40 |             <artifactId>spring-boot-starter-test</artifactId>
```

Incluye JUnit y herramientas de prueba; scope test lo excluye de ejecución productiva.

L41

```
41 |             <scope>test</scope>
```

Solo disponible en compilación/ejecución de tests. No se incorpora como dependencia productiva al JAR Boot.

L42

```
42 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L43

```
43 |         <dependency>
```

Declara una dependencia Maven.

L44

```
44 |             <groupId>org.springframework.security</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.security.

L45

```
45 |             <artifactId>spring-security-test</artifactId>
```

Permite simular identidades y verificar permisos en pruebas aisladas.

L46

```
46 |             <scope>test</scope>
```

Solo disponible en compilación/ejecución de tests. No se incorpora como dependencia productiva al JAR Boot.

L47

```
47 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L48

```
48 |         <dependency>
```

Declara una dependencia Maven.

L49

```
49 |             <groupId>org.postgresql</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.postgresql.

L50

```
50 |             <artifactId>postgresql</artifactId>
```

Driver JDBC de comunicación con PostgreSQL; no instala el servidor SQL.

L51

```
51 |             <scope>runtime</scope>
```

Dependencia necesaria al ejecutar pero no para compilar contra sus tipos; el driver PostgreSQL es descubierto a través de JDBC.

L52

```
52 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L53

```
53 |         <dependency>
```

Declara una dependencia Maven.

L54

```
54 |             <groupId>org.springdoc</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springdoc.

L55

```
55 |             <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
```

Genera OpenAPI de MVC e incorpora recursos Swagger UI.

L56

```
56 |             <version>2.10.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 2.10.0.

L57

```
57 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L58

```
58 |         <dependency>
```

Declara una dependencia Maven.

L59

```
59 |             <groupId>com.auxiruta</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: com.auxiruta.

L60

```
60 |             <artifactId>auxiruta-commons</artifactId>
```

Biblioteca local de seguridad, errores, reloj y geografía; debe construirse antes de los dependientes.

L61

```
61 |             <version>1.0.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 1.0.0.

L62

```
62 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L63

```
63 |     </dependencies>
```

Termina la sección Maven dependencies. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L64

```
64 |     <build>
```

Abre bloque del modelo Maven y agrupa sus hijos.

L65

```
65 |         <finalName>app</finalName>
```

Fija app como nombre de artefacto bajo target. Los Dockerfiles esperan target/app.jar, por lo que el nombre debe coincidir.

L66

```
66 |         <plugins>
```

Abre bloque del modelo Maven y agrupa sus hijos.

L67

```
67 |             <plugin>
```

Plugin de construcción; el de Spring Boot reempaqueta el ejecutable.

L68

```
68 |                 <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L69

```
69 |                 <artifactId>spring-boot-maven-plugin</artifactId>
```

Reempaqueta el JAR con launcher/dependencias para ejecutarlo con java -jar.

L70

```
70 |             </plugin>
```

Termina la sección Maven plugin. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L71

```
71 |         </plugins>
```

Termina la sección Maven plugins. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L72

```
72 |     </build>
```

Termina la sección Maven build. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L73

```
73 | </project>
```

Termina la sección Maven project. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

auxiruta-commons/pom.xml

`Ruta: auxiruta-commons/pom.xml`

Objetivo y funcionalidad

Define la construcción de la biblioteca compartida, que no inicia un servidor. Maven lee coordenadas, versiones, dependencias y plugins para compilar, probar y empaquetar el módulo.

L1

```
1 | <?xml version="1.0" ?>
```

Declara sintaxis XML.

L2

```
2 | <project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

Abre bloque del modelo Maven y agrupa sus hijos.

L3

```
3 |     <modelVersion>4.0.0</modelVersion>
```

Versión del modelo POM, no de Maven. Valor: 4.0.0.

L4

```
4 |     <parent>
```

Herencia del padre que administra plugins y versiones.

L5

```
5 |         <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L6

```
6 |         <artifactId>spring-boot-starter-parent</artifactId>
```

Padre que administra versiones coherentes y valores por defecto de construcción Spring Boot.

L7

```
7 |         <version>3.5.16</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 3.5.16.

L8

```
8 |         <relativePath/>
```

La ruta vacía impide buscar el padre en ../pom.xml; Maven resuelve spring-boot-starter-parent desde repositorios.

L9

```
9 |     </parent>
```

Termina la sección Maven parent. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L10

```
10 |     <groupId>com.auxiruta</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: com.auxiruta.

L11

```
11 |     <artifactId>auxiruta-commons</artifactId>
```

Biblioteca local de seguridad, errores, reloj y geografía; debe construirse antes de los dependientes.

L12

```
12 |     <version>1.0.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 1.0.0.

L13

```
13 |     <properties>
```

Variables de la construcción.

L14

```
14 |         <java.version>21</java.version>
```

Versión de Java para compilar. Valor: 21.

L15

```
15 |         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

Codificación del fuente. Valor: UTF-8.

L16

```
16 |     </properties>
```

Termina la sección Maven properties. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L17

```
17 |     <dependencies>
```

Lista de dependencias.

L18

```
18 |         <dependency>
```

Declara una dependencia Maven.

L19

```
19 |             <groupId>org.springframework</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.

L20

```
20 |             <artifactId>spring-web</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-web.

L21

```
21 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L22

```
22 |         <dependency>
```

Declara una dependencia Maven.

L23

```
23 |             <groupId>org.springframework.security</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.security.

L24

```
24 |             <artifactId>spring-security-oauth2-jose</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-security-oauth2-jose.

L25

```
25 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L26

```
26 |         <dependency>
```

Declara una dependencia Maven.

L27

```
27 |             <groupId>org.springframework.security</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.security.

L28

```
28 |             <artifactId>spring-security-core</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-security-core.

L29

```
29 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L30

```
30 |         <dependency>
```

Declara una dependencia Maven.

L31

```
31 |             <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L32

```
32 |             <artifactId>spring-boot-starter-test</artifactId>
```

Incluye JUnit y herramientas de prueba; scope test lo excluye de ejecución productiva.

L33

```
33 |             <scope>test</scope>
```

Solo disponible en compilación/ejecución de tests. No se incorpora como dependencia productiva al JAR Boot.

L34

```
34 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L35

```
35 |     </dependencies>
```

Termina la sección Maven dependencies. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L36

```
36 | </project>
```

Termina la sección Maven project. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

docker-compose.yml

`Ruta: docker-compose.yml`

Objetivo y funcionalidad

Permite levantar AuxiRuta con un único comando: conecta PostgreSQL, Keycloak, Fleet, Assistance y Gateway, inyecta credenciales por entorno, conserva las bases en un volumen y publica solo Gateway/Keycloak. Las dependencias de salud ordenan el arranque y las anclas comparten configuración sin duplicarla.

L1

```
1 | name: auxiruta
```

Asigna auxiruta como nombre del proyecto Compose; agrupa contenedores, red y volumen. No es un servicio ni el nombre de un usuario.

L2

```
2 | x-logging: &logging
```

Define una extensión Compose: x- permite guardar configuración reutilizable sin crear un servicio. &logging es un ancla YAML, un nombre para el mapa driver/options que sigue, no un valor de texto. Cada logging: *logging reutiliza ese mapa para compartir la rotación de logs entre los cinco contenedores.

L3

```
3 |   driver: json-file
```

json-file es el controlador Docker que guarda stdout/stderr en archivos JSON administrados por el motor. No obliga a que los mensajes de Spring sean JSON ni crea una tabla de logs.

L4

```
4 |   options:
```

Agrupa parámetros del controlador de logs. max-size y max-file trabajan juntos para acotar el almacenamiento por contenedor.

L5

```
5 |     max-size: "10m"
```

"10m" indica 10 MB por archivo de log antes de rotar; aquí m es unidad de tamaño, no minutos. Limita el crecimiento de cada archivo.

L6

```
6 |     max-file: "3"
```

"3" conserva hasta tres archivos de la rotación por contenedor. Al superar el límite se descartan antiguos; no es un historial permanente de negocio.

L7

```
7 | x-jwt: &jwt
```

Define una extensión cuyo mapa recibe el ancla &jwt. Los servicios mezclan sus variables mediante <<: *jwt; esto evita repetir perfil, automatización y direcciones de seguridad. No emite tokens.

L8

```
8 |   SPRING_PROFILES_ACTIVE: ${SPRING_PROFILES_ACTIVE:-docker}
```

Selecciona configuraciones Spring. docker activa conexiones de contenedores; docker,demo habilita los tiempos reducidos interpretados por RulesConfiguration. Compose usa SPRING_PROFILES_ACTIVE si existe y no está vacía; de otro modo usa 'docker'.

L9

```
9 |   AUXIRUTA_AUTOMATION_ENABLED: ${AUXIRUTA_AUTOMATION_ENABLED:-true}
```

Activa/desactiva tareas periódicas de avance y recuperación. No equivale a poner disponible un operador. Compose usa AUXIRUTA_AUTOMATION_ENABLED si existe y no está vacía; de otro modo usa 'true'.

L10

```
10 |   KEYCLOAK_ISSUER_URI: http://localhost:8080/realms/auxiruta
```

Emisor que debe coincidir con iss del JWT. Conserva la URL pública aunque las claves se obtengan por DNS interno.

L11

```
11 |   KEYCLOAK_JWK_SET_URI: http://keycloak:8080/realms/auxiruta/protocol/openid-connect/certs
```

Endpoint interno JWKS de claves públicas para verificar firmas; no contiene contraseñas y no emite tokens.

L12

```
12 | services:
```

Agrupa los cinco contenedores. Sus nombres son DNS de la red Compose, permitiendo llamadas como http://fleet-service:8081 sin conocer una IP fija.

L13

```
13 |   postgres:
```

En services declara el contenedor postgres; dentro de depends_on referencia esa misma dependencia. Aloja tres bases lógicas separadas.

L14

```
14 |     image: postgres:17.11
```

Selecciona una imagen publicada por nombre y etiqueta de versión. Docker la descarga si falta; no compila los fuentes locales con esta instrucción.

L15

```
15 |     environment:
```

Pasa variables al proceso del contenedor. Compose resuelve las referencias al entorno antes de iniciar; la aplicación utiliza los valores para sus conexiones y opciones.

L16

```
16 |       POSTGRES_USER: postgres
```

Superusuario SQL inicial del servidor PostgreSQL; no es una identidad Keycloak.

L17

```
17 |       POSTGRES_DB: postgres
```

Base inicial postgres. El script de inicialización crea además fleet_db, assistance_db y keycloak_db.

L18

```
18 |       POSTGRES_PASSWORD: ${POSTGRES_ADMIN_PASSWORD:?Configurar .env}
```

Contraseña inicial del superusuario SQL. Cambiar esta variable no cambia una contraseña ya guardada en el volumen. Compose exige POSTGRES_ADMIN_PASSWORD no vacía y aborta con "Configurar .env" si falta.

L19

```
19 |       ASSISTANCE_DB_PASSWORD: ${ASSISTANCE_DB_PASSWORD:?Configurar .env}
```

Suministra la contraseña de ASSISTANCE_DB desde el entorno. El contenedor la usa al preparar su usuario de base o identidad demo. Compose exige ASSISTANCE_DB_PASSWORD no vacía y aborta con "Configurar .env" si falta.

L20

```
20 |       FLEET_DB_PASSWORD: ${FLEET_DB_PASSWORD:?Configurar .env}
```

Suministra la contraseña de FLEET_DB desde el entorno. El contenedor la usa al preparar su usuario de base o identidad demo. Compose exige FLEET_DB_PASSWORD no vacía y aborta con "Configurar .env" si falta.

L21

```
21 |       KEYCLOAK_DB_PASSWORD: ${KEYCLOAK_DB_PASSWORD:?Configurar .env}
```

Suministra la contraseña de KEYCLOAK_DB desde el entorno. El contenedor la usa al preparar su usuario de base o identidad demo. Compose exige KEYCLOAK_DB_PASSWORD no vacía y aborta con "Configurar .env" si falta.

L22

```
22 |     volumes:
```

En un servicio define montajes de archivos/datos; en la raíz declara volúmenes nombrados administrados por Docker. Los datos del volumen sobreviven a recrear contenedores.

L23

```
23 |       - postgres-data:/var/lib/postgresql/data
```

Montaje origen:destino. postgres-data conserva datos del servidor bajo /var/lib/postgresql/data.

L24

```
24 |       - ./infra/postgres/01-databases.sh:/docker-entrypoint-initdb.d/01-databases.sh:ro
```

Montaje origen:destino de solo lectura por :ro. El archivo del repositorio se hace visible dentro del contenedor. La imagen PostgreSQL ejecuta este script solo al inicializar un directorio de datos vacío.

L25

```
25 |     healthcheck:
```

Define la sonda del contenedor. Docker marca healthy/unhealthy según sus resultados y los usa para dependencias de arranque.

L26

```
26 |       test: [CMD-SHELL, "pg_isready -U postgres -d postgres"]
```

CMD-SHELL ejecuta la sonda en un shell. pg_isready comprueba si PostgreSQL acepta conexiones a la base postgres; no prueba tablas ni credenciales de negocio.

L27

```
27 |       interval: 5s
```

Tiempo entre sondas; s significa segundos. No controla el scheduler de búsquedas de grúas.

L28

```
28 |       timeout: 3s
```

Tiempo máximo de una sonda antes de considerarla fallida. Es independiente del timeout de las APIs externas.

L29

```
29 |       retries: 30
```

Fallos consecutivos de la sonda necesarios para marcar unhealthy. No es un cupo de propuestas.

L30

```
30 |     logging: *logging
```

*logging reutiliza el mapa definido por &logging: json-file, 10 MB y tres archivos. Es una referencia YAML, no una llamada a un servidor de logging.

L31

```
31 |   keycloak:
```

En services declara el contenedor keycloak; dentro de depends_on referencia esa misma dependencia. Emite tokens y administra identidades.

L32

```
32 |     image: quay.io/keycloak/keycloak:26.7.5
```

Selecciona una imagen publicada por nombre y etiqueta de versión. Docker la descarga si falta; no compila los fuentes locales con esta instrucción.

L33

```
33 |     command: [start-dev, --import-realm]
```

start-dev inicia Keycloak en modo de desarrollo; --import-realm carga los JSON de data/import cuando corresponde. No sobrescribe un realm ya existente.

L34

```
34 |     environment:
```

Pasa variables al proceso del contenedor. Compose resuelve las referencias al entorno antes de iniciar; la aplicación utiliza los valores para sus conexiones y opciones.

L35

```
35 |       KC_DB: postgres
```

Selecciona PostgreSQL como almacenamiento persistente de Keycloak para usuarios, roles, clientes y sesiones.

L36

```
36 |       KC_DB_URL: jdbc:postgresql://postgres:5432/keycloak_db
```

URL JDBC de keycloak_db; postgres es el DNS del contenedor y 5432 su puerto interno. localhost apuntaría al propio contenedor Keycloak.

L37

```
37 |       KC_DB_USERNAME: keycloak_user
```

Cuenta SQL exclusiva de Keycloak, distinta de bootstrap-admin y de los usuarios humanos.

L38

```
38 |       KC_DB_PASSWORD: ${KEYCLOAK_DB_PASSWORD:?Configurar .env}
```

Contraseña SQL de keycloak_user, suministrada por entorno y no compartida con los servicios de negocio. Compose exige KEYCLOAK_DB_PASSWORD no vacía y aborta con "Configurar .env" si falta.

L39

```
39 |       KC_BOOTSTRAP_ADMIN_USERNAME: bootstrap-admin
```

Cuenta administrativa inicial bootstrap-admin. Configura identidades y realms; no representa al ADMIN del negocio.

L40

```
40 |       KC_BOOTSTRAP_ADMIN_PASSWORD: ${KC_BOOTSTRAP_ADMIN_PASSWORD:?Configurar .env}
```

Contraseña inicial del administrador técnico, obtenida del entorno. Compose exige KC_BOOTSTRAP_ADMIN_PASSWORD no vacía y aborta con "Configurar .env" si falta.

L41

```
41 |       KC_HOSTNAME: http://localhost:8080
```

URL pública de Keycloak para navegador/Postman y coherencia del emisor de tokens. No es la dirección interna de acceso desde Docker.

L42

```
42 |       KC_HOSTNAME_BACKCHANNEL_DYNAMIC: "true"
```

Permite resolver direcciones de comunicación interna desde la petición mientras conserva el hostname público configurado.

L43

```
43 |       KC_HEALTH_ENABLED: "true"
```

Habilita sondas de salud Keycloak en el puerto de gestión 9000 que consulta el healthcheck.

L44

```
44 |       KEYCLOAK_CLIENT_SECRET: ${KEYCLOAK_CLIENT_SECRET:?Configurar .env}
```

Secreto que autentica al cliente técnico. Debe coincidir entre Keycloak y Assistance; no se usa como Bearer de usuario. Compose exige KEYCLOAK_CLIENT_SECRET no vacía y aborta con "Configurar .env" si falta.

L45

```
45 |       DEMO_DRIVER_PASSWORD: ${DEMO_DRIVER_PASSWORD:?Configurar .env}
```

Suministra la contraseña de DEMO_DRIVER desde el entorno. El contenedor la usa al preparar su usuario de base o identidad demo. Compose exige DEMO_DRIVER_PASSWORD no vacía y aborta con "Configurar .env" si falta.

L46

```
46 |       DEMO_OPERATOR_PASSWORD: ${DEMO_OPERATOR_PASSWORD:?Configurar .env}
```

Suministra la contraseña de DEMO_OPERATOR desde el entorno. El contenedor la usa al preparar su usuario de base o identidad demo. Compose exige DEMO_OPERATOR_PASSWORD no vacía y aborta con "Configurar .env" si falta.

L47

```
47 |       DEMO_ADMIN_PASSWORD: ${DEMO_ADMIN_PASSWORD:?Configurar .env}
```

Suministra la contraseña de DEMO_ADMIN desde el entorno. El contenedor la usa al preparar su usuario de base o identidad demo. Compose exige DEMO_ADMIN_PASSWORD no vacía y aborta con "Configurar .env" si falta.

L48

```
48 |     ports: ["127.0.0.1:8080:8080"]
```

Publica host:contenedor. 127.0.0.1 restringe al equipo local; el primer puerto pertenece a Windows y el segundo al contenedor. Fleet y Assistance no publican puertos.

L49

```
49 |     volumes:
```

En un servicio define montajes de archivos/datos; en la raíz declara volúmenes nombrados administrados por Docker. Los datos del volumen sobreviven a recrear contenedores.

L50

```
50 |       - ./infra/keycloak/auxiruta-realm.json:/opt/keycloak/data/import/auxiruta-realm.json:ro
```

Montaje origen:destino de solo lectura por :ro. El archivo del repositorio se hace visible dentro del contenedor. Keycloak lo lee con --import-realm para crear el realm cuando falta.

L51

```
51 |     depends_on:
```

Ordena el arranque con dependencias. La condición de salud espera disponibilidad inicial, pero no garantiza disponibilidad continua.

L52

```
52 |       postgres:
```

En services declara el contenedor postgres; dentro de depends_on referencia esa misma dependencia. Aloja tres bases lógicas separadas.

L53

```
53 |         condition: service_healthy
```

service_healthy espera una sonda aprobada de la dependencia antes de iniciar este servicio. No verifica un caso de negocio completo.

L54

```
54 |     healthcheck:
```

Define la sonda del contenedor. Docker marca healthy/unhealthy según sus resultados y los usa para dependencias de arranque.

L55

```
55 |       test: [CMD-SHELL, "bash -c 'exec 3<>/dev/tcp/127.0.0.1/9000; printf \"GET /health/ready
     HTTP/1.0\\r\\n\\r\\n\" >&3; grep -q \"200 OK\" <&3'"]
```

CMD-SHELL inicia bash; exec 3<> abre un socket TCP al puerto de gestión 9000, printf escribe una petición HTTP a /health/ready y grep busca 200 OK. Un código de salida cero indica readiness.

L56

```
56 |       interval: 10s
```

Tiempo entre sondas; s significa segundos. No controla el scheduler de búsquedas de grúas.

L57

```
57 |       timeout: 5s
```

Tiempo máximo de una sonda antes de considerarla fallida. Es independiente del timeout de las APIs externas.

L58

```
58 |       start_period: 60s
```

Ventana inicial de arranque: aquí Keycloak dispone de 60 segundos antes de que ciertos fallos cuenten como fallos normales de salud.

L59

```
59 |       retries: 30
```

Fallos consecutivos de la sonda necesarios para marcar unhealthy. No es un cupo de propuestas.

L60

```
60 |     logging: *logging
```

*logging reutiliza el mapa definido por &logging: json-file, 10 MB y tres archivos. Es una referencia YAML, no una llamada a un servidor de logging.

L61

```
61 |   fleet-service:
```

En services declara el contenedor fleet-service; dentro de depends_on referencia esa misma dependencia. Administra recursos y reservas.

L62

```
62 |     build:
```

Solicita construir una imagen a partir del código local. context indica los archivos disponibles y dockerfile las instrucciones.

L63

```
63 |       context: .
```

El punto selecciona la raíz del repositorio como contexto. COPY puede acceder a Commons y al módulo; .dockerignore limita lo enviado al motor.

L64

```
64 |       dockerfile: fleet-service/Dockerfile
```

Selecciona el archivo de instrucciones del módulo dentro del contexto. Las fuentes de COPY siguen resolviéndose desde el contexto raíz.

L65

```
65 |     environment:
```

Pasa variables al proceso del contenedor. Compose resuelve las referencias al entorno antes de iniciar; la aplicación utiliza los valores para sus conexiones y opciones.

L66

```
66 |       <<: *jwt
```

Es la mezcla de mapas YAML. *jwt referencia el ancla &jwt e incorpora sus claves dentro de environment; las variables explícitas del servicio completan o sobrescriben esas claves.

L67

```
67 |       DB_URL: jdbc:postgresql://postgres:5432/fleet_db
```

URL JDBC de la base propia del servicio. Compartir servidor PostgreSQL no implica compartir tablas entre servicios.

L68

```
68 |       DB_USER: fleet_user
```

Usuario SQL del servicio para su base. Es independiente de los roles del JWT.

L69

```
69 |       DB_PASSWORD: ${FLEET_DB_PASSWORD:?Configurar .env}
```

Contraseña de ese usuario SQL, obtenida de entorno para no versionar secretos. Compose exige FLEET_DB_PASSWORD no vacía y aborta con "Configurar .env" si falta.

L70

```
70 |     depends_on:
```

Ordena el arranque con dependencias. La condición de salud espera disponibilidad inicial, pero no garantiza disponibilidad continua.

L71

```
71 |       keycloak:
```

En services declara el contenedor keycloak; dentro de depends_on referencia esa misma dependencia. Emite tokens y administra identidades.

L72

```
72 |         condition: service_healthy
```

service_healthy espera una sonda aprobada de la dependencia antes de iniciar este servicio. No verifica un caso de negocio completo.

L73

```
73 |     logging: *logging
```

*logging reutiliza el mapa definido por &logging: json-file, 10 MB y tres archivos. Es una referencia YAML, no una llamada a un servidor de logging.

L74

```
74 |   assistance-service:
```

En services declara el contenedor assistance-service; dentro de depends_on referencia esa misma dependencia. Evalúa y ejecuta solicitudes.

L75

```
75 |     build:
```

Solicita construir una imagen a partir del código local. context indica los archivos disponibles y dockerfile las instrucciones.

L76

```
76 |       context: .
```

El punto selecciona la raíz del repositorio como contexto. COPY puede acceder a Commons y al módulo; .dockerignore limita lo enviado al motor.

L77

```
77 |       dockerfile: assistance-service/Dockerfile
```

Selecciona el archivo de instrucciones del módulo dentro del contexto. Las fuentes de COPY siguen resolviéndose desde el contexto raíz.

L78

```
78 |     environment:
```

Pasa variables al proceso del contenedor. Compose resuelve las referencias al entorno antes de iniciar; la aplicación utiliza los valores para sus conexiones y opciones.

L79

```
79 |       <<: *jwt
```

Es la mezcla de mapas YAML. *jwt referencia el ancla &jwt e incorpora sus claves dentro de environment; las variables explícitas del servicio completan o sobrescriben esas claves.

L80

```
80 |       DB_URL: jdbc:postgresql://postgres:5432/assistance_db
```

URL JDBC de la base propia del servicio. Compartir servidor PostgreSQL no implica compartir tablas entre servicios.

L81

```
81 |       DB_USER: assistance_user
```

Usuario SQL del servicio para su base. Es independiente de los roles del JWT.

L82

```
82 |       DB_PASSWORD: ${ASSISTANCE_DB_PASSWORD:?Configurar .env}
```

Contraseña de ese usuario SQL, obtenida de entorno para no versionar secretos. Compose exige ASSISTANCE_DB_PASSWORD no vacía y aborta con "Configurar .env" si falta.

L83

```
83 |       KEYCLOAK_TOKEN_URI: http://keycloak:8080/realms/auxiruta/protocol/openid-connect/token
```

Endpoint interno donde Assistance obtiene un token de servicio mediante client_credentials.

L84

```
84 |       KEYCLOAK_CLIENT_ID: assistance-service
```

Identifica al cliente técnico assistance-service; no es el usuario conductor ni un secreto.

L85

```
85 |       KEYCLOAK_CLIENT_SECRET: ${KEYCLOAK_CLIENT_SECRET:?Configurar .env}
```

Secreto que autentica al cliente técnico. Debe coincidir entre Keycloak y Assistance; no se usa como Bearer de usuario. Compose exige KEYCLOAK_CLIENT_SECRET no vacía y aborta con "Configurar .env" si falta.

L86

```
86 |       FLEET_URL: http://fleet-service:8081
```

Base HTTP interna de Fleet que usa Assistance para candidatos y reservas; fleet-service es un nombre Docker.

L87

```
87 |       ORS_API_KEY: ${ORS_API_KEY:-}
```

Credencial para calcular rutas viales con OpenRouteService. La referencia no revela su valor local. Compose usa ORS_API_KEY si existe y no está vacía; de otro modo usa ".

L88

```
88 |       ORS_BASE_URL: ${ORS_BASE_URL:-https://api.heigit.org/openrouteservice/v2}
```

Raíz del proveedor de rutas; no establece la cobertura del dominio, que se valida en AuxiRuta. Compose usa ORS_BASE_URL si existe y no está vacía; de otro modo usa 'https://api.heigit.org/openrouteservice/v2'.

L89

```
89 |     depends_on:
```

Ordena el arranque con dependencias. La condición de salud espera disponibilidad inicial, pero no garantiza disponibilidad continua.

L90

```
90 |       keycloak:
```

En services declara el contenedor keycloak; dentro de depends_on referencia esa misma dependencia. Emite tokens y administra identidades.

L91

```
91 |         condition: service_healthy
```

service_healthy espera una sonda aprobada de la dependencia antes de iniciar este servicio. No verifica un caso de negocio completo.

L92

```
92 |     logging: *logging
```

*logging reutiliza el mapa definido por &logging: json-file, 10 MB y tres archivos. Es una referencia YAML, no una llamada a un servidor de logging.

L93

```
93 |   api-gateway:
```

En services declara el contenedor api-gateway; dentro de depends_on referencia esa misma dependencia. Recibe el tráfico público y lo enruta.

L94

```
94 |     build:
```

Solicita construir una imagen a partir del código local. context indica los archivos disponibles y dockerfile las instrucciones.

L95

```
95 |       context: .
```

El punto selecciona la raíz del repositorio como contexto. COPY puede acceder a Commons y al módulo; .dockerignore limita lo enviado al motor.

L96

```
96 |       dockerfile: api-gateway/Dockerfile
```

Selecciona el archivo de instrucciones del módulo dentro del contexto. Las fuentes de COPY siguen resolviéndose desde el contexto raíz.

L97

```
97 |     environment:
```

Pasa variables al proceso del contenedor. Compose resuelve las referencias al entorno antes de iniciar; la aplicación utiliza los valores para sus conexiones y opciones.

L98

```
98 |       <<: *jwt
```

Es la mezcla de mapas YAML. *jwt referencia el ancla &jwt e incorpora sus claves dentro de environment; las variables explícitas del servicio completan o sobrescriben esas claves.

L99

```
99 |     ports: ["127.0.0.1:8000:8000"]
```

Publica host:contenedor. 127.0.0.1 restringe al equipo local; el primer puerto pertenece a Windows y el segundo al contenedor. Fleet y Assistance no publican puertos.

L100

```
100 |     depends_on:
```

Ordena el arranque con dependencias. La condición de salud espera disponibilidad inicial, pero no garantiza disponibilidad continua.

L101

```
101 |       keycloak:
```

En services declara el contenedor keycloak; dentro de depends_on referencia esa misma dependencia. Emite tokens y administra identidades.

L102

```
102 |         condition: service_healthy
```

service_healthy espera una sonda aprobada de la dependencia antes de iniciar este servicio. No verifica un caso de negocio completo.

L103

```
103 |     logging: *logging
```

*logging reutiliza el mapa definido por &logging: json-file, 10 MB y tres archivos. Es una referencia YAML, no una llamada a un servidor de logging.

L104

```
104 | volumes:
```

En un servicio define montajes de archivos/datos; en la raíz declara volúmenes nombrados administrados por Docker. Los datos del volumen sobreviven a recrear contenedores.

L105

```
105 |   postgres-data:
```

Declara el volumen nombrado donde PostgreSQL conserva las tres bases. down sin -v conserva ese volumen; eliminarlo pierde esos datos.

fleet-service/Dockerfile

`Ruta: fleet-service/Dockerfile`

Objetivo y funcionalidad

Construye fleet-service en dos etapas: JDK 21 instala Commons y verifica el módulo; la imagen final usa JRE, app.jar y un usuario sin privilegios. Evita transportar herramientas de compilación al servidor.

L1

```
1 | FROM eclipse-temurin:21-jdk-jammy AS build
```

Etapa build: imagen Temurin con JDK 21 y base Ubuntu Jammy. JDK incluye compilador; AS build permite recuperar luego el artefacto mediante COPY --from=build.

L2

```
2 | WORKDIR /workspace
```

Crea/selecciona /workspace como directorio de trabajo para instrucciones siguientes. Rutas relativas de RUN y destinos COPY se resuelven aquí.

L3

```
3 | COPY auxiruta-commons/ auxiruta-commons/
```

Introduce la carpeta auxiruta-commons/ del contexto en la etapa de construcción. Commons debe estar disponible para compilar el módulo dependiente; no se copia el .env real.

L4

```
4 | COPY fleet-service/ fleet-service/
```

Introduce la carpeta fleet-service/ del contexto en la etapa de construcción. Commons debe estar disponible para compilar el módulo dependiente; no se copia el .env real.

L5

```
5 | RUN chmod +x auxiruta-commons/mvnw fleet-service/mvnw && ./auxiruta-commons/mvnw -B -ntp -f
    auxiruta-commons/pom.xml install && ./fleet-service/mvnw -B -ntp -f fleet-service/pom.xml verify
```

Durante docker build da permiso de ejecución a los Wrapper, instala Commons en el repositorio Maven de esa etapa y ejecuta verify del módulo. && detiene la cadena si algo falla; -B evita interacción, -ntp suprime progreso y -f selecciona el POM.

L6

```
6 | FROM eclipse-temurin:21-jre-jammy
```

Comienza imagen final independiente con JRE 21. Conserva runtime Java sin el compilador ni el repositorio de fuentes de la etapa anterior.

L7

```
7 | RUN groupadd --system auxiruta && useradd --system --gid auxiruta --home-dir /app auxiruta
```

Crea grupo y usuario de sistema auxiruta dentro de la imagen. --system marca cuenta de servicio; --gid asocia grupo y --home-dir fija /app como directorio personal.

L8

```
8 | WORKDIR /app
```

Crea/selecciona /app como directorio de trabajo para instrucciones siguientes. Rutas relativas de RUN y destinos COPY se resuelven aquí.

L9

```
9 | COPY --from=build /workspace/fleet-service/target/app.jar app.jar
```

Copia únicamente app.jar desde la etapa build a la imagen final. Este JAR ya incluye launcher y dependencias Boot; los tests y herramientas de compilación no se trasladan.

L10

```
10 | USER auxiruta
```

Las instrucciones de ejecución siguientes y el proceso final usan auxiruta en vez de root, reduciendo privilegios del backend.

L11

```
11 | ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Forma exec JSON: Docker inicia java directamente con -jar /app/app.jar, sin un shell intermedio. El launcher Boot arranca la aplicación del módulo.

fleet-service/pom.xml

`Ruta: fleet-service/pom.xml`

Objetivo y funcionalidad

Define la construcción de la API de flota y reservas con MVC, JPA y PostgreSQL. Maven lee coordenadas, versiones, dependencias y plugins para compilar, probar y empaquetar el módulo.

L1

```
1 | <?xml version="1.0" ?>
```

Declara sintaxis XML.

L2

```
2 | <project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

Abre bloque del modelo Maven y agrupa sus hijos.

L3

```
3 |     <modelVersion>4.0.0</modelVersion>
```

Versión del modelo POM, no de Maven. Valor: 4.0.0.

L4

```
4 |     <parent>
```

Herencia del padre que administra plugins y versiones.

L5

```
5 |         <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L6

```
6 |         <artifactId>spring-boot-starter-parent</artifactId>
```

Padre que administra versiones coherentes y valores por defecto de construcción Spring Boot.

L7

```
 7 |         <version>3.5.16</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 3.5.16.

L8

```
 8 |         <relativePath/>
```

La ruta vacía impide buscar el padre en ../pom.xml; Maven resuelve spring-boot-starter-parent desde repositorios.

L9

```
 9 |     </parent>
```

Termina la sección Maven parent. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L10

```
10 |     <groupId>com.auxiruta</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: com.auxiruta.

L11

```
11 |     <artifactId>fleet-service</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: fleet-service.

L12

```
12 |     <version>1.0.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 1.0.0.

L13

```
13 |     <properties>
```

Variables de la construcción.

L14

```
14 |         <java.version>21</java.version>
```

Versión de Java para compilar. Valor: 21.

L15

```
15 |         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

Codificación del fuente. Valor: UTF-8.

L16

```
16 |     </properties>
```

Termina la sección Maven properties. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L17

```
17 |     <dependencies>
```

Lista de dependencias.

L18

```
18 |         <dependency>
```

Declara una dependencia Maven.

L19

```
19 |             <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L20

```
20 |             <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
```

Aporta recepción de Bearer JWT y validación de tokens.

L21

```
21 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L22

```
22 |         <dependency>
```

Declara una dependencia Maven.

L23

```
23 |             <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L24

```
24 |             <artifactId>spring-boot-starter-web</artifactId>
```

Incluye Spring MVC, JSON y servidor Servlet embebido para REST.

L25

```
25 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L26

```
26 |         <dependency>
```

Declara una dependencia Maven.

L27

```
27 |             <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L28

```
28 |             <artifactId>spring-boot-starter-data-jpa</artifactId>
```

Incluye repositorios Spring Data, Hibernate y transacciones de persistencia.

L29

```
29 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L30

```
30 |         <dependency>
```

Declara una dependencia Maven.

L31

```
31 |             <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L32

```
32 |             <artifactId>spring-boot-starter-validation</artifactId>
```

Activa restricciones Bean Validation de los DTO, como NotBlank o Size.

L33

```
33 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L34

```
34 |         <dependency>
```

Declara una dependencia Maven.

L35

```
35 |             <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L36

```
36 |             <artifactId>spring-boot-starter-test</artifactId>
```

Incluye JUnit y herramientas de prueba; scope test lo excluye de ejecución productiva.

L37

```
37 |             <scope>test</scope>
```

Solo disponible en compilación/ejecución de tests. No se incorpora como dependencia productiva al JAR Boot.

L38

```
38 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L39

```
39 |         <dependency>
```

Declara una dependencia Maven.

L40

```
40 |             <groupId>org.springframework.security</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.security.

L41

```
41 |             <artifactId>spring-security-test</artifactId>
```

Permite simular identidades y verificar permisos en pruebas aisladas.

L42

```
42 |             <scope>test</scope>
```

Solo disponible en compilación/ejecución de tests. No se incorpora como dependencia productiva al JAR Boot.

L43

```
43 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L44

```
44 |         <dependency>
```

Declara una dependencia Maven.

L45

```
45 |             <groupId>org.postgresql</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.postgresql.

L46

```
46 |             <artifactId>postgresql</artifactId>
```

Driver JDBC de comunicación con PostgreSQL; no instala el servidor SQL.

L47

```
47 |             <scope>runtime</scope>
```

Dependencia necesaria al ejecutar pero no para compilar contra sus tipos; el driver PostgreSQL es descubierto a través de JDBC.

L48

```
48 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L49

```
49 |         <dependency>
```

Declara una dependencia Maven.

L50

```
50 |             <groupId>org.springdoc</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springdoc.

L51

```
51 |             <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
```

Genera OpenAPI de MVC e incorpora recursos Swagger UI.

L52

```
52 |             <version>2.10.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 2.10.0.

L53

```
53 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L54

```
54 |         <dependency>
```

Declara una dependencia Maven.

L55

```
55 |             <groupId>com.auxiruta</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: com.auxiruta.

L56

```
56 |             <artifactId>auxiruta-commons</artifactId>
```

Biblioteca local de seguridad, errores, reloj y geografía; debe construirse antes de los dependientes.

L57

```
57 |             <version>1.0.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 1.0.0.

L58

```
58 |         </dependency>
```

Termina la sección Maven dependency. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L59

```
59 |     </dependencies>
```

Termina la sección Maven dependencies. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L60

```
60 |     <build>
```

Abre bloque del modelo Maven y agrupa sus hijos.

L61

```
61 |         <finalName>app</finalName>
```

Fija app como nombre de artefacto bajo target. Los Dockerfiles esperan target/app.jar, por lo que el nombre debe coincidir.

L62

```
62 |         <plugins>
```

Abre bloque del modelo Maven y agrupa sus hijos.

L63

```
63 |             <plugin>
```

Plugin de construcción; el de Spring Boot reempaqueta el ejecutable.

L64

```
64 |                 <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L65

```
65 |                 <artifactId>spring-boot-maven-plugin</artifactId>
```

Reempaqueta el JAR con launcher/dependencias para ejecutarlo con java -jar.

L66

```
66 |             </plugin>
```

Termina la sección Maven plugin. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L67

```
67 |         </plugins>
```

Termina la sección Maven plugins. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L68

```
68 |     </build>
```

Termina la sección Maven build. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

L69

```
69 | </project>
```

Termina la sección Maven project. Los hijos anteriores pertenecen a ese elemento; a partir de aquí las declaraciones vuelven al nivel contenedor.

infra/postgres/01-databases.sh

`Ruta: infra/postgres/01-databases.sh`

Objetivo y funcionalidad

Inicialización de PostgreSQL: crea usuarios y bases separados para Fleet, Assistance y Keycloak con contraseñas suministradas por el entorno.

L1

```
 1 | #!/bin/sh
```

Comentario o directiva del intérprete; no es regla de negocio.

L2

```
2 | set -eu
```

-e termina el shell ante un comando fallido y -u ante una variable sin definir. Impide continuar una inicialización incompleta con credenciales ausentes.

L3

```
3 | psql --username "$POSTGRES_USER" --dbname "$POSTGRES_DB" --set=ON_ERROR_STOP=1
    --set=assistance_password="$ASSISTANCE_DB_PASSWORD" --set=fleet_password="$FLEET_DB_PASSWORD"
    --set=keycloak_password="$KEYCLOAK_DB_PASSWORD" <<'SQL'
```

Invoca psql con errores fatales y contraseñas parametrizadas en heredoc.

L4

```
4 | CREATE ROLE assistance_user LOGIN PASSWORD :'assistance_password';
```

Crea usuario PostgreSQL assistance_user con clave parametrizada.

L5

```
5 | CREATE ROLE fleet_user LOGIN PASSWORD :'fleet_password';
```

Crea usuario PostgreSQL fleet_user con clave parametrizada.

L6

```
6 | CREATE ROLE keycloak_user LOGIN PASSWORD :'keycloak_password';
```

Crea usuario PostgreSQL keycloak_user con clave parametrizada.

L7

```
7 | CREATE DATABASE assistance_db OWNER assistance_user;
```

Crea base assistance_db propiedad de assistance_user.

L8

```
8 | CREATE DATABASE fleet_db OWNER fleet_user;
```

Crea base fleet_db propiedad de fleet_user.

L9

```
9 | CREATE DATABASE keycloak_db OWNER keycloak_user;
```

Crea base keycloak_db propiedad de keycloak_user.

L10

```
10 | REVOKE ALL ON DATABASE assistance_db FROM PUBLIC;
```

Retira privilegios de PUBLIC, el grupo implícito de todos los usuarios SQL, sobre esta base. Evita que cualquier usuario pueda conectarse por los permisos iniciales; el propietario conserva sus derechos.

L11

```
11 | REVOKE ALL ON DATABASE fleet_db FROM PUBLIC;
```

Retira privilegios de PUBLIC, el grupo implícito de todos los usuarios SQL, sobre esta base. Evita que cualquier usuario pueda conectarse por los permisos iniciales; el propietario conserva sus derechos.

L12

```
12 | REVOKE ALL ON DATABASE keycloak_db FROM PUBLIC;
```

Retira privilegios de PUBLIC, el grupo implícito de todos los usuarios SQL, sobre esta base. Evita que cualquier usuario pueda conectarse por los permisos iniciales; el propietario conserva sus derechos.

L13

```
13 | GRANT CONNECT ON DATABASE assistance_db TO assistance_user;
```

Concede permiso de conexión a la cuenta específica del servicio. La propiedad/permisos internos de tablas determinan después qué puede modificar; CONNECT solo autoriza entrar a esa base.

L14

```
14 | GRANT CONNECT ON DATABASE fleet_db TO fleet_user;
```

Concede permiso de conexión a la cuenta específica del servicio. La propiedad/permisos internos de tablas determinan después qué puede modificar; CONNECT solo autoriza entrar a esa base.

L15

```
15 | GRANT CONNECT ON DATABASE keycloak_db TO keycloak_user;
```

Concede permiso de conexión a la cuenta específica del servicio. La propiedad/permisos internos de tablas determinan después qué puede modificar; CONNECT solo autoriza entrar a esa base.

L16

```
16 | SQL
```

Cierra el heredoc iniciado con <<: las líneas anteriores se enviaron a psql como entrada estándar. SQL es un delimitador de shell, no una sentencia de PostgreSQL.

scripts/Format-Java.ps1

`Ruta: scripts/Format-Java.ps1`

Objetivo y funcionalidad

Aplica google-java-format a fuentes/tests de los cuatro módulos, o verifica el formato con -Check. Descarga una versión fija y comprueba integridad; no forma parte del servidor.

L1

```
 1 | param([switch]$Check)
```

Declara parámetros del script. [switch] es un indicador que vale verdadero al escribir su nombre: Check comprueba formato sin reescribir. No es una variable del backend.

L3

```
 3 | $ErrorActionPreference = 'Stop'
```

Stop convierte errores PowerShell no terminantes en errores que detienen el script. Los programas externos requieren además comprobar LASTEXITCODE; esta variable no reemplaza ese control.

L4

```
 4 | $repo = Split-Path $PSScriptRoot -Parent
```

PSScriptRoot contiene la carpeta del script. Split-Path -Parent obtiene la raíz del repositorio para que las rutas funcionen aunque la terminal se haya abierto en otra carpeta.

L5

```
 5 | $version = '1.29.0'
```

Fija la versión de google-java-format. Mantenerla igual en el equipo evita diferencias de formato por actualizar la herramienta.

L6

```
 6 | $expectedHash = 'AEB4E1831D56011E7E06F3393C4E17340C7CCA0FD7BA9076AB8DC0624759F7F0'
```

Huella esperada del binario descargado: permite detectar descarga corrupta o contenido distinto. Es una comprobación de integridad de la herramienta, no un secreto.

L7

```
 7 | $cacheDirectory = Join-Path $env:LOCALAPPDATA 'AuxiRuta\formatters'
```

Construye una carpeta bajo LOCALAPPDATA del usuario para reutilizar el formateador sin agregar el JAR al repositorio.

L8

```
 8 | $formatter = Join-Path $cacheDirectory "google-java-format-$version-all-deps.jar"
```

Ruta del JAR descargado con nombre que incluye su versión.

L10

```
10 | if (!(Test-Path -LiteralPath $formatter)) {
```

Comprueba existencia de .env sin imprimir valores.

L11

```
11 |     New-Item -ItemType Directory -Force $cacheDirectory | Out-Null
```

Crea la carpeta de caché si falta. -Force permite reutilizarla y Out-Null descarta la salida informativa.

L12

```
12 |     $uri = "https://repo.maven.apache.org/maven2/com/google/googlejavaformat/google-java-format/$
     version/google-java-format-$version-all-deps.jar"
```

Dirección de Maven Central del binario completo del formateador. La interpolación incorpora la versión fijada.

L13

```
13 |     Invoke-WebRequest -UseBasicParsing -Uri $uri -OutFile $formatter
```

Descarga el formateador a la caché local; -OutFile guarda el binario en vez de mostrarlo en consola.

L14

```
14 | }
```

Cierra el arreglo/bloque anterior o abre la alternativa else. Solo organiza el flujo del script; no cambia datos del backend.

L16

```
16 | if ((Get-FileHash -LiteralPath $formatter -Algorithm SHA256).Hash -ne $expectedHash) {
```

Calcula SHA-256 del JAR local y lo compara con la huella esperada; si no coincide aborta antes de ejecutar un binario distinto.

L17

```
17 |     throw 'El hash del formateador no coincide con la version fijada.'
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L18

```
18 | }
```

Cierra el arreglo/bloque anterior o abre la alternativa else. Solo organiza el flujo del script; no cambia datos del backend.

L20

```
20 | $arguments = @(
```

Construye la lista de argumentos que se enviará a Java. PowerShell mantiene cada elemento separado para no concatenar un comando inseguro.

L21

```
21 |     '-jar', $formatter,
```

Pide a Java ejecutar el JAR del formateador, no el app.jar del backend.

L22

```
22 |     '--skip-sorting-imports',
```

Conserva el orden original de imports para limitar el cambio al formato del resto del código.

L23

```
23 |     '--skip-removing-unused-imports',
```

Evita eliminar imports aunque no se usen; la tarea no hace limpieza semántica.

L24

```
24 |     '--skip-reflowing-long-strings',
```

Conserva literales largos sin redistribuirlos; protege SQL y JSON embebidos.

L25

```
25 |     '--skip-javadoc-formatting'
```

Conserva el formato de comentarios Javadoc.

L26

```
26 | )
```

Cierra el arreglo/bloque anterior o abre la alternativa else. Solo organiza el flujo del script; no cambia datos del backend.

L28

```
28 | if ($Check) {
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L29

```
29 |     $arguments += @('--dry-run', '--set-exit-if-changed')
```

Modo comprobación: no escribe archivos y devuelve salida no exitosa si detecta diferencias pendientes.

L30

```
30 | } else {
```

Cierra el arreglo/bloque anterior o abre la alternativa else. Solo organiza el flujo del script; no cambia datos del backend.

L31

```
31 |     $arguments += '--replace'
```

Modo edición: el formateador reescribe los archivos Java con el estilo fijado.

L32

```
32 | }
```

Cierra el arreglo/bloque anterior o abre la alternativa else. Solo organiza el flujo del script; no cambia datos del backend.

L34

```
34 | $files = foreach ($module in @('auxiruta-commons', 'fleet-service', 'assistance-service',
     'api-gateway')) {
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L35

```
35 |     Get-ChildItem -LiteralPath (Join-Path $repo "$module/src") -Recurse -File -Filter '*.java'
```

Enumera recursivamente archivos Java bajo src de cada módulo, incluyendo tests pero no clases de target.

L36

```
36 | }
```

Cierra el arreglo/bloque anterior o abre la alternativa else. Solo organiza el flujo del script; no cambia datos del backend.

L38

```
38 | & java @arguments @($files.FullName | Sort-Object)
```

Invoca Java con splatting de argumentos y rutas ordenadas. @arguments expande la lista en parámetros separados.

L39

```
39 | if ($LASTEXITCODE -ne 0) {
```

Comprueba resultado de comando externo.

L40

```
40 |     throw 'El formateador fallo o -Check detecto archivos pendientes de formato.'
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L41

```
41 | }
```

Cierra el arreglo/bloque anterior o abre la alternativa else. Solo organiza el flujo del script; no cambia datos del backend.

Líneas vacías 2, 9, 15, 19, 27, 33, 37: separadores sin instrucciones.

scripts/Start-AuxiRuta.ps1

`Ruta: scripts/Start-AuxiRuta.ps1`

Objetivo y funcionalidad

Arranca el backend con perfil normal o demo, construye si corresponde y espera respuestas 401 como prueba de disponibilidad protegida.

L1

```
1 | param(
```

Declara parámetros del script. [switch] es un indicador que vale verdadero al escribir su nombre: Profile selecciona configuración y NoBuild evita reconstruir imágenes. No es una variable del backend.

L2

```
2 |     [ValidateSet('docker', 'docker,demo')][string]$Profile = 'docker',
```

Limita perfiles a docker y docker,demo.

L3

```
3 |     [switch]$NoBuild
```

Parámetro booleano: al pasarlo evita reconstruir imágenes y usa docker compose up -d con las existentes.

L4

```
4 | )
```

Cierra el arreglo/bloque anterior o abre la alternativa else. Solo organiza el flujo del script; no cambia datos del backend.

L5

```
5 | $ErrorActionPreference = 'Stop'
```

Stop convierte errores PowerShell no terminantes en errores que detienen el script. Los programas externos requieren además comprobar LASTEXITCODE; esta variable no reemplaza ese control.

L6

```
6 | $repo = Split-Path $PSScriptRoot -Parent
```

PSScriptRoot contiene la carpeta del script. Split-Path -Parent obtiene la raíz del repositorio para que las rutas funcionen aunque la terminal se haya abierto en otra carpeta.

L7

```
7 | $previousProfile = $env:SPRING_PROFILES_ACTIVE
```

Conserva/restaura perfil del proceso; no cambia por sí solo contenedores ya arrancados. Selecciona perfil transmitido a aplicaciones.

L8

```
8 | Push-Location $repo
```

Entra temporalmente en la raíz para resolver rutas/Compose.

L9

```
9 | try {
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L10

```
10 |     if (!(Test-Path -LiteralPath '.env')) { throw 'Copiar .env.example a .env y completar los
     secretos locales antes de iniciar.' }
```

Comprueba existencia de .env sin imprimir valores.

L11

```
11 |     & docker info --format '{{.ServerVersion}}'
```

Verifica motor Docker.

L12

```
12 |     if ($LASTEXITCODE -ne 0) { throw 'Iniciar Docker Desktop y esperar a que el motor este
     disponible.' }
```

Comprueba resultado de comando externo.

L13

```
13 |     $env:SPRING_PROFILES_ACTIVE = $Profile
```

Selecciona perfil transmitido a aplicaciones.

L14

```
14 |     & docker compose config --quiet
```

Valida Compose sin mostrar secretos.

L15

```
15 |     if ($LASTEXITCODE -ne 0) { throw 'Configuracion Compose incompleta.' }
```

Comprueba resultado de comando externo.

L16

```
16 |     if ($NoBuild) { & docker compose up -d } else { & docker compose up -d --build }
```

Construye imágenes e inicia/recrea contenedores. Arranca/recrea servicios sin borrar volúmenes.

L17

```
17 |     if ($LASTEXITCODE -ne 0) { throw 'Fallo la construccion o el arranque.' }
```

Comprueba resultado de comando externo.

L18

```
18 |     $deadline = (Get-Date).AddSeconds(120)
```

Plazo de 120 s para disponibilidad.

L19

```
19 |     do {
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L20

```
20 |         $ready = $true
```

Inicializa la bandera de sondeos como verdadera; cualquiera que falle la vuelve falsa y obliga a reintentar.

L21

```
21 |         foreach ($probe in @('http://fleet-service:8081/api/v1/operators/me',
     'http://assistance-service:8082/api/v1/vehicles', 'http://localhost:8000/api/v1/vehicles')) {
```

Sondea Fleet, Assistance y Gateway.

L22

```
22 |             $result = & docker compose exec -T api-gateway curl -s -o /dev/null --max-time 3 -w
     '%{http_code}' $probe 2>$null
```

Ejecuta curl dentro de Gateway, con DNS interno Docker.

L23

```
23 |             if ($LASTEXITCODE -ne 0 -or $result -ne '401') { $ready = $false }
```

Comprueba resultado de comando externo. 401 demuestra servidor disponible y protegido, no login completo.

L24

```
24 |         }
```

Cierra el arreglo/bloque anterior o abre la alternativa else. Solo organiza el flujo del script; no cambia datos del backend.

L25

```
25 |         if (!$ready) { Start-Sleep -Seconds 2 }
```

Espera dos segundos entre sondeos.

L26

```
26 |     } while (!$ready -and (Get-Date) -lt $deadline)
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L27

```
27 |     if (!$ready) { throw 'Servicios no disponibles en 120 s. Revisar docker compose logs --tail
     80.' }
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L28

```
28 |     Write-Host "AuxiRuta disponible en http://localhost:8000; Keycloak en http://localhost:8080.
     Perfil: $Profile"
```

Informa resultado sin credenciales.

L29

```
29 | } finally {
```

Restaura entorno o ubicación aun con error.

L30

```
30 |     $env:SPRING_PROFILES_ACTIVE = $previousProfile
```

Conserva/restaura perfil del proceso; no cambia por sí solo contenedores ya arrancados. Selecciona perfil transmitido a aplicaciones.

L31

```
31 |     Pop-Location
```

Restaura directorio anterior incluso ante fallo.

L32

```
32 | }
```

Cierra el arreglo/bloque anterior o abre la alternativa else. Solo organiza el flujo del script; no cambia datos del backend.

scripts/Verify-AuxiRuta.ps1

`Ruta: scripts/Verify-AuxiRuta.ps1`

Objetivo y funcionalidad

Compila/verifica los cuatro módulos y opcionalmente ejecuta integración secuencial, con restauración del perfil normal al terminar demo.

L1

```
1 | param([switch]$Integration, [switch]$IncludeDemo)
```

Declara parámetros del script. [switch] es un indicador que vale verdadero al escribir su nombre: Integration habilita contenedores y llamadas reales; IncludeDemo añade pruebas de tiempos acelerados. No es una variable del backend.

L2

```
2 | $ErrorActionPreference = 'Stop'
```

Stop convierte errores PowerShell no terminantes en errores que detienen el script. Los programas externos requieren además comprobar LASTEXITCODE; esta variable no reemplaza ese control.

L3

```
3 | if ($IncludeDemo -and !$Integration) { throw 'IncludeDemo requiere Integration.' }
```

Demo solo procede con integración habilitada.

L4

```
4 | $repo = Split-Path $PSScriptRoot -Parent
```

PSScriptRoot contiene la carpeta del script. Split-Path -Parent obtiene la raíz del repositorio para que las rutas funcionen aunque la terminal se haya abierto en otra carpeta.

L5

```
5 | function Invoke-Maven([string]$Module, [string[]]$Arguments) {
```

Ejecuta Wrapper del módulo y comprueba salida.

L6

```
6 |     & ".\$Module\mvnw.cmd" -f "$Module/pom.xml" @Arguments
```

El operador & ejecuta el Wrapper del módulo; -f selecciona su POM y @Arguments transmite objetivos/propiedades sin unirlos en texto.

L7

```
 7 |     if ($LASTEXITCODE -ne 0) { throw "Fallo Maven en $Module. Revisar target/surefire-reports." }
```

Comprueba resultado de comando externo.

L8

```
 8 | }
```

Cierra el arreglo/bloque anterior o abre la alternativa else. Solo organiza el flujo del script; no cambia datos del backend.

L9

```
 9 | Push-Location $repo
```

Entra temporalmente en la raíz para resolver rutas/Compose.

L10

```
10 | try {
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L11

```
11 |     Invoke-Maven 'auxiruta-commons' @('install')
```

Ejecuta Wrapper del módulo y comprueba salida.

L12

```
12 |     foreach ($module in @('fleet-service', 'assistance-service', 'api-gateway')) { Invoke-Maven
   $module @('verify') }
```

Ejecuta Wrapper del módulo y comprueba salida.

L13

```
13 |     if ($Integration) {
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L14

```
14 |         & "$PSScriptRoot\Start-AuxiRuta.ps1" -Profile docker
```

Llama al script de arranque situado junto a este script. -Profile selecciona tiempos normales o demo; -NoBuild reutiliza imágenes existentes.

L15

```
15 |         Invoke-Maven 'api-gateway' @('test', '-Dauxiruta.integration=true',
   '-Dauxiruta.realRouting=true', '-Dtest=BackendIntegrationTest')
```

Ejecuta Wrapper del módulo y comprueba salida.

L16

```
16 |         Invoke-Maven 'api-gateway' @('test', '-Dauxiruta.phase7=true',
   '-Dtest=Phase7IntegrationTest')
```

Ejecuta Wrapper del módulo y comprueba salida.

L17

```
17 |         Invoke-Maven 'api-gateway' @('test', '-Dauxiruta.phase8=true',
   '-Dtest=Phase8IntegrationTest')
```

Ejecuta Wrapper del módulo y comprueba salida.

L18

```
18 |         Invoke-Maven 'api-gateway' @('test', '-Dauxiruta.delivery=true',
   '-Dtest=DeliveryIntegrationTest')
```

Ejecuta Wrapper del módulo y comprueba salida.

L19

```
19 |         if ($IncludeDemo) {
```

Demo solo procede con integración habilitada.

L20

```
20 |             try {
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L21

```
21 |                 & "$PSScriptRoot\Start-AuxiRuta.ps1" -Profile 'docker,demo' -NoBuild
```

Llama al script de arranque situado junto a este script. -Profile selecciona tiempos normales o demo; -NoBuild reutiliza imágenes existentes.

L22

```
22 |                 Invoke-Maven 'api-gateway' @('test', '-Dauxiruta.phase9=true',
     '-Dtest=Phase9IntegrationTest')
```

Ejecuta Wrapper del módulo y comprueba salida.

L23

```
23 |             } finally { & "$PSScriptRoot\Start-AuxiRuta.ps1" -Profile docker -NoBuild }
```

Llama al script de arranque situado junto a este script. -Profile selecciona tiempos normales o demo; -NoBuild reutiliza imágenes existentes.

L24

```
24 |         }
```

Cierra el arreglo/bloque anterior o abre la alternativa else. Solo organiza el flujo del script; no cambia datos del backend.

L25

```
25 |     }
```

Cierra el arreglo/bloque anterior o abre la alternativa else. Solo organiza el flujo del script; no cambia datos del backend.

L26

```
26 |     Write-Host 'Verificacion completada. Resultados JUnit en target/surefire-reports de cada
     modulo.'
```

Informa resultado sin credenciales.

L27

```
27 | } finally { Pop-Location }
```

Restaura directorio anterior incluso ante fallo. Restaura entorno o ubicación aun con error.

2. Biblioteca común

La biblioteca comparte infraestructura estable, no entidades ni DTO de negocio.

GeoMath (auxiruta-commons)

`Ruta: auxiruta-commons/src/main/java/com/auxiruta/commons/GeoMath.java`

Objetivo y funcionalidad

Calcula distancia geográfica con Haversine para cobertura y preselección. No calcula rutas viales ni reemplaza ORS.

L1

```
1 | package com.auxiruta.commons;
```

Ubica el archivo en com.auxiruta.commons. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de auxiruta-commons sin compartir sus entidades con otro módulo.

L3 a L4

```
3 | /** Pure geographic calculation; coordinates are latitude/longitude in degrees. */
4 | public final class GeoMath {
```

Documentación para quien lee el fuente: Pure geographic calculation; coordinates are latitude/longitude in degrees. */ public final class GeoMath { No es una instrucción ejecutable y no impone la regla por sí sola.

L5

```
5 |   private GeoMath() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L7

```
7 |   public static double distance(double lat1, double lon1, double lat2, double lon2, double
    radius) {
```

Función pura de distancia: recibe dos coordenadas en grados y radio terrestre; devuelve metros si el radio está en metros.

L8 a L11

```
 8 |     double p1 = Math.toRadians(lat1),
 9 |         p2 = Math.toRadians(lat2),
10 |         dp = p2 - p1,
11 |         dl = Math.toRadians(lon2 - lon1);
```

Convierte grados a radianes. Se calcula p1 (p1) para reutilizar ese resultado en los pasos siguientes.

L12 a L13

```
12 |     double a =
13 |         Math.pow(Math.sin(dp / 2), 2) + Math.cos(p1) * Math.cos(p2) * Math.pow(Math.sin(dl / 2),
    2);
```

Calcula término de Haversine con latitudes y diferencia de longitudes. Se calcula a (a) para reutilizar ese resultado en los pasos siguientes.

L14

```
14 |     return 2 * radius * Math.asin(Math.sqrt(Math.max(0, Math.min(1, a))));
```

Obtiene distancia angular; clamp 0..1 evita errores numéricos. Resultado que recibe el llamador: elige el mayor valor, por ejemplo para evitar esperas negativas; elige el menor valor para respetar el límite más restrictivo.

L15

```
15 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L16

```
16 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 6: separadores sin instrucciones.

Problems (auxiruta-commons)

Ruta: auxiruta-commons/src/main/java/com/auxiruta/commons/errors/Problems.java

Objetivo y funcionalidad

Construye errores ProblemDetail homogéneos sin exponer consultas SQL, credenciales ni trazas internas.

L1

```
1 | package com.auxiruta.commons.errors;
```

Ubica el archivo en com.auxiruta.commons.errors. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de auxiruta-commons sin compartir sus entidades con otro módulo.

L3

```
3 | import org.springframework.http.*;
```

Importa tipos: org.springframework.http.*. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | public final class Problems {
```

Declara clase Problems. Construye errores ProblemDetail homogéneos sin exponer consultas SQL, credenciales ni trazas internas.

L6

```
6 |   private Problems() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L8

```
8 |   public static ProblemDetail of(int status) {
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Tipos y datos: status = estado del objeto; su enum o CHECK define valores permitidos; int es entero de 32 bits para límites y contadores.

L9

```
9 |     HttpStatus http = HttpStatus.resolve(status);
```

Busca nombre HTTP conocido; puede ser null para código no enumerado. Se calcula http (http) para reutilizar ese resultado en los pasos siguientes.

L10 a L21

```
10 |     ProblemDetail p =
11 |         ProblemDetail.forStatusAndDetail(
12 |             HttpStatusCode.valueOf(status),
13 |             switch (status) {
14 |               case 400 -> "Entrada invalida";
15 |               case 401 -> "Autenticacion requerida o token invalido";
16 |               case 403 -> "Permiso insuficiente";
17 |               case 404 -> "Recurso no encontrado";
18 |               case 409 -> "Conflicto con el estado del recurso";
19 |               case 503 -> "Servicio temporalmente no disponible";
20 |               default -> status >= 500 ? "Error interno" : "La solicitud no pudo procesarse";
21 |             });
```

Crea problema con estado y descripción elegida por switch. Se calcula p (p) para reutilizar ese resultado en los pasos siguientes. el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario

L22

```
22 |     p.setTitle(http == null ? "Error" : http.getReasonPhrase());
```

Usa razón HTTP estándar como título, con fallback Error. Detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L23

```
23 |     p.setProperty("code", "HTTP_" + status);
```

Añade código estable HTTP_n para facilitar interpretación del cliente.

L24

```
24 |     return p;
```

Entrega p al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: p.

L25

```
25 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L26

```
26 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 4, 7: separadores sin instrucciones.

TokenSupport (auxiruta-commons)

`Ruta: auxiruta-commons/src/main/java/com/auxiruta/commons/security/TokenSupport.java`

Objetivo y funcionalidad

Centraliza los roles admitidos, validaciones complementarias de JWT y propiedad. Distingue tokens humanos de tokens técnicos por audiencia y cliente autorizado; ADMIN no hereda las acciones de DRIVER u OPERATOR.

L1

```
1 | package com.auxiruta.commons.security;
```

Ubica el archivo en com.auxiruta.commons.security. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de auxiruta-commons sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.security.core.authority.SimpleGrantedAuthority;
```

Importa tipos: org.springframework.security.core.authority.SimpleGrantedAuthority. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.security.oauth2.core.*;
```

Importa tipos: org.springframework.security.oauth2.core.*. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.security.oauth2.jwt.*;
```

Importa tipos: org.springframework.security.oauth2.jwt.*. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
 7 | import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
 8 | import org.springframework.http.HttpStatus;
```

Importa tipos: org.springframework.http.HttpStatus. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | public final class TokenSupport {
```

Declara clase TokenSupport. Centraliza los roles admitidos, validaciones complementarias de JWT y propiedad. Distingue tokens humanos de tokens técnicos por audiencia y cliente autorizado; ADMIN no hereda las acciones de DRIVER u OPERATOR.

L11

```
11 |   private TokenSupport() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L13

```
13 |   public static Collection<SimpleGrantedAuthority> roles(Jwt jwt) {
```

Extrae solo roles de negocio reconocidos desde realm_access.roles. Tipos y datos: jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias.

L14

```
14 |     Object realm = jwt.getClaims().get("realm_access");
```

Extrae realm_access como Object porque el claim puede faltar o no tener el tipo esperado. Se calcula realm (realm) para reutilizar ese resultado en los pasos siguientes.

L15 a L16

```
15 |     if (!(realm instanceof Map<?, ?> map) || !(map.get("roles") instanceof Collection<?> roles))
16 |       return List.of();
```

Esta condición decide si se entra en la rama siguiente. crea una lista no modificable de los elementos indicados; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L17 a L23

```
17 |     return roles.stream()
18 |         .filter(String.class::isInstance)
19 |         .map(String.class::cast)
20 |         .filter(Set.of("DRIVER", "OPERATOR", "ADMIN", "FLEET_SERVICE")::contains)
21 |         .distinct()
22 |         .map(role -> new SimpleGrantedAuthority("ROLE_" + role))
23 |         .toList();
```

Conserva solo elementos que cumplen el predicado. Transforma cada elemento al valor/tipo de salida. Materializa la secuencia en lista. Resultado que recibe el llamador: inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente; retiene solo elementos que cumplen la condición; transforma cada elemento al tipo/valor de salida; elimina duplicados para no repetir roles o candidatos; materializa el resultado de la secuencia en una lista; crea un conjunto no modificable y sin duplicados.

L18: retiene solo elementos que cumplen la condición.

L19: transforma cada elemento al tipo/valor de salida.

L20: retiene solo elementos que cumplen la condición; crea un conjunto no modificable y sin duplicados.

L21: elimina duplicados para no repetir roles o candidatos.

L22: transforma cada elemento al tipo/valor de salida.

L23: materializa el resultado de la secuencia en una lista.

L24

```
24 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L26

```
26 |   public static OAuth2TokenValidator<Jwt> validator(String issuer, Set<String> audiences) {
```

Compone validadores de emisor/tiempo con presencia de sub, expiración y audiencia aceptada. Tipos y datos: issuer = emisor Keycloak esperado del JWT; String es texto; no es por sí mismo una referencia de entidad.

L27 a L40

```
27 |     return new DelegatingOAuth2TokenValidator<>(
28 |         JwtValidators.createDefaultWithIssuer(issuer),
29 |         jwt -> {
30 |           boolean valid =
31 |               jwt.getExpiresAt() != null
32 |                   && jwt.getSubject() != null
33 |                   && !jwt.getSubject().isBlank()
34 |                   && jwt.getAudience() != null
35 |                   && jwt.getAudience().stream().anyMatch(audiences::contains);
36 |           return valid
37 |               ? OAuth2TokenValidatorResult.success()
38 |               : OAuth2TokenValidatorResult.failure(
39 |                   new OAuth2Error("invalid_token", "Token no valido para este servicio", null));
40 |         });
```

Combina los validadores estándar de emisor/tiempo con el predicado adicional de audiencia, subject y expiración. Devuelve éxito del validador únicamente si todas las condiciones adicionales se cumplen. Construye error invalid_token sin revelar detalles de la identidad. Resultado que recibe el llamador: obtiene sub, identidad estable del usuario autenticado; obtiene los destinatarios permitidos del token; comprueba texto vacío o solo espacios; devuelve verdadero si al menos un elemento cumple el predicado; inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue; comprueba presencia antes de usar el dato; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L32: obtiene sub, identidad estable del usuario autenticado; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato.

L33: obtiene sub, identidad estable del usuario autenticado; comprueba texto vacío o solo espacios; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L34: obtiene los destinatarios permitidos del token; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato.

L35: obtiene los destinatarios permitidos del token; devuelve verdadero si al menos un elemento cumple el predicado; inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L41

```
41 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L43

```
43 |   public static boolean permits(Jwt jwt, boolean technical, String... requiredRoles) {
```

Autoriza por audiencia, azp y rol, separando acceso humano y técnico. Tipos y datos: jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias; technical = indica si se exigen cliente y audiencia de servicio; boolean es verdadero/falso, sin null.

L44

```
44 |     String audience = technical ? "auxiruta-fleet" : "auxiruta-api";
```

Selecciona auxiruta-fleet para tráfico técnico y auxiruta-api para humano. Se calcula audience (audience) para reutilizar ese resultado en los pasos siguientes. el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario

L45

```
45 |     String client = technical ? "assistance-service" : "auxiruta-api";
```

Exige assistance-service como cliente técnico y auxiruta-api para login humano. Se calcula client (client) para reutilizar ese resultado en los pasos siguientes. el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario

L46 a L53

```
46 |     return jwt.getAudience() != null
47 |         && jwt.getAudience().contains(audience)
48 |         && client.equals(jwt.getClaimAsString("azp"))
49 |         && roles(jwt).stream()
50 |             .anyMatch(
51 |                 a ->
52 |                     Arrays.stream(requiredRoles)
53 |                         .anyMatch(role -> a.getAuthority().equals("ROLE_" + role)));
```

Exige al menos uno de los roles solicitados después de validar audiencia y cliente. Resultado que recibe el llamador: obtiene los destinatarios permitidos del token; obtiene el cliente que solicitó el token; compara contenido/identidad lógica, no referencias Java; verifica pertenencia al conjunto o lista; devuelve verdadero si al menos un elemento cumple el predicado; inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato.

L47: obtiene los destinatarios permitidos del token; verifica pertenencia al conjunto o lista; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L48: obtiene el cliente que solicitó el token; compara contenido/identidad lógica, no referencias Java; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L49: inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L50: devuelve verdadero si al menos un elemento cumple el predicado.

L53: compara contenido/identidad lógica, no referencias Java; devuelve verdadero si al menos un elemento cumple el predicado.

L54

```
54 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L56

```
56 |   public static String subject(Jwt jwt) {
```

Tipos y datos: jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias.

L57

```
57 |     return jwt.getSubject();
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Resultado que recibe el llamador: obtiene sub, identidad estable del usuario autenticado.

L58

```
58 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L60

```
60 |   public static void requireOwner(Jwt jwt, String ownerSubject) {
```

Devuelve 404 si el sub del JWT no coincide con el propietario. Campos: ownerSubject: sub de Keycloak del propietario. Tipos y datos: jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias; ownerSubject = sub de Keycloak del propietario; String es texto; no es por sí mismo una referencia de entidad.

L61 a L62

```
61 |     if (ownerSubject == null || !ownerSubject.equals(subject(jwt)))
62 |       throw new ResponseStatusException(HttpStatus.NOT_FOUND, "Recurso no encontrado");
```

Oculta recursos ajenos con el mismo 404 que los inexistentes. Esta condición decide si se entra en la rama siguiente. compara contenido/identidad lógica, no referencias Java; interrumpe la operación con un estado HTTP explícito; 404 oculta inexistencia o pertenencia ajena; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue; detecta un dato ausente.

L63

```
63 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L64

```
64 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 9, 12, 25, 42, 55, 59: separadores sin instrucciones.

UtcClock (auxiruta-commons)

`Ruta: auxiruta-commons/src/main/java/com/auxiruta/commons/time/UtcClock.java`

Objetivo y funcionalidad

Proporciona un reloj UTC compartido. Inyectar Clock permite controlar el tiempo en pruebas sin alterar la hora del sistema.

L1

```
1 | package com.auxiruta.commons.time;
```

Ubica el archivo en com.auxiruta.commons.time. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de auxiruta-commons sin compartir sus entidades con otro módulo.

L3

```
3 | import java.time.Clock;
```

Importa tipos: java.time.Clock. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5 a L6

```
5 | /** Provee el reloj UTC que las aplicaciones pueden sustituir en pruebas temporales. */
6 | public final class UtcClock {
```

Documentación para quien lee el fuente: Provee el reloj UTC que las aplicaciones pueden sustituir en pruebas temporales. */ public final class UtcClock { No es una instrucción ejecutable y no impone la regla por sí sola.

L7

```
7 |   private UtcClock() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L9

```
9 |   public static Clock system() {
```

Fábrica común del reloj; permite que cada servicio comparta la decisión de usar UTC.

L10

```
10 |     return Clock.systemUTC();
```

Crea reloj de sistema con zona UTC; las pruebas pueden sustituirlo por Clock fijo.

L11

```
11 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L12

```
12 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 4, 8: separadores sin instrucciones.

3. Arranque, seguridad, Gateway y errores

Keycloak autentica. Gateway y servicios verifican JWT por separado; la propiedad se controla adicionalmente en servicios/repositorios.

GatewayApplication (api-gateway)

Ruta: api-gateway/src/main/java/com/auxiruta/gateway/GatewayApplication.java

Objetivo y funcionalidad

Punto de entrada del Gateway reactivo: recibe tráfico público y lo dirige a los microservicios.

L1

```
1 | package com.auxiruta.gateway;
```

Ubica el archivo en com.auxiruta.gateway. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de api-gateway sin compartir sus entidades con otro módulo.

L3

```
3 | import org.springframework.boot.SpringApplication;
```

Importa tipos: org.springframework.boot.SpringApplication. Arranque/autoconfiguración y condiciones de activación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.boot.autoconfigure.SpringBootApplication;
```

Importa tipos: org.springframework.boot.autoconfigure.SpringBootApplication. Arranque/autoconfiguración y condiciones de activación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | @SpringBootApplication
```

Activa autoconfiguración y escaneo de componentes.

L7

```
7 | public class GatewayApplication {
```

Declara clase GatewayApplication. Punto de entrada del Gateway reactivo: recibe tráfico público y lo dirige a los microservicios.

L8

```
8 |   public static void main(String[] args) {
```

Entrada JVM: recibe argumentos y el cuerpo arranca Spring Boot.

L9

```
9 |     SpringApplication.run(GatewayApplication.class, args);
```

Arranca contexto Spring y servidor HTTP del módulo.

L10

```
10 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L11

```
11 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 5: separadores sin instrucciones.

GatewayRoutes (api-gateway)

Ruta: api-gateway/src/main/java/com/auxiruta/gateway/config/GatewayRoutes.java

Objetivo y funcionalidad

Enruta los endpoints públicos hacia Fleet o Assistance. Las rutas /internal no se publican. Ajusta caché DNS para recuperar contenedores recreados.

L1

```
1 | package com.auxiruta.gateway.config;
```

Ubica el archivo en com.auxiruta.gateway.config. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de api-gateway sin compartir sus entidades con otro módulo.

L3

```
3 | import org.springframework.beans.factory.annotation.Value;
```

Importa tipos: org.springframework.beans.factory.annotation.Value. Inyección de dependencias y propiedades de entorno. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.context.annotation.*;
```

Importa tipos: org.springframework.context.annotation.*. Beans, configuración y eventos del contexto Spring. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.cloud.gateway.route.*;
```

Importa tipos: org.springframework.cloud.gateway.route.*. Rutas y builder del Gateway reactivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.cloud.gateway.route.builder.RouteLocatorBuilder;
```

Importa tipos: org.springframework.cloud.gateway.route.builder.RouteLocatorBuilder. Rutas y builder del Gateway reactivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | @Configuration
```

Declara configuración Spring.

L9

```
9 | public class GatewayRoutes {
```

Declara clase GatewayRoutes. Enruta los endpoints públicos hacia Fleet o Assistance. Las rutas /internal no se publican. Ajusta caché DNS para recuperar contenedores recreados.

L10

```
10 |   @Bean
```

Registra el objeto retornado en Spring.

L11

```
11 |   org.springframework.cloud.gateway.config.HttpClientCustomizer containerDns() {
```

Expone personalización del cliente HTTP de Gateway para recuperación DNS.

L12 a L16

```
12 |     return client ->
13 |         client.resolver(
14 |             spec ->
15 |                 spec.cacheMaxTimeToLive(java.time.Duration.ofSeconds(10))
16 |                     .cacheNegativeTimeToLive(java.time.Duration.ZERO));
```

DNS positivo hasta 10 s y negativo 0 s para recuperarse de recreación de contenedores.

L17

```
17 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L19

```
19 |   @Bean
```

Registra el objeto retornado en Spring.

L20 a L23

```
20 |   RouteLocator routes(
21 |       RouteLocatorBuilder builder,
22 |       @Value("${FLEET_URI:http://fleet-service:8081}") String fleet,
23 |       @Value("${ASSISTANCE_URI:http://assistance-service:8082}") String assistance) {
```

Define bean de rutas con builder y URIs de servicios inyectadas. Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición. Tipos y datos: fleet = cliente técnico que consulta o modifica reservas en Fleet; String es texto; no es por sí mismo una referencia de entidad; assistance = assistance; String es texto; no es por sí mismo una referencia de entidad.

L24 a L53

```
24 |     return builder
25 |         .routes()
26 |         .route(
27 |             "fleet",
28 |             r ->
29 |                 r.path(
30 |                         "/api/v1/operators/**",
31 |                         "/api/v1/tow-trucks/**",
32 |                         "/api/v1/configuration/fleet")
33 |                     .uri(fleet))
34 |         .route(
35 |             "assistance",
36 |             r ->
37 |                 r.path(
38 |                         "/api/v1/vehicles/**",
39 |                         "/api/v1/assistance-requests/**",
40 |                         "/api/v1/assignments/**",
41 |                         "/api/v1/operator/assignments",
42 |                         "/api/v1/configuration/assistance")
43 |                     .uri(assistance))
44 |         .route(
45 |             "fleet-docs",
46 |             r -> r.path("/docs/fleet/v3/api-docs").filters(f -> f.stripPrefix(2)).uri(fleet))
47 |         .route(
48 |             "assistance-docs",
49 |             r ->
50 |                 r.path("/docs/assistance/v3/api-docs")
51 |                     .filters(f -> f.stripPrefix(2))
52 |                     .uri(assistance))
53 |         .build();
```

Inicia el builder de rutas; cada .route posterior incorpora un destino. Construye el objeto final a partir del builder configurado en líneas previas. Resultado que recibe el llamador: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; Ruta JSON consultada: /docs/fleet/v3/api-docs./docs/assistance/v3/api-docs.

L54

```
54 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L55

```
55 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 7, 18: separadores sin instrucciones.

RequestLoggingFilter (api-gateway)

Ruta: api-gateway/src/main/java/com/auxiruta/gateway/config/RequestLoggingFilter.java

Objetivo y funcionalidad

Registra método, ruta, estado y duración de las peticiones de api-gateway mediante el flujo reactivo. No guarda tokens/cuerpos y no sustituye el historial de negocio.

L1

```
1 | package com.auxiruta.gateway.config;
```

Ubica el archivo en com.auxiruta.gateway.config. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de api-gateway sin compartir sus entidades con otro módulo.

L3

```
3 | import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.core.annotation.Order;
```

Importa tipos: org.springframework.core.annotation.Order. Tipo auxiliar usado por las expresiones del archivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.web.server.*;
```

Importa tipos: org.springframework.web.server.*. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import reactor.core.publisher.Mono;
```

Importa tipos: reactor.core.publisher.Mono. Mono y operadores de finalización del flujo reactivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.slf4j.LoggerFactory;
```

Importa tipos: org.slf4j.LoggerFactory. Logging de eventos y fallos sin acoplarse a una implementación de consola. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | @Component
```

Registra componente inyectable.

L10

```
10 | @Order(-200)
```

Ordena filtro/manejador.

L11

```
11 | public class RequestLoggingFilter implements WebFilter {
```

Declara clase RequestLoggingFilter. Mide duración y registra método y estado HTTP, sin guardar tokens ni cuerpos. Gateway usa una cadena reactiva; los servicios usan el filtro Servlet.

L12

```
12 |   public Mono<Void> filter(ServerWebExchange x, WebFilterChain chain) {
```

Implementa filtro reactivo; Mono expresa finalización sin cuerpo adicional.

L13

```
13 |     long start = System.nanoTime();
```

Captura tiempo monotónico para medir duración de la petición. No se usa como fecha de negocio ni para los vencimientos persistidos; el contador sirve para restar inicio y fin.

L14 a L23

```
14 |     return chain
15 |         .filter(x)
16 |         .doFinally(
17 |             signal ->
18 |                 LoggerFactory.getLogger(getClass())
19 |                     .info(
20 |                         "HTTP method={} status={} durationMs={}",
21 |                         x.getRequest().getMethod(),
22 |                         x.getResponse().getStatusCode(),
23 |                         (System.nanoTime() - start) / 1_000_000));
```

chain.filter(x) entrega el intercambio HTTP al resto de filtros reactivos; aquí filter no es Stream.filter y no elimina elementos. doFinally registra el resultado al finalizar, fallar o cancelarse el flujo. nanoTime mide tiempo transcurrido; dividir por 1 000 000 lo expresa en milisegundos. Los marcadores {} de SLF4J reciben método, estado y duración sin incluir el token.

L15: filter(x) continúa la cadena de filtros HTTP; no es un filtro de colección.

L16: doFinally ejecuta el registro al terminar la cadena, incluso ante error o cancelación.

L24

```
24 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L25

```
25 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 8: separadores sin instrucciones.

SecurityConfiguration (api-gateway)

`Ruta: api-gateway/src/main/java/com/auxiruta/gateway/config/SecurityConfiguration.java`

Objetivo y funcionalidad

Define permisos por ruta y método en api-gateway y la validación Bearer JWT. Su cadena reactiva protege la entrada pública y no expone endpoints internos.

L1

```
1 | package com.auxiruta.gateway.config;
```

Ubica el archivo en com.auxiruta.gateway.config. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de api-gateway sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.Set;
```

Importa tipos: java.util.Set. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import com.auxiruta.commons.security.TokenSupport;
```

Importa tipos: com.auxiruta.commons.security.TokenSupport. Utilidades compartidas de seguridad, errores, reloj o geografía. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.beans.factory.annotation.Value;
```

Importa tipos: org.springframework.beans.factory.annotation.Value. Inyección de dependencias y propiedades de entorno. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.context.annotation.*;
```

Importa tipos: org.springframework.context.annotation.*. Beans, configuración y eventos del contexto Spring. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.http.HttpMethod;
```

Importa tipos: org.springframework.http.HttpMethod. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.springframework.security.authorization.AuthorizationDecision;
```

Importa tipos: org.springframework.security.authorization.AuthorizationDecision. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | import org.springframework.security.oauth2.jwt.*;
```

Importa tipos: org.springframework.security.oauth2.jwt.*. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import org.springframework.security.oauth2.server.resource.authentication.*;
```

Importa tipos: org.springframework.security.oauth2.server.resource.authentication.*. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | import org.springframework.security.config.web.server.ServerHttpSecurity;
```

Importa tipos: org.springframework.security.config.web.server.ServerHttpSecurity. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12

```
12 | import org.springframework.security.web.server.SecurityWebFilterChain;
```

Importa tipos: org.springframework.security.web.server.SecurityWebFilterChain. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L13

```
13 | import org.springframework.security.web.server.context.NoOpServerSecurityContextRepository;
```

Importa tipos: org.springframework.security.web.server.context.NoOpServerSecurityContextRepository. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L14

```
14 | import org.springframework.security.authorization.ReactiveAuthorizationManager;
```

Importa tipos: org.springframework.security.authorization.ReactiveAuthorizationManager. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L15

```
15 | import org.springframework.security.web.server.authorization.AuthorizationContext;
```

Importa tipos: org.springframework.security.web.server.authorization.AuthorizationContext. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L17

```
17 | @Configuration
```

Declara configuración Spring.

L18

```
18 | public class SecurityConfiguration {
```

Declara clase SecurityConfiguration. Configura validación JWT y permisos por ruta y método. El primer matcher aplicable decide; las rutas específicas preceden a las generales y lo no enumerado se deniega.

L19

```
19 |   @Bean
```

Registra el objeto retornado en Spring.

L20 a L22

```
20 |   ReactiveJwtDecoder jwtDecoder(
21 |       @Value("${spring.security.oauth2.resourceserver.jwt.issuer-uri}") String issuer,
22 |       @Value("${spring.security.oauth2.resourceserver.jwt.jwk-set-uri}") String keys) {
```

Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición. Define bean que decodifica y verifica el JWT; las URLs de emisor/claves provienen de configuración. Tipos y datos: issuer = emisor Keycloak esperado del JWT; String es texto; no es por sí mismo una referencia de entidad; keys = keys; String es texto; no es por sí mismo una referencia de entidad.

L23

```
23 |     var decoder = NimbusReactiveJwtDecoder.withJwkSetUri(keys).build();
```

Construye el objeto final a partir del builder configurado en líneas previas. Se calcula decoder (decoder) para reutilizar ese resultado en los pasos siguientes.

L24

```
24 |     decoder.setJwtValidator(TokenSupport.validator(issuer, Set.of("auxiruta-api")));
```

Añade validación de emisor, tiempo, sub y audiencia. Crea un conjunto no modificable y sin duplicados.

L25

```
25 |     return decoder;
```

Entrega el decodificador ya configurado al contenedor Spring. Resultado que recibe el llamador: decoder.

L26

```
26 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L28

```
28 |   @Bean
```

Registra el objeto retornado en Spring.

L29

```
29 |   SecurityWebFilterChain security(ServerHttpSecurity http, SecurityErrors errors) {
```

Construye la cadena reactiva de seguridad del Gateway.

L30

```
30 |     var converter = new JwtAuthenticationConverter();
```

Construye JwtAuthenticationConverter en memoria y conserva la referencia en converter. Los argumentos del constructor determinan los datos iniciales de este objeto. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L31 a L32

```
31 |     converter.setJwtGrantedAuthoritiesConverter(
32 |         jwt -> new java.util.ArrayList<>(TokenSupport.roles(jwt)));
```

Traduce roles de Keycloak a ROLE_ de Spring.

L33 a L136

```
33 |     http.csrf(ServerHttpSecurity.CsrfSpec::disable)
34 |         .httpBasic(ServerHttpSecurity.HttpBasicSpec::disable)
35 |         .formLogin(ServerHttpSecurity.FormLoginSpec::disable)
36 |         .securityContextRepository(NoOpServerSecurityContextRepository.getInstance())
37 |         .exceptionHandling(e -> e.authenticationEntryPoint(errors).accessDeniedHandler(errors))
38 |         .oauth2ResourceServer(
39 |             o ->
40 |                 o.authenticationEntryPoint(errors)
41 |                     .accessDeniedHandler(errors)
42 |                     .jwt(
43 |                         j ->
44 |                             j.jwtAuthenticationConverter(
45 |                                 new ReactiveJwtAuthenticationConverterAdapter(converter))))
46 |         .authorizeExchange(
47 |             auth ->
48 |                 auth.pathMatchers(HttpMethod.GET, "/api/v1/operators/me")
49 |                     .access(allowed(false, "OPERATOR"))
50 |                     .pathMatchers(HttpMethod.PATCH, "/api/v1/operators/me/availability")
51 |                     .access(allowed(false, "OPERATOR"))
52 |                     .pathMatchers(HttpMethod.GET, "/api/v1/tow-trucks/assigned-to-me")
53 |                     .access(allowed(false, "OPERATOR"))
54 |                     .pathMatchers(HttpMethod.PUT, "/api/v1/tow-trucks/assigned-to-me/location")
55 |                     .access(allowed(false, "OPERATOR"))
56 |                     .pathMatchers(
57 |                         "/api/v1/operators/me",
58 |                         "/api/v1/operators/me/**",
59 |                         "/api/v1/tow-trucks/assigned-to-me",
60 |                         "/api/v1/tow-trucks/assigned-to-me/**")
61 |                     .denyAll()
62 |                     .pathMatchers(HttpMethod.GET, "/api/v1/operators")
63 |                     .access(allowed(false, "ADMIN"))
64 |                     .pathMatchers(HttpMethod.GET, "/api/v1/operators/{id}")
65 |                     .access(allowed(false, "ADMIN"))
66 |                     .pathMatchers(HttpMethod.POST, "/api/v1/operators")
67 |                     .access(allowed(false, "ADMIN"))
68 |                     .pathMatchers(HttpMethod.PATCH, "/api/v1/operators/{id}")
69 |                     .access(allowed(false, "ADMIN"))
70 |                     .pathMatchers(HttpMethod.PATCH, "/api/v1/operators/{id}/registration-status")
71 |                     .access(allowed(false, "ADMIN"))
72 |                     .pathMatchers(HttpMethod.PUT, "/api/v1/operators/{id}/qualification")
73 |                     .access(allowed(false, "ADMIN"))
74 |                     .pathMatchers(HttpMethod.GET, "/api/v1/tow-trucks")
75 |                     .access(allowed(false, "ADMIN"))
76 |                     .pathMatchers(HttpMethod.GET, "/api/v1/tow-trucks/{id}")
77 |                     .access(allowed(false, "ADMIN"))
78 |                     .pathMatchers(HttpMethod.POST, "/api/v1/tow-trucks")
79 |                     .access(allowed(false, "ADMIN"))
80 |                     .pathMatchers(HttpMethod.PATCH, "/api/v1/tow-trucks/{id}")
81 |                     .access(allowed(false, "ADMIN"))
82 |                     .pathMatchers(HttpMethod.PUT, "/api/v1/tow-trucks/{id}/operator")
83 |                     .access(allowed(false, "ADMIN"))
84 |                     .pathMatchers(HttpMethod.DELETE, "/api/v1/tow-trucks/{id}/operator")
85 |                     .access(allowed(false, "ADMIN"))
86 |                     .pathMatchers(HttpMethod.PATCH, "/api/v1/tow-trucks/{id}/operational-status")
87 |                     .access(allowed(false, "ADMIN"))
88 |                     .pathMatchers(HttpMethod.GET, "/api/v1/configuration/fleet")
89 |                     .access(allowed(false, "ADMIN"))
90 |                     .pathMatchers(HttpMethod.POST, "/api/v1/vehicles")
91 |                     .access(allowed(false, "DRIVER"))
92 |                     .pathMatchers(HttpMethod.GET, "/api/v1/vehicles")
```

```
 93 |                     .access(allowed(false, "DRIVER"))
 94 |                     .pathMatchers(HttpMethod.GET, "/api/v1/vehicles/{id}")
 95 |                     .access(allowed(false, "DRIVER"))
 96 |                     .pathMatchers(HttpMethod.PATCH, "/api/v1/vehicles/{id}")
 97 |                     .access(allowed(false, "DRIVER"))
 98 |                     .pathMatchers(HttpMethod.PATCH, "/api/v1/vehicles/{id}/registration-status")
 99 |                     .access(allowed(false, "DRIVER"))
100 |                     .pathMatchers(HttpMethod.POST, "/api/v1/assistance-requests")
101 |                     .access(allowed(false, "DRIVER"))
102 |                     .pathMatchers(HttpMethod.POST,
      "/api/v1/assistance-requests/{id}/cancellations")
103 |                     .access(allowed(false, "DRIVER"))
104 |                     .pathMatchers(HttpMethod.GET, "/api/v1/assistance-requests")
105 |                     .access(allowed(false, "DRIVER", "OPERATOR", "ADMIN"))
106 |                     .pathMatchers(HttpMethod.GET, "/api/v1/assistance-requests/{id}")
107 |                     .access(allowed(false, "DRIVER", "OPERATOR", "ADMIN"))
108 |                     .pathMatchers(HttpMethod.GET, "/api/v1/assistance-requests/{id}/history")
109 |                     .access(allowed(false, "DRIVER", "OPERATOR", "ADMIN"))
110 |                     .pathMatchers(HttpMethod.GET, "/api/v1/operator/assignments")
111 |                     .access(allowed(false, "OPERATOR"))
112 |                     .pathMatchers(HttpMethod.GET, "/api/v1/assignments/{id}")
113 |                     .access(allowed(false, "OPERATOR", "ADMIN"))
114 |                     .pathMatchers(HttpMethod.POST, "/api/v1/assignments/{id}/acceptance")
115 |                     .access(allowed(false, "OPERATOR"))
116 |                     .pathMatchers(HttpMethod.POST, "/api/v1/assignments/{id}/rejection")
117 |                     .access(allowed(false, "OPERATOR"))
118 |                     .pathMatchers(HttpMethod.POST, "/api/v1/assistance-requests/{id}/dispatch")
119 |                     .access(allowed(false, "OPERATOR"))
120 |                     .pathMatchers(HttpMethod.POST, "/api/v1/assistance-requests/{id}/arrival")
121 |                     .access(allowed(false, "OPERATOR"))
122 |                     .pathMatchers(HttpMethod.POST, "/api/v1/assistance-requests/{id}/loading")
123 |                     .access(allowed(false, "OPERATOR"))
124 |                     .pathMatchers(
125 |                         HttpMethod.POST, "/api/v1/assistance-requests/{id}/transport-start")
126 |                     .access(allowed(false, "OPERATOR"))
127 |                     .pathMatchers(HttpMethod.POST, "/api/v1/assistance-requests/{id}/completion")
128 |                     .access(allowed(false, "OPERATOR"))
129 |                     .pathMatchers(HttpMethod.GET, "/api/v1/configuration/assistance")
130 |                     .access(allowed(false, "ADMIN"))
131 |                     .pathMatchers(HttpMethod.GET, "/docs/fleet/v3/api-docs")
132 |                     .access(allowed(false, "DRIVER", "OPERATOR", "ADMIN"))
133 |                     .pathMatchers(HttpMethod.GET, "/docs/assistance/v3/api-docs")
134 |                     .access(allowed(false, "DRIVER", "OPERATOR", "ADMIN"))
135 |                     .anyExchange()
136 |                     .denyAll());
```

Construye la cadena de seguridad. Desactiva login por formulario y HTTP Basic porque los clientes presentan Bearer JWT de Keycloak. El contexto no se guarda como sesión de usuario. El conversor traduce roles del JWT a authorities y los manejadores normalizan 401/403. Después aplica reglas en orden: el primer patrón coincidente decide; las rutas específicas deben preceder a patrones amplios. allowed(false, ...) exige identidad humana; allowed(true, ...) exige cliente técnico. denyAll deniega una coincidencia explícita o, al final, cualquier petición que no haya coincidido. No significa que todas las rutas previas estén denegadas.

L49: permite GET /api/v1/operators/me con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L51: permite PATCH /api/v1/operators/me/availability con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L53: permite GET /api/v1/tow-trucks/assigned-to-me con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L55: permite PUT /api/v1/tow-trucks/assigned-to-me/location con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L61: deniega el patrón previo o cualquier ruta restante si sigue a anyRequest/anyExchange.

L63: permite GET /api/v1/operators con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L65: permite GET /api/v1/operators/{id} con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L67: permite POST /api/v1/operators con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L69: permite PATCH /api/v1/operators/{id} con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L71: permite PATCH /api/v1/operators/{id}/registration-status con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L73: permite PUT /api/v1/operators/{id}/qualification con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L75: permite GET /api/v1/tow-trucks con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L77: permite GET /api/v1/tow-trucks/{id} con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L79: permite POST /api/v1/tow-trucks con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L81: permite PATCH /api/v1/tow-trucks/{id} con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L83: permite PUT /api/v1/tow-trucks/{id}/operator con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L85: permite DELETE /api/v1/tow-trucks/{id}/operator con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L87: permite PATCH /api/v1/tow-trucks/{id}/operational-status con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L89: permite GET /api/v1/configuration/fleet con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L91: permite POST /api/v1/vehicles con DRIVER. Exige token humano; la propiedad se valida además en negocio.

L93: permite GET /api/v1/vehicles con DRIVER. Exige token humano; la propiedad se valida además en negocio.

L95: permite GET /api/v1/vehicles/{id} con DRIVER. Exige token humano; la propiedad se valida además en negocio.

L97: permite PATCH /api/v1/vehicles/{id} con DRIVER. Exige token humano; la propiedad se valida además en negocio.

L99: permite PATCH /api/v1/vehicles/{id}/registration-status con DRIVER. Exige token humano; la propiedad se valida además en negocio.

L101: permite POST /api/v1/assistance-requests con DRIVER. Exige token humano; la propiedad se valida además en negocio.

L103: permite POST /api/v1/assistance-requests/{id}/cancellations con DRIVER. Exige token humano; la propiedad se valida además en negocio.

L105: permite GET /api/v1/assistance-requests con DRIVER, OPERATOR, ADMIN. Exige token humano; la propiedad se valida además en negocio.

L107: permite GET /api/v1/assistance-requests/{id} con DRIVER, OPERATOR, ADMIN. Exige token humano; la propiedad se valida además en negocio.

L109: permite GET /api/v1/assistance-requests/{id}/history con DRIVER, OPERATOR, ADMIN. Exige token humano; la propiedad se valida además en negocio.

L111: permite GET /api/v1/operator/assignments con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L113: permite GET /api/v1/assignments/{id} con OPERATOR, ADMIN. Exige token humano; la propiedad se valida además en negocio.

L115: permite POST /api/v1/assignments/{id}/acceptance con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L117: permite POST /api/v1/assignments/{id}/rejection con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L119: permite POST /api/v1/assistance-requests/{id}/dispatch con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L121: permite POST /api/v1/assistance-requests/{id}/arrival con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L123: permite POST /api/v1/assistance-requests/{id}/loading con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L126: permite POST /api/v1/assistance-requests/{id}/transport-start con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L128: permite POST /api/v1/assistance-requests/{id}/completion con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L130: permite GET /api/v1/configuration/assistance con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L132: permite GET /docs/fleet/v3/api-docs con DRIVER, OPERATOR, ADMIN. Exige token humano; la propiedad se valida además en negocio.

L134: permite GET /docs/assistance/v3/api-docs con DRIVER, OPERATOR, ADMIN. Exige token humano; la propiedad se valida además en negocio.

L136: deniega el patrón previo o cualquier ruta restante si sigue a anyRequest/anyExchange.

L137

```
137 |     return http.build();
```

Construye el objeto final a partir del builder configurado en líneas previas. Termina la cadena de seguridad con todos los matchers y handlers configurados.

L138

```
138 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L140 a L141

```
140 |   private ReactiveAuthorizationManager<AuthorizationContext> allowed(
141 |       boolean technical, String... roles) {
```

Combina autenticación JWT, audiencia, cliente autorizado y roles; no basta con un texto ROLE_ aislado. Tipos y datos: technical = indica si se exigen cliente y audiencia de servicio; boolean es verdadero/falso, sin null.

L142 a L149

```
142 |     return (authentication, context) ->
143 |         authentication
144 |             .map(
145 |                 auth ->
146 |                     new AuthorizationDecision(
147 |                         auth instanceof JwtAuthenticationToken token
148 |                             && TokenSupport.permits(token.getToken(), technical, roles)))
149 |             .defaultIfEmpty(new AuthorizationDecision(false));
```

Solo acepta Authentication basada en JWT y delega la combinación de cliente, audiencia y rol en TokenSupport. Si no existe Authentication, la decisión reactiva por defecto es denegar. Resultado que recibe el llamador: transforma cada elemento al tipo/valor de salida; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L144: transforma cada elemento al tipo/valor de salida.

L148: && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L150

```
150 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L151

```
151 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 16, 27, 139: separadores sin instrucciones.

SecurityErrors (api-gateway)

`Ruta: api-gateway/src/main/java/com/auxiruta/gateway/config/SecurityErrors.java`

Objetivo y funcionalidad

Transforma rechazos de autenticación y permisos de api-gateway en JSON 401/403. También maneja fallos de enrutamiento reactivo. No revela trazas.

L1

```
1 | package com.auxiruta.gateway.config;
```

Ubica el archivo en com.auxiruta.gateway.config. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de api-gateway sin compartir sus entidades con otro módulo.

L3

```
3 | import com.auxiruta.commons.errors.Problems;
```

Importa tipos: com.auxiruta.commons.errors.Problems. Utilidades compartidas de seguridad, errores, reloj o geografía. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import com.fasterxml.jackson.databind.ObjectMapper;
```

Importa tipos: com.fasterxml.jackson.databind.ObjectMapper. Lectura/escritura JSON, nodos y conversión a DTO. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.core.annotation.Order;
```

Importa tipos: org.springframework.core.annotation.Order. Tipo auxiliar usado por las expresiones del archivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.web.server.*;
```

Importa tipos: org.springframework.web.server.*. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.springframework.security.web.server.*;
```

Importa tipos: org.springframework.security.web.server.*. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | import org.springframework.security.web.server.authorization.ServerAccessDeniedHandler;
```

Importa tipos: org.springframework.security.web.server.authorization.ServerAccessDeniedHandler. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import org.springframework.security.core.AuthenticationException;
```

Importa tipos: org.springframework.security.core.AuthenticationException. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | import org.springframework.security.access.AccessDeniedException;
```

Importa tipos: org.springframework.security.access.AccessDeniedException. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12

```
12 | import org.springframework.http.*;
```

Importa tipos: org.springframework.http.*. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L13

```
13 | import reactor.core.publisher.Mono;
```

Importa tipos: reactor.core.publisher.Mono. Mono y operadores de finalización del flujo reactivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L15

```
15 | @Component
```

Registra componente inyectable.

L16

```
16 | @Order(-2)
```

Ordena filtro/manejador.

L17 a L18

```
17 | public class SecurityErrors
18 |     implements ServerAuthenticationEntryPoint, ServerAccessDeniedHandler, WebExceptionHandler {
```

Declara clase SecurityErrors. Convierte fallos de autenticación/autorización a respuestas JSON 401/403. Gateway además transforma errores de enrutamiento. No añade un mecanismo de inicio de sesión propio.

L19

```
19 |   private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L21

```
21 |   public SecurityErrors(ObjectMapper mapper) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada. Tipos y datos: mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L22

```
22 |     this.mapper = mapper;
```

En this guarda conversor JSON de Jackson. Toma el dato de mapper después de las validaciones de este método. conversor JSON de Jackson.

L23

```
23 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L25

```
25 |   public Mono<Void> commence(ServerWebExchange x, AuthenticationException e) {
```

Implementa AuthenticationEntryPoint: se invoca cuando falta autenticación válida y produce la respuesta 401.

L26

```
26 |     return write(x, 401);
```

Una falla de autenticación produce HTTP 401.

L27

```
27 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L29

```
29 |   public Mono<Void> handle(ServerWebExchange x, AccessDeniedException e) {
```

Implementa el tratamiento de acceso denegado: una identidad sin permiso recibe 403; la variante reactiva también adapta errores de la cadena.

L30

```
30 |     return write(x, 403);
```

Una falla de autorización produce HTTP 403.

L31

```
31 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L33

```
33 |   public Mono<Void> handle(ServerWebExchange x, Throwable e) {
```

Implementa el tratamiento de acceso denegado: una identidad sin permiso recibe 403; la variante reactiva también adapta errores de la cadena.

L34

```
34 |     if (x.getResponse().isCommitted()) return Mono.error(e);
```

Propaga la excepción por el flujo reactivo para que la maneje el siguiente componente; no devuelve una respuesta exitosa ni bloquea el hilo esperando. .

L35 a L36

```
35 |     return write(
36 |         x, e instanceof ResponseStatusException status ? status.getStatusCode().value() : 503);
```

Conserva el código de ResponseStatusException; un error de enrutamiento no clasificado se presenta como 503. Resultado que recibe el llamador: el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L37

```
37 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L39

```
39 |   private Mono<Void> write(ServerWebExchange x, int status) {
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Tipos y datos: status = estado del objeto; su enum o CHECK define valores permitidos; int es entero de 32 bits para límites y contadores.

L40

```
40 |     x.getResponse().setStatusCode(HttpStatusCode.valueOf(status));
```

Convierte el entero a un estado HTTP y lo aplica a la respuesta reactiva del Gateway.

L41

```
41 |     x.getResponse().getHeaders().setContentType(MediaType.APPLICATION_PROBLEM_JSON);
```

Declara application/problem+json para errores uniformes.

L42

```
42 |     if (status == 401) x.getResponse().getHeaders().set("WWW-Authenticate", "Bearer");
```

Añade desafío Bearer en 401. Esta condición decide si se entra en la rama siguiente. .

L43

```
43 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L44 a L49

```
44 |       return x.getResponse()
45 |           .writeWith(
46 |               Mono.just(
47 |                   x.getResponse()
48 |                       .bufferFactory()
49 |                       .wrap(mapper.writeValueAsBytes(Problems.of(status)))));
```

Serializa ProblemDetail a bytes y los escribe en el buffer reactivo.

L50

```
50 |     } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L51

```
51 |       return Mono.error(e);
```

Propaga la excepción por el flujo reactivo para que la maneje el siguiente componente; no devuelve una respuesta exitosa ni bloquea el hilo esperando.

L52

```
52 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L53

```
53 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L54

```
54 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 14, 20, 24, 28, 32, 38: separadores sin instrucciones.

application-docker (api-gateway)

`Ruta: api-gateway/src/main/resources/application-docker.yml`

Objetivo y funcionalidad

Configura api-gateway al activar docker: emisor JWT y dirección interna de las claves de Keycloak; no necesita base de negocio.

L1

```
1 | spring:
```

Propiedades estándar de Spring Boot; sus hijos configuran componentes que se crean al arrancar.

L2

```
2 |   security:
```

Configuración de seguridad consumida por Spring Security.

L3

```
3 |     oauth2:
```

Configuración del protocolo de obtención/validación de tokens.

L4

```
4 |       resourceserver:
```

Aplicación receptora de access tokens; no emite tokens ni ofrece login.

L5

```
5 |         jwt:
```

Emisor y claves de validación de tokens firmados.

L6

```
6 |           issuer-uri: ${KEYCLOAK_ISSUER_URI}
```

Emisor esperado del JWT. El valor configurado es ${KEYCLOAK_ISSUER_URI}. El valor real se obtiene de KEYCLOAK_ISSUER_URI; aquí se conserva solo la referencia.

L7

```
7 |           jwk-set-uri: ${KEYCLOAK_JWK_SET_URI}
```

URL de claves públicas para verificar firma. El valor configurado es ${KEYCLOAK_JWK_SET_URI}. El valor real se obtiene de KEYCLOAK_JWK_SET_URI; aquí se conserva solo la referencia.

application (api-gateway)

`Ruta: api-gateway/src/main/resources/application.yml`

Objetivo y funcionalidad

Configura nombre y puerto base de api-gateway. El puerto público 8000 recibe tráfico que GatewayRoutes reenvía a los servicios.

L1

```
1 | spring:
```

Propiedades estándar de Spring Boot; sus hijos configuran componentes que se crean al arrancar.

L2

```
2 |   application:
```

Identidad de la aplicación Spring, no de un usuario.

L3

```
3 |     name: api-gateway
```

Identifica a esta aplicación como api-gateway para el contexto Spring y sus registros; no es el nombre de un contenedor creado aquí.

L4

```
4 | server:
```

Opciones del servidor HTTP embebido, incluido su puerto.

L5

```
5 |   port: ${SERVER_PORT:8000}
```

Puerto de escucha; Compose controla su exposición al host. El valor configurado es ${SERVER_PORT:8000}. Spring busca SERVER_PORT y, si no está definida, usa '8000'.

AssistanceApplication (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/AssistanceApplication.java`

Objetivo y funcionalidad

Punto de entrada del microservicio propietario de vehículos, solicitudes, evaluaciones y ejecución.

L1

```
1 | package com.auxiruta.assistance;
```

Ubica el archivo en com.auxiruta.assistance. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import org.springframework.boot.SpringApplication;
```

Importa tipos: org.springframework.boot.SpringApplication. Arranque/autoconfiguración y condiciones de activación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.boot.autoconfigure.SpringBootApplication;
```

Importa tipos: org.springframework.boot.autoconfigure.SpringBootApplication. Arranque/autoconfiguración y condiciones de activación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | @SpringBootApplication
```

Activa autoconfiguración y escaneo de componentes.

L7

```
7 | public class AssistanceApplication {
```

Declara clase AssistanceApplication. Punto de entrada del microservicio propietario de vehículos, solicitudes, evaluaciones y ejecución.

L8

```
8 |   public static void main(String[] args) {
```

Entrada JVM: recibe argumentos y el cuerpo arranca Spring Boot.

L9

```
9 |     SpringApplication.run(AssistanceApplication.class, args);
```

Arranca contexto Spring y servidor HTTP del módulo.

L10

```
10 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L11

```
11 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 5: separadores sin instrucciones.

ClockConfiguration (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/config/ClockConfiguration.java`

Objetivo y funcionalidad

Provee un Clock UTC a assistance-service para vencimientos y auditoría. La inyección permite que las pruebas controlen el tiempo sin cambiar la hora del equipo.

L1

```
1 | package com.auxiruta.assistance.config;
```

Ubica el archivo en com.auxiruta.assistance.config. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import com.auxiruta.commons.time.UtcClock;
```

Importa tipos: com.auxiruta.commons.time.UtcClock. Utilidades compartidas de seguridad, errores, reloj o geografía. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.Clock;
```

Importa tipos: java.time.Clock. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.context.annotation.Bean;
```

Importa tipos: org.springframework.context.annotation.Bean. Beans, configuración y eventos del contexto Spring. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.context.annotation.Configuration;
```

Importa tipos: org.springframework.context.annotation.Configuration. Beans, configuración y eventos del contexto Spring. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | @Configuration
```

Declara configuración Spring.

L9

```
9 | public class ClockConfiguration {
```

Declara clase ClockConfiguration. Registra el Clock UTC como bean para servicios y reglas temporales.

L10

```
10 |   @Bean
```

Registra el objeto retornado en Spring.

L11

```
11 |   Clock clock() {
```

Método fábrica del bean Clock que Spring inyecta en reglas temporales.

L12

```
12 |     return UtcClock.system();
```

Obtiene el reloj común UTC y lo entrega al contenedor de beans.

L13

```
13 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L14

```
14 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 7: separadores sin instrucciones.

OpenApiConfiguration (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/config/OpenApiConfiguration.java`

Objetivo y funcionalidad

Documenta la API pública de assistance-service y su esquema Bearer JWT para OpenAPI. Describe cómo autenticar peticiones; no concede permisos ni crea usuarios.

L1

```
1 | package com.auxiruta.assistance.config;
```

Ubica el archivo en com.auxiruta.assistance.config. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import org.springframework.context.annotation.Configuration;
```

Importa tipos: org.springframework.context.annotation.Configuration. Beans, configuración y eventos del contexto Spring. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import io.swagger.v3.oas.annotations.OpenAPIDefinition;
```

Importa tipos: io.swagger.v3.oas.annotations.OpenAPIDefinition. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import io.swagger.v3.oas.annotations.info.Info;
```

Importa tipos: io.swagger.v3.oas.annotations.info.Info. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import io.swagger.v3.oas.annotations.servers.Server;
```

Importa tipos: io.swagger.v3.oas.annotations.servers.Server. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import io.swagger.v3.oas.annotations.security.SecurityScheme;
```

Importa tipos: io.swagger.v3.oas.annotations.security.SecurityScheme. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import io.swagger.v3.oas.annotations.enums.SecuritySchemeType;
```

Importa tipos: io.swagger.v3.oas.annotations.enums.SecuritySchemeType. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | @Configuration
```

Declara configuración Spring.

L11 a L24

```
11 | @OpenAPIDefinition(
12 |     info =
13 |         @Info(
14 |             title = "AuxiRuta Assistance API",
15 |             version = "1.0.0",
16 |             description =
17 |                 "Vehículos y solicitudes. Bearer JWT. Coordenadas WGS84 hasta seis decimales.
     Fechas UTC. page>=0, size 1..100. Creación, cancelación, respuestas a propuestas y etapas exigen
     Idempotency-Key de hasta 100 caracteres. Evaluación real con OpenRouteService; recuperación
     automática de operaciones remotas. Una respuesta exitosa puede conservar resourceReleasePending
     hasta confirmar liberación. Los errores usan application/problem+json."),
18 |     servers = @Server(url = "/"))
19 | @SecurityScheme(
20 |     name = "bearerAuth",
21 |     type = SecuritySchemeType.HTTP,
22 |     scheme = "bearer",
23 |     bearerFormat = "JWT")
24 | public class OpenApiConfiguration {}
```

Metadatos documentales de API. Describe Bearer JWT. Declara clase OpenApiConfiguration. Declara metadatos OpenAPI y el esquema Bearer JWT. Una anotación de documentación no concede permisos: esa decisión está en SecurityConfiguration.

Líneas vacías 2, 9: separadores sin instrucciones.

RequestLoggingFilter (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/config/RequestLoggingFilter.java`

Objetivo y funcionalidad

Registra método, ruta, estado y duración de las peticiones de assistance-service mediante la cadena de filtros Servlet. No guarda tokens/cuerpos y no sustituye el historial de negocio.

L1

```
1 | package com.auxiruta.assistance.config;
```

Ubica el archivo en com.auxiruta.assistance.config. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import jakarta.servlet.*;
```

Importa tipos: jakarta.servlet.*. Tipos de petición, respuesta y cadena de filtros Servlet. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import jakarta.servlet.http.*;
```

Importa tipos: jakarta.servlet.http.*. Tipos de petición, respuesta y cadena de filtros Servlet. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.io.IOException;
```

Importa tipos: java.io.IOException. Tipo auxiliar usado por las expresiones del archivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.web.filter.OncePerRequestFilter;
```

Importa tipos: org.springframework.web.filter.OncePerRequestFilter. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.slf4j.LoggerFactory;
```

Importa tipos: org.slf4j.LoggerFactory. Logging de eventos y fallos sin acoplarse a una implementación de consola. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | @Component
```

Registra componente inyectable.

L11

```
11 | public class RequestLoggingFilter extends OncePerRequestFilter {
```

Declara clase RequestLoggingFilter. Mide duración y registra método y estado HTTP, sin guardar tokens ni cuerpos. Gateway usa una cadena reactiva; los servicios usan el filtro Servlet.

L12 a L14

```
12 |   protected void doFilterInternal(
13 |       HttpServletRequest req, HttpServletResponse res, FilterChain chain)
14 |       throws ServletException, IOException {
```

Filtro Servlet por petición: envuelve el resto de la cadena y mide su resultado incluso si falla.

L15

```
15 |     long start = System.nanoTime();
```

Captura tiempo monotónico para medir duración de la petición. No se usa como fecha de negocio ni para los vencimientos persistidos; el contador sirve para restar inicio y fin.

L16

```
16 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L17

```
17 |       chain.doFilter(req, res);
```

Entrega petición y respuesta al siguiente filtro o controlador. El bloque finally posterior registra la duración aun si esa cadena lanza una excepción.

L18

```
18 |     } finally {
```

Inicia limpieza o registro que se ejecuta tanto si el bloque try terminó bien como si lanzó una excepción; no transforma automáticamente un fallo en éxito.

L19 a L24

```
19 |       LoggerFactory.getLogger(getClass())
20 |           .debug(
21 |               "HTTP method={} status={} durationMs={}",
22 |               req.getMethod(),
23 |               res.getStatus(),
24 |               (System.nanoTime() - start) / 1_000_000);
```

Captura tiempo monotónico para medir duración de la petición. No se usa como fecha de negocio ni para los vencimientos persistidos; el contador sirve para restar inicio y fin.

L25

```
25 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L26

```
26 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L27

```
27 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 9: separadores sin instrucciones.

SecurityConfiguration (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/config/SecurityConfiguration.java`

Objetivo y funcionalidad

Define permisos por ruta y método en assistance-service y la validación Bearer JWT. Protege el servicio incluso sin Gateway; diferencia roles y deja los controles de propietario al servicio de negocio.

L1

```
1 | package com.auxiruta.assistance.config;
```

Ubica el archivo en com.auxiruta.assistance.config. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.Set;
```

Importa tipos: java.util.Set. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import com.auxiruta.commons.security.TokenSupport;
```

Importa tipos: com.auxiruta.commons.security.TokenSupport. Utilidades compartidas de seguridad, errores, reloj o geografía. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.beans.factory.annotation.Value;
```

Importa tipos: org.springframework.beans.factory.annotation.Value. Inyección de dependencias y propiedades de entorno. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.context.annotation.*;
```

Importa tipos: org.springframework.context.annotation.*. Beans, configuración y eventos del contexto Spring. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.http.HttpMethod;
```

Importa tipos: org.springframework.http.HttpMethod. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.springframework.security.authorization.AuthorizationDecision;
```

Importa tipos: org.springframework.security.authorization.AuthorizationDecision. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | import org.springframework.security.oauth2.jwt.*;
```

Importa tipos: org.springframework.security.oauth2.jwt.*. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import org.springframework.security.oauth2.server.resource.authentication.*;
```

Importa tipos: org.springframework.security.oauth2.server.resource.authentication.*. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | import org.springframework.security.config.annotation.web.builders.HttpSecurity;
```

Importa tipos: org.springframework.security.config.annotation.web.builders.HttpSecurity. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12

```
12 | import org.springframework.security.web.SecurityFilterChain;
```

Importa tipos: org.springframework.security.web.SecurityFilterChain. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L13

```
13 | import org.springframework.security.config.http.SessionCreationPolicy;
```

Importa tipos: org.springframework.security.config.http.SessionCreationPolicy. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L14

```
14 | import org.springframework.security.authorization.AuthorizationManager;
```

Importa tipos: org.springframework.security.authorization.AuthorizationManager. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L15

```
15 | import org.springframework.security.web.access.intercept.RequestAuthorizationContext;
```

Importa tipos: org.springframework.security.web.access.intercept.RequestAuthorizationContext. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L17

```
17 | @Configuration
```

Declara configuración Spring.

L18

```
18 | public class SecurityConfiguration {
```

Declara clase SecurityConfiguration. Configura validación JWT y permisos por ruta y método. El primer matcher aplicable decide; las rutas específicas preceden a las generales y lo no enumerado se deniega.

L19

```
19 |   @Bean
```

Registra el objeto retornado en Spring.

L20 a L22

```
20 |   JwtDecoder jwtDecoder(
21 |       @Value("${spring.security.oauth2.resourceserver.jwt.issuer-uri}") String issuer,
22 |       @Value("${spring.security.oauth2.resourceserver.jwt.jwk-set-uri}") String keys) {
```

Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición. Define bean que decodifica y verifica el JWT; las URLs de emisor/claves provienen de configuración. Tipos y datos: issuer = emisor Keycloak esperado del JWT; String es texto; no es por sí mismo una referencia de entidad; keys = keys; String es texto; no es por sí mismo una referencia de entidad.

L23

```
23 |     var decoder = NimbusJwtDecoder.withJwkSetUri(keys).build();
```

Construye el objeto final a partir del builder configurado en líneas previas. Se calcula decoder (decoder) para reutilizar ese resultado en los pasos siguientes.

L24

```
24 |     decoder.setJwtValidator(TokenSupport.validator(issuer, Set.of("auxiruta-api")));
```

Añade validación de emisor, tiempo, sub y audiencia. Crea un conjunto no modificable y sin duplicados.

L25

```
25 |     return decoder;
```

Entrega el decodificador ya configurado al contenedor Spring. Resultado que recibe el llamador: decoder.

L26

```
26 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L28

```
28 |   @Bean
```

Registra el objeto retornado en Spring.

L29

```
29 |   SecurityFilterChain security(HttpSecurity http, SecurityErrors errors) throws Exception {
```

Construye la cadena Servlet de seguridad que intercepta peticiones antes del controlador.

L30

```
30 |     var converter = new JwtAuthenticationConverter();
```

Construye JwtAuthenticationConverter en memoria y conserva la referencia en converter. Los argumentos del constructor determinan los datos iniciales de este objeto. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L31 a L32

```
31 |     converter.setJwtGrantedAuthoritiesConverter(
32 |         jwt -> new java.util.ArrayList<>(TokenSupport.roles(jwt)));
```

Traduce roles de Keycloak a ROLE_ de Spring.

L33 a L94

```
33 |     http.csrf(c -> c.disable())
34 |         .httpBasic(c -> c.disable())
35 |         .formLogin(c -> c.disable())
36 |         .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
37 |         .exceptionHandling(e -> e.authenticationEntryPoint(errors).accessDeniedHandler(errors))
38 |         .oauth2ResourceServer(
39 |             o ->
40 |                 o.authenticationEntryPoint(errors)
41 |                     .accessDeniedHandler(errors)
42 |                     .jwt(j -> j.jwtAuthenticationConverter(converter)))
43 |         .authorizeHttpRequests(
44 |             auth ->
45 |                 auth.requestMatchers(HttpMethod.POST, "/api/v1/vehicles")
46 |                     .access(allowed(false, "DRIVER"))
47 |                     .requestMatchers(HttpMethod.GET, "/api/v1/vehicles")
48 |                     .access(allowed(false, "DRIVER"))
49 |                     .requestMatchers(HttpMethod.GET, "/api/v1/vehicles/{id}")
50 |                     .access(allowed(false, "DRIVER"))
51 |                     .requestMatchers(HttpMethod.PATCH, "/api/v1/vehicles/{id}")
52 |                     .access(allowed(false, "DRIVER"))
53 |                     .requestMatchers(HttpMethod.PATCH,
   "/api/v1/vehicles/{id}/registration-status")
54 |                     .access(allowed(false, "DRIVER"))
55 |                     .requestMatchers(HttpMethod.POST, "/api/v1/assistance-requests")
56 |                     .access(allowed(false, "DRIVER"))
57 |                     .requestMatchers(
58 |                         HttpMethod.POST, "/api/v1/assistance-requests/{id}/cancellations")
59 |                     .access(allowed(false, "DRIVER"))
60 |                     .requestMatchers(HttpMethod.GET, "/api/v1/assistance-requests")
61 |                     .access(allowed(false, "DRIVER", "OPERATOR", "ADMIN"))
62 |                     .requestMatchers(HttpMethod.GET, "/api/v1/assistance-requests/{id}")
63 |                     .access(allowed(false, "DRIVER", "OPERATOR", "ADMIN"))
64 |                     .requestMatchers(HttpMethod.GET, "/api/v1/assistance-requests/{id}/history")
65 |                     .access(allowed(false, "DRIVER", "OPERATOR", "ADMIN"))
66 |                     .requestMatchers(HttpMethod.GET, "/api/v1/operator/assignments")
67 |                     .access(allowed(false, "OPERATOR"))
68 |                     .requestMatchers(HttpMethod.GET, "/api/v1/assignments/{id}")
69 |                     .access(allowed(false, "OPERATOR", "ADMIN"))
70 |                     .requestMatchers(HttpMethod.POST, "/api/v1/assignments/{id}/acceptance")
71 |                     .access(allowed(false, "OPERATOR"))
72 |                     .requestMatchers(HttpMethod.POST, "/api/v1/assignments/{id}/rejection")
73 |                     .access(allowed(false, "OPERATOR"))
74 |                     .requestMatchers(HttpMethod.POST,
   "/api/v1/assistance-requests/{id}/dispatch")
75 |                     .access(allowed(false, "OPERATOR"))
76 |                     .requestMatchers(HttpMethod.POST, "/api/v1/assistance-requests/{id}/arrival")
77 |                     .access(allowed(false, "OPERATOR"))
78 |                     .requestMatchers(HttpMethod.POST, "/api/v1/assistance-requests/{id}/loading")
79 |                     .access(allowed(false, "OPERATOR"))
80 |                     .requestMatchers(
81 |                         HttpMethod.POST, "/api/v1/assistance-requests/{id}/transport-start")
82 |                     .access(allowed(false, "OPERATOR"))
83 |                     .requestMatchers(HttpMethod.POST,
   "/api/v1/assistance-requests/{id}/completion")
84 |                     .access(allowed(false, "OPERATOR"))
85 |                     .requestMatchers(HttpMethod.GET, "/api/v1/configuration/assistance")
86 |                     .access(allowed(false, "ADMIN"))
87 |                     .requestMatchers(HttpMethod.GET, "/v3/api-docs/**")
88 |                     .access(allowed(false, "DRIVER", "OPERATOR", "ADMIN"))
89 |                     .requestMatchers(HttpMethod.GET, "/swagger-ui/**")
90 |                     .access(allowed(false, "DRIVER", "OPERATOR", "ADMIN"))
91 |                     .requestMatchers(HttpMethod.GET, "/swagger-ui.html")
92 |                     .access(allowed(false, "DRIVER", "OPERATOR", "ADMIN"))
93 |                     .anyRequest()
94 |                     .denyAll());
```

Construye la cadena de seguridad. Desactiva login por formulario y HTTP Basic porque los clientes presentan Bearer JWT de Keycloak. El contexto no se guarda como sesión de usuario. El conversor traduce roles del JWT a authorities y los manejadores normalizan 401/403. Después aplica reglas en orden: el primer patrón coincidente decide; las rutas específicas deben preceder a patrones amplios. allowed(false, ...) exige identidad humana; allowed(true, ...) exige cliente técnico. denyAll deniega una coincidencia explícita o, al final, cualquier petición que no haya coincidido. No significa que todas las rutas previas estén denegadas.

L46: permite POST /api/v1/vehicles con DRIVER. Exige token humano; la propiedad se valida además en negocio.

L48: permite GET /api/v1/vehicles con DRIVER. Exige token humano; la propiedad se valida además en negocio.

L50: permite GET /api/v1/vehicles/{id} con DRIVER. Exige token humano; la propiedad se valida además en negocio.

L52: permite PATCH /api/v1/vehicles/{id} con DRIVER. Exige token humano; la propiedad se valida además en negocio.

L54: permite PATCH /api/v1/vehicles/{id}/registration-status con DRIVER. Exige token humano; la propiedad se valida además en negocio.

L56: permite POST /api/v1/assistance-requests con DRIVER. Exige token humano; la propiedad se valida además en negocio.

L59: permite POST /api/v1/assistance-requests/{id}/cancellations con DRIVER. Exige token humano; la propiedad se valida además en negocio.

L61: permite GET /api/v1/assistance-requests con DRIVER, OPERATOR, ADMIN. Exige token humano; la propiedad se valida además en negocio.

L63: permite GET /api/v1/assistance-requests/{id} con DRIVER, OPERATOR, ADMIN. Exige token humano; la propiedad se valida además en negocio.

L65: permite GET /api/v1/assistance-requests/{id}/history con DRIVER, OPERATOR, ADMIN. Exige token humano; la propiedad se valida además en negocio.

L67: permite GET /api/v1/operator/assignments con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L69: permite GET /api/v1/assignments/{id} con OPERATOR, ADMIN. Exige token humano; la propiedad se valida además en negocio.

L71: permite POST /api/v1/assignments/{id}/acceptance con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L73: permite POST /api/v1/assignments/{id}/rejection con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L75: permite POST /api/v1/assistance-requests/{id}/dispatch con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L77: permite POST /api/v1/assistance-requests/{id}/arrival con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L79: permite POST /api/v1/assistance-requests/{id}/loading con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L82: permite POST /api/v1/assistance-requests/{id}/transport-start con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L84: permite POST /api/v1/assistance-requests/{id}/completion con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L86: permite GET /api/v1/configuration/assistance con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L88: permite GET /v3/api-docs/** con DRIVER, OPERATOR, ADMIN. Exige token humano; la propiedad se valida además en negocio.

L90: permite GET /swagger-ui/** con DRIVER, OPERATOR, ADMIN. Exige token humano; la propiedad se valida además en negocio.

L92: permite GET /swagger-ui.html con DRIVER, OPERATOR, ADMIN. Exige token humano; la propiedad se valida además en negocio.

L94: deniega el patrón previo o cualquier ruta restante si sigue a anyRequest/anyExchange.

L95

```
95 |     return http.build();
```

Construye el objeto final a partir del builder configurado en líneas previas. Termina la cadena de seguridad con todos los matchers y handlers configurados.

L96

```
96 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L98 a L99

```
98 |   private AuthorizationManager<RequestAuthorizationContext> allowed(
99 |       boolean technical, String... roles) {
```

Combina autenticación JWT, audiencia, cliente autorizado y roles; no basta con un texto ROLE_ aislado. Tipos y datos: technical = indica si se exigen cliente y audiencia de servicio; boolean es verdadero/falso, sin null.

L100 a L103

```
100 |     return (authentication, context) ->
101 |         new AuthorizationDecision(
102 |             authentication.get() instanceof JwtAuthenticationToken token
103 |                 && TokenSupport.permits(token.getToken(), technical, roles));
```

Solo acepta Authentication basada en JWT y delega la combinación de cliente, audiencia y rol en TokenSupport. Devuelve una función de autorización evaluada por petición, no durante el arranque. Resultado que recibe el llamador: && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L103: && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L104

```
104 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L105

```
105 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 16, 27, 97: separadores sin instrucciones.

SecurityErrors (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/config/SecurityErrors.java`

Objetivo y funcionalidad

Transforma rechazos de autenticación y permisos de assistance-service en JSON 401/403. Actúa en la respuesta Servlet antes de llegar al controlador. No revela trazas.

L1

```
1 | package com.auxiruta.assistance.config;
```

Ubica el archivo en com.auxiruta.assistance.config. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import com.auxiruta.commons.errors.Problems;
```

Importa tipos: com.auxiruta.commons.errors.Problems. Utilidades compartidas de seguridad, errores, reloj o geografía. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import com.fasterxml.jackson.databind.ObjectMapper;
```

Importa tipos: com.fasterxml.jackson.databind.ObjectMapper. Lectura/escritura JSON, nodos y conversión a DTO. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import jakarta.servlet.http.*;
```

Importa tipos: jakarta.servlet.http.*. Tipos de petición, respuesta y cadena de filtros Servlet. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import java.io.IOException;
```

Importa tipos: java.io.IOException. Tipo auxiliar usado por las expresiones del archivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
 8 | import org.springframework.security.core.AuthenticationException;
```

Importa tipos: org.springframework.security.core.AuthenticationException. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
 9 | import org.springframework.security.access.AccessDeniedException;
```

Importa tipos: org.springframework.security.access.AccessDeniedException. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import org.springframework.security.web.AuthenticationEntryPoint;
```

Importa tipos: org.springframework.security.web.AuthenticationEntryPoint. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | import org.springframework.security.web.access.AccessDeniedHandler;
```

Importa tipos: org.springframework.security.web.access.AccessDeniedHandler. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L13

```
13 | @Component
```

Registra componente inyectable.

L14

```
14 | public class SecurityErrors implements AuthenticationEntryPoint, AccessDeniedHandler {
```

Declara clase SecurityErrors. Convierte fallos de autenticación/autorización a respuestas JSON 401/403. Gateway además transforma errores de enrutamiento. No añade un mecanismo de inicio de sesión propio.

L15

```
15 |   private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L17

```
17 |   public SecurityErrors(ObjectMapper mapper) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada. Tipos y datos: mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L18

```
18 |     this.mapper = mapper;
```

En this guarda conversor JSON de Jackson. Toma el dato de mapper después de las validaciones de este método. conversor JSON de Jackson.

L19

```
19 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L21 a L22

```
21 |   public void commence(HttpServletRequest req, HttpServletResponse res, AuthenticationException
     e)
22 |       throws IOException {
```

Implementa AuthenticationEntryPoint: se invoca cuando falta autenticación válida y produce la respuesta 401.

L23

```
23 |     write(res, 401);
```

Una falla de autenticación produce HTTP 401 en Servlet.

L24

```
24 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L26 a L27

```
26 |   public void handle(HttpServletRequest req, HttpServletResponse res, AccessDeniedException e)
27 |       throws IOException {
```

Implementa el tratamiento de acceso denegado: una identidad sin permiso recibe 403; la variante reactiva también adapta errores de la cadena.

L28

```
28 |     write(res, 403);
```

Una falla de autorización produce HTTP 403 en Servlet.

L29

```
29 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L31

```
31 |   private void write(HttpServletResponse response, int status) throws IOException {
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Tipos y datos: status = estado del objeto; su enum o CHECK define valores permitidos; int es entero de 32 bits para límites y contadores.

L32

```
32 |     response.setStatus(status);
```

Fija el código HTTP numérico de la respuesta Servlet antes de escribir el cuerpo de error.

L33

```
33 |     response.setContentType("application/problem+json");
```

Declara application/problem+json para errores uniformes.

L34

```
34 |     if (status == 401) response.setHeader("WWW-Authenticate", "Bearer");
```

Añade desafío Bearer en 401. Esta condición decide si se entra en la rama siguiente. .

L35

```
35 |     mapper.writeValue(response.getOutputStream(), Problems.of(status));
```

Serializa ProblemDetail directamente en el cuerpo Servlet.

L36

```
36 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L37

```
37 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 12, 16, 20, 25, 30: separadores sin instrucciones.

ApiExceptionHandler (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/exceptions/ApiExceptionHandler.java

Objetivo y funcionalidad

Convierte validación, conflictos SQL y errores de negocio de assistance-service en respuestas HTTP públicas sin SQL ni trazas. Hace distinguibles las entradas inválidas de los conflictos de estado.

L1

```
1 | package com.auxiruta.assistance.exceptions;
```

Ubica el archivo en com.auxiruta.assistance.exceptions. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import com.auxiruta.commons.errors.Problems;
```

Importa tipos: com.auxiruta.commons.errors.Problems. Utilidades compartidas de seguridad, errores, reloj o geografía. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.http.*;
```

Importa tipos: org.springframework.http.*. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.web.bind.annotation.*;
```

Importa tipos: org.springframework.web.bind.annotation.*. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;
```

Importa tipos: org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.web.context.request.WebRequest;
```

Importa tipos: org.springframework.web.context.request.WebRequest. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | import org.springframework.security.access.AccessDeniedException;
```

Importa tipos: org.springframework.security.access.AccessDeniedException. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import org.slf4j.LoggerFactory;
```

Importa tipos: org.slf4j.LoggerFactory. Logging de eventos y fallos sin acoplarse a una implementación de consola. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12

```
12 | @RestControllerAdvice
```

Centraliza transformación de excepciones.

L13

```
13 | public class ApiExceptionHandler extends ResponseEntityExceptionHandler {
```

Declara clase ApiExceptionHandler. Normaliza errores de controladores, validación y persistencia. Las restricciones de integridad/concurrencia se exponen como 409, no como detalle SQL.

L14

```
14 |   @Override
```

Redefine contrato heredado.

L15 a L16

```
15 |   protected ResponseEntity<Object> handleExceptionInternal(
16 |       Exception ex, Object body, HttpHeaders headers, HttpStatusCode status, WebRequest request)
     {
```

Interviene en errores MVC de validación/parsing y preserva encabezados/estado.

L17

```
17 |     return new ResponseEntity<>(Problems.of(status.value()), headers, status);
```

Reemplaza el cuerpo de error por el formato común conservando estado y headers.

L18

```
18 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L20

```
20 |   @ExceptionHandler(ResponseStatusException.class)
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos.

L21

```
21 |   ResponseEntity<ProblemDetail> status(ResponseStatusException e) {
```

Manejador de la excepción indicada: toma el fallo que llega del servicio/validación y devuelve un ProblemDetail con código HTTP público. La anotación ExceptionHandler anterior define qué fallos llegan aquí.

L22

```
22 |     return ResponseEntity.status(e.getStatusCode()).body(statusProblem(e));
```

Manejador de la excepción indicada: toma el fallo que llega del servicio/validación y devuelve un ProblemDetail con código HTTP público. La anotación ExceptionHandler anterior define qué fallos llegan aquí.

L23

```
23 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L25

```
25 |   private ProblemDetail statusProblem(ResponseStatusException e) {
```

Prepara ProblemDetail y puede conservar el código específico de confirmación pendiente.

L26

```
26 |     var problem = Problems.of(e.getStatusCode().value());
```

Obtiene el error común correspondiente al estado de la excepción. Se calcula problem (problem) para reutilizar ese resultado en los pasos siguientes.

L27 a L28

```
27 |     if ("ACCEPTANCE_CONFIRMATION_PENDING".equals(e.getReason()))
28 |       problem.setProperty("code", "ACCEPTANCE_CONFIRMATION_PENDING");
```

Protege la carrera aceptación/cancelación: una confirmación remota incierta puede haber ocurrido. Impide cancelar hasta reconciliar, con código distinguible en el error. Esta condición decide si se entra en la rama siguiente. compara contenido/identidad lógica, no referencias Java.

L29

```
29 |     return problem;
```

Entrega problem al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: problem.

L30

```
30 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L32

```
32 |   @ExceptionHandler(AccessDeniedException.class)
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos.

L33

```
33 |   ResponseEntity<ProblemDetail> forbidden(AccessDeniedException e) {
```

Manejador de la excepción indicada: toma el fallo que llega del servicio/validación y devuelve un ProblemDetail con código HTTP público. La anotación ExceptionHandler anterior define qué fallos llegan aquí.

L34

```
34 |     return ResponseEntity.status(403).body(Problems.of(403));
```

Manejador de la excepción indicada: toma el fallo que llega del servicio/validación y devuelve un ProblemDetail con código HTTP público. La anotación ExceptionHandler anterior define qué fallos llegan aquí.

L35

```
35 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L37 a L41

```
37 |   @ExceptionHandler({
38 |     org.springframework.dao.DataIntegrityViolationException.class,
39 |     org.springframework.dao.ConcurrencyFailureException.class
40 |   })
41 |   ResponseEntity<ProblemDetail> conflict(Exception e) {
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos. Normaliza errores de controladores, validación y persistencia. Las restricciones de integridad/concurrencia se exponen como 409, no como detalle SQL.

L42

```
42 |     return ResponseEntity.status(409).body(Problems.of(409));
```

Manejador de la excepción indicada: toma el fallo que llega del servicio/validación y devuelve un ProblemDetail con código HTTP público. La anotación ExceptionHandler anterior define qué fallos llegan aquí.

L43

```
43 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L45

```
45 |   @ExceptionHandler(Exception.class)
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos.

L46

```
46 |   ResponseEntity<ProblemDetail> unexpected(Exception e) {
```

Captura errores no clasificados para devolver 500 genérico y registrar solo tipo de excepción.

L47 a L48

```
47 |     LoggerFactory.getLogger(getClass())
48 |         .error("Error inesperado tipo={}", e.getClass().getSimpleName());
```

Registra los campos indicados, sin token ni secreto. Se calcula tipo (tipo) para reutilizar ese resultado en los pasos siguientes.

L49

```
49 |     return ResponseEntity.status(500).body(Problems.of(500));
```

Manejador de la excepción indicada: toma el fallo que llega del servicio/validación y devuelve un ProblemDetail con código HTTP público. La anotación ExceptionHandler anterior define qué fallos llegan aquí.

L50

```
50 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L51

```
51 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 11, 19, 24, 31, 36, 44: separadores sin instrucciones.

application-docker (assistance-service)

`Ruta: assistance-service/src/main/resources/application-docker.yml`

Objetivo y funcionalidad

Configura assistance-service al activar docker: credenciales SQL, scripts de esquema y validación Hibernate sin crear tablas. Añade cliente técnico hacia Fleet y URL/clave ORS.

L1

```
1 | spring:
```

Propiedades estándar de Spring Boot; sus hijos configuran componentes que se crean al arrancar.

L2

```
2 |   security:
```

Configuración de seguridad consumida por Spring Security.

L3

```
3 |     oauth2:
```

Configuración del protocolo de obtención/validación de tokens.

L4

```
4 |       client:
```

Identidad OAuth utilizada cuando el servicio llama a otro servicio.

L5

```
5 |         registration:
```

Registros locales de clientes OAuth y sus credenciales/grants.

L6

```
6 |           fleet-client:
```

Registro local de OAuth utilizado para la comunicación técnica con Fleet.

L7

```
7 |             provider: keycloak
```

Referencia el proveedor OAuth keycloak configurado en el bloque provider; permite ubicar su endpoint de emisión.

L8

```
 8 |             client-id: ${KEYCLOAK_CLIENT_ID}
```

Identificador del cliente OAuth. El valor configurado es ${KEYCLOAK_CLIENT_ID}. El valor real se obtiene de KEYCLOAK_CLIENT_ID; aquí se conserva solo la referencia.

L9

```
 9 |             client-secret: ${KEYCLOAK_CLIENT_SECRET}
```

Secreto técnico resuelto desde entorno, no token de usuario. El valor configurado es ${KEYCLOAK_CLIENT_SECRET}. El valor real se obtiene de KEYCLOAK_CLIENT_SECRET; aquí se conserva solo la referencia.

L10

```
10 |             authorization-grant-type: client_credentials
```

client_credentials autentica al servicio, no al conductor. El valor configurado es client_credentials.

L11

```
11 |         provider:
```

Relación con el emisor OAuth y configuración de su endpoint de token.

L12

```
12 |           keycloak:
```

Identificador local del proveedor de identidad; token-uri define la dirección.

L13

```
13 |             token-uri: ${KEYCLOAK_TOKEN_URI}
```

Endpoint de Keycloak para emitir tokens. El valor configurado es ${KEYCLOAK_TOKEN_URI}. El valor real se obtiene de KEYCLOAK_TOKEN_URI; aquí se conserva solo la referencia.

L14

```
14 |       resourceserver:
```

Aplicación receptora de access tokens; no emite tokens ni ofrece login.

L15

```
15 |         jwt:
```

Emisor y claves de validación de tokens firmados.

L16

```
16 |           issuer-uri: ${KEYCLOAK_ISSUER_URI}
```

Emisor esperado del JWT. El valor configurado es ${KEYCLOAK_ISSUER_URI}. El valor real se obtiene de KEYCLOAK_ISSUER_URI; aquí se conserva solo la referencia.

L17

```
17 |           jwk-set-uri: ${KEYCLOAK_JWK_SET_URI}
```

URL de claves públicas para verificar firma. El valor configurado es ${KEYCLOAK_JWK_SET_URI}. El valor real se obtiene de KEYCLOAK_JWK_SET_URI; aquí se conserva solo la referencia.

L18

```
18 |   datasource:
```

Pool de conexiones SQL: dirección, usuario y contraseña.

L19

```
19 |     url: ${DB_URL}
```

Dirección de conexión o servidor del bloque. El valor configurado es ${DB_URL}. El valor real se obtiene de DB_URL; aquí se conserva solo la referencia.

L20

```
20 |     username: ${DB_USER}
```

Usuario de conexión o identidad. El valor configurado es ${DB_USER}. El valor real se obtiene de DB_USER; aquí se conserva solo la referencia.

L21

```
21 |     password: ${DB_PASSWORD}
```

Referencia de contraseña; no reproduce el secreto local. El valor configurado es ${DB_PASSWORD}. El valor real se obtiene de DB_PASSWORD; aquí se conserva solo la referencia.

L22

```
22 |   sql:
```

Configuración de scripts SQL al iniciar.

L23

```
23 |     init:
```

Momento y orden de aplicación de scripts de esquema.

L24

```
24 |       mode: always
```

always aplica inicialización SQL también a PostgreSQL. El valor configurado es always.

L25

```
25 |       schema-locations: classpath:db/001-assistance.sql,classpath:db/002-evaluation.sql,classpath
     :db/003-assignments.sql,classpath:db/004-execution.sql,classpath:db/005-automation.sql
```

Scripts SQL ordenados para inicializar/extender esquema. El valor configurado es classpath:db/001-assistance.sql,classpath:db/002-evaluation.sql,classpath:db/003-assignments.sql,classpath:db/004-execution.sql,classpath:db/005-automation.sql.

L26

```
26 |   jpa:
```

Opciones de persistencia de entidades mediante JPA.

L27

```
27 |     open-in-view: false
```

false evita mantener EntityManager durante serialización HTTP. El valor configurado es false.

L28

```
28 |     hibernate:
```

Opciones del proveedor JPA que valida mapeos contra tablas.

L29

```
29 |       ddl-auto: validate
```

validate comprueba el esquema; Hibernate no crea tablas. El valor configurado es validate.

L31

```
31 | ors:
```

Dirección y credencial del adaptador de OpenRouteService.

L32

```
32 |   base-url: ${ORS_BASE_URL:https://api.heigit.org/openrouteservice/v2}
```

URL del proveedor ORS. El valor configurado es ${ORS_BASE_URL:https://api.heigit.org/openrouteservice/v2}. Spring busca ORS_BASE_URL y, si no está definida, usa 'https://api.heigit.org/openrouteservice/v2'.

L33

```
33 |   api-key: ${ORS_API_KEY:}
```

Referencia a la clave ORS; nunca versionar su valor local. El valor configurado es ${ORS_API_KEY:}. Spring busca ORS_API_KEY y, si no está definida, usa ''.

Líneas vacías 30: separadores sin instrucciones.

application (assistance-service)

Ruta: assistance-service/src/main/resources/application.yml

Objetivo y funcionalidad

Configura nombre y puerto base de assistance-service. También controla JSON desconocido, publicación OpenAPI, logs SQL y activación de tareas programadas.

L1

```
1 | spring:
```

Propiedades estándar de Spring Boot; sus hijos configuran componentes que se crean al arrancar.

L2

```
2 |   jackson:
```

Jackson transforma JSON recibido a DTO y objetos Java a JSON de respuesta.

L3

```
3 |     deserialization:
```

Opciones de lectura del JSON entrante, antes de aplicar las reglas del servicio.

L4

```
4 |       fail-on-unknown-properties: true
```

true rechaza campos JSON desconocidos. El valor configurado es true.

L5

```
5 |   application:
```

Identidad de la aplicación Spring, no de un usuario.

L6

```
6 |     name: assistance-service
```

Identifica a esta aplicación como assistance-service para el contexto Spring y sus registros; no es el nombre de un contenedor creado aquí.

L7

```
7 | server:
```

Opciones del servidor HTTP embebido, incluido su puerto.

L8

```
8 |   port: ${SERVER_PORT:8082}
```

Puerto de escucha; Compose controla su exposición al host. El valor configurado es ${SERVER_PORT:8082}. Spring busca SERVER_PORT y, si no está definida, usa '8082'.

L10

```
10 | springdoc:
```

Generación de contratos OpenAPI desde controladores y anotaciones.

L11

```
11 |   paths-to-match: /api/v1/**
```

Patrones incluidos en OpenAPI público. El valor configurado es /api/v1/**.

L13

```
13 | logging:
```

Opciones de logs de Spring; no es la extensión x-logging de Compose.

L14

```
14 |   level:
```

Nivel de salida de cada categoría de logger, o su desactivación.

L15

```
15 |     org.hibernate.engine.jdbc.spi.SqlExceptionHelper: "OFF"
```

OFF silencia el logger Hibernate que podría imprimir detalles de restricciones SQL con datos personales. ApiExceptionHandler sigue devolviendo un error público; no se ignora el fallo de la transacción.

L17

```
17 | auxiruta:
```

Propiedades propias del proyecto, leídas por sus componentes.

L18

```
18 |   automation:
```

Interruptor de trabajos periódicos de búsqueda y recuperación.

L19

```
19 |     enabled: ${AUXIRUTA_AUTOMATION_ENABLED:true}
```

Activa el componente o identidad del bloque. El valor configurado es ${AUXIRUTA_AUTOMATION_ENABLED:true}. Spring busca AUXIRUTA_AUTOMATION_ENABLED y, si no está definida, usa 'true'.

Líneas vacías 9, 12, 16: separadores sin instrucciones.

FleetApplication (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/FleetApplication.java`

Objetivo y funcionalidad

Punto de entrada de Spring Boot para el microservicio propietario de operadores, grúas y reservas.

L1

```
1 | package com.auxiruta.fleet;
```

Ubica el archivo en com.auxiruta.fleet. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import org.springframework.boot.SpringApplication;
```

Importa tipos: org.springframework.boot.SpringApplication. Arranque/autoconfiguración y condiciones de activación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.boot.autoconfigure.SpringBootApplication;
```

Importa tipos: org.springframework.boot.autoconfigure.SpringBootApplication. Arranque/autoconfiguración y condiciones de activación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | @SpringBootApplication
```

Activa autoconfiguración y escaneo de componentes.

L7

```
7 | public class FleetApplication {
```

Declara clase FleetApplication. Punto de entrada de Spring Boot para el microservicio propietario de operadores, grúas y reservas.

L8

```
8 |   public static void main(String[] args) {
```

Entrada JVM: recibe argumentos y el cuerpo arranca Spring Boot.

L9

```
9 |     SpringApplication.run(FleetApplication.class, args);
```

Arranca contexto Spring y servidor HTTP del módulo.

L10

```
10 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L11

```
11 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 5: separadores sin instrucciones.

ClockConfiguration (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/config/ClockConfiguration.java`

Objetivo y funcionalidad

Provee un Clock UTC a fleet-service para vencimientos y auditoría. La inyección permite que las pruebas controlen el tiempo sin cambiar la hora del equipo.

L1

```
1 | package com.auxiruta.fleet.config;
```

Ubica el archivo en com.auxiruta.fleet.config. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import com.auxiruta.commons.time.UtcClock;
```

Importa tipos: com.auxiruta.commons.time.UtcClock. Utilidades compartidas de seguridad, errores, reloj o geografía. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.Clock;
```

Importa tipos: java.time.Clock. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.context.annotation.Bean;
```

Importa tipos: org.springframework.context.annotation.Bean. Beans, configuración y eventos del contexto Spring. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.context.annotation.Configuration;
```

Importa tipos: org.springframework.context.annotation.Configuration. Beans, configuración y eventos del contexto Spring. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | @Configuration
```

Declara configuración Spring.

L9

```
9 | public class ClockConfiguration {
```

Declara clase ClockConfiguration. Registra el Clock UTC como bean para servicios y reglas temporales.

L10

```
10 |   @Bean
```

Registra el objeto retornado en Spring.

L11

```
11 |   Clock clock() {
```

Método fábrica del bean Clock que Spring inyecta en reglas temporales.

L12

```
12 |     return UtcClock.system();
```

Obtiene el reloj común UTC y lo entrega al contenedor de beans.

L13

```
13 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L14

```
14 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 7: separadores sin instrucciones.

OpenApiConfiguration (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/config/OpenApiConfiguration.java`

Objetivo y funcionalidad

Documenta la API pública de fleet-service y su esquema Bearer JWT para OpenAPI. Describe cómo autenticar peticiones; no concede permisos ni crea usuarios.

L1

```
1 | package com.auxiruta.fleet.config;
```

Ubica el archivo en com.auxiruta.fleet.config. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import org.springframework.context.annotation.Configuration;
```

Importa tipos: org.springframework.context.annotation.Configuration. Beans, configuración y eventos del contexto Spring. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import io.swagger.v3.oas.annotations.OpenAPIDefinition;
```

Importa tipos: io.swagger.v3.oas.annotations.OpenAPIDefinition. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import io.swagger.v3.oas.annotations.info.Info;
```

Importa tipos: io.swagger.v3.oas.annotations.info.Info. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import io.swagger.v3.oas.annotations.servers.Server;
```

Importa tipos: io.swagger.v3.oas.annotations.servers.Server. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import io.swagger.v3.oas.annotations.security.SecurityScheme;
```

Importa tipos: io.swagger.v3.oas.annotations.security.SecurityScheme. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import io.swagger.v3.oas.annotations.enums.SecuritySchemeType;
```

Importa tipos: io.swagger.v3.oas.annotations.enums.SecuritySchemeType. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | @Configuration
```

Declara configuración Spring.

L11 a L24

```
11 | @OpenAPIDefinition(
12 |     info =
13 |         @Info(
14 |             title = "AuxiRuta Fleet API",
15 |             version = "1.0.0",
16 |             description =
17 |                 "Administración de flota. Fechas UTC ISO-8601. Listados page>=0 y size 1..100.
     PATCH modifica campos presentes; null no borra campos. Borrar teléfono usando cadena vacía.
     Identidades y roles se administran previamente en Keycloak."),
18 |     servers = @Server(url = "/"))
19 | @SecurityScheme(
20 |     name = "bearerAuth",
21 |     type = SecuritySchemeType.HTTP,
22 |     scheme = "bearer",
23 |     bearerFormat = "JWT")
24 | public class OpenApiConfiguration {}
```

Metadatos documentales de API. Describe Bearer JWT. Declara clase OpenApiConfiguration. Declara metadatos OpenAPI y el esquema Bearer JWT. Una anotación de documentación no concede permisos: esa decisión está en SecurityConfiguration.

Líneas vacías 2, 9: separadores sin instrucciones.

RequestLoggingFilter (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/config/RequestLoggingFilter.java`

Objetivo y funcionalidad

Registra método, ruta, estado y duración de las peticiones de fleet-service mediante la cadena de filtros Servlet. No guarda tokens/cuerpos y no sustituye el historial de negocio.

L1

```
1 | package com.auxiruta.fleet.config;
```

Ubica el archivo en com.auxiruta.fleet.config. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import jakarta.servlet.*;
```

Importa tipos: jakarta.servlet.*. Tipos de petición, respuesta y cadena de filtros Servlet. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import jakarta.servlet.http.*;
```

Importa tipos: jakarta.servlet.http.*. Tipos de petición, respuesta y cadena de filtros Servlet. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.io.IOException;
```

Importa tipos: java.io.IOException. Tipo auxiliar usado por las expresiones del archivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.web.filter.OncePerRequestFilter;
```

Importa tipos: org.springframework.web.filter.OncePerRequestFilter. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.slf4j.LoggerFactory;
```

Importa tipos: org.slf4j.LoggerFactory. Logging de eventos y fallos sin acoplarse a una implementación de consola. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | @Component
```

Registra componente inyectable.

L11

```
11 | public class RequestLoggingFilter extends OncePerRequestFilter {
```

Declara clase RequestLoggingFilter. Mide duración y registra método y estado HTTP, sin guardar tokens ni cuerpos. Gateway usa una cadena reactiva; los servicios usan el filtro Servlet.

L12 a L14

```
12 |   protected void doFilterInternal(
13 |       HttpServletRequest req, HttpServletResponse res, FilterChain chain)
14 |       throws ServletException, IOException {
```

Filtro Servlet por petición: envuelve el resto de la cadena y mide su resultado incluso si falla.

L15

```
15 |     long start = System.nanoTime();
```

Captura tiempo monotónico para medir duración de la petición. No se usa como fecha de negocio ni para los vencimientos persistidos; el contador sirve para restar inicio y fin.

L16

```
16 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L17

```
17 |       chain.doFilter(req, res);
```

Entrega petición y respuesta al siguiente filtro o controlador. El bloque finally posterior registra la duración aun si esa cadena lanza una excepción.

L18

```
18 |     } finally {
```

Inicia limpieza o registro que se ejecuta tanto si el bloque try terminó bien como si lanzó una excepción; no transforma automáticamente un fallo en éxito.

L19 a L24

```
19 |       LoggerFactory.getLogger(getClass())
20 |           .debug(
21 |               "HTTP method={} status={} durationMs={}",
22 |               req.getMethod(),
23 |               res.getStatus(),
24 |               (System.nanoTime() - start) / 1_000_000);
```

Captura tiempo monotónico para medir duración de la petición. No se usa como fecha de negocio ni para los vencimientos persistidos; el contador sirve para restar inicio y fin.

L25

```
25 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L26

```
26 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L27

```
27 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 9: separadores sin instrucciones.

SecurityConfiguration (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/config/SecurityConfiguration.java`

Objetivo y funcionalidad

Define permisos por ruta y método en fleet-service y la validación Bearer JWT. Protege el servicio incluso sin Gateway; diferencia roles y deja los controles de propietario al servicio de negocio.

L1

```
1 | package com.auxiruta.fleet.config;
```

Ubica el archivo en com.auxiruta.fleet.config. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.Set;
```

Importa tipos: java.util.Set. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import com.auxiruta.commons.security.TokenSupport;
```

Importa tipos: com.auxiruta.commons.security.TokenSupport. Utilidades compartidas de seguridad, errores, reloj o geografía. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.beans.factory.annotation.Value;
```

Importa tipos: org.springframework.beans.factory.annotation.Value. Inyección de dependencias y propiedades de entorno. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.context.annotation.*;
```

Importa tipos: org.springframework.context.annotation.*. Beans, configuración y eventos del contexto Spring. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.http.HttpMethod;
```

Importa tipos: org.springframework.http.HttpMethod. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.springframework.security.authorization.AuthorizationDecision;
```

Importa tipos: org.springframework.security.authorization.AuthorizationDecision. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | import org.springframework.security.oauth2.jwt.*;
```

Importa tipos: org.springframework.security.oauth2.jwt.*. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import org.springframework.security.oauth2.server.resource.authentication.*;
```

Importa tipos: org.springframework.security.oauth2.server.resource.authentication.*. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | import org.springframework.security.config.annotation.web.builders.HttpSecurity;
```

Importa tipos: org.springframework.security.config.annotation.web.builders.HttpSecurity. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12

```
12 | import org.springframework.security.web.SecurityFilterChain;
```

Importa tipos: org.springframework.security.web.SecurityFilterChain. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L13

```
13 | import org.springframework.security.config.http.SessionCreationPolicy;
```

Importa tipos: org.springframework.security.config.http.SessionCreationPolicy. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L14

```
14 | import org.springframework.security.authorization.AuthorizationManager;
```

Importa tipos: org.springframework.security.authorization.AuthorizationManager. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L15

```
15 | import org.springframework.security.web.access.intercept.RequestAuthorizationContext;
```

Importa tipos: org.springframework.security.web.access.intercept.RequestAuthorizationContext. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L17

```
17 | @Configuration
```

Declara configuración Spring.

L18

```
18 | public class SecurityConfiguration {
```

Declara clase SecurityConfiguration. Configura validación JWT y permisos por ruta y método. El primer matcher aplicable decide; las rutas específicas preceden a las generales y lo no enumerado se deniega.

L19

```
19 |   @Bean
```

Registra el objeto retornado en Spring.

L20 a L22

```
20 |   JwtDecoder jwtDecoder(
21 |       @Value("${spring.security.oauth2.resourceserver.jwt.issuer-uri}") String issuer,
22 |       @Value("${spring.security.oauth2.resourceserver.jwt.jwk-set-uri}") String keys) {
```

Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición. Define bean que decodifica y verifica el JWT; las URLs de emisor/claves provienen de configuración. Tipos y datos: issuer = emisor Keycloak esperado del JWT; String es texto; no es por sí mismo una referencia de entidad; keys = keys; String es texto; no es por sí mismo una referencia de entidad.

L23

```
23 |     var decoder = NimbusJwtDecoder.withJwkSetUri(keys).build();
```

Construye el objeto final a partir del builder configurado en líneas previas. Se calcula decoder (decoder) para reutilizar ese resultado en los pasos siguientes.

L24 a L25

```
24 |     decoder.setJwtValidator(
25 |         TokenSupport.validator(issuer, Set.of("auxiruta-api", "auxiruta-fleet")));
```

Añade validación de emisor, tiempo, sub y audiencia. Crea un conjunto no modificable y sin duplicados.

L26

```
26 |     return decoder;
```

Entrega el decodificador ya configurado al contenedor Spring. Resultado que recibe el llamador: decoder.

L27

```
27 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L29

```
29 |   @Bean
```

Registra el objeto retornado en Spring.

L30

```
30 |   SecurityFilterChain security(HttpSecurity http, SecurityErrors errors) throws Exception {
```

Construye la cadena Servlet de seguridad que intercepta peticiones antes del controlador.

L31

```
31 |     var converter = new JwtAuthenticationConverter();
```

Construye JwtAuthenticationConverter en memoria y conserva la referencia en converter. Los argumentos del constructor determinan los datos iniciales de este objeto. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L32 a L33

```
32 |     converter.setJwtGrantedAuthoritiesConverter(
33 |         jwt -> new java.util.ArrayList<>(TokenSupport.roles(jwt)));
```

Traduce roles de Keycloak a ROLE_ de Spring.

L34 a L109

```
    http.csrf(c -> c.disable())
        .httpBasic(c -> c.disable())
        .formLogin(c -> c.disable())
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .exceptionHandling(e -> e.authenticationEntryPoint(errors).accessDeniedHandler(errors))
        .oauth2ResourceServer(
            o ->
                o.authenticationEntryPoint(errors)
                    .accessDeniedHandler(errors)
                    .jwt(j -> j.jwtAuthenticationConverter(converter)))
        .authorizeHttpRequests(
            auth ->
                auth.requestMatchers(HttpMethod.GET, "/api/v1/operators/me")
                    .access(allowed(false, "OPERATOR"))
                    .requestMatchers(HttpMethod.PATCH, "/api/v1/operators/me/availability")
                    .access(allowed(false, "OPERATOR"))
                    .requestMatchers(HttpMethod.GET, "/api/v1/tow-trucks/assigned-to-me")
                    .access(allowed(false, "OPERATOR"))
                    .requestMatchers(HttpMethod.PUT,
"/api/v1/tow-trucks/assigned-to-me/location")
                    .access(allowed(false, "OPERATOR"))
                    .requestMatchers(
                        "/api/v1/operators/me",
                        "/api/v1/operators/me/**",
                        "/api/v1/tow-trucks/assigned-to-me",
                        "/api/v1/tow-trucks/assigned-to-me/**")
                    .denyAll()
                    .requestMatchers(HttpMethod.GET, "/api/v1/operators")
                    .access(allowed(false, "ADMIN"))
                    .requestMatchers(HttpMethod.GET, "/api/v1/operators/{id}")
                    .access(allowed(false, "ADMIN"))
                    .requestMatchers(HttpMethod.POST, "/api/v1/operators")
                    .access(allowed(false, "ADMIN"))
                    .requestMatchers(HttpMethod.PATCH, "/api/v1/operators/{id}")
                    .access(allowed(false, "ADMIN"))
                    .requestMatchers(HttpMethod.PATCH,
"/api/v1/operators/{id}/registration-status")
                    .access(allowed(false, "ADMIN"))
                    .requestMatchers(HttpMethod.PUT, "/api/v1/operators/{id}/qualification")
                    .access(allowed(false, "ADMIN"))
                    .requestMatchers(HttpMethod.GET, "/api/v1/tow-trucks")
                    .access(allowed(false, "ADMIN"))
                    .requestMatchers(HttpMethod.GET, "/api/v1/tow-trucks/{id}")
                    .access(allowed(false, "ADMIN"))
                    .requestMatchers(HttpMethod.POST, "/api/v1/tow-trucks")
                    .access(allowed(false, "ADMIN"))
                    .requestMatchers(HttpMethod.PATCH, "/api/v1/tow-trucks/{id}")
                    .access(allowed(false, "ADMIN"))
                    .requestMatchers(HttpMethod.PUT, "/api/v1/tow-trucks/{id}/operator")
                    .access(allowed(false, "ADMIN"))
                    .requestMatchers(HttpMethod.DELETE, "/api/v1/tow-trucks/{id}/operator")
                    .access(allowed(false, "ADMIN"))
                    .requestMatchers(HttpMethod.PATCH,
"/api/v1/tow-trucks/{id}/operational-status")
                    .access(allowed(false, "ADMIN"))
                    .requestMatchers(HttpMethod.GET, "/api/v1/configuration/fleet")
                    .access(allowed(false, "ADMIN"))
                    .requestMatchers(HttpMethod.GET, "/internal/fleet/candidates")
                    .access(allowed(true, "FLEET_SERVICE"))
                    .requestMatchers(HttpMethod.PUT, "/internal/reservations/{id}")
                    .access(allowed(true, "FLEET_SERVICE"))
                    .requestMatchers(HttpMethod.GET, "/internal/reservations/{id}")
                    .access(allowed(true, "FLEET_SERVICE"))
                    .requestMatchers(HttpMethod.POST, "/internal/reservations/{id}/confirmation")
                    .access(allowed(true, "FLEET_SERVICE"))
                    .requestMatchers(HttpMethod.POST, "/internal/reservations/{id}/release")
```

```
 97 |                     .access(allowed(true, "FLEET_SERVICE"))
 98 |                     .requestMatchers(HttpMethod.GET, "/internal/reservations/{id}/eligibility")
 99 |                     .access(allowed(true, "FLEET_SERVICE"))
100 |                     .requestMatchers(HttpMethod.POST, "/internal/reservations/status-queries")
101 |                     .access(allowed(true, "FLEET_SERVICE"))
102 |                     .requestMatchers(HttpMethod.GET, "/v3/api-docs/**")
103 |                     .access(allowed(false, "DRIVER", "OPERATOR", "ADMIN"))
104 |                     .requestMatchers(HttpMethod.GET, "/swagger-ui/**")
105 |                     .access(allowed(false, "DRIVER", "OPERATOR", "ADMIN"))
106 |                     .requestMatchers(HttpMethod.GET, "/swagger-ui.html")
107 |                     .access(allowed(false, "DRIVER", "OPERATOR", "ADMIN"))
108 |                     .anyRequest()
109 |                     .denyAll());
```

Construye la cadena de seguridad. Desactiva login por formulario y HTTP Basic porque los clientes presentan Bearer JWT de Keycloak. El contexto no se guarda como sesión de usuario. El conversor traduce roles del JWT a authorities y los manejadores normalizan 401/403. Después aplica reglas en orden: el primer patrón coincidente decide; las rutas específicas deben preceder a patrones amplios. allowed(false, ...) exige identidad humana; allowed(true, ...) exige cliente técnico. denyAll deniega una coincidencia explícita o, al final, cualquier petición que no haya coincidido. No significa que todas las rutas previas estén denegadas.

L47: permite GET /api/v1/operators/me con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L49: permite PATCH /api/v1/operators/me/availability con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L51: permite GET /api/v1/tow-trucks/assigned-to-me con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L53: permite PUT /api/v1/tow-trucks/assigned-to-me/location con OPERATOR. Exige token humano; la propiedad se valida además en negocio.

L59: deniega el patrón previo o cualquier ruta restante si sigue a anyRequest/anyExchange.

L61: permite GET /api/v1/operators con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L63: permite GET /api/v1/operators/{id} con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L65: permite POST /api/v1/operators con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L67: permite PATCH /api/v1/operators/{id} con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L69: permite PATCH /api/v1/operators/{id}/registration-status con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L71: permite PUT /api/v1/operators/{id}/qualification con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L73: permite GET /api/v1/tow-trucks con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L75: permite GET /api/v1/tow-trucks/{id} con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L77: permite POST /api/v1/tow-trucks con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L79: permite PATCH /api/v1/tow-trucks/{id} con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L81: permite PUT /api/v1/tow-trucks/{id}/operator con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L83: permite DELETE /api/v1/tow-trucks/{id}/operator con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L85: permite PATCH /api/v1/tow-trucks/{id}/operational-status con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L87: permite GET /api/v1/configuration/fleet con ADMIN. Exige token humano; la propiedad se valida además en negocio.

L89: permite GET /internal/fleet/candidates con FLEET_SERVICE. Exige token técnico.

L91: permite PUT /internal/reservations/{id} con FLEET_SERVICE. Exige token técnico.

L93: permite GET /internal/reservations/{id} con FLEET_SERVICE. Exige token técnico.

L95: permite POST /internal/reservations/{id}/confirmation con FLEET_SERVICE. Exige token técnico.

L97: permite POST /internal/reservations/{id}/release con FLEET_SERVICE. Exige token técnico.

L99: permite GET /internal/reservations/{id}/eligibility con FLEET_SERVICE. Exige token técnico.

L101: permite POST /internal/reservations/status-queries con FLEET_SERVICE. Exige token técnico.

L103: permite GET /v3/api-docs/** con DRIVER, OPERATOR, ADMIN. Exige token humano; la propiedad se valida además en negocio.

L105: permite GET /swagger-ui/** con DRIVER, OPERATOR, ADMIN. Exige token humano; la propiedad se valida además en negocio.

L107: permite GET /swagger-ui.html con DRIVER, OPERATOR, ADMIN. Exige token humano; la propiedad se valida además en negocio.

L109: deniega el patrón previo o cualquier ruta restante si sigue a anyRequest/anyExchange.

L110

```
110 |     return http.build();
```

Construye el objeto final a partir del builder configurado en líneas previas. Termina la cadena de seguridad con todos los matchers y handlers configurados.

L111

```
111 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L113 a L114

```
113 |   private AuthorizationManager<RequestAuthorizationContext> allowed(
114 |       boolean technical, String... roles) {
```

Combina autenticación JWT, audiencia, cliente autorizado y roles; no basta con un texto ROLE_ aislado. Tipos y datos: technical = indica si se exigen cliente y audiencia de servicio; boolean es verdadero/falso, sin null.

L115 a L118

```
115 |     return (authentication, context) ->
116 |         new AuthorizationDecision(
117 |             authentication.get() instanceof JwtAuthenticationToken token
118 |                 && TokenSupport.permits(token.getToken(), technical, roles));
```

Solo acepta Authentication basada en JWT y delega la combinación de cliente, audiencia y rol en TokenSupport. Devuelve una función de autorización evaluada por petición, no durante el arranque. Resultado que recibe el llamador: && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L118: && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L119

```
119 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L120

```
120 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 16, 28, 112: separadores sin instrucciones.

SecurityErrors (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/config/SecurityErrors.java`

Objetivo y funcionalidad

Transforma rechazos de autenticación y permisos de fleet-service en JSON 401/403. Actúa en la respuesta Servlet antes de llegar al controlador. No revela trazas.

L1

```
  1 | package com.auxiruta.fleet.config;
```

Ubica el archivo en com.auxiruta.fleet.config. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
  3 | import com.auxiruta.commons.errors.Problems;
```

Importa tipos: com.auxiruta.commons.errors.Problems. Utilidades compartidas de seguridad, errores, reloj o geografía. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import com.fasterxml.jackson.databind.ObjectMapper;
```

Importa tipos: com.fasterxml.jackson.databind.ObjectMapper. Lectura/escritura JSON, nodos y conversión a DTO. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import jakarta.servlet.http.*;
```

Importa tipos: jakarta.servlet.http.*. Tipos de petición, respuesta y cadena de filtros Servlet. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import java.io.IOException;
```

Importa tipos: java.io.IOException. Tipo auxiliar usado por las expresiones del archivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.springframework.security.core.AuthenticationException;
```

Importa tipos: org.springframework.security.core.AuthenticationException. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | import org.springframework.security.access.AccessDeniedException;
```

Importa tipos: org.springframework.security.access.AccessDeniedException. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import org.springframework.security.web.AuthenticationEntryPoint;
```

Importa tipos: org.springframework.security.web.AuthenticationEntryPoint. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | import org.springframework.security.web.access.AccessDeniedHandler;
```

Importa tipos: org.springframework.security.web.access.AccessDeniedHandler. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L13

```
13 | @Component
```

Registra componente inyectable.

L14

```
14 | public class SecurityErrors implements AuthenticationEntryPoint, AccessDeniedHandler {
```

Declara clase SecurityErrors. Convierte fallos de autenticación/autorización a respuestas JSON 401/403. Gateway además transforma errores de enrutamiento. No añade un mecanismo de inicio de sesión propio.

L15

```
15 |   private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L17

```
17 |   public SecurityErrors(ObjectMapper mapper) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada. Tipos y datos: mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L18

```
18 |     this.mapper = mapper;
```

En this guarda conversor JSON de Jackson. Toma el dato de mapper después de las validaciones de este método. conversor JSON de Jackson.

L19

```
19 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L21 a L22

```
21 |   public void commence(HttpServletRequest req, HttpServletResponse res, AuthenticationException
     e)
22 |       throws IOException {
```

Implementa AuthenticationEntryPoint: se invoca cuando falta autenticación válida y produce la respuesta 401.

L23

```
23 |     write(res, 401);
```

Una falla de autenticación produce HTTP 401 en Servlet.

L24

```
24 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L26 a L27

```
26 |   public void handle(HttpServletRequest req, HttpServletResponse res, AccessDeniedException e)
27 |       throws IOException {
```

Implementa el tratamiento de acceso denegado: una identidad sin permiso recibe 403; la variante reactiva también adapta errores de la cadena.

L28

```
28 |     write(res, 403);
```

Una falla de autorización produce HTTP 403 en Servlet.

L29

```
29 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L31

```
31 |   private void write(HttpServletResponse response, int status) throws IOException {
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Tipos y datos: status = estado del objeto; su enum o CHECK define valores permitidos; int es entero de 32 bits para límites y contadores.

L32

```
32 |     response.setStatus(status);
```

Fija el código HTTP numérico de la respuesta Servlet antes de escribir el cuerpo de error.

L33

```
33 |     response.setContentType("application/problem+json");
```

Declara application/problem+json para errores uniformes.

L34

```
34 |     if (status == 401) response.setHeader("WWW-Authenticate", "Bearer");
```

Añade desafío Bearer en 401. Esta condición decide si se entra en la rama siguiente. .

L35

```
35 |     mapper.writeValue(response.getOutputStream(), Problems.of(status));
```

Serializa ProblemDetail directamente en el cuerpo Servlet.

L36

```
36 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L37

```
37 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 12, 16, 20, 25, 30: separadores sin instrucciones.

ApiExceptionHandler (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/exceptions/ApiExceptionHandler.java`

Objetivo y funcionalidad

Convierte validación, conflictos SQL y errores de negocio de fleet-service en respuestas HTTP públicas sin SQL ni trazas. Hace distinguibles las entradas inválidas de los conflictos de estado.

L1

```
1 | package com.auxiruta.fleet.exceptions;
```

Ubica el archivo en com.auxiruta.fleet.exceptions. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import com.auxiruta.commons.errors.Problems;
```

Importa tipos: com.auxiruta.commons.errors.Problems. Utilidades compartidas de seguridad, errores, reloj o geografía. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.http.*;
```

Importa tipos: org.springframework.http.*. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.web.bind.annotation.*;
```

Importa tipos: org.springframework.web.bind.annotation.*. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;
```

Importa tipos: org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.web.context.request.WebRequest;
```

Importa tipos: org.springframework.web.context.request.WebRequest. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
 8 | import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
 9 | import org.springframework.security.access.AccessDeniedException;
```

Importa tipos: org.springframework.security.access.AccessDeniedException. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import org.slf4j.LoggerFactory;
```

Importa tipos: org.slf4j.LoggerFactory. Logging de eventos y fallos sin acoplarse a una implementación de consola. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12

```
12 | @RestControllerAdvice
```

Centraliza transformación de excepciones.

L13

```
13 | public class ApiExceptionHandler extends ResponseEntityExceptionHandler {
```

Declara clase ApiExceptionHandler. Normaliza errores de controladores, validación y persistencia. Las restricciones de integridad/concurrencia se exponen como 409, no como detalle SQL.

L14

```
14 |   @Override
```

Redefine contrato heredado.

L15 a L16

```
15 |   protected ResponseEntity<Object> handleExceptionInternal(
16 |       Exception ex, Object body, HttpHeaders headers, HttpStatusCode status, WebRequest request)
     {
```

Interviene en errores MVC de validación/parsing y preserva encabezados/estado.

L17

```
17 |     return new ResponseEntity<>(Problems.of(status.value()), headers, status);
```

Reemplaza el cuerpo de error por el formato común conservando estado y headers.

L18

```
18 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L20

```
20 |   @ExceptionHandler(ResponseStatusException.class)
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos.

L21

```
21 |   ResponseEntity<ProblemDetail> status(ResponseStatusException e) {
```

Manejador de la excepción indicada: toma el fallo que llega del servicio/validación y devuelve un ProblemDetail con código HTTP público. La anotación ExceptionHandler anterior define qué fallos llegan aquí.

L22

```
22 |     return ResponseEntity.status(e.getStatusCode()).body(Problems.of(e.getStatusCode().value()));
```

Manejador de la excepción indicada: toma el fallo que llega del servicio/validación y devuelve un ProblemDetail con código HTTP público. La anotación ExceptionHandler anterior define qué fallos llegan aquí.

L23

```
23 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L25

```
25 |   @ExceptionHandler(AccessDeniedException.class)
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos.

L26

```
26 |   ResponseEntity<ProblemDetail> forbidden(AccessDeniedException e) {
```

Manejador de la excepción indicada: toma el fallo que llega del servicio/validación y devuelve un ProblemDetail con código HTTP público. La anotación ExceptionHandler anterior define qué fallos llegan aquí.

L27

```
27 |     return ResponseEntity.status(403).body(Problems.of(403));
```

Manejador de la excepción indicada: toma el fallo que llega del servicio/validación y devuelve un ProblemDetail con código HTTP público. La anotación ExceptionHandler anterior define qué fallos llegan aquí.

L28

```
28 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L30 a L34

```
30 |   @ExceptionHandler({
31 |     org.springframework.dao.DataIntegrityViolationException.class,
32 |     org.springframework.dao.ConcurrencyFailureException.class
33 |   })
34 |   ResponseEntity<ProblemDetail> conflict(Exception e) {
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos. Normaliza errores de controladores, validación y persistencia. Las restricciones de integridad/concurrencia se exponen como 409, no como detalle SQL.

L35

```
35 |     return ResponseEntity.status(409).body(Problems.of(409));
```

Manejador de la excepción indicada: toma el fallo que llega del servicio/validación y devuelve un ProblemDetail con código HTTP público. La anotación ExceptionHandler anterior define qué fallos llegan aquí.

L36

```
36 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L38

```
38 |   @ExceptionHandler(jakarta.validation.ConstraintViolationException.class)
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos.

L39

```
39 |   ResponseEntity<ProblemDetail> invalidConstraint(Exception e) {
```

Manejador de la excepción indicada: toma el fallo que llega del servicio/validación y devuelve un ProblemDetail con código HTTP público. La anotación ExceptionHandler anterior define qué fallos llegan aquí.

L40

```
40 |     return ResponseEntity.badRequest().body(Problems.of(400));
```

Devuelve 400 ante validación de restricción inválida.

L41

```
41 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L43

```
43 |   @ExceptionHandler(Exception.class)
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos.

L44

```
44 |   ResponseEntity<ProblemDetail> unexpected(Exception e) {
```

Captura errores no clasificados para devolver 500 genérico y registrar solo tipo de excepción.

L45 a L46

```
45 |     LoggerFactory.getLogger(getClass())
46 |         .error("Error inesperado tipo={}", e.getClass().getSimpleName());
```

Registra los campos indicados, sin token ni secreto. Se calcula tipo (tipo) para reutilizar ese resultado en los pasos siguientes.

L47

```
47 |     return ResponseEntity.status(500).body(Problems.of(500));
```

Manejador de la excepción indicada: toma el fallo que llega del servicio/validación y devuelve un ProblemDetail con código HTTP público. La anotación ExceptionHandler anterior define qué fallos llegan aquí.

L48

```
48 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L49

```
49 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 11, 19, 24, 29, 37, 42: separadores sin instrucciones.

application-docker (fleet-service)

`Ruta: fleet-service/src/main/resources/application-docker.yml`

Objetivo y funcionalidad

Configura fleet-service al activar docker: credenciales SQL, scripts de esquema y validación Hibernate sin crear tablas. Define también emisor y claves para validar tokens.

L1

```
1 | spring:
```

Propiedades estándar de Spring Boot; sus hijos configuran componentes que se crean al arrancar.

L2

```
2 |   security:
```

Configuración de seguridad consumida por Spring Security.

L3

```
3 |     oauth2:
```

Configuración del protocolo de obtención/validación de tokens.

L4

```
4 |       resourceserver:
```

Aplicación receptora de access tokens; no emite tokens ni ofrece login.

L5

```
5 |         jwt:
```

Emisor y claves de validación de tokens firmados.

L6

```
6 |           issuer-uri: ${KEYCLOAK_ISSUER_URI}
```

Emisor esperado del JWT. El valor configurado es ${KEYCLOAK_ISSUER_URI}. El valor real se obtiene de KEYCLOAK_ISSUER_URI; aquí se conserva solo la referencia.

L7

```
7 |           jwk-set-uri: ${KEYCLOAK_JWK_SET_URI}
```

URL de claves públicas para verificar firma. El valor configurado es ${KEYCLOAK_JWK_SET_URI}. El valor real se obtiene de KEYCLOAK_JWK_SET_URI; aquí se conserva solo la referencia.

L8

```
8 |   datasource:
```

Pool de conexiones SQL: dirección, usuario y contraseña.

L9

```
9 |     url: ${DB_URL}
```

Dirección de conexión o servidor del bloque. El valor configurado es ${DB_URL}. El valor real se obtiene de DB_URL; aquí se conserva solo la referencia.

L10

```
10 |     username: ${DB_USER}
```

Usuario de conexión o identidad. El valor configurado es ${DB_USER}. El valor real se obtiene de DB_USER; aquí se conserva solo la referencia.

L11

```
11 |     password: ${DB_PASSWORD}
```

Referencia de contraseña; no reproduce el secreto local. El valor configurado es ${DB_PASSWORD}. El valor real se obtiene de DB_PASSWORD; aquí se conserva solo la referencia.

L12

```
12 |   sql:
```

Configuración de scripts SQL al iniciar.

L13

```
13 |     init:
```

Momento y orden de aplicación de scripts de esquema.

L14

```
14 |       mode: always
```

always aplica inicialización SQL también a PostgreSQL. El valor configurado es always.

L15

```
15 |       schema-locations:
     classpath:db/001-fleet-catalog.sql,classpath:db/002-reservations.sql,classpath:db/003-automation.
     sql
```

Scripts SQL ordenados para inicializar/extender esquema. El valor configurado es classpath:db/001-fleet-catalog.sql,classpath:db/002-reservations.sql,classpath:db/003-automation.sql.

L16

```
16 |   jpa:
```

Opciones de persistencia de entidades mediante JPA.

L17

```
17 |     open-in-view: false
```

false evita mantener EntityManager durante serialización HTTP. El valor configurado es false.

L18

```
18 |     hibernate:
```

Opciones del proveedor JPA que valida mapeos contra tablas.

L19

```
19 |       ddl-auto: validate
```

validate comprueba el esquema; Hibernate no crea tablas. El valor configurado es validate.

application (fleet-service)

`Ruta: fleet-service/src/main/resources/application.yml`

Objetivo y funcionalidad

Configura nombre y puerto base de fleet-service. También controla JSON desconocido, publicación OpenAPI, logs SQL y activación de tareas programadas.

L1

```
1 | spring:
```

Propiedades estándar de Spring Boot; sus hijos configuran componentes que se crean al arrancar.

L2

```
2 |   jackson:
```

Jackson transforma JSON recibido a DTO y objetos Java a JSON de respuesta.

L3

```
3 |     deserialization:
```

Opciones de lectura del JSON entrante, antes de aplicar las reglas del servicio.

L4

```
4 |       fail-on-unknown-properties: true
```

true rechaza campos JSON desconocidos. El valor configurado es true.

L5

```
5 |   application:
```

Identidad de la aplicación Spring, no de un usuario.

L6

```
6 |     name: fleet-service
```

Identifica a esta aplicación como fleet-service para el contexto Spring y sus registros; no es el nombre de un contenedor creado aquí.

L7

```
7 | server:
```

Opciones del servidor HTTP embebido, incluido su puerto.

L8

```
8 |   port: ${SERVER_PORT:8081}
```

Puerto de escucha; Compose controla su exposición al host. El valor configurado es ${SERVER_PORT:8081}. Spring busca SERVER_PORT y, si no está definida, usa '8081'.

L10

```
10 | springdoc:
```

Generación de contratos OpenAPI desde controladores y anotaciones.

L11

```
11 |   paths-to-match: /api/v1/**
```

Patrones incluidos en OpenAPI público. El valor configurado es /api/v1/**.

L13

```
13 | # SQL constraint messages may contain personal data; API errors are handled centrally.
```

Comentario o directiva del intérprete; no es regla de negocio.

L14

```
14 | logging:
```

Opciones de logs de Spring; no es la extensión x-logging de Compose.

L15

```
15 |   level:
```

Nivel de salida de cada categoría de logger, o su desactivación.

L16

```
16 |     org.hibernate.engine.jdbc.spi.SqlExceptionHelper: "OFF"
```

OFF silencia el logger Hibernate que podría imprimir detalles de restricciones SQL con datos personales. ApiExceptionHandler sigue devolviendo un error público; no se ignora el fallo de la transacción.

L18

```
18 | auxiruta:
```

Propiedades propias del proyecto, leídas por sus componentes.

L19

```
19 |   automation:
```

Interruptor de trabajos periódicos de búsqueda y recuperación.

L20

```
20 |     enabled: ${AUXIRUTA_AUTOMATION_ENABLED:true}
```

Activa el componente o identidad del bloque. El valor configurado es ${AUXIRUTA_AUTOMATION_ENABLED:true}. Spring busca AUXIRUTA_AUTOMATION_ENABLED y, si no está definida, usa 'true'.

Líneas vacías 9, 12, 17: separadores sin instrucciones.

auxiruta-realm (infra)

`Ruta: infra/keycloak/auxiruta-realm.json`

Objetivo y funcionalidad

Importación inicial de Keycloak: realm, roles, clientes, scopes, audiencias y cuentas demo. Los valores secretos son referencias de entorno. Un realm persistido no se reimporta al reiniciar.

L1

```
 1 | {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L2

```
 2 |   "realm": "auxiruta",
```

Espacio aislado de identidades auxiruta, separado del realm administrativo master.

L3

```
 3 |   "enabled": true,
```

Habilita este realm/cliente/usuario para su uso; false desactiva el elemento.

L4

```
4 |   "sslRequired": "external",
```

external exige HTTPS para peticiones consideradas externas; el entorno mostrado es de desarrollo local.

L5

```
5 |   "registrationAllowed": false,
```

false deshabilita autorregistro desde login: las cuentas se preparan administrativamente.

L6

```
6 |   "resetPasswordAllowed": false,
```

false deshabilita recuperación desde login; no configura envío de correo.

L7

```
7 |   "roles": {
```

Catálogo de roles de negocio y técnico que se asignarán a identidades.

L8

```
8 |     "realm": [
```

Espacio aislado de identidades auxiruta, separado del realm administrativo master.

L9

```
9 |       {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L10

```
10 |         "name": "DRIVER"
```

Nombre del rol o mapper dentro del objeto actual: "DRIVER". En roles identifica un permiso asignable; en protocolMappers etiqueta la regla que incorpora claims.

L11

```
11 |       },
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L12

```
12 |       {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L13

```
13 |         "name": "OPERATOR"
```

Nombre del rol o mapper dentro del objeto actual: "OPERATOR". En roles identifica un permiso asignable; en protocolMappers etiqueta la regla que incorpora claims.

L14

```
14 |       },
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L15

```
15 |       {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L16

```
16 |         "name": "ADMIN"
```

Nombre del rol o mapper dentro del objeto actual: "ADMIN". En roles identifica un permiso asignable; en protocolMappers etiqueta la regla que incorpora claims.

L17

```
17 |       },
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L18

```
18 |       {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L19

```
19 |         "name": "FLEET_SERVICE"
```

Nombre del rol o mapper dentro del objeto actual: "FLEET_SERVICE". En roles identifica un permiso asignable; en protocolMappers etiqueta la regla que incorpora claims.

L20

```
20 |       }
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L21

```
21 |     ]
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L22

```
22 |   },
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L23

```
23 |   "clients": [
```

Aplicaciones autorizadas a obtener tokens: cliente humano y cliente técnico.

L24

```
24 |     {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L25

```
25 |       "clientId": "auxiruta-api",
```

Identificador presentado por Postman o el servicio. No es username ni contraseña.

L26

```
26 |       "enabled": true,
```

Habilita este realm/cliente/usuario para su uso; false desactiva el elemento.

L27

```
27 |       "publicClient": true,
```

true indica cliente sin secreto que usa PKCE; false permite cliente confidencial con secreto.

L28

```
28 |       "standardFlowEnabled": true,
```

Habilita Authorization Code con login, el flujo humano usado en Postman.

L29

```
29 |       "directAccessGrantsEnabled": false,
```

false impide password grant; no se intercambian directamente usuario/contraseña por token desde el cliente.

L30

```
30 |       "fullScopeAllowed": false,
```

false restringe roles emitidos mediante scopeMappings, evitando incluir todos los roles sin control.

L31

```
31 |       "redirectUris": [
```

Callbacks admitidos al retornar del login; Keycloak valida el destino del código de autorización.

L32

```
32 |         "https://oauth.pstmn.io/v1/callback",
```

Elemento de la lista del bloque anterior: "https://oauth.pstmn.io/v1/callback". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L33

```
33 |         "http://127.0.0.1:8765/callback"
```

Elemento de la lista del bloque anterior: "http://127.0.0.1:8765/callback". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L34

```
34 |       ],
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L35

```
35 |       "webOrigins": [],
```

Orígenes CORS permitidos para clientes web. Vacío no autoriza cualquier origen.

L36

```
36 |       "attributes": {
```

Opciones adicionales del cliente, como exigir PKCE S256.

L37

```
37 |         "pkce.code.challenge.method": "S256"
```

S256 exige desafío SHA-256 del verificador PKCE; quien intercambia el código debe probar que inició el flujo.

L38

```
38 |       },
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L39

```
39 |       "defaultClientScopes": [
```

Scopes añadidos al token por defecto; profile/email/roles/basic controlan información y mapeos.

L40

```
40 |         "profile",
```

Elemento de la lista del bloque anterior: "profile". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L41

```
41 |         "email",
```

Elemento de la lista del bloque anterior: "email". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L42

```
42 |         "roles",
```

Elemento de la lista del bloque anterior: "roles". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L43

```
43 |         "basic"
```

Elemento de la lista del bloque anterior: "basic". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L44

```
44 |       ],
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L45

```
45 |       "protocolMappers": [
```

Reglas para incorporar claims. Aquí agregan la audiencia esperada por la API.

L46

```
46 |         {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L47

```
47 |           "name": "audience-auxiruta-api",
```

Nombre del rol o mapper dentro del objeto actual: "audience-auxiruta-api". En roles identifica un permiso asignable; en protocolMappers etiqueta la regla que incorpora claims.

L48

```
48 |           "protocol": "openid-connect",
```

openid-connect selecciona OIDC, la capa de identidad sobre OAuth 2.0.

L49

```
49 |           "protocolMapper": "oidc-audience-mapper",
```

oidc-audience-mapper agrega el destinatario lógico del token.

L50

```
50 |           "config": {
```

Parámetros del mapper: audiencia y tipos de token afectados.

L51

```
51 |             "included.custom.audience": "auxiruta-api",
```

auxiruta-api identifica API humana; auxiruta-fleet, la API técnica. TokenSupport verifica esta separación.

L52

```
52 |             "access.token.claim": "true",
```

true añade el claim al access token, el que se envía como Bearer.

L53

```
53 |             "id.token.claim": "false"
```

false evita añadirlo al ID token, que no es el token de acceso de la API.

L54

```
54 |           }
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L55

```
55 |         }
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L56

```
56 |       ]
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L57

```
57 |     },
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L58

```
58 |     {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L59

```
59 |       "clientId": "assistance-service",
```

Identificador presentado por Postman o el servicio. No es username ni contraseña.

L60

```
60 |       "enabled": true,
```

Habilita este realm/cliente/usuario para su uso; false desactiva el elemento.

L61

```
61 |       "publicClient": false,
```

true indica cliente sin secreto que usa PKCE; false permite cliente confidencial con secreto.

L62

```
62 |       "secret": "${KEYCLOAK_CLIENT_SECRET}",
```

Secreto del cliente confidencial, resuelto desde entorno al importar. El valor real se obtiene de KEYCLOAK_CLIENT_SECRET; aquí se conserva solo la referencia.

L63

```
63 |       "serviceAccountsEnabled": true,
```

Permite client_credentials con la identidad del cliente, sin login humano.

L64

```
64 |       "standardFlowEnabled": false,
```

Habilita Authorization Code con login, el flujo humano usado en Postman.

L65

```
65 |       "directAccessGrantsEnabled": false,
```

false impide password grant; no se intercambian directamente usuario/contraseña por token desde el cliente.

L66

```
66 |       "fullScopeAllowed": false,
```

false restringe roles emitidos mediante scopeMappings, evitando incluir todos los roles sin control.

L67

```
67 |       "defaultClientScopes": [
```

Scopes añadidos al token por defecto; profile/email/roles/basic controlan información y mapeos.

L68

```
68 |         "roles",
```

Elemento de la lista del bloque anterior: "roles". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L69

```
69 |         "basic"
```

Elemento de la lista del bloque anterior: "basic". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L70

```
70 |       ],
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L71

```
71 |       "protocolMappers": [
```

Reglas para incorporar claims. Aquí agregan la audiencia esperada por la API.

L72

```
72 |         {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L73

```
73 |           "name": "audience-auxiruta-fleet",
```

Nombre del rol o mapper dentro del objeto actual: "audience-auxiruta-fleet". En roles identifica un permiso asignable; en protocolMappers etiqueta la regla que incorpora claims.

L74

```
74 |           "protocol": "openid-connect",
```

openid-connect selecciona OIDC, la capa de identidad sobre OAuth 2.0.

L75

```
75 |           "protocolMapper": "oidc-audience-mapper",
```

oidc-audience-mapper agrega el destinatario lógico del token.

L76

```
76 |           "config": {
```

Parámetros del mapper: audiencia y tipos de token afectados.

L77

```
77 |             "included.custom.audience": "auxiruta-fleet",
```

auxiruta-api identifica API humana; auxiruta-fleet, la API técnica. TokenSupport verifica esta separación.

L78

```
78 |             "access.token.claim": "true",
```

true añade el claim al access token, el que se envía como Bearer.

L79

```
79 |             "id.token.claim": "false"
```

false evita añadirlo al ID token, que no es el token de acceso de la API.

L80

```
80 |           }
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L81

```
81 |         }
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L82

```
82 |       ]
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L83

```
83 |     }
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L84

```
84 |   ],
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L85

```
85 |   "scopeMappings": [
```

Limita los roles que puede incluir el token de cada cliente.

L86

```
86 |     {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L87

```
87 |       "client": "auxiruta-api",
```

Cliente al que se asigna este alcance de roles.

L88

```
88 |       "roles": [
```

Catálogo de roles de negocio y técnico que se asignarán a identidades.

L89

```
89 |         "DRIVER",
```

Elemento de la lista del bloque anterior: "DRIVER". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L90

```
90 |         "OPERATOR",
```

Elemento de la lista del bloque anterior: "OPERATOR". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L91

```
91 |         "ADMIN"
```

Elemento de la lista del bloque anterior: "ADMIN". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L92

```
92 |       ]
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L93

```
93 |     },
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L94

```
94 |     {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L95

```
95 |       "client": "assistance-service",
```

Cliente al que se asigna este alcance de roles.

L96

```
96 |       "roles": [
```

Catálogo de roles de negocio y técnico que se asignarán a identidades.

L97

```
97 |         "FLEET_SERVICE"
```

Elemento de la lista del bloque anterior: "FLEET_SERVICE". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L98

```
98 |       ]
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L99

```
99 |     }
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L100

```
100 |   ],
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L101

```
101 |   "users": [
```

Cuentas iniciales. No crea perfiles Fleet ni vehículos de Assistance.

L102

```
102 |     {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L103

```
103 |       "username": "service-account-assistance-service",
```

Nombre de login visible. La propiedad de negocio usa sub y no este nombre.

L104

```
104 |       "enabled": true,
```

Habilita este realm/cliente/usuario para su uso; false desactiva el elemento.

L105

```
105 |       "serviceAccountClientId": "assistance-service",
```

Vincula la cuenta técnica a su cliente OAuth para asignarle roles.

L106

```
106 |       "realmRoles": [
```

Roles de esta identidad: DRIVER, OPERATOR, ADMIN o FLEET_SERVICE técnico.

L107

```
107 |         "FLEET_SERVICE"
```

Elemento de la lista del bloque anterior: "FLEET_SERVICE". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L108

```
108 |       ]
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L109

```
109 |     },
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L110

```
110 |     {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L111

```
111 |       "username": "driver.demo",
```

Nombre de login visible. La propiedad de negocio usa sub y no este nombre.

L112

```
112 |       "enabled": true,
```

Habilita este realm/cliente/usuario para su uso; false desactiva el elemento.

L113

```
113 |       "emailVerified": true,
```

Declara correo verificado para demo, sin enviar mensajes.

L114

```
114 |       "firstName": "Driver",
```

Nombre visible del perfil de identidad, sin efecto en permisos.

L115

```
115 |       "lastName": "Demo",
```

Apellido visible del perfil, sin efecto en propiedad.

L116

```
116 |       "email": "driver@auxiruta.example",
```

Dirección del perfil; no implementa notificaciones por correo.

L117

```
117 |       "credentials": [
```

Credenciales iniciales que Keycloak administra; los servicios no guardan contraseñas.

L118

```
118 |         {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L119

```
119 |           "type": "password",
```

password identifica la clase de credencial inicial.

L120

```
120 |           "value": "${DEMO_DRIVER_PASSWORD}",
```

Referencia a la contraseña del entorno, no un access token. El valor real se obtiene de DEMO_DRIVER_PASSWORD; aquí se conserva solo la referencia.

L121

```
121 |           "temporary": false
```

false evita exigir un cambio de clave al primer login demo.

L122

```
122 |         }
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L123

```
123 |       ],
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L124

```
124 |       "realmRoles": [
```

Roles de esta identidad: DRIVER, OPERATOR, ADMIN o FLEET_SERVICE técnico.

L125

```
125 |         "DRIVER"
```

Elemento de la lista del bloque anterior: "DRIVER". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L126

```
126 |       ]
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L127

```
127 |     },
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L128

```
128 |     {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L129

```
129 |       "username": "operator.demo",
```

Nombre de login visible. La propiedad de negocio usa sub y no este nombre.

L130

```
130 |       "enabled": true,
```

Habilita este realm/cliente/usuario para su uso; false desactiva el elemento.

L131

```
131 |       "emailVerified": true,
```

Declara correo verificado para demo, sin enviar mensajes.

L132

```
132 |       "firstName": "Operator",
```

Nombre visible del perfil de identidad, sin efecto en permisos.

L133

```
133 |       "lastName": "Demo",
```

Apellido visible del perfil, sin efecto en propiedad.

L134

```
134 |       "email": "operator@auxiruta.example",
```

Dirección del perfil; no implementa notificaciones por correo.

L135

```
135 |       "credentials": [
```

Credenciales iniciales que Keycloak administra; los servicios no guardan contraseñas.

L136

```
136 |         {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L137

```
137 |           "type": "password",
```

password identifica la clase de credencial inicial.

L138

```
138 |           "value": "${DEMO_OPERATOR_PASSWORD}",
```

Referencia a la contraseña del entorno, no un access token. El valor real se obtiene de DEMO_OPERATOR_PASSWORD; aquí se conserva solo la referencia.

L139

```
139 |           "temporary": false
```

false evita exigir un cambio de clave al primer login demo.

L140

```
140 |         }
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L141

```
141 |       ],
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L142

```
142 |       "realmRoles": [
```

Roles de esta identidad: DRIVER, OPERATOR, ADMIN o FLEET_SERVICE técnico.

L143

```
143 |         "OPERATOR"
```

Elemento de la lista del bloque anterior: "OPERATOR". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L144

```
144 |       ]
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L145

```
145 |     },
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L146

```
146 |     {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L147

```
147 |       "username": "admin.demo",
```

Nombre de login visible. La propiedad de negocio usa sub y no este nombre.

L148

```
148 |       "enabled": true,
```

Habilita este realm/cliente/usuario para su uso; false desactiva el elemento.

L149

```
149 |       "emailVerified": true,
```

Declara correo verificado para demo, sin enviar mensajes.

L150

```
150 |       "firstName": "Admin",
```

Nombre visible del perfil de identidad, sin efecto en permisos.

L151

```
151 |       "lastName": "Demo",
```

Apellido visible del perfil, sin efecto en propiedad.

L152

```
152 |       "email": "admin@auxiruta.example",
```

Dirección del perfil; no implementa notificaciones por correo.

L153

```
153 |       "credentials": [
```

Credenciales iniciales que Keycloak administra; los servicios no guardan contraseñas.

L154

```
154 |         {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L155

```
155 |           "type": "password",
```

password identifica la clase de credencial inicial.

L156

```
156 |           "value": "${DEMO_ADMIN_PASSWORD}",
```

Referencia a la contraseña del entorno, no un access token. El valor real se obtiene de DEMO_ADMIN_PASSWORD; aquí se conserva solo la referencia.

L157

```
157 |           "temporary": false
```

false evita exigir un cambio de clave al primer login demo.

L158

```
158 |         }
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L159

```
159 |       ],
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L160

```
160 |       "realmRoles": [
```

Roles de esta identidad: DRIVER, OPERATOR, ADMIN o FLEET_SERVICE técnico.

L161

```
161 |         "ADMIN"
```

Elemento de la lista del bloque anterior: "ADMIN". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L162

```
162 |       ]
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L163

```
163 |     },
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L164

```
164 |     {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L165

```
165 |       "username": "driver2.demo",
```

Nombre de login visible. La propiedad de negocio usa sub y no este nombre.

L166

```
166 |       "enabled": true,
```

Habilita este realm/cliente/usuario para su uso; false desactiva el elemento.

L167

```
167 |       "emailVerified": true,
```

Declara correo verificado para demo, sin enviar mensajes.

L168

```
168 |       "firstName": "Driver",
```

Nombre visible del perfil de identidad, sin efecto en permisos.

L169

```
169 |       "lastName": "Demo 2",
```

Apellido visible del perfil, sin efecto en propiedad.

L170

```
170 |       "email": "driver2@auxiruta.example",
```

Dirección del perfil; no implementa notificaciones por correo.

L171

```
171 |       "credentials": [
```

Credenciales iniciales que Keycloak administra; los servicios no guardan contraseñas.

L172

```
172 |         {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L173

```
173 |           "type": "password",
```

password identifica la clase de credencial inicial.

L174

```
174 |           "value": "${DEMO_DRIVER_PASSWORD}",
```

Referencia a la contraseña del entorno, no un access token. El valor real se obtiene de DEMO_DRIVER_PASSWORD; aquí se conserva solo la referencia.

L175

```
175 |           "temporary": false
```

false evita exigir un cambio de clave al primer login demo.

L176

```
176 |         }
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L177

```
177 |       ],
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L178

```
178 |       "realmRoles": [
```

Roles de esta identidad: DRIVER, OPERATOR, ADMIN o FLEET_SERVICE técnico.

L179

```
179 |         "DRIVER"
```

Elemento de la lista del bloque anterior: "DRIVER". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L180

```
180 |       ]
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L181

```
181 |     },
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L182

```
182 |     {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L183

```
183 |       "username": "operator2.demo",
```

Nombre de login visible. La propiedad de negocio usa sub y no este nombre.

L184

```
184 |       "enabled": true,
```

Habilita este realm/cliente/usuario para su uso; false desactiva el elemento.

L185

```
185 |       "emailVerified": true,
```

Declara correo verificado para demo, sin enviar mensajes.

L186

```
186 |       "firstName": "Operator",
```

Nombre visible del perfil de identidad, sin efecto en permisos.

L187

```
187 |       "lastName": "Demo 2",
```

Apellido visible del perfil, sin efecto en propiedad.

L188

```
188 |       "email": "operator2@auxiruta.example",
```

Dirección del perfil; no implementa notificaciones por correo.

L189

```
189 |       "credentials": [
```

Credenciales iniciales que Keycloak administra; los servicios no guardan contraseñas.

L190

```
190 |         {
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L191

```
191 |           "type": "password",
```

password identifica la clase de credencial inicial.

L192

```
192 |           "value": "${DEMO_OPERATOR_PASSWORD}",
```

Referencia a la contraseña del entorno, no un access token. El valor real se obtiene de DEMO_OPERATOR_PASSWORD; aquí se conserva solo la referencia.

L193

```
193 |           "temporary": false
```

false evita exigir un cambio de clave al primer login demo.

L194

```
194 |         }
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L195

```
195 |       ],
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L196

```
196 |       "realmRoles": [
```

Roles de esta identidad: DRIVER, OPERATOR, ADMIN o FLEET_SERVICE técnico.

L197

```
197 |         "OPERATOR"
```

Elemento de la lista del bloque anterior: "OPERATOR". Un rol limita permisos, un scope selecciona información del token y un callback limita destinos de retorno; el nombre de la propiedad contenedora indica cuál se está definiendo.

L198

```
198 |       ]
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L199

```
199 |     }
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L200

```
200 |   ]
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

L201

```
201 | }
```

Llave/corchete JSON que cierra o separa objetos y listas. Mantiene la estructura de importación; no crea por sí solo una identidad.

4. Persistencia: entidades, repositorios y SQL

Cada servicio administra su base. UUID remotos no son FK entre bases. Keycloak administra sus tablas internas, no definidas por Java del proyecto.

AssistanceRequest (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/AssistanceRequest.java

Objetivo y funcionalidad

Entidad central con estado, ubicación, datos declarados del vehículo, primera aceptación y snapshot de reglas. Los campos de automatización permiten reanudar después de reinicios.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | @Entity
```

Marca entidad JPA. JPA administra instancias de esta clase como filas; se necesita @Id y un esquema compatible.

L10 a L13

```
10 | @Table(
11 |     name = "assistance_requests",
12 |     uniqueConstraints = @UniqueConstraint(columnNames = {"driver_subject", "idempotency_key"}))
13 | public class AssistanceRequest {
```

Define tabla y restricciones de mapeo. Declara clase AssistanceRequest. Entidad central con estado, ubicación, datos declarados del vehículo, primera aceptación y snapshot de reglas. Los campos de automatización permiten reanudar después de reinicios.

L14

```
14 |   protected AssistanceRequest() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L16

```
16 |   @Id UUID id = UUID.randomUUID();
```

Identifica clave primaria. Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro.

L17

```
17 |   @Version long version;
```

Activa concurrencia optimista mediante comparación/incremento de versión. Campos: version: versión para control optimista de concurrencia. Hibernate agrega la versión a la condición del UPDATE; si otra transacción la cambió, detecta conflicto en vez de pisar datos. Tipos y datos: version = versión para control optimista de concurrencia; long es entero de 64 bits para cantidades y duraciones mayores.

L19

```
19 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L20

```
20 |   Instant createdAt;
```

Campos: createdAt: fecha de creación. Tipos y datos: createdAt = fecha de creación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L22

```
22 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L23

```
23 |   Instant updatedAt;
```

Campos: updatedAt: fecha de modificación. Tipos y datos: updatedAt = fecha de modificación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L25

```
25 |   public UUID getId() {
```

L26

```
26 |     return id;
```

Entrega id al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: identificador UUID del registro.

L27

```
27 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L29

```
29 |   @ManyToOne(fetch = FetchType.LAZY, optional = false)
```

Relaciona muchas filas con una entidad; LAZY difiere la carga.

L30

```
30 |   @JoinColumn(name = "vehicle_id")
```

Nombra FK local.

L31

```
31 |   Vehicle vehicle;
```

Referencia Java a Vehicle para vehicle. Las anotaciones inmediatamente anteriores fijan la relación JPA y la columna de enlace; no comparte tablas entre servicios.

L33

```
33 |   @Column(nullable = false, length = 100)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L34

```
34 |   String driverSubject;
```

Campos: driverSubject: sub del conductor que creó la solicitud. Tipos y datos: driverSubject = sub del conductor que creó la solicitud; String es texto; no es por sí mismo una referencia de entidad.

L36

```
36 |   @Enumerated(EnumType.STRING)
```

Guarda nombre del enum, no posición ordinal.

L37

```
37 |   @Column(nullable = false, length = 30)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L38

```
38 |   Status status = Status.REQUESTED;
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. REQUESTED: alternativa del ciclo de vida definido en el enum

L40

```
40 |   Instant firstAssignedAt;
```

Campos: firstAssignedAt: primera aceptación que impide cancelar incluso durante reemplazos. Tipos y datos: firstAssignedAt = primera aceptación que impide cancelar incluso durante reemplazos; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L42

```
42 |   @Column(length = 100)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L43

```
43 |   String closureReasonCode;
```

Campos: closureReasonCode: causa codificada del cierre. Tipos y datos: closureReasonCode = causa codificada del cierre; String es texto; no es por sí mismo una referencia de entidad.

L45

```
45 |   @Column(nullable = false, precision = 9, scale = 6)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L46

```
46 |   BigDecimal originLatitude;
```

Campos: originLatitude: latitud de recogida. Tipos y datos: originLatitude = latitud de recogida; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L48

```
48 |   @Column(nullable = false, precision = 9, scale = 6)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L49

```
49 |   BigDecimal originLongitude;
```

Campos: originLongitude: longitud de recogida. Tipos y datos: originLongitude = longitud de recogida; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L51

```
51 |   @Column(nullable = false, precision = 9, scale = 6)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L52

```
52 |   BigDecimal destinationLatitude;
```

Campos: destinationLatitude: latitud de entrega. Tipos y datos: destinationLatitude = latitud de entrega; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L54

```
54 |   @Column(nullable = false, precision = 9, scale = 6)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L55

```
55 |   BigDecimal destinationLongitude;
```

Campos: destinationLongitude: longitud de entrega. Tipos y datos: destinationLongitude = longitud de entrega; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L57

```
57 |   @Enumerated(EnumType.STRING)
```

Guarda nombre del enum, no posición ordinal.

L58

```
58 |   @Column(nullable = false, length = 20)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L59

```
59 |   VehicleType declaredVehicleType;
```

Campos: declaredVehicleType: copia del tipo de vehículo al solicitar.

L61

```
61 |   @Column(nullable = false, precision = 9, scale = 2)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L62

```
62 |   BigDecimal declaredWeightKg;
```

Campos: declaredWeightKg: peso declarado en kg. Tipos y datos: declaredWeightKg = peso declarado en kg; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L64

```
64 |   @Enumerated(EnumType.STRING)
```

Guarda nombre del enum, no posición ordinal.

L65

```
65 |   @Column(nullable = false, length = 20)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L66

```
66 |   Mobility mobilityCondition;
```

Campos: mobilityCondition: condición ROLLABLE/NON_ROLLABLE.

L68

```
68 |   @Column(nullable = false, length = 1000)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L69

```
69 |   String description;
```

Campos: description: descripción de asistencia. Tipos y datos: description = descripción de asistencia; String es texto; no es por sí mismo una referencia de entidad.

L71

```
71 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L72

```
72 |   int searchAttemptNumber = 1;
```

Campos: searchAttemptNumber: número de búsqueda de 1 a 3. Se calcula searchAttemptNumber (número de búsqueda de 1 a 3) para reutilizar ese resultado en los pasos siguientes. 1

L74

```
74 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L75

```
75 |   Instant searchDeadline;
```

Campos: searchDeadline: vencimiento del intento. Tipos y datos: searchDeadline = vencimiento del intento; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L77

```
77 |   @Column(nullable = false, length = 30)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L78

```
78 |   String configurationRevision;
```

Campos: configurationRevision: versión de reglas. Tipos y datos: configurationRevision = versión de reglas; String es texto; no es por sí mismo una referencia de entidad.

L80

```
80 |   @Column(nullable = false, columnDefinition = "text")
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L81

```
81 |   String configurationSnapshot;
```

Campos: configurationSnapshot: JSON de parámetros vigente al crear. Tipos y datos: configurationSnapshot = JSON de parámetros vigente al crear; String es texto; no es por sí mismo una referencia de entidad.

L83

```
83 |   @Column(nullable = false, length = 100)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L84

```
84 |   String idempotencyKey;
```

Campos: idempotencyKey: clave para repetir sin duplicar. Tipos y datos: idempotencyKey = clave para repetir sin duplicar; String es texto; no es por sí mismo una referencia de entidad.

L86

```
86 |   @Column(nullable = false, length = 64)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L87

```
87 |   String requestFingerprint;
```

Campos: requestFingerprint: SHA-256 del contenido asociado a la clave. Tipos y datos: requestFingerprint = SHA-256 del contenido asociado a la clave; String es texto; no es por sí mismo una referencia de entidad.

L89

```
89 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L90

```
90 |   boolean resourceReleasePending;
```

Campos: resourceReleasePending: liberación remota pendiente que impide reutilización. Tipos y datos: resourceReleasePending = liberación remota pendiente que impide reutilización; boolean es verdadero/falso, sin null.

L92

```
92 |   @org.hibernate.annotations.OptimisticLock(excluded = true)
```

excluded=true evita incrementar versión por este campo de planificación.

L93

```
93 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L94

```
94 |   Instant nextEvaluationAt = Instant.EPOCH;
```

Usa 1970-01-01 UTC como planificación inicialmente vencida. Campos: nextEvaluationAt: próximo trabajo de evaluación. Se calcula nextEvaluationAt (próximo trabajo de evaluación) para reutilizar ese resultado en los pasos siguientes.

L96

```
96 |   @org.hibernate.annotations.OptimisticLock(excluded = true)
```

excluded=true evita incrementar versión por este campo de planificación.

L97

```
97 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L98

```
98 |   Instant nextReconciliationAt = Instant.EPOCH;
```

Usa 1970-01-01 UTC como planificación inicialmente vencida. Campos: nextReconciliationAt: próxima revisión del compromiso. Se calcula nextReconciliationAt (próxima revisión del compromiso) para reutilizar ese resultado en los pasos siguientes.

L100

```
100 |   @org.hibernate.annotations.OptimisticLock(excluded = true)
```

excluded=true evita incrementar versión por este campo de planificación.

L101

```
101 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L102

```
102 |   Instant evaluationLeaseUntil = Instant.EPOCH;
```

Usa 1970-01-01 UTC como planificación inicialmente vencida. Campos: evaluationLeaseUntil: límite de trabajo de evaluación en curso. Se calcula evaluationLeaseUntil (límite de trabajo de evaluación en curso) para reutilizar ese resultado en los pasos siguientes.

L104

```
104 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L105

```
105 |   boolean replacementPending;
```

Campos: replacementPending: necesidad de iniciar reemplazo después de liberar. Tipos y datos: replacementPending = necesidad de iniciar reemplazo después de liberar; boolean es verdadero/falso, sin null.

L107

```
107 |   Instant closedAt;
```

Campos: closedAt: fecha de cierre; null indica abierto. Tipos y datos: closedAt = fecha de cierre; null indica abierto; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L108

```
108 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 8, 15, 18, 21, 24, 28, 32, 35, 39, 41, 44, 47, 50, 53, 56, 60, 63, 67, 70, 73, 76, 79, 82, 85, 88, 91, 95, 99, 103, 106: separadores sin instrucciones.

CandidateEvaluation (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/CandidateEvaluation.java`

Objetivo y funcionalidad

Snapshot por candidato de compatibilidad, elegibilidad, recorridos y posición en el ranking. Referencia UUID remotos sin FK hacia Fleet.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | @Entity
```

Marca entidad JPA. JPA administra instancias de esta clase como filas; se necesita @Id y un esquema compatible.

L9

```
9 | @Table(name = "candidate_evaluations")
```

Define tabla y restricciones de mapeo.

L10

```
10 | public class CandidateEvaluation {
```

Declara clase CandidateEvaluation. Snapshot por candidato de compatibilidad, elegibilidad, recorridos y posición en el ranking. Referencia UUID remotos sin FK hacia Fleet.

L11

```
11 |   protected CandidateEvaluation() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L13

```
13 |   @Id UUID id = UUID.randomUUID();
```

Identifica clave primaria. Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro.

L15

```
15 |   @ManyToOne(fetch = FetchType.LAZY, optional = false)
```

Relaciona muchas filas con una entidad; LAZY difiere la carga.

L16

```
16 |   @JoinColumn(name = "evaluation_id")
```

Nombra FK local.

L17

```
17 |   Evaluation evaluation;
```

Referencia Java a Evaluation para evaluation. Las anotaciones inmediatamente anteriores fijan la relación JPA y la columna de enlace; no comparte tablas entre servicios.

L19

```
19 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L20

```
20 |   UUID towTruckId;
```

Campos: towTruckId: UUID de grúa. Tipos y datos: towTruckId = UUID de grúa; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L22

```
22 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L23

```
23 |   UUID operatorId;
```

Campos: operatorId: UUID del perfil operador. Tipos y datos: operatorId = UUID del perfil operador; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L25

```
25 |   @Column(nullable = false, length = 100)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L26

```
26 |   String operatorSubject;
```

Campos: operatorSubject: sub del operador destinatario. Tipos y datos: operatorSubject = sub del operador destinatario; String es texto; no es por sí mismo una referencia de entidad.

L28

```
28 |   @Column(nullable = false, precision = 9, scale = 2)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L29

```
29 |   BigDecimal truckCapacityKg;
```

Campos: truckCapacityKg: snapshot de capacidad. Tipos y datos: truckCapacityKg = snapshot de capacidad; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L31

```
31 |   @Column(nullable = false, length = 30)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L32

```
32 |   String qualificationCategory;
```

Campos: qualificationCategory: habilitación LIGHT_TOWING. Tipos y datos: qualificationCategory = habilitación LIGHT_TOWING; String es texto; no es por sí mismo una referencia de entidad.

L34

```
34 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L35

```
35 |   Instant qualificationValidUntil;
```

Campos: qualificationValidUntil: fin de habilitación que debe superar el servicio estimado. Tipos y datos: qualificationValidUntil = fin de habilitación que debe superar el servicio estimado; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L37

```
37 |   @Column(nullable = false, precision = 10, scale = 7)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L38

```
38 |   BigDecimal truckLatitude;
```

Campos: truckLatitude: snapshot de latitud de grúa. Tipos y datos: truckLatitude = snapshot de latitud de grúa; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L40

```
40 |   @Column(nullable = false, precision = 10, scale = 7)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L41

```
41 |   BigDecimal truckLongitude;
```

Campos: truckLongitude: snapshot de longitud de grúa. Tipos y datos: truckLongitude = snapshot de longitud de grúa; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L43

```
43 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L44

```
44 |   Instant truckLocationUpdatedAt;
```

Campos: truckLocationUpdatedAt: fecha de posición usada al evaluar. Tipos y datos: truckLocationUpdatedAt = fecha de posición usada al evaluar; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L46

```
46 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L47

```
47 |   boolean compatible;
```

Campos: compatible: compatibilidad de capacidad, tipo y movilidad. Tipos y datos: compatible = compatibilidad de capacidad, tipo y movilidad; boolean es verdadero/falso, sin null.

L49

```
49 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L50

```
50 |   boolean eligible;
```

Campos: eligible: aptitud completa del candidato/compromiso. Tipos y datos: eligible = aptitud completa del candidato/compromiso; boolean es verdadero/falso, sin null.

L52

```
52 |   @Column(nullable = true, length = 100)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L53

```
53 |   String reasonCode;
```

Campos: reasonCode: causa codificada. Tipos y datos: reasonCode = causa codificada; String es texto; no es por sí mismo una referencia de entidad.

L55

```
55 |   @Column(name = "approach_distance_m", columnDefinition = "numeric")
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L56

```
56 |   BigDecimal approachDistanceM;
```

Campos: approachDistanceM: distancia vial grúa-recogida. Tipos y datos: approachDistanceM = distancia vial grúa-recogida; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L58

```
58 |   @Column(nullable = true, columnDefinition = "numeric")
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L59

```
59 |   BigDecimal approachDurationSeconds;
```

Campos: approachDurationSeconds: duración original de aproximación. Tipos y datos: approachDurationSeconds = duración original de aproximación; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L61

```
61 |   @Column(nullable = true)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L62

```
62 |   Integer adjustedApproachSeconds;
```

Campos: adjustedApproachSeconds: aproximación ajustada.

L64

```
64 |   @Column(nullable = true)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L65

```
65 |   Instant approachRouteObtainedAt;
```

Campos: approachRouteObtainedAt: fecha de ruta de aproximación. Tipos y datos: approachRouteObtainedAt = fecha de ruta de aproximación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L67

```
67 |   @Column(name = "transport_distance_m", columnDefinition = "numeric")
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L68

```
68 |   BigDecimal transportDistanceM;
```

Campos: transportDistanceM: distancia vial origen-destino en metros. Tipos y datos: transportDistanceM = distancia vial origen-destino en metros; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L70

```
70 |   @Column(nullable = true, columnDefinition = "numeric")
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L71

```
71 |   BigDecimal transportDurationSeconds;
```

Campos: transportDurationSeconds: duración vial original del traslado. Tipos y datos: transportDurationSeconds = duración vial original del traslado; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L73

```
73 |   @Column(nullable = true)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L74

```
74 |   Integer adjustedTransportSeconds;
```

Campos: adjustedTransportSeconds: duración ajustada y redondeada hacia arriba.

L76

```
76 |   @Column(nullable = true)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L77

```
77 |   Instant transportRouteObtainedAt;
```

Campos: transportRouteObtainedAt: fecha de ruta de traslado. Tipos y datos: transportRouteObtainedAt = fecha de ruta de traslado; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L79

```
79 |   @Column(nullable = true)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L80

```
80 |   Instant estimatedServiceEndAt;
```

Campos: estimatedServiceEndAt: fin planificado para controlar habilitación. Tipos y datos: estimatedServiceEndAt = fin planificado para controlar habilitación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L82

```
82 |   @Column(nullable = true)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L83

```
83 |   Integer rankingPosition;
```

Campos: rankingPosition: posición entre elegibles, comenzando en 1.

L84

```
84 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 7, 12, 14, 18, 21, 24, 27, 30, 33, 36, 39, 42, 45, 48, 51, 54, 57, 60, 63, 66, 69, 72, 75, 78, 81: separadores sin instrucciones.

CandidateEvaluationRepository (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/CandidateEvaluationRepository.java`

Objetivo y funcionalidad

Recupera candidatos asociados a una evaluación, métricas y posición de ranking. Expone los recursos considerados sin consultar directamente tablas de Fleet.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.data.jpa.repository.JpaRepository;
```

Importa tipos: org.springframework.data.jpa.repository.JpaRepository. Repositorios Spring Data y estructuras de paginación/orden. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | public interface CandidateEvaluationRepository extends JpaRepository<CandidateEvaluation, UUID> {
```

Declara interfaz CandidateEvaluationRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L7

```
7 |   List<CandidateEvaluation> findByEvaluationId(UUID id);
```

Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L8

```
8 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 5: separadores sin instrucciones.

Evaluation (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/Evaluation.java`

Objetivo y funcionalidad

Entidad del resultado de una evaluación, con distancias, tiempos, motivos y datos usados. No equivale a una asignación aceptada.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | @Entity
```

Marca entidad JPA. JPA administra instancias de esta clase como filas; se necesita @Id y un esquema compatible.

L9

```
9 | @Table(name = "evaluations")
```

Define tabla y restricciones de mapeo.

L10

```
10 | public class Evaluation {
```

Declara clase Evaluation. Entidad del resultado de una evaluación, con distancias, tiempos, motivos y datos usados. No equivale a una asignación aceptada.

L11

```
11 |   protected Evaluation() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L13

```
13 |   @Id UUID id = UUID.randomUUID();
```

Identifica clave primaria. Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro.

L15

```
15 |   @ManyToOne(fetch = FetchType.LAZY, optional = false)
```

Relaciona muchas filas con una entidad; LAZY difiere la carga.

L16

```
16 |   @JoinColumn(name = "request_id")
```

Nombra FK local.

L17

```
17 |   AssistanceRequest request;
```

Referencia Java a AssistanceRequest para request. Las anotaciones inmediatamente anteriores fijan la relación JPA y la columna de enlace; no comparte tablas entre servicios.

L19

```
19 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L20

```
20 |   int searchAttemptNumber;
```

Campos: searchAttemptNumber: número de búsqueda de 1 a 3. Tipos y datos: searchAttemptNumber = número de búsqueda de 1 a 3; int es entero de 32 bits para límites y contadores.

L22

```
22 |   @Column(nullable = false, precision = 9, scale = 6)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L23

```
23 |   BigDecimal originLatitude;
```

Campos: originLatitude: latitud de recogida. Tipos y datos: originLatitude = latitud de recogida; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L25

```
25 |   @Column(nullable = false, precision = 9, scale = 6)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L26

```
26 |   BigDecimal originLongitude;
```

Campos: originLongitude: longitud de recogida. Tipos y datos: originLongitude = longitud de recogida; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L28

```
28 |   @Column(nullable = false, precision = 9, scale = 6)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L29

```
29 |   BigDecimal destinationLatitude;
```

Campos: destinationLatitude: latitud de entrega. Tipos y datos: destinationLatitude = latitud de entrega; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L31

```
31 |   @Column(nullable = false, precision = 9, scale = 6)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L32

```
32 |   BigDecimal destinationLongitude;
```

Campos: destinationLongitude: longitud de entrega. Tipos y datos: destinationLongitude = longitud de entrega; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L34

```
34 |   @Column(name = "transport_distance_m", columnDefinition = "numeric")
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L35

```
35 |   BigDecimal transportDistanceM;
```

Campos: transportDistanceM: distancia vial origen-destino en metros. Tipos y datos: transportDistanceM = distancia vial origen-destino en metros; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L37

```
37 |   @Column(nullable = true, columnDefinition = "numeric")
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L38

```
38 |   BigDecimal transportDurationSeconds;
```

Campos: transportDurationSeconds: duración vial original del traslado. Tipos y datos: transportDurationSeconds = duración vial original del traslado; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L40

```
40 |   @Column(nullable = true)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L41

```
41 |   Integer adjustedTransportSeconds;
```

Campos: adjustedTransportSeconds: duración ajustada y redondeada hacia arriba.

L43

```
43 |   @Column(nullable = true)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L44

```
44 |   Instant routeObtainedAt;
```

Campos: routeObtainedAt: fecha del recorrido. Tipos y datos: routeObtainedAt = fecha del recorrido; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L46

```
46 |   @Column(nullable = false, length = 20)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L47

```
47 |   String vehicleType;
```

Campos: vehicleType: tipo CAR/SUV/PICKUP. Tipos y datos: vehicleType = tipo CAR/SUV/PICKUP; String es texto; no es por sí mismo una referencia de entidad.

L49

```
49 |   @Column(nullable = false, precision = 9, scale = 2)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L50

```
50 |   BigDecimal declaredWeightKg;
```

Campos: declaredWeightKg: peso declarado en kg. Tipos y datos: declaredWeightKg = peso declarado en kg; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L52

```
52 |   @Column(nullable = false, length = 20)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L53

```
53 |   String mobilityCondition;
```

Campos: mobilityCondition: condición ROLLABLE/NON_ROLLABLE. Tipos y datos: mobilityCondition = condición ROLLABLE/NON_ROLLABLE; String es texto; no es por sí mismo una referencia de entidad.

L55

```
55 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L56

```
56 |   int maximumArrivalSeconds;
```

Campos: maximumArrivalSeconds: presupuesto máximo hasta llegada. Tipos y datos: maximumArrivalSeconds = presupuesto máximo hasta llegada; int es entero de 32 bits para límites y contadores.

L58

```
58 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L59

```
59 |   int loadingSeconds;
```

Campos: loadingSeconds: carga prevista. Tipos y datos: loadingSeconds = carga prevista; int es entero de 32 bits para límites y contadores.

L61

```
61 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L62

```
62 |   int unloadingSeconds;
```

Campos: unloadingSeconds: descarga prevista. Tipos y datos: unloadingSeconds = descarga prevista; int es entero de 32 bits para límites y contadores.

L64

```
64 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L65

```
65 |   int operationalMarginSeconds;
```

Campos: operationalMarginSeconds: contingencia de planificación. Tipos y datos: operationalMarginSeconds = contingencia de planificación; int es entero de 32 bits para límites y contadores.

L67

```
67 |   @Column(nullable = false, length = 30)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L68

```
68 |   String configurationRevision;
```

Campos: configurationRevision: versión de reglas. Tipos y datos: configurationRevision = versión de reglas; String es texto; no es por sí mismo una referencia de entidad.

L70

```
70 |   @Column(nullable = false, columnDefinition = "text")
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L71

```
71 |   String coverageAndAdmissionSummary;
```

Campos: coverageAndAdmissionSummary: resumen de cobertura y admisión. Tipos y datos: coverageAndAdmissionSummary = resumen de cobertura y admisión; String es texto; no es por sí mismo una referencia de entidad.

L73

```
73 |   @Column(nullable = false, length = 30)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L74

```
74 |   String result;
```

Campos: result: resultado de evaluación, distinto del estado de solicitud. Tipos y datos: result = resultado de evaluación, distinto del estado de solicitud; String es texto; no es por sí mismo una referencia de entidad.

L76

```
76 |   @Column(nullable = true, length = 100)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L77

```
77 |   String reasonCode;
```

Campos: reasonCode: causa codificada. Tipos y datos: reasonCode = causa codificada; String es texto; no es por sí mismo una referencia de entidad.

L79

```
79 |   @Column(nullable = true, columnDefinition = "text")
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L80

```
80 |   String reasonDetail;
```

Campos: reasonDetail: detalle diagnóstico acotado. Tipos y datos: reasonDetail = detalle diagnóstico acotado; String es texto; no es por sí mismo una referencia de entidad.

L82

```
82 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L83

```
83 |   Instant evaluatedAt;
```

Campos: evaluatedAt: instante de evaluación. Tipos y datos: evaluatedAt = instante de evaluación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L84

```
84 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 7, 12, 14, 18, 21, 24, 27, 30, 33, 36, 39, 42, 45, 48, 51, 54, 57, 60, 63, 66, 69, 72, 75, 78, 81: separadores sin instrucciones.

EvaluationRepository (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/EvaluationRepository.java`

Objetivo y funcionalidad

Localiza evaluaciones persistidas, incluida la más reciente. Conserva evidencia separada del estado actual para justificar por qué se aprobó o rechazó una evaluación.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.data.jpa.repository.JpaRepository;
```

Importa tipos: org.springframework.data.jpa.repository.JpaRepository. Repositorios Spring Data y estructuras de paginación/orden. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | public interface EvaluationRepository extends JpaRepository<Evaluation, UUID> {
```

Declara interfaz EvaluationRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L7

```
7 |   Optional<Evaluation> findFirstByRequestIdOrderByEvaluatedAtDescIdDesc(UUID id);
```

Recupera evaluación más reciente por fecha e ID. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L8

```
8 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 5: separadores sin instrucciones.

HistoryRepository (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/HistoryRepository.java`

Objetivo y funcionalidad

Recupera eventos de solicitud y acciones registradas. Permite presentar una cronología y reconocer la repetición de una operación sin volver a cambiar el estado.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.data.jpa.repository.JpaRepository;
```

Importa tipos: org.springframework.data.jpa.repository.JpaRepository. Repositorios Spring Data y estructuras de paginación/orden. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.data.domain.*;
```

Importa tipos: org.springframework.data.domain.*. Repositorios Spring Data y estructuras de paginación/orden. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | public interface HistoryRepository extends JpaRepository<RequestStatusHistory, UUID> {
```

Declara interfaz HistoryRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L8

```
8 |   Page<RequestStatusHistory> findByRequestId(UUID id, Pageable page);
```

Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; page = page; Pageable es número, tamaño y orden de una página SQL.

L10

```
10 |   Optional<RequestStatusHistory> findByRequestIdAndOperationKey(UUID id, String key);
```

Busca resultado anterior para repetir sin duplicar. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; key = key; String es texto; no es por sí mismo una referencia de entidad.

L11

```
11 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 6, 9: separadores sin instrucciones.

RequestRepository (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/RequestRepository.java`

Objetivo y funcionalidad

Lee solicitudes, busca operaciones idempotentes y bloquea la solicitud cuando varias acciones compiten. Alimenta cancelación, evaluación, propuestas y ejecución en assistance_db.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.data.jpa.repository.*;
```

Importa tipos: org.springframework.data.jpa.repository.*. Repositorios Spring Data y estructuras de paginación/orden. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.data.domain.*;
```

Importa tipos: org.springframework.data.domain.*. Repositorios Spring Data y estructuras de paginación/orden. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import jakarta.persistence.LockModeType;
```

Importa tipos: jakarta.persistence.LockModeType. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | public interface RequestRepository extends JpaRepository<AssistanceRequest, UUID> {
```

Declara interfaz RequestRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L9

```
9 |   @Query("select r.vehicle.id from AssistanceRequest r where r.id=:id")
```

Consulta AssistanceRequest con el filtro y proyección escritos. Cambios: r.id recibe :id").

L10

```
10 |   Optional<UUID> vehicleId(UUID id);
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L12

```
12 |   Optional<AssistanceRequest> findByDriverSubjectAndIdempotencyKey(String subject, String key);
```

Busca resultado anterior para repetir sin duplicar. Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad; key = key; String es texto; no es por sí mismo una referencia de entidad.

L14

```
14 |   @Lock(LockModeType.PESSIMISTIC_WRITE)
```

Adquiere bloqueo pesimista.

L15

```
15 |   @Query("select r from AssistanceRequest r where r.id=:id")
```

Consulta AssistanceRequest con el filtro y proyección escritos. Cambios: r.id recibe :id").

L16

```
16 |   Optional<AssistanceRequest> lockById(UUID id);
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L18 a L20

```
18 |   @Query(
19 |       "select count(r)>0 from AssistanceRequest r where r.vehicle.id=:id and (r.closedAt is null
     or r.resourceReleasePending=true)")
20 |   boolean hasActive(UUID id);
```

Consulta AssistanceRequest contando coincidencias. Campos: id: identificador UUID del registro. Cambios: r.resourceReleasePending recibe true)") boolean hasActive(UUID id).

L20: Dato del contrato: id: identificador UUID del registro.

L22 a L24

```
22 |   @Query(
23 |       "select r from AssistanceRequest r where (:subject is null or r.driverSubject=:subject) and
      (:status is null or r.status=:status)")
24 |   Page<AssistanceRequest> visible(String subject, DomainTypes.Status status, Pageable page);
```

Consulta AssistanceRequest con el filtro y proyección escritos. Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L26 a L32

```
26 |   @Query(
27 |       value =
28 |           "select r.* from assistance_requests r where exists(select 1 from assignments a where
     a.request_id=r.id and a.operator_subject=:subject) and (:status is null or r.status=:status)",
29 |       countQuery =
30 |           "select count(*) from assistance_requests r where exists(select 1 from assignments a
     where a.request_id=r.id and a.operator_subject=:subject) and (:status is null or
     r.status=:status)",
31 |       nativeQuery = true)
32 |   Page<AssistanceRequest> forOperator(String subject, String status, Pageable page);
```

Consulta assistance_requests, assignments con el filtro y proyección escritos. Consulta assistance_requests, assignments contando coincidencias. Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L33

```
33 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 7, 11, 13, 17, 21, 25: separadores sin instrucciones.

RequestStatusHistory (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/RequestStatusHistory.java`

Objetivo y funcionalidad

Auditoría de cambios de solicitud con actor, motivo, fecha, idempotencia y evidencia. Es persistencia de negocio, distinta de los logs de consola.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | @Entity
```

Marca entidad JPA. JPA administra instancias de esta clase como filas; se necesita @Id y un esquema compatible.

L10 a L13

```
10 | @Table(
11 |     name = "request_status_history",
12 |     uniqueConstraints = @UniqueConstraint(columnNames = {"request_id", "operation_key"}))
13 | public class RequestStatusHistory {
```

Define tabla y restricciones de mapeo. Declara clase RequestStatusHistory. Auditoría de cambios de solicitud con actor, motivo, fecha, idempotencia y evidencia. Es persistencia de negocio, distinta de los logs de consola.

L14

```
14 |   protected RequestStatusHistory() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L16

```
16 |   @Id UUID id = UUID.randomUUID();
```

Identifica clave primaria. Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro.

L18

```
18 |   @ManyToOne(fetch = FetchType.LAZY, optional = false)
```

Relaciona muchas filas con una entidad; LAZY difiere la carga.

L19

```
19 |   @JoinColumn(name = "request_id")
```

Nombra FK local.

L20

```
20 |   AssistanceRequest request;
```

Referencia Java a AssistanceRequest para request. Las anotaciones inmediatamente anteriores fijan la relación JPA y la columna de enlace; no comparte tablas entre servicios.

L22

```
22 |   @Enumerated(EnumType.STRING)
```

Guarda nombre del enum, no posición ordinal.

L23

```
23 |   @Column(length = 30)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L24

```
24 |   Status previousStatus;
```

Campos: previousStatus: estado previo.

L26

```
26 |   @Enumerated(EnumType.STRING)
```

Guarda nombre del enum, no posición ordinal.

L27

```
27 |   @Column(nullable = false, length = 30)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L28

```
28 |   Status newStatus;
```

Campos: newStatus: estado resultante.

L30

```
30 |   @Column(length = 100)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L31

```
31 |   String actorSubject;
```

Campos: actorSubject: sub del usuario que produjo el evento. Tipos y datos: actorSubject = sub del usuario que produjo el evento; String es texto; no es por sí mismo una referencia de entidad.

L33

```
33 |   @Column(nullable = false, length = 20)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L34

```
34 |   String actorType;
```

Campos: actorType: tipo de actor SYSTEM/DRIVER/OPERATOR. Tipos y datos: actorType = tipo de actor SYSTEM/DRIVER/OPERATOR; String es texto; no es por sí mismo una referencia de entidad.

L36

```
36 |   @Column(columnDefinition = "text")
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L37

```
37 |   String reason;
```

Campos: reason: motivo legible. Tipos y datos: reason = motivo legible; String es texto; no es por sí mismo una referencia de entidad.

L39

```
39 |   @Column(columnDefinition = "text")
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L40

```
40 |   String details;
```

Campos: details: evidencia adicional. Tipos y datos: details = evidencia adicional; String es texto; no es por sí mismo una referencia de entidad.

L42

```
42 |   @Column(length = 100)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L43

```
43 |   String operationKey;
```

Campos: operationKey: clave durable de acción. Tipos y datos: operationKey = clave durable de acción; String es texto; no es por sí mismo una referencia de entidad.

L45

```
45 |   @Column(length = 64)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L46

```
46 |   String requestFingerprint;
```

Campos: requestFingerprint: SHA-256 del contenido asociado a la clave. Tipos y datos: requestFingerprint = SHA-256 del contenido asociado a la clave; String es texto; no es por sí mismo una referencia de entidad.

L48

```
48 |   @Column(length = 100)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L49

```
49 |   String correlationId;
```

Campos: correlationId: identificador opcional de correlación; su campo no implica generación automática. Tipos y datos: correlationId = identificador opcional de correlación; su campo no implica generación automática; String es texto; no es por sí mismo una referencia de entidad.

L51

```
51 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L52

```
52 |   Instant occurredAt;
```

Campos: occurredAt: instante del evento. Tipos y datos: occurredAt = instante del evento; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L53

```
53 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 8, 15, 17, 21, 25, 29, 32, 35, 38, 41, 44, 47, 50: separadores sin instrucciones.

Vehicle (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/Vehicle.java`

Objetivo y funcionalidad

Entidad de vehículo del conductor. ownerSubject enlaza con sub de Keycloak sin FK entre bases; el repositorio filtra por propietario.

L1

```
 1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | @Entity
```

Marca entidad JPA. JPA administra instancias de esta clase como filas; se necesita @Id y un esquema compatible.

L10

```
10 | @Table(name = "vehicles")
```

Define tabla y restricciones de mapeo.

L11

```
11 | public class Vehicle {
```

Declara clase Vehicle. Entidad de vehículo del conductor. ownerSubject enlaza con sub de Keycloak sin FK entre bases; el repositorio filtra por propietario.

L12

```
12 |   protected Vehicle() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L14

```
14 |   @Id UUID id = UUID.randomUUID();
```

Identifica clave primaria. Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro.

L15

```
15 |   @Version long version;
```

Activa concurrencia optimista mediante comparación/incremento de versión. Campos: version: versión para control optimista de concurrencia. Hibernate agrega la versión a la condición del UPDATE; si otra transacción la cambió, detecta conflicto en vez de pisar datos. Tipos y datos: version = versión para control optimista de concurrencia; long es entero de 64 bits para cantidades y duraciones mayores.

L17

```
17 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L18

```
18 |   Instant createdAt;
```

Campos: createdAt: fecha de creación. Tipos y datos: createdAt = fecha de creación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L20

```
20 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L21

```
21 |   Instant updatedAt;
```

Campos: updatedAt: fecha de modificación. Tipos y datos: updatedAt = fecha de modificación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L23

```
23 |   public UUID getId() {
```

L24

```
24 |     return id;
```

Entrega id al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: identificador UUID del registro.

L25

```
25 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L27

```
27 |   @Column(nullable = false, length = 100)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L28

```
28 |   String ownerSubject;
```

Campos: ownerSubject: sub de Keycloak del propietario. Tipos y datos: ownerSubject = sub de Keycloak del propietario; String es texto; no es por sí mismo una referencia de entidad.

L30

```
30 |   @Column(nullable = false, unique = true, length = 10)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L31

```
31 |   String licensePlate;
```

Campos: licensePlate: patente normalizada y única. Tipos y datos: licensePlate = patente normalizada y única; String es texto; no es por sí mismo una referencia de entidad.

L33

```
33 |   @Column(length = 100)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L34

```
34 |   String brand;
```

Campos: brand: marca declarada. Tipos y datos: brand = marca declarada; String es texto; no es por sí mismo una referencia de entidad.

L36

```
36 |   @Column(length = 100)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L37

```
37 |   String model;
```

Campos: model: modelo declarado. Tipos y datos: model = modelo declarado; String es texto; no es por sí mismo una referencia de entidad.

L39

```
39 |   @Enumerated(EnumType.STRING)
```

Guarda nombre del enum, no posición ordinal.

L40

```
40 |   @Column(nullable = false, length = 20)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L41

```
41 |   VehicleType vehicleType;
```

Campos: vehicleType: tipo CAR/SUV/PICKUP.

L43

```
43 |   @Column(nullable = false, precision = 9, scale = 2)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L44

```
44 |   BigDecimal declaredWeightKg;
```

Campos: declaredWeightKg: peso declarado en kg. Tipos y datos: declaredWeightKg = peso declarado en kg; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L46

```
46 |   @Enumerated(EnumType.STRING)
```

Guarda nombre del enum, no posición ordinal.

L47

```
47 |   @Column(nullable = false, length = 20)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L48

```
48 |   Registration registrationStatus = Registration.ACTIVE;
```

Campos: registrationStatus: alta administrativa ACTIVE/INACTIVE. Se calcula registrationStatus (alta administrativa ACTIVE/INACTIVE) para reutilizar ese resultado en los pasos siguientes. ACTIVE: registro habilitado administrativamente

L49

```
49 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 8, 13, 16, 19, 22, 26, 29, 32, 35, 38, 42, 45: separadores sin instrucciones.

VehicleRepository (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/VehicleRepository.java

Objetivo y funcionalidad

Busca vehículos por propietario e identificador, y ofrece bloqueo para serializar solicitudes y edición del mismo vehículo. Conocer un UUID no debe permitir leer un vehículo ajeno.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.data.jpa.repository.*;
```

Importa tipos: org.springframework.data.jpa.repository.*. Repositorios Spring Data y estructuras de paginación/orden. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.data.domain.*;
```

Importa tipos: org.springframework.data.domain.*. Repositorios Spring Data y estructuras de paginación/orden. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import jakarta.persistence.LockModeType;
```

Importa tipos: jakarta.persistence.LockModeType. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | public interface VehicleRepository extends JpaRepository<Vehicle, UUID> {
```

Declara interfaz VehicleRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L9

```
9 |   Optional<Vehicle> findByIdAndOwnerSubject(UUID id, String subject);
```

Filtra por propietario en la consulta misma. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; subject = subject; String es texto; no es por sí mismo una referencia de entidad.

L11

```
11 |   @Lock(LockModeType.PESSIMISTIC_WRITE)
```

Adquiere bloqueo pesimista.

L12

```
12 |   @Query("select v from Vehicle v where v.id=:id and v.ownerSubject=:subject")
```

Consulta Vehicle con el filtro y proyección escritos.

L13

```
13 |   Optional<Vehicle> lockOwned(UUID id, String subject);
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; subject = subject; String es texto; no es por sí mismo una referencia de entidad.

L15

```
15 |   Page<Vehicle> findByOwnerSubject(String subject, Pageable page);
```

Filtra por propietario en la consulta misma. Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad; page = page; Pageable es número, tamaño y orden de una página SQL.

L17 a L18

```
17 |   Page<Vehicle> findByOwnerSubjectAndRegistrationStatus(
18 |       String subject, DomainTypes.Registration status, Pageable page);
```

Filtra por propietario en la consulta misma. Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad; page = page; Pageable es número, tamaño y orden de una página SQL.

L18: Dato del contrato: status: estado del objeto; su enum o CHECK define valores permitidos.

L19

```
19 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 7, 10, 14, 16: separadores sin instrucciones.

001-assistance (assistance-service)

`Ruta: assistance-service/src/main/resources/db/001-assistance.sql`

Objetivo y funcionalidad

Crea vehículos, solicitudes y eventos. Guarda propiedad por sub, snapshots e idempotencia y restringe solicitudes activas del mismo vehículo.

L1 a L13

```
 1 | CREATE TABLE IF NOT EXISTS vehicles (
 2 |   id UUID PRIMARY KEY,
 3 |   owner_subject VARCHAR(100) NOT NULL,
 4 |   license_plate VARCHAR(10) NOT NULL UNIQUE CHECK(license_plate ~ '^[A-Z0-9]{6,10}$'),
 5 |   brand VARCHAR(100),
 6 |   model VARCHAR(100),
 7 |   vehicle_type VARCHAR(20) NOT NULL CHECK(vehicle_type IN ('CAR',  'SUV',  'PICKUP')),
 8 |   declared_weight_kg NUMERIC(9,  2) NOT NULL CHECK(declared_weight_kg BETWEEN 500 AND 3000),
 9 |   registration_status VARCHAR(20) NOT NULL CHECK(registration_status IN ('ACTIVE',  'INACTIVE')),
10 |   created_at TIMESTAMPTZ NOT NULL,
11 |   updated_at TIMESTAMPTZ NOT NULL,
12 |   version BIGINT NOT NULL
13 | );
```

Crea vehicles solo si la tabla no existe. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. La unicidad impide repetir la combinación indicada. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: id, owner_subject, license_plate, brand, model, vehicle_type, declared_weight_kg, registration_status, created_at, updated_at, version. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro. La columna owner_subject conserva sub de Keycloak del propietario. La columna license_plate conserva patente normalizada y única. La columna brand conserva marca declarada. La columna model conserva modelo declarado. La columna vehicle_type conserva tipo CAR/SUV/PICKUP. La columna declared_weight_kg conserva peso declarado en kg. La columna registration_status conserva alta administrativa ACTIVE/INACTIVE. La columna created_at conserva fecha de creación. La columna updated_at conserva fecha de modificación. La columna version conserva versión para control optimista de concurrencia. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L2: PRIMARY KEY impide ID nulo o repetido. Columnas: id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro.

L3: NOT NULL exige valor. Columnas: owner_subject. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna owner_subject conserva sub de Keycloak del propietario.

L4: NOT NULL exige valor. La unicidad impide repetir la combinación indicada. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: license_plate. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

La columna license_plate conserva patente normalizada y única. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes.

L5: Columnas: brand. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna brand conserva marca declarada.

L6: Columnas: model. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna model conserva modelo declarado.

L7: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: vehicle_type. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna vehicle_type conserva tipo CAR/SUV/PICKUP. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L8: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: declared_weight_kg. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna declared_weight_kg conserva peso declarado en kg. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L9: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: registration_status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna registration_status conserva alta administrativa ACTIVE/INACTIVE. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L10: NOT NULL exige valor. Columnas: created_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna created_at conserva fecha de creación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L11: NOT NULL exige valor. Columnas: updated_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna updated_at conserva fecha de modificación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L12: NOT NULL exige valor. Columnas: version. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna version conserva versión para control optimista de concurrencia.

L15 a L45

```
15 | CREATE TABLE IF NOT EXISTS assistance_requests (
16 |   id UUID PRIMARY KEY,
17 |   vehicle_id UUID NOT NULL REFERENCES vehicles(id),
18 |   driver_subject VARCHAR(100) NOT NULL,
19 |   status VARCHAR(30) NOT NULL CHECK(status IN ('REQUESTED',  'PENDING_EVALUATION',
     'AWAITING_ACCEPTANCE',  'ASSIGNED',  'SEARCHING_ALTERNATIVE',  'DISPATCHED',  'ARRIVED',
     'VEHICLE_LOADED',  'IN_TRANSIT',  'COMPLETED',  'REJECTED',  'UNFULFILLED',  'CANCELLED')),
20 |   first_assigned_at TIMESTAMPTZ,
21 |   closure_reason_code VARCHAR(100),
22 |   origin_latitude NUMERIC(9,  6) NOT NULL CHECK(origin_latitude BETWEEN -90 AND 90),
23 |   origin_longitude NUMERIC(9,  6) NOT NULL CHECK(origin_longitude BETWEEN -180 AND 180),
24 |   destination_latitude NUMERIC(9,  6) NOT NULL CHECK(destination_latitude BETWEEN -90 AND 90),
25 |   destination_longitude NUMERIC(9,  6) NOT NULL CHECK(destination_longitude BETWEEN -180 AND
     180),
26 |   declared_vehicle_type VARCHAR(20) NOT NULL CHECK(declared_vehicle_type IN ('CAR',  'SUV',
     'PICKUP')),
27 |   declared_weight_kg NUMERIC(9,  2) NOT NULL CHECK(declared_weight_kg BETWEEN 500 AND 3000),
28 |   mobility_condition VARCHAR(20) NOT NULL CHECK(mobility_condition IN ('ROLLABLE',
     'NON_ROLLABLE')),
29 |   description VARCHAR(1000) NOT NULL CHECK(length(trim(description))>0),
30 |   search_attempt_number INTEGER NOT NULL CHECK(search_attempt_number BETWEEN 1 AND 3),
31 |   search_deadline TIMESTAMPTZ NOT NULL,
```

```
32 |   configuration_revision VARCHAR(30) NOT NULL,
33 |   configuration_snapshot TEXT NOT NULL,
34 |   idempotency_key VARCHAR(100) NOT NULL CHECK(length(trim(idempotency_key))>0),
35 |   request_fingerprint VARCHAR(64) NOT NULL,
36 |   created_at TIMESTAMPTZ NOT NULL,
37 |   updated_at TIMESTAMPTZ NOT NULL,
38 |   closed_at TIMESTAMPTZ,
39 |   version BIGINT NOT NULL,
40 |   UNIQUE(driver_subject,  idempotency_key),
41 |   CHECK(origin_latitude<>destination_latitude OR origin_longitude<>destination_longitude),
42 |   CHECK(search_deadline>created_at),
43 |   CHECK((closed_at IS NOT NULL)=(status IN ('COMPLETED',  'REJECTED',  'UNFULFILLED',
     'CANCELLED'))),
44 |   CHECK(status<>'CANCELLED' OR first_assigned_at IS NULL)
45 | );
```

Crea assistance_requests solo si la tabla no existe. La FK exige un registro relacionado en esta misma base; no cruza microservicios. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. La unicidad impide repetir la combinación indicada. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: id, vehicle_id, driver_subject, status, first_assigned_at, closure_reason_code, origin_latitude, origin_longitude, destination_latitude, destination_longitude, declared_vehicle_type, declared_weight_kg, mobility_condition, description, search_attempt_number, search_deadline, configuration_revision, configuration_snapshot, idempotency_key, request_fingerprint, created_at, updated_at, closed_at, version. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro. La columna driver_subject conserva sub del conductor que creó la solicitud. La columna status conserva estado del objeto; su enum o CHECK define valores permitidos. La columna first_assigned_at conserva primera aceptación que impide cancelar incluso durante reemplazos. La columna closure_reason_code conserva causa codificada del cierre. La columna origin_latitude conserva latitud de recogida. La columna origin_longitude conserva longitud de recogida. La columna destination_latitude conserva latitud de entrega. La columna destination_longitude conserva longitud de entrega. La columna declared_vehicle_type conserva copia del tipo de vehículo al solicitar. La columna declared_weight_kg conserva peso declarado en kg. La columna mobility_condition conserva condición ROLLABLE/NON_ROLLABLE. La columna description conserva descripción de asistencia. La columna search_attempt_number conserva número de búsqueda de 1 a 3. La columna search_deadline conserva vencimiento del intento. La columna configuration_revision conserva versión de reglas. La columna configuration_snapshot conserva JSON de parámetros vigente al crear. La columna idempotency_key conserva clave para repetir sin duplicar. La columna request_fingerprint conserva SHA-256 del contenido asociado a la clave. La columna created_at conserva fecha de creación. La columna updated_at conserva fecha de modificación. La columna closed_at conserva fecha de cierre; null indica abierto. La columna version conserva versión para control optimista de concurrencia. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L16: PRIMARY KEY impide ID nulo o repetido. Columnas: id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro.

L17: La FK exige un registro relacionado en esta misma base; no cruza microservicios. NOT NULL exige valor. Columnas: vehicle_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L18: NOT NULL exige valor. Columnas: driver_subject. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna driver_subject conserva sub del conductor que creó la solicitud.

L19: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna status conserva estado del objeto; su enum o CHECK define valores permitidos. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L20: Columnas: first_assigned_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna first_assigned_at conserva primera aceptación que impide cancelar incluso durante reemplazos. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L21: Columnas: closure_reason_code. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna closure_reason_code conserva causa codificada del cierre.

L22: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: origin_latitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna origin_latitude conserva latitud de recogida. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L23: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: origin_longitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna origin_longitude

conserva longitud de recogida. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L24: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: destination_latitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna destination_latitude conserva latitud de entrega. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L25: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: destination_longitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna destination_longitude conserva longitud de entrega. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L26: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: declared_vehicle_type. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna declared_vehicle_type conserva copia del tipo de vehículo al solicitar. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L27: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: declared_weight_kg. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna declared_weight_kg conserva peso declarado en kg. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L28: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: mobility_condition. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna mobility_condition conserva condición ROLLABLE/NON_ROLLABLE. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L29: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: description. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna description conserva descripción de asistencia. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L30: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: search_attempt_number. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna search_attempt_number conserva número de búsqueda de 1 a 3. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L31: NOT NULL exige valor. Columnas: search_deadline. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna search_deadline conserva vencimiento del intento. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L32: NOT NULL exige valor. Columnas: configuration_revision. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna configuration_revision conserva versión de reglas.

L33: NOT NULL exige valor. Columnas: configuration_snapshot. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna configuration_snapshot conserva JSON de parámetros vigente al crear.

L34: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: idempotency_key. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna idempotency_key conserva clave para repetir sin duplicar. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L35: NOT NULL exige valor. Columnas: request_fingerprint. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna request_fingerprint conserva SHA-256 del contenido asociado a la clave.

L36: NOT NULL exige valor. Columnas: created_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna created_at conserva fecha de creación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L37: NOT NULL exige valor. Columnas: updated_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna updated_at conserva fecha de modificación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L38: Columnas: closed_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna closed_at conserva fecha de cierre; null indica abierto. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L39: NOT NULL exige valor. Columnas: version. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna version conserva versión para control optimista de concurrencia.

L40: La unicidad impide repetir la combinación indicada. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes.

L41: CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L42: CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L43: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L44: CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L47

```
47 | CREATE UNIQUE INDEX IF NOT EXISTS ux_requests_active_vehicle ON assistance_requests(vehicle_id)
     WHERE closed_at IS NULL;
```

Índice único ux_requests_active_vehicle sobre assistance_requests. La unicidad impide repetir la combinación indicada. El índice es parcial: solo restringe registros de los estados indicados; permite conservar historial cerrado. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes. El predicado WHERE delimita las filas que participan: las filas históricas que no cumplen ese estado quedan fuera de la exclusividad.

L49

```
49 | CREATE INDEX IF NOT EXISTS ix_requests_driver_status ON assistance_requests(driver_subject,
     status,  created_at,  id);
```

Índice de búsqueda ix_requests_driver_status sobre assistance_requests.

L51

```
51 | CREATE INDEX IF NOT EXISTS ix_vehicle_owner ON vehicles(owner_subject,  created_at,  id);
```

Índice de búsqueda ix_vehicle_owner sobre vehicles.

L53 a L67

```
53 | CREATE TABLE IF NOT EXISTS request_status_history (
54 |   id UUID PRIMARY KEY,
55 |   request_id UUID NOT NULL REFERENCES assistance_requests(id),
56 |   previous_status VARCHAR(30) CHECK(previous_status IN ('REQUESTED',  'PENDING_EVALUATION',
     'AWAITING_ACCEPTANCE',  'ASSIGNED',  'SEARCHING_ALTERNATIVE',  'DISPATCHED',  'ARRIVED',
     'VEHICLE_LOADED',  'IN_TRANSIT',  'COMPLETED',  'REJECTED',  'UNFULFILLED',  'CANCELLED')),
57 |   new_status VARCHAR(30) NOT NULL CHECK(new_status IN ('REQUESTED',  'PENDING_EVALUATION',
     'AWAITING_ACCEPTANCE',  'ASSIGNED',  'SEARCHING_ALTERNATIVE',  'DISPATCHED',  'ARRIVED',
     'VEHICLE_LOADED',  'IN_TRANSIT',  'COMPLETED',  'REJECTED',  'UNFULFILLED',  'CANCELLED')),
58 |   actor_subject VARCHAR(100),
59 |   actor_type VARCHAR(20) NOT NULL,
60 |   reason TEXT,
61 |   details TEXT,
62 |   operation_key VARCHAR(100),
63 |   request_fingerprint VARCHAR(64),
64 |   correlation_id VARCHAR(100),
65 |   occurred_at TIMESTAMPTZ NOT NULL,
66 |   UNIQUE(request_id,  operation_key)
67 | );
```

Crea request_status_history solo si la tabla no existe. La FK exige un registro relacionado en esta misma base; no cruza microservicios. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. La unicidad impide repetir la combinación indicada. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: id, request_id, previous_status, new_status, actor_subject, actor_type, reason, details, operation_key, request_fingerprint, correlation_id, occurred_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro. La columna request_id conserva UUID de solicitud. La columna previous_status conserva estado previo. La columna new_status conserva estado resultante. La columna actor_subject conserva sub del usuario que produjo el evento. La columna actor_type conserva tipo de actor SYSTEM/DRIVER/OPERATOR. La columna reason conserva motivo legible. La columna details conserva evidencia adicional. La columna operation_key conserva clave durable de acción. La columna request_fingerprint conserva SHA-256 del contenido asociado a la clave. La columna correlation_id conserva identificador opcional de correlación; su campo no implica generación automática. La columna occurred_at conserva instante del evento. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L54: PRIMARY KEY impide ID nulo o repetido. Columnas: id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro.

L55: La FK exige un registro relacionado en esta misma base; no cruza microservicios. NOT NULL exige valor. Columnas: request_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna request_id conserva UUID de solicitud.

L56: CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: previous_status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna previous_status conserva estado previo. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L57: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: new_status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna new_status conserva estado resultante. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L58: Columnas: actor_subject. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna actor_subject conserva sub del usuario que produjo el evento.

L59: NOT NULL exige valor. Columnas: actor_type. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna actor_type conserva tipo de actor SYSTEM/DRIVER/OPERATOR.

L60: Columnas: reason. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna reason conserva motivo legible.

L61: Columnas: details. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna details conserva evidencia adicional.

L62: Columnas: operation_key. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna operation_key conserva clave durable de acción.

L63: Columnas: request_fingerprint. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna request_fingerprint conserva SHA-256 del contenido asociado a la clave.

L64: Columnas: correlation_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna correlation_id conserva identificador opcional de correlación; su campo no implica generación automática.

L65: NOT NULL exige valor. Columnas: occurred_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna occurred_at conserva instante del evento. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L66: La unicidad impide repetir la combinación indicada. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes.

L69

```
69 | CREATE INDEX IF NOT EXISTS ix_request_history ON request_status_history(request_id,  occurred_at,
       id);
```

Índice de búsqueda ix_request_history sobre request_status_history.

Líneas vacías 14, 46, 48, 50, 52, 68: separadores sin instrucciones.

002-evaluation (assistance-service)

`Ruta: assistance-service/src/main/resources/db/002-evaluation.sql`

Objetivo y funcionalidad

Añade evaluaciones y candidatos con distancias, tiempos y motivos. Conserva también el presupuesto de llamadas externas, evitando que reiniciar Assistance reinicie la cuota ORS.

L1 a L26

```
CREATE TABLE IF NOT EXISTS evaluations (
  id UUID PRIMARY KEY,
  request_id UUID NOT NULL REFERENCES assistance_requests(id),
  search_attempt_number integer NOT NULL,
  origin_latitude numeric(9,  6) NOT NULL,
  origin_longitude numeric(9,  6) NOT NULL,
  destination_latitude numeric(9,  6) NOT NULL,
  destination_longitude numeric(9,  6) NOT NULL,
  transport_distance_m numeric,
  transport_duration_seconds numeric,
  adjusted_transport_seconds integer,
  route_obtained_at timestamptz,
  vehicle_type varchar(20) NOT NULL,
  declared_weight_kg numeric(9,  2) NOT NULL,
  mobility_condition varchar(20) NOT NULL,
  maximum_arrival_seconds integer NOT NULL,
  loading_seconds integer NOT NULL,
  unloading_seconds integer NOT NULL,
  operational_margin_seconds integer NOT NULL,
  configuration_revision varchar(30) NOT NULL,
  coverage_and_admission_summary text NOT NULL,
  result varchar(30) NOT NULL,
  reason_code varchar(100),
  reason_detail text,
  evaluated_at timestamptz NOT NULL
);
```

Crea evaluations solo si la tabla no existe. La FK exige un registro relacionado en esta misma base; no cruza microservicios. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. Columnas: id, request_id, search_attempt_number, origin_latitude, origin_longitude, destination_latitude, destination_longitude, transport_distance_m, transport_duration_seconds, adjusted_transport_seconds, route_obtained_at, vehicle_type, declared_weight_kg, mobility_condition, maximum_arrival_seconds, loading_seconds, unloading_seconds, operational_margin_seconds, configuration_revision, coverage_and_admission_summary, result, reason_code, reason_detail, evaluated_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro. La columna request_id conserva UUID de solicitud. La columna search_attempt_number conserva número de búsqueda de 1 a 3. La columna origin_latitude conserva latitud de recogida. La columna origin_longitude conserva longitud de recogida. La columna destination_latitude conserva latitud de entrega. La columna destination_longitude conserva longitud de entrega. La columna transport_distance_m conserva distancia vial origen-destino en metros. La columna transport_duration_seconds conserva duración vial original del traslado. La columna adjusted_transport_seconds conserva duración ajustada y redondeada hacia arriba. La columna route_obtained_at conserva fecha del recorrido. La columna vehicle_type conserva tipo CAR/SUV/PICKUP. La columna declared_weight_kg conserva peso declarado en kg. La columna mobility_condition conserva condición ROLLABLE/NON_ROLLABLE. La columna maximum_arrival_seconds conserva presupuesto máximo hasta llegada. La columna loading_seconds conserva carga prevista. La columna unloading_seconds conserva descarga prevista. La columna operational_margin_seconds conserva contingencia de planificación. La columna configuration_revision conserva versión de reglas. La columna coverage_and_admission_summary conserva resumen de cobertura y admisión. La columna result conserva resultado de evaluación, distinto del estado de solicitud. La columna reason_code conserva causa codificada. La columna reason_detail conserva detalle diagnóstico acotado. La columna evaluated_at conserva instante de evaluación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L2: PRIMARY KEY impide ID nulo o repetido. Columnas: id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro.

L3: La FK exige un registro relacionado en esta misma base; no cruza microservicios. NOT NULL exige valor. Columnas: request_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna request_id conserva UUID de solicitud.

L4: NOT NULL exige valor. Columnas: search_attempt_number. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna search_attempt_number conserva número de búsqueda de 1 a 3.

L5: NOT NULL exige valor. Columnas: origin_latitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna origin_latitude conserva latitud de recogida.

L6: NOT NULL exige valor. Columnas: origin_longitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna origin_longitude conserva longitud de recogida.

L7: NOT NULL exige valor. Columnas: destination_latitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna destination_latitude conserva latitud de entrega.

L8: NOT NULL exige valor. Columnas: destination_longitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna destination_longitude conserva longitud de entrega.

L9: Columnas: transport_distance_m. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna transport_distance_m conserva distancia vial origen-destino en metros.

L10: Columnas: transport_duration_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna transport_duration_seconds conserva duración vial original del traslado.

L11: Columnas: adjusted_transport_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna adjusted_transport_seconds conserva duración ajustada y redondeada hacia arriba.

L12: Columnas: route_obtained_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna route_obtained_at conserva fecha del recorrido. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L13: NOT NULL exige valor. Columnas: vehicle_type. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna vehicle_type conserva tipo CAR/SUV/PICKUP.

L14: NOT NULL exige valor. Columnas: declared_weight_kg. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna declared_weight_kg conserva peso declarado en kg.

L15: NOT NULL exige valor. Columnas: mobility_condition. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna mobility_condition conserva condición ROLLABLE/NON_ROLLABLE.

L16: NOT NULL exige valor. Columnas: maximum_arrival_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna maximum_arrival_seconds conserva presupuesto máximo hasta llegada.

L17: NOT NULL exige valor. Columnas: loading_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna loading_seconds conserva carga prevista.

L18: NOT NULL exige valor. Columnas: unloading_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna unloading_seconds conserva descarga prevista.

L19: NOT NULL exige valor. Columnas: operational_margin_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna operational_margin_seconds conserva contingencia de planificación.

L20: NOT NULL exige valor. Columnas: configuration_revision. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna configuration_revision conserva versión de reglas.

L21: NOT NULL exige valor. Columnas: coverage_and_admission_summary. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna coverage_and_admission_summary conserva resumen de cobertura y admisión.

L22: NOT NULL exige valor. Columnas: result. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna result conserva resultado de evaluación, distinto del estado de solicitud.

L23: Columnas: reason_code. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna reason_code conserva causa codificada.

L24: Columnas: reason_detail. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna reason_detail conserva detalle diagnóstico acotado.

L25: NOT NULL exige valor. Columnas: evaluated_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna evaluated_at conserva instante de evaluación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L28 a L53

```
28 | CREATE TABLE IF NOT EXISTS candidate_evaluations (
29 |   id UUID PRIMARY KEY,
30 |   evaluation_id UUID NOT NULL REFERENCES evaluations(id),
31 |   tow_truck_id uuid NOT NULL,
32 |   operator_id uuid NOT NULL,
33 |   operator_subject varchar(100) NOT NULL,
34 |   truck_capacity_kg numeric(9,  2) NOT NULL,
35 |   qualification_category varchar(30) NOT NULL,
36 |   qualification_valid_until timestamptz NOT NULL,
37 |   truck_latitude numeric(10,  7) NOT NULL,
38 |   truck_longitude numeric(10,  7) NOT NULL,
39 |   truck_location_updated_at timestamptz NOT NULL,
40 |   compatible boolean NOT NULL,
41 |   eligible boolean NOT NULL,
42 |   reason_code varchar(100),
43 |   approach_distance_m numeric,
44 |   approach_duration_seconds numeric,
45 |   adjusted_approach_seconds integer,
46 |   approach_route_obtained_at timestamptz,
47 |   transport_distance_m numeric,
48 |   transport_duration_seconds numeric,
49 |   adjusted_transport_seconds integer,
50 |   transport_route_obtained_at timestamptz,
51 |   estimated_service_end_at timestamptz,
52 |   ranking_position integer
53 | );
```

Crea candidate_evaluations solo si la tabla no existe. La FK exige un registro relacionado en esta misma base; no cruza microservicios. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. Columnas: id, evaluation_id, tow_truck_id, operator_id, operator_subject, truck_capacity_kg, qualification_category, qualification_valid_until, truck_latitude, truck_longitude, truck_location_updated_at, compatible, eligible, reason_code, approach_distance_m, approach_duration_seconds, adjusted_approach_seconds, approach_route_obtained_at, transport_distance_m, transport_duration_seconds, adjusted_transport_seconds, transport_route_obtained_at, estimated_service_end_at, ranking_position. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro. La columna tow_truck_id conserva UUID de grúa. La columna operator_id conserva UUID del perfil operador. La columna operator_subject conserva sub del operador destinatario. La columna truck_capacity_kg conserva snapshot de capacidad. La columna qualification_category conserva habilitación LIGHT_TOWING. La columna qualification_valid_until conserva fin de habilitación que debe superar el servicio estimado. La columna truck_latitude conserva snapshot de latitud de grúa. La columna truck_longitude conserva snapshot de longitud de grúa. La columna truck_location_updated_at conserva fecha de posición usada al evaluar. La columna compatible conserva compatibilidad de capacidad, tipo y movilidad. La columna eligible conserva aptitud completa del candidato/compromiso. La columna reason_code conserva causa codificada. La columna approach_distance_m conserva distancia vial grúa-recogida. La columna approach_duration_seconds conserva duración original de aproximación. La columna adjusted_approach_seconds conserva aproximación ajustada. La columna approach_route_obtained_at conserva fecha de ruta de aproximación. La columna transport_distance_m conserva distancia vial origen-destino en metros. La columna transport_duration_seconds conserva duración vial original del traslado. La columna adjusted_transport_seconds conserva duración ajustada y redondeada hacia arriba. La columna transport_route_obtained_at conserva fecha de ruta de traslado. La columna estimated_service_end_at conserva fin planificado para controlar habilitación. La columna ranking_position conserva posición entre elegibles, comenzando en 1. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L29: PRIMARY KEY impide ID nulo o repetido. Columnas: id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro.

L30: La FK exige un registro relacionado en esta misma base; no cruza microservicios. NOT NULL exige valor. Columnas: evaluation_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L31: NOT NULL exige valor. Columnas: tow_truck_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna tow_truck_id conserva UUID de grúa.

L32: NOT NULL exige valor. Columnas: operator_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna operator_id conserva UUID del perfil operador.

L33: NOT NULL exige valor. Columnas: operator_subject. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna operator_subject conserva sub del operador destinatario.

L34: NOT NULL exige valor. Columnas: truck_capacity_kg. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna truck_capacity_kg conserva snapshot de capacidad.

L35: NOT NULL exige valor. Columnas: qualification_category. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna qualification_category conserva habilitación LIGHT_TOWING.

L36: NOT NULL exige valor. Columnas: qualification_valid_until. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna qualification_valid_until conserva fin de habilitación que debe superar el servicio estimado. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L37: NOT NULL exige valor. Columnas: truck_latitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna truck_latitude conserva snapshot de latitud de grúa.

L38: NOT NULL exige valor. Columnas: truck_longitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna truck_longitude conserva snapshot de longitud de grúa.

L39: NOT NULL exige valor. Columnas: truck_location_updated_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna truck_location_updated_at conserva fecha de posición usada al evaluar. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L40: NOT NULL exige valor. Columnas: compatible. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna compatible conserva compatibilidad de capacidad, tipo y movilidad.

L41: NOT NULL exige valor. Columnas: eligible. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna eligible conserva aptitud completa del candidato/compromiso.

L42: Columnas: reason_code. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna reason_code conserva causa codificada.

L43: Columnas: approach_distance_m. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna approach_distance_m conserva distancia vial grúa-recogida.

L44: Columnas: approach_duration_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna approach_duration_seconds conserva duración original de aproximación.

L45: Columnas: adjusted_approach_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna adjusted_approach_seconds conserva aproximación ajustada.

L46: Columnas: approach_route_obtained_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna approach_route_obtained_at conserva fecha de ruta de aproximación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L47: Columnas: transport_distance_m. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna transport_distance_m conserva distancia vial origen-destino en metros.

L48: Columnas: transport_duration_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna transport_duration_seconds conserva duración vial original del traslado.

L49: Columnas: adjusted_transport_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna adjusted_transport_seconds conserva duración ajustada y redondeada hacia arriba.

L50: Columnas: transport_route_obtained_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna transport_route_obtained_at conserva fecha de ruta de traslado. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L51: Columnas: estimated_service_end_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna estimated_service_end_at conserva fin planificado para controlar habilitación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L52: Columnas: ranking_position. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna ranking_position conserva posición entre elegibles, comenzando en 1.

L55

```
55 | CREATE INDEX IF NOT EXISTS ix_evaluations_request ON evaluations(request_id,  evaluated_at,  id);
```

Índice de búsqueda ix_evaluations_request sobre evaluations.

L57

```
57 | CREATE INDEX IF NOT EXISTS ix_candidate_evaluation ON candidate_evaluations(evaluation_id,
     ranking_position);
```

Índice de búsqueda ix_candidate_evaluation sobre candidate_evaluations.

L59 a L62

```
59 | CREATE TABLE IF NOT EXISTS routing_budget(
60 |   id INTEGER PRIMARY KEY CHECK(id=1),
61 |   blocked_until TIMESTAMPTZ NOT NULL
62 | );
```

Crea routing_budget solo si la tabla no existe. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: id, blocked_until. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L60: PRIMARY KEY impide ID nulo o repetido. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L61: NOT NULL exige valor. Columnas: blocked_until. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L64

```
64 | INSERT INTO routing_budget(id,  blocked_until) VALUES(1,  '1970-01-01T00:00:00Z') ON CONFLICT(id)
      DO NOTHING;
```

Inserta en routing_budget. DO NOTHING evita duplicar la intención/fila.

L66 a L69

```
66 | CREATE TABLE IF NOT EXISTS routing_calls(
67 |   id UUID PRIMARY KEY,
68 |   called_at TIMESTAMPTZ NOT NULL
69 | );
```

Crea routing_calls solo si la tabla no existe. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. Columnas: id, called_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L67: PRIMARY KEY impide ID nulo o repetido. Columnas: id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro.

L68: NOT NULL exige valor. Columnas: called_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L71

```
71 | CREATE INDEX IF NOT EXISTS ix_routing_calls_time ON routing_calls(called_at);
```

Índice de búsqueda ix_routing_calls_time sobre routing_calls.

Líneas vacías 27, 54, 56, 58, 63, 65, 70: separadores sin instrucciones.

003-assignments (assistance-service)

`Ruta: assistance-service/src/main/resources/db/003-assignments.sql`

Objetivo y funcionalidad

Añade asignaciones y acciones remotas pendientes para reservar, confirmar y liberar en Fleet sin una transacción SQL entre dos bases.

L1

```
1 | ALTER TABLE assistance_requests ADD COLUMN IF NOT EXISTS resource_release_pending boolean NOT
    NULL DEFAULT false;
```

Añade resource_release_pending a assistance_requests sin borrar filas; omite si ya existe. NOT NULL exige valor. DEFAULT aporta el valor indicado cuando no se suministra otro. Columnas: resource_release_pending. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna resource_release_pending conserva liberación remota pendiente que impide reutilización.

L3 a L21

```
 3 | CREATE TABLE IF NOT EXISTS assignments (
 4 |   id uuid PRIMARY KEY,
 5 |   request_id uuid NOT NULL REFERENCES assistance_requests(id),
 6 |   candidate_evaluation_id uuid NOT NULL REFERENCES candidate_evaluations(id),
 7 |   tow_truck_id uuid NOT NULL,
 8 |   operator_id uuid NOT NULL,
 9 |   operator_subject varchar(100) NOT NULL,
10 |   status varchar(20) NOT NULL CHECK(status IN ('RESERVING',  'PROPOSED',  'ACCEPTED',
    'REJECTED',  'EXPIRED',  'CANCELLED',  'CLOSED')),
11 |   created_at timestamptz NOT NULL,
12 |   expires_at timestamptz,
13 |   accepted_at timestamptz,
14 |   closed_at timestamptz,
15 |   response_key varchar(100),
16 |   response_fingerprint varchar(64),
17 |   response_kind varchar(20),
18 |   search_attempt_number integer NOT NULL,
19 |   reserve_payload text NOT NULL,
20 |   reason text
21 | );
```

Crea assignments solo si la tabla no existe. La FK exige un registro relacionado en esta misma base; no cruza microservicios. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: id, request_id, candidate_evaluation_id, tow_truck_id, operator_id, operator_subject, status, created_at, expires_at, accepted_at, closed_at, response_key, response_fingerprint, response_kind, search_attempt_number, reserve_payload, reason. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro. La columna request_id conserva UUID de solicitud. La columna tow_truck_id conserva UUID de grúa. La columna operator_id conserva UUID del perfil operador. La columna operator_subject conserva sub del operador destinatario. La columna status conserva estado del objeto; su enum o CHECK define valores permitidos. La columna created_at conserva fecha de creación. La columna expires_at conserva fin exclusivo de retención/propuesta. La columna accepted_at conserva instante de aceptación efectiva. La columna closed_at conserva fecha de cierre; null indica abierto. La columna search_attempt_number conserva número de búsqueda de 1 a 3. La columna reason conserva motivo legible. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L4: PRIMARY KEY impide ID nulo o repetido. Columnas: id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro.

L5: La FK exige un registro relacionado en esta misma base; no cruza microservicios. NOT NULL exige valor. Columnas: request_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna request_id conserva UUID de solicitud.

L6: La FK exige un registro relacionado en esta misma base; no cruza microservicios. NOT NULL exige valor. Columnas: candidate_evaluation_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L7: NOT NULL exige valor. Columnas: tow_truck_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna tow_truck_id conserva UUID de grúa.

L8: NOT NULL exige valor. Columnas: operator_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna operator_id conserva UUID del perfil operador.

L9: NOT NULL exige valor. Columnas: operator_subject. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna operator_subject conserva sub del operador destinatario.

L10: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna status conserva estado del objeto; su enum o CHECK define valores permitidos. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L11: NOT NULL exige valor. Columnas: created_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna created_at conserva fecha de creación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L12: Columnas: expires_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna expires_at conserva fin exclusivo de retención/propuesta. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L13: Columnas: accepted_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna accepted_at conserva instante de aceptación efectiva. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L14: Columnas: closed_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna closed_at conserva fecha de cierre; null indica abierto. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L15: Columnas: response_key. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L16: Columnas: response_fingerprint. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L17: Columnas: response_kind. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L18: NOT NULL exige valor. Columnas: search_attempt_number. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna search_attempt_number conserva número de búsqueda de 1 a 3.

L19: NOT NULL exige valor. Columnas: reserve_payload. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L20: Columnas: reason. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna reason conserva motivo legible.

L23

```
23 | CREATE UNIQUE INDEX IF NOT EXISTS ux_assignment_active ON assignments(request_id) WHERE status IN
     ('RESERVING',  'PROPOSED',  'ACCEPTED');
```

Índice único ux_assignment_active sobre assignments. La unicidad impide repetir la combinación indicada. El índice es parcial: solo restringe registros de los estados indicados; permite conservar historial cerrado. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes. El predicado WHERE delimita las filas que participan: las filas históricas que no cumplen ese estado quedan fuera de la exclusividad.

L25

```
25 | CREATE INDEX IF NOT EXISTS ix_assignment_operator ON assignments(operator_subject,  created_at,
     id);
```

Índice de búsqueda ix_assignment_operator sobre assignments.

L27 a L35

```
27 | CREATE TABLE IF NOT EXISTS pending_actions (
28 |   id uuid PRIMARY KEY,
29 |   assignment_id uuid NOT NULL REFERENCES assignments(id),
30 |   kind varchar(20) NOT NULL CHECK(kind IN ('RESERVE',  'CONFIRM',  'RELEASE')),
31 |   status varchar(20) NOT NULL CHECK(status IN ('PENDING',  'COMPLETED',  'RETRYING')),
32 |   created_at timestamptz NOT NULL,
33 |   updated_at timestamptz NOT NULL,
34 |   UNIQUE(assignment_id,  kind)
35 | );
```

Crea pending_actions solo si la tabla no existe. La FK exige un registro relacionado en esta misma base; no cruza microservicios. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. La unicidad impide repetir la combinación indicada. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: id, assignment_id, kind, status, created_at, updated_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro. La columna status conserva estado del objeto; su enum o CHECK define valores permitidos. La columna created_at conserva fecha de creación. La columna updated_at conserva fecha de modificación. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L28: PRIMARY KEY impide ID nulo o repetido. Columnas: id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro.

L29: La FK exige un registro relacionado en esta misma base; no cruza microservicios. NOT NULL exige valor. Columnas: assignment_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L30: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: kind. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L31: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna status conserva estado del objeto; su enum o CHECK define valores permitidos. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L32: NOT NULL exige valor. Columnas: created_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna created_at conserva fecha de creación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L33: NOT NULL exige valor. Columnas: updated_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna updated_at conserva fecha de modificación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L34: La unicidad impide repetir la combinación indicada. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes.

Líneas vacías 2, 22, 24, 26: separadores sin instrucciones.

004-execution (assistance-service)

`Ruta: assistance-service/src/main/resources/db/004-execution.sql`

Objetivo y funcionalidad

Añade a assignments las fechas dispatched_at, arrived_at, loaded_at, transport_started_at y completed_at, además de la nota de entrega. Conserva cuándo el operador declaró cada etapa del servicio.

L1

```
1 | ALTER TABLE assignments ADD COLUMN IF NOT EXISTS dispatched_at timestamptz;
```

Añade dispatched_at a assignments sin borrar filas; omite si ya existe. Columnas: dispatched_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L3

```
3 | ALTER TABLE assignments ADD COLUMN IF NOT EXISTS arrived_at timestamptz;
```

Añade arrived_at a assignments sin borrar filas; omite si ya existe. Columnas: arrived_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L5

```
5 | ALTER TABLE assignments ADD COLUMN IF NOT EXISTS loaded_at timestamptz;
```

Añade loaded_at a assignments sin borrar filas; omite si ya existe. Columnas: loaded_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L7

```
7 | ALTER TABLE assignments ADD COLUMN IF NOT EXISTS transport_started_at timestamptz;
```

Añade transport_started_at a assignments sin borrar filas; omite si ya existe. Columnas: transport_started_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L9

```
9 | ALTER TABLE assignments ADD COLUMN IF NOT EXISTS completed_at timestamptz;
```

Añade completed_at a assignments sin borrar filas; omite si ya existe. Columnas: completed_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L11

```
11 | ALTER TABLE assignments ADD COLUMN IF NOT EXISTS completion_note varchar(500);
```

Añade completion_note a assignments sin borrar filas; omite si ya existe. Columnas: completion_note. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

Líneas vacías 2, 4, 6, 8, 10: separadores sin instrucciones.

005-automation (assistance-service)

`Ruta: assistance-service/src/main/resources/db/005-automation.sql`

Objetivo y funcionalidad

Añade planificación persistente para retomar búsquedas, cupos, vencimientos y reconciliaciones después de reinicios.

L1

```
1 | ALTER TABLE pending_actions ADD COLUMN IF NOT EXISTS failure_count integer NOT NULL DEFAULT 0;
```

Añade failure_count a pending_actions sin borrar filas; omite si ya existe. NOT NULL exige valor. DEFAULT aporta el valor indicado cuando no se suministra otro. Columnas: failure_count. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L3

```
3 | ALTER TABLE pending_actions ADD COLUMN IF NOT EXISTS next_attempt_at timestamptz NOT NULL DEFAULT
    now();
```

Añade next_attempt_at a pending_actions sin borrar filas; omite si ya existe. NOT NULL exige valor. DEFAULT aporta el valor indicado cuando no se suministra otro. Columnas: next_attempt_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L5

```
5 | CREATE INDEX IF NOT EXISTS ix_pending_actions_due ON pending_actions(next_attempt_at) WHERE
    status<>'COMPLETED';
```

Índice de búsqueda ix_pending_actions_due sobre pending_actions. El predicado WHERE delimita las filas que participan: las filas históricas que no cumplen ese estado quedan fuera de la exclusividad.

L7

```
7 | ALTER TABLE assistance_requests ADD COLUMN IF NOT EXISTS next_evaluation_at timestamptz NOT NULL
    DEFAULT now();
```

Añade next_evaluation_at a assistance_requests sin borrar filas; omite si ya existe. NOT NULL exige valor. DEFAULT aporta el valor indicado cuando no se suministra otro. Columnas: next_evaluation_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna next_evaluation_at conserva próximo trabajo de evaluación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L9

```
9 | ALTER TABLE assistance_requests ADD COLUMN IF NOT EXISTS next_reconciliation_at timestamptz NOT
    NULL DEFAULT now();
```

Añade next_reconciliation_at a assistance_requests sin borrar filas; omite si ya existe. NOT NULL exige valor. DEFAULT aporta el valor indicado cuando no se suministra otro. Columnas: next_reconciliation_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna next_reconciliation_at conserva próxima revisión del compromiso. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L11

```
11 | CREATE INDEX IF NOT EXISTS ix_requests_automation ON assistance_requests(status,
    next_evaluation_at,  created_at);
```

Índice de búsqueda ix_requests_automation sobre assistance_requests.

L13

```
13 | ALTER TABLE assistance_requests ADD COLUMN IF NOT EXISTS evaluation_lease_until timestamptz NOT
    NULL DEFAULT '1970-01-01T00:00:00Z';
```

Añade evaluation_lease_until a assistance_requests sin borrar filas; omite si ya existe. NOT NULL exige valor. DEFAULT aporta el valor indicado cuando no se suministra otro. Columnas: evaluation_lease_until. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna evaluation_lease_until conserva límite de trabajo de evaluación en curso. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L15

```
15 | ALTER TABLE assistance_requests ADD COLUMN IF NOT EXISTS replacement_pending boolean NOT NULL
     DEFAULT false;
```

Añade replacement_pending a assistance_requests sin borrar filas; omite si ya existe. NOT NULL exige valor. DEFAULT aporta el valor indicado cuando no se suministra otro. Columnas: replacement_pending. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna replacement_pending conserva necesidad de iniciar reemplazo después de liberar.

Líneas vacías 2, 4, 6, 8, 10, 12, 14: separadores sin instrucciones.

Operator (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/Operator.java`

Objetivo y funcionalidad

Entidad JPA del perfil de negocio del operador. keycloakSubject identifica la cuenta externa; no guarda contraseña ni reemplaza Keycloak.

L1

```
1 | package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import static com.auxiruta.fleet.catalog.FleetTypes.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetTypes.*. Tipos y DTO locales de Fleet. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | @Entity
```

Marca entidad JPA. JPA administra instancias de esta clase como filas; se necesita @Id y un esquema compatible.

L9

```
9 | @Table(name = "operators")
```

Define tabla y restricciones de mapeo.

L10

```
10 | public class Operator {
```

Declara clase Operator. Entidad JPA del perfil de negocio del operador. keycloakSubject identifica la cuenta externa; no guarda contraseña ni reemplaza Keycloak.

L11

```
11 |   protected Operator() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L13

```
13 |   @Id UUID id = UUID.randomUUID();
```

Identifica clave primaria. Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro.

L14

```
14 |   @Version long version;
```

Activa concurrencia optimista mediante comparación/incremento de versión. Campos: version: versión para control optimista de concurrencia. Hibernate agrega la versión a la condición del UPDATE; si otra transacción la cambió, detecta conflicto en vez de pisar datos. Tipos y datos: version = versión para control optimista de concurrencia; long es entero de 64 bits para cantidades y duraciones mayores.

L16

```
16 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L17

```
17 |   Instant createdAt;
```

Campos: createdAt: fecha de creación. Tipos y datos: createdAt = fecha de creación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L19

```
19 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L20

```
20 |   Instant updatedAt;
```

Campos: updatedAt: fecha de modificación. Tipos y datos: updatedAt = fecha de modificación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L22

```
22 |   @Column(nullable = false, unique = true, length = 100)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L23

```
23 |   String keycloakSubject;
```

Campos: keycloakSubject: identidad externa del operador. Tipos y datos: keycloakSubject = identidad externa del operador; String es texto; no es por sí mismo una referencia de entidad.

L25

```
25 |   @Column(nullable = false, length = 150)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L26

```
26 |   String fullName;
```

Campos: fullName: nombre del perfil de negocio. Tipos y datos: fullName = nombre del perfil de negocio; String es texto; no es por sí mismo una referencia de entidad.

L28

```
28 |   @Column(length = 40)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L29

```
29 |   String contactPhone;
```

Campos: contactPhone: teléfono opcional. Tipos y datos: contactPhone = teléfono opcional; String es texto; no es por sí mismo una referencia de entidad.

L31

```
31 |   @Enumerated(EnumType.STRING)
```

Guarda nombre del enum, no posición ordinal.

L32

```
32 |   @Column(nullable = false, length = 20)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L33

```
33 |   Registration registrationStatus = Registration.ACTIVE;
```

Campos: registrationStatus: alta administrativa ACTIVE/INACTIVE. Se calcula registrationStatus (alta administrativa ACTIVE/INACTIVE) para reutilizar ese resultado en los pasos siguientes. ACTIVE: registro habilitado administrativamente

L35

```
35 |   @Enumerated(EnumType.STRING)
```

Guarda nombre del enum, no posición ordinal.

L36

```
36 |   @Column(nullable = false, length = 20)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L37

```
37 |   Availability availabilityStatus = Availability.UNAVAILABLE;
```

Campos: availabilityStatus: disposición a recibir propuestas. Se calcula availabilityStatus (disposición a recibir propuestas) para reutilizar ese resultado en los pasos siguientes. UNAVAILABLE: operador no disponible

L39

```
39 |   @Enumerated(EnumType.STRING)
```

Guarda nombre del enum, no posición ordinal.

L40

```
40 |   @Column(length = 30)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L41

```
41 |   Qualification qualificationCategory;
```

Campos: qualificationCategory: habilitación LIGHT_TOWING.

L43

```
43 |   Instant qualificationValidFrom;
```

Campos: qualificationValidFrom: inicio de habilitación. Tipos y datos: qualificationValidFrom = inicio de habilitación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L44

```
44 |   Instant qualificationValidUntil;
```

Campos: qualificationValidUntil: fin de habilitación que debe superar el servicio estimado. Tipos y datos: qualificationValidUntil = fin de habilitación que debe superar el servicio estimado; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L45

```
45 |   Instant availableSince;
```

Campos: availableSince: inicio de disponibilidad usado para desempate. Tipos y datos: availableSince = inicio de disponibilidad usado para desempate; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L47

```
47 |   public UUID getId() {
```

L48

```
48 |     return id;
```

Entrega id al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: identificador UUID del registro.

L49

```
49 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L51

```
51 |   public String getKeycloakSubject() {
```

Expone identidad del operador para comprobar propiedad del equipo.

L52

```
52 |     return keycloakSubject;
```

Entrega keycloakSubject al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: identidad externa del operador.

L53

```
53 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L54

```
54 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 7, 12, 15, 18, 21, 24, 27, 30, 34, 38, 42, 46, 50: separadores sin instrucciones.

OperatorRepository (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/OperatorRepository.java`

Objetivo y funcionalidad

Consulta perfiles por UUID o por el sub de Keycloak y permite bloquear el operador cuando se compromete su disponibilidad. FleetService y ReservationService lo utilizan para impedir decisiones concurrentes incompatibles sobre la misma persona.

L1

```
 1 | package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
 3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.data.jpa.repository.*;
```

Importa tipos: org.springframework.data.jpa.repository.*. Repositorios Spring Data y estructuras de paginación/orden. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.data.domain.*;
```

Importa tipos: org.springframework.data.domain.*. Repositorios Spring Data y estructuras de paginación/orden. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import jakarta.persistence.LockModeType;
```

Importa tipos: jakarta.persistence.LockModeType. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | public interface OperatorRepository extends JpaRepository<Operator, UUID> {
```

Declara interfaz OperatorRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L9

```
9 |   Optional<Operator> findByKeycloakSubject(String subject);
```

Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad.

L11

```
11 |   @Lock(LockModeType.PESSIMISTIC_WRITE)
```

Adquiere bloqueo pesimista.

L12

```
12 |   @Query("select o from Operator o where o.keycloakSubject = :subject")
```

Consulta Operator con el filtro y proyección escritos. Cambios: o.keycloakSubject recibe :subject").

L13

```
13 |   Optional<Operator> lockBySubject(String subject);
```

Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad.

L15

```
15 |   Page<Operator> findByRegistrationStatus(FleetTypes.Registration status, Pageable page);
```

Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Tipos y datos: page = page; Pageable es número, tamaño y orden de una página SQL.

L17

```
17 |   @Lock(LockModeType.PESSIMISTIC_WRITE)
```

Adquiere bloqueo pesimista.

L18

```
18 |   @Query("select o from Operator o where o.id = :id")
```

Consulta Operator con el filtro y proyección escritos. Cambios: o.id recibe :id").

L19

```
19 |   Optional<Operator> lockById(UUID id);
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L20

```
20 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 7, 10, 14, 16: separadores sin instrucciones.

ReservationRepository (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/ReservationRepository.java`

Objetivo y funcionalidad

Consulta compromisos y retenciones vencidas en Fleet. Es el acceso JPA a ResourceReservation; ReservationService decide cuáles pueden confirmarse, liberarse o expirar.

L1

```
1 | package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.data.jpa.repository.*;
```

Importa tipos: org.springframework.data.jpa.repository.*. Repositorios Spring Data y estructuras de paginación/orden. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import jakarta.persistence.LockModeType;
```

Importa tipos: jakarta.persistence.LockModeType. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | public interface ReservationRepository extends JpaRepository<ResourceReservation, UUID> {
```

Declara interfaz ReservationRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L8

```
8 |   @Lock(LockModeType.PESSIMISTIC_WRITE)
```

Adquiere bloqueo pesimista.

L9

```
9 |   @Query("select r from ResourceReservation r where r.id=:id")
```

Consulta ResourceReservation con el filtro y proyección escritos. Cambios: r.id recibe :id").

L10

```
10 |   Optional<ResourceReservation> lockById(UUID id);
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L12 a L14

```
12 |   @Query(
13 |       "select count(r)>0 from ResourceReservation r where (r.towTruckId=:truck or
     r.operatorId=:operator) and r.status in ('HELD','CONFIRMED')")
14 |   boolean busy(UUID truck, UUID operator);
```

Consulta ResourceReservation contando coincidencias. Cambios: r.towTruckId recibe :truck or r.operatorId=:operator) and r.status in ('HELD','CONFIRMED')") boolean busy(UUID truck, UUID operator).

L15

```
15 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 6, 11: separadores sin instrucciones.

ResourceReservation (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/ResourceReservation.java`

Objetivo y funcionalidad

Entidad Fleet del compromiso exclusivo de una grúa y un operador: HELD, CONFIRMED, RELEASED o EXPIRED. El ID coincide con el intento de asignación remoto.

L1

```
1 | package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | @Entity
```

Marca entidad JPA. JPA administra instancias de esta clase como filas; se necesita @Id y un esquema compatible.

L8

```
8 | @Table(name = "resource_reservations")
```

Define tabla y restricciones de mapeo.

L9

```
9 | public class ResourceReservation {
```

Declara clase ResourceReservation. Entidad Fleet del compromiso exclusivo de una grúa y un operador: HELD, CONFIRMED, RELEASED o EXPIRED. El ID coincide con el intento de asignación remoto.

L10

```
10 |   @Id UUID id;
```

Identifica clave primaria. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L11

```
11 |   @Version long version;
```

Activa concurrencia optimista mediante comparación/incremento de versión. Campos: version: versión para control optimista de concurrencia. Hibernate agrega la versión a la condición del UPDATE; si otra transacción la cambió, detecta conflicto en vez de pisar datos. Tipos y datos: version = versión para control optimista de concurrencia; long es entero de 64 bits para cantidades y duraciones mayores.

L13

```
13 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L14

```
14 |   boolean createdHold;
```

Campos: createdHold: indica si el intento llegó a retener recursos. Tipos y datos: createdHold = indica si el intento llegó a retener recursos; boolean es verdadero/falso, sin null.

L16

```
16 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L17

```
17 |   UUID requestId;
```

Campos: requestId: UUID de solicitud. Tipos y datos: requestId = UUID de solicitud; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L19

```
19 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L20

```
20 |   UUID towTruckId;
```

Campos: towTruckId: UUID de grúa. Tipos y datos: towTruckId = UUID de grúa; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L22

```
22 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L23

```
23 |   UUID operatorId;
```

Campos: operatorId: UUID del perfil operador. Tipos y datos: operatorId = UUID del perfil operador; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L25

```
25 |   @Column(nullable = false, length = 20)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L26

```
26 |   String status;
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Tipos y datos: status = estado del objeto; su enum o CHECK define valores permitidos; String es texto; no es por sí mismo una referencia de entidad.

L28

```
28 |   @Column(nullable = false, columnDefinition = "text")
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L29

```
29 |   String payload;
```

Campos: payload: contenido normalizado para reintentos. Tipos y datos: payload = contenido normalizado para reintentos; String es texto; no es por sí mismo una referencia de entidad.

L31

```
31 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L32

```
32 |   Instant createdAt;
```

Campos: createdAt: fecha de creación. Tipos y datos: createdAt = fecha de creación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L34

```
34 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L35

```
35 |   Instant expiresAt;
```

Campos: expiresAt: fin exclusivo de retención/propuesta. Tipos y datos: expiresAt = fin exclusivo de retención/propuesta; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L37

```
37 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L38

```
38 |   Instant searchDeadline;
```

Campos: searchDeadline: vencimiento del intento. Tipos y datos: searchDeadline = vencimiento del intento; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L40

```
40 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L41

```
41 |   Instant estimatedServiceEndAt;
```

Campos: estimatedServiceEndAt: fin planificado para controlar habilitación. Tipos y datos: estimatedServiceEndAt = fin planificado para controlar habilitación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L43

```
43 |   Instant confirmedAt;
```

Campos: confirmedAt: instante de confirmación en Fleet. Tipos y datos: confirmedAt = instante de confirmación en Fleet; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L44

```
44 |   Instant releasedAt;
```

Campos: releasedAt: instante de liberación. Tipos y datos: releasedAt = instante de liberación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L46

```
46 |   protected ResourceReservation() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L47

```
47 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 6, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39, 42, 45: separadores sin instrucciones.

TowTruck (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/TowTruck.java`

Objetivo y funcionalidad

Entidad JPA de la grúa: capacidad, tipos admitidos, vínculo, estado y última ubicación declarada. La disponibilidad humana está en Operator y el compromiso en ResourceReservation.

L1

```
1 | package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import static com.auxiruta.fleet.catalog.FleetTypes.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetTypes.*. Tipos y DTO locales de Fleet. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | @Entity
```

Marca entidad JPA. JPA administra instancias de esta clase como filas; se necesita @Id y un esquema compatible.

L10

```
10 | @Table(name = "tow_trucks")
```

Define tabla y restricciones de mapeo.

L11

```
11 | public class TowTruck {
```

Declara clase TowTruck. Entidad JPA de la grúa: capacidad, tipos admitidos, vínculo, estado y última ubicación declarada. La disponibilidad humana está en Operator y el compromiso en ResourceReservation.

L12

```
12 |   protected TowTruck() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L14

```
14 |   @Id UUID id = UUID.randomUUID();
```

Identifica clave primaria. Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro.

L15

```
15 |   @Version long version;
```

Activa concurrencia optimista mediante comparación/incremento de versión. Campos: version: versión para control optimista de concurrencia. Hibernate agrega la versión a la condición del UPDATE; si otra transacción la cambió, detecta conflicto en vez de pisar datos. Tipos y datos: version = versión para control optimista de concurrencia; long es entero de 64 bits para cantidades y duraciones mayores.

L17

```
17 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L18

```
18 |   Instant createdAt;
```

Campos: createdAt: fecha de creación. Tipos y datos: createdAt = fecha de creación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L20

```
20 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L21

```
21 |   Instant updatedAt;
```

Campos: updatedAt: fecha de modificación. Tipos y datos: updatedAt = fecha de modificación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L23

```
23 |   @Column(nullable = false, unique = true, length = 10)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L24

```
24 |   String licensePlate;
```

Campos: licensePlate: patente normalizada y única. Tipos y datos: licensePlate = patente normalizada y única; String es texto; no es por sí mismo una referencia de entidad.

L26

```
26 |   @ManyToOne(fetch = FetchType.LAZY)
```

Relaciona muchas filas con una entidad; LAZY difiere la carga.

L27

```
27 |   @JoinColumn(name = "operator_id", unique = true)
```

Nombra FK local.

L28

```
28 |   Operator operator;
```

Referencia Java a Operator para operator. Las anotaciones inmediatamente anteriores fijan la relación JPA y la columna de enlace; no comparte tablas entre servicios.

L30

```
30 |   @Column(nullable = false, precision = 9, scale = 2)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L31

```
31 |   BigDecimal maximumLoadKg;
```

Campos: maximumLoadKg: capacidad de carga de la grúa en kg. Tipos y datos: maximumLoadKg = capacidad de carga de la grúa en kg; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L33

```
33 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L34

```
34 |   boolean supportsNonRollable;
```

Campos: supportsNonRollable: capacidad para atender vehículos que no ruedan. Tipos y datos: supportsNonRollable = capacidad para atender vehículos que no ruedan; boolean es verdadero/falso, sin null.

L36

```
36 |   @Enumerated(EnumType.STRING)
```

Guarda nombre del enum, no posición ordinal.

L37

```
37 |   @Column(nullable = false, length = 20)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L38

```
38 |   Operational operationalStatus = Operational.INACTIVE;
```

Campos: operationalStatus: estado administrativo de la grúa. Se calcula operationalStatus (estado administrativo de la grúa) para reutilizar ese resultado en los pasos siguientes. INACTIVE: registro inactivo

L40

```
40 |   @Column(precision = 10, scale = 7)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L41

```
41 |   BigDecimal latitude;
```

Campos: latitude: latitud declarada. Tipos y datos: latitude = latitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L43

```
43 |   @Column(precision = 10, scale = 7)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L44

```
44 |   BigDecimal longitude;
```

Campos: longitude: longitud declarada. Tipos y datos: longitude = longitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L46

```
46 |   Instant locationUpdatedAt;
```

Campos: locationUpdatedAt: instante servidor de última ubicación declarada. Tipos y datos: locationUpdatedAt = instante servidor de última ubicación declarada; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L48

```
48 |   @ElementCollection
```

Persiste valores en tabla auxiliar.

L49 a L52

```
49 |   @CollectionTable(
50 |       name = "tow_truck_supported_types",
51 |       joinColumns = @JoinColumn(name = "tow_truck_id"))
52 |   @Enumerated(EnumType.STRING)
```

Nombra tabla auxiliar y referencia al dueño. Nombra FK local. Guarda nombre del enum, no posición ordinal.

L53

```
53 |   @Column(name = "vehicle_type", length = 20)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L54

```
54 |   Set<VehicleType> supportedTypes = new HashSet<>();
```

Construye HashSet<> en memoria y conserva la referencia en supportedTypes. Los argumentos del constructor determinan los datos iniciales de este objeto. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L56

```
56 |   public UUID getId() {
```

L57

```
57 |     return id;
```

Entrega id al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: identificador UUID del registro.

L58

```
58 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L59

```
59 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 8, 13, 16, 19, 22, 25, 29, 32, 35, 39, 42, 45, 47, 55: separadores sin instrucciones.

TowTruckRepository (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/TowTruckRepository.java`

Objetivo y funcionalidad

Accede a grúas y vínculos; sus consultas con bloqueo serializan cambios de equipo, ubicación y reservas. Trabaja en fleet_db y no consulta vehículos de conductores.

L1

```
1 | package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.data.jpa.repository.*;
```

Importa tipos: org.springframework.data.jpa.repository.*. Repositorios Spring Data y estructuras de paginación/orden. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.data.domain.*;
```

Importa tipos: org.springframework.data.domain.*. Repositorios Spring Data y estructuras de paginación/orden. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import jakarta.persistence.LockModeType;
```

Importa tipos: jakarta.persistence.LockModeType. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | public interface TowTruckRepository extends JpaRepository<TowTruck, UUID> {
```

Declara interfaz TowTruckRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L9

```
9 |   @EntityGraph(attributePaths = {"operator", "supportedTypes"})
```

Carga asociaciones indicadas con la consulta. JPA administra instancias de esta clase como filas; se necesita @Id y un esquema compatible.

L10 a L15

```
10 |   @Query(
11 |       "select t from TowTruck t where t.operationalStatus=:operational and
     t.operator.registrationStatus=:registration and t.operator.availabilityStatus=:availability and
     not exists (select r.id from ResourceReservation r where (r.towTruckId=t.id or
     r.operatorId=t.operator.id) and r.status in ('HELD','CONFIRMED'))")
12 |   List<TowTruck> findCandidates(
13 |       FleetTypes.Operational operational,
14 |       FleetTypes.Registration registration,
15 |       FleetTypes.Availability availability);
```

La firma aplica el JPQL y EntityGraph anteriores para obtener recursos libres por estado.

L17

```
17 |   Optional<TowTruck> findByOperatorId(UUID id);
```

Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L19

```
19 |   @Query("select t.id from TowTruck t where t.operator.keycloakSubject = :subject")
```

Consulta TowTruck con el filtro y proyección escritos.

L20

```
20 |   Optional<UUID> assignedId(String subject);
```

Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad.

L22

```
22 |   Page<TowTruck> findByOperationalStatus(FleetTypes.Operational status, Pageable page);
```

Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Tipos y datos: page = page; Pageable es número, tamaño y orden de una página SQL.

L24

```
24 |   @Lock(LockModeType.PESSIMISTIC_WRITE)
```

Adquiere bloqueo pesimista.

L25

```
25 |   @Query("select t from TowTruck t where t.id = :id")
```

Consulta TowTruck con el filtro y proyección escritos. Cambios: t.id recibe :id").

L26

```
26 |   Optional<TowTruck> lockById(UUID id);
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L27

```
27 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 7, 16, 18, 21, 23: separadores sin instrucciones.

TruckStatusHistory (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/TruckStatusHistory.java`

Objetivo y funcionalidad

Entidad de historial administrativo de la grúa. Conserva antes/después, motivo, actor y clave de idempotencia.

L1

```
1 | package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import static com.auxiruta.fleet.catalog.FleetTypes.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetTypes.*. Tipos y DTO locales de Fleet. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | @Entity
```

Marca entidad JPA. JPA administra instancias de esta clase como filas; se necesita @Id y un esquema compatible.

L9

```
9 | @Table(name = "truck_status_history")
```

Define tabla y restricciones de mapeo.

L10

```
10 | public class TruckStatusHistory {
```

Declara clase TruckStatusHistory. Entidad de historial administrativo de la grúa. Conserva antes/después, motivo, actor y clave de idempotencia.

L11

```
11 |   protected TruckStatusHistory() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L13

```
13 |   @Id UUID id = UUID.randomUUID();
```

Identifica clave primaria. Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro.

L15

```
15 |   @ManyToOne(fetch = FetchType.LAZY, optional = false)
```

Relaciona muchas filas con una entidad; LAZY difiere la carga.

L16

```
16 |   @JoinColumn(name = "tow_truck_id")
```

Nombra FK local.

L17

```
17 |   TowTruck truck;
```

Referencia Java a TowTruck para truck. Las anotaciones inmediatamente anteriores fijan la relación JPA y la columna de enlace; no comparte tablas entre servicios.

L19

```
19 |   @Enumerated(EnumType.STRING)
```

Guarda nombre del enum, no posición ordinal.

L20

```
20 |   @Column(nullable = false, length = 20)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L21

```
21 |   Operational previousStatus;
```

Campos: previousStatus: estado previo.

L23

```
23 |   @Enumerated(EnumType.STRING)
```

Guarda nombre del enum, no posición ordinal.

L24

```
24 |   @Column(nullable = false, length = 20)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L25

```
25 |   Operational newStatus;
```

Campos: newStatus: estado resultante.

L27

```
27 |   @Column(nullable = false, length = 500)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L28

```
28 |   String reason;
```

Campos: reason: motivo legible. Tipos y datos: reason = motivo legible; String es texto; no es por sí mismo una referencia de entidad.

L30

```
30 |   @Column(nullable = false, length = 100)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L31

```
31 |   String changedBySubject;
```

Campos: changedBySubject: sub del administrador responsable. Tipos y datos: changedBySubject = sub del administrador responsable; String es texto; no es por sí mismo una referencia de entidad.

L33

```
33 |   @Column(unique = true, length = 100)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L34

```
34 |   String operationKey;
```

Campos: operationKey: clave durable de acción. Tipos y datos: operationKey = clave durable de acción; String es texto; no es por sí mismo una referencia de entidad.

L36

```
36 |   @Column(length = 100)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L37

```
37 |   String correlationId;
```

Campos: correlationId: identificador opcional de correlación; su campo no implica generación automática. Tipos y datos: correlationId = identificador opcional de correlación; su campo no implica generación automática; String es texto; no es por sí mismo una referencia de entidad.

L39

```
39 |   @Column(nullable = false)
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Estos atributos describen el mapeo esperado; las restricciones reales también se definen en los scripts SQL porque ddl-auto es validate.

L40

```
40 |   Instant changedAt;
```

Campos: changedAt: instante del cambio. Tipos y datos: changedAt = instante del cambio; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L41

```
41 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 7, 12, 14, 18, 22, 26, 29, 32, 35, 38: separadores sin instrucciones.

TruckStatusHistoryRepository (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/TruckStatusHistoryRepository.java`

Objetivo y funcionalidad

Recupera cambios administrativos de grúas e identifica operaciones ya procesadas por su clave. Permite devolver el mismo resultado al reintentar sin duplicar el historial.

L1

```
1 | package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.data.jpa.repository.JpaRepository;
```

Importa tipos: org.springframework.data.jpa.repository.JpaRepository. Repositorios Spring Data y estructuras de paginación/orden. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | public interface TruckStatusHistoryRepository extends JpaRepository<TruckStatusHistory, UUID> {
```

Declara interfaz TruckStatusHistoryRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L7

```
7 |   Optional<TruckStatusHistory> findByOperationKey(String key);
```

Busca resultado anterior para repetir sin duplicar. Tipos y datos: key = key; String es texto; no es por sí mismo una referencia de entidad.

L8

```
8 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 5: separadores sin instrucciones.

001-fleet-catalog (fleet-service)

`Ruta: fleet-service/src/main/resources/db/001-fleet-catalog.sql`

Objetivo y funcionalidad

Crea operadores, grúas, tipos admitidos e historial administrativo. Sus claves, CHECK e índices impiden datos incompatibles, vínculos duplicados y estados inválidos incluso ante escrituras ajenas a los controladores.

L1 a L20

```
-- Additive bootstrap for new and existing phase-3 databases. Hibernate validates the result.
CREATE TABLE IF NOT EXISTS operators (
  id UUID PRIMARY KEY,
  keycloak_subject VARCHAR(100) NOT NULL UNIQUE,
  full_name VARCHAR(150) NOT NULL CHECK (length(trim(full_name)) > 0),
  contact_phone VARCHAR(40),
  registration_status VARCHAR(20) NOT NULL CHECK (registration_status IN ('ACTIVE',
'INACTIVE')),
  availability_status VARCHAR(20) NOT NULL CHECK (availability_status IN ('AVAILABLE',
'UNAVAILABLE')),
  qualification_category VARCHAR(30) CHECK (qualification_category = 'LIGHT_TOWING'),
  qualification_valid_from TIMESTAMPTZ,
  qualification_valid_until TIMESTAMPTZ,
  available_since TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL,
  updated_at TIMESTAMPTZ NOT NULL,
  version BIGINT NOT NULL,
  CHECK (length(trim(keycloak_subject)) > 0),
  CHECK ((qualification_category IS NULL AND qualification_valid_from IS NULL AND
qualification_valid_until IS NULL)
  OR (qualification_category IS NOT NULL AND qualification_valid_from IS NOT NULL AND
qualification_valid_until IS NOT NULL AND qualification_valid_from < qualification_valid_until)),
  CHECK ((availability_status='AVAILABLE' AND available_since IS NOT NULL AND
registration_status='ACTIVE') OR (availability_status='UNAVAILABLE' AND available_since IS NULL))
);
```

Crea operators solo si la tabla no existe. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. La unicidad impide repetir la combinación indicada. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: id, keycloak_subject, full_name, contact_phone, registration_status, availability_status, qualification_category, qualification_valid_from, qualification_valid_until, available_since, created_at, updated_at, version. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro. La columna keycloak_subject conserva identidad externa del operador. La columna full_name conserva nombre del perfil de negocio. La columna contact_phone conserva teléfono opcional. La columna registration_status conserva alta administrativa ACTIVE/INACTIVE. La columna availability_status conserva disposición a recibir propuestas. La columna qualification_category conserva habilitación LIGHT_TOWING. La columna qualification_valid_from conserva inicio de habilitación. La columna qualification_valid_until conserva fin de habilitación que debe superar el servicio estimado. La columna available_since conserva inicio de disponibilidad usado para desempate. La columna created_at conserva fecha de creación. La columna updated_at conserva fecha de modificación. La columna version conserva versión para control optimista de concurrencia. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L2: Crea operators solo si la tabla no existe.

L3: PRIMARY KEY impide ID nulo o repetido. Columnas: id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro.

L4: NOT NULL exige valor. La unicidad impide repetir la combinación indicada. Columnas: keycloak_subject. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna keycloak_subject conserva identidad externa del operador. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes.

L5: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: full_name. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna full_name conserva nombre del perfil de negocio. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L6: Columnas: contact_phone. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna contact_phone conserva teléfono opcional.

L7: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: registration_status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna registration_status conserva alta administrativa ACTIVE/INACTIVE. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L8: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: availability_status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna availability_status conserva disposición a recibir propuestas. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L9: CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: qualification_category. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna qualification_category conserva habilitación LIGHT_TOWING. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L10: Columnas: qualification_valid_from. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna qualification_valid_from conserva inicio de habilitación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L11: Columnas: qualification_valid_until. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna qualification_valid_until conserva fin de habilitación que debe superar el servicio estimado. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L12: Columnas: available_since. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna available_since conserva inicio de disponibilidad usado para desempate. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L13: NOT NULL exige valor. Columnas: created_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna created_at conserva fecha de creación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L14: NOT NULL exige valor. Columnas: updated_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna updated_at conserva fecha de modificación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L15: NOT NULL exige valor. Columnas: version. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna version conserva versión para control optimista de concurrencia.

L16: CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L17: CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L18: NOT NULL exige valor.

L19: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L22 a L36

```
22 | CREATE TABLE IF NOT EXISTS tow_trucks (
23 |   id UUID PRIMARY KEY,
24 |   license_plate VARCHAR(10) NOT NULL UNIQUE CHECK (license_plate ~ '^[A-Z0-9]{6,10}$'),
25 |   operator_id UUID UNIQUE REFERENCES operators(id),
26 |   maximum_load_kg NUMERIC(9,  2) NOT NULL CHECK (maximum_load_kg BETWEEN 500 AND 5000),
27 |   supports_non_rollable BOOLEAN NOT NULL,
28 |   operational_status VARCHAR(20) NOT NULL CHECK (operational_status IN ('ACTIVE',  'INACTIVE',
     'MAINTENANCE')),
29 |   latitude NUMERIC(10,  7) CHECK (latitude BETWEEN -90 AND 90),
30 |   longitude NUMERIC(10,  7) CHECK (longitude BETWEEN -180 AND 180),
31 |   location_updated_at TIMESTAMPTZ,
32 |   created_at TIMESTAMPTZ NOT NULL,
33 |   updated_at TIMESTAMPTZ NOT NULL,
34 |   version BIGINT NOT NULL,
35 |   CHECK ((latitude IS NULL AND longitude IS NULL AND location_updated_at IS NULL) OR (latitude IS
      NOT NULL AND longitude IS NOT NULL AND location_updated_at IS NOT NULL))
36 | );
```

Crea tow_trucks solo si la tabla no existe. La FK exige un registro relacionado en esta misma base; no cruza microservicios. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. La unicidad impide repetir la combinación indicada. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: id, license_plate, operator_id, maximum_load_kg, supports_non_rollable, operational_status, latitude, longitude, location_updated_at, created_at, updated_at, version. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro. La columna license_plate conserva patente normalizada y única. La columna operator_id conserva UUID del perfil operador. La columna maximum_load_kg conserva capacidad de carga de la grúa en kg. La columna supports_non_rollable conserva capacidad para atender vehículos que no ruedan. La columna operational_status conserva estado administrativo de la grúa. La columna latitude conserva latitud declarada. La columna longitude conserva longitud declarada. La columna location_updated_at conserva instante servidor de última ubicación declarada. La columna created_at conserva fecha de creación. La columna updated_at conserva fecha de modificación. La columna version conserva versión para control optimista de concurrencia. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L23: PRIMARY KEY impide ID nulo o repetido. Columnas: id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro.

L24: NOT NULL exige valor. La unicidad impide repetir la combinación indicada. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: license_plate. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna license_plate conserva patente normalizada y única. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes.

L25: La FK exige un registro relacionado en esta misma base; no cruza microservicios. La unicidad impide repetir la combinación indicada. Columnas: operator_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna operator_id conserva UUID del perfil operador. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes.

L26: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: maximum_load_kg. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna maximum_load_kg conserva capacidad de carga de la grúa en kg. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L27: NOT NULL exige valor. Columnas: supports_non_rollable. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna supports_non_rollable conserva capacidad para atender vehículos que no ruedan.

L28: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: operational_status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna operational_status conserva estado administrativo de la grúa. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L29: CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: latitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna latitude conserva latitud declarada. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L30: CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: longitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna longitude conserva longitud declarada. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L31: Columnas: location_updated_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna location_updated_at conserva instante servidor de última ubicación declarada. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L32: NOT NULL exige valor. Columnas: created_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna created_at conserva fecha de creación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L33: NOT NULL exige valor. Columnas: updated_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna updated_at conserva fecha de modificación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L34: NOT NULL exige valor. Columnas: version. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna version conserva versión para control optimista de concurrencia.

L35: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L38 a L42

```
38 | CREATE TABLE IF NOT EXISTS tow_truck_supported_types (
39 |   tow_truck_id UUID NOT NULL REFERENCES tow_trucks(id),
40 |   vehicle_type VARCHAR(20) NOT NULL CHECK (vehicle_type IN ('CAR',  'SUV',  'PICKUP')),
41 |   PRIMARY KEY(tow_truck_id,  vehicle_type)
42 | );
```

Crea tow_truck_supported_types solo si la tabla no existe. La FK exige un registro relacionado en esta misma base; no cruza microservicios. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: tow_truck_id, vehicle_type. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna tow_truck_id conserva UUID de grúa. La columna vehicle_type conserva tipo CAR/SUV/PICKUP. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L39: La FK exige un registro relacionado en esta misma base; no cruza microservicios. NOT NULL exige valor. Columnas: tow_truck_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna tow_truck_id conserva UUID de grúa.

L40: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: vehicle_type. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna vehicle_type conserva tipo CAR/SUV/PICKUP. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L41: PRIMARY KEY impide ID nulo o repetido.

L44 a L55

```
44 | CREATE TABLE IF NOT EXISTS truck_status_history (
45 |   id UUID PRIMARY KEY,
46 |   tow_truck_id UUID NOT NULL REFERENCES tow_trucks(id),
47 |   previous_status VARCHAR(20) NOT NULL CHECK (previous_status IN ('ACTIVE',  'INACTIVE',
   'MAINTENANCE')),
48 |   new_status VARCHAR(20) NOT NULL CHECK (new_status IN ('ACTIVE',  'INACTIVE',  'MAINTENANCE')),
49 |   reason VARCHAR(500) NOT NULL CHECK (length(trim(reason))>0),
50 |   changed_by_subject VARCHAR(100) NOT NULL,
51 |   operation_key VARCHAR(100) UNIQUE,
52 |   correlation_id VARCHAR(100),
53 |   changed_at TIMESTAMPTZ NOT NULL,
54 |   CHECK(previous_status<>new_status)
55 | );
```

Crea truck_status_history solo si la tabla no existe. La FK exige un registro relacionado en esta misma base; no cruza microservicios. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. La unicidad impide repetir la combinación indicada. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: id, tow_truck_id, previous_status, new_status, reason, changed_by_subject, operation_key, correlation_id, changed_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro. La columna tow_truck_id conserva UUID de grúa. La columna previous_status conserva estado previo. La columna new_status conserva estado resultante. La columna reason conserva motivo legible. La columna changed_by_subject conserva sub del administrador responsable. La columna operation_key conserva clave durable de acción. La columna correlation_id conserva identificador opcional de correlación; su campo no implica generación automática. La columna changed_at conserva instante del cambio. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L45: PRIMARY KEY impide ID nulo o repetido. Columnas: id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro.

L46: La FK exige un registro relacionado en esta misma base; no cruza microservicios. NOT NULL exige valor. Columnas: tow_truck_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna tow_truck_id conserva UUID de grúa.

L47: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: previous_status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna previous_status conserva estado previo. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L48: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: new_status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna new_status conserva estado resultante. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L49: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: reason. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna reason conserva motivo legible. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L50: NOT NULL exige valor. Columnas: changed_by_subject. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna changed_by_subject conserva sub del administrador responsable.

L51: La unicidad impide repetir la combinación indicada. Columnas: operation_key. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna operation_key conserva clave durable de acción. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes.

L52: Columnas: correlation_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna correlation_id conserva identificador opcional de correlación; su campo no implica generación automática.

L53: NOT NULL exige valor. Columnas: changed_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna changed_at conserva instante del cambio. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L54: CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L57

```
57 | CREATE INDEX IF NOT EXISTS ix_operators_status ON operators(registration_status,  created_at,
     id);
```

Índice de búsqueda ix_operators_status sobre operators.

L59

```
59 | CREATE INDEX IF NOT EXISTS ix_trucks_status ON tow_trucks(operational_status,  created_at,  id);
```

Índice de búsqueda ix_trucks_status sobre tow_trucks.

L61

```
61 | CREATE INDEX IF NOT EXISTS ix_truck_history ON truck_status_history(tow_truck_id,  changed_at);
```

Índice de búsqueda ix_truck_history sobre truck_status_history.

Líneas vacías 21, 37, 43, 56, 58, 60: separadores sin instrucciones.

002-reservations (fleet-service)

`Ruta: fleet-service/src/main/resources/db/002-reservations.sql`

Objetivo y funcionalidad

Añade reservas de grúa y operador. Los índices únicos parciales impiden comprometer el mismo recurso en dos reservas activas; las cerradas se conservan.

L1 a L17

```
CREATE TABLE IF NOT EXISTS resource_reservations (
  id UUID PRIMARY KEY,
  version bigint NOT NULL,
  request_id UUID NOT NULL,
  tow_truck_id UUID NOT NULL REFERENCES tow_trucks(id),
  operator_id UUID NOT NULL REFERENCES operators(id),
  status varchar(20) NOT NULL CHECK(status IN ('HELD',  'CONFIRMED',  'RELEASED',  'EXPIRED')),
  payload text NOT NULL,
  created_at timestamptz NOT NULL,
  expires_at timestamptz NOT NULL,
  search_deadline timestamptz NOT NULL,
  estimated_service_end_at timestamptz NOT NULL,
  confirmed_at timestamptz,
  released_at timestamptz,
  CHECK(expires_at<=search_deadline),
  CHECK(status<>'CONFIRMED' OR confirmed_at IS NOT NULL)
);
```

Crea resource_reservations solo si la tabla no existe. La FK exige un registro relacionado en esta misma base; no cruza microservicios. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: id, version, request_id, tow_truck_id, operator_id, status, payload, created_at, expires_at, search_deadline, estimated_service_end_at, confirmed_at, released_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro. La columna version conserva versión para control optimista de concurrencia. La columna request_id conserva UUID de solicitud. La columna tow_truck_id conserva UUID de grúa. La columna operator_id conserva UUID del perfil operador. La columna status conserva estado del objeto; su enum o CHECK define valores permitidos. La columna payload conserva contenido normalizado para reintentos. La columna created_at conserva fecha de creación. La columna expires_at conserva fin exclusivo de retención/propuesta. La columna search_deadline conserva vencimiento del intento. La columna estimated_service_end_at conserva fin planificado para controlar habilitación. La columna confirmed_at conserva instante de confirmación en Fleet. La columna released_at conserva instante de liberación. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L2: PRIMARY KEY impide ID nulo o repetido. Columnas: id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna id conserva identificador UUID del registro.

L3: NOT NULL exige valor. Columnas: version. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna version conserva versión para control optimista de concurrencia.

L4: NOT NULL exige valor. Columnas: request_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna request_id conserva UUID de solicitud.

L5: La FK exige un registro relacionado en esta misma base; no cruza microservicios. NOT NULL exige valor. Columnas: tow_truck_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna tow_truck_id conserva UUID de grúa.

L6: La FK exige un registro relacionado en esta misma base; no cruza microservicios. NOT NULL exige valor. Columnas: operator_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna operator_id conserva UUID del perfil operador.

L7: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna status conserva estado del objeto; su enum o CHECK define valores permitidos. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L8: NOT NULL exige valor. Columnas: payload. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna payload conserva contenido normalizado para reintentos.

L9: NOT NULL exige valor. Columnas: created_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna created_at conserva fecha de creación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L10: NOT NULL exige valor. Columnas: expires_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna expires_at conserva fin exclusivo de retención/propuesta. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L11: NOT NULL exige valor. Columnas: search_deadline. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna search_deadline conserva vencimiento del intento. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L12: NOT NULL exige valor. Columnas: estimated_service_end_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna estimated_service_end_at conserva fin planificado para controlar habilitación. TIMESTAMPTZ representa

instantes comparables; la zona de visualización no cambia el instante almacenado.

L13: Columnas: confirmed_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna confirmed_at conserva instante de confirmación en Fleet. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L14: Columnas: released_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna released_at conserva instante de liberación. TIMESTAMPTZ representa instantes comparables; la zona de visualización no cambia el instante almacenado.

L15: CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L16: NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. PostgreSQL rechaza INSERT/UPDATE que hagan falsa esta condición. El servicio traduce el conflicto; la restricción protege también ante escrituras directas.

L19

```
19 | CREATE UNIQUE INDEX IF NOT EXISTS ux_reservation_truck ON resource_reservations(tow_truck_id)
     WHERE status IN ('HELD',  'CONFIRMED');
```

Índice único ux_reservation_truck sobre resource_reservations. La unicidad impide repetir la combinación indicada. El índice es parcial: solo restringe registros de los estados indicados; permite conservar historial cerrado. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes. El predicado WHERE delimita las filas que participan: las filas históricas que no cumplen ese estado quedan fuera de la exclusividad.

L21

```
21 | CREATE UNIQUE INDEX IF NOT EXISTS ux_reservation_operator ON resource_reservations(operator_id)
     WHERE status IN ('HELD',  'CONFIRMED');
```

Índice único ux_reservation_operator sobre resource_reservations. La unicidad impide repetir la combinación indicada. El índice es parcial: solo restringe registros de los estados indicados; permite conservar historial cerrado. La base comprueba exclusividad de forma atómica; una consulta previa en Java no bastaría para evitar dos escrituras concurrentes. El predicado WHERE delimita las filas que participan: las filas históricas que no cumplen ese estado quedan fuera de la exclusividad.

Líneas vacías 18, 20: separadores sin instrucciones.

003-automation (fleet-service)

`Ruta: fleet-service/src/main/resources/db/003-automation.sql`

Objetivo y funcionalidad

Añade created_hold a las reservas de Fleet y rellena ese indicador en registros que sí alcanzaron retención. Permite distinguir una reserva realmente creada de una respuesta negativa, necesario para contar ofertas y reconciliar sin consumir cupos por intentos fallidos.

L1

```
1 | ALTER TABLE resource_reservations ADD COLUMN IF NOT EXISTS created_hold boolean NOT NULL DEFAULT
    false;
```

Añade created_hold a resource_reservations sin borrar filas; omite si ya existe. NOT NULL exige valor. DEFAULT aporta el valor indicado cuando no se suministra otro. Columnas: created_hold. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal. La columna created_hold conserva indica si el intento llegó a retener recursos.

L3

```
3 | UPDATE resource_reservations SET created_hold=true WHERE status IN ('HELD',  'CONFIRMED',
    'RELEASED') AND created_hold=false;
```

Actualiza resource_reservations para las filas del filtro.

Líneas vacías 2: separadores sin instrucciones.

5. Flota: administración, candidatos y reservas

ACTIVE, AVAILABLE y HELD representan registro, disponibilidad y compromiso diferentes; no deben confundirse con contenedor running.

CandidateController (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/CandidateController.java`

Objetivo y funcionalidad

API interna de preselección, consumida por Assistance con credenciales técnicas. Valida parámetros y delega el filtrado.

L1

```
1 | package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.List;
```

Importa tipos: java.util.List. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.web.bind.annotation.*;
```

Importa tipos: org.springframework.web.bind.annotation.*. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.validation.annotation.Validated;
```

Importa tipos: org.springframework.validation.annotation.Validated. Tipo auxiliar usado por las expresiones del archivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import jakarta.validation.constraints.*;
```

Importa tipos: jakarta.validation.constraints.*. Restricciones de DTO y activación de validación de entradas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import io.swagger.v3.oas.annotations.Hidden;
```

Importa tipos: io.swagger.v3.oas.annotations.Hidden. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | @RestController
```

Expone HTTP y serializa JSON.

L11

```
11 | @Validated
```

Activa validación de parámetros de métodos del componente, como límites de coordenadas o pesos. Si falla un parámetro, se rechaza la petición antes de usarlo en preselección.

L12

```
12 | @Hidden
```

Oculta operación interna en OpenAPI.

L13

```
13 | public class CandidateController {
```

Declara clase CandidateController. API interna de preselección, consumida por Assistance con credenciales técnicas. Valida parámetros y delega el filtrado.

L14

```
14 |   private final CandidateService service;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L16

```
16 |   public CandidateController(CandidateService service) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L17

```
17 |     this.service = service;
```

En this guarda service. Toma el dato de service después de las validaciones de este método. service.

L18

```
18 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L20

```
20 |   @GetMapping("/internal/fleet/candidates")
```

Ruta /internal/fleet/candidates combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L21 a L29

```
21 |   public List<CandidateService.Candidate> candidates(
22 |       @RequestParam @DecimalMin("-90") @DecimalMax("90") @Digits(integer = 3, fraction = 6)
23 |           BigDecimal latitude,
24 |       @RequestParam @DecimalMin("-180") @DecimalMax("180") @Digits(integer = 3, fraction = 6)
25 |           BigDecimal longitude,
26 |       @RequestParam FleetTypes.VehicleType vehicleType,
27 |       @RequestParam @DecimalMin("500") @DecimalMax("3000") @Digits(integer = 4, fraction = 2)
28 |           BigDecimal weightKg,
29 |       @RequestParam boolean nonRollable) {
```

Endpoint técnico de candidatos: los parámetros de líneas siguientes se validan antes de delegar. Se calcula fraction (fraction) para reutilizar ese resultado en los pasos siguientes. Tipos y datos: latitude = latitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; longitude = longitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; weightKg = weightKg; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; nonRollable = nonRollable; boolean es verdadero/falso, sin null.

L23: Dato del contrato: latitude: latitud declarada.

L25: Dato del contrato: longitude: longitud declarada.

L26: Dato del contrato: vehicleType: tipo CAR/SUV/PICKUP.

L30

```
30 |     return service.find(latitude, longitude, vehicleType, weightKg, nonRollable);
```

Delega preselección en CandidateService con origen, tipo, peso y movilidad. Devuelve candidatos aptos por reglas Fleet; Assistance calculará rutas viales para el ranking final.

L31

```
31 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L32

```
32 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 9, 15, 19: separadores sin instrucciones.

CandidateService (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/CandidateService.java`

Objetivo y funcionalidad

Preselecciona como máximo cinco grúas libres y compatibles, dentro de cobertura y con ubicación vigente. Ordena por distancia recta; Assistance realiza después el ranking vial.

L1

```
1 | package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import com.auxiruta.commons.GeoMath;
```

Importa tipos: com.auxiruta.commons.GeoMath. Utilidades compartidas de seguridad, errores, reloj o geografía. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
 9 | import static com.auxiruta.fleet.catalog.FleetTypes.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetTypes.*. Tipos y DTO locales de Fleet. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | @Service
```

Registra lógica de aplicación como bean. El escaneo de Spring crea este componente e inyecta sus dependencias; no hay que construirlo manualmente en el controlador.

L12

```
12 | public class CandidateService {
```

Declara clase CandidateService. Preselecciona como máximo cinco grúas libres y compatibles, dentro de cobertura y con ubicación vigente. Ordena por distancia recta; Assistance realiza después el ranking vial.

L13

```
13 |   private final TowTruckRepository trucks;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L14

```
14 |   private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L16

```
16 |   public CandidateService(TowTruckRepository trucks, Clock clock) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L17

```
17 |     this.trucks = trucks;
```

En this guarda repositorio de grúas. Toma el dato de trucks después de las validaciones de este método. repositorio de grúas.

L18

```
18 |     this.clock = clock;
```

En this guarda reloj usado para evaluar plazos. Toma el dato de clock después de las validaciones de este método. reloj usado para evaluar plazos.

L19

```
19 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L21 a L35

```
21 |   public record Candidate(
22 |       UUID towTruckId,
23 |       UUID operatorId,
24 |       String operatorSubject,
25 |       BigDecimal capacityKg,
26 |       Set<VehicleType> supportedTypes,
27 |       boolean supportsNonRollable,
28 |       String qualificationCategory,
29 |       Instant qualificationValidFrom,
30 |       Instant qualificationValidUntil,
31 |       Instant availableSince,
32 |       BigDecimal latitude,
33 |       BigDecimal longitude,
34 |       Instant locationUpdatedAt,
35 |       double straightDistanceM) {}
```

Declara record inmutable Candidate. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: towTruckId: UUID de grúa; operatorId: UUID del perfil operador; operatorSubject: sub del operador destinatario; supportsNonRollable: capacidad para atender vehículos que no ruedan; qualificationCategory: habilitación LIGHT_TOWING; qualificationValidFrom: inicio de habilitación; qualificationValidUntil: fin de habilitación que debe superar el servicio estimado; availableSince: inicio de disponibilidad usado para desempate; latitude: latitud declarada; longitude: longitud declarada; locationUpdatedAt: instante servidor de última ubicación declarada. Preselecciona como máximo cinco grúas libres y compatibles, dentro de cobertura y con ubicación vigente. Ordena por distancia recta; Assistance realiza después el ranking vial. Tipos y datos: towTruckId = UUID de grúa; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; operatorId = UUID del perfil operador; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; operatorSubject = sub del operador destinatario; String es texto; no es por sí mismo una referencia de entidad; capacityKg = capacityKg; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; supportsNonRollable = capacidad para atender vehículos que no ruedan; boolean es verdadero/falso, sin null; qualificationCategory = habilitación LIGHT_TOWING; String es texto; no es por sí mismo una referencia de entidad; qualificationValidFrom = inicio de habilitación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; qualificationValidUntil = fin de habilitación que debe superar el servicio estimado; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; availableSince = inicio de disponibilidad usado para desempate; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; latitude = latitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; longitude = longitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; locationUpdatedAt = instante servidor de última ubicación declarada; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L22: Dato del contrato: towTruckId: UUID de grúa.

L23: Dato del contrato: operatorId: UUID del perfil operador.

L24: Dato del contrato: operatorSubject: sub del operador destinatario.

L27: Dato del contrato: supportsNonRollable: capacidad para atender vehículos que no ruedan.

L28: Dato del contrato: qualificationCategory: habilitación LIGHT_TOWING.

L29: Dato del contrato: qualificationValidFrom: inicio de habilitación.

L30: Dato del contrato: qualificationValidUntil: fin de habilitación que debe superar el servicio estimado.

L31: Dato del contrato: availableSince: inicio de disponibilidad usado para desempate.

L32: Dato del contrato: latitude: latitud declarada.

L33: Dato del contrato: longitude: longitud declarada.

L34: Dato del contrato: locationUpdatedAt: instante servidor de última ubicación declarada.

L37

```
37 |   @Transactional(readOnly = true)
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L38 a L43

```
38 |   public List<Candidate> find(
39 |       BigDecimal latitude,
40 |       BigDecimal longitude,
41 |       VehicleType type,
42 |       BigDecimal weight,
43 |       boolean nonRollable) {
```

Campos: latitude: latitud declarada; longitude: longitud declarada. Tipos y datos: latitude = latitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; longitude = longitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; weight = weight; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; nonRollable = nonRollable; boolean es verdadero/falso, sin null.

L39: Dato del contrato: latitude: latitud declarada.

L40: Dato del contrato: longitude: longitud declarada.

L44

```
44 |     Instant now = clock.instant();
```

Captura un instante consistente de referencia para comparar condiciones temporales en esta operación. Se calcula now (instante actual obtenido del reloj inyectado) para reutilizar ese resultado en los pasos siguientes. lee el instante del reloj inyectado

L45 a L105

```
45 |     return trucks
46 |         .findCandidates(Operational.ACTIVE, Registration.ACTIVE, Availability.AVAILABLE)
47 |         .stream()
48 |         .filter(
49 |             t ->
50 |                 t.maximumLoadKg.compareTo(weight) >= 0
51 |                     && t.supportedTypes.contains(type)
52 |                     && (!nonRollable || t.supportsNonRollable))
53 |         .filter(
54 |             t ->
55 |                 t.latitude != null
56 |                     && t.longitude != null
57 |                     && t.locationUpdatedAt != null
58 |                     && !t.locationUpdatedAt.isAfter(now)
59 |                     && !t.locationUpdatedAt.isBefore(now.minusSeconds(300)))
60 |         .filter(
61 |             t ->
62 |                 GeoMath.distance(
63 |                         -31.4167,
64 |                         -64.1833,
65 |                         t.latitude.doubleValue(),
66 |                         t.longitude.doubleValue(),
67 |                         6371000)
68 |                     <= 15000)
69 |         .map(
70 |             t -> {
71 |               Operator o = org.hibernate.Hibernate.unproxy(t.operator, Operator.class);
72 |               return new Candidate(
73 |                   t.id,
74 |                   o.id,
75 |                   o.keycloakSubject,
76 |                   t.maximumLoadKg,
77 |                   Set.copyOf(t.supportedTypes),
78 |                   t.supportsNonRollable,
79 |                   o.qualificationCategory == null ? null : o.qualificationCategory.name(),
80 |                   o.qualificationValidFrom,
81 |                   o.qualificationValidUntil,
82 |                   o.availableSince,
83 |                   t.latitude,
84 |                   t.longitude,
85 |                   t.locationUpdatedAt,
86 |                   GeoMath.distance(
87 |                       latitude.doubleValue(),
88 |                       longitude.doubleValue(),
```

```
                        t.latitude.doubleValue(),
                        t.longitude.doubleValue(),
                        6371000));
            })
        .filter(
            c ->
                "LIGHT_TOWING".equals(c.qualificationCategory())
                    && c.qualificationValidFrom() != null
                    && c.qualificationValidUntil() != null
                    && !c.qualificationValidFrom().isAfter(now)
                    && c.qualificationValidUntil().isAfter(now)
                    && c.availableSince() != null)
        .sorted(
            Comparator.comparingDouble(Candidate::straightDistanceM)
                .thenComparing(c -> c.towTruckId().toString()))
        .limit(5)
        .toList();
```

Obtiene grúas ACTIVE, operadores ACTIVE/AVAILABLE y sin reservas; inicia filtros adicionales en memoria. Resultado que recibe el llamador: calcula un instante anterior restando segundos; compara si el instante es estrictamente posterior al otro; compara si el instante es estrictamente anterior al otro; compara decimales: negativo menor, cero igual, positivo mayor; compara contenido/identidad lógica, no referencias Java; verifica pertenencia al conjunto o lista; inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente; retiene solo elementos que cumplen la condición; transforma cada elemento al tipo/valor de salida; materializa el resultado de la secuencia en una lista; representa el valor como texto; en un nodo JSON produce JSON serializado; ACTIVE: registro habilitado administrativamente; AVAILABLE: operador dispuesto a recibir propuestas; Datos implicados: capacidad de carga de la grúa en kg, tipos de vehículo admitidos, capacidad para atender vehículos que no ruedan, latitud declarada, longitud declarada, instante servidor de última ubicación declarada, identificador UUID del registro, identidad externa del operador, habilitación LIGHT_TOWING, inicio de habilitación, fin de habilitación que debe superar el servicio estimado, inicio de disponibilidad usado para desempate, UUID de grúa; && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue; comprueba presencia antes de usar el dato; detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L46: ACTIVE: registro habilitado administrativamente; AVAILABLE: operador dispuesto a recibir propuestas.

L47: inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente.

L48: retiene solo elementos que cumplen la condición.

L51: verifica pertenencia al conjunto o lista; Datos implicados: tipos de vehículo admitidos; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L52: Datos implicados: capacidad para atender vehículos que no ruedan; && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L53: retiene solo elementos que cumplen la condición.

L56: Datos implicados: longitud declarada; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato.

L57: Datos implicados: instante servidor de última ubicación declarada; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato.

L58: compara si el instante es estrictamente posterior al otro; Datos implicados: instante servidor de última ubicación declarada; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L59: calcula un instante anterior restando segundos; compara si el instante es estrictamente anterior al otro; Datos implicados: instante servidor de última ubicación declarada; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L60: retiene solo elementos que cumplen la condición.

L69: transforma cada elemento al tipo/valor de salida.

L93: retiene solo elementos que cumplen la condición.

L96: Datos implicados: inicio de habilitación; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato.

L97: Datos implicados: fin de habilitación que debe superar el servicio estimado; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato.

L98: compara si el instante es estrictamente posterior al otro; Datos implicados: inicio de habilitación; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L99: compara si el instante es estrictamente posterior al otro; Datos implicados: fin de habilitación que debe superar el servicio estimado; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L100: Datos implicados: inicio de disponibilidad usado para desempate; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato.

L103: representa el valor como texto; en un nodo JSON produce JSON serializado; Datos implicados: UUID de grúa.

L105: materializa el resultado de la secuencia en una lista.

L106

```
106 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L107

```
107 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 10, 15, 20, 36: separadores sin instrucciones.

FleetController (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/FleetController.java`

Objetivo y funcionalidad

API pública de flota: ADMIN administra recursos y OPERATOR opera su propio perfil/equipo. Delega las reglas y transacciones en FleetService.

L1

```
1 | package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.net.URI;
```

Importa tipos: java.net.URI. Tipo auxiliar usado por las expresiones del archivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import jakarta.validation.Valid;
```

Importa tipos: jakarta.validation.Valid. Restricciones de DTO y activación de validación de entradas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.web.bind.annotation.*;
```

Importa tipos: org.springframework.web.bind.annotation.*. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.http.ResponseEntity;
```

Importa tipos: org.springframework.http.ResponseEntity. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
 8 | import org.springframework.security.core.annotation.AuthenticationPrincipal;
```

Importa tipos: org.springframework.security.core.annotation.AuthenticationPrincipal. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
 9 | import org.springframework.security.oauth2.jwt.Jwt;
```

Importa tipos: org.springframework.security.oauth2.jwt.Jwt. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import io.swagger.v3.oas.annotations.Operation;
```

Importa tipos: io.swagger.v3.oas.annotations.Operation. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | import io.swagger.v3.oas.annotations.security.SecurityRequirement;
```

Importa tipos: io.swagger.v3.oas.annotations.security.SecurityRequirement. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12

```
12 | import io.swagger.v3.oas.annotations.tags.Tag;
```

Importa tipos: io.swagger.v3.oas.annotations.tags.Tag. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L13

```
13 | import io.swagger.v3.oas.annotations.responses.*;
```

Importa tipos: io.swagger.v3.oas.annotations.responses.*. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L14

```
14 | import static com.auxiruta.fleet.catalog.FleetDtos.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetDtos.*. Tipos y DTO locales de Fleet. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L15

```
15 | import static com.auxiruta.fleet.catalog.FleetTypes.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetTypes.*. Tipos y DTO locales de Fleet. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L17

```
17 | @RestController
```

Expone HTTP y serializa JSON.

L18

```
18 | @RequestMapping("/api/v1")
```

Ruta /api/v1 combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L19 a L23

```
19 | @Tag(
20 |     name = "Fleet",
21 |     description =
22 |         "Administración: ADMIN. Rutas me y assigned-to-me: OPERATOR, identidad tomada del JWT.")
23 | @SecurityRequirement(name = "bearerAuth")
```

Agrupa operaciones Fleet en OpenAPI y describe alcance de roles.

L24 a L31

```
24 | @ApiResponses({
25 |   @ApiResponse(responseCode = "400", description = "Entrada inválida"),
26 |   @ApiResponse(responseCode = "401", description = "JWT ausente o inválido"),
27 |   @ApiResponse(responseCode = "403", description = "Rol insuficiente"),
28 |   @ApiResponse(responseCode = "404", description = "Recurso inexistente o ajeno"),
29 |   @ApiResponse(responseCode = "409", description = "Duplicado o conflicto de estado")
30 | })
31 | public class FleetController {
```

Documenta código HTTP. Declara clase FleetController. API pública de flota: ADMIN administra recursos y OPERATOR opera su propio perfil/equipo. Delega las reglas y transacciones en FleetService.

L32

```
32 |   private final FleetService service;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L34

```
34 |   public FleetController(FleetService service) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L35

```
35 |     this.service = service;
```

En this guarda service. Toma el dato de service después de las validaciones de este método. service.

L36

```
36 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L38

```
38 |   @ApiResponse(responseCode = "201", description = "Perfil creado; Location contiene su URL")
```

Documenta código HTTP.

L39

```
39 |   @PostMapping("/operators")
```

Ruta /operators combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L40 a L42

```
40 |   @Operation(
41 |       summary = "ADMIN: registrar perfil de una identidad OPERATOR creada previamente en
     Keycloak")
42 |   public ResponseEntity<OperatorView> createOperator(@Valid @RequestBody CreateOperator d) {
```

Documenta en OpenAPI: ADMIN: registrar perfil de una identidad OPERATOR creada previamente en Keycloak. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L43

```
43 |     var o = service.create(d);
```

Se calcula o (o) para reutilizar ese resultado en los pasos siguientes.

L44

```
44 |     return ResponseEntity.created(URI.create("/api/v1/operators/" + o.id())).body(o);
```

Responde 201 con Location. Resultado que recibe el llamador: crea una respuesta 201 con dirección Location del recurso; Datos implicados: identificador UUID del registro.

L45

```
45 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L47

```
47 |   @GetMapping("/operators")
```

Ruta /operators combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L48

```
48 |   @Operation(summary = "ADMIN: consultar operadores paginados y filtrar por registrationStatus")
```

Documenta en OpenAPI: ADMIN: consultar operadores paginados y filtrar por registrationStatus. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L49 a L52

```
49 |   public PageView<OperatorView> operators(
50 |       @RequestParam(defaultValue = "0") int page,
51 |       @RequestParam(defaultValue = "20") int size,
52 |       @RequestParam(required = false) Registration registrationStatus) {
```

Lee parámetro/filtro URL y su default. Campos: registrationStatus: alta administrativa ACTIVE/INACTIVE. Tipos y datos: page = page; int es entero de 32 bits para límites y contadores; size = size; int es entero de 32 bits para límites y contadores.

L52: Dato del contrato: registrationStatus: alta administrativa ACTIVE/INACTIVE.

L53

```
53 |     return service.operators(page, size, registrationStatus);
```

Consulta paginada de operadores con filtro administrativo; solo el endpoint ADMIN la expone. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L54

```
54 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L56

```
56 |   @GetMapping("/operators/{id}")
```

Ruta /operators/{id} combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L57

```
57 |   @Operation(summary = "ADMIN: consultar un operador")
```

Documenta en OpenAPI: ADMIN: consultar un operador. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L58

```
58 |   public OperatorView operator(@PathVariable UUID id) {
```

Extrae ID de la ruta. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L59

```
59 |     return service.getOperator(id);
```

Obtiene el perfil administrativo del operador por UUID y lo proyecta a DTO. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L60

```
60 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L62

```
62 |   @PatchMapping("/operators/{id}")
```

Ruta /operators/{id} combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L63

```
63 |   @Operation(summary = "ADMIN: actualizar nombre o teléfono; subject inmutable")
```

Documenta en OpenAPI: ADMIN: actualizar nombre o teléfono; subject inmutable. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L64

```
64 |   public OperatorView editOperator(@PathVariable UUID id, @Valid @RequestBody EditOperator d) {
```

Extrae ID de la ruta. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L65

```
65 |     return service.edit(id, d);
```

Actualiza solo campos admitidos después de validar restricciones y recursos comprometidos; no sustituye la identidad Keycloak. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L66

```
66 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L68

```
68 |   @PatchMapping("/operators/{id}/registration-status")
```

Ruta /operators/{id}/registration-status combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L69

```
69 |   @Operation(summary = "ADMIN: activar o inactivar el perfil; inactivar retira su
     disponibilidad")
```

Documenta en OpenAPI: ADMIN: activar o inactivar el perfil; inactivar retira su disponibilidad. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L70 a L71

```
70 |   public OperatorView registration(
71 |       @PathVariable UUID id, @Valid @RequestBody RegistrationChange d) {
```

Extrae ID de la ruta. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L71: Dato del contrato: id: identificador UUID del registro.

L72

```
72 |     return service.registration(id, d);
```

Cambia ACTIVE/INACTIVE del perfil y retira disponibilidad al inactivarlo si las reglas permiten la operación. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L73

```
73 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L75

```
75 |   @PutMapping("/operators/{id}/qualification")
```

Ruta /operators/{id}/qualification combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L76

```
76 |   @Operation(summary = "ADMIN: registrar o renovar la habilitación LIGHT_TOWING")
```

Documenta en OpenAPI: ADMIN: registrar o renovar la habilitación LIGHT_TOWING. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L77 a L78

```
77 |   public OperatorView qualification(
78 |       @PathVariable UUID id, @Valid @RequestBody QualificationChange d) {
```

Extrae ID de la ruta. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L78: Dato del contrato: id: identificador UUID del registro.

L79

```
79 |     return service.qualification(id, d);
```

Registra categoría y período de vigencia de la habilitación; esa vigencia se usa al decidir elegibilidad. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L80

```
80 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L82

```
82 |   @GetMapping("/operators/me")
```

Ruta /operators/me combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L83

```
83 |   @Operation(summary = "OPERATOR: consultar el perfil asociado al sub del JWT")
```

Documenta en OpenAPI: OPERATOR: consultar el perfil asociado al sub del JWT. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L84

```
84 |   public OperatorView me(@AuthenticationPrincipal Jwt jwt) {
```

Obtiene JWT validado, no identidad suministrada por el cliente. Tipos y datos: jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias.

L85

```
85 |     return service.me(jwt.getSubject());
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Resultado que recibe el llamador: obtiene sub, identidad estable del usuario autenticado.

L86

```
86 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L88

```
88 |   @PatchMapping("/operators/me/availability")
```

Ruta /operators/me/availability combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L89

```
89 |   @Operation(summary = "OPERATOR: declarar disponibilidad propia")
```

Documenta en OpenAPI: OPERATOR: declarar disponibilidad propia. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L90 a L91

```
90 |   public OperatorView availability(
91 |       @AuthenticationPrincipal Jwt jwt, @Valid @RequestBody AvailabilityChange d) {
```

Obtiene JWT validado, no identidad suministrada por el cliente. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Tipos y datos: jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias.

L92

```
92 |     return service.availability(jwt.getSubject(), d);
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Resultado que recibe el llamador: obtiene sub, identidad estable del usuario autenticado.

L93

```
93 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L95

```
95 |   @ApiResponse(responseCode = "201", description = "Grúa creada; Location contiene su URL")
```

Documenta código HTTP.

L96

```
96 |   @PostMapping("/tow-trucks")
```

Ruta /tow-trucks combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L97

```
97 |   @Operation(summary = "ADMIN: registrar una grúa INACTIVE, sin operador ni ubicación")
```

Documenta en OpenAPI: ADMIN: registrar una grúa INACTIVE, sin operador ni ubicación. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L98

```
98 |   public ResponseEntity<TruckView> createTruck(@Valid @RequestBody CreateTruck d) {
```

Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON.

L99

```
99 |     var t = service.create(d);
```

Se calcula t (t) para reutilizar ese resultado en los pasos siguientes.

L100

```
100 |     return ResponseEntity.created(URI.create("/api/v1/tow-trucks/" + t.id())).body(t);
```

Responde 201 con Location. Resultado que recibe el llamador: crea una respuesta 201 con dirección Location del recurso; Datos implicados: identificador UUID del registro.

L101

```
101 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L103

```
103 |   @GetMapping("/tow-trucks")
```

Ruta /tow-trucks combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L104

```
104 |   @Operation(summary = "ADMIN: consultar grúas paginadas y filtrar por operationalStatus")
```

Documenta en OpenAPI: ADMIN: consultar grúas paginadas y filtrar por operationalStatus. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L105 a L108

```
105 |   public PageView<TruckView> trucks(
106 |       @RequestParam(defaultValue = "0") int page,
107 |       @RequestParam(defaultValue = "20") int size,
108 |       @RequestParam(required = false) Operational operationalStatus) {
```

Lee parámetro/filtro URL y su default. Campos: operationalStatus: estado administrativo de la grúa. Tipos y datos: page = page; int es entero de 32 bits para límites y contadores; size = size; int es entero de 32 bits para límites y contadores.

L108: Dato del contrato: operationalStatus: estado administrativo de la grúa.

L109

```
109 |     return service.trucks(page, size, operationalStatus);
```

Consulta paginada de grúas y su estado administrativo. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L110

```
110 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L112

```
112 |   @GetMapping("/tow-trucks/{id}")
```

Ruta /tow-trucks/{id} combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L113

```
113 |   @Operation(summary = "ADMIN: consultar capacidades, vínculo y ubicación de una grúa")
```

Documenta en OpenAPI: ADMIN: consultar capacidades, vínculo y ubicación de una grúa. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L114

```
114 |   public TruckView truck(@PathVariable UUID id) {
```

Extrae ID de la ruta. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L115

```
115 |     return service.getTruck(id);
```

Obtiene capacidad, tipos, vínculo y ubicación de la grúa para consulta administrativa. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L116

```
116 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L118

```
118 |   @PatchMapping("/tow-trucks/{id}")
```

Ruta /tow-trucks/{id} combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L119

```
119 |   @Operation(summary = "ADMIN: actualizar datos y capacidades de una grúa")
```

Documenta en OpenAPI: ADMIN: actualizar datos y capacidades de una grúa. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L120

```
120 |   public TruckView editTruck(@PathVariable UUID id, @Valid @RequestBody EditTruck d) {
```

Extrae ID de la ruta. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L121

```
121 |     return service.edit(id, d);
```

Actualiza solo campos admitidos después de validar restricciones y recursos comprometidos; no sustituye la identidad Keycloak. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L122

```
122 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L124

```
124 |   @PutMapping("/tow-trucks/{id}/operator")
```

Ruta /tow-trucks/{id}/operator combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L125 a L127

```
125 |   @Operation(
126 |       summary = "ADMIN: vincular un operador; rechaza sustitución implícita y vínculos
      duplicados")
127 |   public TruckView link(@PathVariable UUID id, @Valid @RequestBody LinkOperator d) {
```

Documenta en OpenAPI: ADMIN: vincular un operador; rechaza sustitución implícita y vínculos duplicados. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L128

```
128 |     return service.link(id, d);
```

Vincula operador y grúa comprobando exclusividad y ausencia de compromiso incompatible. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L129

```
129 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L131

```
131 |   @ApiResponse(responseCode = "204", description = "Vínculo retirado")
```

Documenta código HTTP.

L132

```
132 |   @DeleteMapping("/tow-trucks/{id}/operator")
```

Ruta /tow-trucks/{id}/operator combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L133

```
133 |   @Operation(summary = "ADMIN: desvincular el operador; repetición sin vínculo devuelve 204")
```

Documenta en OpenAPI: ADMIN: desvincular el operador; repetición sin vínculo devuelve 204. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L134

```
134 |   public ResponseEntity<Void> unlink(@PathVariable UUID id) {
```

Extrae ID de la ruta. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L135

```
135 |     service.unlink(id);
```

Desvincula operador de una grúa libre; repetir sobre un vínculo ausente no crea un efecto extra. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L136

```
136 |     return ResponseEntity.noContent().build();
```

Construye el objeto final a partir del builder configurado en líneas previas. Resultado que recibe el llamador: crea una respuesta 204 sin cuerpo.

L137

```
137 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L139

```
139 |   @PatchMapping("/tow-trucks/{id}/operational-status")
```

Ruta /tow-trucks/{id}/operational-status combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L140 a L146

```
140 |   @Operation(
141 |       summary = "ADMIN: cambiar estado con motivo e Idempotency-Key; devuelve el evento durable")
142 |   public StatusView status(
143 |       @PathVariable UUID id,
144 |       @Valid @RequestBody StatusChange d,
145 |       @RequestHeader("Idempotency-Key") String key,
146 |       @AuthenticationPrincipal Jwt jwt) {
```

Documenta en OpenAPI: ADMIN: cambiar estado con motivo e Idempotency-Key; devuelve el evento durable. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L143: Dato del contrato: id: identificador UUID del registro.

L147

```
147 |     return service.status(id, d, jwt.getSubject(), key);
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Resultado que recibe el llamador: obtiene sub, identidad estable del usuario autenticado.

L148

```
148 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L150

```
150 |   @GetMapping("/tow-trucks/assigned-to-me")
```

Ruta /tow-trucks/assigned-to-me combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L151

```
151 |   @Operation(summary = "OPERATOR: consultar exclusivamente su grúa vinculada")
```

Documenta en OpenAPI: OPERATOR: consultar exclusivamente su grúa vinculada. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L152

```
152 |   public TruckView assigned(@AuthenticationPrincipal Jwt jwt) {
```

Obtiene JWT validado, no identidad suministrada por el cliente. Tipos y datos: jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias.

L153

```
153 |     return service.assigned(jwt.getSubject());
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Resultado que recibe el llamador: obtiene sub, identidad estable del usuario autenticado.

L154

```
154 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L156

```
156 |   @PutMapping("/tow-trucks/assigned-to-me/location")
```

Ruta /tow-trucks/assigned-to-me/location combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L157 a L159

```
157 |   @Operation(
158 |       summary = "OPERATOR: registrar coordenadas de su grúa; el servidor fija locationUpdatedAt")
159 |   public TruckView location(@AuthenticationPrincipal Jwt jwt, @Valid @RequestBody Location d) {
```

Documenta en OpenAPI: OPERATOR: registrar coordenadas de su grúa; el servidor fija locationUpdatedAt. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L160

```
160 |     return service.location(jwt.getSubject(), d);
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Resultado que recibe el llamador: obtiene sub, identidad estable del usuario autenticado.

L161

```
161 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L163

```
163 |   @GetMapping("/configuration/fleet")
```

Ruta /configuration/fleet combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L164

```
164 |   @Operation(summary = "ADMIN: consultar límites efectivos de administración de Fleet")
```

Documenta en OpenAPI: ADMIN: consultar límites efectivos de administración de Fleet. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L165

```
165 |   public AdministrationRules configuration() {
```

Endpoint de consulta de reglas efectivas para ADMIN. Permite verificar límites/perfil sin devolver contraseñas ni claves externas.

L166

```
166 |     return new AdministrationRules("AR-MVP-5", 500, 5000, 100, Qualification.LIGHT_TOWING);
```

Devuelve límites administrativos 500..5000 kg, paginación máxima 100 y categoría LIGHT_TOWING.

L167

```
167 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L169 a L174

```
169 |   public record AdministrationRules(
170 |       String rulesVersion,
171 |       int minTruckCapacityKg,
172 |       int maxTruckCapacityKg,
173 |       int maxPageSize,
174 |       Qualification requiredQualification) {}
```

Declara record inmutable AdministrationRules. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. API pública de flota: ADMIN administra recursos y OPERATOR opera su propio perfil/equipo. Delega las reglas y transacciones en FleetService. Tipos y datos: rulesVersion = rulesVersion; String es texto; no es por sí mismo una referencia de entidad; minTruckCapacityKg = minTruckCapacityKg; int es entero de 32 bits para límites y contadores; maxTruckCapacityKg = maxTruckCapacityKg; int es entero de 32 bits para límites y contadores; maxPageSize = maxPageSize; int es entero de 32 bits para límites y contadores.

L175

```
175 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 16, 33, 37, 46, 55, 61, 67, 74, 81, 87, 94, 102, 111, 117, 123, 130, 138, 149, 155, 162, 168: separadores sin instrucciones.

FleetDtos (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/FleetDtos.java`

Objetivo y funcionalidad

Contratos de entrada/salida de Fleet mediante records y Bean Validation. Los DTO no son tablas y evitan exponer directamente entidades JPA.

L1

```
1 | package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import jakarta.validation.constraints.*;
```

Importa tipos: jakarta.validation.constraints.*. Restricciones de DTO y activación de validación de entradas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
 7 | import static com.auxiruta.fleet.catalog.FleetTypes.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetTypes.*. Tipos y DTO locales de Fleet. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
 9 | public final class FleetDtos {
```

Declara clase FleetDtos. Contratos de entrada/salida de Fleet mediante records y Bean Validation. Los DTO no son tablas y evitan exponer directamente entidades JPA.

L10

```
10 |   private FleetDtos() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L12 a L15

```
12 |   public record CreateOperator(
13 |       @NotBlank @Size(max = 100) String keycloakSubject,
14 |       @NotBlank @Size(max = 150) String fullName,
15 |       @Size(max = 40) String contactPhone) {}
```

Exige texto no vacío ni solo espacios. Limita caracteres/elementos según min/max. Declara record inmutable CreateOperator. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: keycloakSubject: identidad externa del operador; fullName: nombre del perfil de negocio; contactPhone: teléfono opcional. Contratos de entrada/salida de Fleet mediante records y Bean Validation. Los DTO no son tablas y evitan exponer directamente entidades JPA. Tipos y datos: keycloakSubject = identidad externa del operador; String es texto; no es por sí mismo una referencia de entidad; fullName = nombre del perfil de negocio; String es texto; no es por sí mismo una referencia de entidad; contactPhone = teléfono opcional; String es texto; no es por sí mismo una referencia de entidad.

L13: Dato del contrato: keycloakSubject: identidad externa del operador.

L14: Dato del contrato: fullName: nombre del perfil de negocio.

L15: Dato del contrato: contactPhone: teléfono opcional.

L17 a L18

```
17 |   public record EditOperator(
18 |       @Size(min = 1, max = 150) String fullName, @Size(max = 40) String contactPhone) {}
```

Limita caracteres/elementos según min/max. Declara record inmutable EditOperator. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: fullName: nombre del perfil de negocio; contactPhone: teléfono opcional. Se calcula max (max) para reutilizar ese resultado en los pasos siguientes. Contratos de entrada/salida de Fleet mediante records y Bean Validation. Los DTO no son tablas y evitan exponer directamente entidades JPA. Tipos y datos: fullName = nombre del perfil de negocio; String es texto; no es por sí mismo una referencia de entidad; contactPhone = teléfono opcional; String es texto; no es por sí mismo una referencia de entidad.

L18: Dato del contrato: fullName: nombre del perfil de negocio; contactPhone: teléfono opcional.

L20

```
20 |   public record RegistrationChange(@NotNull Registration status) {}
```

Exige no null. Declara record inmutable RegistrationChange. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Contratos de entrada/salida de Fleet mediante records y Bean Validation. Los DTO no son tablas y evitan exponer directamente entidades JPA.

L22

```
22 |   public record AvailabilityChange(@NotNull Availability status) {}
```

Exige no null. Declara record inmutable AvailabilityChange. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Contratos de entrada/salida de Fleet mediante records y Bean Validation. Los DTO no son tablas y evitan exponer directamente entidades JPA.

L24 a L25

```
24 |   public record QualificationChange(
25 |       @NotNull Qualification category, @NotNull Instant validFrom, @NotNull Instant validUntil)
     {}
```

Exige no null. Declara record inmutable QualificationChange. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Contratos de entrada/salida de Fleet mediante records y Bean Validation. Los DTO no son tablas y evitan exponer directamente entidades JPA. Tipos y datos: validFrom = validFrom; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; validUntil = límite temporal de validez de evidencia o autorización; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L27 a L32

```
27 |   public record CreateTruck(
28 |       @NotBlank @Size(max = 30) String licensePlate,
29 |       @NotNull @DecimalMin("500") @DecimalMax("5000") @Digits(integer = 4, fraction = 2)
30 |           BigDecimal maximumLoadKg,
31 |       @NotNull Boolean supportsNonRollable,
32 |       @NotEmpty Set<@NotNull VehicleType> supportedTypes) {}
```

Exige texto no vacío ni solo espacios. Limita caracteres/elementos según min/max. Exige no null. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Exige colección/texto no vacío. Declara record inmutable CreateTruck. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: licensePlate: patente normalizada y única; maximumLoadKg: capacidad de carga de la grúa en kg; supportsNonRollable: capacidad para atender vehículos que no ruedan. Se calcula fraction (fraction) para reutilizar ese resultado en los pasos siguientes. Contratos de entrada/salida de Fleet mediante records y Bean Validation. Los DTO no son tablas y evitan exponer directamente entidades JPA. Tipos y datos: licensePlate = patente normalizada y única; String es texto; no es por sí mismo una referencia de entidad; maximumLoadKg = capacidad de carga de la grúa en kg; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; supportsNonRollable = capacidad para atender vehículos que no ruedan; Boolean es booleano envoltorio que puede ser null y validarse con NotNull.

L28: Dato del contrato: licensePlate: patente normalizada y única.

L30: Dato del contrato: maximumLoadKg: capacidad de carga de la grúa en kg.

L31: Dato del contrato: supportsNonRollable: capacidad para atender vehículos que no ruedan.

L34 a L39

```
34 |   public record EditTruck(
35 |       @Size(max = 30) String licensePlate,
36 |       @DecimalMin("500") @DecimalMax("5000") @Digits(integer = 4, fraction = 2)
37 |           BigDecimal maximumLoadKg,
38 |       Boolean supportsNonRollable,
39 |       @Size(min = 1) Set<@NotNull VehicleType> supportedTypes) {}
```

Limita caracteres/elementos según min/max. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Exige no null. Declara record inmutable EditTruck. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: licensePlate: patente normalizada y única; maximumLoadKg: capacidad de carga de la grúa en kg; supportsNonRollable: capacidad para atender vehículos que no ruedan. Se calcula fraction (fraction) para reutilizar ese resultado en los pasos siguientes. Contratos de entrada/salida de Fleet mediante records y Bean Validation. Los DTO no son tablas y evitan exponer directamente entidades JPA. Tipos y datos: licensePlate = patente normalizada y única; String es texto; no es por sí mismo una referencia de entidad; maximumLoadKg = capacidad de carga de la grúa en kg; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; supportsNonRollable = capacidad para atender vehículos que no ruedan; Boolean es booleano envoltorio que puede ser null y validarse con NotNull.

L35: Dato del contrato: licensePlate: patente normalizada y única.

L37: Dato del contrato: maximumLoadKg: capacidad de carga de la grúa en kg.

L41

```
41 |   public record LinkOperator(@NotNull UUID operatorId) {}
```

Exige no null. Declara record inmutable LinkOperator. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: operatorId: UUID del perfil operador. Contratos de entrada/salida de Fleet mediante records y Bean Validation. Los DTO no son tablas y evitan exponer directamente entidades JPA. Tipos y datos: operatorId = UUID del perfil operador; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L43 a L44

```
43 |   public record StatusChange(
44 |       @NotNull Operational status, @NotBlank @Size(max = 500) String reason) {}
```

Exige no null. Exige texto no vacío ni solo espacios. Limita caracteres/elementos según min/max. Declara record inmutable StatusChange. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: status: estado del objeto; su enum o CHECK define valores permitidos; reason: motivo legible. Contratos de entrada/salida de Fleet mediante records y Bean Validation. Los DTO no son tablas y evitan exponer directamente entidades JPA. Tipos y datos: reason = motivo legible; String es texto; no es por sí mismo una referencia de entidad.

L44: Dato del contrato: status: estado del objeto; su enum o CHECK define valores permitidos; reason: motivo legible.

L46 a L50

```
46 |   public record Location(
47 |       @NotNull @DecimalMin("-90") @DecimalMax("90") @Digits(integer = 3, fraction = 6)
48 |           BigDecimal latitude,
49 |       @NotNull @DecimalMin("-180") @DecimalMax("180") @Digits(integer = 3, fraction = 6)
50 |           BigDecimal longitude) {}
```

Exige no null. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Declara record inmutable Location. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: latitude: latitud declarada; longitude: longitud declarada. Se calcula fraction (fraction) para reutilizar ese resultado en los pasos siguientes. Contratos de entrada/salida de Fleet mediante records y Bean Validation. Los DTO no son tablas y evitan exponer directamente entidades JPA. Tipos y datos: latitude = latitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; longitude = longitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L48: Dato del contrato: latitude: latitud declarada.

L50: Dato del contrato: longitude: longitud declarada.

L52 a L60

```
52 |   public record StatusView(
53 |       UUID id,
54 |       UUID towTruckId,
55 |       Operational previousStatus,
56 |       Operational newStatus,
57 |       String reason,
58 |       String changedBySubject,
59 |       String operationKey,
60 |       Instant changedAt) {}
```

Declara record inmutable StatusView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; towTruckId: UUID de grúa; previousStatus: estado previo; newStatus: estado resultante; reason: motivo legible; changedBySubject: sub del administrador responsable; operationKey: clave durable de acción; changedAt: instante del cambio. Contratos de entrada/salida de Fleet mediante records y Bean Validation. Los DTO no son tablas y evitan exponer directamente entidades JPA. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; towTruckId = UUID de grúa; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; reason = motivo legible; String es texto; no es por sí mismo una referencia de entidad; changedBySubject = sub del administrador responsable; String es texto; no es por sí mismo una referencia de entidad; operationKey = clave durable de acción; String es texto; no es por sí mismo una referencia de entidad; changedAt = instante del cambio; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L53: Dato del contrato: id: identificador UUID del registro.

L54: Dato del contrato: towTruckId: UUID de grúa.

L57: Dato del contrato: reason: motivo legible.

L58: Dato del contrato: changedBySubject: sub del administrador responsable.

L59: Dato del contrato: operationKey: clave durable de acción.

L60: Dato del contrato: changedAt: instante del cambio.

L62 a L75

```
62 |   public record OperatorView(
63 |       UUID id,
64 |       String keycloakSubject,
65 |       String fullName,
66 |       String contactPhone,
67 |       Registration registrationStatus,
68 |       Availability availabilityStatus,
69 |       Qualification qualificationCategory,
70 |       Instant qualificationValidFrom,
71 |       Instant qualificationValidUntil,
72 |       Instant availableSince,
73 |       Instant createdAt,
74 |       Instant updatedAt,
75 |       long version) {}
```

Declara record inmutable OperatorView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; keycloakSubject: identidad externa del operador; fullName: nombre del perfil de negocio; contactPhone: teléfono opcional; registrationStatus: alta administrativa ACTIVE/INACTIVE; availabilityStatus: disposición a recibir propuestas; qualificationCategory: habilitación LIGHT_TOWING; qualificationValidFrom: inicio de habilitación; qualificationValidUntil: fin de habilitación que debe superar el servicio estimado; availableSince: inicio de disponibilidad usado para desempate; createdAt: fecha de creación; updatedAt: fecha de modificación; version: versión para control optimista de concurrencia. Contratos de entrada/salida de Fleet mediante records y Bean Validation. Los DTO no son tablas y evitan exponer directamente entidades JPA. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; keycloakSubject = identidad externa del operador; String es texto; no es por sí mismo una referencia de entidad; fullName = nombre del perfil de negocio; String es texto; no es por sí mismo una referencia de entidad; contactPhone = teléfono opcional; String es texto; no es por sí mismo una referencia de entidad; qualificationValidFrom = inicio de habilitación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; qualificationValidUntil = fin de habilitación que debe superar el servicio estimado; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; availableSince = inicio de disponibilidad usado para desempate; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; createdAt = fecha de creación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; updatedAt = fecha de modificación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; version = versión para control optimista de concurrencia; long es entero de 64 bits para cantidades y duraciones mayores.

L63: Dato del contrato: id: identificador UUID del registro.

L64: Dato del contrato: keycloakSubject: identidad externa del operador.

L65: Dato del contrato: fullName: nombre del perfil de negocio.

L66: Dato del contrato: contactPhone: teléfono opcional.

L70: Dato del contrato: qualificationValidFrom: inicio de habilitación.

L71: Dato del contrato: qualificationValidUntil: fin de habilitación que debe superar el servicio estimado.

L72: Dato del contrato: availableSince: inicio de disponibilidad usado para desempate.

L73: Dato del contrato: createdAt: fecha de creación.

L74: Dato del contrato: updatedAt: fecha de modificación.

L75: Dato del contrato: version: versión para control optimista de concurrencia.

L77 a L90

```
77 |   public record TruckView(
78 |       UUID id,
79 |       String licensePlate,
80 |       UUID operatorId,
81 |       BigDecimal maximumLoadKg,
82 |       boolean supportsNonRollable,
83 |       Set<VehicleType> supportedTypes,
84 |       Operational operationalStatus,
85 |       BigDecimal latitude,
86 |       BigDecimal longitude,
87 |       Instant locationUpdatedAt,
88 |       Instant createdAt,
89 |       Instant updatedAt,
90 |       long version) {}
```

Declara record inmutable TruckView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; licensePlate: patente normalizada y única; operatorId: UUID del perfil operador; maximumLoadKg: capacidad de carga de la grúa en kg; supportsNonRollable: capacidad para atender vehículos que no ruedan; operationalStatus: estado administrativo de la grúa; latitude: latitud declarada; longitude: longitud declarada; locationUpdatedAt: instante servidor de última ubicación declarada; createdAt: fecha de creación; updatedAt: fecha de modificación; version: versión para control optimista de concurrencia. Contratos de entrada/salida de Fleet mediante records y Bean Validation. Los DTO no son tablas y evitan exponer directamente entidades JPA. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; licensePlate = patente normalizada y única; String es texto; no es por sí mismo una referencia de entidad; operatorId = UUID del perfil operador; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; maximumLoadKg = capacidad de carga de la grúa en kg; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; supportsNonRollable = capacidad para atender vehículos que no ruedan; boolean es verdadero/falso, sin null; latitude = latitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; longitude = longitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; locationUpdatedAt = instante servidor de última ubicación declarada; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; createdAt = fecha de creación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; updatedAt = fecha de modificación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; version = versión para control optimista de concurrencia; long es entero de 64 bits para cantidades y duraciones mayores.

L78: Dato del contrato: id: identificador UUID del registro.

L79: Dato del contrato: licensePlate: patente normalizada y única.

L80: Dato del contrato: operatorId: UUID del perfil operador.

L81: Dato del contrato: maximumLoadKg: capacidad de carga de la grúa en kg.

L82: Dato del contrato: supportsNonRollable: capacidad para atender vehículos que no ruedan.

L85: Dato del contrato: latitude: latitud declarada.

L86: Dato del contrato: longitude: longitud declarada.

L87: Dato del contrato: locationUpdatedAt: instante servidor de última ubicación declarada.

L88: Dato del contrato: createdAt: fecha de creación.

L89: Dato del contrato: updatedAt: fecha de modificación.

L90: Dato del contrato: version: versión para control optimista de concurrencia.

L92 a L93

```
92 |   public record PageView<T>(
93 |       List<T> content, int page, int size, long totalElements, int totalPages) {}
```

Declara record inmutable PageView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Contratos de entrada/salida de Fleet mediante records y Bean Validation. Los DTO no son tablas y evitan exponer directamente entidades JPA. Tipos y datos: page = page; int es entero de 32 bits para límites y contadores; size = size; int es entero de 32 bits para límites y contadores; totalElements = totalElements; long es entero de 64 bits para cantidades y duraciones mayores; totalPages = totalPages; int es entero de 32 bits para límites y contadores.

L94

```
94 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 8, 11, 16, 19, 21, 23, 26, 33, 40, 42, 45, 51, 61, 76, 91: separadores sin instrucciones.

FleetService (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/FleetService.java`

Objetivo y funcionalidad

Implementa administración de flota, propiedad, vínculos exclusivos, disponibilidad y ubicación. Los cambios de recursos ocupados fallan con 409; bloquear la grúa evita moverla mientras existe HELD/CONFIRMED.

L1

```
1 | package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.data.domain.*;
```

Importa tipos: org.springframework.data.domain.*. Repositorios Spring Data y estructuras de paginación/orden. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.springframework.http.HttpStatus;
```

Importa tipos: org.springframework.http.HttpStatus. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import org.slf4j.LoggerFactory;
```

Importa tipos: org.slf4j.LoggerFactory. Logging de eventos y fallos sin acoplarse a una implementación de consola. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | import static com.auxiruta.fleet.catalog.FleetDtos.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetDtos.*. Tipos y DTO locales de Fleet. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12

```
12 | import static com.auxiruta.fleet.catalog.FleetTypes.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetTypes.*. Tipos y DTO locales de Fleet. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L14

```
14 | @Service
```

Registra lógica de aplicación como bean. El escaneo de Spring crea este componente e inyecta sus dependencias; no hay que construirlo manualmente en el controlador.

L15

```
15 | @Transactional(readOnly = true)
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L16

```
16 | public class FleetService {
```

Declara clase FleetService. Implementa administración de flota, propiedad, vínculos exclusivos, disponibilidad y ubicación. Los cambios de recursos ocupados fallan con 409; bloquear la grúa evita moverla mientras existe HELD/CONFIRMED.

L17

```
17 |   private final OperatorRepository operators;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L18

```
18 |   private final TowTruckRepository trucks;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L19

```
19 |   private final TruckStatusHistoryRepository history;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L20

```
20 |   private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L22

```
22 |   @org.springframework.beans.factory.annotation.Autowired
```

Inyecta dependencia Spring.

L23

```
23 |   private ReservationRepository reservations;
```

Dependencia inyectada que consulta reservas activas antes de editar recursos. Evita modificar un operador o grúa que ya está retenido o confirmado.

L25

```
25 |   private void freeTruck(UUID id) {
```

Impide mutar una grúa retenida o confirmada. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L26

```
26 |     require(!reservations.busy(id, null), HttpStatus.CONFLICT);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. 409 comunica que el estado/regla actual impide la acción; ! invierte el booleano de la expresión que sigue

L27

```
27 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L29

```
29 |   private void freeOperator(UUID id) {
```

Impide mutar un operador retenido o confirmado. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L30

```
30 |     require(!reservations.busy(null, id), HttpStatus.CONFLICT);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. 409 comunica que el estado/regla actual impide la acción; ! invierte el booleano de la expresión que sigue

L31

```
31 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L33 a L37

```
33 |   public FleetService(
34 |       OperatorRepository operators,
35 |       TowTruckRepository trucks,
36 |       TruckStatusHistoryRepository history,
37 |       Clock clock) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L38

```
38 |     this.operators = operators;
```

En this guarda repositorio de perfiles de operadores. Toma el dato de operators después de las validaciones de este método. repositorio de perfiles de operadores.

L39

```
39 |     this.trucks = trucks;
```

En this guarda repositorio de grúas. Toma el dato de trucks después de las validaciones de este método. repositorio de grúas.

L40

```
40 |     this.history = history;
```

En this guarda historial persistente de cambios. Toma el dato de history después de las validaciones de este método. historial persistente de cambios.

L41

```
41 |     this.clock = clock;
```

En this guarda reloj usado para evaluar plazos. Toma el dato de clock después de las validaciones de este método. reloj usado para evaluar plazos.

L42

```
42 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L44

```
44 |   private void committed(String event, UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: event = event; String es texto; no es por sí mismo una referencia de entidad; id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L45 a L53

```
45 |     org.springframework.transaction.support.TransactionSynchronizationManager
46 |         .registerSynchronization(
47 |             new org.springframework.transaction.support.TransactionSynchronization() {
48 |               @Override
49 |               public void afterCommit() {
50 |                 LoggerFactory.getLogger(FleetService.class)
51 |                     .info("Fleet event={} resourceId={}", event, id);
52 |               }
53 |             });
```

Registra un callback del ciclo transaccional; no ejecuta el log antes del commit. Implementa callback afterCommit para anunciar solo cambios confirmados. Se calcula event (event) para reutilizar ese resultado en los pasos siguientes.

L54

```
54 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L56

```
56 |   private Instant now() {
```

Obtiene el instante del reloj inyectado; truncar a microsegundos mantiene consistencia con PostgreSQL.

L57

```
57 |     return clock.instant().truncatedTo(java.time.temporal.ChronoUnit.MICROS);
```

Resultado que recibe el llamador: lee el instante del reloj inyectado.

L58

```
58 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L60

```
60 |   private static ResponseStatusException missing() {
```

Construye 404; puede representar recurso inexistente o ajeno para no revelar su existencia.

L61

```
61 |     return new ResponseStatusException(HttpStatus.NOT_FOUND);
```

Resultado que recibe el llamador: interrumpe la operación con un estado HTTP explícito; 404 oculta inexistencia o pertenencia ajena.

L62

```
62 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L64

```
64 |   private static void require(boolean condition, HttpStatus status) {
```

Expresa una precondición; si resulta falsa se aborta con el estado HTTP indicado. Tipos y datos: condition = condition; boolean es verdadero/falso, sin null.

L65

```
65 |     if (!condition) throw new ResponseStatusException(status);
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. interrumpe la operación con un estado HTTP explícito; ! invierte el booleano de la expresión que sigue.

L66

```
66 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L68

```
68 |   private Operator operator(UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L69

```
69 |     return operators.findById(id).orElseThrow(FleetService::missing);
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Resultado que recibe el llamador: si no existe valor, lanza la excepción en vez de seguir con null; busca por clave primaria; el resultado puede faltar.

L70

```
70 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L72

```
72 |   private Operator own(String subject) {
```

Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad.

L73

```
73 |     return operators.findByKeycloakSubject(subject).orElseThrow(FleetService::missing);
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Resultado que recibe el llamador: si no existe valor, lanza la excepción en vez de seguir con null.

L74

```
74 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L76

```
76 |   private Operator lockedOperator(UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L77 a L78

```
77 |     return org.hibernate.Hibernate.unproxy(
78 |         operators.lockById(id).orElseThrow(FleetService::missing), Operator.class);
```

Si no encuentra resultado lanza la excepción indicada. Resultado que recibe el llamador: si no existe valor, lanza la excepción en vez de seguir con null.

L79

```
79 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L81

```
81 |   private TowTruck truck(UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L82

```
82 |     return trucks.findById(id).orElseThrow(FleetService::missing);
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Resultado que recibe el llamador: si no existe valor, lanza la excepción en vez de seguir con null; busca por clave primaria; el resultado puede faltar.

L83

```
83 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L85

```
85 |   private TowTruck lockedTruck(UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L86

```
86 |     return trucks.lockById(id).orElseThrow(FleetService::missing);
```

Si no encuentra resultado lanza la excepción indicada. Resultado que recibe el llamador: si no existe valor, lanza la excepción en vez de seguir con null.

L87

```
87 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L89

```
89 |   private TowTruck ownTruck(String subject) {
```

Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad.

L90

```
90 |     return trucks.findByOperatorId(own(subject).id).orElseThrow(FleetService::missing);
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Resultado que recibe el llamador: si no existe valor, lanza la excepción en vez de seguir con null.

L91

```
91 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L93

```
93 |   private static String plate(String value) {
```

Quita espacios/guiones, convierte a mayúsculas y exige entre 6 y 10 caracteres alfanuméricos. Tipos y datos: value = value; String es texto; no es por sí mismo una referencia de entidad.

L94

```
94 |     String normalized = value.replace(" ", "").replace("-", "").toUpperCase(Locale.ROOT);
```

Normaliza patente quitando espacios/guiones y convirtiendo a mayúsculas con Locale.ROOT. Se calcula normalized (normalized) para reutilizar ese resultado en los pasos siguientes.

L95

```
95 |     require(normalized.matches("[A-Z0-9]{6,10}"), HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. 400 indica entrada inválida

L96

```
96 |     return normalized;
```

Entrega normalized al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: normalized.

L97

```
97 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L99

```
99 |   private static String name(String value) {
```

Tipos y datos: value = value; String es texto; no es por sí mismo una referencia de entidad.

L100

```
100 |     require(!value.isBlank(), HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. comprueba texto vacío o solo espacios; 400 indica entrada inválida; ! invierte el booleano de la expresión que sigue

L101

```
101 |     return value.strip();
```

Elimina espacios Unicode al comienzo y al final para normalizar el dato; no elimina espacios internos.

L102

```
102 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L104

```
104 |   private static Pageable page(int page, int size) {
```

Valida número/tamaño y aplica orden estable para paginar. Tipos y datos: size = size; int es entero de 32 bits para límites y contadores.

L105

```
105 |     require(page >= 0 && size >= 1 && size <= 100, HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. 400 indica entrada inválida; && exige todas las condiciones y deja de evaluar al encontrar una falsa

L106

```
106 |     return PageRequest.of(page, size, Sort.by("createdAt").ascending().and(Sort.by("id")));
```

Construye página ordenada por fecha y UUID, estable incluso cuando coinciden fechas.

L107

```
107 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L109

```
109 |   private static <T> PageView<T> result(Page<T> p) {
```

Método genérico que proyecta Page de Spring a PageView del contrato público. T conserva el tipo de cada elemento y expone metadatos sin depender de la serialización interna de Spring.

L110 a L111

```
110 |     return new PageView<>(
111 |         p.getContent(), p.getNumber(), p.getSize(), p.getTotalElements(), p.getTotalPages());
```

Proyecta contenido, página, tamaño, total de elementos y páginas al contrato REST.

L112

```
112 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L114

```
114 |   private OperatorView view(Operator o) {
```

Transforma entidad/datos internos en DTO; no cambia la base por realizar esta proyección.

L115 a L128

```
115 |     return new OperatorView(
116 |         o.id,
117 |         o.keycloakSubject,
118 |         o.fullName,
119 |         o.contactPhone,
120 |         o.registrationStatus,
121 |         o.availabilityStatus,
122 |         o.qualificationCategory,
123 |         o.qualificationValidFrom,
124 |         o.qualificationValidUntil,
125 |         o.availableSince,
126 |         o.createdAt,
127 |         o.updatedAt,
128 |         o.version);
```

Resultado que recibe el llamador: Datos implicados: identificador UUID del registro, identidad externa del operador, nombre del perfil de negocio, teléfono opcional, alta administrativa ACTIVE/INACTIVE, disposición a recibir propuestas, habilitación LIGHT_TOWING, inicio de habilitación, fin de habilitación que debe superar el servicio estimado, inicio de disponibilidad usado para desempate, fecha de creación, fecha de modificación, versión para control optimista de concurrencia.

L129

```
129 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L131

```
131 |   private TruckView view(TowTruck t) {
```

Transforma entidad/datos internos en DTO; no cambia la base por realizar esta proyección.

L132 a L145

```
132 |     return new TruckView(
133 |         t.id,
134 |         t.licensePlate,
135 |         t.operator == null ? null : t.operator.getId(),
136 |         t.maximumLoadKg,
137 |         t.supportsNonRollable,
138 |         Set.copyOf(t.supportedTypes),
139 |         t.operationalStatus,
140 |         t.latitude,
141 |         t.longitude,
142 |         t.locationUpdatedAt,
143 |         t.createdAt,
144 |         t.updatedAt,
145 |         t.version);
```

Resultado que recibe el llamador: Datos implicados: identificador UUID del registro, patente normalizada y única, capacidad de carga de la grúa en kg, capacidad para atender vehículos que no ruedan, tipos de vehículo admitidos, estado administrativo de la grúa, latitud declarada, longitud declarada, instante servidor de última ubicación declarada, fecha de creación, fecha de modificación, versión para control optimista de concurrencia; detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L146

```
146 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L148

```
148 |   public OperatorView getOperator(UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L149

```
149 |     return view(operator(id));
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L150

```
150 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L152

```
152 |   public OperatorView me(String subject) {
```

Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad.

L153

```
153 |     return view(own(subject));
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L154

```
154 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L156

```
156 |   public TruckView getTruck(UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L157

```
157 |     return view(truck(id));
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L158

```
158 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L160

```
160 |   public TruckView assigned(String subject) {
```

Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad.

L161

```
161 |     return view(ownTruck(subject));
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L162

```
162 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L164

```
164 |   public PageView<OperatorView> operators(int p, int s, Registration status) {
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Tipos y datos: p = p; int es entero de 32 bits para límites y contadores; s = s; int es entero de 32 bits para límites y contadores.

L165

```
165 |     Pageable pageable = page(p, s);
```

Se calcula pageable (pageable) para reutilizar ese resultado en los pasos siguientes.

L166 a L170

```
166 |     return result(
167 |         (status == null
168 |                 ? operators.findAll(pageable)
169 |                 : operators.findByRegistrationStatus(status, pageable))
170 |             .map(this::view));
```

Transforma cada elemento al valor/tipo de salida. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Resultado que recibe el llamador: transforma cada elemento al tipo/valor de salida; detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L170: transforma cada elemento al tipo/valor de salida.

L171

```
171 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L173

```
173 |   public PageView<TruckView> trucks(int p, int s, Operational status) {
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Tipos y datos: p = p; int es entero de 32 bits para límites y contadores; s = s; int es entero de 32 bits para límites y contadores.

L174

```
174 |     Pageable pageable = page(p, s);
```

Se calcula pageable (pageable) para reutilizar ese resultado en los pasos siguientes.

L175 a L179

```
175 |     return result(
176 |         (status == null
177 |                 ? trucks.findAll(pageable)
178 |                 : trucks.findByOperationalStatus(status, pageable))
179 |             .map(this::view));
```

Transforma cada elemento al valor/tipo de salida. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Resultado que recibe el llamador: transforma cada elemento al tipo/valor de salida; detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L179: transforma cada elemento al tipo/valor de salida.

L180

```
180 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L182

```
182 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L183

```
183 |   public OperatorView create(CreateOperator d) {
```

Alta administrativa de perfil de operador: recibe el DTO, valida sub único y prepara un registro independiente de la cuenta Keycloak.

L184

```
184 |     Operator o = new Operator();
```

Construye Operator en memoria y conserva la referencia en o. Entidad JPA del perfil de negocio del operador. keycloakSubject identifica la cuenta externa; no guarda contraseña ni reemplaza Keycloak. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L185

```
185 |     o.keycloakSubject = d.keycloakSubject().strip();
```

En o guarda identidad externa del operador. Toma el dato de d.keycloakSubject().strip() después de las validaciones de este método.

L186

```
186 |     o.fullName = name(d.fullName());
```

En o guarda nombre del perfil de negocio. Toma el dato de name(d.fullName()) después de las validaciones de este método.

L187

```
187 |     o.contactPhone = d.contactPhone();
```

En o guarda teléfono opcional. Toma el dato de d.contactPhone() después de las validaciones de este método.

L188

```
188 |     o.createdAt = o.updatedAt = now();
```

En o guarda fecha de creación. Marca el instante de esta operación para ordenar y auditar su evolución. Datos implicados: fecha de modificación.

L189

```
189 |     operators.saveAndFlush(o);
```

Persiste y fuerza sincronización SQL para detectar restricciones antes de retornar; commit ocurre al terminar transacción. Guarda y sincroniza SQL para detectar conflictos antes de retornar; no confirma por sí solo la transacción.

L190

```
190 |     committed("OPERATOR_CREATED", o.id);
```

Datos implicados: identificador UUID del registro.

L191

```
191 |     return view(o);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L192

```
192 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L194

```
194 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L195

```
195 |   public OperatorView edit(UUID id, EditOperator d) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L196

```
196 |     require(d.fullName() != null || d.contactPhone() != null, HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. 400 indica entrada inválida; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; comprueba presencia antes de usar el dato

L197

```
197 |     Operator o = lockedOperator(id);
```

Se calcula o (o) para reutilizar ese resultado en los pasos siguientes.

L198

```
198 |     if (d.fullName() != null) o.fullName = name(d.fullName());
```

Cambios: o.fullName recibe name(d.fullName()). Esta condición decide si se entra en la rama siguiente. Datos implicados: nombre del perfil de negocio; comprueba presencia antes de usar el dato.

L199

```
199 |     if (d.contactPhone() != null) o.contactPhone = d.contactPhone();
```

Cambios: o.contactPhone recibe d.contactPhone(). Esta condición decide si se entra en la rama siguiente. Datos implicados: teléfono opcional; comprueba presencia antes de usar el dato.

L200

```
200 |     o.updatedAt = now();
```

En o guarda fecha de modificación. Marca el instante de esta operación para ordenar y auditar su evolución.

L201

```
201 |     operators.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente.

L202

```
202 |     return view(o);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L203

```
203 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L205

```
205 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L206

```
206 |   public OperatorView registration(UUID id, RegistrationChange d) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L207

```
207 |     Operator o = lockedOperator(id);
```

Se calcula o (o) para reutilizar ese resultado en los pasos siguientes.

L208

```
208 |     freeOperator(id);
```

Impide mutar un operador retenido o confirmado.

L209

```
209 |     o.registrationStatus = d.status();
```

En o guarda alta administrativa ACTIVE/INACTIVE. La transición efectiva proviene de d.status() y debe estar autorizada por las validaciones previas.

L210

```
210 |     if (d.status() == Registration.INACTIVE) {
```

Inactivar también retira disponibilidad y limpia availableSince. Esta condición decide si se entra en la rama siguiente. INACTIVE: registro inactivo.

L211

```
211 |       o.availabilityStatus = Availability.UNAVAILABLE;
```

En o guarda disposición a recibir propuestas. operador que no debe recibir nuevas propuestas UNAVAILABLE: operador no disponible.

L212

```
212 |       o.availableSince = null;
```

En o guarda inicio de disponibilidad usado para desempate. Toma el dato de null después de las validaciones de este método. null.

L213

```
213 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L214

```
214 |     o.updatedAt = now();
```

En o guarda fecha de modificación. Marca el instante de esta operación para ordenar y auditar su evolución.

L215

```
215 |     operators.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente.

L216

```
216 |     return view(o);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L217

```
217 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L219

```
219 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L220

```
220 |   public OperatorView qualification(UUID id, QualificationChange d) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L221

```
221 |     require(d.validFrom().isBefore(d.validUntil()), HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara si el instante es estrictamente anterior al otro; 400 indica entrada inválida

L222

```
222 |     Operator o = lockedOperator(id);
```

Se calcula o (o) para reutilizar ese resultado en los pasos siguientes.

L223

```
223 |     freeOperator(id);
```

Impide mutar un operador retenido o confirmado.

L224

```
224 |     o.qualificationCategory = d.category();
```

En o guarda habilitación LIGHT_TOWING. Toma el dato de d.category() después de las validaciones de este método.

L225

```
225 |     o.qualificationValidFrom = d.validFrom();
```

En o guarda inicio de habilitación. Toma el dato de d.validFrom() después de las validaciones de este método.

L226

```
226 |     o.qualificationValidUntil = d.validUntil();
```

En o guarda fin de habilitación que debe superar el servicio estimado. Toma el dato de d.validUntil() después de las validaciones de este método.

L227

```
227 |     o.updatedAt = now();
```

En o guarda fecha de modificación. Marca el instante de esta operación para ordenar y auditar su evolución.

L228

```
228 |     operators.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente.

L229

```
229 |     committed("QUALIFICATION_UPDATED", o.id);
```

Programa log de renovación después del commit y retorna perfil actualizado. Datos implicados: identificador UUID del registro.

L230

```
230 |     return view(o);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L231

```
231 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L233

```
233 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L234

```
234 |   public OperatorView availability(String subject, AvailabilityChange d) {
```

Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad.

L235

```
235 |     Operator o = operators.lockBySubject(subject).orElseThrow(FleetService::missing);
```

Si no encuentra resultado lanza la excepción indicada. Se calcula o (o) para reutilizar ese resultado en los pasos siguientes. si no existe valor, lanza la excepción en vez de seguir con null

L236

```
236 |     freeOperator(o.id);
```

Impide cambiar disponibilidad si el operador está retenido/confirmado. Datos implicados: identificador UUID del registro.

L237 a L239

```
237 |     require(
238 |         d.status() != Availability.AVAILABLE || o.registrationStatus == Registration.ACTIVE,
239 |         HttpStatus.CONFLICT);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. 409 comunica que el estado/regla actual impide la acción; AVAILABLE: operador dispuesto a recibir propuestas; ACTIVE: registro habilitado administrativamente; Datos implicados: alta administrativa ACTIVE/INACTIVE; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera

L240

```
240 |     if (o.availabilityStatus != d.status()) {
```

availableSince solo cambia si cambia disponibilidad: repetir AVAILABLE no altera el desempate. Esta condición decide si se entra en la rama siguiente. Datos implicados: disposición a recibir propuestas.

L241

```
241 |       o.availabilityStatus = d.status();
```

En o guarda disposición a recibir propuestas. La transición efectiva proviene de d.status() y debe estar autorizada por las validaciones previas.

L242

```
242 |       o.availableSince = d.status() == Availability.AVAILABLE ? now() : null;
```

En o guarda inicio de disponibilidad usado para desempate. Toma el dato de d.status() == Availability.AVAILABLE ? now() : null después de las validaciones de este método. AVAILABLE: operador dispuesto a recibir propuestas; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L243

```
243 |       o.updatedAt = now();
```

En o guarda fecha de modificación. Marca el instante de esta operación para ordenar y auditar su evolución.

L244

```
244 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L245

```
245 |     operators.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente.

L246

```
246 |     return view(o);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L247

```
247 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L249

```
249 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L250

```
250 |   public TruckView create(CreateTruck d) {
```

Alta administrativa de grúa: valida patente y capacidad y crea el registro inicial sin convertirla automáticamente en recurso elegible.

L251

```
251 |     TowTruck t = new TowTruck();
```

Construye TowTruck en memoria y conserva la referencia en t. Entidad JPA de la grúa: capacidad, tipos admitidos, vínculo, estado y última ubicación declarada. La disponibilidad humana está en Operator y el compromiso en ResourceReservation. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L252

```
252 |     t.licensePlate = plate(d.licensePlate());
```

En t guarda patente normalizada y única. Toma el dato de plate(d.licensePlate()) después de las validaciones de este método.

L253

```
253 |     t.maximumLoadKg = d.maximumLoadKg();
```

En t guarda capacidad de carga de la grúa en kg. Toma el dato de d.maximumLoadKg() después de las validaciones de este método.

L254

```
254 |     t.supportsNonRollable = d.supportsNonRollable();
```

En t guarda capacidad para atender vehículos que no ruedan. Toma el dato de d.supportsNonRollable() después de las validaciones de este método.

L255

```
255 |     t.supportedTypes.addAll(d.supportedTypes());
```

Datos implicados: tipos de vehículo admitidos.

L256

```
256 |     t.createdAt = t.updatedAt = now();
```

En t guarda fecha de creación. Marca el instante de esta operación para ordenar y auditar su evolución. Datos implicados: fecha de modificación.

L257

```
257 |     trucks.saveAndFlush(t);
```

Persiste y fuerza sincronización SQL para detectar restricciones antes de retornar; commit ocurre al terminar transacción. Guarda y sincroniza SQL para detectar conflictos antes de retornar; no confirma por sí solo la transacción.

L258

```
258 |     committed("TRUCK_CREATED", t.id);
```

Datos implicados: identificador UUID del registro.

L259

```
259 |     return view(t);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L260

```
260 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L262

```
262 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L263

```
263 |   public TruckView edit(UUID id, EditTruck d) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L264 a L269

```
264 |     require(
265 |         d.licensePlate() != null
266 |             || d.maximumLoadKg() != null
267 |             || d.supportsNonRollable() != null
268 |             || d.supportedTypes() != null,
269 |         HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. 400 indica entrada inválida; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; comprueba presencia antes de usar el dato

L266: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; comprueba presencia antes de usar el dato.

L267: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; comprueba presencia antes de usar el dato.

L268: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; comprueba presencia antes de usar el dato.

L270

```
270 |     TowTruck t = lockedTruck(id);
```

Bloquea la grúa antes de decidir la modificación; freeTruck impide cambiar un compromiso activo cuando aparece en esta línea. Se calcula t (t) para reutilizar ese resultado en los pasos siguientes.

L271

```
271 |     freeTruck(id);
```

Rechaza mutación de grúa con reserva HELD/CONFIRMED.

L272

```
272 |     if (d.licensePlate() != null) t.licensePlate = plate(d.licensePlate());
```

Cambios: t.licensePlate recibe plate(d.licensePlate()). Esta condición decide si se entra en la rama siguiente. Datos implicados: patente normalizada y única; comprueba presencia antes de usar el dato.

L273

```
273 |     if (d.maximumLoadKg() != null) t.maximumLoadKg = d.maximumLoadKg();
```

Cambios: t.maximumLoadKg recibe d.maximumLoadKg(). Esta condición decide si se entra en la rama siguiente. Datos implicados: capacidad de carga de la grúa en kg; comprueba presencia antes de usar el dato.

L274

```
274 |     if (d.supportsNonRollable() != null) t.supportsNonRollable = d.supportsNonRollable();
```

Cambios: t.supportsNonRollable recibe d.supportsNonRollable(). Esta condición decide si se entra en la rama siguiente. Datos implicados: capacidad para atender vehículos que no ruedan; comprueba presencia antes de usar el dato.

L275

```
275 |     if (d.supportedTypes() != null) {
```

Esta condición decide si se entra en la rama siguiente. comprueba presencia antes de usar el dato.

L276

```
276 |       t.supportedTypes.clear();
```

Datos implicados: tipos de vehículo admitidos.

L277

```
277 |       t.supportedTypes.addAll(d.supportedTypes());
```

Datos implicados: tipos de vehículo admitidos.

L278

```
278 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L279

```
279 |     t.updatedAt = now();
```

En t guarda fecha de modificación. Marca el instante de esta operación para ordenar y auditar su evolución.

L280

```
280 |     trucks.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente.

L281

```
281 |     return view(t);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L282

```
282 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L284

```
284 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L285

```
285 |   public TruckView link(UUID id, LinkOperator d) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L286

```
286 |     // All operations needing both locks take the truck first, then the operator.
```

Documentación para quien lee el fuente: All operations needing both locks take the truck first, then the operator. No es una instrucción ejecutable y no impone la regla por sí sola.

L287

```
287 |     TowTruck t = lockedTruck(id);
```

Bloquea la grúa antes de decidir la modificación; freeTruck impide cambiar un compromiso activo cuando aparece en esta línea. Se calcula t (t) para reutilizar ese resultado en los pasos siguientes.

L288

```
288 |     Operator o = lockedOperator(d.operatorId());
```

Se calcula o (o) para reutilizar ese resultado en los pasos siguientes.

L289

```
289 |     freeTruck(id);
```

Rechaza mutación de grúa con reserva HELD/CONFIRMED.

L290

```
290 |     freeOperator(o.id);
```

Impide cambiar disponibilidad si el operador está retenido/confirmado. Datos implicados: identificador UUID del registro.

L291

```
291 |     require(t.operator == null || t.operator.getId().equals(o.id), HttpStatus.CONFLICT);
```

No sustituye otro operador silenciosamente; admite vacío o el mismo vínculo. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara contenido/identidad lógica, no referencias Java; 409 comunica que el estado/regla actual impide la acción; Datos implicados: identificador UUID del registro; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente

L292

```
292 |     var assigned = trucks.findByOperatorId(o.id);
```

Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Se calcula assigned (assigned) para reutilizar ese resultado en los pasos siguientes. Datos implicados: identificador UUID del registro

L293

```
293 |     require(assigned.isEmpty() || assigned.get().id.equals(t.id), HttpStatus.CONFLICT);
```

El operador no puede estar vinculado a otra grúa. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara contenido/identidad lógica, no referencias Java; comprueba ausencia de elementos o valor; 409 comunica que el estado/regla actual impide la acción; Datos implicados: identificador UUID del registro; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera

L294

```
294 |     t.operator = o;
```

En t guarda operator. Toma el dato de o después de las validaciones de este método. o.

L295

```
295 |     t.updatedAt = now();
```

En t guarda fecha de modificación. Marca el instante de esta operación para ordenar y auditar su evolución.

L296

```
296 |     trucks.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente.

L297

```
297 |     return view(t);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L298

```
298 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L300

```
300 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L301

```
301 |   public void unlink(UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L302

```
302 |     TowTruck t = lockedTruck(id);
```

Bloquea la grúa antes de decidir la modificación; freeTruck impide cambiar un compromiso activo cuando aparece en esta línea. Se calcula t (t) para reutilizar ese resultado en los pasos siguientes.

L303

```
303 |     freeTruck(id);
```

Rechaza mutación de grúa con reserva HELD/CONFIRMED.

L304

```
304 |     t.operator = null;
```

En t guarda operator. Toma el dato de null después de las validaciones de este método. null.

L305

```
305 |     t.updatedAt = now();
```

En t guarda fecha de modificación. Marca el instante de esta operación para ordenar y auditar su evolución.

L306

```
306 |     trucks.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente.

L307

```
307 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L309

```
309 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L310

```
310 |   public StatusView status(UUID id, StatusChange d, String subject, String key) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; subject = subject; String es texto; no es por sí mismo una referencia de entidad; key = key; String es texto; no es por sí mismo una referencia de entidad.

L311

```
311 |     require(key != null && !key.isBlank() && key.length() <= 100, HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. comprueba texto vacío o solo espacios; 400 indica entrada inválida; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue; comprueba presencia antes de usar el dato

L312

```
312 |     TowTruck t = lockedTruck(id);
```

Bloquea la grúa antes de decidir la modificación; freeTruck impide cambiar un compromiso activo cuando aparece en esta línea. Se calcula t (t) para reutilizar ese resultado en los pasos siguientes.

L313

```
313 |     var previous = history.findByOperationKey(key);
```

Busca resultado anterior para repetir sin duplicar. Se calcula previous (previous) para reutilizar ese resultado en los pasos siguientes.

L314

```
314 |     if (previous.isPresent()) {
```

Esta condición decide si se entra en la rama siguiente. .

L315

```
315 |       TruckStatusHistory h = previous.get();
```

Recupera el evento previo asociado a la clave; sirve para devolver la misma respuesta. Se calcula h (h) para reutilizar ese resultado en los pasos siguientes.

L316 a L321

```
316 |       require(
317 |           h.truck.getId().equals(id)
318 |               && h.newStatus == d.status()
319 |               && h.reason.equals(d.reason().strip())
320 |               && h.changedBySubject.equals(subject),
321 |           HttpStatus.CONFLICT);
```

Una clave de estado de grúa exige mismo actor, grúa, estado y motivo. Repetir devuelve el evento sin reaplicarlo. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara contenido/identidad lógica, no referencias Java; 409 comunica que el estado/regla actual impide la acción; Datos implicados: estado resultante, motivo legible, sub del administrador responsable; && exige todas las condiciones y deja de evaluar al encontrar una falsa

L318: Datos implicados: estado resultante; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L319: compara contenido/identidad lógica, no referencias Java; Datos implicados: motivo legible; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L320: compara contenido/identidad lógica, no referencias Java; Datos implicados: sub del administrador responsable; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L322

```
322 |       return statusView(h);
```

Proyecta el evento de estado sin aplicar de nuevo el cambio.

L323

```
323 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L324

```
324 |     freeTruck(id);
```

Rechaza mutación de grúa con reserva HELD/CONFIRMED.

L325

```
325 |     require(t.operationalStatus != d.status(), HttpStatus.CONFLICT);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. 409 comunica que el estado/regla actual impide la acción; Datos implicados: estado administrativo de la grúa

L326

```
326 |     TruckStatusHistory h = new TruckStatusHistory();
```

Construye TruckStatusHistory en memoria y conserva la referencia en h. Entidad de historial administrativo de la grúa. Conserva antes/después, motivo, actor y clave de idempotencia. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L327

```
327 |     h.truck = t;
```

En h guarda truck. Toma el dato de t después de las validaciones de este método. t.

L328

```
328 |     h.previousStatus = t.operationalStatus;
```

En h guarda estado previo. La transición efectiva proviene de t.operationalStatus y debe estar autorizada por las validaciones previas. Datos implicados: estado administrativo de la grúa.

L329

```
329 |     h.newStatus = d.status();
```

En h guarda estado resultante. La transición efectiva proviene de d.status() y debe estar autorizada por las validaciones previas.

L330

```
330 |     h.reason = d.reason().strip();
```

En h guarda motivo legible. Toma el dato de d.reason().strip() después de las validaciones de este método.

L331

```
331 |     h.changedBySubject = subject;
```

En h guarda sub del administrador responsable. Identifica a la persona por sub de Keycloak, no por un nombre editable ni una contraseña. subject.

L332

```
332 |     h.operationKey = key;
```

En h guarda clave durable de acción. Toma el dato de key después de las validaciones de este método. key.

L333

```
333 |     h.changedAt = now();
```

En h guarda instante del cambio. Marca el instante de esta operación para ordenar y auditar su evolución.

L334

```
334 |     history.save(h);
```

Solicita persistir la entidad en el contexto JPA.

L335

```
335 |     t.operationalStatus = d.status();
```

En t guarda estado administrativo de la grúa. La transición efectiva proviene de d.status() y debe estar autorizada por las validaciones previas.

L336

```
336 |     t.updatedAt = h.changedAt;
```

En t guarda fecha de modificación. Marca el instante de esta operación para ordenar y auditar su evolución. Datos implicados: instante del cambio.

L337

```
337 |     trucks.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente.

L338

```
338 |     committed("TRUCK_" + h.previousStatus + "_TO_" + h.newStatus, t.id);
```

Datos implicados: estado previo, estado resultante, identificador UUID del registro.

L339

```
339 |     return statusView(h);
```

Proyecta el evento de estado sin aplicar de nuevo el cambio.

L340

```
340 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L342

```
342 |   private StatusView statusView(TruckStatusHistory h) {
```

Proyecta el evento de cambio de estado administrativo, no una nueva transición.

L343 a L351

```
343 |     return new StatusView(
344 |         h.id,
345 |         h.truck.getId(),
346 |         h.previousStatus,
347 |         h.newStatus,
348 |         h.reason,
349 |         h.changedBySubject,
350 |         h.operationKey,
351 |         h.changedAt);
```

Resultado que recibe el llamador: Datos implicados: identificador UUID del registro, estado previo, estado resultante, motivo legible, sub del administrador responsable, clave durable de acción, instante del cambio.

L352

```
352 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L354

```
354 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L355

```
355 |   public TruckView location(String subject, Location d) {
```

Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad.

L356

```
356 |     TowTruck t = lockedTruck(trucks.assignedId(subject).orElseThrow(FleetService::missing));
```

Si no encuentra resultado lanza la excepción indicada. Se calcula t (t) para reutilizar ese resultado en los pasos siguientes. si no existe valor, lanza la excepción en vez de seguir con null

L357 a L359

```
357 |     require(
358 |         t.operator != null && t.operator.getKeycloakSubject().equals(subject),
359 |         HttpStatus.NOT_FOUND);
```

Expone identidad del operador para comprobar propiedad del equipo. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara contenido/identidad lógica, no referencias Java; 404 oculta inexistencia o pertenencia ajena; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato

L360

```
360 |     freeTruck(t.id);
```

Impide actualizar incluso las mismas coordenadas cuando existe HELD/CONFIRMED. Así no se renueva artificialmente la vigencia ni cambia la ruta durante una propuesta. Datos implicados: identificador UUID del registro.

L361

```
361 |     t.latitude = d.latitude();
```

En t guarda latitud declarada. Forma parte de la posición declarada usada para cobertura y elegibilidad; la fecha permite comprobar sus cinco minutos de vigencia.

L362

```
362 |     t.longitude = d.longitude();
```

En t guarda longitud declarada. Forma parte de la posición declarada usada para cobertura y elegibilidad; la fecha permite comprobar sus cinco minutos de vigencia.

L363

```
363 |     t.locationUpdatedAt = t.updatedAt = now();
```

En t guarda instante servidor de última ubicación declarada. Forma parte de la posición declarada usada para cobertura y elegibilidad; la fecha permite comprobar sus cinco minutos de vigencia. Datos implicados: fecha de modificación.

L364

```
364 |     trucks.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente.

L365

```
365 |     return view(t);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L366

```
366 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L367

```
367 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 13, 21, 24, 28, 32, 43, 55, 59, 63, 67, 71, 75, 80, 84, 88, 92, 98, 103, 108, 113, 130, 147, 151, 155, 159, 163, 172, 181, 193, 204, 218, 232, 248, 261, 283, 299, 308, 341, 353: separadores sin instrucciones.

FleetTypes (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/FleetTypes.java`

Objetivo y funcionalidad

Enumeraciones propias de Fleet. ACTIVE/INACTIVE/MAINTENANCE es el estado administrativo de la grúa; no representa los estados del traslado.

L1

```
1 | package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | public final class FleetTypes {
```

Declara clase FleetTypes. Enumeraciones propias de Fleet. ACTIVE/INACTIVE/MAINTENANCE es el estado administrativo de la grúa; no representa los estados del traslado.

L4

```
4 |   private FleetTypes() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L6

```
6 |   public enum Registration {
```

Declara enumeración cerrada Registration. Enumeraciones propias de Fleet. ACTIVE/INACTIVE/MAINTENANCE es el estado administrativo de la grúa; no representa los estados del traslado.

L7 a L9

```
7 |     ACTIVE,
8 |     INACTIVE
9 |   }
```

Valores cerrados de esta enumeración: ACTIVE, INACTIVE. El tipo impide usar cadenas arbitrarias; representan registro, disponibilidad, capacidad o categoría según el enum contenedor, no el estado HTTP del contenedor Docker.

L11

```
11 |   public enum Availability {
```

Declara enumeración cerrada Availability. Enumeraciones propias de Fleet. ACTIVE/INACTIVE/MAINTENANCE es el estado administrativo de la grúa; no representa los estados del traslado.

L12 a L14

```
12 |     AVAILABLE,
13 |     UNAVAILABLE
14 |   }
```

Valores cerrados de esta enumeración: AVAILABLE, UNAVAILABLE. El tipo impide usar cadenas arbitrarias; representan registro, disponibilidad, capacidad o categoría según el enum contenedor, no el estado HTTP del contenedor Docker.

L16

```
16 |   public enum Operational {
```

Declara enumeración cerrada Operational. Enumeraciones propias de Fleet. ACTIVE/INACTIVE/MAINTENANCE es el estado administrativo de la grúa; no representa los estados del traslado.

L17 a L20

```
17 |     ACTIVE,
18 |     MAINTENANCE,
19 |     INACTIVE
20 |   }
```

Valores cerrados de esta enumeración: ACTIVE, MAINTENANCE, INACTIVE. El tipo impide usar cadenas arbitrarias; representan registro, disponibilidad, capacidad o categoría según el enum contenedor, no el estado HTTP del contenedor Docker.

L22

```
22 |   public enum VehicleType {
```

Declara enumeración cerrada VehicleType. Enumeraciones propias de Fleet. ACTIVE/INACTIVE/MAINTENANCE es el estado administrativo de la grúa; no representa los estados del traslado.

L23 a L26

```
23 |     CAR,
24 |     SUV,
25 |     PICKUP
26 |   }
```

Valores cerrados de esta enumeración: CAR, SUV, PICKUP. El tipo impide usar cadenas arbitrarias; representan registro, disponibilidad, capacidad o categoría según el enum contenedor, no el estado HTTP del contenedor Docker.

L28

```
28 |   public enum Qualification {
```

Declara enumeración cerrada Qualification. Enumeraciones propias de Fleet. ACTIVE/INACTIVE/MAINTENANCE es el estado administrativo de la grúa; no representa los estados del traslado.

L29 a L30

```
29 |     LIGHT_TOWING
30 |   }
```

Valores cerrados de esta enumeración: LIGHT_TOWING. El tipo impide usar cadenas arbitrarias; representan registro, disponibilidad, capacidad o categoría según el enum contenedor, no el estado HTTP del contenedor Docker.

L31

```
31 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 5, 10, 15, 21, 27: separadores sin instrucciones.

ReservationController (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/ReservationController.java`

Objetivo y funcionalidad

API interna de reservar, confirmar, liberar y consultar elegibilidad/estado. Se usa el mismo UUID para reintentos idempotentes.

L1

```
1 | package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import jakarta.validation.Valid;
```

Importa tipos: jakarta.validation.Valid. Restricciones de DTO y activación de validación de entradas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.web.bind.annotation.*;
```

Importa tipos: org.springframework.web.bind.annotation.*. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import io.swagger.v3.oas.annotations.Hidden;
```

Importa tipos: io.swagger.v3.oas.annotations.Hidden. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | @Hidden
```

Oculta operación interna en OpenAPI.

L9

```
9 | @RestController
```

Expone HTTP y serializa JSON.

L10

```
10 | @RequestMapping("/internal/reservations")
```

Ruta /internal/reservations combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L11

```
11 | public class ReservationController {
```

Declara clase ReservationController. API interna de reservar, confirmar, liberar y consultar elegibilidad/estado. Se usa el mismo UUID para reintentos idempotentes.

L12

```
12 |   private final ReservationService service;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L14

```
14 |   public ReservationController(ReservationService service) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L15

```
15 |     this.service = service;
```

En this guarda service. Toma el dato de service después de las validaciones de este método. service.

L16

```
16 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L18

```
18 |   @PutMapping("/{id}")
```

Ruta /{id} combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L19 a L20

```
19 |   public ReservationService.View reserve(
20 |       @PathVariable UUID id, @Valid @RequestBody ReservationService.Input input) {
```

Extrae ID de la ruta. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L20: Dato del contrato: id: identificador UUID del registro.

L21

```
21 |     return service.reserve(id, input);
```

Intenta retener de manera exclusiva grúa/operador con el UUID compartido de asignación; la repetición debe conservar payload. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L22

```
22 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L24 a L27

```
24 |   public record StatusQueries(
25 |       @jakarta.validation.constraints.NotNull
26 |           @jakarta.validation.constraints.Size(min = 1, max = 100)
27 |           java.util.List<@jakarta.validation.constraints.NotNull UUID> assignmentAttemptIds) {}
```

Exige no null. Limita caracteres/elementos según min/max. Declara record inmutable StatusQueries. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Se calcula max (max) para reutilizar ese resultado en los pasos siguientes. API interna de reservar, confirmar, liberar y consultar elegibilidad/estado. Se usa el mismo UUID para reintentos idempotentes.

L29

```
29 |   @PostMapping("/status-queries")
```

Ruta /status-queries combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L30 a L31

```
30 |   public java.util.List<ReservationService.StatusQuery> statuses(
31 |       @Valid @RequestBody StatusQueries input) {
```

Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON.

L32

```
32 |     return input.assignmentAttemptIds().stream().distinct().map(service::statusQuery).toList();
```

Transforma cada elemento al valor/tipo de salida. Materializa la secuencia en lista. Resultado que recibe el llamador: inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente; transforma cada elemento al tipo/valor de salida; elimina duplicados para no repetir roles o candidatos; materializa el resultado de la secuencia en una lista.

L33

```
33 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L35

```
35 |   @GetMapping("/{id}")
```

Ruta /{id} combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L36

```
36 |   public ReservationService.View get(@PathVariable UUID id) {
```

Extrae ID de la ruta. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L37

```
37 |     return service.get(id);
```

Consulta la reserva por identificador para reconciliar lo que realmente ocurrió en Fleet. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L38

```
38 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L40

```
40 |   @PostMapping("/{id}/confirmation")
```

Ruta /{id}/confirmation combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L41

```
41 |   public ReservationService.View confirm(@PathVariable UUID id) {
```

Extrae ID de la ruta. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L42

```
42 |     return service.confirm(id);
```

Confirma la retención vigente después de revalidar las condiciones de aceptación. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L43

```
43 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L45

```
45 |   @GetMapping("/{id}/eligibility")
```

Ruta /{id}/eligibility combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L46

```
46 |   public ReservationService.Eligibility eligibility(@PathVariable UUID id) {
```

Extrae ID de la ruta. Consulta si un compromiso confirmado conserva aptitud; no aplica caducidad de ubicación de una propuesta nueva. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L47

```
47 |     return service.eligibility(id);
```

Consulta si el compromiso existente sigue siendo apto; no busca un candidato nuevo. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L48

```
48 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L50

```
50 |   @PostMapping("/{id}/release")
```

Ruta /{id}/release combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L51

```
51 |   public ReservationService.View release(@PathVariable UUID id) {
```

Extrae ID de la ruta. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L52

```
52 |     return service.release(id);
```

Delega la liberación idempotente de la reserva identificada en Fleet. El servicio comprueba el estado y devuelve la representación actual sin liberar otro recurso.

L53

```
53 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L54

```
54 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 7, 13, 17, 23, 28, 34, 39, 44, 49: separadores sin instrucciones.

ReservationJobs (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/ReservationJobs.java`

Objetivo y funcionalidad

Materializa la expiración de retenciones mediante una tarea periódica; no cancela servicios ya confirmados.

L1

```
1 | package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.Clock;
```

Importa tipos: java.time.Clock. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.sql.Timestamp;
```

Importa tipos: java.sql.Timestamp. Timestamp y ResultSet para puentear fechas/filas de JDBC con objetos Java. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.jdbc.core.JdbcTemplate;
```

Importa tipos: org.springframework.jdbc.core.JdbcTemplate. JdbcTemplate para SQL parametrizado, resultados y acciones durables. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
 8 | import org.springframework.scheduling.annotation.*;
```

Importa tipos: org.springframework.scheduling.annotation.*. Tareas periódicas y scheduler. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
 9 | import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
```

Importa tipos: org.springframework.boot.autoconfigure.condition.ConditionalOnProperty. Arranque/autoconfiguración y condiciones de activación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | @Component
```

Registra componente inyectable.

L12

```
12 | @EnableScheduling
```

Habilita tareas programadas.

L13 a L17

```
13 | @ConditionalOnProperty(
14 |     name = "auxiruta.automation.enabled",
15 |     havingValue = "true",
16 |     matchIfMissing = true)
17 | public class ReservationJobs {
```

Activa bean solo si cumple propiedad configurada. Declara clase ReservationJobs. Materializa la expiración de retenciones mediante una tarea periódica; no cancela servicios ya confirmados.

L18

```
18 |   private final JdbcTemplate jdbc;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: jdbc = ejecutor SQL parametrizado; JdbcTemplate es ejecutor de SQL parametrizado de Spring dentro de transacciones locales.

L19

```
19 |   private final ReservationService reservations;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L20

```
20 |   private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L22

```
22 |   public ReservationJobs(JdbcTemplate jdbc, ReservationService reservations, Clock clock) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada. Tipos y datos: jdbc = ejecutor SQL parametrizado; JdbcTemplate es ejecutor de SQL parametrizado de Spring dentro de transacciones locales; clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L23

```
23 |     this.jdbc = jdbc;
```

En this guarda ejecutor SQL parametrizado. Toma el dato de jdbc después de las validaciones de este método. ejecutor SQL parametrizado.

L24

```
24 |     this.reservations = reservations;
```

En this guarda acceso al compromiso temporal/confirmado de recursos. Toma el dato de reservations después de las validaciones de este método. acceso al compromiso temporal/confirmado de recursos.

L25

```
25 |     this.clock = clock;
```

En this guarda reloj usado para evaluar plazos. Toma el dato de clock después de las validaciones de este método. reloj usado para evaluar plazos.

L26

```
26 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L28

```
28 |   @Scheduled(fixedDelayString = "${auxiruta.automation.fleet-delay-ms:15000}", initialDelay =
     15000)
```

Tarea periódica: fixedDelay se mide desde fin de ejecución previa. El scheduler invoca el método sin una petición de Postman; fixedDelay espera desde que termina la ejecución anterior.

L29

```
29 |   public void expire() {
```

Solo cambia HELD a EXPIRED al alcanzar expiresAt; no vence CONFIRMED.

L30 a L35

```
30 |     for (UUID id :
31 |         jdbc.queryForList(
32 |             "select id from resource_reservations where status='HELD' and expires_at<=? order by
     expires_at,id limit 100",
33 |             UUID.class,
34 |             Timestamp.from(clock.instant())))
35 |       try {
```

Consulta resource_reservations con el filtro y proyección escritos. Campos: id: identificador UUID del registro. El bucle repite el bloque por cada elemento; el nombre a la izquierda de : es la variable local y a la derecha está la colección. Lee el instante del reloj inyectado; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L36

```
36 |         reservations.expireDue(id);
```

Solicita expirar esta retención si sigue vencida y en estado expirables al verificarla bajo transacción. No cancela una reserva confirmada por el antiguo plazo de oferta.

L37

```
37 |       } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L38 a L40

```
38 |         org.slf4j.LoggerFactory.getLogger(getClass())
39 |             .warn(
40 |                 "Reservation expiration deferred id={} type={}", id,
     e.getClass().getSimpleName());
```

Registra los campos indicados, sin token ni secreto. Se calcula id (identificador UUID del registro) para reutilizar ese resultado en los pasos siguientes.

L41

```
41 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L42

```
42 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L43

```
43 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 10, 21, 27: separadores sin instrucciones.

ReservationService (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/ReservationService.java`

Objetivo y funcionalidad

Protege la exclusividad mediante bloqueos y restricciones. HELD caduca; CONFIRMED no caduca por el plazo antiguo de la oferta. Exige ubicación de hasta 300 s al reservar/confirmar, pero no vuelve a caducarla después de aceptar.

L1

```
1 | package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de fleet-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import jakarta.validation.constraints.*;
```

Importa tipos: jakarta.validation.constraints.*. Restricciones de DTO y activación de validación de entradas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
 9 | import org.springframework.http.HttpStatus;
```

Importa tipos: org.springframework.http.HttpStatus. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | import com.fasterxml.jackson.databind.ObjectMapper;
```

Importa tipos: com.fasterxml.jackson.databind.ObjectMapper. Lectura/escritura JSON, nodos y conversión a DTO. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12

```
12 | import static com.auxiruta.fleet.catalog.FleetTypes.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetTypes.*. Tipos y DTO locales de Fleet. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L14

```
14 | @Service
```

Registra lógica de aplicación como bean. El escaneo de Spring crea este componente e inyecta sus dependencias; no hay que construirlo manualmente en el controlador.

L15

```
15 | @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L16

```
16 | public class ReservationService {
```

Declara clase ReservationService. Protege la exclusividad mediante bloqueos y restricciones. HELD caduca; CONFIRMED no caduca por el plazo antiguo de la oferta. Exige ubicación de hasta 300 s al reservar/confirmar, pero no vuelve a caducarla después de aceptar.

L17 a L27

```
17 |   public record Input(
18 |       @NotNull UUID requestId,
19 |       @NotNull UUID towTruckId,
20 |       @NotNull UUID operatorId,
21 |       @NotNull VehicleType vehicleType,
22 |       @NotNull @DecimalMin("500") @DecimalMax("3000") BigDecimal weightKg,
23 |       boolean nonRollable,
24 |       @NotNull Instant searchDeadline,
25 |       @NotNull Instant estimatedServiceEndAt,
26 |       Integer holdSeconds,
27 |       Integer minimumOfferSeconds) {
```

Exige no null. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Declara record inmutable Input. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: requestId: UUID de solicitud; towTruckId: UUID de grúa; operatorId: UUID del perfil operador; vehicleType: tipo CAR/SUV/PICKUP; searchDeadline: vencimiento del intento; estimatedServiceEndAt: fin planificado para controlar habilitación. Protege la exclusividad mediante bloqueos y restricciones. HELD caduca; CONFIRMED no caduca por el plazo antiguo de la oferta. Exige ubicación de hasta 300 s al reservar/confirmar, pero no vuelve a caducarla después de aceptar. Tipos y datos: requestId = UUID de solicitud; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; towTruckId = UUID de grúa; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; operatorId = UUID del perfil operador; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; weightKg = weightKg; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; nonRollable = nonRollable; boolean es verdadero/falso, sin null; searchDeadline = vencimiento del intento; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; estimatedServiceEndAt = fin planificado para controlar habilitación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L18: Dato del contrato: requestId: UUID de solicitud.

L19: Dato del contrato: towTruckId: UUID de grúa.

L20: Dato del contrato: operatorId: UUID del perfil operador.

L21: Dato del contrato: vehicleType: tipo CAR/SUV/PICKUP.

L24: Dato del contrato: searchDeadline: vencimiento del intento.

L25: Dato del contrato: estimatedServiceEndAt: fin planificado para controlar habilitación.

L28

```
28 |     public Input {
```

Constructor compacto de un record: valida o normaliza los parámetros antes de que Java los guarde en campos finales. Evita crear un DTO que incumpla esta condición local.

L29

```
29 |       if (holdSeconds == null) holdSeconds = 60;
```

Esta condición decide si se entra en la rama siguiente. detecta un dato ausente.

L30

```
30 |       if (minimumOfferSeconds == null) minimumOfferSeconds = 15;
```

Esta condición decide si se entra en la rama siguiente. detecta un dato ausente.

L31

```
31 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L33 a L41

```
33 |     public Input(
34 |         UUID requestId,
35 |         UUID towTruckId,
36 |         UUID operatorId,
37 |         VehicleType vehicleType,
38 |         BigDecimal weightKg,
39 |         boolean nonRollable,
40 |         Instant searchDeadline,
41 |         Instant estimatedServiceEndAt) {
```

Campos: requestId: UUID de solicitud; towTruckId: UUID de grúa; operatorId: UUID del perfil operador; vehicleType: tipo CAR/SUV/PICKUP; searchDeadline: vencimiento del intento; estimatedServiceEndAt: fin planificado para controlar habilitación. Tipos y datos: requestId = UUID de solicitud; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; towTruckId = UUID de grúa; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; operatorId = UUID del perfil operador; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; weightKg = weightKg; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; nonRollable = nonRollable; boolean es verdadero/falso, sin null; searchDeadline = vencimiento del intento; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; estimatedServiceEndAt = fin planificado para controlar habilitación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L34: Dato del contrato: requestId: UUID de solicitud.

L35: Dato del contrato: towTruckId: UUID de grúa.

L36: Dato del contrato: operatorId: UUID del perfil operador.

L40: Dato del contrato: searchDeadline: vencimiento del intento.

L41: Dato del contrato: estimatedServiceEndAt: fin planificado para controlar habilitación.

L42 a L52

```
42 |       this(
43 |           requestId,
44 |           towTruckId,
45 |           operatorId,
46 |           vehicleType,
47 |           weightKg,
48 |           nonRollable,
49 |           searchDeadline,
50 |           estimatedServiceEndAt,
51 |           60,
52 |           15);
```

Constructor alternativo que completa la petición con holdSeconds=60 y minimumOfferSeconds=15. this(...) reutiliza el constructor principal y conserva los mismos datos de vehículo, recurso y vencimientos.

L53

```
53 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L54

```
54 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L56 a L65

```
56 |   public record View(
57 |       UUID assignmentAttemptId,
58 |       UUID requestId,
59 |       UUID towTruckId,
60 |       UUID operatorId,
61 |       String status,
62 |       Instant expiresAt,
63 |       Instant confirmedAt,
64 |       Instant releasedAt,
65 |       boolean createdHold) {}
```

Declara record inmutable View. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: requestId: UUID de solicitud; towTruckId: UUID de grúa; operatorId: UUID del perfil operador; status: estado del objeto; su enum o CHECK define valores permitidos; expiresAt: fin exclusivo de retención/propuesta; confirmedAt: instante de confirmación en Fleet; releasedAt: instante de liberación; createdHold: indica si el intento llegó a retener recursos. Protege la exclusividad mediante bloqueos y restricciones. HELD caduca; CONFIRMED no caduca por el plazo antiguo de la oferta. Exige ubicación de hasta 300 s al reservar/confirmar, pero no vuelve a caducarla después de aceptar. Tipos y datos: assignmentAttemptId = assignmentAttemptId; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; requestId = UUID de solicitud; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; towTruckId = UUID de grúa; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; operatorId = UUID del perfil operador; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; status = estado del objeto; su enum o CHECK define valores permitidos; String es texto; no es por sí mismo una referencia de entidad; expiresAt = fin exclusivo de retención/propuesta; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; confirmedAt = instante de confirmación en Fleet; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; releasedAt = instante de liberación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; createdHold = indica si el intento llegó a retener recursos; boolean es verdadero/falso, sin null.

L58: Dato del contrato: requestId: UUID de solicitud.

L59: Dato del contrato: towTruckId: UUID de grúa.

L60: Dato del contrato: operatorId: UUID del perfil operador.

L61: Dato del contrato: status: estado del objeto; su enum o CHECK define valores permitidos.

L62: Dato del contrato: expiresAt: fin exclusivo de retención/propuesta.

L63: Dato del contrato: confirmedAt: instante de confirmación en Fleet.

L64: Dato del contrato: releasedAt: instante de liberación.

L65: Dato del contrato: createdHold: indica si el intento llegó a retener recursos.

L67

```
67 |   private final ReservationRepository reservations;
```

Dependencia inyectada que consulta reservas activas antes de editar recursos. Evita modificar un operador o grúa que ya está retenido o confirmado.

L68

```
68 |   private final TowTruckRepository trucks;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L69

```
69 |   private final OperatorRepository operators;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L70

```
70 |   private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L71

```
71 |   private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L73 a L78

```
73 |   public ReservationService(
74 |       ReservationRepository reservations,
75 |       TowTruckRepository trucks,
76 |       OperatorRepository operators,
77 |       Clock clock,
78 |       ObjectMapper mapper) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos; mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L79

```
79 |     this.reservations = reservations;
```

En this guarda acceso al compromiso temporal/confirmado de recursos. Toma el dato de reservations después de las validaciones de este método. acceso al compromiso temporal/confirmado de recursos.

L80

```
80 |     this.trucks = trucks;
```

En this guarda repositorio de grúas. Toma el dato de trucks después de las validaciones de este método. repositorio de grúas.

L81

```
81 |     this.operators = operators;
```

En this guarda repositorio de perfiles de operadores. Toma el dato de operators después de las validaciones de este método. repositorio de perfiles de operadores.

L82

```
82 |     this.clock = clock;
```

En this guarda reloj usado para evaluar plazos. Toma el dato de clock después de las validaciones de este método. reloj usado para evaluar plazos.

L83

```
83 |     this.mapper = mapper;
```

En this guarda conversor JSON de Jackson. Toma el dato de mapper después de las validaciones de este método. conversor JSON de Jackson.

L84

```
84 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L86

```
86 |   @jakarta.persistence.PersistenceContext private jakarta.persistence.EntityManager
     entityManager;
```

Inyecta EntityManager transaccional. Tipos y datos: entityManager = contexto JPA de la transacción; EntityManager es contexto JPA que administra entidades y sincroniza cambios con SQL.

L88

```
88 |   private Instant now() {
```

Obtiene el instante del reloj inyectado; truncar a microsegundos mantiene consistencia con PostgreSQL.

L89

```
89 |     return clock.instant().truncatedTo(java.time.temporal.ChronoUnit.MICROS);
```

Resultado que recibe el llamador: lee el instante del reloj inyectado.

L90

```
90 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L92

```
92 |   private static void require(boolean ok) {
```

Expresa una precondición; si resulta falsa se aborta con el estado HTTP indicado. Tipos y datos: ok = ok; boolean es verdadero/falso, sin null.

L93

```
93 |     if (!ok) throw new ResponseStatusException(HttpStatus.CONFLICT);
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. interrumpe la operación con un estado HTTP explícito; 409 comunica que el estado/regla actual impide la acción; ! invierte el booleano de la expresión que sigue.

L94

```
94 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L96

```
96 |   private static ResponseStatusException missing() {
```

Construye 404; puede representar recurso inexistente o ajeno para no revelar su existencia.

L97

```
97 |     return new ResponseStatusException(HttpStatus.NOT_FOUND);
```

Resultado que recibe el llamador: interrumpe la operación con un estado HTTP explícito; 404 oculta inexistencia o pertenencia ajena.

L98

```
98 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L100

```
100 |   private View view(ResourceReservation r) {
```

Transforma entidad/datos internos en DTO; no cambia la base por realizar esta proyección.

L101 a L110

```
101 |     return new View(
102 |         r.id,
103 |         r.requestId,
104 |         r.towTruckId,
105 |         r.operatorId,
106 |         r.status,
107 |         r.expiresAt,
108 |         r.confirmedAt,
109 |         r.releasedAt,
110 |         r.createdHold);
```

Resultado que recibe el llamador: Datos implicados: identificador UUID del registro, UUID de solicitud, UUID de grúa, UUID del perfil operador, estado del objeto; su enum o CHECK define valores permitidos, fin exclusivo de retención/propuesta, instante de confirmación en Fleet, instante de liberación, indica si el intento llegó a retener recursos.

L111

```
111 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L113

```
113 |   private String payload(Input d) {
```

Serializa el contenido normalizado que se conserva para reintentar la misma reserva.

L114

```
114 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L115 a L126

```
115 |       return mapper.writeValueAsString(
116 |           new Input(
117 |               d.requestId(),
118 |               d.towTruckId(),
119 |               d.operatorId(),
120 |               d.vehicleType(),
121 |               d.weightKg().stripTrailingZeros(),
122 |               d.nonRollable(),
123 |               d.searchDeadline(),
124 |               d.estimatedServiceEndAt(),
125 |               d.holdSeconds(),
126 |               d.minimumOfferSeconds()));
```

Serializa una entrada de reserva normalizada como JSON. El peso se normaliza sin ceros decimales redundantes; ese payload se conserva para comparar reintentos y no aceptar la misma clave con datos diferentes.

L127

```
127 |     } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L128

```
128 |       throw new IllegalArgumentException("Invalid reservation");
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L129

```
129 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L130

```
130 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L132

```
132 |   private String normalizeStored(String stored) {
```

Reconstruye un payload antiguo con valores por defecto y lo normaliza para idempotencia. Tipos y datos: stored = stored; String es texto; no es por sí mismo una referencia de entidad.

L133

```
133 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L134

```
134 |       return payload(mapper.readValue(stored, Input.class));
```

Reconstruye Input a partir del JSON guardado y vuelve a normalizarlo con payload. Compara el contenido lógico de la reserva, no diferencias superficiales de representación JSON.

L135

```
135 |     } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L136

```
136 |       throw new IllegalStateException("Invalid reservation payload");
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L137

```
137 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L138

```
138 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L140

```
140 |   private void eligible(TowTruck t, Operator o, Input d, Instant now) {
```

Agrupa validaciones de vínculo, registro, disponibilidad, capacidad, cobertura, ubicación y habilitación. Tipos y datos: now = instante actual obtenido del reloj inyectado; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L141

```
141 |     eligible(t, o, d, now, true);
```

Valida conjuntamente grúa, operador, solicitud de reserva e instante. La elegibilidad de recibir/aceptar exige ubicación vigente y recursos compatibles/libres según el paso.

L142

```
142 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L144

```
144 |   private void eligible(TowTruck t, Operator o, Input d, Instant now, boolean freshLocation) {
```

Agrupa validaciones de vínculo, registro, disponibilidad, capacidad, cobertura, ubicación y habilitación. Tipos y datos: now = instante actual obtenido del reloj inyectado; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; freshLocation = freshLocation; boolean es verdadero/falso, sin null.

L145

```
145 |     require(t.operator != null && t.operator.getId().equals(o.id));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara contenido/identidad lógica, no referencias Java; Datos implicados: identificador UUID del registro; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato

L146 a L149

```
146 |     require(
147 |         t.operationalStatus == Operational.ACTIVE
148 |             && o.registrationStatus == Registration.ACTIVE
149 |             && o.availabilityStatus == Availability.AVAILABLE);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. ACTIVE: registro habilitado administrativamente; AVAILABLE: operador dispuesto a recibir propuestas; Datos implicados: estado administrativo de la grúa, alta administrativa ACTIVE/INACTIVE, disposición a recibir propuestas; && exige todas las condiciones y deja de evaluar al encontrar una falsa

L148: ACTIVE: registro habilitado administrativamente; Datos implicados: alta administrativa ACTIVE/INACTIVE; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L149: AVAILABLE: operador dispuesto a recibir propuestas; Datos implicados: disposición a recibir propuestas; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L150 a L153

```
150 |     require(
151 |         t.maximumLoadKg.compareTo(d.weightKg()) >= 0
152 |             && t.supportedTypes.contains(d.vehicleType())
153 |             && (!d.nonRollable() || t.supportsNonRollable));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara decimales: negativo menor, cero igual, positivo mayor; verifica pertenencia al conjunto o lista; Datos implicados: capacidad de carga de la grúa en kg, tipos de vehículo admitidos, capacidad para atender vehículos que no ruedan; && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue

L152: verifica pertenencia al conjunto o lista; Datos implicados: tipos de vehículo admitidos; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L153: Datos implicados: capacidad para atender vehículos que no ruedan; && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L154 a L157

```
154 |     require(
155 |         t.locationUpdatedAt != null
156 |             && !t.locationUpdatedAt.isAfter(now)
157 |             && (!freshLocation || !t.locationUpdatedAt.isBefore(now.minusSeconds(300))));
```

Al reservar/confirmar exige ubicación no futura de hasta 300 s. Para elegibilidad de una reserva ya confirmada freshLocation=false elimina solo el límite de antigüedad. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. calcula un instante anterior restando segundos; compara si el instante es estrictamente posterior al otro; compara si el instante es estrictamente anterior al otro; Datos implicados: instante servidor de última ubicación declarada; && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue; comprueba presencia antes de usar el dato

L156: compara si el instante es estrictamente posterior al otro; Datos implicados: instante servidor de última ubicación declarada; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L157: calcula un instante anterior restando segundos; compara si el instante es estrictamente anterior al otro; Datos implicados: instante servidor de última ubicación declarada; && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L158 a L167

```
158 |     require(
159 |         t.latitude != null
160 |             && t.longitude != null
161 |             && com.auxiruta.commons.GeoMath.distance(
162 |                     -31.4167,
163 |                     -64.1833,
164 |                     t.latitude.doubleValue(),
165 |                     t.longitude.doubleValue(),
166 |                     6371000)
167 |                 <= 15000);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. Datos implicados: latitud declarada, longitud declarada; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato

L160: Datos implicados: longitud declarada; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato.

L161: && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L168 a L173

```
168 |     require(
169 |         o.qualificationCategory == Qualification.LIGHT_TOWING
170 |             && o.qualificationValidFrom != null
171 |             && !o.qualificationValidFrom.isAfter(now)
172 |             && o.qualificationValidUntil != null
173 |             && o.qualificationValidUntil.isAfter(d.estimatedServiceEndAt()));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara si el instante es estrictamente posterior al otro; Datos implicados: habilitación LIGHT_TOWING, inicio de habilitación, fin de habilitación que debe superar el servicio estimado; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue; comprueba presencia antes de usar el dato

L170: Datos implicados: inicio de habilitación; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato.

L171: compara si el instante es estrictamente posterior al otro; Datos implicados: inicio de habilitación; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L172: Datos implicados: fin de habilitación que debe superar el servicio estimado; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato.

L173: compara si el instante es estrictamente posterior al otro; Datos implicados: fin de habilitación que debe superar el servicio estimado; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L174

```
174 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L176

```
176 |   public View reserve(UUID id, Input d) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L177 a L179

```
177 |     require(
178 |         (d.holdSeconds() == 60 && d.minimumOfferSeconds() == 15)
179 |             || (d.holdSeconds() == 10 && d.minimumOfferSeconds() == 3));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera

L179: && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L180

```
180 |     // Every operation involving both resources locks truck, then operator, then reservation.
```

Documentación para quien lee el fuente: Every operation involving both resources locks truck, then operator, then reservation. No es una instrucción ejecutable y no impone la regla por sí sola.

L181

```
181 |     TowTruck t = trucks.lockById(d.towTruckId()).orElseThrow(ReservationService::missing);
```

Si no encuentra resultado lanza la excepción indicada. Se calcula t (t) para reutilizar ese resultado en los pasos siguientes. si no existe valor, lanza la excepción en vez de seguir con null

L182 a L185

```
182 |     Operator o =
183 |         org.hibernate.Hibernate.unproxy(
184 |             operators.lockById(d.operatorId()).orElseThrow(ReservationService::missing),
185 |             Operator.class);
```

Si no encuentra resultado lanza la excepción indicada. Se calcula o (o) para reutilizar ese resultado en los pasos siguientes. si no existe valor, lanza la excepción en vez de seguir con null

L186

```
186 |     var previous = reservations.lockById(id);
```

Se calcula previous (previous) para reutilizar ese resultado en los pasos siguientes.

L187

```
187 |     String payload = payload(d);
```

Campos: payload: contenido normalizado para reintentos. Se calcula payload (contenido normalizado para reintentos) para reutilizar ese resultado en los pasos siguientes.

L188

```
188 |     if (previous.isPresent()) {
```

Esta condición decide si se entra en la rama siguiente. .

L189

```
189 |       require(normalizeStored(previous.get().payload).equals(payload));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara contenido/identidad lógica, no referencias Java

L190

```
190 |       expire(previous.get());
```

Solo cambia HELD a EXPIRED al alcanzar expiresAt; no vence CONFIRMED.

L191

```
191 |       return view(previous.get());
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L192

```
192 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L193

```
193 |     Instant now = now();
```

Se calcula now (instante actual obtenido del reloj inyectado) para reutilizar ese resultado en los pasos siguientes.

L194

```
194 |     boolean eligible = true;
```

Campos: eligible: aptitud completa del candidato/compromiso. Se calcula eligible (aptitud completa del candidato/compromiso) para reutilizar ese resultado en los pasos siguientes. true

L195

```
195 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L196 a L198

```
196 |       require(
197 |           Duration.between(now, d.searchDeadline()).getSeconds() >= d.minimumOfferSeconds()
198 |               && d.estimatedServiceEndAt().isAfter(now));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara si el instante es estrictamente posterior al otro; && exige todas las condiciones y deja de evaluar al encontrar una falsa

L198: compara si el instante es estrictamente posterior al otro; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L199

```
199 |       eligible(t, o, d, now);
```

Valida conjuntamente grúa, operador, solicitud de reserva e instante. La elegibilidad de recibir/aceptar exige ubicación vigente y recursos compatibles/libres según el paso.

L200

```
200 |       require(!reservations.busy(t.id, o.id));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. Datos implicados: identificador UUID del registro; ! invierte el booleano de la expresión que sigue

L201

```
201 |     } catch (ResponseStatusException e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L202

```
202 |       eligible = false;
```

Registra que el recurso no superó la comprobación de elegibilidad; impide confirmar aptitud a ciegas.

L203

```
203 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L204

```
204 |     ResourceReservation r = new ResourceReservation();
```

Construye ResourceReservation en memoria y conserva la referencia en r. Entidad Fleet del compromiso exclusivo de una grúa y un operador: HELD, CONFIRMED, RELEASED o EXPIRED. El ID coincide con el intento de asignación remoto. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L205

```
205 |     r.id = id;
```

En r guarda identificador UUID del registro. El valor id relaciona esta fila con el recurso correspondiente; los UUID remotos se conservan sin FK entre microservicios. identificador UUID del registro.

L206

```
206 |     r.requestId = d.requestId();
```

En r guarda UUID de solicitud. El valor d.requestId() relaciona esta fila con el recurso correspondiente; los UUID remotos se conservan sin FK entre microservicios.

L207

```
207 |     r.towTruckId = t.id;
```

En r guarda UUID de grúa. El valor t.id relaciona esta fila con el recurso correspondiente; los UUID remotos se conservan sin FK entre microservicios. Datos implicados: identificador UUID del registro.

L208

```
208 |     r.operatorId = o.id;
```

En r guarda UUID del perfil operador. El valor o.id relaciona esta fila con el recurso correspondiente; los UUID remotos se conservan sin FK entre microservicios. Datos implicados: identificador UUID del registro.

L209

```
209 |     r.payload = payload;
```

En r guarda contenido normalizado para reintentos. Conserva evidencia estable de la operación para comparar reintentos o reconstruir las reglas utilizadas, aunque cambie la configuración futura. contenido normalizado para reintentos.

L210

```
210 |     r.status = "HELD";
```

En r guarda estado del objeto; su enum o CHECK define valores permitidos. retiene temporalmente recursos mientras el operador responde; excluye otras reservas activas

L211

```
211 |     r.createdAt = now;
```

En r guarda fecha de creación. Marca el instante de esta operación para ordenar y auditar su evolución. instante actual obtenido del reloj inyectado.

L212

```
212 |     r.searchDeadline = d.searchDeadline();
```

En r guarda vencimiento del intento. Toma el dato de d.searchDeadline() después de las validaciones de este método.

L213

```
213 |     r.estimatedServiceEndAt = d.estimatedServiceEndAt();
```

En r guarda fin planificado para controlar habilitación. Toma el dato de d.estimatedServiceEndAt() después de las validaciones de este método.

L214 a L217

```
214 |     r.expiresAt =
215 |         now.plusSeconds(d.holdSeconds()).isBefore(d.searchDeadline())
216 |             ? now.plusSeconds(d.holdSeconds())
217 |             : d.searchDeadline();
```

En r guarda fin exclusivo de retención/propuesta. Determina hasta cuándo es válida la retención/propuesta. El cálculo elige el límite temporal más cercano para no exceder el cierre de búsqueda. calcula un instante futuro sumando segundos, sin modificar el instante original; compara si el instante es estrictamente anterior al otro; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L216: calcula un instante futuro sumando segundos, sin modificar el instante original.

L218

```
218 |     r.createdHold = eligible;
```

En r guarda indica si el intento llegó a retener recursos. Toma el dato de eligible después de las validaciones de este método. aptitud completa del candidato/compromiso.

L219

```
219 |     if (!eligible) {
```

Esta condición decide si se entra en la rama siguiente. ! invierte el booleano de la expresión que sigue.

L220

```
220 |       r.status = "EXPIRED";
```

En r guarda estado del objeto; su enum o CHECK define valores permitidos. la ventana de validez terminó y ya no puede aceptarse como vigente

L221

```
221 |       r.releasedAt = now;
```

En r guarda instante de liberación. Toma el dato de now después de las validaciones de este método. instante actual obtenido del reloj inyectado.

L222

```
222 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L223

```
223 |     return view(reservations.saveAndFlush(r));
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L224

```
224 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L226

```
226 |   private ResourceReservation lock(UUID id) {
```

Adquiere bloqueos en orden para serializar operaciones competidoras. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L227

```
227 |     // Read only IDs before taking resource locks; refresh state under reservation lock.
```

Documentación para quien lee el fuente: Read only IDs before taking resource locks; refresh state under reservation lock. No es una instrucción ejecutable y no impone la regla por sí sola.

L228

```
228 |     ResourceReservation first =
      reservations.findById(id).orElseThrow(ReservationService::missing);
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Se calcula first (first) para reutilizar ese resultado en los pasos siguientes. si no existe valor, lanza la excepción en vez de seguir con null; busca por clave primaria; el resultado puede faltar

L229

```
229 |     trucks.lockById(first.towTruckId).orElseThrow(ReservationService::missing);
```

Si no encuentra resultado lanza la excepción indicada. Si no existe valor, lanza la excepción en vez de seguir con null.

L230

```
230 |     operators.lockById(first.operatorId).orElseThrow(ReservationService::missing);
```

Si no encuentra resultado lanza la excepción indicada. Si no existe valor, lanza la excepción en vez de seguir con null.

L231

```
231 |     entityManager.refresh(first, jakarta.persistence.LockModeType.PESSIMISTIC_WRITE);
```

Recarga bajo bloqueo pesimista después de bloquear grúa y operador: evita usar el estado leído antes de tomar los locks.

L232

```
232 |     return first;
```

Entrega first al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: first.

L233

```
233 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L235

```
235 |   private void expire(ResourceReservation r) {
```

Solo cambia HELD a EXPIRED al alcanzar expiresAt; no vence CONFIRMED.

L236

```
236 |     if (r.status.equals("HELD") && !r.expiresAt.isAfter(now())) {
```

Esta condición decide si se entra en la rama siguiente. compara si el instante es estrictamente posterior al otro; compara contenido/identidad lógica, no referencias Java; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos, fin exclusivo de retención/propuesta; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L237

```
237 |       r.status = "EXPIRED";
```

En r guarda estado del objeto; su enum o CHECK define valores permitidos. la ventana de validez terminó y ya no puede aceptarse como vigente

L238

```
238 |       r.releasedAt = now();
```

En r guarda instante de liberación. Toma el dato de now() después de las validaciones de este método.

L239

```
239 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L240

```
240 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L242

```
242 |   public void expireDue(UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L243

```
243 |     expire(lock(id));
```

Solo cambia HELD a EXPIRED al alcanzar expiresAt; no vence CONFIRMED.

L244

```
244 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L246

```
246 |   public record StatusQuery(UUID assignmentAttemptId, String status, Eligibility resource) {}
```

Declara record inmutable StatusQuery. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Protege la exclusividad mediante bloqueos y restricciones. HELD caduca; CONFIRMED no caduca por el plazo antiguo de la oferta. Exige ubicación de hasta 300 s al reservar/confirmar, pero no vuelve a caducarla después de aceptar. Tipos y datos: assignmentAttemptId = assignmentAttemptId; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; status = estado del objeto; su enum o CHECK define valores permitidos; String es texto; no es por sí mismo una referencia de entidad.

L248

```
248 |   public StatusQuery statusQuery(UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L249

```
249 |     var found = reservations.findById(id);
```

Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Se calcula found (found) para reutilizar ese resultado en los pasos siguientes. busca por clave primaria; el resultado puede faltar

L250

```
250 |     if (found.isEmpty()) return new StatusQuery(id, "MISSING", null);
```

Esta condición decide si se entra en la rama siguiente. comprueba ausencia de elementos o valor.

L251

```
251 |     var r = found.get();
```

Se calcula r (r) para reutilizar ese resultado en los pasos siguientes.

L252

```
252 |     String state = r.status.equals("HELD") && !r.expiresAt.isAfter(now()) ? "EXPIRED" : r.status;
```

Se calcula state (state) para reutilizar ese resultado en los pasos siguientes. compara si el instante es estrictamente posterior al otro; compara contenido/identidad lógica, no referencias Java; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos, fin exclusivo de retención/propuesta; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario

L253

```
253 |     return new StatusQuery(id, state, state.equals("CONFIRMED") ? eligibility(id) : null);
```

Resultado que recibe el llamador: compara contenido/identidad lógica, no referencias Java; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L254

```
254 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L256

```
256 |   public View get(UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L257

```
257 |     ResourceReservation r = lock(id);
```

Se calcula r (r) para reutilizar ese resultado en los pasos siguientes.

L258

```
258 |     expire(r);
```

Solo cambia HELD a EXPIRED al alcanzar expiresAt; no vence CONFIRMED.

L259

```
259 |     return view(r);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L260

```
260 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L262

```
262 |   public View confirm(UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L263

```
263 |     ResourceReservation r = lock(id);
```

Se calcula r (r) para reutilizar ese resultado en los pasos siguientes.

L264

```
264 |     if (r.status.equals("CONFIRMED")) return view(r);
```

Una confirmación repetida devuelve el compromiso ya existente antes de evaluar el vencimiento de la antigua oferta. Esta condición decide si se entra en la rama siguiente. compara contenido/identidad lógica, no referencias Java; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos.

L265 a L267

```
265 |     if (!r.status.equals("HELD")
266 |         || !r.expiresAt.isAfter(now())
267 |         || !r.searchDeadline.isAfter(now())) {
```

Esta condición decide si se entra en la rama siguiente. compara si el instante es estrictamente posterior al otro; compara contenido/identidad lógica, no referencias Java; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos, fin exclusivo de retención/propuesta, vencimiento del intento; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L266: compara si el instante es estrictamente posterior al otro; Datos implicados: fin exclusivo de retención/propuesta; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L267: compara si el instante es estrictamente posterior al otro; Datos implicados: vencimiento del intento; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L268

```
268 |       expire(r);
```

Solo cambia HELD a EXPIRED al alcanzar expiresAt; no vence CONFIRMED.

L269

```
269 |       return view(r);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L270

```
270 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L271

```
271 |     Input d;
```

Declara una variable local para la entrada de reserva reconstruida desde el payload persistido. Se asigna dentro del try siguiente; todavía no crea una reserva.

L272

```
272 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L273

```
273 |       d = mapper.readValue(r.payload, Input.class);
```

Datos implicados: contenido normalizado para reintentos.

L274

```
274 |     } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L275

```
275 |       throw new IllegalStateException("Invalid reservation payload");
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L276

```
276 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L277

```
277 |     TowTruck t = trucks.findById(r.towTruckId).orElseThrow();
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Se calcula t (t) para reutilizar ese resultado en los pasos siguientes. si no existe valor, lanza la excepción en vez de seguir con null; busca por clave primaria; el resultado puede faltar; Datos implicados: UUID de grúa

L278 a L280

```
278 |     Operator o =
279 |         org.hibernate.Hibernate.unproxy(
280 |             operators.findById(r.operatorId).orElseThrow(), Operator.class);
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Se calcula o (o) para reutilizar ese resultado en los pasos siguientes. si no existe valor, lanza la excepción en vez de seguir con null; busca por clave primaria; el resultado puede faltar; Datos implicados: UUID del perfil operador

L281

```
281 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L282

```
282 |       eligible(t, o, d, now());
```

Valida conjuntamente grúa, operador, solicitud de reserva e instante. La elegibilidad de recibir/aceptar exige ubicación vigente y recursos compatibles/libres según el paso.

L283

```
283 |     } catch (ResponseStatusException e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L284

```
284 |       r.status = "RELEASED";
```

En r guarda estado del objeto; su enum o CHECK define valores permitidos. libera el compromiso y conserva el registro como historial

L285

```
285 |       r.releasedAt = now();
```

En r guarda instante de liberación. Toma el dato de now() después de las validaciones de este método.

L286

```
286 |       return view(r);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L287

```
287 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L288

```
288 |     if (!r.expiresAt.isAfter(now())) {
```

Esta condición decide si se entra en la rama siguiente. compara si el instante es estrictamente posterior al otro; Datos implicados: fin exclusivo de retención/propuesta; ! invierte el booleano de la expresión que sigue.

L289

```
289 |       expire(r);
```

Solo cambia HELD a EXPIRED al alcanzar expiresAt; no vence CONFIRMED.

L290

```
290 |       return view(r);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L291

```
291 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L292

```
292 |     r.status = "CONFIRMED";
```

En r guarda estado del objeto; su enum o CHECK define valores permitidos. mantiene el compromiso después de aceptar; no caduca por el vencimiento original de la oferta

L293

```
293 |     r.confirmedAt = now();
```

En r guarda instante de confirmación en Fleet. Toma el dato de now() después de las validaciones de este método.

L294

```
294 |     reservations.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente.

L295

```
295 |     return view(r);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L296

```
296 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L298 a L310

```
298 |   public record Eligibility(
299 |       UUID assignmentAttemptId,
300 |       UUID requestId,
301 |       UUID towTruckId,
302 |       UUID operatorId,
303 |       String operatorSubject,
304 |       boolean eligible,
305 |       String reason,
306 |       BigDecimal latitude,
307 |       BigDecimal longitude,
308 |       Instant locationUpdatedAt,
309 |       Instant qualificationValidUntil,
310 |       Instant checkedAt) {}
```

Declara record inmutable Eligibility. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: requestId: UUID de solicitud; towTruckId: UUID de grúa; operatorId: UUID del perfil operador; operatorSubject: sub del operador destinatario; eligible: aptitud completa del candidato/compromiso; reason: motivo legible; latitude: latitud declarada; longitude: longitud declarada; locationUpdatedAt: instante servidor de última ubicación declarada; qualificationValidUntil: fin de habilitación que debe superar el servicio estimado. Protege la exclusividad mediante bloqueos y restricciones. HELD caduca; CONFIRMED no caduca por el plazo antiguo de la oferta. Exige ubicación de hasta 300 s al reservar/confirmar, pero no vuelve a caducarla después de aceptar. Tipos y datos: assignmentAttemptId = assignmentAttemptId; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; requestId = UUID de solicitud; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; towTruckId = UUID de grúa; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; operatorId = UUID del perfil operador; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; operatorSubject = sub del operador destinatario; String es texto; no es por sí mismo una referencia de entidad; eligible = aptitud completa del candidato/compromiso; boolean es verdadero/falso, sin null; reason = motivo legible; String es texto; no es por sí mismo una referencia de entidad; latitude = latitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; longitude = longitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; locationUpdatedAt = instante servidor de última ubicación declarada; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; qualificationValidUntil = fin de habilitación que debe superar el servicio estimado; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; checkedAt = checkedAt; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L300: Dato del contrato: requestId: UUID de solicitud.

L301: Dato del contrato: towTruckId: UUID de grúa.

L302: Dato del contrato: operatorId: UUID del perfil operador.

L303: Dato del contrato: operatorSubject: sub del operador destinatario.

L304: Dato del contrato: eligible: aptitud completa del candidato/compromiso.

L305: Dato del contrato: reason: motivo legible.

L306: Dato del contrato: latitude: latitud declarada.

L307: Dato del contrato: longitude: longitud declarada.

L308: Dato del contrato: locationUpdatedAt: instante servidor de última ubicación declarada.

L309: Dato del contrato: qualificationValidUntil: fin de habilitación que debe superar el servicio estimado.

L312

```
312 |   public Eligibility eligibility(UUID id) {
```

Consulta si un compromiso confirmado conserva aptitud; no aplica caducidad de ubicación de una propuesta nueva. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L313

```
313 |     ResourceReservation r = lock(id);
```

Se calcula r (r) para reutilizar ese resultado en los pasos siguientes.

L314

```
314 |     TowTruck t = trucks.findById(r.towTruckId).orElseThrow();
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Se calcula t (t) para reutilizar ese resultado en los pasos siguientes. si no existe valor, lanza la excepción en vez de seguir con null; busca por clave primaria; el resultado puede faltar; Datos implicados: UUID de grúa

L315 a L317

```
315 |     Operator o =
316 |         org.hibernate.Hibernate.unproxy(
317 |             operators.findById(r.operatorId).orElseThrow(), Operator.class);
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Se calcula o (o) para reutilizar ese resultado en los pasos siguientes. si no existe valor, lanza la excepción en vez de seguir con null; busca por clave primaria; el resultado puede faltar; Datos implicados: UUID del perfil operador

L318

```
318 |     boolean valid = r.status.equals("CONFIRMED");
```

Se calcula valid (valid) para reutilizar ese resultado en los pasos siguientes. compara contenido/identidad lógica, no referencias Java; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos

L319

```
319 |     String reason = valid ? "ELIGIBLE" : "RESERVATION_NOT_CONFIRMED";
```

Campos: reason: motivo legible. Se calcula reason (motivo legible) para reutilizar ese resultado en los pasos siguientes. el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario

L320 a L321

```
320 |     if (valid)
321 |       try {
```

Esta condición decide si se entra en la rama siguiente. .

L322

```
322 |         Input input = mapper.readValue(r.payload, Input.class);
```

Se calcula input (snapshot de entrada que evita trabajar con una entidad mutable fuera de transacción) para reutilizar ese resultado en los pasos siguientes. Datos implicados: contenido normalizado para reintentos

L323

```
323 |         eligible(t, o, input, now(), false);
```

Revalida recurso confirmado sin caducidad por edad de ubicación. Conserva compatibilidad, disponibilidad, vínculo, cobertura y habilitación; transforma incumplimiento en eligible=false.

L324

```
324 |       } catch (ResponseStatusException e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L325

```
325 |         valid = false;
```

Marca el resultado de validación como negativo; las líneas siguientes conservan el motivo de inelegibilidad.

L326

```
326 |         reason = "RESOURCE_NOT_ELIGIBLE";
```

Conserva una causa estable de rechazo de elegibilidad para comunicarla a Assistance sin exponer detalles internos de la base.

L327

```
327 |       } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L328

```
328 |         throw new IllegalStateException("Invalid reservation payload");
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L329

```
329 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L330 a L342

```
330 |     return new Eligibility(
331 |         r.id,
332 |         r.requestId,
333 |         r.towTruckId,
334 |         r.operatorId,
335 |         o.keycloakSubject,
336 |         valid,
337 |         reason,
338 |         t.latitude,
339 |         t.longitude,
340 |         t.locationUpdatedAt,
341 |         o.qualificationValidUntil,
342 |         now());
```

Devuelve snapshot de aptitud y coordenadas para revalidar despacho; no crea reserva nueva. Resultado que recibe el llamador: Datos implicados: identificador UUID del registro, UUID de solicitud, UUID de grúa, UUID del perfil operador, identidad externa del operador, latitud declarada, longitud declarada, instante servidor de última ubicación declarada, fin de habilitación que debe superar el servicio estimado.

L343

```
343 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L345

```
345 |   public View release(UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L346

```
346 |     ResourceReservation r = lock(id);
```

Se calcula r (r) para reutilizar ese resultado en los pasos siguientes.

L347

```
347 |     if (!Set.of("RELEASED", "EXPIRED").contains(r.status)) {
```

Esta condición decide si se entra en la rama siguiente. verifica pertenencia al conjunto o lista; crea un conjunto no modificable y sin duplicados; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos; ! invierte el booleano de la expresión que sigue.

L348

```
348 |       if (r.status.equals("CONFIRMED")) {
```

Esta condición decide si se entra en la rama siguiente. compara contenido/identidad lógica, no referencias Java; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos.

L349 a L351

```
349 |         Operator o =
350 |             org.hibernate.Hibernate.unproxy(
351 |                 operators.findById(r.operatorId).orElseThrow(), Operator.class);
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Se calcula o (o) para reutilizar ese resultado en los pasos siguientes. si no existe valor, lanza la excepción en vez de seguir con null; busca por clave primaria; el resultado puede faltar; Datos implicados: UUID del perfil operador

L352

```
352 |         if (o.availabilityStatus == Availability.AVAILABLE) {
```

Esta condición decide si se entra en la rama siguiente. AVAILABLE: operador dispuesto a recibir propuestas; Datos implicados: disposición a recibir propuestas.

L353

```
353 |           o.availableSince = now();
```

En o guarda inicio de disponibilidad usado para desempate. Toma el dato de now() después de las validaciones de este método.

L354

```
354 |           o.updatedAt = now();
```

En o guarda fecha de modificación. Marca el instante de esta operación para ordenar y auditar su evolución.

L355

```
355 |         }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L356

```
356 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L357

```
357 |       r.status = "RELEASED";
```

En r guarda estado del objeto; su enum o CHECK define valores permitidos. libera el compromiso y conserva el registro como historial

L358

```
358 |       r.releasedAt = now();
```

En r guarda instante de liberación. Toma el dato de now() después de las validaciones de este método.

L359

```
359 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L360

```
360 |     return view(r);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L361

```
361 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L362

```
362 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 13, 32, 55, 66, 72, 85, 87, 91, 95, 99, 112, 131, 139, 143, 175, 225, 234, 241, 245, 247, 255, 261, 297, 311, 344: separadores sin instrucciones.

6. Vehículos, solicitudes y reglas

La exclusividad es por vehículo, no por conductor. firstAssignedAt hace irreversible la prohibición de cancelar.

DomainTypes (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/DomainTypes.java

Objetivo y funcionalidad

Enumeraciones de Assistance: tipo de vehículo, movilidad, registro y ciclo de vida de la solicitud. Los estados de asignación y reserva se almacenan aparte.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | public final class DomainTypes {
```

Declara clase DomainTypes. Enumeraciones de Assistance: tipo de vehículo, movilidad, registro y ciclo de vida de la solicitud. Los estados de asignación y reserva se almacenan aparte.

L4

```
4 |   private DomainTypes() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L6

```
6 |   public enum VehicleType {
```

Declara enumeración cerrada VehicleType. Enumeraciones de Assistance: tipo de vehículo, movilidad, registro y ciclo de vida de la solicitud. Los estados de asignación y reserva se almacenan aparte.

L7 a L10

```
7 |     CAR,
8 |     SUV,
9 |     PICKUP
10 |   }
```

Valores cerrados de esta enumeración: CAR, SUV, PICKUP. El tipo impide usar cadenas arbitrarias; representan alternativas del dominio. Las transiciones válidas se controlan en los servicios; enumerar estados no permite saltar entre ellos libremente.

L12

```
12 |   public enum Registration {
```

Declara enumeración cerrada Registration. Enumeraciones de Assistance: tipo de vehículo, movilidad, registro y ciclo de vida de la solicitud. Los estados de asignación y reserva se almacenan aparte.

L13 a L15

```
13 |     ACTIVE,
14 |     INACTIVE
15 |   }
```

Valores cerrados de esta enumeración: ACTIVE, INACTIVE. El tipo impide usar cadenas arbitrarias; representan alternativas del dominio. Las transiciones válidas se controlan en los servicios; enumerar estados no permite saltar entre ellos libremente.

L17

```
17 |   public enum Mobility {
```

Declara enumeración cerrada Mobility. Enumeraciones de Assistance: tipo de vehículo, movilidad, registro y ciclo de vida de la solicitud. Los estados de asignación y reserva se almacenan aparte.

L18 a L20

```
18 |     ROLLABLE,
19 |     NON_ROLLABLE
20 |   }
```

Valores cerrados de esta enumeración: ROLLABLE, NON_ROLLABLE. El tipo impide usar cadenas arbitrarias; representan alternativas del dominio. Las transiciones válidas se controlan en los servicios; enumerar estados no permite saltar entre ellos libremente.

L22

```
22 |   public enum Status {
```

Declara enumeración cerrada Status. Enumeraciones de Assistance: tipo de vehículo, movilidad, registro y ciclo de vida de la solicitud. Los estados de asignación y reserva se almacenan aparte.

L23 a L36

```
23 |     REQUESTED,
24 |     PENDING_EVALUATION,
25 |     AWAITING_ACCEPTANCE,
26 |     ASSIGNED,
27 |     SEARCHING_ALTERNATIVE,
28 |     DISPATCHED,
29 |     ARRIVED,
30 |     VEHICLE_LOADED,
31 |     IN_TRANSIT,
32 |     COMPLETED,
33 |     REJECTED,
34 |     UNFULFILLED,
35 |     CANCELLED
36 |   }
```

Valores cerrados de esta enumeración: REQUESTED, PENDING_EVALUATION, AWAITING_ACCEPTANCE, ASSIGNED, SEARCHING_ALTERNATIVE, DISPATCHED, ARRIVED, VEHICLE_LOADED, IN_TRANSIT, COMPLETED, REJECTED, UNFULFILLED, CANCELLED. El tipo impide usar cadenas arbitrarias; representan alternativas del dominio. Las transiciones válidas se controlan en los servicios; enumerar estados no permite saltar entre ellos libremente.

L37

```
37 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 5, 11, 16, 21: separadores sin instrucciones.

RequestController (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/RequestController.java`

Objetivo y funcionalidad

API de vehículos propios y solicitudes. Crear una solicitud persiste, evalúa y propone; devolver 201 indica creación, no necesariamente asignación exitosa.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.net.URI;
```

Importa tipos: java.net.URI. Tipo auxiliar usado por las expresiones del archivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import jakarta.validation.Valid;
```

Importa tipos: jakarta.validation.Valid. Restricciones de DTO y activación de validación de entradas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.web.bind.annotation.*;
```

Importa tipos: org.springframework.web.bind.annotation.*. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.http.ResponseEntity;
```

Importa tipos: org.springframework.http.ResponseEntity. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.springframework.security.core.annotation.AuthenticationPrincipal;
```

Importa tipos: org.springframework.security.core.annotation.AuthenticationPrincipal. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | import org.springframework.security.oauth2.jwt.Jwt;
```

Importa tipos: org.springframework.security.oauth2.jwt.Jwt. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import io.swagger.v3.oas.annotations.Operation;
```

Importa tipos: io.swagger.v3.oas.annotations.Operation. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | import io.swagger.v3.oas.annotations.responses.*;
```

Importa tipos: io.swagger.v3.oas.annotations.responses.*. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12

```
12 | import io.swagger.v3.oas.annotations.security.SecurityRequirement;
```

Importa tipos: io.swagger.v3.oas.annotations.security.SecurityRequirement. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L13

```
13 | import static com.auxiruta.assistance.requests.RequestDtos.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.RequestDtos.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L14

```
14 | import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L16

```
16 | @RestController
```

Expone HTTP y serializa JSON.

L17

```
17 | @RequestMapping("/api/v1")
```

Ruta /api/v1 combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L18

```
18 | @SecurityRequirement(name = "bearerAuth")
```

Documenta autenticación; no concede permisos.

L19 a L28

```
19 | @ApiResponses({
20 |   @ApiResponse(responseCode = "400", description = "Entrada inválida"),
21 |   @ApiResponse(responseCode = "401", description = "Autenticación requerida"),
22 |   @ApiResponse(responseCode = "403", description = "Rol insuficiente"),
23 |   @ApiResponse(responseCode = "404", description = "Recurso inexistente o ajeno"),
24 |   @ApiResponse(
25 |       responseCode = "409",
26 |       description = "Conflicto de estado, exclusividad o idempotencia")
27 | })
28 | public class RequestController {
```

Documenta código HTTP. Declara clase RequestController. API de vehículos propios y solicitudes. Crear una solicitud persiste, evalúa y propone; devolver 201 indica creación, no necesariamente asignación exitosa.

L29

```
29 |   private final RequestService service;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L30

```
30 |   private final RulesConfiguration rules;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L31

```
31 |   private final EvaluationService evaluation;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L32

```
32 |   private final AssignmentCoordinator assignments;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L34 a L38

```
34 |   public RequestController(
35 |       RequestService service,
36 |       RulesConfiguration rules,
37 |       EvaluationService evaluation,
38 |       AssignmentCoordinator assignments) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L39

```
39 |     this.service = service;
```

En this guarda service. Toma el dato de service después de las validaciones de este método. service.

L40

```
40 |     this.rules = rules;
```

En this guarda configuración o snapshot de reglas que rige esta operación. Toma el dato de rules después de las validaciones de este método. configuración o snapshot de reglas que rige esta operación.

L41

```
41 |     this.evaluation = evaluation;
```

En this guarda evaluation. Toma el dato de evaluation después de las validaciones de este método. evaluation.

L42

```
42 |     this.assignments = assignments;
```

En this guarda assignments. Toma el dato de assignments después de las validaciones de este método. assignments.

L43

```
43 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L45

```
45 |   @ApiResponse(responseCode = "201", description = "Vehículo creado con Location")
```

Documenta código HTTP.

L46

```
46 |   @PostMapping("/vehicles")
```

Ruta /vehicles combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L47

```
47 |   @Operation(summary = "DRIVER: registrar vehículo propio ACTIVE")
```

Documenta en OpenAPI: DRIVER: registrar vehículo propio ACTIVE. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L48 a L49

```
48 |   public ResponseEntity<VehicleView> createVehicle(
49 |       @AuthenticationPrincipal Jwt jwt, @Valid @RequestBody CreateVehicle d) {
```

Obtiene JWT validado, no identidad suministrada por el cliente. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Tipos y datos: jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias.

L50

```
50 |     var v = service.createVehicle(jwt.getSubject(), d);
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Se calcula v (v) para reutilizar ese resultado en los pasos siguientes. obtiene sub, identidad estable del usuario autenticado

L51

```
51 |     return ResponseEntity.created(URI.create("/api/v1/vehicles/" + v.id())).body(v);
```

Responde 201 con Location. Resultado que recibe el llamador: crea una respuesta 201 con dirección Location del recurso; Datos implicados: identificador UUID del registro.

L52

```
52 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L54

```
54 |   @GetMapping("/vehicles")
```

Ruta /vehicles combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L55

```
55 |   @Operation(summary = "DRIVER: listar exclusivamente vehículos propios")
```

Documenta en OpenAPI: DRIVER: listar exclusivamente vehículos propios. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L56 a L60

```
56 |   public PageView<VehicleView> vehicles(
57 |       @AuthenticationPrincipal Jwt jwt,
58 |       @RequestParam(defaultValue = "0") int page,
59 |       @RequestParam(defaultValue = "20") int size,
60 |       @RequestParam(required = false) Registration registrationStatus) {
```

Obtiene JWT validado, no identidad suministrada por el cliente. Lee parámetro/filtro URL y su default. Campos: registrationStatus: alta administrativa ACTIVE/INACTIVE. Tipos y datos: jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias; page = page; int es entero de 32 bits para límites y contadores; size = size; int es entero de 32 bits para límites y contadores.

L60: Dato del contrato: registrationStatus: alta administrativa ACTIVE/INACTIVE.

L61

```
61 |     return service.vehicles(jwt.getSubject(), page, size, registrationStatus);
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Resultado que recibe el llamador: obtiene sub, identidad estable del usuario autenticado.

L62

```
62 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L64

```
64 |   @GetMapping("/vehicles/{id}")
```

Ruta /vehicles/{id} combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L65

```
65 |   @Operation(summary = "DRIVER: consultar vehículo propio; ajeno e inexistente devuelven 404")
```

Documenta en OpenAPI: DRIVER: consultar vehículo propio; ajeno e inexistente devuelven 404. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L66

```
66 |   public VehicleView vehicle(@PathVariable UUID id, @AuthenticationPrincipal Jwt jwt) {
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias.

L67

```
67 |     return service.vehicle(id, jwt.getSubject());
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Resultado que recibe el llamador: obtiene sub, identidad estable del usuario autenticado.

L68

```
68 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L70

```
70 |   @PatchMapping("/vehicles/{id}")
```

Ruta /vehicles/{id} combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L71

```
71 |   @Operation(summary = "DRIVER: editar vehículo propio sin solicitud activa")
```

Documenta en OpenAPI: DRIVER: editar vehículo propio sin solicitud activa. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L72 a L73

```
72 |   public VehicleView edit(
73 |       @PathVariable UUID id, @AuthenticationPrincipal Jwt jwt, @Valid @RequestBody EditVehicle d)
      {
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias.

L73: Dato del contrato: id: identificador UUID del registro.

L74

```
74 |     return service.editVehicle(id, jwt.getSubject(), d);
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Resultado que recibe el llamador: obtiene sub, identidad estable del usuario autenticado.

L75

```
75 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L77

```
77 |   @PatchMapping("/vehicles/{id}/registration-status")
```

Ruta /vehicles/{id}/registration-status combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L78

```
78 |   @Operation(summary = "DRIVER: activar/inactivar vehículo sin solicitud activa")
```

Documenta en OpenAPI: DRIVER: activar/inactivar vehículo sin solicitud activa. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L79 a L82

```
79 |   public VehicleView registration(
80 |       @PathVariable UUID id,
81 |       @AuthenticationPrincipal Jwt jwt,
82 |       @Valid @RequestBody RegistrationChange d) {
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias.

L80: Dato del contrato: id: identificador UUID del registro.

L83

```
83 |     return service.registration(id, jwt.getSubject(), d);
```

Cambia ACTIVE/INACTIVE del perfil y retira disponibilidad al inactivarlo si las reglas permiten la operación. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L84

```
84 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L86 a L90

```
86 |   @ApiResponses({
87 |     @ApiResponse(responseCode = "201", description = "Solicitud creada con Location"),
88 |     @ApiResponse(responseCode = "200", description = "Solicitud recuperada por Idempotency-Key")
89 |   })
90 |   @PostMapping("/assistance-requests")
```

Documenta código HTTP. Ruta /assistance-requests combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L91 a L97

```
91 |   @Operation(
92 |       summary =
93 |           "DRIVER: crear y evaluar solicitud con snapshot; repetir Idempotency-Key y cuerpo
     recupera la solicitud")
94 |   public ResponseEntity<RequestView> create(
95 |       @AuthenticationPrincipal Jwt jwt,
96 |       @RequestHeader("Idempotency-Key") String key,
97 |       @Valid @RequestBody CreateRequest d) {
```

Documenta en OpenAPI: DRIVER: crear y evaluar solicitud con snapshot; repetir Idempotency-Key y cuerpo recupera la solicitud. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L98

```
98 |     var result = service.createRequest(jwt.getSubject(), key, d);
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Se calcula result (resultado de evaluación, distinto del estado de solicitud) para reutilizar ese resultado en los pasos siguientes. obtiene sub, identidad estable del usuario autenticado

L99

```
99 |     if (result.created()) {
```

Esta condición decide si se entra en la rama siguiente. .

L100

```
100 |       evaluation.evaluate(result.request().id());
```

Calcula y persiste evaluación con cobertura, Fleet y rutas ORS; puede dejar pendiente por un fallo temporal. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L101

```
101 |       assignments.propose(result.request().id());
```

Selecciona candidato evaluado e inicia reserva/propuesta si siguen vigentes las condiciones. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L102

```
102 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L103 a L105

```
103 |     return ResponseEntity.status(result.created() ? 201 : 200)
104 |         .location(URI.create("/api/v1/assistance-requests/" + result.request().id()))
105 |         .body(service.request(result.request().id(), jwt));
```

Construye respuesta con estado explícito y cuerpo ProblemDetail. Resultado que recibe el llamador: el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L106

```
106 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L108

```
108 |   @GetMapping("/assistance-requests")
```

Ruta /assistance-requests combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L109 a L116

```
109 |   @Operation(
110 |       summary =
111 |           "DRIVER: solicitudes propias; ADMIN: globales; OPERATOR: solicitudes con propuestas o
      asignaciones propias")
112 |   public PageView<RequestView> requests(
113 |       @AuthenticationPrincipal Jwt jwt,
114 |       @RequestParam(defaultValue = "0") int page,
115 |       @RequestParam(defaultValue = "20") int size,
116 |       @RequestParam(required = false) Status status) {
```

Documenta en OpenAPI: DRIVER: solicitudes propias; ADMIN: globales; OPERATOR: solicitudes con propuestas o asignaciones propias. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L116: Dato del contrato: status: estado del objeto; su enum o CHECK define valores permitidos.

L117

```
117 |     return service.requests(jwt, page, size, status);
```

Lista solicitudes dentro del ámbito del JWT, con página, tamaño y filtro de estado. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L118

```
118 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L120

```
120 |   @GetMapping("/assistance-requests/{id}")
```

Ruta /assistance-requests/{id} combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L121

```
121 |   @Operation(summary = "Consultar solicitud autorizada; fuera del ámbito devuelve 404")
```

Documenta en OpenAPI: Consultar solicitud autorizada; fuera del ámbito devuelve 404. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L122

```
122 |   public RequestView request(@PathVariable UUID id, @AuthenticationPrincipal Jwt jwt) {
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias.

L123

```
123 |     return service.request(id, jwt);
```

Devuelve la solicitud solo si la identidad tiene alcance sobre ella. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L124

```
124 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L126

```
126 |   @GetMapping("/assistance-requests/{id}/history")
```

Ruta /assistance-requests/{id}/history combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L127

```
127 |   @Operation(summary = "Consultar historial paginado dentro del mismo ámbito de la solicitud")
```

Documenta en OpenAPI: Consultar historial paginado dentro del mismo ámbito de la solicitud. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L128 a L132

```
128 |   public PageView<HistoryView> history(
129 |       @PathVariable UUID id,
130 |       @AuthenticationPrincipal Jwt jwt,
131 |       @RequestParam(defaultValue = "0") int page,
132 |       @RequestParam(defaultValue = "20") int size) {
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Lee parámetro/filtro URL y su default. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias; page = page; int es entero de 32 bits para límites y contadores; size = size; int es entero de 32 bits para límites y contadores.

L129: Dato del contrato: id: identificador UUID del registro.

L133

```
133 |     return service.history(id, jwt, page, size);
```

Consulta eventos de la solicitud con el mismo control de alcance y paginación. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L134

```
134 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L136

```
136 |   @PostMapping("/assistance-requests/{id}/cancellations")
```

Ruta /assistance-requests/{id}/cancellations combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L137 a L144

```
137 |   @Operation(
138 |       summary =
139 |           "DRIVER propietario: cancelar antes de primera aceptación; Idempotency-Key identifica
      el evento durable")
140 |   public HistoryView cancel(
141 |       @PathVariable UUID id,
142 |       @AuthenticationPrincipal Jwt jwt,
143 |       @RequestHeader("Idempotency-Key") String key,
144 |       @Valid @RequestBody Cancellation d) {
```

Documenta en OpenAPI: DRIVER propietario: cancelar antes de primera aceptación; Idempotency-Key identifica el evento durable. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L141: Dato del contrato: id: identificador UUID del registro.

L145

```
145 |     var result = service.cancel(id, jwt.getSubject(), key, d);
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Se calcula result (resultado de evaluación, distinto del estado de solicitud) para reutilizar ese resultado en los pasos siguientes. obtiene sub, identidad estable del usuario autenticado

L146

```
146 |     assignments.recover(id);
```

Retoma acciones remotas durables pendientes de esta solicitud. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L147

```
147 |     return result;
```

Entrega result al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: resultado de evaluación, distinto del estado de solicitud.

L148

```
148 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L150

```
150 |   @GetMapping("/configuration/assistance")
```

Ruta /configuration/assistance combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L151

```
151 |   @Operation(summary = "ADMIN: consultar reglas efectivas sin secretos")
```

Documenta en OpenAPI: ADMIN: consultar reglas efectivas sin secretos. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L152

```
152 |   public java.util.Map<String, Object> configuration() {
```

Endpoint de consulta de reglas efectivas para ADMIN. Permite verificar límites/perfil sin devolver contraseñas ni claves externas.

L153

```
153 |     return rules.effective();
```

Expone reglas efectivas al ADMIN; no devuelve secretos de infraestructura.

L154

```
154 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L155

```
155 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 15, 33, 44, 53, 63, 69, 76, 85, 107, 119, 125, 135, 149: separadores sin instrucciones.

RequestDtos (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/RequestDtos.java`

Objetivo y funcionalidad

DTO de vehículos, solicitudes, historial y paginación. Los cuerpos se validan antes de ingresar en las reglas del servicio.

L1

```
  1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import jakarta.validation.constraints.*;
```

Importa tipos: jakarta.validation.constraints.*. Restricciones de DTO y activación de validación de entradas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | public final class RequestDtos {
```

Declara clase RequestDtos. DTO de vehículos, solicitudes, historial y paginación. Los cuerpos se validan antes de ingresar en las reglas del servicio.

L10

```
10 |   private RequestDtos() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L12 a L18

```
12 |   public record CreateVehicle(
13 |       @NotBlank @Size(max = 30) String licensePlate,
14 |       @Size(max = 100) String brand,
15 |       @Size(max = 100) String model,
16 |       @NotNull VehicleType vehicleType,
17 |       @NotNull @DecimalMin("500") @DecimalMax("3000") @Digits(integer = 4, fraction = 2)
18 |           BigDecimal declaredWeightKg) {}
```

Exige texto no vacío ni solo espacios. Limita caracteres/elementos según min/max. Exige no null. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Declara record inmutable CreateVehicle. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: licensePlate: patente normalizada y única; brand: marca declarada; model: modelo declarado; vehicleType: tipo CAR/SUV/PICKUP; declaredWeightKg: peso declarado en kg. Se calcula fraction (fraction) para reutilizar ese resultado en los pasos siguientes. DTO de vehículos, solicitudes, historial y paginación. Los cuerpos se validan antes de ingresar en las reglas del servicio. Tipos y datos: licensePlate = patente normalizada y única; String es texto; no es por sí mismo una referencia de entidad; brand = marca declarada; String es texto; no es por sí mismo una referencia de entidad; model = modelo declarado; String es texto; no es por sí mismo una referencia de entidad; declaredWeightKg = peso declarado en kg; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L13: Dato del contrato: licensePlate: patente normalizada y única.

L14: Dato del contrato: brand: marca declarada.

L15: Dato del contrato: model: modelo declarado.

L16: Dato del contrato: vehicleType: tipo CAR/SUV/PICKUP.

L18: Dato del contrato: declaredWeightKg: peso declarado en kg.

L20 a L26

```
20 |   public record EditVehicle(
21 |       @Size(max = 30) String licensePlate,
22 |       @Size(max = 100) String brand,
23 |       @Size(max = 100) String model,
24 |       VehicleType vehicleType,
25 |       @DecimalMin("500") @DecimalMax("3000") @Digits(integer = 4, fraction = 2)
26 |           BigDecimal declaredWeightKg) {}
```

Limita caracteres/elementos según min/max. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Declara record inmutable EditVehicle. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: licensePlate: patente normalizada y única; brand: marca declarada; model: modelo declarado; vehicleType: tipo CAR/SUV/PICKUP; declaredWeightKg: peso declarado en kg. Se calcula fraction (fraction) para reutilizar ese resultado en los pasos siguientes. DTO de vehículos, solicitudes, historial y paginación. Los cuerpos se validan antes de ingresar en las reglas del servicio. Tipos y datos: licensePlate = patente normalizada y única; String es texto; no es por sí mismo una referencia de entidad; brand = marca declarada; String es texto; no es por sí mismo una referencia de entidad; model = modelo declarado; String es texto; no es por sí mismo una referencia de entidad; declaredWeightKg = peso declarado en kg; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L21: Dato del contrato: licensePlate: patente normalizada y única.

L22: Dato del contrato: brand: marca declarada.

L23: Dato del contrato: model: modelo declarado.

L26: Dato del contrato: declaredWeightKg: peso declarado en kg.

L28

```
28 |   public record RegistrationChange(@NotNull Registration status) {}
```

Exige no null. Declara record inmutable RegistrationChange. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: status: estado del objeto; su enum o CHECK define valores permitidos. DTO de vehículos, solicitudes, historial y paginación. Los cuerpos se validan antes de ingresar en las reglas del servicio.

L30 a L41

```
30 |   public record CreateRequest(
31 |       @NotNull UUID vehicleId,
32 |       @NotNull @DecimalMin("-90") @DecimalMax("90") @Digits(integer = 3, fraction = 6)
33 |           BigDecimal originLatitude,
34 |       @NotNull @DecimalMin("-180") @DecimalMax("180") @Digits(integer = 3, fraction = 6)
35 |           BigDecimal originLongitude,
36 |       @NotNull @DecimalMin("-90") @DecimalMax("90") @Digits(integer = 3, fraction = 6)
37 |           BigDecimal destinationLatitude,
38 |       @NotNull @DecimalMin("-180") @DecimalMax("180") @Digits(integer = 3, fraction = 6)
39 |           BigDecimal destinationLongitude,
40 |       @NotNull Mobility mobilityCondition,
41 |       @NotBlank @Size(max = 1000) String description) {}
```

Exige no null. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Exige texto no vacío ni solo espacios. Limita caracteres/elementos según min/max. Declara record inmutable CreateRequest. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: originLatitude: latitud de recogida; originLongitude: longitud de recogida; destinationLatitude: latitud de entrega; destinationLongitude: longitud de entrega; mobilityCondition: condición ROLLABLE/NON_ROLLABLE; description: descripción de asistencia. Se calcula fraction (fraction) para reutilizar ese resultado en los pasos siguientes. DTO de vehículos, solicitudes, historial y paginación. Los cuerpos se validan antes de ingresar en las reglas del servicio. Tipos y datos: vehicleId = vehicleId; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; originLatitude = latitud de recogida; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; originLongitude = longitud de recogida; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; destinationLatitude = latitud de entrega; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; destinationLongitude = longitud de entrega; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; description = descripción de asistencia; String es texto; no es por sí mismo una referencia de entidad.

L33: Dato del contrato: originLatitude: latitud de recogida.

L35: Dato del contrato: originLongitude: longitud de recogida.

L37: Dato del contrato: destinationLatitude: latitud de entrega.

L39: Dato del contrato: destinationLongitude: longitud de entrega.

L40: Dato del contrato: mobilityCondition: condición ROLLABLE/NON_ROLLABLE.

L41: Dato del contrato: description: descripción de asistencia.

L43

```
43 |   public record Cancellation(@NotBlank @Size(max = 500) String reason) {}
```

Exige texto no vacío ni solo espacios. Limita caracteres/elementos según min/max. Declara record inmutable Cancellation. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: reason: motivo legible. DTO de vehículos, solicitudes, historial y paginación. Los cuerpos se validan antes de ingresar en las reglas del servicio. Tipos y datos: reason = motivo legible; String es texto; no es por sí mismo una referencia de entidad.

L45 a L55

```
45 |   public record VehicleView(
46 |       UUID id,
47 |       String licensePlate,
48 |       String brand,
49 |       String model,
50 |       VehicleType vehicleType,
51 |       BigDecimal declaredWeightKg,
52 |       Registration registrationStatus,
53 |       Instant createdAt,
54 |       Instant updatedAt,
55 |       long version) {}
```

Declara record inmutable VehicleView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; licensePlate: patente normalizada y única; brand: marca declarada; model: modelo declarado; vehicleType: tipo CAR/SUV/PICKUP; declaredWeightKg: peso declarado en kg; registrationStatus: alta administrativa ACTIVE/INACTIVE; createdAt: fecha de creación; updatedAt: fecha de modificación; version: versión para control optimista de concurrencia. DTO de vehículos, solicitudes, historial y paginación. Los cuerpos se validan antes de ingresar en las reglas del servicio. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; licensePlate = patente normalizada y única; String es texto; no es por sí mismo una referencia de entidad; brand = marca declarada; String es texto; no es por sí mismo una referencia de entidad; model = modelo declarado; String es texto; no es por sí mismo una referencia de entidad; declaredWeightKg = peso declarado en kg; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; createdAt = fecha de creación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; updatedAt = fecha de modificación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; version = versión para control optimista de concurrencia; long es entero de 64 bits para cantidades y duraciones mayores.

L46: Dato del contrato: id: identificador UUID del registro.

L47: Dato del contrato: licensePlate: patente normalizada y única.

L48: Dato del contrato: brand: marca declarada.

L49: Dato del contrato: model: modelo declarado.

L51: Dato del contrato: declaredWeightKg: peso declarado en kg.

L53: Dato del contrato: createdAt: fecha de creación.

L54: Dato del contrato: updatedAt: fecha de modificación.

L55: Dato del contrato: version: versión para control optimista de concurrencia.

L57 a L80

```
57 |   public record RequestView(
58 |       UUID id,
59 |       UUID vehicleId,
60 |       Status status,
61 |       Instant firstAssignedAt,
62 |       String closureReasonCode,
63 |       BigDecimal originLatitude,
64 |       BigDecimal originLongitude,
65 |       BigDecimal destinationLatitude,
66 |       BigDecimal destinationLongitude,
67 |       VehicleType declaredVehicleType,
68 |       BigDecimal declaredWeightKg,
69 |       Mobility mobilityCondition,
70 |       String description,
71 |       int searchAttemptNumber,
72 |       Instant searchDeadline,
73 |       String configurationRevision,
74 |       String configurationSnapshot,
75 |       Instant createdAt,
76 |       Instant updatedAt,
77 |       Instant closedAt,
78 |       long version,
79 |       boolean resourceReleasePending,
80 |       EvaluationReadService.EvaluationView latestEvaluation) {}
```

Declara record inmutable RequestView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; status: estado del objeto; su enum o CHECK define valores permitidos; firstAssignedAt: primera aceptación que impide cancelar incluso durante reemplazos; closureReasonCode: causa codificada del cierre; originLatitude: latitud de recogida; originLongitude: longitud de recogida; destinationLatitude: latitud de entrega; destinationLongitude: longitud de entrega; declaredVehicleType: copia del tipo de vehículo al solicitar; declaredWeightKg: peso declarado en kg; mobilityCondition: condición ROLLABLE/NON_ROLLABLE; description: descripción de asistencia; searchAttemptNumber: número de búsqueda de 1 a 3; searchDeadline: vencimiento del intento; configurationRevision: versión de reglas; configurationSnapshot: JSON de parámetros vigente al crear; createdAt: fecha de creación; updatedAt: fecha de modificación; closedAt: fecha de cierre; null indica abierto; version: versión para control optimista de concurrencia; resourceReleasePending: liberación remota pendiente que impide reutilización. DTO de vehículos, solicitudes, historial y paginación. Los cuerpos se validan antes de ingresar en las reglas del servicio. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; vehicleId = vehicleId; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; firstAssignedAt = primera aceptación que impide cancelar incluso durante reemplazos; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; closureReasonCode = causa codificada del cierre; String es texto; no es por sí mismo una referencia de entidad; originLatitude = latitud de recogida; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; originLongitude = longitud de recogida; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; destinationLatitude = latitud de entrega; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; destinationLongitude = longitud de entrega; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; declaredWeightKg = peso declarado en kg; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; description = descripción de asistencia; String es texto; no es por sí mismo una referencia de entidad; searchAttemptNumber = número de búsqueda de 1 a 3; int es entero de 32 bits para límites y contadores; searchDeadline = vencimiento del intento; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; configurationRevision = versión de reglas; String es texto; no es por sí mismo una referencia de entidad; configurationSnapshot = JSON de parámetros vigente al crear; String es texto; no es por sí mismo una referencia de entidad; createdAt = fecha de creación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; updatedAt = fecha de modificación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; closedAt = fecha de cierre; null indica abierto; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; version = versión para control optimista de concurrencia; long es entero de 64 bits para cantidades y duraciones mayores; resourceReleasePending = liberación remota pendiente que impide reutilización; boolean es verdadero/falso, sin null.

L58: Dato del contrato: id: identificador UUID del registro.

L61: Dato del contrato: firstAssignedAt: primera aceptación que impide cancelar incluso durante reemplazos.

L62: Dato del contrato: closureReasonCode: causa codificada del cierre.

L63: Dato del contrato: originLatitude: latitud de recogida.

L64: Dato del contrato: originLongitude: longitud de recogida.

L65: Dato del contrato: destinationLatitude: latitud de entrega.

L66: Dato del contrato: destinationLongitude: longitud de entrega.

L68: Dato del contrato: declaredWeightKg: peso declarado en kg.

L70: Dato del contrato: description: descripción de asistencia.

L71: Dato del contrato: searchAttemptNumber: número de búsqueda de 1 a 3.

L72: Dato del contrato: searchDeadline: vencimiento del intento.

L73: Dato del contrato: configurationRevision: versión de reglas.

L74: Dato del contrato: configurationSnapshot: JSON de parámetros vigente al crear.

L75: Dato del contrato: createdAt: fecha de creación.

L76: Dato del contrato: updatedAt: fecha de modificación.

L77: Dato del contrato: closedAt: fecha de cierre; null indica abierto.

L78: Dato del contrato: version: versión para control optimista de concurrencia.

L79: Dato del contrato: resourceReleasePending: liberación remota pendiente que impide reutilización.

L82 a L92

```
82 |   public record HistoryView(
83 |       UUID id,
84 |       UUID requestId,
85 |       Status previousStatus,
86 |       Status newStatus,
87 |       String actorType,
88 |       String actorSubject,
89 |       String reason,
90 |       String details,
91 |       String operationKey,
92 |       Instant occurredAt) {}
```

Declara record inmutable HistoryView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; requestId: UUID de solicitud; previousStatus: estado previo; newStatus: estado resultante; actorType: tipo de actor SYSTEM/DRIVER/OPERATOR; actorSubject: sub del usuario que produjo el evento; reason: motivo legible; details: evidencia adicional; operationKey: clave durable de acción; occurredAt: instante del evento. DTO de vehículos, solicitudes, historial y paginación. Los cuerpos se validan antes de ingresar en las reglas del servicio. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; requestId = UUID de solicitud; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; actorType = tipo de actor SYSTEM/DRIVER/OPERATOR; String es texto; no es por sí mismo una referencia de entidad; actorSubject = sub del usuario que produjo el evento; String es texto; no es por sí mismo una referencia de entidad; reason = motivo legible; String es texto; no es por sí mismo una referencia de entidad; details = evidencia adicional; String es texto; no es por sí mismo una referencia de entidad; operationKey = clave durable de acción; String es texto; no es por sí mismo una referencia de entidad; occurredAt = instante del evento; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L83: Dato del contrato: id: identificador UUID del registro.

L84: Dato del contrato: requestId: UUID de solicitud.

L87: Dato del contrato: actorType: tipo de actor SYSTEM/DRIVER/OPERATOR.

L88: Dato del contrato: actorSubject: sub del usuario que produjo el evento.

L89: Dato del contrato: reason: motivo legible.

L90: Dato del contrato: details: evidencia adicional.

L91: Dato del contrato: operationKey: clave durable de acción.

L92: Dato del contrato: occurredAt: instante del evento.

L94 a L95

```
94 |   public record PageView<T>(
95 |       List<T> content, int page, int size, long totalElements, int totalPages) {}
```

Declara record inmutable PageView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. DTO de vehículos, solicitudes, historial y paginación. Los cuerpos se validan antes de ingresar en las reglas del servicio. Tipos y datos: page = page; int es entero de 32 bits para límites y contadores; size = size; int es entero de 32 bits para límites y contadores; totalElements = totalElements; long es entero de 64 bits para cantidades y duraciones mayores; totalPages = totalPages; int es entero de 32 bits para límites y contadores.

L97

```
97 |   public record Created(RequestView request, boolean created) {}
```

Declara record inmutable Created. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. DTO de vehículos, solicitudes, historial y paginación. Los cuerpos se validan antes de ingresar en las reglas del servicio. Tipos y datos: created = created; boolean es verdadero/falso, sin null.

L98

```
98 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 8, 11, 19, 27, 29, 42, 44, 56, 81, 93, 96: separadores sin instrucciones.

RequestService (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/RequestService.java`

Objetivo y funcionalidad

Gestiona propiedad, normalización, exclusividad por vehículo y cancelación previa a primera aceptación. Serializa operaciones con bloqueo del vehículo y conserva huellas de idempotencia.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import java.nio.charset.StandardCharsets;
```

Importa tipos: java.nio.charset.StandardCharsets. Codificación UTF-8 explícita al construir huellas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import java.security.MessageDigest;
```

Importa tipos: java.security.MessageDigest. MessageDigest para SHA-256; no almacena contraseñas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import org.springframework.data.domain.*;
```

Importa tipos: org.springframework.data.domain.*. Repositorios Spring Data y estructuras de paginación/orden. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | import org.springframework.http.HttpStatus;
```

Importa tipos: org.springframework.http.HttpStatus. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12

```
12 | import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L13

```
13 | import org.springframework.security.oauth2.jwt.Jwt;
```

Importa tipos: org.springframework.security.oauth2.jwt.Jwt. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L14

```
14 | import com.auxiruta.commons.security.TokenSupport;
```

Importa tipos: com.auxiruta.commons.security.TokenSupport. Utilidades compartidas de seguridad, errores, reloj o geografía. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L15

```
15 | import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L16

```
16 | import static com.auxiruta.assistance.requests.RequestDtos.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.RequestDtos.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L18

```
18 | @Service
```

Registra lógica de aplicación como bean. El escaneo de Spring crea este componente e inyecta sus dependencias; no hay que construirlo manualmente en el controlador.

L19

```
19 | @Transactional(readOnly = true)
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L20

```
20 | public class RequestService {
```

Declara clase RequestService. Gestiona propiedad, normalización, exclusividad por vehículo y cancelación previa a primera aceptación. Serializa operaciones con bloqueo del vehículo y conserva huellas de idempotencia.

L21

```
21 |   private final VehicleRepository vehicles;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L22

```
22 |   private final RequestRepository requests;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L23

```
23 |   private final HistoryRepository histories;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L24

```
24 |   private final RulesConfiguration rules;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L25

```
25 |   private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L26

```
26 |   private final EvaluationReadService evaluations;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L28

```
28 |   @org.springframework.beans.factory.annotation.Autowired
```

Inyecta dependencia Spring.

L29

```
29 |   private org.springframework.jdbc.core.JdbcTemplate jdbc;
```

Tipos y datos: jdbc = ejecutor SQL parametrizado; JdbcTemplate es ejecutor de SQL parametrizado de Spring dentro de transacciones locales.

L31 a L37

```
31 |   public RequestService(
32 |       VehicleRepository vehicles,
33 |       RequestRepository requests,
34 |       HistoryRepository histories,
35 |       RulesConfiguration rules,
36 |       Clock clock,
37 |       EvaluationReadService evaluations) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L38

```
38 |     this.vehicles = vehicles;
```

En this guarda repositorio de vehículos. Toma el dato de vehicles después de las validaciones de este método. repositorio de vehículos.

L39

```
39 |     this.requests = requests;
```

En this guarda repositorio de solicitudes. Toma el dato de requests después de las validaciones de este método. repositorio de solicitudes.

L40

```
40 |     this.histories = histories;
```

En this guarda histories. Toma el dato de histories después de las validaciones de este método. histories.

L41

```
41 |     this.rules = rules;
```

En this guarda configuración o snapshot de reglas que rige esta operación. Toma el dato de rules después de las validaciones de este método. configuración o snapshot de reglas que rige esta operación.

L42

```
42 |     this.clock = clock;
```

En this guarda reloj usado para evaluar plazos. Toma el dato de clock después de las validaciones de este método. reloj usado para evaluar plazos.

L43

```
43 |     this.evaluations = evaluations;
```

En this guarda acceso a evaluaciones persistidas. Toma el dato de evaluations después de las validaciones de este método. acceso a evaluaciones persistidas.

L44

```
44 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L46

```
46 |   private Instant now() {
```

Obtiene el instante del reloj inyectado; truncar a microsegundos mantiene consistencia con PostgreSQL.

L47

```
47 |     return clock.instant().truncatedTo(java.time.temporal.ChronoUnit.MICROS);
```

Resultado que recibe el llamador: lee el instante del reloj inyectado.

L48

```
48 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L50

```
50 |   private static ResponseStatusException missing() {
```

Construye 404; puede representar recurso inexistente o ajeno para no revelar su existencia.

L51

```
51 |     return new ResponseStatusException(HttpStatus.NOT_FOUND);
```

Resultado que recibe el llamador: interrumpe la operación con un estado HTTP explícito; 404 oculta inexistencia o pertenencia ajena.

L52

```
52 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L54

```
54 |   static void require(boolean ok, HttpStatus status) {
```

Expresa una precondición; si resulta falsa se aborta con el estado HTTP indicado. Tipos y datos: ok = ok; boolean es verdadero/falso, sin null.

L55

```
55 |     if (!ok) throw new ResponseStatusException(status);
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. interrumpe la operación con un estado HTTP explícito; ! invierte el booleano de la expresión que sigue.

L56

```
56 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L58

```
58 |   private static void key(String value) {
```

Tipos y datos: value = value; String es texto; no es por sí mismo una referencia de entidad.

L59

```
59 |     require(value != null && !value.isBlank() && value.length() <= 100, HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. comprueba texto vacío o solo espacios; 400 indica entrada inválida; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue; comprueba presencia antes de usar el dato

L60

```
60 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L62

```
62 |   static String plate(String value) {
```

Quita espacios/guiones, convierte a mayúsculas y exige entre 6 y 10 caracteres alfanuméricos. Tipos y datos: value = value; String es texto; no es por sí mismo una referencia de entidad.

L63

```
63 |     String p = value.replace(" ", "").replace("-", "").toUpperCase(Locale.ROOT);
```

Se calcula p (p) para reutilizar ese resultado en los pasos siguientes.

L64

```
64 |     require(p.matches("[A-Z0-9]{6,10}"), HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. 400 indica entrada inválida

L65

```
65 |     return p;
```

Entrega p al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: p.

L66

```
66 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L68

```
68 |   private static Pageable page(int p, int size, String sort) {
```

Valida número/tamaño y aplica orden estable para paginar. Tipos y datos: p = p; int es entero de 32 bits para límites y contadores; size = size; int es entero de 32 bits para límites y contadores; sort = sort; String es texto; no es por sí mismo una referencia de entidad.

L69

```
69 |     require(p >= 0 && size >= 1 && size <= 100, HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. 400 indica entrada inválida; && exige todas las condiciones y deja de evaluar al encontrar una falsa

L70

```
70 |     return PageRequest.of(p, size, Sort.by(sort).and(Sort.by("id")));
```

Construye página ordenada por fecha y UUID, estable incluso cuando coinciden fechas.

L71

```
71 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L73

```
73 |   private static <T> PageView<T> paged(Page<T> p) {
```

Método genérico que proyecta Page de Spring a PageView del contrato público. T conserva el tipo de cada elemento y expone metadatos sin depender de la serialización interna de Spring.

L74 a L75

```
74 |     return new PageView<>(
75 |         p.getContent(), p.getNumber(), p.getSize(), p.getTotalElements(), p.getTotalPages());
```

Proyecta contenido, página, tamaño, total de elementos y páginas al contrato REST.

L76

```
76 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L78

```
78 |   private Vehicle owned(UUID id, String subject) {
```

Busca por ID y propietario conjuntamente, ocultando recursos ajenos con 404. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; subject = subject; String es texto; no es por sí mismo una referencia de entidad.

L79

```
79 |     return vehicles.findByIdAndOwnerSubject(id, subject).orElseThrow(RequestService::missing);
```

Si no encuentra resultado lanza la excepción indicada. Filtra por propietario en la consulta misma. Resultado que recibe el llamador: si no existe valor, lanza la excepción en vez de seguir con null.

L80

```
80 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L82

```
82 |   private Vehicle locked(UUID id, String subject) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; subject = subject; String es texto; no es por sí mismo una referencia de entidad.

L83

```
83 |     return vehicles.lockOwned(id, subject).orElseThrow(RequestService::missing);
```

Si no encuentra resultado lanza la excepción indicada. Resultado que recibe el llamador: si no existe valor, lanza la excepción en vez de seguir con null.

L84

```
84 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L86

```
86 |   private void free(UUID id) {
```

Impide editar/reutilizar un vehículo con solicitud activa o liberación pendiente. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L87

```
87 |     require(!requests.hasActive(id), HttpStatus.CONFLICT);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. 409 comunica que el estado/regla actual impide la acción; ! invierte el booleano de la expresión que sigue

L88

```
88 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L90

```
90 |   private boolean admin(Jwt jwt) {
```

Tipos y datos: jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias.

L91

```
91 |     return TokenSupport.permits(jwt, false, "ADMIN");
```

Comprueba ADMIN humano con audiencia/cliente válidos, no solo el texto del rol.

L92

```
92 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L94

```
94 |   private boolean driver(Jwt jwt) {
```

Tipos y datos: jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias.

L95

```
95 |     return TokenSupport.permits(jwt, false, "DRIVER");
```

Comprueba DRIVER humano; ADMIN no se trata automáticamente como conductor.

L96

```
96 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L98

```
98 |   private AssistanceRequest visible(UUID id, Jwt jwt) {
```

Aplica alcance por identidad/rol antes de devolver un recurso. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias.

L99

```
99 |     AssistanceRequest req = requests.findById(id).orElseThrow(RequestService::missing);
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Se calcula req (req) para reutilizar ese resultado en los pasos siguientes. si no existe valor, lanza la excepción en vez de seguir con null; busca por clave primaria; el resultado puede faltar

L100 a L110

```
100 |     if (!admin(jwt))
101 |       require(
102 |           (driver(jwt) && req.driverSubject.equals(jwt.getSubject()))
103 |               || (TokenSupport.permits(jwt, false, "OPERATOR")
104 |                   && jdbc.queryForObject(
105 |                           "select count(*) from assignments where request_id=? and
      operator_subject=?",
106 |                           Long.class,
107 |                           id,
108 |                           jwt.getSubject())
109 |                       > 0),
110 |           HttpStatus.NOT_FOUND);
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Consulta assignments contando coincidencias. Cada require exige su condición; el helper aborta con el HTTP definido si falla. Esta condición decide si se entra en la rama siguiente. obtiene sub, identidad estable del usuario autenticado; compara contenido/identidad lógica, no referencias Java; 404 oculta inexistencia o pertenencia ajena; ejecuta SQL parametrizado y espera un resultado escalar o una fila; Datos implicados: sub del conductor que creó la solicitud; && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. obtiene sub, identidad estable del usuario autenticado; compara contenido/identidad lógica, no referencias Java; 404 oculta inexistencia o pertenencia ajena; ejecuta SQL parametrizado y espera un resultado escalar o una fila; Datos implicados: sub del conductor que creó la solicitud; && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue

L103: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L104: ejecuta SQL parametrizado y espera un resultado escalar o una fila; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L111

```
111 |     return req;
```

Entrega req al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: req.

L112

```
112 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L114

```
114 |   private VehicleView view(Vehicle v) {
```

Transforma entidad/datos internos en DTO; no cambia la base por realizar esta proyección.

L115 a L125

```
115 |     return new VehicleView(
116 |         v.id,
117 |         v.licensePlate,
118 |         v.brand,
119 |         v.model,
120 |         v.vehicleType,
121 |         v.declaredWeightKg,
122 |         v.registrationStatus,
123 |         v.createdAt,
124 |         v.updatedAt,
125 |         v.version);
```

Resultado que recibe el llamador: Datos implicados: identificador UUID del registro, patente normalizada y única, marca declarada, modelo declarado, tipo CAR/SUV/PICKUP, peso declarado en kg, alta administrativa ACTIVE/INACTIVE, fecha de creación, fecha de modificación, versión para control optimista de concurrencia.

L126

```
126 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L128

```
128 |   private RequestView view(AssistanceRequest r) {
```

Transforma entidad/datos internos en DTO; no cambia la base por realizar esta proyección.

L129 a L152

```
129 |     return new RequestView(
130 |         r.id,
131 |         r.vehicle.getId(),
132 |         r.status,
133 |         r.firstAssignedAt,
134 |         r.closureReasonCode,
135 |         r.originLatitude,
136 |         r.originLongitude,
137 |         r.destinationLatitude,
138 |         r.destinationLongitude,
139 |         r.declaredVehicleType,
140 |         r.declaredWeightKg,
141 |         r.mobilityCondition,
142 |         r.description,
143 |         r.searchAttemptNumber,
144 |         r.searchDeadline,
145 |         r.configurationRevision,
146 |         r.configurationSnapshot,
147 |         r.createdAt,
148 |         r.updatedAt,
149 |         r.closedAt,
150 |         r.version,
151 |         r.resourceReleasePending,
152 |         evaluations.latest(r.id));
```

Resultado que recibe el llamador: Datos implicados: identificador UUID del registro, estado del objeto; su enum o CHECK define valores permitidos, primera aceptación que impide cancelar incluso durante reemplazos, causa codificada del cierre, latitud de recogida, longitud de recogida, latitud de entrega, longitud de entrega, copia del tipo de vehículo al solicitar, peso declarado en kg, condición ROLLABLE/NON_ROLLABLE, descripción de asistencia, número de búsqueda de 1 a 3, vencimiento del intento, versión de reglas, JSON de parámetros vigente al crear, fecha de creación, fecha de modificación, fecha de cierre; null indica abierto, versión para control optimista de concurrencia, liberación remota pendiente que impide reutilización.

L153

```
153 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L155

```
155 |   private HistoryView view(RequestStatusHistory h) {
```

Transforma entidad/datos internos en DTO; no cambia la base por realizar esta proyección.

L156 a L166

```
156 |     return new HistoryView(
157 |         h.id,
158 |         h.request.getId(),
159 |         h.previousStatus,
160 |         h.newStatus,
161 |         h.actorType,
162 |         h.actorSubject,
163 |         h.reason,
164 |         h.details,
165 |         h.operationKey,
166 |         h.occurredAt);
```

Resultado que recibe el llamador: Datos implicados: identificador UUID del registro, estado previo, estado resultante, tipo de actor SYSTEM/DRIVER/OPERATOR, sub del usuario que produjo el evento, motivo legible, evidencia adicional, clave durable de acción, instante del evento.

L167

```
167 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L169

```
169 |   public VehicleView vehicle(UUID id, String subject) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; subject = subject; String es texto; no es por sí mismo una referencia de entidad.

L170

```
170 |     return view(owned(id, subject));
```

Busca por identificador y propietario a la vez, devuelve 404 si no pertenece al sujeto y convierte la entidad propia a DTO.

L171

```
171 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L173

```
173 |   public PageView<VehicleView> vehicles(String subject, int p, int size, Registration status) {
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad; p = p; int es entero de 32 bits para límites y contadores; size = size; int es entero de 32 bits para límites y contadores.

L174

```
174 |     Pageable pageable = page(p, size, "createdAt");
```

Valida paginación y ordena solicitudes por creación con UUID de desempate. Se calcula pageable (pageable) para reutilizar ese resultado en los pasos siguientes.

L175 a L179

```
175 |     return paged(
176 |         (status == null
177 |                 ? vehicles.findByOwnerSubject(subject, pageable)
178 |                 : vehicles.findByOwnerSubjectAndRegistrationStatus(subject, status, pageable))
179 |             .map(this::view));
```

Transforma cada elemento al valor/tipo de salida. Filtra por propietario en la consulta misma. Resultado que recibe el llamador: transforma cada elemento al tipo/valor de salida; detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L179: transforma cada elemento al tipo/valor de salida.

L180

```
180 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L182

```
182 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L183

```
183 |   public VehicleView createVehicle(String subject, CreateVehicle d) {
```

Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad.

L184

```
184 |     Vehicle v = new Vehicle();
```

Construye Vehicle en memoria y conserva la referencia en v. Entidad de vehículo del conductor. ownerSubject enlaza con sub de Keycloak sin FK entre bases; el repositorio filtra por propietario. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L185

```
185 |     v.ownerSubject = subject;
```

En v guarda sub de Keycloak del propietario. Identifica a la persona por sub de Keycloak, no por un nombre editable ni una contraseña. subject.

L186

```
186 |     v.licensePlate = plate(d.licensePlate());
```

En v guarda patente normalizada y única. Toma el dato de plate(d.licensePlate()) después de las validaciones de este método.

L187

```
187 |     v.brand = d.brand();
```

En v guarda marca declarada. Toma el dato de d.brand() después de las validaciones de este método.

L188

```
188 |     v.model = d.model();
```

En v guarda modelo declarado. Toma el dato de d.model() después de las validaciones de este método.

L189

```
189 |     v.vehicleType = d.vehicleType();
```

En v guarda tipo CAR/SUV/PICKUP. Toma el dato de d.vehicleType() después de las validaciones de este método.

L190

```
190 |     v.declaredWeightKg = d.declaredWeightKg();
```

En v guarda peso declarado en kg. Toma el dato de d.declaredWeightKg() después de las validaciones de este método.

L191

```
191 |     v.createdAt = v.updatedAt = now();
```

En v guarda fecha de creación. Marca el instante de esta operación para ordenar y auditar su evolución. Datos implicados: fecha de modificación.

L192

```
192 |     vehicles.saveAndFlush(v);
```

Persiste y fuerza sincronización SQL para detectar restricciones antes de retornar; commit ocurre al terminar transacción. Guarda y sincroniza SQL para detectar conflictos antes de retornar; no confirma por sí solo la transacción.

L193

```
193 |     return view(v);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L194

```
194 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L196

```
196 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L197

```
197 |   public VehicleView editVehicle(UUID id, String subject, EditVehicle d) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; subject = subject; String es texto; no es por sí mismo una referencia de entidad.

L198 a L204

```
198 |     require(
199 |         d.licensePlate() != null
200 |             || d.brand() != null
201 |             || d.model() != null
202 |             || d.vehicleType() != null
203 |             || d.declaredWeightKg() != null,
204 |         HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. 400 indica entrada inválida; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; comprueba presencia antes de usar el dato

L200: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; comprueba presencia antes de usar el dato.

L201: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; comprueba presencia antes de usar el dato.

L202: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; comprueba presencia antes de usar el dato.

L203: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; comprueba presencia antes de usar el dato.

L205

```
205 |     Vehicle v = locked(id, subject);
```

Se calcula v (v) para reutilizar ese resultado en los pasos siguientes.

L206

```
206 |     free(id);
```

Impide editar/reutilizar un vehículo con solicitud activa o liberación pendiente.

L207

```
207 |     if (d.licensePlate() != null) v.licensePlate = plate(d.licensePlate());
```

Esta condición decide si se entra en la rama siguiente. Datos implicados: patente normalizada y única; comprueba presencia antes de usar el dato.

L208

```
208 |     if (d.brand() != null) v.brand = d.brand();
```

Esta condición decide si se entra en la rama siguiente. Datos implicados: marca declarada; comprueba presencia antes de usar el dato.

L209

```
209 |     if (d.model() != null) v.model = d.model();
```

Esta condición decide si se entra en la rama siguiente. Datos implicados: modelo declarado; comprueba presencia antes de usar el dato.

L210

```
210 |     if (d.vehicleType() != null) v.vehicleType = d.vehicleType();
```

Esta condición decide si se entra en la rama siguiente. Datos implicados: tipo CAR/SUV/PICKUP; comprueba presencia antes de usar el dato.

L211

```
211 |     if (d.declaredWeightKg() != null) v.declaredWeightKg = d.declaredWeightKg();
```

Esta condición decide si se entra en la rama siguiente. Datos implicados: peso declarado en kg; comprueba presencia antes de usar el dato.

L212

```
212 |     v.updatedAt = now();
```

En v guarda fecha de modificación. Marca el instante de esta operación para ordenar y auditar su evolución.

L213

```
213 |     vehicles.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente.

L214

```
214 |     return view(v);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L215

```
215 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L217

```
217 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L218

```
218 |   public VehicleView registration(UUID id, String subject, RegistrationChange d) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; subject = subject; String es texto; no es por sí mismo una referencia de entidad.

L219

```
219 |     Vehicle v = locked(id, subject);
```

Se calcula v (v) para reutilizar ese resultado en los pasos siguientes.

L220

```
220 |     free(id);
```

Impide editar/reutilizar un vehículo con solicitud activa o liberación pendiente.

L221

```
221 |     v.registrationStatus = d.status();
```

En v guarda alta administrativa ACTIVE/INACTIVE. La transición efectiva proviene de d.status() y debe estar autorizada por las validaciones previas.

L222

```
222 |     v.updatedAt = now();
```

En v guarda fecha de modificación. Marca el instante de esta operación para ordenar y auditar su evolución.

L223

```
223 |     vehicles.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente.

L224

```
224 |     return view(v);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L225

```
225 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L227

```
227 |   static String digest(String text) {
```

Genera SHA-256 hexadecimal usando UTF-8; es una huella de contenido, no cifrado de contraseñas. Tipos y datos: text = text; String es texto; no es por sí mismo una referencia de entidad.

L228

```
228 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L229 a L231

```
229 |       return HexFormat.of()
230 |           .formatHex(
231 |
      MessageDigest.getInstance("SHA-256").digest(text.getBytes(StandardCharsets.UTF_8)));
```

Resultado que recibe el llamador: codifica texto a bytes, necesario para calcular la huella estable; calcula una huella de contenido usada para idempotencia.

L232

```
232 |     } catch (java.security.NoSuchAlgorithmException e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L233

```
233 |       throw new IllegalStateException(e);
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L234

```
234 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L235

```
235 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L237

```
237 |   private static String coordinate(BigDecimal n) {
```

Normaliza decimales para que valores equivalentes generen la misma clave o huella. Tipos y datos: n = n; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L238

```
238 |     return n.stripTrailingZeros().toPlainString();
```

Normaliza el decimal a texto sin ceros finales superfluos ni notación exponencial; números equivalentes producen la misma representación para caché/huellas.

L239

```
239 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L241

```
241 |   static String fingerprint(CreateRequest d) {
```

Calcula una huella del contenido para distinguir repetición idéntica de reutilización conflictiva de una clave.

L242

```
242 |     // Length-prefixed fields prevent ambiguities when free text contains delimiters.
```

Documentación para quien lee el fuente: Length-prefixed fields prevent ambiguities when free text contains delimiters. No es una instrucción ejecutable y no impone la regla por sí sola.

L243 a L251

```
243 |     List<String> fields =
244 |         List.of(
245 |             d.vehicleId().toString(),
246 |             coordinate(d.originLatitude()),
247 |             coordinate(d.originLongitude()),
248 |             coordinate(d.destinationLatitude()),
249 |             coordinate(d.destinationLongitude()),
250 |             d.mobilityCondition().name(),
251 |             d.description().strip());
```

Define los campos de contenido que identifican semánticamente la solicitud para idempotencia. Se calcula fields (fields) para reutilizar ese resultado en los pasos siguientes. representa el valor como texto; en un nodo JSON produce JSON serializado; crea una lista no modificable de los elementos indicados

L252 a L255

```
252 |     return digest(
253 |         fields.stream()
254 |             .map(v -> v.length() + ":" + v)
255 |             .collect(java.util.stream.Collectors.joining()));
```

Prefija longitud de cada campo antes de hashear; evita ambigüedad por delimitadores en texto libre. Resultado que recibe el llamador: inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente; transforma cada elemento al tipo/valor de salida.

L254: transforma cada elemento al tipo/valor de salida.

L256

```
256 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L258

```
258 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L259

```
259 |   public Created createRequest(String subject, String operationKey, CreateRequest d) {
```

Campos: operationKey: clave durable de acción. Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad; operationKey = clave durable de acción; String es texto; no es por sí mismo una referencia de entidad.

L260

```
260 |     key(operationKey);
```

Exige clave no nula, no vacía y de hasta cien caracteres antes de operar.

L261 a L264

```
261 |     require(
262 |         d.originLatitude().compareTo(d.destinationLatitude()) != 0
263 |             || d.originLongitude().compareTo(d.destinationLongitude()) != 0,
264 |         HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara decimales: negativo menor, cero igual, positivo mayor; 400 indica entrada inválida; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera

L263: compara decimales: negativo menor, cero igual, positivo mayor; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L265

```
265 |     String fingerprint = fingerprint(d);
```

Calcula huella normalizada del cuerpo antes de buscar una operación previa. Se calcula fingerprint (fingerprint) para reutilizar ese resultado en los pasos siguientes.

L266

```
266 |     // Locking the vehicle also serializes competing creation, edits and cancellation.
```

Documentación para quien lee el fuente: Locking the vehicle also serializes competing creation, edits and cancellation. No es una instrucción ejecutable y no impone la regla por sí sola.

L267

```
267 |     Vehicle v = locked(d.vehicleId(), subject);
```

Bloquea únicamente el vehículo del conductor autenticado; serializa creación concurrente, edición y cancelación. Se calcula v (v) para reutilizar ese resultado en los pasos siguientes.

L268

```
268 |     var previous = requests.findByDriverSubjectAndIdempotencyKey(subject, operationKey);
```

Busca resultado anterior para repetir sin duplicar. Se calcula previous (previous) para reutilizar ese resultado en los pasos siguientes.

L269

```
269 |     if (previous.isPresent()) {
```

Esta condición decide si se entra en la rama siguiente. .

L270

```
270 |       require(previous.get().requestFingerprint.equals(fingerprint), HttpStatus.CONFLICT);
```

Una clave del conductor con mismo contenido devuelve el recurso anterior; contenido distinto es conflicto 409. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara contenido/identidad lógica, no referencias Java; 409 comunica que el estado/regla actual impide la acción

L271

```
271 |       return new Created(view(previous.get()), false);
```

Devuelve el recurso ya creado y marca false: fue una repetición idempotente. El controlador no debe reevaluar ni generar otra propuesta por ese reintento.

L272

```
272 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L273

```
273 |     require(v.registrationStatus == Registration.ACTIVE, HttpStatus.CONFLICT);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. 409 comunica que el estado/regla actual impide la acción; ACTIVE: registro habilitado administrativamente; Datos implicados: alta administrativa ACTIVE/INACTIVE

L274

```
274 |     free(v.id);
```

Exige vehículo ACTIVE y sin solicitud abierta/liberación pendiente. No impide solicitudes de otros vehículos del mismo conductor. Datos implicados: identificador UUID del registro.

L275

```
275 |     AssistanceRequest req = new AssistanceRequest();
```

Construye AssistanceRequest en memoria y conserva la referencia en req. Entidad central con estado, ubicación, datos declarados del vehículo, primera aceptación y snapshot de reglas. Los campos de automatización permiten reanudar después de reinicios. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L276

```
276 |     req.vehicle = v;
```

En req guarda vehicle. Toma el dato de v después de las validaciones de este método. v.

L277

```
277 |     req.driverSubject = subject;
```

En req guarda sub del conductor que creó la solicitud. Toma el dato de subject después de las validaciones de este método. subject.

L278

```
278 |     req.originLatitude = d.originLatitude();
```

En req guarda latitud de recogida. Toma el dato de d.originLatitude() después de las validaciones de este método.

L279

```
279 |     req.originLongitude = d.originLongitude();
```

En req guarda longitud de recogida. Toma el dato de d.originLongitude() después de las validaciones de este método.

L280

```
280 |     req.destinationLatitude = d.destinationLatitude();
```

En req guarda latitud de entrega. Toma el dato de d.destinationLatitude() después de las validaciones de este método.

L281

```
281 |     req.destinationLongitude = d.destinationLongitude();
```

En req guarda longitud de entrega. Toma el dato de d.destinationLongitude() después de las validaciones de este método.

L282

```
282 |     req.declaredVehicleType = v.vehicleType;
```

En req guarda copia del tipo de vehículo al solicitar. Toma el dato de v.vehicleType después de las validaciones de este método. Datos implicados: tipo CAR/SUV/PICKUP.

L283

```
283 |     req.declaredWeightKg = v.declaredWeightKg;
```

En req guarda peso declarado en kg. Toma el dato de v.declaredWeightKg después de las validaciones de este método. Datos implicados: peso declarado en kg.

L284

```
284 |     req.mobilityCondition = d.mobilityCondition();
```

En req guarda condición ROLLABLE/NON_ROLLABLE. Toma el dato de d.mobilityCondition() después de las validaciones de este método.

L285

```
285 |     req.description = d.description().strip();
```

En req guarda descripción de asistencia. Toma el dato de d.description().strip() después de las validaciones de este método.

L286

```
286 |     req.configurationRevision = rules.revision();
```

En req guarda versión de reglas. Toma el dato de rules.revision() después de las validaciones de este método.

L287

```
287 |     req.configurationSnapshot = rules.snapshot();
```

En req guarda JSON de parámetros vigente al crear. Conserva evidencia estable de la operación para comparar reintentos o reconstruir las reglas utilizadas, aunque cambie la configuración futura.

L288

```
288 |     req.idempotencyKey = operationKey;
```

En req guarda clave para repetir sin duplicar. Toma el dato de operationKey después de las validaciones de este método. clave durable de acción.

L289

```
289 |     req.requestFingerprint = fingerprint;
```

En req guarda SHA-256 del contenido asociado a la clave. Conserva evidencia estable de la operación para comparar reintentos o reconstruir las reglas utilizadas, aunque cambie la configuración futura. fingerprint.

L290

```
290 |     req.createdAt = req.updatedAt = now();
```

En req guarda fecha de creación. Marca el instante de esta operación para ordenar y auditar su evolución. Datos implicados: fecha de modificación.

L291

```
291 |     req.searchDeadline = req.createdAt.plusSeconds(rules.searchSeconds());
```

En req guarda vencimiento del intento. Toma el dato de req.createdAt.plusSeconds(rules.searchSeconds()) después de las validaciones de este método. calcula un instante futuro sumando segundos, sin modificar el instante original; Datos implicados: fecha de creación.

L292

```
292 |     req = requests.saveAndFlush(req);
```

Persiste y fuerza sincronización SQL para detectar restricciones antes de retornar; commit ocurre al terminar transacción. Guarda y sincroniza SQL para detectar conflictos antes de retornar; no confirma por sí solo la transacción.

L293

```
293 |     event(req, null, Status.REQUESTED, subject, "CREATED", null, null);
```

Registra evento inicial CREATED: no hay estado previo y el nuevo es REQUESTED. REQUESTED: alternativa del ciclo de vida definido en el enum.

L294

```
294 |     return new Created(view(req), true);
```

Devuelve DTO y bandera de creación nueva; el controlador elige 201 en vez de 200 de una repetición.

L295

```
295 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L297 a L304

```
297 |   private RequestStatusHistory event(
298 |       AssistanceRequest req,
299 |       Status previous,
300 |       Status next,
301 |       String subject,
302 |       String reason,
303 |       String operationKey,
304 |       String fingerprint) {
```

Persiste el cambio de estado con actor/motivo y difiere el log hasta confirmar la transacción. Campos: reason: motivo legible; operationKey: clave durable de acción. Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad; reason = motivo legible; String es texto; no es por sí mismo una referencia de entidad; operationKey = clave durable de acción; String es texto; no es por sí mismo una referencia de entidad; fingerprint = fingerprint; String es texto; no es por sí mismo una referencia de entidad.

L302: Dato del contrato: reason: motivo legible.

L303: Dato del contrato: operationKey: clave durable de acción.

L305

```
305 |     RequestStatusHistory h = new RequestStatusHistory();
```

Construye RequestStatusHistory en memoria y conserva la referencia en h. Auditoría de cambios de solicitud con actor, motivo, fecha, idempotencia y evidencia. Es persistencia de negocio, distinta de los logs de consola. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L306

```
306 |     h.request = req;
```

En h guarda request. Toma el dato de req después de las validaciones de este método. req.

L307

```
307 |     h.previousStatus = previous;
```

En h guarda estado previo. La transición efectiva proviene de previous y debe estar autorizada por las validaciones previas. previous.

L308

```
308 |     h.newStatus = next;
```

En h guarda estado resultante. La transición efectiva proviene de next y debe estar autorizada por las validaciones previas. next.

L309

```
309 |     h.actorSubject = subject;
```

En h guarda sub del usuario que produjo el evento. Toma el dato de subject después de las validaciones de este método. subject.

L310

```
310 |     h.actorType = "DRIVER";
```

En h guarda tipo de actor SYSTEM/DRIVER/OPERATOR. Toma el dato de "DRIVER" después de las validaciones de este método.

L311

```
311 |     h.reason = reason;
```

En h guarda motivo legible. Toma el dato de reason después de las validaciones de este método. motivo legible.

L312

```
312 |     h.operationKey = operationKey;
```

En h guarda clave durable de acción. Toma el dato de operationKey después de las validaciones de este método. clave durable de acción.

L313

```
313 |     h.requestFingerprint = fingerprint;
```

En h guarda SHA-256 del contenido asociado a la clave. Conserva evidencia estable de la operación para comparar reintentos o reconstruir las reglas utilizadas, aunque cambie la configuración futura. fingerprint.

L314

```
314 |     h.occurredAt = now();
```

En h guarda instante del evento. Toma el dato de now() después de las validaciones de este método.

L315

```
315 |     h.details = next == Status.REQUESTED ? "searchDeadline=" + req.searchDeadline : null;
```

En h guarda evidencia adicional. Toma el dato de next == Status.REQUESTED ? "searchDeadline=" + req.searchDeadline : null después de las validaciones de este método. REQUESTED: alternativa del ciclo de vida definido en el enum; Datos implicados: vencimiento del intento; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L316

```
316 |     histories.saveAndFlush(h);
```

Persiste y fuerza sincronización SQL para detectar restricciones antes de retornar; commit ocurre al terminar transacción. Guarda y sincroniza SQL para detectar conflictos antes de retornar; no confirma por sí solo la transacción.

L317 a L326

```
317 |     org.springframework.transaction.support.TransactionSynchronizationManager
318 |         .registerSynchronization(
319 |             new org.springframework.transaction.support.TransactionSynchronization() {
320 |               @Override
321 |               public void afterCommit() {
322 |                 org.slf4j.LoggerFactory.getLogger(RequestService.class)
323 |                     .info(
324 |                         "Request state changed requestId={} from={} to={}", req.id, previous,
      next);
325 |               }
326 |             });
```

Registra un callback del ciclo transaccional; no ejecuta el log antes del commit. Implementa callback afterCommit para anunciar solo cambios confirmados. Se calcula requestId (UUID de solicitud) para reutilizar ese resultado en los pasos siguientes. Datos implicados: identificador UUID del registro

L327

```
327 |     return h;
```

Entrega h al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: h.

L328

```
328 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L330

```
330 |   public RequestView request(UUID id, Jwt jwt) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias.

L331

```
331 |     return view(visible(id, jwt));
```

Verifica visibilidad según identidad/rol y después construye el DTO. Saber el UUID no evita este control de alcance.

L332

```
332 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L334

```
334 |   public PageView<RequestView> requests(Jwt jwt, int p, int size, Status status) {
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Tipos y datos: jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias; p = p; int es entero de 32 bits para límites y contadores; size = size; int es entero de 32 bits para límites y contadores.

L335

```
335 |     Pageable pageable = page(p, size, "createdAt");
```

Valida paginación y ordena solicitudes por creación con UUID de desempate. Se calcula pageable (pageable) para reutilizar ese resultado en los pasos siguientes.

L336 a L343

```
336 |     if (!admin(jwt) && !driver(jwt))
337 |       return paged(
338 |           requests
339 |               .forOperator(
340 |                   jwt.getSubject(),
341 |                   status == null ? null : status.name(),
342 |                   PageRequest.of(p, size, Sort.by("created_at").and(Sort.by("id"))))
343 |               .map(this::view));
```

Transforma cada elemento al valor/tipo de salida. Identidad obtenida del JWT, no del JSON enviado por el usuario. Esta condición decide si se entra en la rama siguiente. obtiene sub, identidad estable del usuario autenticado; transforma cada elemento al tipo/valor de salida; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue; detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L343: transforma cada elemento al tipo/valor de salida.

L344 a L345

```
344 |     return paged(
345 |         requests.visible(admin(jwt) ? null : jwt.getSubject(), status,
      pageable).map(this::view));
```

Transforma cada elemento al valor/tipo de salida. Identidad obtenida del JWT, no del JSON enviado por el usuario. Resultado que recibe el llamador: obtiene sub, identidad estable del usuario autenticado; transforma cada elemento al tipo/valor de salida; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L346

```
346 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L348

```
348 |   public PageView<HistoryView> history(UUID id, Jwt jwt, int p, int size) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias; p = p; int es entero de 32 bits para límites y contadores; size = size; int es entero de 32 bits para límites y contadores.

L349

```
349 |     visible(id, jwt);
```

Aplica alcance por identidad/rol antes de devolver un recurso.

L350

```
350 |     return paged(histories.findByRequestId(id, page(p, size, "occurredAt")).map(this::view));
```

Transforma cada elemento al valor/tipo de salida. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Resultado que recibe el llamador: transforma cada elemento al tipo/valor de salida.

L351

```
351 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L353

```
353 |   static boolean cancellable(AssistanceRequest r) {
```

Solo permite cancelar antes de firstAssignedAt y en estados previos a aceptación.

L354 a L360

```
354 |     return r.firstAssignedAt == null
355 |         && EnumSet.of(
356 |                 Status.REQUESTED,
357 |                 Status.PENDING_EVALUATION,
358 |                 Status.AWAITING_ACCEPTANCE,
359 |                 Status.SEARCHING_ALTERNATIVE)
360 |             .contains(r.status);
```

Resultado que recibe el llamador: verifica pertenencia al conjunto o lista; crea un conjunto no modificable y sin duplicados; REQUESTED: alternativa del ciclo de vida definido en el enum; PENDING_EVALUATION: solicitud pendiente de evaluación; AWAITING_ACCEPTANCE: alternativa del ciclo de vida definido en el enum; SEARCHING_ALTERNATIVE: alternativa del ciclo de vida definido en el enum; Datos implicados: primera aceptación que impide cancelar incluso durante reemplazos, estado del objeto; su enum o CHECK define valores permitidos; && exige todas las condiciones y deja de evaluar al encontrar una falsa; detecta un dato ausente.

L355: crea un conjunto no modificable y sin duplicados; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L360: verifica pertenencia al conjunto o lista; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos.

L361

```
361 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L363

```
363 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L364

```
364 |   public HistoryView cancel(UUID id, String subject, String operationKey, Cancellation d) {
```

Campos: id: identificador UUID del registro; operationKey: clave durable de acción. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; subject = subject; String es texto; no es por sí mismo una referencia de entidad; operationKey = clave durable de acción; String es texto; no es por sí mismo una referencia de entidad.

L365

```
365 |     key(operationKey);
```

Exige clave no nula, no vacía y de hasta cien caracteres antes de operar.

L366

```
366 |     UUID vehicleId = requests.vehicleId(id).orElseThrow(RequestService::missing);
```

Si no encuentra resultado lanza la excepción indicada. Se calcula vehicleId (vehicleId) para reutilizar ese resultado en los pasos siguientes. si no existe valor, lanza la excepción en vez de seguir con null

L367

```
367 |     locked(vehicleId, subject);
```

Busca y bloquea el vehículo del propietario antes de continuar; serializa operaciones del mismo vehículo y no bloquea los demás vehículos del conductor.

L368

```
368 |     AssistanceRequest req = requests.lockById(id).orElseThrow(RequestService::missing);
```

Si no encuentra resultado lanza la excepción indicada. Se calcula req (req) para reutilizar ese resultado en los pasos siguientes. si no existe valor, lanza la excepción en vez de seguir con null

L369

```
369 |     require(req.driverSubject.equals(subject), HttpStatus.NOT_FOUND);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara contenido/identidad lógica, no referencias Java; 404 oculta inexistencia o pertenencia ajena; Datos implicados: sub del conductor que creó la solicitud

L370

```
370 |     String fingerprint = digest(d.reason().strip());
```

Se calcula fingerprint (fingerprint) para reutilizar ese resultado en los pasos siguientes.

L371

```
371 |     var previous = histories.findByRequestIdAndOperationKey(id, operationKey);
```

Busca resultado anterior para repetir sin duplicar. Se calcula previous (previous) para reutilizar ese resultado en los pasos siguientes.

L372

```
372 |     if (previous.isPresent()) {
```

Esta condición decide si se entra en la rama siguiente. .

L373 a L376

```
373 |       require(
374 |           previous.get().actorSubject.equals(subject)
375 |               && previous.get().requestFingerprint.equals(fingerprint),
376 |           HttpStatus.CONFLICT);
```

Una clave del conductor con mismo contenido devuelve el recurso anterior; contenido distinto es conflicto 409. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara contenido/identidad lógica, no referencias Java; 409 comunica que el estado/regla actual impide la acción; && exige todas las condiciones y deja de evaluar al encontrar una falsa

L375: compara contenido/identidad lógica, no referencias Java; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L377

```
377 |       return view(previous.get());
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L378

```
378 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L379

```
379 |     require(cancellable(req), HttpStatus.CONFLICT);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. 409 comunica que el estado/regla actual impide la acción

L380 a L385

```
380 |     if (jdbc.queryForObject(
381 |             "select count(*) from pending_actions p join assignments a on a.id=p.assignment_id
      where a.request_id=? and p.kind='CONFIRM' and p.status<>'COMPLETED'",
382 |             Long.class,
383 |             id)
384 |         > 0)
385 |       throw new ResponseStatusException(HttpStatus.CONFLICT, "ACCEPTANCE_CONFIRMATION_PENDING");
```

Protege la carrera aceptación/cancelación: una confirmación remota incierta puede haber ocurrido. Impide cancelar hasta reconciliar, con código distinguible en el error. Esta condición decide si se entra en la rama siguiente. interrumpe la operación con un estado HTTP explícito; 409 comunica que el estado/regla actual impide la acción; ejecuta SQL parametrizado y espera un resultado escalar o una fila.

L386 a L390

```
386 |     var assignmentIds =
387 |         jdbc.queryForList(
388 |             "select id from assignments where request_id=? and status in
      ('RESERVING','PROPOSED')",
389 |             UUID.class,
390 |             id);
```

Consulta assignments con el filtro y proyección escritos. Se calcula assignmentIds (assignmentIds) para reutilizar ese resultado en los pasos siguientes.

L391

```
391 |     for (UUID assignment : assignmentIds) {
```

El bucle repite el bloque por cada elemento; el nombre a la izquierda de : es la variable local y a la derecha está la colección. Tipos y datos: assignment = assignment; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L392 a L395

```
392 |       jdbc.update(
393 |           "update assignments set status='CANCELLED',closed_at=? where id=?",
394 |           java.sql.Timestamp.from(now()),
395 |           assignment);
```

Actualiza assignments para las filas del filtro. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L396

```
396 |       // Resolve an uncertain RESERVE first; its completion creates the compensating RELEASE.
```

Documentación para quien lee el fuente: Resolve an uncertain RESERVE first; its completion creates the compensating RELEASE. No es una instrucción ejecutable y no impone la regla por sí sola.

L397 a L401

```
397 |       long pending =
398 |           jdbc.queryForObject(
399 |               "select count(*) from pending_actions where assignment_id=? and kind='RESERVE' and
      status<>'COMPLETED'",
400 |               Long.class,
401 |               assignment);
```

Consulta pending_actions contando coincidencias. Se calcula pending (pending) para reutilizar ese resultado en los pasos siguientes. ejecuta SQL parametrizado y espera un resultado escalar o una fila

L402 a L408

```
402 |       if (pending == 0)
403 |         jdbc.update(
404 |             "insert into pending_actions(id,assignment_id,kind,status,created_at,updated_at)
      values (?,?,'RELEASE','PENDING',?,?) on conflict(assignment_id,kind) do nothing",
405 |             UUID.randomUUID(),
406 |             assignment,
407 |             java.sql.Timestamp.from(now()),
408 |             java.sql.Timestamp.from(now()));
```

Genera ID nuevo en la aplicación. Inserta en pending_actions. DO NOTHING evita duplicar la intención/fila. Esta condición decide si se entra en la rama siguiente. genera un nuevo identificador UUID; ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L405: genera un nuevo identificador UUID.

L409

```
409 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L410

```
410 |     req.resourceReleasePending = req.resourceReleasePending || !assignmentIds.isEmpty();
```

En req guarda liberación remota pendiente que impide reutilización. Es una bandera durable de trabajo pendiente: después de reiniciar el sistema debe terminar esta recuperación antes de comprometer recursos incompatibles. comprueba ausencia de elementos o valor; Datos implicados: liberación remota pendiente que impide reutilización; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L411

```
411 |     Status old = req.status;
```

Se calcula old (old) para reutilizar ese resultado en los pasos siguientes. Datos implicados: estado del objeto; su enum o CHECK define valores permitidos

L412

```
412 |     req.status = Status.CANCELLED;
```

En req guarda estado del objeto; su enum o CHECK define valores permitidos. solicitud cancelada por el conductor antes de la primera aceptación CANCELLED: solicitud cancelada.

L413

```
413 |     req.closureReasonCode = "DRIVER_CANCELLED";
```

En req guarda causa codificada del cierre. Toma el dato de "DRIVER_CANCELLED" después de las validaciones de este método.

L414

```
414 |     req.closedAt = req.updatedAt = now();
```

En req guarda fecha de cierre; null indica abierto. Toma el dato de req.updatedAt = now() después de las validaciones de este método. Datos implicados: fecha de modificación.

L415

```
415 |     requests.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente.

L416 a L417

```
416 |     return view(
417 |         event(req, old, Status.CANCELLED, subject, d.reason().strip(), operationKey,
      fingerprint));
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L418

```
418 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L419

```
419 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 17, 27, 30, 45, 49, 53, 57, 61, 67, 72, 77, 81, 85, 89, 93, 97, 113, 127, 154, 168, 172, 181, 195, 216, 226, 236, 240, 257, 296, 329, 333, 347, 352, 362: separadores sin instrucciones.

RulesConfiguration (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/RulesConfiguration.java`

Objetivo y funcionalidad

Lee rules/mvp.yml y construye el snapshot JSON. Si está activo demo sustituye cinco tiempos de espera; no acelera cuotas ni modifica solicitudes existentes.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.Map;
```

Importa tipos: java.util.Map. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.io.InputStream;
```

Importa tipos: java.io.InputStream. Tipo auxiliar usado por las expresiones del archivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.core.io.ClassPathResource;
```

Importa tipos: org.springframework.core.io.ClassPathResource. Tipo auxiliar usado por las expresiones del archivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import com.fasterxml.jackson.databind.ObjectMapper;
```

Importa tipos: com.fasterxml.jackson.databind.ObjectMapper. Lectura/escritura JSON, nodos y conversión a DTO. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.yaml.snakeyaml.Yaml;
```

Importa tipos: org.yaml.snakeyaml.Yaml. Tipo auxiliar usado por las expresiones del archivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | @Component
```

Registra componente inyectable.

L11

```
11 | public class RulesConfiguration {
```

Declara clase RulesConfiguration. Lee rules/mvp.yml y construye el snapshot JSON. Si está activo demo sustituye cinco tiempos de espera; no acelera cuotas ni modifica solicitudes existentes.

L12

```
12 |   private final String snapshot;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: snapshot = snapshot; String es texto; no es por sí mismo una referencia de entidad.

L13

```
13 |   private final Map<String, Object> rules;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L15

```
15 |   public RulesConfiguration(ObjectMapper mapper) throws Exception {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada. Tipos y datos: mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L16

```
16 |     this(mapper, new org.springframework.core.env.StandardEnvironment());
```

Reutiliza el otro constructor con un entorno estándar. Evita duplicar la carga de reglas y permite instanciar esta configuración fuera de la inyección habitual.

L17

```
17 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L19

```
19 |   @org.springframework.beans.factory.annotation.Autowired
```

Inyecta dependencia Spring.

L20 a L21

```
20 |   public RulesConfiguration(
21 |       ObjectMapper mapper, org.springframework.core.env.Environment environment) throws Exception
       {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada. Tipos y datos: mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L22

```
22 |     try (InputStream in = new ClassPathResource("rules/mvp.yml").getInputStream()) {
```

Abre YAML empaquetado en classpath; try-with-resources cierra el stream automáticamente. Se calcula in (in) para reutilizar ese resultado en los pasos siguientes.

L23

```
23 |       Map<String, Object> document = new Yaml().load(in);
```

Parsea YAML a mapa de datos; no ejecuta instrucciones del archivo. Se calcula document (document) para reutilizar ese resultado en los pasos siguientes.

L24 a L27

```
24 |       rules =
25 |           mapper.convertValue(
26 |               document.get("auxiruta"),
27 |               new com.fasterxml.jackson.core.type.TypeReference<Map<String, Object>>() {});
```

Extrae el bloque auxiruta y lo convierte a mapa apto para serialización JSON.

L28

```
28 |       if (java.util.Arrays.asList(environment.getActiveProfiles()).contains("demo")) {
```

Esta condición decide si se entra en la rama siguiente. verifica pertenencia al conjunto o lista.

L29

```
29 |         rules.put("rules-version", "AR-MVP-5-DEMO");
```

Identifica en el snapshot que los plazos reducidos son de demostración. Añade o reemplaza una propiedad en el objeto JSON o mapa.

L30

```
30 |         override("search", "attempt-timeout-seconds", 60);
```

Solo en perfil demo sustituye search.attempt-timeout-seconds por 60 segundos. Plazo por intento de búsqueda. Las solicitudes existentes conservan su snapshot.

L31

```
31 |         override("search", "evaluation-interval-seconds", 2);
```

Solo en perfil demo sustituye search.evaluation-interval-seconds por 2 segundos. Intervalo de reevaluación. Las solicitudes existentes conservan su snapshot.

L32

```
32 |         override("search", "minimum-offer-window-seconds", 3);
```

Solo en perfil demo sustituye search.minimum-offer-window-seconds por 3 segundos. Tiempo mínimo para publicar una propuesta. Las solicitudes existentes conservan su snapshot.

L33

```
33 |         override("fleet", "hold-seconds", 10);
```

Solo en perfil demo sustituye fleet.hold-seconds por 10 segundos. Retención temporal máxima. Las solicitudes existentes conservan su snapshot.

L34

```
34 |         override("execution", "dispatch-deadline-seconds", 20);
```

Solo en perfil demo sustituye execution.dispatch-deadline-seconds por 20 segundos. Plazo exclusivo para declarar salida. Las solicitudes existentes conservan su snapshot.

L35

```
35 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L36

```
36 |       snapshot = mapper.writeValueAsString(rules);
```

Serializa reglas efectivas una vez para copiarlas a solicitudes nuevas.

L37

```
37 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L38

```
38 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L40

```
40 |   @SuppressWarnings("unchecked")
```

Suprime una advertencia del compilador para esta conversión controlada; no desactiva validaciones de datos en ejecución. unchecked se refiere a genéricos que Java no puede verificar completamente después del borrado de tipos.

L41

```
41 |   private void override(String group, String name, int value) {
```

Sustituye una clave dentro del submapa de reglas; el cast se basa en estructura YAML conocida. Tipos y datos: group = group; String es texto; no es por sí mismo una referencia de entidad; name = name; String es texto; no es por sí mismo una referencia de entidad; value = value; int es entero de 32 bits para límites y contadores.

L42

```
42 |     ((Map<String, Object>) rules.get(group)).put(name, value);
```

Añade o reemplaza una propiedad en el objeto JSON o mapa.

L43

```
43 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L45

```
45 |   public String snapshot() {
```

Devuelve JSON de reglas a persistir en la solicitud.

L46

```
46 |     return snapshot;
```

Entrega snapshot al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: snapshot.

L47

```
47 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L49

```
49 |   public String revision() {
```

Devuelve identificador de revisión de reglas.

L50

```
50 |     return rules.get("rules-version").toString();
```

Resultado que recibe el llamador: representa el valor como texto; en un nodo JSON produce JSON serializado.

L51

```
51 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L53

```
53 |   public int searchSeconds() {
```

Devuelve duración de búsqueda del perfil efectivo para calcular deadline.

L54

```
54 |     return ((Number) ((Map<?, ?>)
     rules.get("search")).get("attempt-timeout-seconds")).intValue();
```

Lee search.attempt-timeout-seconds desde el mapa de reglas, convierte el resultado numérico a int y devuelve el plazo efectivo. Los casts Map/Number son necesarios porque la configuración se cargó como objetos genéricos.

L55

```
55 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L57

```
57 |   public Map<String, Object> effective() {
```

Devuelve mapa de configuración efectiva usado por consulta administrativa.

L58

```
58 |     return rules;
```

Entrega rules al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: configuración o snapshot de reglas que rige esta operación.

L59

```
59 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L60

```
60 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 9, 14, 18, 39, 44, 48, 52, 56: separadores sin instrucciones.

mvp (assistance-service)

`Ruta: assistance-service/src/main/resources/rules/mvp.yml`

Objetivo y funcionalidad

Centraliza la referencia de reglas AR-MVP-5: cobertura de Córdoba, pesos, distancias, tiempos y cupos. RulesConfiguration carga estos valores y guarda una copia en cada solicitud para que las decisiones posteriores puedan justificarse con las reglas originales.

L1

```
1 | # AR-MVP-5: referencia funcional; distribuir en cada servicio según su responsabilidad.
```

Comentario o directiva del intérprete; no es regla de negocio.

L2

```
2 | # Valores del proyecto, no límites legales ni garantía de tiempos reales.
```

Comentario o directiva del intérprete; no es regla de negocio.

L3

```
3 | auxiruta:
```

Propiedades propias del proyecto, leídas por sus componentes.

L4

```
4 |   rules-version: AR-MVP-5
```

Identifica las reglas copiadas al snapshot. El valor configurado es AR-MVP-5. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L5

```
5 |   display-time-zone: America/Argentina/Cordoba
```

Zona de presentación; fechas persistidas en UTC. El valor configurado es America/Argentina/Cordoba. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L6

```
6 |   coverage:
```

Círculo comercial de Córdoba validado geográficamente antes de pedir rutas.

L7

```
7 |     center-latitude: -31.4167
```

Latitud del centro próximo a Plaza San Martín. El valor configurado es -31.4167. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L8

```
8 |     center-longitude: -64.1833
```

Longitud del centro de cobertura. El valor configurado es -64.1833. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L9

```
9 |     radius-meters: 15000
```

Radio comercial, no límite municipal. El valor configurado es 15000. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L10

```
10 |     haversine-earth-radius-meters: 6371000
```

Radio terrestre usado en Haversine. El valor configurado es 6371000. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L11

```
11 |     boundary-inclusive: true
```

Admite exactamente el borde. El valor configurado es true. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L12

```
12 |   vehicles:
```

Tipos/pesos/capacidades usados para compatibilidad de recursos.

L13

```
13 |     allowed-types: [CAR, SUV, PICKUP]
```

Tipos de vehículo admitidos. El valor configurado es [CAR, SUV, PICKUP]. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L14

```
14 |     min-weight-kg: 500
```

Peso mínimo admitido del vehículo. El valor configurado es 500. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L15

```
15 |     max-weight-kg: 3000
```

Peso máximo admitido del vehículo. El valor configurado es 3000. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L16

```
16 |     min-truck-capacity-kg: 500
```

Capacidad mínima registrable de grúa. El valor configurado es 500. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L17

```
17 |     max-truck-capacity-kg: 5000
```

Capacidad máxima registrable de grúa. El valor configurado es 5000. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L18

```
18 |     required-qualification: LIGHT_TOWING
```

Categoría exigida. El valor configurado es LIGHT_TOWING. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L19

```
19 |   routing:
```

Límites aplicados a distancias y tiempos devueltos por ORS.

L20

```
20 |     profile: driving-car
```

Perfil vial ORS driving-car; distinto del perfil Spring demo. El valor configurado es driving-car. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L21

```
21 |     min-transport-meters: 500
```

Traslado vial mínimo. El valor configurado es 500. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L22

```
22 |     max-transport-meters: 30000
```

Traslado vial máximo. El valor configurado es 30000. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L23

```
23 |     max-approach-meters: 15000
```

Aproximación vial máxima. El valor configurado es 15000. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L24

```
24 |     max-arrival-budget-seconds: 2700
```

Presupuesto de llegada con despacho y margen. El valor configurado es 2700. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L25

```
25 |     max-adjusted-transport-seconds: 5400
```

Duración ajustada máxima del traslado. El valor configurado es 5400. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L26

```
26 |     duration-factor: 1.25
```

Multiplicador conservador de tiempos ORS. El valor configurado es 1.25. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L27

```
27 |     arrival-margin-seconds: 300
```

Margen del presupuesto de llegada. El valor configurado es 300. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L28

```
28 |     planning-contingency-seconds: 600
```

Contingencia del fin estimado. El valor configurado es 600. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L29

```
29 |     max-route-age-seconds: 300
```

Vigencia vial al evaluar/proponer y despachar. El valor configurado es 300. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L30

```
30 |     max-snap-distance-meters: 100
```

Desviación máxima al ajustar puntos a la red vial. El valor configurado es 100. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L31

```
31 |   search:
```

Ventanas y cupos de búsqueda automática.

L32

```
32 |     attempt-timeout-seconds: 600
```

Plazo por intento de búsqueda. El valor configurado es 600. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L33

```
33 |     max-offers-per-attempt: 5
```

Cupo de propuestas por intento. El valor configurado es 5. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L34

```
34 |     max-candidates-per-round: 5
```

Máximo de candidatos por preselección. El valor configurado es 5. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L35

```
35 |     evaluation-interval-seconds: 30
```

Intervalo de reevaluación. El valor configurado es 30. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L36

```
36 |     max-reassignments: 2
```

Reemplazos adicionales al intento inicial. El valor configurado es 2. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L37

```
37 |     minimum-offer-window-seconds: 15
```

Tiempo mínimo para publicar una propuesta. El valor configurado es 15. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L38

```
38 |   fleet:
```

Vigencia de ubicación y retención temporal de recursos.

L39

```
39 |     hold-seconds: 60
```

Retención temporal máxima. El valor configurado es 60. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L40

```
40 |     max-location-age-seconds: 300
```

Vigencia de posición para recibir y aceptar propuestas. El valor configurado es 300. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L41

```
41 |     reconciliation-interval-seconds: 15
```

Intervalo de verificación antes de despacho. El valor configurado es 15. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L42

```
42 |   cancellation:
```

Restricción de cancelación antes de la primera aceptación.

L43

```
43 |     allowed-only-before-first-acceptance: true
```

Prohíbe cancelar tras primera aceptación. El valor configurado es true. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L44

```
44 |   assumptions:
```

Supuestos del MVP; no implementan sensores ni verificaciones del mundo real.

L45

```
45 |     declared-vehicle-data-correct: true
```

Se asume que tipo/peso declarados son correctos; no hay inspección física.

L46

```
46 |     pickup-accessible: true
```

Se asume que la grúa puede acceder al vehículo; no se evalúan obstáculos.

L47

```
47 |     destination-available: true
```

Se asume que el destino recibe el vehículo; no existe reserva con un taller.

L48

```
48 |     operator-continuity: true
```

Se asume continuidad del operador; no se modelan imprevistos personales.

L49

```
49 |     operational-continuity-after-dispatch: true
```

Se excluyen averías/interrupciones después de despacho; no se implementa rescate de la propia grúa.

L50

```
50 |   execution:
```

Plazos y tiempos planificados de despacho, carga y descarga.

L51

```
51 |     dispatch-deadline-seconds: 300
```

Plazo exclusivo para declarar salida. El valor configurado es 300. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L52

```
52 |     rollable-loading-seconds: 900
```

Carga prevista si el vehículo rueda. El valor configurado es 900. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L53

```
53 |     non-rollable-loading-seconds: 1800
```

Carga prevista si no rueda. El valor configurado es 1800. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L54

```
54 |     unloading-seconds: 600
```

Descarga prevista. El valor configurado es 600. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L55

```
55 |   integration:
```

Timeouts y esperas para recuperar operaciones remotas.

L56

```
56 |     connect-timeout-seconds: 2
```

Límite de conexión HTTP. El valor configurado es 2. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L57

```
57 |     response-timeout-seconds: 5
```

Límite de respuesta HTTP. El valor configurado es 5. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L58

```
58 |     recovery-delays-seconds: [5, 15, 30]
```

Primeras esperas de recuperación. El valor configurado es [5, 15, 30]. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L59

```
59 |     subsequent-recovery-delay-seconds: 60
```

Espera posterior acotada. El valor configurado es 60. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

L60

```
60 |     rate-limit-default-wait-seconds: 60
```

Espera si no hay Retry-After utilizable. El valor configurado es 60. RulesConfiguration lo incorpora al snapshot de nuevas solicitudes; cambiarlo no modifica solicitudes ya creadas. Es una decisión del MVP, no una garantía de tiempos reales.

7. Evaluación, rutas e integración externa

Haversine aporta cobertura/preselección; ORS, recorridos viales. Un fallo temporal no justifica rechazo de dominio.

DispatchValidator (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/DispatchValidator.java`

Objetivo y funcionalidad

Valida el recurso ya comprometido y refresca evidencia vial antes de despachar. No busca otro candidato ni exige actualizar una ubicación ya aceptada por su antigüedad.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.http.HttpStatus;
```

Importa tipos: org.springframework.http.HttpStatus. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import com.fasterxml.jackson.databind.*;
```

Importa tipos: com.fasterxml.jackson.databind.*. Lectura/escritura JSON, nodos y conversión a DTO. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | import com.auxiruta.assistance.routing.*;
```

Importa tipos: com.auxiruta.assistance.routing.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import static com.auxiruta.assistance.routing.RouteProvider.*;
```

Importa miembros estáticos: com.auxiruta.assistance.routing.RouteProvider.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12 a L13

```
12 | /** Rechecks the currently committed resource, not the list of free candidates. */
13 | @Service
```

Documentación para quien lee el fuente: Rechecks the currently committed resource, not the list of free candidates. */ @Service No es una instrucción ejecutable y no impone la regla por sí sola.

L14

```
14 | public class DispatchValidator {
```

Declara clase DispatchValidator. Valida el recurso ya comprometido y refresca evidencia vial antes de despachar. No busca otro candidato ni exige actualizar una ubicación ya aceptada por su antigüedad.

L15 a L16

```
15 |   public record Evidence(
16 |       String json, Instant validUntil, Instant qualificationUntil, long plannedSeconds) {}
```

Declara record inmutable Evidence. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Valida el recurso ya comprometido y refresca evidencia vial antes de despachar. No busca otro candidato ni exige actualizar una ubicación ya aceptada por su antigüedad. Tipos y datos: json = json; String es texto; no es por sí mismo una referencia de entidad; validUntil = límite temporal de validez de evidencia o autorización; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; qualificationUntil = qualificationUntil; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; plannedSeconds = plannedSeconds; long es entero de 64 bits para cantidades y duraciones mayores.

L18

```
18 |   private final FleetClient fleet;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L19

```
19 |   private final RouteProvider routes;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L20

```
20 |   private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L21

```
21 |   private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L23 a L24

```
23 |   public DispatchValidator(
24 |       FleetClient fleet, RouteProvider routes, ObjectMapper mapper, Clock clock) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada. Tipos y datos: mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java; clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L25

```
25 |     this.fleet = fleet;
```

En this guarda cliente técnico que consulta o modifica reservas en Fleet. Toma el dato de fleet después de las validaciones de este método. cliente técnico que consulta o modifica reservas en Fleet.

L26

```
26 |     this.routes = routes;
```

En this guarda proveedor de recorridos viales, implementado por OpenRouteServiceClient. Toma el dato de routes después de las validaciones de este método. proveedor de recorridos viales, implementado por OpenRouteServiceClient.

L27

```
27 |     this.mapper = mapper;
```

En this guarda conversor JSON de Jackson. Toma el dato de mapper después de las validaciones de este método. conversor JSON de Jackson.

L28

```
28 |     this.clock = clock;
```

En this guarda reloj usado para evaluar plazos. Toma el dato de clock después de las validaciones de este método. reloj usado para evaluar plazos.

L29

```
29 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L31

```
31 |   private static void require(boolean value) {
```

Expresa una precondición; si resulta falsa se aborta con el estado HTTP indicado. Tipos y datos: value = value; boolean es verdadero/falso, sin null.

L32

```
32 |     if (!value) throw new ResponseStatusException(HttpStatus.CONFLICT, "DISPATCH_NOT_ELIGIBLE");
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. interrumpe la operación con un estado HTTP explícito; 409 comunica que el estado/regla actual impide la acción; ! invierte el booleano de la expresión que sigue.

L33

```
33 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L35

```
35 |   private static int value(JsonNode rules, String group, String name) {
```

Lee un parámetro entero del snapshot JSON. Tipos y datos: rules = configuración o snapshot de reglas que rige esta operación; JsonNode es árbol JSON navegable usado para snapshots y respuestas externas; group = group; String es texto; no es por sí mismo una referencia de entidad; name = name; String es texto; no es por sí mismo una referencia de entidad.

L36

```
36 |     return rules.path(group).path(name).asInt();
```

Resultado que recibe el llamador: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a entero.

L37

```
37 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L39

```
39 |   public Evidence validate(ExecutionStore.Input input) {
```

Revalida recurso y recorridos fuera de la transacción SQL y produce evidencia para commit.

L40

```
40 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L41

```
41 |       JsonNode rules = mapper.readTree(input.rules());
```

Se calcula rules (configuración o snapshot de reglas que rige esta operación) para reutilizar ese resultado en los pasos siguientes. analiza texto JSON para consultar su estructura; Datos implicados: configuración o snapshot de reglas que rige esta operación

L42

```
42 |       var resource = fleet.eligibility(input.assignmentId());
```

Obtiene snapshot actual del compromiso Fleet, no la lista de grúas disponibles. Se calcula resource (estado del recurso comprometido devuelto por Fleet) para reutilizar ese resultado en los pasos siguientes.

L43 a L49

```
43 |       if (resource == null
44 |           || !input.assignmentId().equals(resource.assignmentAttemptId())
45 |           || !input.requestId().equals(resource.requestId())
46 |           || !input.truckId().equals(resource.towTruckId())
47 |           || !input.operatorId().equals(resource.operatorId())
48 |           || !input.subject().equals(resource.operatorSubject()))
49 |         throw new IllegalStateException("FLEET_INVALID_RESPONSE");
```

Valida identidad del intento, solicitud, grúa, operador y subject de la respuesta Fleet. Esta condición decide si se entra en la rama siguiente. compara contenido/identidad lógica, no referencias Java; Datos implicados: UUID de solicitud, UUID de grúa, UUID del perfil operador, sub del operador destinatario; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue; detecta un dato ausente.

L44: compara contenido/identidad lógica, no referencias Java; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L45: compara contenido/identidad lógica, no referencias Java; Datos implicados: UUID de solicitud; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L46: compara contenido/identidad lógica, no referencias Java; Datos implicados: UUID de grúa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L47: compara contenido/identidad lógica, no referencias Java; Datos implicados: UUID del perfil operador; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L48: compara contenido/identidad lógica, no referencias Java; Datos implicados: sub del operador destinatario; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L50

```
50 |       require(resource.eligible());
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. Datos implicados: aptitud completa del candidato/compromiso

L51

```
51 |       Instant now = clock.instant();
```

Captura un instante consistente de referencia para comparar condiciones temporales en esta operación. Se calcula now (instante actual obtenido del reloj inyectado) para reutilizar ese resultado en los pasos siguientes. lee el instante del reloj inyectado

L52 a L53

```
52 |       int routeAge = value(rules, "routing", "max-route-age-seconds"),
53 |           snap = value(rules, "routing", "max-snap-distance-meters");
```

Lee del snapshot vigencia de ruta y snap máximo; captura instante de control. Se calcula routeAge (máxima edad permitida de la evidencia vial) para reutilizar ese resultado en los pasos siguientes.

L54 a L59

```
54 |       if (resource.latitude() == null
55 |           || resource.longitude() == null
56 |           || resource.locationUpdatedAt() == null
57 |           || resource.qualificationValidUntil() == null
58 |           || resource.checkedAt() == null)
59 |         throw new IllegalStateException("FLEET_INVALID_RESPONSE");
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. Datos implicados: latitud declarada, longitud declarada, instante servidor de última ubicación declarada, fin de habilitación que debe superar el servicio estimado; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L55: Datos implicados: longitud declarada; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L56: Datos implicados: instante servidor de última ubicación declarada; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L57: Datos implicados: fin de habilitación que debe superar el servicio estimado; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L58: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L60

```
60 |       require(!resource.locationUpdatedAt().isAfter(now) && !resource.checkedAt().isAfter(now));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara si el instante es estrictamente posterior al otro; Datos implicados: instante servidor de última ubicación declarada; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue

L61 a L65

```
61 |       require(
62 |           EvaluationService.inCoverage(resource.latitude(), resource.longitude(), rules)
63 |               && EvaluationService.inCoverage(input.originLat(), input.originLon(), rules)
64 |               && EvaluationService.inCoverage(
65 |                   input.destinationLat(), input.destinationLon(), rules));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. Datos implicados: latitud declarada, longitud declarada; && exige todas las condiciones y deja de evaluar al encontrar una falsa

L63: && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L64: && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L66 a L68

```
66 |       Point origin = new Point(input.originLat(), input.originLon()),
67 |           destination = new Point(input.destinationLat(), input.destinationLon()),
68 |           truck = new Point(resource.latitude(), resource.longitude());
```

Construye Point en memoria y conserva la referencia en origin. Los argumentos del constructor determinan los datos iniciales de este objeto. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L69

```
69 |       String revision = rules.path("rules-version").asText();
```

Se calcula revision (revision) para reutilizar ese resultado en los pasos siguientes. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a texto; Ruta JSON consultada: rules-version; Identifica las reglas copiadas al snapshot.

L70

```
70 |       Route transport = routes.route(origin, destination, revision, routeAge, snap);
```

Enruta el patrón al microservicio destino; no ejecuta la regla de negocio. Se calcula transport (transport) para reutilizar ese resultado en los pasos siguientes.

L71 a L75

```
71 |       Route approach =
72 |           truck.latitude().compareTo(origin.latitude()) == 0
73 |                   && truck.longitude().compareTo(origin.longitude()) == 0
74 |               ? new Route(BigDecimal.ZERO, BigDecimal.ZERO, transport.obtainedAt())
75 |               : routes.route(truck, origin, revision, routeAge, snap);
```

Enruta el patrón al microservicio destino; no ejecuta la regla de negocio. Se calcula approach (approach) para reutilizar ese resultado en los pasos siguientes. compara decimales: negativo menor, cero igual, positivo mayor; && exige todas las condiciones y deja de evaluar al encontrar una falsa; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario

L73: compara decimales: negativo menor, cero igual, positivo mayor; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L76

```
76 |       BigDecimal factor = rules.path("routing").path("duration-factor").decimalValue();
```

Obtiene factor decimal y calcula tiempos ajustados con redondeo hacia arriba. Se calcula factor (factor) para reutilizar ese resultado en los pasos siguientes. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; lee el número JSON como BigDecimal; Ruta JSON consultada: routing.duration-factor; Multiplicador conservador de tiempos ORS.

L77 a L79

```
77 |       int adjustedTransport = EvaluationService.adjusted(transport.durationSeconds(), factor),
78 |           adjustedApproach = EvaluationService.adjusted(approach.durationSeconds(), factor),
79 |           dispatch = value(rules, "execution", "dispatch-deadline-seconds");
```

Se calcula adjustedTransport (tiempo de traslado con factor conservador) para reutilizar ese resultado en los pasos siguientes.

L80 a L91

```
80 |       require(
81 |           transport
82 |                       .distanceM()
83 |                       .compareTo(
84 |                           BigDecimal.valueOf(value(rules, "routing", "min-transport-meters")))
85 |                   >= 0
86 |               && transport
87 |                       .distanceM()
88 |                       .compareTo(
89 |                           BigDecimal.valueOf(value(rules, "routing", "max-transport-meters")))
90 |                   <= 0
91 |               && adjustedTransport <= value(rules, "routing", "max-adjusted-transport-seconds"));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara decimales: negativo menor, cero igual, positivo mayor; && exige todas las condiciones y deja de evaluar al encontrar una falsa

L83: compara decimales: negativo menor, cero igual, positivo mayor.

L86: && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L88: compara decimales: negativo menor, cero igual, positivo mayor.

L91: && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L92 a L100

```
92 |       require(
93 |           approach
94 |                       .distanceM()
95 |                       .compareTo(BigDecimal.valueOf(value(rules, "routing",
     "max-approach-meters")))
96 |                   <= 0
97 |               && (long) dispatch
98 |                       + adjustedApproach
99 |                       + value(rules, "routing", "arrival-margin-seconds")
100 |                  <= value(rules, "routing", "max-arrival-budget-seconds"));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara decimales: negativo menor, cero igual, positivo mayor; && exige todas las condiciones y deja de evaluar al encontrar una falsa

L95: compara decimales: negativo menor, cero igual, positivo mayor.

L97: && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L101 a L112

```
101 |       long planned =
102 |           (long) dispatch
103 |               + adjustedApproach
104 |               + adjustedTransport
105 |               + value(
106 |                   rules,
107 |                   "execution",
108 |                   input.mobility().equals("ROLLABLE")
109 |                       ? "rollable-loading-seconds"
110 |                       : "non-rollable-loading-seconds")
111 |               + value(rules, "execution", "unloading-seconds")
112 |               + value(rules, "routing", "planning-contingency-seconds");
```

Planifica desde ahora despacho, aproximación, traslado, carga, descarga y contingencia para exigir habilitación suficiente. Se calcula planned (duración planificada total para comprobar vigencia de habilitación) para reutilizar ese resultado en los pasos siguientes. compara contenido/identidad lógica, no referencias Java; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario

L113

```
113 |       Instant validUntil = transport.obtainedAt().plusSeconds(routeAge);
```

Vence con la más antigua de ambas rutas. No agrega caducidad de ubicación posterior a aceptación. Se calcula validUntil (límite temporal de validez de evidencia o autorización) para reutilizar ese resultado en los pasos siguientes. calcula un instante futuro sumando segundos, sin modificar el instante original

L114 a L118

```
114 |       for (Instant expiry :
115 |           java.util.List.of(
116 |               approach.obtainedAt().plusSeconds(routeAge),
117 |               transport.obtainedAt().plusSeconds(routeAge)))
118 |         if (expiry.isBefore(validUntil)) validUntil = expiry;
```

Compara los vencimientos de evidencia de aproximación y traslado y conserva el más cercano en validUntil. Las dos rutas deben estar vigentes simultáneamente; una ruta reciente no compensa que la otra haya vencido.

L119

```
119 |       now = clock.instant();
```

Lee el instante del reloj inyectado.

L120 a L122

```
120 |       require(
121 |           !now.isAfter(validUntil)
122 |               && resource.qualificationValidUntil().isAfter(now.plusSeconds(planned)));
```

Impide despachar si la evidencia vial ya venció o si la habilitación no permanece vigente más allá de todo el servicio planificado. !isAfter admite el instante exacto validUntil; isAfter exige que qualificationValidUntil sea estrictamente posterior a now + planned. Fallar produce el conflicto definido por require.

L122: calcula un instante futuro sumando segundos, sin modificar el instante original; compara si el instante es estrictamente posterior al otro; Datos implicados: fin de habilitación que debe superar el servicio estimado; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L123

```
123 |       var evidence = mapper.createObjectNode();
```

Se calcula evidence (JSON de datos usados para justificar la decisión) para reutilizar ese resultado en los pasos siguientes. crea un objeto JSON vacío que se completará como evidencia

L124

```
124 |       evidence.put("assignmentId", input.assignmentId().toString());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Representa el valor como texto; en un nodo JSON produce JSON serializado; añade o reemplaza una propiedad en el objeto JSON o mapa.

L125

```
125 |       evidence.put("fleetCheckedAt", resource.checkedAt().toString());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Representa el valor como texto; en un nodo JSON produce JSON serializado; añade o reemplaza una propiedad en el objeto JSON o mapa.

L126

```
126 |       evidence.put("locationUpdatedAt", resource.locationUpdatedAt().toString());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Representa el valor como texto; en un nodo JSON produce JSON serializado; añade o reemplaza una propiedad en el objeto JSON o mapa; Datos implicados: instante servidor de última ubicación declarada.

L127

```
127 |       evidence.put("latitude", resource.latitude());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Añade o reemplaza una propiedad en el objeto JSON o mapa; Datos implicados: latitud declarada.

L128

```
128 |       evidence.put("longitude", resource.longitude());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Añade o reemplaza una propiedad en el objeto JSON o mapa; Datos implicados: longitud declarada.

L129

```
129 |       evidence.put("approachDistanceM", approach.distanceM());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Añade o reemplaza una propiedad en el objeto JSON o mapa.

L130

```
130 |       evidence.put("transportDistanceM", transport.distanceM());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Añade o reemplaza una propiedad en el objeto JSON o mapa.

L131

```
131 |       evidence.put("adjustedApproachSeconds", adjustedApproach);
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Añade o reemplaza una propiedad en el objeto JSON o mapa.

L132

```
132 |       evidence.put("adjustedTransportSeconds", adjustedTransport);
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Añade o reemplaza una propiedad en el objeto JSON o mapa.

L133

```
133 |       evidence.put("approachRouteObtainedAt", approach.obtainedAt().toString());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Representa el valor como texto; en un nodo JSON produce JSON serializado; añade o reemplaza una propiedad en el objeto JSON o mapa.

L134

```
134 |       evidence.put("transportRouteObtainedAt", transport.obtainedAt().toString());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Representa el valor como texto; en un nodo JSON produce JSON serializado; añade o reemplaza una propiedad en el objeto JSON o mapa.

L135

```
135 |       evidence.put("qualificationValidUntil", resource.qualificationValidUntil().toString());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Representa el valor como texto; en un nodo JSON produce JSON serializado; añade o reemplaza una propiedad en el objeto JSON o mapa; Datos implicados: fin de habilitación que debe superar el servicio estimado.

L136

```
136 |       evidence.put("estimatedServiceEndAt", now.plusSeconds(planned).toString());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Calcula un instante futuro sumando segundos, sin modificar el instante original; representa el valor como texto; en un nodo JSON produce JSON serializado; añade o reemplaza una propiedad en el objeto JSON o mapa.

L137 a L138

```
137 |       return new Evidence(
138 |           evidence.toString(), validUntil, resource.qualificationValidUntil(), planned);
```

Devuelve evidencia JSON, vencimiento vial, fin de habilitación y duración planificada para revalidación transaccional final. Resultado que recibe el llamador: representa el valor como texto; en un nodo JSON produce JSON serializado; Datos implicados: fin de habilitación que debe superar el servicio estimado.

L139

```
139 |     } catch (ResponseStatusException e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L140

```
140 |       throw e;
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L141

```
141 |     } catch (RoutingFailure e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L142 a L143

```
142 |       throw new ResponseStatusException(
143 |           e.domainRejection() ? HttpStatus.CONFLICT : HttpStatus.SERVICE_UNAVAILABLE);
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde. interrumpe la operación con un estado HTTP explícito; 409 comunica que el estado/regla actual impide la acción; 503 comunica dependencia temporalmente indisponible; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario

L144

```
144 |     } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L145

```
145 |       throw new ResponseStatusException(HttpStatus.SERVICE_UNAVAILABLE);
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde. interrumpe la operación con un estado HTTP explícito; 503 comunica dependencia temporalmente indisponible

L146

```
146 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L147

```
147 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L148

```
148 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 11, 17, 22, 30, 34, 38: separadores sin instrucciones.

EvaluationReadService (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/EvaluationReadService.java`

Objetivo y funcionalidad

Proyecta la evaluación más reciente y sus candidatos hacia DTO para consultar la justificación de la decisión.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | @Service
```

Registra lógica de aplicación como bean. El escaneo de Spring crea este componente e inyecta sus dependencias; no hay que construirlo manualmente en el controlador.

L10

```
10 | @Transactional(readOnly = true)
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L11

```
11 | public class EvaluationReadService {
```

Declara clase EvaluationReadService. Proyecta la evaluación más reciente y sus candidatos hacia DTO para consultar la justificación de la decisión.

L12 a L19

```
12 |   public record CandidateView(
13 |       UUID towTruckId,
14 |       boolean eligible,
15 |       String reasonCode,
16 |       BigDecimal approachDistanceM,
17 |       Integer adjustedApproachSeconds,
18 |       Instant estimatedServiceEndAt,
19 |       Integer rankingPosition) {}
```

Declara record inmutable CandidateView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: towTruckId: UUID de grúa; eligible: aptitud completa del candidato/compromiso; reasonCode: causa codificada; approachDistanceM: distancia vial grúa-recogida; adjustedApproachSeconds: aproximación ajustada; estimatedServiceEndAt: fin planificado para controlar habilitación; rankingPosition: posición entre elegibles, comenzando en 1. Proyecta la evaluación más reciente y sus candidatos hacia DTO para consultar la justificación de la decisión. Tipos y datos: towTruckId = UUID de grúa; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; eligible = aptitud completa del candidato/compromiso; boolean es verdadero/falso, sin null; reasonCode = causa codificada; String es texto; no es por sí mismo una referencia de entidad; approachDistanceM = distancia vial grúa-recogida; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; estimatedServiceEndAt = fin planificado para controlar habilitación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L13: Dato del contrato: towTruckId: UUID de grúa.

L14: Dato del contrato: eligible: aptitud completa del candidato/compromiso.

L15: Dato del contrato: reasonCode: causa codificada.

L16: Dato del contrato: approachDistanceM: distancia vial grúa-recogida.

L18: Dato del contrato: estimatedServiceEndAt: fin planificado para controlar habilitación.

L21 a L28

```
21 |   public record EvaluationView(
22 |       UUID id,
23 |       String result,
24 |       String reasonCode,
25 |       Instant evaluatedAt,
26 |       BigDecimal transportDistanceM,
27 |       Integer adjustedTransportSeconds,
28 |       List<CandidateView> candidates) {}
```

Declara record inmutable EvaluationView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; result: resultado de evaluación, distinto del estado de solicitud; reasonCode: causa codificada; evaluatedAt: instante de evaluación; transportDistanceM: distancia vial origen-destino en metros; adjustedTransportSeconds: duración ajustada y redondeada hacia arriba. Proyecta la evaluación más reciente y sus candidatos hacia DTO para consultar la justificación de la decisión. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; result = resultado de evaluación, distinto del estado de solicitud; String es texto; no es por sí mismo una referencia de entidad; reasonCode = causa codificada; String es texto; no es por sí mismo una referencia de entidad; evaluatedAt = instante de evaluación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; transportDistanceM = distancia vial origen-destino en metros; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L22: Dato del contrato: id: identificador UUID del registro.

L23: Dato del contrato: result: resultado de evaluación, distinto del estado de solicitud.

L24: Dato del contrato: reasonCode: causa codificada.

L25: Dato del contrato: evaluatedAt: instante de evaluación.

L26: Dato del contrato: transportDistanceM: distancia vial origen-destino en metros.

L30

```
30 |   private final EvaluationRepository evaluations;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L31

```
31 |   private final CandidateEvaluationRepository candidates;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L33 a L34

```
33 |   public EvaluationReadService(
34 |       EvaluationRepository evaluations, CandidateEvaluationRepository candidates) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L35

```
35 |     this.evaluations = evaluations;
```

En this guarda acceso a evaluaciones persistidas. Toma el dato de evaluations después de las validaciones de este método. acceso a evaluaciones persistidas.

L36

```
36 |     this.candidates = candidates;
```

En this guarda candidates. Toma el dato de candidates después de las validaciones de este método. candidates.

L37

```
37 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L39

```
39 |   public EvaluationView latest(UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L40 a L68

```
40 |     return evaluations
41 |         .findFirstByRequestIdOrderByEvaluatedAtDescIdDesc(id)
42 |         .map(
43 |             e ->
44 |                 new EvaluationView(
45 |                     e.id,
46 |                     e.result,
47 |                     e.reasonCode,
48 |                     e.evaluatedAt,
49 |                     e.transportDistanceM,
50 |                     e.adjustedTransportSeconds,
51 |                     candidates.findByEvaluationId(e.id).stream()
52 |                         .sorted(
53 |                             Comparator.comparing(
54 |                                     (CandidateEvaluation c) -> c.rankingPosition,
55 |                                     Comparator.nullsLast(Comparator.naturalOrder()))
56 |                                 .thenComparing(c -> c.towTruckId.toString()))
57 |                         .map(
58 |                             c ->
59 |                                 new CandidateView(
60 |                                     c.towTruckId,
61 |                                     c.eligible,
```

```
62 |                                     c.reasonCode,
63 |                                     c.approachDistanceM,
64 |                                     c.adjustedApproachSeconds,
65 |                                     c.estimatedServiceEndAt,
66 |                                     c.rankingPosition))
67 |                         .toList()))
68 |         .orElse(null);
```

Ordena por el comparador explícito para desempate reproducible. Transforma cada elemento al valor/tipo de salida. Materializa la secuencia en lista. Recupera evaluación más reciente por fecha e ID. Resultado que recibe el llamador: inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente; transforma cada elemento al tipo/valor de salida; materializa el resultado de la secuencia en una lista; representa el valor como texto; en un nodo JSON produce JSON serializado; usa un valor alternativo cuando no hay resultado; Datos implicados: identificador UUID del registro, resultado de evaluación, distinto del estado de solicitud, causa codificada, instante de evaluación, distancia vial origen-destino en metros, duración ajustada y redondeada hacia arriba, posición entre elegibles, comenzando en 1, UUID de grúa, aptitud completa del candidato/compromiso, distancia vial grúa-recogida, aproximación ajustada, fin planificado para controlar habilitación.

L42: transforma cada elemento al tipo/valor de salida.

L56: representa el valor como texto; en un nodo JSON produce JSON serializado; Datos implicados: UUID de grúa.

L57: transforma cada elemento al tipo/valor de salida.

L67: materializa el resultado de la secuencia en una lista.

L68: usa un valor alternativo cuando no hay resultado.

L69

```
69 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L70

```
70 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 8, 20, 29, 32, 38: separadores sin instrucciones.

EvaluationService (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/EvaluationService.java`

Objetivo y funcionalidad

Calcula cobertura, traslado y aproximación con Fleet/ORS fuera de transacciones largas. Distingue rechazo de dominio de dependencia temporal; rankea por tiempo ajustado, antigüedad de disponibilidad y UUID.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.math.*;
```

Importa tipos: java.math.*. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import com.fasterxml.jackson.databind.*;
```

Importa tipos: com.fasterxml.jackson.databind.*. Lectura/escritura JSON, nodos y conversión a DTO. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import com.auxiruta.commons.GeoMath;
```

Importa tipos: com.auxiruta.commons.GeoMath. Utilidades compartidas de seguridad, errores, reloj o geografía. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | import com.auxiruta.assistance.routing.*;
```

Importa tipos: com.auxiruta.assistance.routing.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import static com.auxiruta.assistance.routing.RouteProvider.*;
```

Importa miembros estáticos: com.auxiruta.assistance.routing.RouteProvider.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12

```
12 | @Service
```

Registra lógica de aplicación como bean. El escaneo de Spring crea este componente e inyecta sus dependencias; no hay que construirlo manualmente en el controlador.

L13

```
13 | public class EvaluationService {
```

Declara clase EvaluationService. Calcula cobertura, traslado y aproximación con Fleet/ORS fuera de transacciones largas. Distingue rechazo de dominio de dependencia temporal; rankea por tiempo ajustado, antigüedad de disponibilidad y UUID.

L14

```
14 |   private final EvaluationStore store;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L15

```
15 |   private final RouteProvider routes;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L16

```
16 |   private final FleetClient fleet;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L17

```
17 |   private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L18

```
18 |   private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L20 a L25

```
20 |   public EvaluationService(
21 |       EvaluationStore store,
22 |       RouteProvider routes,
23 |       FleetClient fleet,
24 |       Clock clock,
25 |       ObjectMapper mapper) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos; mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L26

```
26 |     this.store = store;
```

En this guarda componente que conserva decisiones en transacciones locales. Toma el dato de store después de las validaciones de este método. componente que conserva decisiones en transacciones locales.

L27

```
27 |     this.routes = routes;
```

En this guarda proveedor de recorridos viales, implementado por OpenRouteServiceClient. Toma el dato de routes después de las validaciones de este método. proveedor de recorridos viales, implementado por OpenRouteServiceClient.

L28

```
28 |     this.fleet = fleet;
```

En this guarda cliente técnico que consulta o modifica reservas en Fleet. Toma el dato de fleet después de las validaciones de este método. cliente técnico que consulta o modifica reservas en Fleet.

L29

```
29 |     this.clock = clock;
```

En this guarda reloj usado para evaluar plazos. Toma el dato de clock después de las validaciones de este método. reloj usado para evaluar plazos.

L30

```
30 |     this.mapper = mapper;
```

En this guarda conversor JSON de Jackson. Toma el dato de mapper después de las validaciones de este método. conversor JSON de Jackson.

L31

```
31 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L33

```
33 |   static int adjusted(BigDecimal seconds, BigDecimal factor) {
```

Multiplica segundos por el factor y redondea hacia arriba, evitando subestimar duración. Tipos y datos: seconds = seconds; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; factor = factor; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L34

```
34 |     return seconds.multiply(factor).setScale(0, RoundingMode.CEILING).intValueExact();
```

Resultado que recibe el llamador: redondea hacia arriba el cálculo decimal; fija precisión decimal y regla de redondeo.

L35

```
35 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L37

```
37 |   static boolean inCoverage(BigDecimal lat, BigDecimal lon, JsonNode rules) {
```

Calcula Haversine y admite el borde inclusive del radio comercial. Tipos y datos: lat = lat; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; lon = lon; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; rules = configuración o snapshot de reglas que rige esta operación; JsonNode es árbol JSON navegable usado para snapshots y respuestas externas.

L38

```
38 |     var c = rules.path("coverage");
```

Se calcula c (c) para reutilizar ese resultado en los pasos siguientes. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; Ruta JSON consultada: coverage

L39 a L45

```
39 |     return GeoMath.distance(
40 |             c.path("center-latitude").asDouble(),
41 |             c.path("center-longitude").asDouble(),
42 |             lat.doubleValue(),
43 |             lon.doubleValue(),
44 |             c.path("haversine-earth-radius-meters").asDouble())
45 |         <= c.path("radius-meters").asDouble();
```

Resultado que recibe el llamador: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; Ruta JSON consultada: center-latitude.center-longitude.haversine-earth-radius-meters.radius-meters; Latitud del centro próximo a Plaza San Martín.; Longitud del centro de cobertura.; Radio terrestre usado en Haversine.; Radio comercial, no límite municipal..

L46

```
46 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L48

```
48 |   static int value(JsonNode rules, String group, String name) {
```

Lee un parámetro entero del snapshot JSON. Tipos y datos: rules = configuración o snapshot de reglas que rige esta operación; JsonNode es árbol JSON navegable usado para snapshots y respuestas externas; group = group; String es texto; no es por sí mismo una referencia de entidad; name = name; String es texto; no es por sí mismo una referencia de entidad.

L49

```
49 |     return rules.path(group).path(name).asInt();
```

Resultado que recibe el llamador: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a entero.

L50

```
50 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L52

```
52 |   private static boolean same(Point a, Point b) {
```

Compara coordenadas por valor numérico, no por escala decimal.

L53

```
53 |     return a.latitude().compareTo(b.latitude()) == 0 && a.longitude().compareTo(b.longitude()) ==
       0;
```

Resultado que recibe el llamador: compara decimales: negativo menor, cero igual, positivo mayor; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L54

```
54 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L56

```
56 |   public void evaluate(UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L57

```
57 |     EvaluationStore.Input input = store.load(id);
```

Se calcula input (snapshot de entrada que evita trabajar con una entidad mutable fuera de transacción) para reutilizar ese resultado en los pasos siguientes.

L58

```
58 |     if (input == null) return;
```

Esta condición decide si se entra en la rama siguiente. detecta un dato ausente.

L59

```
59 |     JsonNode rules;
```

Tipos y datos: rules = configuración o snapshot de reglas que rige esta operación; JsonNode es árbol JSON navegable usado para snapshots y respuestas externas.

L60

```
60 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L61

```
61 |       rules = mapper.readTree(input.rules());
```

Analiza texto JSON para consultar su estructura; Datos implicados: configuración o snapshot de reglas que rige esta operación.

L62

```
62 |     } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L63

```
63 |       throw new IllegalStateException("INVALID_RULE_SNAPSHOT");
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L64

```
64 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L65

```
65 |     Evaluation e = new Evaluation();
```

Construye Evaluation en memoria y conserva la referencia en e. Entidad del resultado de una evaluación, con distancias, tiempos, motivos y datos usados. No equivale a una asignación aceptada. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L66

```
66 |     e.searchAttemptNumber = input.attemptNumber();
```

En e guarda número de búsqueda de 1 a 3. Toma el dato de input.attemptNumber() después de las validaciones de este método.

L67

```
67 |     e.originLatitude = input.originLat();
```

En e guarda latitud de recogida. Toma el dato de input.originLat() después de las validaciones de este método.

L68

```
68 |     e.originLongitude = input.originLon();
```

En e guarda longitud de recogida. Toma el dato de input.originLon() después de las validaciones de este método.

L69

```
69 |     e.destinationLatitude = input.destinationLat();
```

En e guarda latitud de entrega. Toma el dato de input.destinationLat() después de las validaciones de este método.

L70

```
70 |     e.destinationLongitude = input.destinationLon();
```

En e guarda longitud de entrega. Toma el dato de input.destinationLon() después de las validaciones de este método.

L71

```
71 |     e.vehicleType = input.vehicleType();
```

En e guarda tipo CAR/SUV/PICKUP. Toma el dato de input.vehicleType() después de las validaciones de este método. Datos implicados: tipo CAR/SUV/PICKUP.

L72

```
72 |     e.declaredWeightKg = input.weight();
```

En e guarda peso declarado en kg. Toma el dato de input.weight() después de las validaciones de este método.

L73

```
73 |     e.mobilityCondition = input.mobility();
```

En e guarda condición ROLLABLE/NON_ROLLABLE. Toma el dato de input.mobility() después de las validaciones de este método.

L74

```
74 |     e.configurationRevision = rules.path("rules-version").asText();
```

En e guarda versión de reglas. Toma el dato de rules.path("rules-version").asText() después de las validaciones de este método. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a texto; Ruta JSON consultada: rules-version; Identifica las reglas copiadas al snapshot..

L75

```
75 |     e.maximumArrivalSeconds = value(rules, "routing", "max-arrival-budget-seconds");
```

En e guarda presupuesto máximo hasta llegada. Toma el dato de value(rules, "routing", "max-arrival-budget-seconds") después de las validaciones de este método.

L76 a L82

```
76 |     e.loadingSeconds =
77 |         value(
78 |             rules,
79 |             "execution",
80 |             input.mobility().equals("ROLLABLE")
81 |                 ? "rollable-loading-seconds"
82 |                 : "non-rollable-loading-seconds");
```

En e guarda carga prevista. Toma el dato de value(rules, "execution", input.mobility().equals("ROLLABLE") ? "rollable-loading-seconds" : "non-rollable-loading-seconds") después de las validaciones de este método. compara contenido/identidad lógica, no referencias Java; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L83

```
83 |     e.unloadingSeconds = value(rules, "execution", "unloading-seconds");
```

En e guarda descarga prevista. Toma el dato de value(rules, "execution", "unloading-seconds") después de las validaciones de este método.

L84

```
84 |     e.operationalMarginSeconds = value(rules, "routing", "planning-contingency-seconds");
```

En e guarda contingencia de planificación. Toma el dato de value(rules, "routing", "planning-contingency-seconds") después de las validaciones de este método.

L85

```
85 |     e.coverageAndAdmissionSummary = "AR-MVP-5 admission";
```

En e guarda resumen de cobertura y admisión. Toma el dato de "AR-MVP-5 admission" después de las validaciones de este método.

L86

```
86 |     List<CandidateEvaluation> details = new ArrayList<>();
```

Construye ArrayList<> en memoria y conserva la referencia en details. Los argumentos del constructor determinan los datos iniciales de este objeto. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L87 a L88

```
87 |     if (!inCoverage(input.originLat(), input.originLon(), rules)
88 |         || !inCoverage(input.destinationLat(), input.destinationLon(), rules)) {
```

Esta condición decide si se entra en la rama siguiente. || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L88: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L89

```
89 |       e.result = "REJECTED";
```

En e guarda resultado de evaluación, distinto del estado de solicitud. Toma el dato de "REJECTED" después de las validaciones de este método.

L90

```
90 |       e.reasonCode = "OUTSIDE_COVERAGE";
```

En e guarda causa codificada. Toma el dato de "OUTSIDE_COVERAGE" después de las validaciones de este método.

L91

```
91 |       store.save(input, e, details);
```

Entrega resultado y candidatos al store, que decide bajo bloqueo si aún corresponde persistirlos. Solicita persistir la entidad en el contexto JPA.

L92

```
92 |       return;
```

Return.

L93

```
93 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L94

```
94 |     e.coverageAndAdmissionSummary = "Origin and destination inside coverage";
```

En e guarda resumen de cobertura y admisión. Toma el dato de "Origin and destination inside coverage" después de las validaciones de este método.

L95 a L96

```
95 |     int age = value(rules, "routing", "max-route-age-seconds"),
96 |         snap = value(rules, "routing", "max-snap-distance-meters");
```

Lee antigüedad máxima de ruta, tolerancia de snap y factor del snapshot. Se calcula age (age) para reutilizar ese resultado en los pasos siguientes.

L97

```
97 |     BigDecimal factor = rules.path("routing").path("duration-factor").decimalValue();
```

Obtiene factor decimal y calcula tiempos ajustados con redondeo hacia arriba. Se calcula factor (factor) para reutilizar ese resultado en los pasos siguientes. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; lee el número JSON como BigDecimal; Ruta JSON consultada: routing.duration-factor; Multiplicador conservador de tiempos ORS.

L98 a L99

```
98 |     Point origin = new Point(input.originLat(), input.originLon()),
99 |         destination = new Point(input.destinationLat(), input.destinationLon());
```

Construye Point en memoria y conserva la referencia en origin. Los argumentos del constructor determinan los datos iniciales de este objeto. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L100

```
100 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L101

```
101 |       Route transport = routes.route(origin, destination, e.configurationRevision, age, snap);
```

Enruta el patrón al microservicio destino; no ejecuta la regla de negocio. Se calcula transport (transport) para reutilizar ese resultado en los pasos siguientes. Datos implicados: versión de reglas

L102

```
102 |       e.transportDistanceM = transport.distanceM();
```

En e guarda distancia vial origen-destino en metros. Toma el dato de transport.distanceM() después de las validaciones de este método.

L103

```
103 |       e.transportDurationSeconds = transport.durationSeconds();
```

En e guarda duración vial original del traslado. Toma el dato de transport.durationSeconds() después de las validaciones de este método.

L104

```
104 |       e.adjustedTransportSeconds = adjusted(transport.durationSeconds(), factor);
```

En e guarda duración ajustada y redondeada hacia arriba. Toma el dato de adjusted(transport.durationSeconds(), factor) después de las validaciones de este método.

L105

```
105 |       e.routeObtainedAt = transport.obtainedAt();
```

En e guarda fecha del recorrido. Toma el dato de transport.obtainedAt() después de las validaciones de este método.

L106 a L113

```
106 |       if (transport
107 |                   .distanceM()
108 |                   .compareTo(BigDecimal.valueOf(value(rules, "routing", "min-transport-meters")))
109 |               < 0
110 |           || transport
111 |                   .distanceM()
112 |                   .compareTo(BigDecimal.valueOf(value(rules, "routing", "max-transport-meters")))
113 |               > 0) {
```

Esta condición decide si se entra en la rama siguiente. compara decimales: negativo menor, cero igual, positivo mayor; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L108: compara decimales: negativo menor, cero igual, positivo mayor.

L110: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L112: compara decimales: negativo menor, cero igual, positivo mayor.

L114

```
114 |         e.result = "REJECTED";
```

En e guarda resultado de evaluación, distinto del estado de solicitud. Toma el dato de "REJECTED" después de las validaciones de este método.

L115

```
115 |         e.reasonCode = "TRANSPORT_DISTANCE";
```

En e guarda causa codificada. Toma el dato de "TRANSPORT_DISTANCE" después de las validaciones de este método.

L116

```
116 |         store.save(input, e, details);
```

Entrega resultado y candidatos al store, que decide bajo bloqueo si aún corresponde persistirlos. Solicita persistir la entidad en el contexto JPA.

L117

```
117 |         return;
```

Return.

L118

```
118 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L119

```
119 |       if (e.adjustedTransportSeconds > value(rules, "routing", "max-adjusted-transport-seconds"))
        {
```

Esta condición decide si se entra en la rama siguiente. Datos implicados: duración ajustada y redondeada hacia arriba.

L120

```
120 |         e.result = "REJECTED";
```

En e guarda resultado de evaluación, distinto del estado de solicitud. Toma el dato de "REJECTED" después de las validaciones de este método.

L121

```
121 |         e.reasonCode = "TRANSPORT_DURATION";
```

En e guarda causa codificada. Toma el dato de "TRANSPORT_DURATION" después de las validaciones de este método.

L122

```
122 |         store.save(input, e, details);
```

Entrega resultado y candidatos al store, que decide bajo bloqueo si aún corresponde persistirlos. Solicita persistir la entidad en el contexto JPA.

L123

```
123 |         return;
```

Return.

L124

```
124 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L125 a L131

```
125 |       List<FleetClient.Candidate> resources =
126 |           fleet.candidates(
127 |               input.originLat(),
128 |               input.originLon(),
129 |               input.vehicleType(),
130 |               input.weight(),
131 |               input.mobility().equals("NON_ROLLABLE"));
```

Consulta hasta cinco recursos de Fleet según origen, tipo, peso y movilidad. Se calcula resources (resources) para reutilizar ese resultado en los pasos siguientes. compara contenido/identidad lógica, no referencias Java; Datos implicados: tipo CAR/SUV/PICKUP

L132

```
132 |       Map<UUID, Instant> freeSince = new HashMap<>();
```

Prepara antigüedad de disponibilidad para desempate y bandera de dependencia pendiente. Se calcula freeSince (freeSince) para reutilizar ese resultado en los pasos siguientes.

L133

```
133 |       boolean pending = false;
```

Se calcula pending (pending) para reutilizar ese resultado en los pasos siguientes. false

L134

```
134 |       for (var resource : resources) {
```

El bucle repite el bloque por cada elemento; el nombre a la izquierda de : es la variable local y a la derecha está la colección.

L135 a L147

```
135 |         if (resource.towTruckId() == null
136 |             || resource.operatorId() == null
137 |             || resource.operatorSubject() == null
138 |             || resource.capacityKg() == null
139 |             || resource.supportedTypes() == null
140 |             || resource.qualificationCategory() == null
141 |             || resource.qualificationValidFrom() == null
142 |             || resource.qualificationValidUntil() == null
143 |             || resource.availableSince() == null
144 |             || resource.latitude() == null
145 |             || resource.longitude() == null
146 |             || resource.locationUpdatedAt() == null)
147 |           throw new IllegalStateException("FLEET_INVALID_RESPONSE");
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. Datos implicados: UUID de grúa, UUID del perfil operador, sub del operador destinatario, tipos de vehículo admitidos, habilitación LIGHT_TOWING, inicio de habilitación, fin de habilitación que debe superar el servicio estimado, inicio de disponibilidad usado para desempate, latitud declarada, longitud declarada, instante servidor de última ubicación declarada; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L136: Datos implicados: UUID del perfil operador; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L137: Datos implicados: sub del operador destinatario; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L138: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L139: Datos implicados: tipos de vehículo admitidos; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L140: Datos implicados: habilitación LIGHT_TOWING; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L141: Datos implicados: inicio de habilitación; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L142: Datos implicados: fin de habilitación que debe superar el servicio estimado; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L143: Datos implicados: inicio de disponibilidad usado para desempate; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L144: Datos implicados: latitud declarada; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L145: Datos implicados: longitud declarada; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L146: Datos implicados: instante servidor de última ubicación declarada; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L148

```
148 |         CandidateEvaluation c = new CandidateEvaluation();
```

Construye CandidateEvaluation en memoria y conserva la referencia en c. Snapshot por candidato de compatibilidad, elegibilidad, recorridos y posición en el ranking. Referencia UUID remotos sin FK hacia Fleet. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L149

```
149 |         c.towTruckId = resource.towTruckId();
```

En c guarda UUID de grúa. El valor resource.towTruckId() relaciona esta fila con el recurso correspondiente; los UUID remotos se conservan sin FK entre microservicios. Datos implicados: UUID de grúa.

L150

```
150 |         c.operatorId = resource.operatorId();
```

En c guarda UUID del perfil operador. El valor resource.operatorId() relaciona esta fila con el recurso correspondiente; los UUID remotos se conservan sin FK entre microservicios. Datos implicados: UUID del perfil operador.

L151

```
151 |         c.operatorSubject = resource.operatorSubject();
```

En c guarda sub del operador destinatario. Identifica a la persona por sub de Keycloak, no por un nombre editable ni una contraseña. Datos implicados: sub del operador destinatario.

L152

```
152 |         c.truckCapacityKg = resource.capacityKg();
```

En c guarda snapshot de capacidad. Toma el dato de resource.capacityKg() después de las validaciones de este método.

L153

```
153 |         c.qualificationCategory = resource.qualificationCategory();
```

En c guarda habilitación LIGHT_TOWING. Toma el dato de resource.qualificationCategory() después de las validaciones de este método. Datos implicados: habilitación LIGHT_TOWING.

L154

```
154 |         c.qualificationValidUntil = resource.qualificationValidUntil();
```

En c guarda fin de habilitación que debe superar el servicio estimado. Toma el dato de resource.qualificationValidUntil() después de las validaciones de este método. Datos implicados: fin de habilitación que debe superar el servicio estimado.

L155

```
155 |         c.truckLatitude = resource.latitude();
```

En c guarda snapshot de latitud de grúa. Toma el dato de resource.latitude() después de las validaciones de este método. Datos implicados: latitud declarada.

L156

```
156 |         c.truckLongitude = resource.longitude();
```

En c guarda snapshot de longitud de grúa. Toma el dato de resource.longitude() después de las validaciones de este método. Datos implicados: longitud declarada.

L157

```
157 |         c.truckLocationUpdatedAt = resource.locationUpdatedAt();
```

En c guarda fecha de posición usada al evaluar. Toma el dato de resource.locationUpdatedAt() después de las validaciones de este método. Datos implicados: instante servidor de última ubicación declarada.

L158 a L161

```
158 |         c.compatible =
159 |             resource.capacityKg().compareTo(input.weight()) >= 0
160 |                 && resource.supportedTypes().contains(input.vehicleType())
161 |                 && (!input.mobility().equals("NON_ROLLABLE") || resource.supportsNonRollable());
```

En c guarda compatibilidad de capacidad, tipo y movilidad. Toma el dato de resource.capacityKg().compareTo(input.weight()) >= 0 && resource.supportedTypes().contains(input.vehicleType()) && (!input.mobility().equals("NON_ROLLABLE") || resource.supportsNonRollable()) después de las validaciones de este método. compara decimales: negativo menor, cero igual, positivo mayor; compara contenido/identidad lógica, no referencias Java; verifica pertenencia al conjunto o lista; Datos implicados: tipos de vehículo admitidos, tipo CAR/SUV/PICKUP, capacidad para atender vehículos que no ruedan; && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L160: verifica pertenencia al conjunto o lista; Datos implicados: tipos de vehículo admitidos, tipo CAR/SUV/PICKUP; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L161: compara contenido/identidad lógica, no referencias Java; Datos implicados: capacidad para atender vehículos que no ruedan; && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L162

```
162 |         c.transportDistanceM = e.transportDistanceM;
```

En c guarda distancia vial origen-destino en metros. Toma el dato de e.transportDistanceM después de las validaciones de este método. Datos implicados: distancia vial origen-destino en metros.

L163

```
163 |         c.transportDurationSeconds = e.transportDurationSeconds;
```

En c guarda duración vial original del traslado. Toma el dato de e.transportDurationSeconds después de las validaciones de este método. Datos implicados: duración vial original del traslado.

L164

```
164 |         c.adjustedTransportSeconds = e.adjustedTransportSeconds;
```

En c guarda duración ajustada y redondeada hacia arriba. Toma el dato de e.adjustedTransportSeconds después de las validaciones de este método. Datos implicados: duración ajustada y redondeada hacia arriba.

L165

```
165 |         c.transportRouteObtainedAt = e.routeObtainedAt;
```

En c guarda fecha de ruta de traslado. Toma el dato de e.routeObtainedAt después de las validaciones de este método. Datos implicados: fecha del recorrido.

L166

```
166 |         details.add(c);
```

Conserva la evaluación individual del candidato para devolver evidencia, incluso cuando ese candidato no queda seleccionado.

L167

```
167 |         freeSince.put(c.towTruckId, resource.availableSince());
```

Añade o reemplaza una propiedad en el objeto JSON o mapa; Datos implicados: UUID de grúa, inicio de disponibilidad usado para desempate.

L168

```
168 |         Instant validation = clock.instant();
```

Toma instante fresco porque las consultas HTTP previas pueden haber consumido tiempo. Se calcula validation (validation) para reutilizar ese resultado en los pasos siguientes. lee el instante del reloj inyectado

L169

```
169 |         if (!c.compatible) {
```

Esta condición decide si se entra en la rama siguiente. Datos implicados: compatibilidad de capacidad, tipo y movilidad; ! invierte el booleano de la expresión que sigue.

L170

```
170 |           c.reasonCode = "INCOMPATIBLE";
```

En c guarda causa codificada. Toma el dato de "INCOMPATIBLE" después de las validaciones de este método.

L171

```
171 |           continue;
```

Continue.

L172

```
172 |         }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L173 a L176

```
173 |         if (!inCoverage(c.truckLatitude, c.truckLongitude, rules)
174 |             || c.truckLocationUpdatedAt.isAfter(validation)
175 |             || c.truckLocationUpdatedAt.isBefore(
176 |                 validation.minusSeconds(value(rules, "fleet", "max-location-age-seconds")))) {
```

Esta condición decide si se entra en la rama siguiente. calcula un instante anterior restando segundos; compara si el instante es estrictamente posterior al otro; compara si el instante es estrictamente anterior al otro; Datos implicados: snapshot de latitud de grúa, snapshot de longitud de grúa, fecha de posición usada al evaluar; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L174: compara si el instante es estrictamente posterior al otro; Datos implicados: fecha de posición usada al evaluar; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L175: compara si el instante es estrictamente anterior al otro; Datos implicados: fecha de posición usada al evaluar; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L177

```
177 |           c.reasonCode = "LOCATION_INVALID";
```

En c guarda causa codificada. Toma el dato de "LOCATION_INVALID" después de las validaciones de este método.

L178

```
178 |           continue;
```

Continue.

L179

```
179 |         }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L180

```
180 |         try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L181

```
181 |           Point truck = new Point(c.truckLatitude, c.truckLongitude);
```

Construye Point en memoria y conserva la referencia en truck. Los argumentos del constructor determinan los datos iniciales de este objeto. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L182 a L185

```
182 |           Route approach =
183 |               same(truck, origin)
184 |                   ? new Route(BigDecimal.ZERO, BigDecimal.ZERO, transport.obtainedAt())
185 |                   : routes.route(truck, origin, e.configurationRevision, age, snap);
```

Si la grúa ya está exactamente en origen, aproximación cero; en caso contrario pide ruta vial. No inventa una distancia para reemplazar ORS. Se calcula approach (approach) para reutilizar ese resultado en los pasos siguientes. Datos implicados: versión de reglas; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario

L185: Datos implicados: versión de reglas.

L186

```
186 |           c.approachDistanceM = approach.distanceM();
```

En c guarda distancia vial grúa-recogida. Toma el dato de approach.distanceM() después de las validaciones de este método.

L187

```
187 |           c.approachDurationSeconds = approach.durationSeconds();
```

En c guarda duración original de aproximación. Toma el dato de approach.durationSeconds() después de las validaciones de este método.

L188

```
188 |           c.adjustedApproachSeconds = adjusted(approach.durationSeconds(), factor);
```

En c guarda aproximación ajustada. Toma el dato de adjusted(approach.durationSeconds(), factor) después de las validaciones de este método.

L189

```
189 |           c.approachRouteObtainedAt = approach.obtainedAt();
```

En c guarda fecha de ruta de aproximación. Toma el dato de approach.obtainedAt() después de las validaciones de este método.

L190 a L191

```
190 |           int dispatch = value(rules, "execution", "dispatch-deadline-seconds"),
191 |               arrivalMargin = value(rules, "routing", "arrival-margin-seconds");
```

Lee plazo de despacho y margen de llegada para evaluar el presupuesto total. Se calcula dispatch (dispatch) para reutilizar ese resultado en los pasos siguientes.

L192 a L196

```
192 |           if (c.approachDistanceM.compareTo(
193 |                       BigDecimal.valueOf(value(rules, "routing", "max-approach-meters")))
194 |                   > 0
195 |               || (long) dispatch + c.adjustedApproachSeconds + arrivalMargin
196 |                   > e.maximumArrivalSeconds) {
```

Esta condición decide si se entra en la rama siguiente. compara decimales: negativo menor, cero igual, positivo mayor; Datos implicados: distancia vial grúa-recogida, aproximación ajustada, presupuesto máximo hasta llegada; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L195: Datos implicados: aproximación ajustada; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L197

```
197 |             c.reasonCode = "APPROACH_LIMIT";
```

En c guarda causa codificada. Toma el dato de "APPROACH_LIMIT" después de las validaciones de este método.

L198

```
198 |             continue;
```

Continue.

L199

```
199 |           }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L200

```
200 |           validation = clock.instant();
```

Lee el instante del reloj inyectado.

L201 a L204

```
201 |           long remaining =
202 |               Math.min(
203 |                   value(rules, "fleet", "hold-seconds"),
204 |                   Duration.between(validation, input.deadline()).getSeconds());
```

Se calcula remaining (remaining) para reutilizar ese resultado en los pasos siguientes. elige el menor valor para respetar el límite más restrictivo

L205

```
205 |           if (remaining < value(rules, "search", "minimum-offer-window-seconds")) {
```

Esta condición decide si se entra en la rama siguiente. .

L206

```
206 |             c.reasonCode = "SEARCH_WINDOW_TOO_SHORT";
```

En c guarda causa codificada. Toma el dato de "SEARCH_WINDOW_TOO_SHORT" después de las validaciones de este método.

L207

```
207 |             continue;
```

Continue.

L208

```
208 |           }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L209 a L219

```
209 |           c.estimatedServiceEndAt =
210 |               validation
211 |                   .plusSeconds(
212 |                       remaining
213 |                           + dispatch
214 |                           + c.adjustedApproachSeconds
215 |                           + e.loadingSeconds
216 |                           + e.adjustedTransportSeconds
217 |                           + e.unloadingSeconds
218 |                           + e.operationalMarginSeconds)
219 |                   .truncatedTo(java.time.temporal.ChronoUnit.MICROS);
```

En c guarda fin planificado para controlar habilitación. Toma el dato de validation .plusSeconds(remaining + dispatch + c.adjustedApproachSeconds + e.loadingSeconds + e.adjustedTransportSeconds + e.unloadingSeconds + e.operationalMarginSeconds) .truncatedTo(java.time.temporal.ChronoUnit.MICROS) después de las validaciones de este método. calcula un instante futuro sumando segundos, sin modificar el instante original; Datos implicados: aproximación ajustada, carga prevista, duración ajustada y redondeada hacia arriba, descarga prevista, contingencia de planificación.

L211: calcula un instante futuro sumando segundos, sin modificar el instante original.

L220 a L222

```
220 |           if (!"LIGHT_TOWING".equals(c.qualificationCategory)
221 |               || resource.qualificationValidFrom().isAfter(validation)
222 |               || !c.qualificationValidUntil.isAfter(c.estimatedServiceEndAt)) {
```

Esta condición decide si se entra en la rama siguiente. compara si el instante es estrictamente posterior al otro; compara contenido/identidad lógica, no referencias Java; Datos implicados: habilitación LIGHT_TOWING, inicio de habilitación, fin de habilitación que debe superar el servicio estimado, fin planificado para controlar habilitación; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L221: compara si el instante es estrictamente posterior al otro; Datos implicados: inicio de habilitación; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L222: compara si el instante es estrictamente posterior al otro; Datos implicados: fin de habilitación que debe superar el servicio estimado, fin planificado para controlar habilitación; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L223

```
223 |             c.reasonCode = "QUALIFICATION_EXPIRES";
```

En c guarda causa codificada. Toma el dato de "QUALIFICATION_EXPIRES" después de las validaciones de este método.

L224

```
224 |             continue;
```

Continue.

L225

```
225 |           }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L226 a L229

```
226 |           if (c.truckLocationUpdatedAt.isBefore(
227 |                   validation.minusSeconds(value(rules, "fleet", "max-location-age-seconds")))
228 |               || c.approachRouteObtainedAt.isBefore(validation.minusSeconds(age))
229 |               || c.transportRouteObtainedAt.isBefore(validation.minusSeconds(age))) {
```

Esta condición decide si se entra en la rama siguiente. calcula un instante anterior restando segundos; compara si el instante es estrictamente anterior al otro; Datos implicados: fecha de posición usada al evaluar, fecha de ruta de aproximación, fecha de ruta de traslado; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L228: calcula un instante anterior restando segundos; compara si el instante es estrictamente anterior al otro; Datos implicados: fecha de ruta de aproximación; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L229: calcula un instante anterior restando segundos; compara si el instante es estrictamente anterior al otro; Datos implicados: fecha de ruta de traslado; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L230

```
230 |             c.reasonCode = "STALE_INPUT";
```

En c guarda causa codificada. Toma el dato de "STALE_INPUT" después de las validaciones de este método.

L231

```
231 |             pending = true;
```

Marca que un fallo temporal dejó trabajo de evaluación pendiente. No debe interpretarse como ausencia definitiva de grúas aptas.

L232

```
232 |             continue;
```

Continue.

L233

```
233 |           }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L234

```
234 |           c.eligible = true;
```

En c guarda aptitud completa del candidato/compromiso. Toma el dato de true después de las validaciones de este método. true.

L235

```
235 |           c.reasonCode = "ELIGIBLE";
```

En c guarda causa codificada. Toma el dato de "ELIGIBLE" después de las validaciones de este método.

L236

```
236 |         } catch (RoutingFailure failure) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L237

```
237 |           c.reasonCode = failure.reason().name();
```

En c guarda causa codificada. Toma el dato de failure.reason().name() después de las validaciones de este método.

L238

```
238 |           pending |= !failure.domainRejection();
```

Un fallo de dominio invalida ese candidato; uno temporal señala que la evaluación debe mantenerse pendiente si no hay otro viable. ! invierte el booleano de la expresión que sigue.

L239

```
239 |         }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L240

```
240 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L241

```
241 |       Instant finished = clock.instant();
```

Captura fin del cálculo para volver a comprobar frescura de todos los candidatos. Se calcula finished (finished) para reutilizar ese resultado en los pasos siguientes. lee el instante del reloj inyectado

L242 a L247

```
242 |       for (var c : details)
243 |         if (c.eligible
244 |             && (c.truckLocationUpdatedAt.isBefore(
245 |                     finished.minusSeconds(value(rules, "fleet", "max-location-age-seconds")))
246 |                 || c.approachRouteObtainedAt.isBefore(finished.minusSeconds(age))
247 |                 || c.transportRouteObtainedAt.isBefore(finished.minusSeconds(age)))) {
```

Invalida evidencia que envejeció durante cálculo de otros candidatos antes de rankear. El bucle repite el bloque por cada elemento; el nombre a la izquierda de : es la variable local y a la derecha está la colección. Calcula un instante anterior restando segundos; compara si el instante es estrictamente anterior al otro; Datos implicados: aptitud completa del candidato/compromiso, fecha de posición usada al evaluar, fecha de ruta de aproximación, fecha de ruta de traslado; && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L244: compara si el instante es estrictamente anterior al otro; Datos implicados: fecha de posición usada al evaluar; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L246: calcula un instante anterior restando segundos; compara si el instante es estrictamente anterior al otro; Datos implicados: fecha de ruta de aproximación; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L247: calcula un instante anterior restando segundos; compara si el instante es estrictamente anterior al otro; Datos implicados: fecha de ruta de traslado; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L248

```
248 |           c.eligible = false;
```

En c guarda aptitud completa del candidato/compromiso. Toma el dato de false después de las validaciones de este método. false.

L249

```
249 |           c.reasonCode = "STALE_INPUT";
```

En c guarda causa codificada. Toma el dato de "STALE_INPUT" después de las validaciones de este método.

L250

```
250 |           pending = true;
```

Marca que un fallo temporal dejó trabajo de evaluación pendiente. No debe interpretarse como ausencia definitiva de grúas aptas.

L251

```
251 |         }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L252 a L259

```
252 |       List<CandidateEvaluation> ranked =
253 |           details.stream()
254 |               .filter(c -> c.eligible)
255 |               .sorted(
256 |                   Comparator.comparingInt((CandidateEvaluation c) -> c.adjustedApproachSeconds)
257 |                       .thenComparing(c -> freeSince.get(c.towTruckId))
258 |                       .thenComparing(c -> c.towTruckId.toString()))
259 |               .toList();
```

Ordena candidatos elegibles por menor aproximación ajustada, luego por mayor antigüedad de disponibilidad y finalmente por UUID para desempate determinista. Se calcula ranked (ranked) para reutilizar ese resultado en los pasos siguientes. inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente; retiene solo elementos que cumplen la condición; materializa el resultado de la secuencia en una lista; representa el valor como texto; en un nodo JSON produce JSON serializado; Datos implicados: aptitud completa del candidato/compromiso, aproximación ajustada, UUID de grúa

L254: retiene solo elementos que cumplen la condición; Datos implicados: aptitud completa del candidato/compromiso.

L257: Datos implicados: UUID de grúa.

L258: representa el valor como texto; en un nodo JSON produce JSON serializado; Datos implicados: UUID de grúa.

L259: materializa el resultado de la secuencia en una lista.

L260

```
260 |       for (int i = 0; i < ranked.size(); i++) ranked.get(i).rankingPosition = i + 1;
```

El bucle repite el bloque por cada elemento; el nombre a la izquierda de : es la variable local y a la derecha está la colección.

L261

```
261 |       e.result = !ranked.isEmpty() ? "VIABLE" : pending ? "PENDING_DEPENDENCY" : "SEARCHING";
```

En e guarda resultado de evaluación, distinto del estado de solicitud. Toma el dato de !ranked.isEmpty() ? "VIABLE" : pending ? "PENDING_DEPENDENCY" : "SEARCHING" después de las validaciones de este método. comprueba ausencia de elementos o valor; ! invierte el booleano de la expresión que sigue; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L262 a L267

```
262 |       e.reasonCode =
263 |           !ranked.isEmpty()
264 |               ? "VIABLE_CANDIDATE"
265 |               : pending
266 |                   ? "DEPENDENCY_UNAVAILABLE"
267 |                   : resources.isEmpty() ? "NO_AVAILABLE_RESOURCE" : "NO_VIABLE_CANDIDATE";
```

En e guarda causa codificada. Toma el dato de !ranked.isEmpty() ? "VIABLE_CANDIDATE" : pending ? "DEPENDENCY_UNAVAILABLE" : resources.isEmpty() ? "NO_AVAILABLE_RESOURCE" : "NO_VIABLE_CANDIDATE" después de las validaciones de este método. comprueba ausencia de elementos o valor; ! invierte el booleano de la expresión que sigue; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L267: comprueba ausencia de elementos o valor; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L268

```
268 |     } catch (RoutingFailure failure) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L269

```
269 |       e.result = failure.domainRejection() ? "REJECTED" : "PENDING_DEPENDENCY";
```

En e guarda resultado de evaluación, distinto del estado de solicitud. Toma el dato de failure.domainRejection() ? "REJECTED" : "PENDING_DEPENDENCY" después de las validaciones de este método. el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L270

```
270 |       e.reasonCode = failure.reason().name();
```

En e guarda causa codificada. Toma el dato de failure.reason().name() después de las validaciones de este método.

L271

```
271 |     } catch (Exception failure) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L272

```
272 |       e.result = "PENDING_DEPENDENCY";
```

En e guarda resultado de evaluación, distinto del estado de solicitud. Toma el dato de "PENDING_DEPENDENCY" después de las validaciones de este método.

L273

```
273 |       e.reasonCode = "FLEET_OR_EVALUATION_UNAVAILABLE";
```

En e guarda causa codificada. Toma el dato de "FLEET_OR_EVALUATION_UNAVAILABLE" después de las validaciones de este método.

L274

```
274 |       e.reasonDetail = failure.getClass().getSimpleName();
```

En e guarda detalle diagnóstico acotado. Toma el dato de failure.getClass().getSimpleName() después de las validaciones de este método.

L275

```
275 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L276

```
276 |     store.save(input, e, details);
```

Entrega resultado y candidatos al store, que decide bajo bloqueo si aún corresponde persistirlos. Solicita persistir la entidad en el contexto JPA.

L277

```
277 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L278

```
278 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 11, 19, 32, 36, 47, 51, 55: separadores sin instrucciones.

EvaluationStore (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/EvaluationStore.java`

Objetivo y funcionalidad

Carga un snapshot bajo bloqueo, cede una ventana de evaluación y persiste solo si versión/estado/plazo siguen siendo compatibles. Evita sobrescribir una cancelación concurrente.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
 8 | import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | @Service
```

Registra lógica de aplicación como bean. El escaneo de Spring crea este componente e inyecta sus dependencias; no hay que construirlo manualmente en el controlador.

L11

```
11 | public class EvaluationStore {
```

Declara clase EvaluationStore. Carga un snapshot bajo bloqueo, cede una ventana de evaluación y persiste solo si versión/estado/plazo siguen siendo compatibles. Evita sobrescribir una cancelación concurrente.

L12 a L25

```
12 |   public record Input(
13 |       UUID requestId,
14 |       long version,
15 |       Status status,
16 |       int attemptNumber,
17 |       Instant deadline,
18 |       String rules,
19 |       BigDecimal originLat,
20 |       BigDecimal originLon,
21 |       BigDecimal destinationLat,
22 |       BigDecimal destinationLon,
23 |       String vehicleType,
24 |       BigDecimal weight,
25 |       String mobility) {}
```

Declara record inmutable Input. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: requestId: UUID de solicitud; version: versión para control optimista de concurrencia; status: estado del objeto; su enum o CHECK define valores permitidos; vehicleType: tipo CAR/SUV/PICKUP. Carga un snapshot bajo bloqueo, cede una ventana de evaluación y persiste solo si versión/estado/plazo siguen siendo compatibles. Evita sobrescribir una cancelación concurrente. Tipos y datos: requestId = UUID de solicitud; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; version = versión para control optimista de concurrencia; long es entero de 64 bits para cantidades y duraciones mayores; attemptNumber = attemptNumber; int es entero de 32 bits para límites y contadores; deadline = deadline; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; rules = configuración o snapshot de reglas que rige esta operación; String es texto; no es por sí mismo una referencia de entidad; originLat = originLat; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; originLon = originLon; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; destinationLat = destinationLat; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; destinationLon = destinationLon; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; vehicleType = tipo CAR/SUV/PICKUP; String es texto; no es por sí mismo una referencia de entidad; weight = weight; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; mobility = mobility; String es texto; no es por sí mismo una referencia de entidad.

L13: Dato del contrato: requestId: UUID de solicitud.

L14: Dato del contrato: version: versión para control optimista de concurrencia.

L23: Dato del contrato: vehicleType: tipo CAR/SUV/PICKUP.

L27

```
27 |   private final RequestRepository requests;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L28

```
28 |   private final EvaluationRepository evaluations;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L29

```
29 |   private final CandidateEvaluationRepository candidates;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L30

```
30 |   private final HistoryRepository history;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L31

```
31 |   private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L33 a L38

```
33 |   public EvaluationStore(
34 |       RequestRepository requests,
35 |       EvaluationRepository evaluations,
36 |       CandidateEvaluationRepository candidates,
37 |       HistoryRepository history,
38 |       Clock clock) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L39

```
39 |     this.requests = requests;
```

En this guarda repositorio de solicitudes. Toma el dato de requests después de las validaciones de este método. repositorio de solicitudes.

L40

```
40 |     this.evaluations = evaluations;
```

En this guarda acceso a evaluaciones persistidas. Toma el dato de evaluations después de las validaciones de este método. acceso a evaluaciones persistidas.

L41

```
41 |     this.candidates = candidates;
```

En this guarda candidates. Toma el dato de candidates después de las validaciones de este método. candidates.

L42

```
42 |     this.history = history;
```

En this guarda historial persistente de cambios. Toma el dato de history después de las validaciones de este método. historial persistente de cambios.

L43

```
43 |     this.clock = clock;
```

En this guarda reloj usado para evaluar plazos. Toma el dato de clock después de las validaciones de este método. reloj usado para evaluar plazos.

L44

```
44 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L46

```
46 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L47

```
47 |   public Input load(UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L48

```
48 |     var maybe = requests.lockById(id);
```

Se calcula maybe (maybe) para reutilizar ese resultado en los pasos siguientes.

L49

```
49 |     if (maybe.isEmpty()) return null;
```

La rama condicional retorna sin trabajo si no aplica. Esta condición decide si se entra en la rama siguiente. comprueba ausencia de elementos o valor.

L50

```
50 |     var r = maybe.get();
```

Se calcula r (r) para reutilizar ese resultado en los pasos siguientes.

L51 a L55

```
51 |     if (!Set.of(Status.REQUESTED, Status.PENDING_EVALUATION, Status.SEARCHING_ALTERNATIVE)
52 |             .contains(r.status)
53 |         || r.resourceReleasePending
54 |         || r.evaluationLeaseUntil.isAfter(clock.instant())
55 |         || !r.searchDeadline.isAfter(clock.instant())) return null;
```

La rama condicional retorna sin trabajo si no aplica. Esta condición decide si se entra en la rama siguiente. lee el instante del reloj inyectado; compara si el instante es estrictamente posterior al otro; verifica pertenencia al conjunto o lista; crea un conjunto no modificable y sin duplicados; REQUESTED: alternativa del ciclo de vida definido en el enum; PENDING_EVALUATION: solicitud pendiente de evaluación; SEARCHING_ALTERNATIVE: alternativa del ciclo de vida definido en el enum; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos, liberación remota pendiente que impide reutilización, límite de trabajo de evaluación en curso, vencimiento del intento; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L52: verifica pertenencia al conjunto o lista; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos.

L53: Datos implicados: liberación remota pendiente que impide reutilización; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L54: lee el instante del reloj inyectado; compara si el instante es estrictamente posterior al otro; Datos implicados: límite de trabajo de evaluación en curso; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L55: lee el instante del reloj inyectado; compara si el instante es estrictamente posterior al otro; Datos implicados: vencimiento del intento; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L56

```
56 |     Instant lease = clock.instant().plusSeconds(120);
```

Se calcula lease (lease) para reutilizar ese resultado en los pasos siguientes. lee el instante del reloj inyectado; calcula un instante futuro sumando segundos, sin modificar el instante original

L57

```
57 |     r.evaluationLeaseUntil = lease;
```

En r guarda límite de trabajo de evaluación en curso. Toma el dato de lease después de las validaciones de este método. lease.

L58

```
58 |     r.nextEvaluationAt = lease.isBefore(r.searchDeadline) ? lease : r.searchDeadline;
```

En r guarda próximo trabajo de evaluación. Toma el dato de lease.isBefore(r.searchDeadline) ? lease : r.searchDeadline después de las validaciones de este método. compara si el instante es estrictamente anterior al otro; Datos implicados: vencimiento del intento; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L59

```
59 |     requests.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente.

L60 a L73

```
60 |     return new Input(
61 |         id,
62 |         r.version,
63 |         r.status,
64 |         r.searchAttemptNumber,
65 |         r.searchDeadline,
66 |         r.configurationSnapshot,
67 |         r.originLatitude,
68 |         r.originLongitude,
69 |         r.destinationLatitude,
70 |         r.destinationLongitude,
71 |         r.declaredVehicleType.name(),
72 |         r.declaredWeightKg,
73 |         r.mobilityCondition.name());
```

Devuelve snapshot inmutable con versión/estado y datos necesarios; HTTP se hará fuera del bloqueo local. Resultado que recibe el llamador: Datos implicados: versión para control optimista de concurrencia, estado del objeto; su enum o CHECK define valores permitidos, número de búsqueda de 1 a 3, vencimiento del intento, JSON de parámetros vigente al crear, latitud de recogida, longitud de recogida, latitud de entrega, longitud de entrega, copia del tipo de vehículo al solicitar, peso declarado en kg, condición ROLLABLE/NON_ROLLABLE.

L74

```
74 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L76

```
76 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L77

```
77 |   public boolean save(Input input, Evaluation result, List<CandidateEvaluation> details) {
```

Crea lista que conservará tanto candidatos aptos como descartados y sus motivos. Reingresa a transacción para aplicar solo una evaluación compatible con el snapshot original.

L78

```
78 |     var maybe = requests.lockById(input.requestId());
```

Se calcula maybe (maybe) para reutilizar ese resultado en los pasos siguientes. Datos implicados: UUID de solicitud

L79

```
79 |     if (maybe.isEmpty()) return false;
```

Esta condición decide si se entra en la rama siguiente. comprueba ausencia de elementos o valor.

L80

```
80 |     var r = maybe.get();
```

Se calcula r (r) para reutilizar ese resultado en los pasos siguientes.

L81 a L83

```
81 |     if (r.version != input.version()
82 |         || r.status != input.status()
83 |         || !r.searchDeadline.isAfter(clock.instant())) return false;
```

Descarta el resultado si hubo un cambio concurrente de versión/estado o venció la búsqueda. La evaluación externa no pisa una cancelación. Esta condición decide si se entra en la rama siguiente. lee el instante del reloj inyectado; compara si el instante es estrictamente posterior al otro; Datos implicados: versión para control optimista de concurrencia, estado del objeto; su enum o CHECK define valores permitidos, vencimiento del intento; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L82: Datos implicados: estado del objeto; su enum o CHECK define valores permitidos; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L83: lee el instante del reloj inyectado; compara si el instante es estrictamente posterior al otro; Datos implicados: vencimiento del intento; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L84

```
84 |     r.evaluationLeaseUntil = Instant.EPOCH;
```

En r guarda límite de trabajo de evaluación en curso. Toma el dato de Instant.EPOCH después de las validaciones de este método.

L85

```
85 |     result.request = r;
```

En result guarda request. Toma el dato de r después de las validaciones de este método. r.

L86

```
86 |     result.evaluatedAt = clock.instant().truncatedTo(java.time.temporal.ChronoUnit.MICROS);
```

En result guarda instante de evaluación. Marca el instante de esta operación para ordenar y auditar su evolución. lee el instante del reloj inyectado.

L87

```
87 |     var stored = evaluations.saveAndFlush(result);
```

Persiste y fuerza sincronización SQL para detectar restricciones antes de retornar; commit ocurre al terminar transacción. Se calcula stored (stored) para reutilizar ese resultado en los pasos siguientes. guarda y sincroniza SQL para detectar conflictos antes de retornar; no confirma por sí solo la transacción

L88

```
88 |     for (var c : details) {
```

El bucle repite el bloque por cada elemento; el nombre a la izquierda de : es la variable local y a la derecha está la colección.

L89

```
89 |       c.evaluation = stored;
```

En c guarda evaluation. Toma el dato de stored después de las validaciones de este método. stored.

L90

```
90 |       candidates.save(c);
```

Solicita persistir la entidad en el contexto JPA.

L91

```
91 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L92

```
92 |     candidates.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente.

L93

```
93 |     Status previous = r.status;
```

Se calcula previous (previous) para reutilizar ese resultado en los pasos siguientes. Datos implicados: estado del objeto; su enum o CHECK define valores permitidos

L94 a L97

```
94 |     r.status =
95 |         "REJECTED".equals(result.result)
96 |             ? (r.firstAssignedAt == null ? Status.REJECTED : Status.UNFULFILLED)
97 |             : Status.PENDING_EVALUATION;
```

En r guarda estado del objeto; su enum o CHECK define valores permitidos. rechazo registrado; se conserva para excluir al operador según la regla rechazo registrado; se conserva para excluir al operador según la regla solicitud cerrada sin poder prestar el servicio bajo sus plazos/cupos solicitud que espera evaluación de candidatos y rutas compara contenido/identidad lógica, no referencias Java; REJECTED: evaluación rechazada; UNFULFILLED: alternativa del ciclo de vida definido en el enum; PENDING_EVALUATION: solicitud pendiente de evaluación; Datos implicados: primera aceptación que impide cancelar incluso durante reemplazos; detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L96: REJECTED: evaluación rechazada; UNFULFILLED: alternativa del ciclo de vida definido en el enum; Datos implicados: primera aceptación que impide cancelar incluso durante reemplazos; detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L97: PENDING_EVALUATION: solicitud pendiente de evaluación.

L98

```
98 |     r.updatedAt = result.evaluatedAt;
```

En r guarda fecha de modificación. Marca el instante de esta operación para ordenar y auditar su evolución.

L99

```
99 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L100 a L105

```
100 |       int interval =
101 |           new com.fasterxml.jackson.databind.ObjectMapper()
102 |               .readTree(r.configurationSnapshot)
103 |               .path("search")
104 |               .path("evaluation-interval-seconds")
105 |               .asInt();
```

Construye com.fasterxml.jackson.databind.ObjectMapper en memoria y conserva la referencia en interval. conversor Jackson entre JSON y objetos/árboles Java Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L102: analiza texto JSON para consultar su estructura; Datos implicados: JSON de parámetros vigente al crear.

L103: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; Ruta JSON consultada: search.

L104: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; Ruta JSON consultada: evaluation-interval-seconds; Intervalo de reevaluación..

L105: convierte el nodo a entero.

L106

```
106 |       Instant next = result.evaluatedAt.plusSeconds(interval);
```

Se calcula next (next) para reutilizar ese resultado en los pasos siguientes. calcula un instante futuro sumando segundos, sin modificar el instante original

L107

```
107 |       r.nextEvaluationAt = next.isBefore(r.searchDeadline) ? next : r.searchDeadline;
```

En r guarda próximo trabajo de evaluación. Toma el dato de next.isBefore(r.searchDeadline) ? next : r.searchDeadline después de las validaciones de este método. compara si el instante es estrictamente anterior al otro; Datos implicados: vencimiento del intento; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L108

```
108 |     } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L109

```
109 |       throw new IllegalStateException("Invalid rules");
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L110

```
110 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L111

```
111 |     if (r.status == Status.REJECTED || r.status == Status.UNFULFILLED) {
```

Esta condición decide si se entra en la rama siguiente. REJECTED: evaluación rechazada; UNFULFILLED: alternativa del ciclo de vida definido en el enum; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L112

```
112 |       r.closedAt = result.evaluatedAt;
```

En r guarda fecha de cierre; null indica abierto. Toma el dato de result.evaluatedAt después de las validaciones de este método.

L113

```
113 |       r.closureReasonCode = result.reasonCode;
```

En r guarda causa codificada del cierre. Toma el dato de result.reasonCode después de las validaciones de este método.

L114

```
114 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L115

```
115 |     RequestStatusHistory h = new RequestStatusHistory();
```

Construye RequestStatusHistory en memoria y conserva la referencia en h. Auditoría de cambios de solicitud con actor, motivo, fecha, idempotencia y evidencia. Es persistencia de negocio, distinta de los logs de consola. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L116

```
116 |     h.request = r;
```

En h guarda request. Toma el dato de r después de las validaciones de este método. r.

L117

```
117 |     h.previousStatus = previous;
```

En h guarda estado previo. La transición efectiva proviene de previous y debe estar autorizada por las validaciones previas. previous.

L118

```
118 |     h.newStatus = r.status;
```

En h guarda estado resultante. La transición efectiva proviene de r.status y debe estar autorizada por las validaciones previas. Datos implicados: estado del objeto; su enum o CHECK define valores permitidos.

L119

```
119 |     h.actorType = "SYSTEM";
```

En h guarda tipo de actor SYSTEM/DRIVER/OPERATOR. Toma el dato de "SYSTEM" después de las validaciones de este método.

L120

```
120 |     h.reason = result.reasonCode;
```

En h guarda motivo legible. Toma el dato de result.reasonCode después de las validaciones de este método.

L121

```
121 |     h.details = "evaluationId=" + stored.id + " result=" + result.result;
```

En h guarda evidencia adicional. Toma el dato de "evaluationId=" + stored.id + " result=" + result.result después de las validaciones de este método.

L122

```
122 |     h.occurredAt = result.evaluatedAt;
```

En h guarda instante del evento. Toma el dato de result.evaluatedAt después de las validaciones de este método.

L123

```
123 |     history.saveAndFlush(h);
```

Persiste y fuerza sincronización SQL para detectar restricciones antes de retornar; commit ocurre al terminar transacción. Guarda y sincroniza SQL para detectar conflictos antes de retornar; no confirma por sí solo la transacción.

L124 a L136

```
124 |     org.springframework.transaction.support.TransactionSynchronizationManager
125 |         .registerSynchronization(
126 |             new org.springframework.transaction.support.TransactionSynchronization() {
127 |               @Override
128 |               public void afterCommit() {
129 |                 org.slf4j.LoggerFactory.getLogger(EvaluationStore.class)
130 |                     .info(
131 |                         "Evaluation completed requestId={} result={} reason={}",
132 |                         input.requestId(),
133 |                         result.result,
134 |                         result.reasonCode);
135 |               }
136 |             });
```

Registra un callback del ciclo transaccional; no ejecuta el log antes del commit. Implementa callback afterCommit para anunciar solo cambios confirmados. Se calcula requestId (UUID de solicitud) para reutilizar ese resultado en los pasos siguientes. Datos implicados: UUID de solicitud

L137

```
137 |     return true;
```

Entrega true al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: true.

L138

```
138 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L139

```
139 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 9, 26, 32, 45, 75: separadores sin instrucciones.

FleetClient (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/routing/FleetClient.java`

Objetivo y funcionalidad

Cliente HTTP técnico. Obtiene y reutiliza un token client_credentials; consulta candidatos y opera reservas. No propaga el token del conductor.

L1

```
1 | package com.auxiruta.assistance.routing;
```

Ubica el archivo en com.auxiruta.assistance.routing. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.beans.factory.annotation.Value;
```

Importa tipos: org.springframework.beans.factory.annotation.Value. Inyección de dependencias y propiedades de entorno. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.springframework.web.client.RestClient;
```

Importa tipos: org.springframework.web.client.RestClient. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | import org.springframework.http.*;
```

Importa tipos: org.springframework.http.*. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import org.springframework.http.client.JdkClientHttpRequestFactory;
```

Importa tipos: org.springframework.http.client.JdkClientHttpRequestFactory. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | import org.springframework.core.ParameterizedTypeReference;
```

Importa tipos: org.springframework.core.ParameterizedTypeReference. Tipo auxiliar usado por las expresiones del archivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12

```
12 | import org.springframework.util.LinkedMultiValueMap;
```

Importa tipos: org.springframework.util.LinkedMultiValueMap. Tipo auxiliar usado por las expresiones del archivo. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L13

```
13 | import com.fasterxml.jackson.databind.JsonNode;
```

Importa tipos: com.fasterxml.jackson.databind.JsonNode. Lectura/escritura JSON, nodos y conversión a DTO. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L15

```
15 | @Component
```

Registra componente inyectable.

L16

```
16 | public class FleetClient {
```

Declara clase FleetClient. Cliente HTTP técnico. Obtiene y reutiliza un token client_credentials; consulta candidatos y opera reservas. No propaga el token del conductor.

L17 a L31

```
17 |   public record Candidate(
18 |       UUID towTruckId,
19 |       UUID operatorId,
20 |       String operatorSubject,
21 |       BigDecimal capacityKg,
22 |       Set<String> supportedTypes,
23 |       boolean supportsNonRollable,
24 |       String qualificationCategory,
25 |       Instant qualificationValidFrom,
26 |       Instant qualificationValidUntil,
27 |       Instant availableSince,
28 |       BigDecimal latitude,
29 |       BigDecimal longitude,
30 |       Instant locationUpdatedAt,
31 |       double straightDistanceM) {}
```

Declara record inmutable Candidate. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: towTruckId: UUID de grúa; operatorId: UUID del perfil operador; operatorSubject: sub del operador destinatario; supportsNonRollable: capacidad para atender vehículos que no ruedan; qualificationCategory: habilitación LIGHT_TOWING; qualificationValidFrom: inicio de habilitación; qualificationValidUntil: fin de habilitación que debe superar el servicio estimado; availableSince: inicio de disponibilidad usado para desempate; latitude: latitud declarada; longitude: longitud declarada; locationUpdatedAt: instante servidor de última ubicación declarada. Cliente HTTP técnico. Obtiene y reutiliza un token client_credentials; consulta candidatos y opera reservas. No propaga el token del conductor. Tipos y datos: towTruckId = UUID de grúa; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; operatorId = UUID del perfil operador; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; operatorSubject = sub del operador destinatario; String es texto; no es por sí mismo una referencia de entidad; capacityKg = capacityKg; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; supportsNonRollable = capacidad para atender vehículos que no ruedan; boolean es verdadero/falso, sin null; qualificationCategory = habilitación LIGHT_TOWING; String es texto; no es por sí mismo una referencia de entidad; qualificationValidFrom = inicio de habilitación; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; qualificationValidUntil = fin de habilitación que debe superar el servicio estimado; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; availableSince = inicio de disponibilidad usado para desempate; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; latitude = latitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; longitude = longitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; locationUpdatedAt = instante servidor de última ubicación declarada; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L18: Dato del contrato: towTruckId: UUID de grúa.

L19: Dato del contrato: operatorId: UUID del perfil operador.

L20: Dato del contrato: operatorSubject: sub del operador destinatario.

L23: Dato del contrato: supportsNonRollable: capacidad para atender vehículos que no ruedan.

L24: Dato del contrato: qualificationCategory: habilitación LIGHT_TOWING.

L25: Dato del contrato: qualificationValidFrom: inicio de habilitación.

L26: Dato del contrato: qualificationValidUntil: fin de habilitación que debe superar el servicio estimado.

L27: Dato del contrato: availableSince: inicio de disponibilidad usado para desempate.

L28: Dato del contrato: latitude: latitud declarada.

L29: Dato del contrato: longitude: longitud declarada.

L30: Dato del contrato: locationUpdatedAt: instante servidor de última ubicación declarada.

L33

```
33 |   private final RestClient http;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L34

```
34 |   private final String base, tokenUrl, clientId, secret;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: base = base; String es texto; no es por sí mismo una referencia de entidad.

L35

```
35 |   private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L36

```
36 |   private String token;
```

Tipos y datos: token = token; String es texto; no es por sí mismo una referencia de entidad.

L37

```
37 |   private Instant expires = Instant.EPOCH;
```

Usa 1970-01-01 UTC como planificación inicialmente vencida. Se calcula expires (expires) para reutilizar ese resultado en los pasos siguientes. Tipos y datos: expires = expires; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L39 a L46

```
39 |   public FleetClient(
40 |       @Value("${FLEET_URL:http://localhost:8081}") String base,
41 |       @Value(
42 |
     "${KEYCLOAK_TOKEN_URI:http://localhost:8080/realms/auxiruta/protocol/openid-connect/token}")
43 |           String tokenUrl,
44 |       @Value("${KEYCLOAK_CLIENT_ID:assistance-service}") String clientId,
45 |       @Value("${KEYCLOAK_CLIENT_SECRET:}") String secret,
46 |       Clock clock) {
```

Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición. Tipos y datos: base = base; String es texto; no es por sí mismo una referencia de entidad; tokenUrl = tokenUrl; String es texto; no es por sí mismo una referencia de entidad; clientId = clientId; String es texto; no es por sí mismo una referencia de entidad; secret = secret; String es texto; no es por sí mismo una referencia de entidad; clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L47

```
47 |     this.base = base;
```

En this guarda base. Toma el dato de base después de las validaciones de este método. base.

L48

```
48 |     this.tokenUrl = tokenUrl;
```

En this guarda tokenUrl. Toma el dato de tokenUrl después de las validaciones de este método. tokenUrl.

L49

```
49 |     this.clientId = clientId;
```

En this guarda clientId. Toma el dato de clientId después de las validaciones de este método. clientId.

L50

```
50 |     this.secret = secret;
```

En this guarda secret. Toma el dato de secret después de las validaciones de este método. secret.

L51

```
51 |     this.clock = clock;
```

En this guarda reloj usado para evaluar plazos. Toma el dato de clock después de las validaciones de este método. reloj usado para evaluar plazos.

L52 a L57

```
52 |     var factory =
53 |         new JdkClientHttpRequestFactory(
54 |             java.net.http.HttpClient.newBuilder()
55 |                 .connectTimeout(Duration.ofSeconds(2))
56 |                 .followRedirects(java.net.http.HttpClient.Redirect.NEVER)
57 |                 .build());
```

Construye JdkClientHttpRequestFactory en memoria y conserva la referencia en factory. Los argumentos del constructor determinan los datos iniciales de este objeto. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L58

```
58 |     factory.setReadTimeout(Duration.ofSeconds(5));
```

Limita espera de respuesta a cinco segundos.

L59

```
59 |     http = RestClient.builder().requestFactory(factory).build();
```

Construye el objeto final a partir del builder configurado en líneas previas.

L60

```
60 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L62

```
62 |   private synchronized String token() {
```

Reutiliza el token técnico mientras conserve margen de diez segundos; de lo contrario pide uno nuevo a Keycloak.

L63

```
63 |     if (token != null && expires.isAfter(clock.instant().plusSeconds(10))) return token;
```

Esta condición decide si se entra en la rama siguiente. lee el instante del reloj inyectado; calcula un instante futuro sumando segundos, sin modificar el instante original; compara si el instante es estrictamente posterior al otro; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato.

L64

```
64 |     var form = new LinkedMultiValueMap<String, String>();
```

Se calcula form (form) para reutilizar ese resultado en los pasos siguientes.

L65

```
65 |     form.add("grant_type", "client_credentials");
```

Construye formulario OAuth con cliente técnico y secreto, no con usuario/contraseña del conductor.

L66

```
66 |     form.add("client_id", clientId);
```

Añade al formulario OAuth el identificador del cliente técnico de Assistance.

L67

```
67 |     form.add("client_secret", secret);
```

Añade la credencial confidencial al formulario de emisión de token. Se envía a Keycloak, no se registra en logs ni se entrega al usuario.

L68 a L74

```
68 |     JsonNode response =
69 |         http.post()
70 |             .uri(tokenUrl)
71 |             .contentType(MediaType.APPLICATION_FORM_URLENCODED)
72 |             .body(form)
73 |             .retrieve()
74 |             .body(JsonNode.class);
```

Prepara HTTP al endpoint indicado; retrieve ejecuta y deserializa o lanza ante error. Se calcula response (response) para reutilizar ese resultado en los pasos siguientes.

L75 a L76

```
75 |     if (response == null || response.path("access_token").asText().isBlank())
76 |       throw new IllegalStateException("FLEET_AUTH_UNAVAILABLE");
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a texto; comprueba texto vacío o solo espacios; Ruta JSON consultada: access_token; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L77

```
77 |     token = response.path("access_token").asText();
```

Guarda token en memoria y calcula vencimiento usando expires_in; no lo persiste en tablas. Navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a texto; Ruta JSON consultada: access_token.

L78

```
78 |     expires = clock.instant().plusSeconds(response.path("expires_in").asLong(0));
```

Lee el instante del reloj inyectado; navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; calcula un instante futuro sumando segundos, sin modificar el instante original; Ruta JSON consultada: expires_in.

L79

```
79 |     return token;
```

Entrega token al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: token.

L80

```
80 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L82 a L94

```
 82 |   public record Eligibility(
 83 |       UUID assignmentAttemptId,
 84 |       UUID requestId,
 85 |       UUID towTruckId,
 86 |       UUID operatorId,
 87 |       String operatorSubject,
 88 |       boolean eligible,
 89 |       String reason,
 90 |       BigDecimal latitude,
 91 |       BigDecimal longitude,
 92 |       Instant locationUpdatedAt,
 93 |       Instant qualificationValidUntil,
 94 |       Instant checkedAt) {}
```

Declara record inmutable Eligibility. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: requestId: UUID de solicitud; towTruckId: UUID de grúa; operatorId: UUID del perfil operador; operatorSubject: sub del operador destinatario; eligible: aptitud completa del candidato/compromiso; reason: motivo legible; latitude: latitud declarada; longitude: longitud declarada; locationUpdatedAt: instante servidor de última ubicación declarada; qualificationValidUntil: fin de habilitación que debe superar el servicio estimado. Cliente HTTP técnico. Obtiene y reutiliza un token client_credentials; consulta candidatos y opera reservas. No propaga el token del conductor. Tipos y datos: assignmentAttemptId = assignmentAttemptId; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; requestId = UUID de solicitud; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; towTruckId = UUID de grúa; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; operatorId = UUID del perfil operador; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; operatorSubject = sub del operador destinatario; String es texto; no es por sí mismo una referencia de entidad; eligible = aptitud completa del candidato/compromiso; boolean es verdadero/falso, sin null; reason = motivo legible; String es texto; no es por sí mismo una referencia de entidad; latitude = latitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; longitude = longitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; locationUpdatedAt = instante servidor de última ubicación declarada; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; qualificationValidUntil = fin de habilitación que debe superar el servicio estimado; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; checkedAt = checkedAt; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L84: Dato del contrato: requestId: UUID de solicitud.

L85: Dato del contrato: towTruckId: UUID de grúa.

L86: Dato del contrato: operatorId: UUID del perfil operador.

L87: Dato del contrato: operatorSubject: sub del operador destinatario.

L88: Dato del contrato: eligible: aptitud completa del candidato/compromiso.

L89: Dato del contrato: reason: motivo legible.

L90: Dato del contrato: latitude: latitud declarada.

L91: Dato del contrato: longitude: longitud declarada.

L92: Dato del contrato: locationUpdatedAt: instante servidor de última ubicación declarada.

L93: Dato del contrato: qualificationValidUntil: fin de habilitación que debe superar el servicio estimado.

L96

```
 96 |   public Eligibility eligibility(UUID id) {
```

Consulta si un compromiso confirmado conserva aptitud; no aplica caducidad de ubicación de una propuesta nueva. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L97 a L101

```
 97 |     return http.get()
 98 |         .uri(base + "/internal/reservations/" + id + "/eligibility")
 99 |         .header(HttpHeaders.AUTHORIZATION, "Bearer " + token())
100 |         .retrieve()
101 |         .body(Eligibility.class);
```

Prepara HTTP al endpoint indicado; retrieve ejecuta y deserializa o lanza ante error.

L102

```
102 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L104

```
104 |   public java.util.List<JsonNode> statuses(java.util.List<UUID> ids) {
```

Consulta en lote el estado de varias reservas. Reduce llamadas al reconciliar muchas solicitudes y permite comparar el compromiso local con el real en Fleet.

L105 a L112

```
105 |     return http.post()
106 |         .uri(base + "/internal/reservations/status-queries")
107 |         .header(HttpHeaders.AUTHORIZATION, "Bearer " + token())
108 |         .contentType(MediaType.APPLICATION_JSON)
109 |         .body(java.util.Map.of("assignmentAttemptIds", ids))
110 |         .retrieve()
111 |         .body(
112 |             new org.springframework.core.ParameterizedTypeReference<java.util.List<JsonNode>>()
      {});
```

Prepara HTTP al endpoint indicado; retrieve ejecuta y deserializa o lanza ante error.

L113

```
113 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L115

```
115 |   public JsonNode reservation(UUID id, String kind, String payload) {
```

Campos: id: identificador UUID del registro; payload: contenido normalizado para reintentos. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; kind = kind; String es texto; no es por sí mismo una referencia de entidad; payload = contenido normalizado para reintentos; String es texto; no es por sí mismo una referencia de entidad.

L116

```
116 |     if (kind.equals("CONFIRM")) {
```

Consulta estado remoto previo; si ya no está HELD devuelve ese estado durable. Esta condición decide si se entra en la rama siguiente. compara contenido/identidad lógica, no referencias Java.

L117 a L122

```
117 |       JsonNode existing =
118 |           http.get()
119 |               .uri(base + "/internal/reservations/" + id)
120 |               .header(HttpHeaders.AUTHORIZATION, "Bearer " + token())
121 |               .retrieve()
122 |               .body(JsonNode.class);
```

Prepara HTTP al endpoint indicado; retrieve ejecuta y deserializa o lanza ante error. Se calcula existing (existing) para reutilizar ese resultado en los pasos siguientes.

L123 a L125

```
123 |       if (existing != null
124 |           && id.toString().equals(existing.path("assignmentAttemptId").asText())
125 |           && !"HELD".equals(existing.path("status").asText())) return existing;
```

Devuelve una ruta cacheada solo si su fecha no es futura y su antigüedad no supera el máximo solicitado. Evita consumir cuota en ese caso. Esta condición decide si se entra en la rama siguiente. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a texto; compara contenido/identidad lógica, no referencias Java; representa el valor como texto; en un nodo JSON produce JSON serializado; Ruta JSON consultada: assignmentAttemptId.status; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato.

L124: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a texto; compara contenido/identidad lógica, no referencias Java; representa el valor como texto; en un nodo JSON produce JSON serializado; Ruta JSON

consultada: assignmentAttemptId; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L125: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a texto; compara contenido/identidad lógica, no referencias Java; Ruta JSON consultada: status; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L126

```
126 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L127 a L135

```
127 |     var request =
128 |         kind.equals("RESERVE")
129 |             ? http.put().uri(base + "/internal/reservations/" + id)
130 |             : http.post()
131 |                 .uri(
132 |                     base
133 |                         + "/internal/reservations/"
134 |                         + id
135 |                         + (kind.equals("CONFIRM") ? "/confirmation" : "/release"));
```

Prepara HTTP al endpoint indicado; retrieve ejecuta y deserializa o lanza ante error. Se calcula request (request) para reutilizar ese resultado en los pasos siguientes. compara contenido/identidad lógica, no referencias Java; añade o reemplaza una propiedad en el objeto JSON o mapa; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario

L129: añade o reemplaza una propiedad en el objeto JSON o mapa.

L136

```
136 |     request.header(HttpHeaders.AUTHORIZATION, "Bearer " + token());
```

Añade Bearer técnico a la operación de reserva.

L137

```
137 |     if (kind.equals("RESERVE")) request.contentType(MediaType.APPLICATION_JSON).body(payload);
```

Esta condición decide si se entra en la rama siguiente. compara contenido/identidad lógica, no referencias Java.

L138

```
138 |     JsonNode result = request.retrieve().body(JsonNode.class);
```

Ejecuta HTTP y deserializa respuesta; un estado HTTP de error lanza excepción. Se calcula result (resultado de evaluación, distinto del estado de solicitud) para reutilizar ese resultado en los pasos siguientes.

L139 a L140

```
139 |     if (result == null || !id.toString().equals(result.path("assignmentAttemptId").asText()))
140 |       throw new IllegalStateException("FLEET_INVALID_RESERVATION_RESPONSE");
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a texto; compara contenido/identidad lógica, no referencias Java; representa el valor como texto; en un nodo JSON produce JSON serializado; Ruta JSON consultada: assignmentAttemptId; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue; detecta un dato ausente.

L141

```
141 |     return result;
```

Entrega result al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: resultado de evaluación, distinto del estado de solicitud.

L142

```
142 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L144 a L149

```
144 |   public List<Candidate> candidates(
145 |       BigDecimal latitude,
146 |       BigDecimal longitude,
147 |       String type,
148 |       BigDecimal weight,
149 |       boolean nonRollable) {
```

Campos: latitude: latitud declarada; longitude: longitud declarada. Tipos y datos: latitude = latitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; longitude = longitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; type = type; String es texto; no es por sí mismo una referencia de entidad; weight = weight; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; nonRollable = nonRollable; boolean es verdadero/falso, sin null.

L145: Dato del contrato: latitude: latitud declarada.

L146: Dato del contrato: longitude: longitud declarada.

L150

```
150 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L151 a L160

```
151 |       var uri =
152 |           org.springframework.web.util.UriComponentsBuilder.fromUriString(
153 |                   base + "/internal/fleet/candidates")
154 |               .queryParam("latitude", latitude)
155 |               .queryParam("longitude", longitude)
156 |               .queryParam("vehicleType", type)
157 |               .queryParam("weightKg", weight)
158 |               .queryParam("nonRollable", nonRollable)
159 |               .build()
160 |               .toUri();
```

Construye URL codificando parámetros de búsqueda, sin concatenar datos sin escape. Se calcula uri (uri) para reutilizar ese resultado en los pasos siguientes.

L161 a L166

```
161 |       List<Candidate> result =
162 |           http.get()
163 |               .uri(uri)
164 |               .header(HttpHeaders.AUTHORIZATION, "Bearer " + token())
165 |               .retrieve()
166 |               .body(new ParameterizedTypeReference<List<Candidate>>() {});
```

Prepara HTTP al endpoint indicado; retrieve ejecuta y deserializa o lanza ante error. Se calcula result (resultado de evaluación, distinto del estado de solicitud) para reutilizar ese resultado en los pasos siguientes.

L167 a L168

```
167 |       if (result == null || result.size() > 5)
168 |         throw new IllegalStateException("FLEET_INVALID_RESPONSE");
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L169

```
169 |       return result;
```

Entrega result al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: resultado de evaluación, distinto del estado de solicitud.

L170

```
170 |     } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L171

```
171 |       throw new IllegalStateException("FLEET_UNAVAILABLE");
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L172

```
172 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L173

```
173 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L174

```
174 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 14, 32, 38, 61, 81, 95, 103, 114, 143: separadores sin instrucciones.

OpenRouteServiceClient (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/routing/OpenRouteServiceClient.java`

Objetivo y funcionalidad

Adaptador HTTP ORS con timeouts, presupuesto persistente, caché por coordenadas/revisión y control del ajuste a red vial. Maneja 429, cuota diaria y respuesta inválida sin reintento inmediato.

L1

```
1 | package com.auxiruta.assistance.routing;
```

Ubica el archivo en com.auxiruta.assistance.routing. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.format.DateTimeFormatter;
```

Importa tipos: java.time.format.DateTimeFormatter. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import java.util.concurrent.ConcurrentHashMap;
```

Importa tipos: java.util.concurrent.ConcurrentHashMap. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
 8 | import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
 9 | import org.springframework.beans.factory.annotation.Value;
```

Importa tipos: org.springframework.beans.factory.annotation.Value. Inyección de dependencias y propiedades de entorno. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import org.springframework.web.client.*;
```

Importa tipos: org.springframework.web.client.*. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | import org.springframework.http.*;
```

Importa tipos: org.springframework.http.*. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12

```
12 | import org.springframework.http.client.JdkClientHttpRequestFactory;
```

Importa tipos: org.springframework.http.client.JdkClientHttpRequestFactory. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L13

```
13 | import com.fasterxml.jackson.databind.*;
```

Importa tipos: com.fasterxml.jackson.databind.*. Lectura/escritura JSON, nodos y conversión a DTO. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L14

```
14 | import com.auxiruta.commons.GeoMath;
```

Importa tipos: com.auxiruta.commons.GeoMath. Utilidades compartidas de seguridad, errores, reloj o geografía. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L15

```
15 | import static com.auxiruta.assistance.routing.RoutingFailure.Reason.*;
```

Importa miembros estáticos: com.auxiruta.assistance.routing.RoutingFailure.Reason.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L17

```
17 | @Component
```

Registra componente inyectable.

L18

```
18 | public class OpenRouteServiceClient implements RouteProvider {
```

Declara clase OpenRouteServiceClient. Adaptador HTTP ORS con timeouts, presupuesto persistente, caché por coordenadas/revisión y control del ajuste a red vial. Maneja 429, cuota diaria y respuesta inválida sin reintento inmediato.

L19

```
19 |   private final RestClient http;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L20

```
20 |   private final String key;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: key = key; String es texto; no es por sí mismo una referencia de entidad.

L21

```
21 |   private final RoutingBudget budget;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L22

```
22 |   private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L23

```
23 |   private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L24

```
24 |   private final Map<String, Route> cache = new ConcurrentHashMap<>();
```

Conserva dependencias/configuración; final impide reasignar referencias. Se calcula cache (cache) para reutilizar ese resultado en los pasos siguientes.

L26 a L31

```
26 |   public OpenRouteServiceClient(
27 |       @Value("${ors.base-url:https://api.heigit.org/openrouteservice/v2}") String url,
28 |       @Value("${ors.api-key:}") String key,
29 |       RoutingBudget budget,
30 |       Clock clock,
31 |       ObjectMapper mapper) {
```

Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición. Tipos y datos: url = url; String es texto; no es por sí mismo una referencia de entidad; key = key; String es texto; no es por sí mismo una referencia de entidad; clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos; mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L32

```
32 |     this.key = key;
```

En this guarda key. Toma el dato de key después de las validaciones de este método. key.

L33

```
33 |     this.budget = budget;
```

En this guarda budget. Toma el dato de budget después de las validaciones de este método. budget.

L34

```
34 |     this.clock = clock;
```

En this guarda reloj usado para evaluar plazos. Toma el dato de clock después de las validaciones de este método. reloj usado para evaluar plazos.

L35

```
35 |     this.mapper = mapper;
```

En this guarda conversor JSON de Jackson. Toma el dato de mapper después de las validaciones de este método. conversor JSON de Jackson.

L36 a L40

```
36 |     var client =
37 |         java.net.http.HttpClient.newBuilder()
38 |             .connectTimeout(Duration.ofSeconds(2))
39 |             .followRedirects(java.net.http.HttpClient.Redirect.NEVER)
40 |             .build();
```

Construye el objeto final a partir del builder configurado en líneas previas. Se calcula client (client) para reutilizar ese resultado en los pasos siguientes.

L41

```
41 |     var factory = new JdkClientHttpRequestFactory(client);
```

Construye JdkClientHttpRequestFactory en memoria y conserva la referencia en factory. Los argumentos del constructor determinan los datos iniciales de este objeto. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L42

```
42 |     factory.setReadTimeout(Duration.ofSeconds(5));
```

Limita espera de respuesta a cinco segundos.

L43

```
43 |     this.http = RestClient.builder().baseUrl(url).requestFactory(factory).build();
```

En this guarda http. Toma el dato de RestClient.builder().baseUrl(url).requestFactory(factory).build() después de las validaciones de este método.

L44

```
44 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L46

```
46 |   private static String coordinate(BigDecimal n) {
```

Normaliza decimales para que valores equivalentes generen la misma clave o huella. Tipos y datos: n = n; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L47

```
47 |     return n.stripTrailingZeros().toPlainString();
```

Normaliza el decimal a texto sin ceros finales superfluos ni notación exponencial; números equivalentes producen la misma representación para caché/huellas.

L48

```
48 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L50

```
50 |   @Override
```

Redefine contrato heredado.

L51

```
51 |   public Route route(Point from, Point to, String revision, int maxAgeSeconds, int maxSnapMeters)
       {
```

Contrato proveedor: dos puntos, revisión, edad máxima y ajuste máximo; devuelve recorrido o falla clasificada. Tipos y datos: revision = revision; String es texto; no es por sí mismo una referencia de entidad; maxAgeSeconds = maxAgeSeconds; int es entero de 32 bits para límites y contadores; maxSnapMeters = maxSnapMeters; int es entero de 32 bits para límites y contadores.

L52 a L63

```
52 |     String cacheKey =
53 |         coordinate(from.latitude())
54 |             + ","
55 |             + coordinate(from.longitude())
56 |             + ":"
57 |             + coordinate(to.latitude())
58 |             + ","
59 |             + coordinate(to.longitude())
60 |             + ":driving-car:"
61 |             + revision
62 |             + ":"
63 |             + maxSnapMeters;
```

La clave de caché incluye origen, destino, perfil driving-car, revisión de reglas y tolerancia de ajuste vial. No reutiliza recorridos con coordenadas distintas. Se calcula cacheKey (cacheKey) para reutilizar ese resultado en los pasos siguientes.

L64

```
64 |     Route existing = cache.get(cacheKey);
```

Busca recorrido por clave exacta y captura ahora para controlar su vigencia. Se calcula existing (existing) para reutilizar ese resultado en los pasos siguientes.

L65

```
65 |     Instant now = clock.instant();
```

Captura un instante consistente de referencia para comparar condiciones temporales en esta operación. Se calcula now (instante actual obtenido del reloj inyectado) para reutilizar ese resultado en los pasos siguientes. lee el instante del reloj inyectado

L66 a L68

```
66 |     if (existing != null
67 |         && !existing.obtainedAt().isAfter(now)
68 |         && !existing.obtainedAt().isBefore(now.minusSeconds(maxAgeSeconds))) return existing;
```

Devuelve una ruta cacheada solo si su fecha no es futura y su antigüedad no supera el máximo solicitado. Evita consumir cuota en ese caso. Esta condición decide si se entra en la rama siguiente. calcula un instante anterior restando segundos; compara si el instante es estrictamente posterior al otro; compara si el instante es estrictamente anterior al otro; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue; comprueba presencia antes de usar el dato.

L67: compara si el instante es estrictamente posterior al otro; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L68: calcula un instante anterior restando segundos; compara si el instante es estrictamente anterior al otro; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L69

```
69 |     if (key.isBlank()) throw new RoutingFailure(CONFIGURATION);
```

Sin clave ORS falla como configuración antes de reservar cuota; la capa de evaluación conserva el resultado pendiente. Esta condición decide si se entra en la rama siguiente. comprueba texto vacío o solo espacios.

L70

```
70 |     if (!budget.reserve()) throw new RoutingFailure(RATE_LIMIT);
```

Si no hay presupuesto no sale a la red; reporta límite temporal para reprogramar. Esta condición decide si se entra en la rama siguiente. ! invierte el booleano de la expresión que sigue.

L71

```
71 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L72 a L88

```
72 |       ResponseEntity<JsonNode> response =
73 |           http.post()
74 |               .uri("/directions/driving-car/geojson")
75 |               .header(HttpHeaders.AUTHORIZATION, key)
76 |               .contentType(MediaType.APPLICATION_JSON)
77 |               .body(
78 |                   Map.of(
79 |                       "coordinates",
80 |                       List.of(
81 |                           List.of(from.longitude(), from.latitude()),
82 |                           List.of(to.longitude(), to.latitude())),
83 |                       "preference",
84 |                       "recommended",
85 |                       "units",
86 |                       "m"))
87 |               .retrieve()
88 |               .toEntity(JsonNode.class);
```

ORS recibe pares [longitud, latitud], aunque Point almacena latitud y longitud. Solicita unidades métricas y ruta recomendada. Ejecuta consulta ORS y conserva cuerpo y headers para evaluar cuota. Se calcula response (response) para reutilizar ese resultado en los pasos siguientes. crea una lista no modificable de los elementos indicados

L89 a L90

```
89 |       if ("0".equals(response.getHeaders().getFirst("x-ratelimit-remaining")))
90 |         budget.defer(reset(response.getHeaders(), now, now.plusSeconds(86400)));
```

Esta condición decide si se entra en la rama siguiente. calcula un instante futuro sumando segundos, sin modificar el instante original; compara contenido/identidad lógica, no referencias Java.

L91

```
91 |       Route result = parse(response.getBody(), from, to, maxSnapMeters, clock.instant());
```

Se calcula result (resultado de evaluación, distinto del estado de solicitud) para reutilizar ese resultado en los pasos siguientes. lee el instante del reloj inyectado

L92

```
92 |       if (cache.size() >= 500) cache.clear();
```

Esta condición decide si se entra en la rama siguiente. .

L93

```
93 |       cache.put(cacheKey, result);
```

Añade o reemplaza una propiedad en el objeto JSON o mapa.

L94

```
94 |       return result;
```

Entrega result al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: resultado de evaluación, distinto del estado de solicitud.

L95

```
95 |     } catch (RestClientResponseException e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L96 a L97

```
96 |       HttpHeaders headers =
97 |           e.getResponseHeaders() == null ? new HttpHeaders() : e.getResponseHeaders();
```

Extrae headers/cuerpo de error para clasificar cuota, autenticación o ausencia de ruta. Se calcula headers (headers) para reutilizar ese resultado en los pasos siguientes. detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario

L98

```
98 |       String body = e.getResponseBodyAsString();
```

Se calcula body (body) para reutilizar ese resultado en los pasos siguientes.

L99

```
 99 |       int code = 0;
```

Se calcula code (code) para reutilizar ese resultado en los pasos siguientes. 0

L100

```
100 |       try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L101

```
101 |         code = mapper.readTree(body).path("error").path("code").asInt();
```

Navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a entero; analiza texto JSON para consultar su estructura; Ruta JSON consultada: error.code.

L102

```
102 |       } catch (Exception ignored) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L103

```
103 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L104 a L105

```
104 |       if ((e.getStatusCode().value() == 400 || e.getStatusCode().value() == 404)
105 |           && (code == 2009 || code == 2010)) throw new RoutingFailure(NO_ROUTE);
```

Solo 400/404 con código reconocido significa ausencia de ruta; no transforma un 5xx en rechazo comercial. Esta condición decide si se entra en la rama siguiente. && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L105: && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L106

```
106 |       if (e.getStatusCode().value() == 429) {
```

Esta condición decide si se entra en la rama siguiente. .

L107

```
107 |         budget.defer(retryAfter(headers, now));
```

Pospone la recuperación; respeta la espera mínima del proveedor y no descarta la intención.

L108

```
108 |         throw new RoutingFailure(RATE_LIMIT);
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L109

```
109 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L110 a L112

```
110 |       if (e.getStatusCode().value() == 403
111 |           && (body.toLowerCase(Locale.ROOT).contains("quota")
112 |               || "0".equals(headers.getFirst("x-ratelimit-remaining")))) {
```

Esta condición decide si se entra en la rama siguiente. compara contenido/identidad lógica, no referencias Java; verifica pertenencia al conjunto o lista; && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L111: verifica pertenencia al conjunto o lista; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L112: compara contenido/identidad lógica, no referencias Java; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L113

```
113 |         budget.defer(reset(headers, now, now.plusSeconds(86400)));
```

Calcula un instante futuro sumando segundos, sin modificar el instante original.

L114

```
114 |         throw new RoutingFailure(DAILY_QUOTA);
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L115

```
115 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L116

```
116 |       if (e.getStatusCode().value() == 401 || e.getStatusCode().value() == 403) {
```

Esta condición decide si se entra en la rama siguiente. || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L117

```
117 |         budget.defer(now.plusSeconds(300));
```

Calcula un instante futuro sumando segundos, sin modificar el instante original.

L118

```
118 |         throw new RoutingFailure(CONFIGURATION);
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L119

```
119 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L120

```
120 |       throw new RoutingFailure(UNAVAILABLE);
```

Clasifica error no recuperado como indisponibilidad, no rechazo comercial. La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L121

```
121 |     } catch (ResourceAccessException e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L122

```
122 |       throw new RoutingFailure(UNAVAILABLE);
```

Clasifica error no recuperado como indisponibilidad, no rechazo comercial. La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L123

```
123 |     } catch (RestClientException e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L124

```
124 |       throw new RoutingFailure(INVALID_RESPONSE);
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L125

```
125 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L126

```
126 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L128

```
128 |   static Instant retryAfter(HttpHeaders headers, Instant now) {
```

Interpreta Retry-After en segundos o fecha HTTP y aplica una espera segura si no puede leerlo. Tipos y datos: now = instante actual obtenido del reloj inyectado; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L129

```
129 |     String value = headers.getFirst("Retry-After");
```

Se calcula value (value) para reutilizar ese resultado en los pasos siguientes.

L130 a L131

```
130 |     if (value != null)
131 |       try {
```

Esta condición decide si se entra en la rama siguiente. comprueba presencia antes de usar el dato.

L132

```
132 |         return now.plusSeconds(Math.max(1, Long.parseLong(value)));
```

Resultado que recibe el llamador: calcula un instante futuro sumando segundos, sin modificar el instante original; elige el mayor valor, por ejemplo para evitar esperas negativas.

L133

```
133 |       } catch (NumberFormatException ignored) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L134

```
134 |         try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L135 a L139

```
135 |           return ZonedDateTime.parse(value, DateTimeFormatter.RFC_1123_DATE_TIME)
136 |                   .toInstant()
137 |                   .isAfter(now)
138 |               ? ZonedDateTime.parse(value, DateTimeFormatter.RFC_1123_DATE_TIME).toInstant()
139 |               : now.plusSeconds(60);
```

Resultado que recibe el llamador: calcula un instante futuro sumando segundos, sin modificar el instante original; compara si el instante es estrictamente posterior al otro; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L137: compara si el instante es estrictamente posterior al otro.

L139: calcula un instante futuro sumando segundos, sin modificar el instante original.

L140

```
140 |         } catch (Exception ignoredDate) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L141

```
141 |         }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L142

```
142 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L143

```
143 |     return now.plusSeconds(60);
```

Fallback de espera de un minuto cuando no hay indicación válida del proveedor. Resultado que recibe el llamador: calcula un instante futuro sumando segundos, sin modificar el instante original.

L144

```
144 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L146

```
146 |   static Instant reset(HttpHeaders headers, Instant now, Instant fallback) {
```

Interpreta el instante de reinicio de cuota y utiliza el fallback si es inválido o anterior. Tipos y datos: now = instante actual obtenido del reloj inyectado; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; fallback = fallback; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L147

```
147 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L148

```
148 |       Instant reset =
      Instant.ofEpochSecond(Long.parseLong(headers.getFirst("x-ratelimit-reset")));
```

Se calcula reset (reset) para reutilizar ese resultado en los pasos siguientes.

L149

```
149 |       return reset.isAfter(now) ? reset : fallback;
```

Resultado que recibe el llamador: compara si el instante es estrictamente posterior al otro; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L150

```
150 |     } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L151

```
151 |       return fallback;
```

Entrega fallback al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: fallback.

L152

```
152 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L153

```
153 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L155

```
155 |   static Route parse(JsonNode root, Point from, Point to, int maxSnap, Instant now) {
```

Interpreta GeoJSON y valida métricas, geometría y distancia de ajuste a red. Tipos y datos: root = root; JsonNode es árbol JSON navegable usado para snapshots y respuestas externas; maxSnap = maxSnap; int es entero de 32 bits para límites y contadores; now = instante actual obtenido del reloj inyectado; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L156

```
156 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L157 a L160

```
157 |       JsonNode feature = root.path("features").path(0),
158 |           summary = feature.path("properties").path("summary"),
159 |           geometry = feature.path("geometry").path("coordinates"),
160 |           way = feature.path("properties").path("way_points");
```

Selecciona primera feature, resumen, geometría y waypoints de la respuesta ORS. Se calcula feature (feature) para reutilizar ese resultado en los pasos siguientes. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; Ruta JSON consultada: features.properties.summary.geometry.coordinates.properties.way_points; Variables de la construcción.

L161 a L164

```
161 |       if (!summary.path("distance").isNumber()
162 |           || !summary.path("duration").isNumber()
163 |           || geometry.size() < 2
164 |           || way.size() != 2) throw new RoutingFailure(INVALID_RESPONSE);
```

Valida resumen, geometría y dos waypoints antes de aceptar el GeoJSON. Esta condición decide si se entra en la rama siguiente. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; Ruta JSON consultada: distance.duration; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L162: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; Ruta JSON consultada: duration; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L163: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L164: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L165 a L166

```
165 |       BigDecimal distance = summary.get("distance").decimalValue(),
166 |           duration = summary.get("duration").decimalValue();
```

Extrae distancia y duración como decimales para evitar pérdida innecesaria de precisión. Se calcula distance (distance) para reutilizar ese resultado en los pasos siguientes. lee el número JSON como BigDecimal

L167 a L168

```
167 |       if (distance.signum() < 0 || duration.signum() < 0)
168 |         throw new RoutingFailure(INVALID_RESPONSE);
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L169

```
169 |       int first = way.get(0).asInt(-1), last = way.get(1).asInt(-1);
```

Se calcula first (first) para reutilizar ese resultado en los pasos siguientes.

L170 a L171

```
170 |       if (first < 0 || last < first || last >= geometry.size())
171 |         throw new RoutingFailure(INVALID_RESPONSE);
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L172

```
172 |       JsonNode a = geometry.get(first), z = geometry.get(last);
```

Obtiene puntos de geometría correspondientes a los dos waypoints validados. Se calcula a (a) para reutilizar ese resultado en los pasos siguientes.

L173 a L178

```
173 |       if (a.size() < 2
174 |           || z.size() < 2
175 |           || !a.get(0).isNumber()
176 |           || !a.get(1).isNumber()
177 |           || !z.get(0).isNumber()
178 |           || !z.get(1).isNumber()) throw new RoutingFailure(INVALID_RESPONSE);
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L174: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L175: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L176: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L177: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L178: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L179 a L183

```
179 |       for (JsonNode point : List.of(a, z))
180 |         if (!Double.isFinite(point.get(0).asDouble())
181 |             || !Double.isFinite(point.get(1).asDouble())
182 |             || Math.abs(point.get(0).asDouble()) > 180
183 |             || Math.abs(point.get(1).asDouble()) > 90) throw new
      RoutingFailure(INVALID_RESPONSE);
```

Verifica extremos finitos y dentro de rangos geográficos. El bucle repite el bloque por cada elemento; el nombre a la izquierda de : es la variable local y a la derecha está la colección. Crea una lista no modificable de los elementos indicados; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue. Tipos y datos: point = point; JsonNode es árbol JSON navegable usado para snapshots y respuestas externas.

L181: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L182: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L183: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L184 a L197

```
184 |       if (GeoMath.distance(
185 |                   from.latitude().doubleValue(),
186 |                   from.longitude().doubleValue(),
187 |                   a.get(1).asDouble(),
188 |                   a.get(0).asDouble(),
189 |                   6371000)
190 |               > maxSnap
191 |           || GeoMath.distance(
192 |                   to.latitude().doubleValue(),
193 |                   to.longitude().doubleValue(),
194 |                   z.get(1).asDouble(),
195 |                   z.get(0).asDouble(),
196 |                   6371000)
197 |               > maxSnap) throw new RoutingFailure(SNAP_TOO_FAR);
```

Compara puntos pedidos con extremos ajustados a la red vial; si la desviación supera el máximo devuelve SNAP_TOO_FAR. Esta condición decide si se entra en la rama siguiente. || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L191: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L198

```
198 |       return new Route(distance, duration,
      now.truncatedTo(java.time.temporal.ChronoUnit.MICROS));
```

Devuelve recorrido métrico con instante truncado a microsegundos.

L199

```
199 |     } catch (RoutingFailure e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L200

```
200 |       throw e;
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L201

```
201 |     } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L202

```
202 |       throw new RoutingFailure(INVALID_RESPONSE);
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L203

```
203 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L204

```
204 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L205

```
205 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 16, 25, 45, 49, 127, 145, 154: separadores sin instrucciones.

RouteProvider (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/routing/RouteProvider.java

Objetivo y funcionalidad

Contrato local del proveedor de rutas; Point y Route expresan coordenadas, distancia, duración y fecha de obtención.

L1

```
1 | package com.auxiruta.assistance.routing;
```

Ubica el archivo en com.auxiruta.assistance.routing. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | public interface RouteProvider {
```

Declara interfaz RouteProvider. Contrato local del proveedor de rutas; Point y Route expresan coordenadas, distancia, duración y fecha de obtención.

L7

```
7 |   record Point(BigDecimal latitude, BigDecimal longitude) {}
```

Declara record inmutable Point. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: latitude: latitud declarada; longitude: longitud declarada. Contrato local del proveedor de rutas; Point y Route expresan coordenadas, distancia, duración y fecha de obtención. Tipos y datos: latitude = latitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; longitude = longitud declarada; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos.

L9

```
9 |   record Route(BigDecimal distanceM, BigDecimal durationSeconds, Instant obtainedAt) {}
```

Declara record inmutable Route. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Contrato local del proveedor de rutas; Point y Route expresan coordenadas, distancia, duración y fecha de obtención. Tipos y datos: distanceM = distanceM; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; durationSeconds = durationSeconds; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; obtainedAt = obtainedAt; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L11

```
11 |   Route route(Point from, Point to, String revision, int maxAgeSeconds, int maxSnapMeters);
```

Contrato proveedor: dos puntos, revisión, edad máxima y ajuste máximo; devuelve recorrido o falla clasificada. Tipos y datos: revision = revision; String es texto; no es por sí mismo una referencia de entidad; maxAgeSeconds = maxAgeSeconds; int es entero de 32 bits para límites y contadores; maxSnapMeters = maxSnapMeters; int es entero de 32 bits para límites y contadores.

L12

```
12 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 5, 8, 10: separadores sin instrucciones.

RoutingBudget (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/routing/RoutingBudget.java

Objetivo y funcionalidad

Reserva cuota antes de llamar a ORS mediante una transacción con bloqueo. Conserva ventanas móviles de 60 s y 24 h y un bloqueo hasta fecha indicada por el proveedor.

L1

```
1 | package com.auxiruta.assistance.routing;
```

Ubica el archivo en com.auxiruta.assistance.routing. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.jdbc.core.JdbcTemplate;
```

Importa tipos: org.springframework.jdbc.core.JdbcTemplate. JdbcTemplate para SQL parametrizado, resultados y acciones durables. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.beans.factory.annotation.Value;
```

Importa tipos: org.springframework.beans.factory.annotation.Value. Inyección de dependencias y propiedades de entorno. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | @Service
```

Registra lógica de aplicación como bean. El escaneo de Spring crea este componente e inyecta sus dependencias; no hay que construirlo manualmente en el controlador.

L10

```
10 | public class RoutingBudget {
```

Declara clase RoutingBudget. Reserva cuota antes de llamar a ORS mediante una transacción con bloqueo. Conserva ventanas móviles de 60 s y 24 h y un bloqueo hasta fecha indicada por el proveedor.

L11

```
11 |   private final JdbcTemplate jdbc;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: jdbc = ejecutor SQL parametrizado; JdbcTemplate es ejecutor de SQL parametrizado de Spring dentro de transacciones locales.

L12

```
12 |   private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L13

```
13 |   private final int perMinute, perDay;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: perMinute = perMinute; int es entero de 32 bits para límites y contadores.

L15 a L19

```
15 |   public RoutingBudget(
16 |       JdbcTemplate jdbc,
17 |       Clock clock,
18 |       @Value("${ors.per-minute:40}") int minute,
19 |       @Value("${ors.per-day:2000}") int day) {
```

Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición. Tipos y datos: jdbc = ejecutor SQL parametrizado; JdbcTemplate es ejecutor de SQL parametrizado de Spring dentro de transacciones locales; clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos; minute = minute; int es entero de 32 bits para límites y contadores; day = day; int es entero de 32 bits para límites y contadores.

L20

```
20 |     this.jdbc = jdbc;
```

En this guarda ejecutor SQL parametrizado. Toma el dato de jdbc después de las validaciones de este método. ejecutor SQL parametrizado.

L21

```
21 |     this.clock = clock;
```

En this guarda reloj usado para evaluar plazos. Toma el dato de clock después de las validaciones de este método. reloj usado para evaluar plazos.

L22

```
22 |     this.perMinute = minute;
```

En this guarda perMinute. Toma el dato de minute después de las validaciones de este método. minute.

L23

```
23 |     this.perDay = day;
```

En this guarda perDay. Toma el dato de day después de las validaciones de este método. day.

L24

```
24 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L26

```
26 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L27

```
27 |   public boolean reserve() {
```

Reserva una unidad de presupuesto de ORS de manera transaccional. Devuelve false si hay bloqueo o cuota agotada; true solo después de registrar la llamada.

L28

```
28 |     Instant now = clock.instant();
```

Captura un instante consistente de referencia para comparar condiciones temporales en esta operación. Se calcula now (instante actual obtenido del reloj inyectado) para reutilizar ese resultado en los pasos siguientes. lee el instante del reloj inyectado

L29 a L32

```
29 |     java.sql.Timestamp blocked =
30 |         jdbc.queryForObject(
31 |             "select blocked_until from routing_budget where id=1 for update",
32 |             java.sql.Timestamp.class);
```

Consulta routing_budget con el filtro y proyección escritos. Bloquea las filas durante la transacción. Se calcula blocked (blocked) para reutilizar ese resultado en los pasos siguientes. ejecuta SQL parametrizado y espera un resultado escalar o una fila

L33

```
33 |     if (blocked != null && blocked.toInstant().isAfter(now)) return false;
```

Esta condición decide si se entra en la rama siguiente. compara si el instante es estrictamente posterior al otro; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato.

L34 a L36

```
34 |     jdbc.update(
35 |         "delete from routing_calls where called_at < ?",
36 |         java.sql.Timestamp.from(now.minusSeconds(86400)));
```

Poda llamadas fuera de la ventana móvil de 24 h; esta tabla es control de cuota, no historial permanente de todas las consultas ORS. Calcula un instante anterior restando segundos; ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L37

```
37 |     long daily = jdbc.queryForObject("select count(*) from routing_calls", Long.class);
```

Consulta routing_calls contando coincidencias. Se calcula daily (daily) para reutilizar ese resultado en los pasos siguientes. ejecuta SQL parametrizado y espera un resultado escalar o una fila

L38 a L42

```
38 |     long minute =
39 |         jdbc.queryForObject(
40 |             "select count(*) from routing_calls where called_at > ?",
41 |             Long.class,
42 |             java.sql.Timestamp.from(now.minusSeconds(60)));
```

Consulta routing_calls contando coincidencias. Se calcula minute (minute) para reutilizar ese resultado en los pasos siguientes. calcula un instante anterior restando segundos; ejecuta SQL parametrizado y espera un resultado escalar o una fila

L43

```
43 |     if (daily >= perDay || minute >= perMinute) return false;
```

Esta condición decide si se entra en la rama siguiente. || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L44 a L47

```
44 |     jdbc.update(
45 |         "insert into routing_calls(id,called_at) values (?,?)",
46 |         java.util.UUID.randomUUID(),
47 |         java.sql.Timestamp.from(now));
```

Genera ID nuevo en la aplicación. Inserta en routing_calls. Genera un nuevo identificador UUID; ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L48

```
48 |     return true;
```

Entrega true al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: true.

L49

```
49 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L51

```
51 |   @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L52

```
52 |   public void defer(Instant until) {
```

Pospone la recuperación; respeta la espera mínima del proveedor y no descarta la intención. Tipos y datos: until = until; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L53 a L55

```
53 |     jdbc.update(
54 |         "update routing_budget set blocked_until=greatest(blocked_until,?) where id=1",
55 |         java.sql.Timestamp.from(until));
```

Extiende blocked_until sin acortarlo por una respuesta posterior con espera menor. Se calcula blocked_until (blocked_until) para reutilizar ese resultado en los pasos siguientes. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L56

```
56 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L57

```
57 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 8, 14, 25, 50: separadores sin instrucciones.

RoutingFailure (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/routing/RoutingFailure.java`

Objetivo y funcionalidad

Clasifica fallos del proveedor. Algunas causas justifican rechazo del dominio y otras dejan evaluación pendiente.

L1

```
1 | package com.auxiruta.assistance.routing;
```

Ubica el archivo en com.auxiruta.assistance.routing. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | public class RoutingFailure extends RuntimeException {
```

Declara clase RoutingFailure. Clasifica fallos del proveedor. Algunas causas justifican rechazo del dominio y otras dejan evaluación pendiente.

L4

```
4 |   public enum Reason {
```

Declara enumeración cerrada Reason. Clasifica fallos del proveedor. Algunas causas justifican rechazo del dominio y otras dejan evaluación pendiente.

L5 a L12

```
 5 |     NO_ROUTE,
 6 |     SNAP_TOO_FAR,
 7 |     RATE_LIMIT,
 8 |     DAILY_QUOTA,
 9 |     CONFIGURATION,
10 |     UNAVAILABLE,
11 |     INVALID_RESPONSE
12 |   }
```

Valores cerrados de esta enumeración: NO_ROUTE, SNAP_TOO_FAR, RATE_LIMIT, DAILY_QUOTA, CONFIGURATION, UNAVAILABLE, INVALID_RESPONSE. El tipo impide usar cadenas arbitrarias; representan alternativas del dominio. Las transiciones válidas se controlan en los servicios; enumerar estados no permite saltar entre ellos libremente.

L14

```
14 |   private final Reason reason;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L16

```
16 |   public RoutingFailure(Reason reason) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L17

```
17 |     super(reason.name());
```

Inicializa el mensaje de la excepción padre con el nombre estable de la causa; el objeto conserva además datos para clasificar rechazo o fallo temporal.

L18

```
18 |     this.reason = reason;
```

En this guarda motivo legible. Toma el dato de reason después de las validaciones de este método. motivo legible.

L19

```
19 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L21

```
21 |   public Reason reason() {
```

Permite a la capa de negocio conocer categoría del fallo sin depender de la excepción HTTP.

L22

```
22 |     return reason;
```

Entrega reason al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: motivo legible.

L23

```
23 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L25

```
25 |   public boolean domainRejection() {
```

Solo NO_ROUTE y SNAP_TOO_FAR representan rechazo de dominio; el resto exige recuperación temporal/configuración.

L26

```
26 |     return reason == Reason.NO_ROUTE || reason == Reason.SNAP_TOO_FAR;
```

Resultado que recibe el llamador: || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L27

```
27 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L28

```
28 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 13, 15, 20, 24: separadores sin instrucciones.

8. Propuestas, aceptación y recuperación durable

AssignmentCoordinator es una clase técnica, no un rol humano. Las acciones remotas inciertas se conservan y recuperan.

AssignmentController (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/AssignmentController.java

Objetivo y funcionalidad

Expone consultas y respuestas de propuestas. Solo el operador destinatario puede aceptar/rechazar; ADMIN consulta pero no actúa por él.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import jakarta.validation.Valid;
```

Importa tipos: jakarta.validation.Valid. Restricciones de DTO y activación de validación de entradas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import jakarta.validation.constraints.*;
```

Importa tipos: jakarta.validation.constraints.*. Restricciones de DTO y activación de validación de entradas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.web.bind.annotation.*;
```

Importa tipos: org.springframework.web.bind.annotation.*. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.security.core.annotation.AuthenticationPrincipal;
```

Importa tipos: org.springframework.security.core.annotation.AuthenticationPrincipal. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.springframework.security.oauth2.jwt.Jwt;
```

Importa tipos: org.springframework.security.oauth2.jwt.Jwt. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | import io.swagger.v3.oas.annotations.Operation;
```

Importa tipos: io.swagger.v3.oas.annotations.Operation. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import io.swagger.v3.oas.annotations.responses.*;
```

Importa tipos: io.swagger.v3.oas.annotations.responses.*. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | import io.swagger.v3.oas.annotations.security.SecurityRequirement;
```

Importa tipos: io.swagger.v3.oas.annotations.security.SecurityRequirement. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L13

```
13 | @RestController
```

Expone HTTP y serializa JSON.

L14

```
14 | @RequestMapping("/api/v1")
```

Ruta /api/v1 combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L15

```
15 | @SecurityRequirement(name = "bearerAuth")
```

Documenta autenticación; no concede permisos.

L16 a L26

```
16 | @ApiResponses({
17 |   @ApiResponse(responseCode = "400", description = "Entrada invalida"),
18 |   @ApiResponse(responseCode = "401", description = "JWT ausente o invalido"),
19 |   @ApiResponse(responseCode = "403", description = "Rol insuficiente"),
20 |   @ApiResponse(responseCode = "404", description = "Recurso inexistente o ajeno"),
21 |   @ApiResponse(responseCode = "409", description = "Conflicto de estado, plazo o idempotencia"),
22 |   @ApiResponse(
23 |       responseCode = "503",
24 |       description = "Dependencia temporalmente no disponible; consultar estado antes de
     reintentar")
25 | })
26 | public class AssignmentController {
```

Documenta código HTTP. Declara clase AssignmentController. Expone consultas y respuestas de propuestas. Solo el operador destinatario puede aceptar/rechazar; ADMIN consulta pero no actúa por él.

L27

```
27 |   public record Rejection(@NotBlank @Size(min = 10, max = 500) String reason) {}
```

Exige texto no vacío ni solo espacios. Limita caracteres/elementos según min/max. Declara record inmutable Rejection. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: reason: motivo legible. Se calcula max (max) para reutilizar ese resultado en los pasos siguientes. Expone consultas y respuestas de propuestas. Solo el operador destinatario puede aceptar/rechazar; ADMIN consulta pero no actúa por él. Tipos y datos: reason = motivo legible; String es texto; no es por sí mismo una referencia de entidad.

L29

```
29 |   private final AssignmentStore store;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L30

```
30 |   private final AssignmentCoordinator coordinator;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L32

```
32 |   public AssignmentController(AssignmentStore store, AssignmentCoordinator coordinator) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L33

```
33 |     this.store = store;
```

En this guarda componente que conserva decisiones en transacciones locales. Toma el dato de store después de las validaciones de este método. componente que conserva decisiones en transacciones locales.

L34

```
34 |     this.coordinator = coordinator;
```

En this guarda orquestador técnico de reserva/confirmación/liberación. Toma el dato de coordinator después de las validaciones de este método. orquestador técnico de reserva/confirmación/liberación.

L35

```
35 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L37

```
37 |   @GetMapping("/operator/assignments")
```

Ruta /operator/assignments combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L38

```
38 |   @Operation(summary = "OPERATOR: consultar propuestas y servicios propios")
```

Documenta en OpenAPI: OPERATOR: consultar propuestas y servicios propios. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L39 a L42

```
39 |   public RequestDtos.PageView<AssignmentStore.View> own(
40 |       @AuthenticationPrincipal Jwt jwt,
41 |       @RequestParam(defaultValue = "0") int page,
42 |       @RequestParam(defaultValue = "20") int size) {
```

Obtiene JWT validado, no identidad suministrada por el cliente. Lee parámetro/filtro URL y su default. Tipos y datos: jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias; page = page; int es entero de 32 bits para límites y contadores; size = size; int es entero de 32 bits para límites y contadores.

L43

```
43 |     return store.own(jwt.getSubject(), page, size);
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Resultado que recibe el llamador: obtiene sub, identidad estable del usuario autenticado.

L44

```
44 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L46

```
46 |   @GetMapping("/assignments/{id}")
```

Ruta /assignments/{id} combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L47

```
47 |   @Operation(summary = "Consultar asignación propia; ADMIN puede consultar globalmente")
```

Documenta en OpenAPI: Consultar asignación propia; ADMIN puede consultar globalmente. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L48

```
48 |   public AssignmentStore.View get(@PathVariable UUID id, @AuthenticationPrincipal Jwt jwt) {
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias.

L49

```
49 |     return store.visible(id, jwt);
```

Consulta la asignación con control de alcance: operador destinatario o administrador autorizado a consultar. No permite actuar en nombre del operador.

L50

```
50 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L52

```
52 |   @PostMapping("/assignments/{id}/acceptance")
```

Ruta /assignments/{id}/acceptance combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L53

```
53 |   @Operation(summary = "OPERATOR destinatario: confirmar una propuesta vigente con
     Idempotency-Key")
```

Documenta en OpenAPI: OPERATOR destinatario: confirmar una propuesta vigente con Idempotency-Key. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L54 a L57

```
54 |   public AssignmentStore.View accept(
55 |       @PathVariable UUID id,
56 |       @AuthenticationPrincipal Jwt jwt,
57 |       @RequestHeader("Idempotency-Key") String key) {
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Lee header indicado, normalmente Idempotency-Key. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias; key = key; String es texto; no es por sí mismo una referencia de entidad.

L55: Dato del contrato: id: identificador UUID del registro.

L58

```
58 |     coordinator.execute(store.respond(id, jwt.getSubject(), key, true, null));
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Obtiene sub, identidad estable del usuario autenticado.

L59

```
59 |     return store.visible(id, jwt);
```

Consulta la asignación con control de alcance: operador destinatario o administrador autorizado a consultar. No permite actuar en nombre del operador.

L60

```
60 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L62

```
62 |   @PostMapping("/assignments/{id}/rejection")
```

Ruta /assignments/{id}/rejection combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L63 a L69

```
63 |   @Operation(
64 |       summary = "OPERATOR destinatario: rechazar con motivo y liberar antes de otra propuesta")
65 |   public AssignmentStore.View reject(
66 |       @PathVariable UUID id,
67 |       @AuthenticationPrincipal Jwt jwt,
68 |       @RequestHeader("Idempotency-Key") String key,
69 |       @Valid @RequestBody Rejection body) {
```

Documenta en OpenAPI: OPERATOR destinatario: rechazar con motivo y liberar antes de otra propuesta. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L66: Dato del contrato: id: identificador UUID del registro.

L70

```
70 |     coordinator.execute(store.respond(id, jwt.getSubject(), key, false, body.reason()));
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Obtiene sub, identidad estable del usuario autenticado.

L71

```
71 |     var result = store.visible(id, jwt);
```

Se calcula result (resultado de evaluación, distinto del estado de solicitud) para reutilizar ese resultado en los pasos siguientes.

L72

```
72 |     coordinator.propose(result.requestId());
```

Selecciona candidato evaluado e inicia reserva/propuesta si siguen vigentes las condiciones. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L73

```
73 |     return result;
```

Entrega result al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: resultado de evaluación, distinto del estado de solicitud.

L74

```
74 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L75

```
75 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 12, 28, 31, 36, 45, 51, 61: separadores sin instrucciones.

AssignmentCoordinator (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/AssignmentCoordinator.java`

Objetivo y funcionalidad

Orquestador técnico, no rol humano. Ejecuta HTTP fuera de las transacciones de AssignmentStore y reprograma acciones inciertas con el mismo identificador.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import com.auxiruta.assistance.routing.FleetClient;
```

Importa tipos: com.auxiruta.assistance.routing.FleetClient. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.web.client.RestClientResponseException;
```

Importa tipos: org.springframework.web.client.RestClientResponseException. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | @Service
```

Registra lógica de aplicación como bean. El escaneo de Spring crea este componente e inyecta sus dependencias; no hay que construirlo manualmente en el controlador.

L9

```
9 | public class AssignmentCoordinator {
```

Declara clase AssignmentCoordinator. Orquestador técnico, no rol humano. Ejecuta HTTP fuera de las transacciones de AssignmentStore y reprograma acciones inciertas con el mismo identificador.

L10

```
10 |   private final AssignmentStore store;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L11

```
11 |   private final FleetClient fleet;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L12

```
12 |   private final java.time.Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L14

```
14 |   public AssignmentCoordinator(AssignmentStore store, FleetClient fleet) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L15

```
15 |     this(store, fleet, java.time.Clock.systemUTC());
```

Crea reloj de sistema con zona UTC; las pruebas pueden sustituirlo por Clock fijo.

L16

```
16 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L18

```
18 |   @org.springframework.beans.factory.annotation.Autowired
```

Inyecta dependencia Spring.

L19

```
19 |   public AssignmentCoordinator(AssignmentStore store, FleetClient fleet, java.time.Clock clock) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L20

```
20 |     this.store = store;
```

En this guarda componente que conserva decisiones en transacciones locales. Toma el dato de store después de las validaciones de este método. componente que conserva decisiones en transacciones locales.

L21

```
21 |     this.fleet = fleet;
```

En this guarda cliente técnico que consulta o modifica reservas en Fleet. Toma el dato de fleet después de las validaciones de este método. cliente técnico que consulta o modifica reservas en Fleet.

L22

```
22 |     this.clock = clock;
```

En this guarda reloj usado para evaluar plazos. Toma el dato de clock después de las validaciones de este método. reloj usado para evaluar plazos.

L23

```
23 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L25

```
25 |   private int retryAfter(RestClientResponseException e) {
```

Interpreta Retry-After en segundos o fecha HTTP y aplica una espera segura si no puede leerlo.

L26 a L28

```
26 |     return retryAfter(
27 |         e.getResponseHeaders() == null ? null : e.getResponseHeaders().getFirst("Retry-After"),
28 |         clock.instant());
```

Resultado que recibe el llamador: lee el instante del reloj inyectado; detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L29

```
29 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L31

```
31 |   static int retryAfter(String header, java.time.Instant now) {
```

Interpreta Retry-After en segundos o fecha HTTP y aplica una espera segura si no puede leerlo. Tipos y datos: header = header; String es texto; no es por sí mismo una referencia de entidad; now = instante actual obtenido del reloj inyectado; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L32

```
32 |     if (header == null) return 60;
```

Esta condición decide si se entra en la rama siguiente. detecta un dato ausente.

L33

```
33 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L34

```
34 |       return (int) Math.min(Integer.MAX_VALUE, Math.max(1, Long.parseLong(header.strip())));
```

Resultado que recibe el llamador: elige el mayor valor, por ejemplo para evitar esperas negativas; elige el menor valor para respetar el límite más restrictivo.

L35

```
35 |     } catch (Exception ignored) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L36

```
36 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L37

```
37 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L38 a L41

```
38 |       var until =
39 |           java.time.ZonedDateTime.parse(
40 |                   header, java.time.format.DateTimeFormatter.RFC_1123_DATE_TIME)
41 |               .toInstant();
```

Se calcula until (until) para reutilizar ese resultado en los pasos siguientes.

L42 a L45

```
42 |       return (int)
43 |           Math.min(
44 |               Integer.MAX_VALUE,
45 |               Math.max(1, java.time.Duration.between(now, until).getSeconds() + 1));
```

Resultado que recibe el llamador: elige el mayor valor, por ejemplo para evitar esperas negativas; elige el menor valor para respetar el límite más restrictivo.

L46

```
46 |     } catch (Exception ignored) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L47

```
47 |       return 60;
```

Entrega 60 al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: 60.

L48

```
48 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L49

```
49 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L51

```
51 |   public void recoverDue() {
```

Punto de entrada para reintentar acciones pendientes cuyo próximo intento ya corresponde. Recupera trabajo persistido después de fallos o reinicios.

L52

```
52 |     for (var action : store.dueActions()) execute(action);
```

Ejecuta la acción remota preparada, fuera de la transacción que la registró; después guarda éxito o reprograma el reintento.

L53

```
53 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L55

```
55 |   public void propose(UUID request) {
```

Tipos y datos: request = request; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L56

```
56 |     for (int i = 0; i < 5; i++) {
```

El bucle repite el bloque por cada elemento; el nombre a la izquierda de : es la variable local y a la derecha está la colección.

L57

```
57 |       var action = store.propose(request);
```

Se calcula action (action) para reutilizar ese resultado en los pasos siguientes.

L58

```
58 |       if (action == null) return;
```

Esta condición decide si se entra en la rama siguiente. detecta un dato ausente.

L59

```
59 |       execute(action);
```

Ejecuta la acción remota preparada, fuera de la transacción que la registró; después guarda éxito o reprograma el reintento.

L60

```
60 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L61

```
61 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L63

```
63 |   public void recover(UUID request) {
```

Tipos y datos: request = request; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L64

```
64 |     for (var action : store.pendingForRequest(request)) execute(action);
```

Ejecuta la acción remota preparada, fuera de la transacción que la registró; después guarda éxito o reprograma el reintento.

L65

```
65 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L67

```
67 |   public void execute(AssignmentStore.Action action) {
```

Ejecuta una intención durable con claim y manejo de errores, sin transacción extendida alrededor de HTTP.

L68

```
68 |     // Each network call is outside database transactions. Continuations are compensation, not
```

Documentación para quien lee el fuente: Each network call is outside database transactions. Continuations are compensation, not No es una instrucción ejecutable y no impone la regla por sí sola.

L69

```
69 |     // retries.
```

Documentación para quien lee el fuente: retries. No es una instrucción ejecutable y no impone la regla por sí sola.

L70

```
70 |     for (int step = 0; action != null && step < 3; step++) {
```

Máximo tres continuaciones compensatorias, no tres reintentos inmediatos del fallo. El bucle repite el bloque por cada elemento; el nombre a la izquierda de : es la variable local y a la derecha está la colección. && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato.

L71

```
71 |       if (!store.claim(action)) return;
```

Solo quien obtiene ventana de trabajo hace HTTP; el otro abandona. Esta condición decide si se entra en la rama siguiente. ! invierte el booleano de la expresión que sigue.

L72

```
72 |       try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L73

```
73 |         var response = fleet.reservation(action.assignmentId(), action.kind(), action.payload());
```

Se calcula response (response) para reutilizar ese resultado en los pasos siguientes.

L74

```
74 |         action = store.completed(action, response);
```

Registra la respuesta remota como completada en una transacción local y obtiene la siguiente acción si el flujo requiere compensación o continuación.

L75

```
75 |       } catch (RestClientResponseException e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L76 a L77

```
76 |         if (action.kind().equals("RESERVE")
77 |             && (e.getStatusCode().value() == 409 || e.getStatusCode().value() == 404)) {
```

Esta condición decide si se entra en la rama siguiente. compara contenido/identidad lógica, no referencias Java; && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L77: && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L78

```
78 |           store.reserveRejected(action);
```

Cierra el intento de reserva rechazado explícitamente y vuelve a evaluar si la solicitud sigue abierta.

L79

```
79 |           return;
```

Return.

L80

```
80 |         }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L81 a L85

```
81 |         store.defer(
82 |             action,
83 |             e.getStatusCode().value() == 401 || e.getStatusCode().value() == 403
84 |                 ? 300
85 |                 : e.getStatusCode().value() == 429 ? retryAfter(e) : 0);
```

|| acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L85: el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L86

```
86 |         return;
```

Return.

L87

```
87 |       } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L88

```
88 |         store.failed(action);
```

Conserva el fallo y programa el próximo intento; no borra la intención durable ni supone que el remoto no recibió la operación.

L89

```
89 |         return;
```

Return.

L90

```
90 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L91

```
91 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L92

```
92 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L93

```
93 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 7, 13, 17, 24, 30, 50, 54, 62, 66: separadores sin instrucciones.

AssignmentStore (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/AssignmentStore.java`

Objetivo y funcionalidad

Persistencia transaccional de propuestas y acciones remotas pendientes. Excluye al operador que rechazó durante toda la solicitud y conserva aceptación/cancelación idempotentes.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.sql.*;
```

Importa tipos: java.sql.*. Timestamp y ResultSet para puentear fechas/filas de JDBC con objetos Java. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.jdbc.core.JdbcTemplate;
```

Importa tipos: org.springframework.jdbc.core.JdbcTemplate. JdbcTemplate para SQL parametrizado, resultados y acciones durables. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | import org.springframework.http.HttpStatus;
```

Importa tipos: org.springframework.http.HttpStatus. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | import org.springframework.security.oauth2.jwt.Jwt;
```

Importa tipos: org.springframework.security.oauth2.jwt.Jwt. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12

```
12 | import com.auxiruta.commons.security.TokenSupport;
```

Importa tipos: com.auxiruta.commons.security.TokenSupport. Utilidades compartidas de seguridad, errores, reloj o geografía. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L13

```
13 | import com.fasterxml.jackson.databind.*;
```

Importa tipos: com.fasterxml.jackson.databind.*. Lectura/escritura JSON, nodos y conversión a DTO. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L14

```
14 | import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L16

```
16 | @Service
```

Registra lógica de aplicación como bean. El escaneo de Spring crea este componente e inyecta sus dependencias; no hay que construirlo manualmente en el controlador.

L17

```
17 | @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L18

```
18 | public class AssignmentStore {
```

Declara clase AssignmentStore. Persistencia transaccional de propuestas y acciones remotas pendientes. Excluye al operador que rechazó durante toda la solicitud y conserva aceptación/cancelación idempotentes.

L19 a L28

```
19 |   public record View(
20 |       UUID id,
21 |       UUID requestId,
22 |       UUID towTruckId,
23 |       UUID operatorId,
24 |       String operatorSubject,
25 |       String status,
26 |       Instant expiresAt,
27 |       Instant acceptedAt,
28 |       String reason) {}
```

Declara record inmutable View. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; requestId: UUID de solicitud; towTruckId: UUID de grúa; operatorId: UUID del perfil operador; operatorSubject: sub del operador destinatario; status: estado del objeto; su enum o CHECK define valores permitidos; expiresAt: fin exclusivo de retención/propuesta; acceptedAt: instante de aceptación efectiva; reason: motivo legible. Persistencia transaccional de propuestas y acciones remotas pendientes. Excluye al operador que rechazó durante toda la solicitud y conserva aceptación/cancelación idempotentes. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; requestId = UUID de solicitud; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; towTruckId = UUID de grúa; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; operatorId = UUID del perfil operador; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; operatorSubject = sub del operador destinatario; String es texto; no es por sí mismo una referencia de entidad; status = estado del objeto; su enum o CHECK define valores permitidos; String es texto; no es por sí mismo una referencia de entidad; expiresAt = fin exclusivo de retención/propuesta; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; acceptedAt = instante de aceptación efectiva; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; reason = motivo legible; String es texto; no es por sí mismo una referencia de entidad.

L20: Dato del contrato: id: identificador UUID del registro.

L21: Dato del contrato: requestId: UUID de solicitud.

L22: Dato del contrato: towTruckId: UUID de grúa.

L23: Dato del contrato: operatorId: UUID del perfil operador.

L24: Dato del contrato: operatorSubject: sub del operador destinatario.

L25: Dato del contrato: status: estado del objeto; su enum o CHECK define valores permitidos.

L26: Dato del contrato: expiresAt: fin exclusivo de retención/propuesta.

L27: Dato del contrato: acceptedAt: instante de aceptación efectiva.

L28: Dato del contrato: reason: motivo legible.

L30

```
30 |   public record Action(UUID id, UUID assignmentId, String kind, String payload) {}
```

Declara record inmutable Action. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; payload: contenido normalizado para reintentos. Persistencia transaccional de propuestas y acciones remotas pendientes. Excluye al operador que rechazó durante toda la solicitud y conserva aceptación/cancelación idempotentes. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; assignmentId = assignmentId; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; kind = kind; String es texto; no es por sí mismo una referencia de entidad; payload = contenido normalizado para reintentos; String es texto; no es por sí mismo una referencia de entidad.

L32

```
32 |   private final JdbcTemplate jdbc;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: jdbc = ejecutor SQL parametrizado; JdbcTemplate es ejecutor de SQL parametrizado de Spring dentro de transacciones locales.

L33

```
33 |   private final RequestRepository requests;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L34

```
34 |   private final CandidateEvaluationRepository candidates;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L35

```
35 |   private final EvaluationRepository evaluations;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L36

```
36 |   private final HistoryRepository history;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L37

```
37 |   private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L38

```
38 |   private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L40 a L47

```
40 |   public AssignmentStore(
41 |       JdbcTemplate jdbc,
42 |       RequestRepository requests,
43 |       CandidateEvaluationRepository candidates,
44 |       EvaluationRepository evaluations,
45 |       HistoryRepository history,
46 |       Clock clock,
47 |       ObjectMapper mapper) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada. Tipos y datos: jdbc = ejecutor SQL parametrizado; JdbcTemplate es ejecutor de SQL parametrizado de Spring dentro de transacciones locales; clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos; mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L48

```
48 |     this.jdbc = jdbc;
```

En this guarda ejecutor SQL parametrizado. Toma el dato de jdbc después de las validaciones de este método. ejecutor SQL parametrizado.

L49

```
49 |     this.requests = requests;
```

En this guarda repositorio de solicitudes. Toma el dato de requests después de las validaciones de este método. repositorio de solicitudes.

L50

```
50 |     this.candidates = candidates;
```

En this guarda candidates. Toma el dato de candidates después de las validaciones de este método. candidates.

L51

```
51 |     this.evaluations = evaluations;
```

En this guarda acceso a evaluaciones persistidas. Toma el dato de evaluations después de las validaciones de este método. acceso a evaluaciones persistidas.

L52

```
52 |     this.history = history;
```

En this guarda historial persistente de cambios. Toma el dato de history después de las validaciones de este método. historial persistente de cambios.

L53

```
53 |     this.clock = clock;
```

En this guarda reloj usado para evaluar plazos. Toma el dato de clock después de las validaciones de este método. reloj usado para evaluar plazos.

L54

```
54 |     this.mapper = mapper;
```

En this guarda conversor JSON de Jackson. Toma el dato de mapper después de las validaciones de este método. conversor JSON de Jackson.

L55

```
55 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L57

```
57 |   private int rule(AssistanceRequest r, String group, String key) {
```

Tipos y datos: group = group; String es texto; no es por sí mismo una referencia de entidad; key = key; String es texto; no es por sí mismo una referencia de entidad.

L58

```
58 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L59

```
59 |       return mapper.readTree(r.configurationSnapshot).path(group).path(key).asInt();
```

Resultado que recibe el llamador: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a entero; analiza texto JSON para consultar su estructura; Datos implicados: JSON de parámetros vigente al crear.

L60

```
60 |     } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L61

```
61 |       throw new IllegalStateException("Invalid rules");
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L62

```
62 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L63

```
63 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L65

```
65 |   private Instant now() {
```

Obtiene el instante del reloj inyectado; truncar a microsegundos mantiene consistencia con PostgreSQL.

L66

```
66 |     return clock.instant().truncatedTo(java.time.temporal.ChronoUnit.MICROS);
```

Resultado que recibe el llamador: lee el instante del reloj inyectado.

L67

```
67 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L69

```
69 |   private static Timestamp ts(Instant i) {
```

Convierte Instant a Timestamp JDBC; la variante nullable preserva null. Tipos y datos: i = i; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L70

```
70 |     return i == null ? null : Timestamp.from(i);
```

Resultado que recibe el llamador: detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L71

```
71 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L73

```
73 |   private static Instant instant(ResultSet r, String name) throws SQLException {
```

Lee columna temporal JDBC y la convierte a Instant sin inventar fecha cuando es nula. Tipos y datos: name = name; String es texto; no es por sí mismo una referencia de entidad.

L74

```
74 |     Timestamp t = r.getTimestamp(name);
```

Se calcula t (t) para reutilizar ese resultado en los pasos siguientes.

L75

```
75 |     return t == null ? null : t.toInstant();
```

Resultado que recibe el llamador: detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L76

```
76 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L78

```
78 |   private static void require(boolean ok) {
```

Expresa una precondición; si resulta falsa se aborta con el estado HTTP indicado. Tipos y datos: ok = ok; boolean es verdadero/falso, sin null.

L79

```
79 |     if (!ok) throw new ResponseStatusException(HttpStatus.CONFLICT);
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. interrumpe la operación con un estado HTTP explícito; 409 comunica que el estado/regla actual impide la acción; ! invierte el booleano de la expresión que sigue.

L80

```
80 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L82

```
82 |   private static ResponseStatusException missing() {
```

Construye 404; puede representar recurso inexistente o ajeno para no revelar su existencia.

L83

```
83 |     return new ResponseStatusException(HttpStatus.NOT_FOUND);
```

Resultado que recibe el llamador: interrumpe la operación con un estado HTTP explícito; 404 oculta inexistencia o pertenencia ajena.

L84

```
84 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L86

```
86 |   private View map(ResultSet r, int row) throws SQLException {
```

Convierte una fila ResultSet a un record; respeta tipos UUID y fechas opcionales. Tipos y datos: row = row; int es entero de 32 bits para límites y contadores.

L87 a L96

```
87 |     return new View(
88 |         r.getObject("id", UUID.class),
89 |         r.getObject("request_id", UUID.class),
90 |         r.getObject("tow_truck_id", UUID.class),
91 |         r.getObject("operator_id", UUID.class),
92 |         r.getString("operator_subject"),
93 |         r.getString("status"),
94 |         instant(r, "expires_at"),
95 |         instant(r, "accepted_at"),
96 |         r.getString("reason"));
```

Construye el DTO de asignación desde una fila JDBC. getObject(..., UUID.class) conserva tipos de identificadores; getString lee estado/sujeto/motivo; instant adapta fechas opcionales. No devuelve el ResultSet ni una entidad mutable.

L97

```
97 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L99

```
99 |   private View get(UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L100 a L102

```
100 |     return jdbc.query("select * from assignments where id=?", this::map, id).stream()
101 |         .findFirst()
102 |         .orElseThrow(AssignmentStore::missing);
```

Si no encuentra resultado lanza la excepción indicada. Consulta assignments con el filtro y proyección escritos. Resultado que recibe el llamador: inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente; si no existe valor, lanza la excepción en vez de seguir con null; ejecuta una consulta SQL y transforma filas mediante el mapper; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L102: si no existe valor, lanza la excepción en vez de seguir con null.

L103

```
103 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L105

```
105 |   private AssistanceRequest lock(UUID requestId) {
```

Adquiere bloqueos en orden para serializar operaciones competidoras. Campos: requestId: UUID de solicitud. Tipos y datos: requestId = UUID de solicitud; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L106

```
106 |     UUID vehicle = requests.vehicleId(requestId).orElseThrow(AssignmentStore::missing);
```

Si no encuentra resultado lanza la excepción indicada. Se calcula vehicle (vehicle) para reutilizar ese resultado en los pasos siguientes. si no existe valor, lanza la excepción en vez de seguir con null

L107

```
107 |     jdbc.queryForObject("select id from vehicles where id=? for update", UUID.class, vehicle);
```

Consulta vehicles con el filtro y proyección escritos. Bloquea las filas durante la transacción. Se calcula id (identificador UUID del registro) para reutilizar ese resultado en los pasos siguientes. Ejecuta SQL parametrizado y espera un resultado escalar o una fila.

L108

```
108 |     return requests.lockById(requestId).orElseThrow(AssignmentStore::missing);
```

Si no encuentra resultado lanza la excepción indicada. Resultado que recibe el llamador: si no existe valor, lanza la excepción en vez de seguir con null.

L109

```
109 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L111

```
111 |   public View visible(UUID id, Jwt jwt) {
```

Aplica alcance por identidad/rol antes de devolver un recurso. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias.

L112

```
112 |     View a = get(id);
```

Se calcula a (a) para reutilizar ese resultado en los pasos siguientes.

L113 a L114

```
113 |     if (!TokenSupport.permits(jwt, false, "ADMIN") &&
      !a.operatorSubject().equals(jwt.getSubject()))
114 |       throw missing();
```

Comprueba ADMIN humano con audiencia/cliente válidos, no solo el texto del rol. Esta condición decide si se entra en la rama siguiente. obtiene sub, identidad estable del usuario autenticado; compara contenido/identidad lógica, no referencias Java; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L115

```
115 |     return a;
```

Entrega a al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: a.

L116

```
116 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L118

```
118 |   public RequestDtos.PageView<View> own(String subject, int page, int size) {
```

Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad; page = page; int es entero de 32 bits para límites y contadores; size = size; int es entero de 32 bits para límites y contadores.

L119 a L120

```
119 |     if (page < 0 || size < 1 || size > 100)
120 |       throw new ResponseStatusException(HttpStatus.BAD_REQUEST);
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. interrumpe la operación con un estado HTTP explícito; 400 indica entrada inválida; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L121 a L123

```
121 |     long count =
122 |         jdbc.queryForObject(
123 |             "select count(*) from assignments where operator_subject=?", Long.class, subject);
```

Consulta assignments contando coincidencias. Se calcula count (count) para reutilizar ese resultado en los pasos siguientes. ejecuta SQL parametrizado y espera un resultado escalar o una fila

L124 a L134

```
124 |     return new RequestDtos.PageView<>(
125 |         jdbc.query(
126 |             "select * from assignments where operator_subject=? order by created_at,id limit ?
      offset ?",
127 |             this::map,
128 |             subject,
129 |             size,
130 |             (long) page * size),
131 |         page,
132 |         size,
133 |         count,
134 |         (int) ((count + size - 1) / size));
```

Consulta assignments con el filtro y proyección escritos. Resultado que recibe el llamador: ejecuta una consulta SQL y transforma filas mediante el mapper; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L135

```
135 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L137

```
137 |   private void event(AssistanceRequest r, Status next, String reason, String actor) {
```

Persiste el cambio de estado con actor/motivo y difiere el log hasta confirmar la transacción. Campos: reason: motivo legible. Tipos y datos: reason = motivo legible; String es texto; no es por sí mismo una referencia de entidad; actor = actor; String es texto; no es por sí mismo una referencia de entidad.

L138

```
138 |     Status previous = r.status;
```

Se calcula previous (previous) para reutilizar ese resultado en los pasos siguientes. Datos implicados: estado del objeto; su enum o CHECK define valores permitidos

L139

```
139 |     r.status = next;
```

En r guarda estado del objeto; su enum o CHECK define valores permitidos. La transición efectiva proviene de next y debe estar autorizada por las validaciones previas. next.

L140

```
140 |     r.updatedAt = now();
```

En r guarda fecha de modificación. Marca el instante de esta operación para ordenar y auditar su evolución.

L141

```
141 |     r.nextEvaluationAt = now();
```

En r guarda próximo trabajo de evaluación. Toma el dato de now() después de las validaciones de este método.

L142

```
142 |     if (next == Status.ASSIGNED) r.nextReconciliationAt = now();
```

Cambios: r.nextReconciliationAt recibe now(). Esta condición decide si se entra en la rama siguiente. ASSIGNED: solicitud aceptada/asignada; Datos implicados: próxima revisión del compromiso.

L143

```
143 |     RequestStatusHistory h = new RequestStatusHistory();
```

Construye RequestStatusHistory en memoria y conserva la referencia en h. Auditoría de cambios de solicitud con actor, motivo, fecha, idempotencia y evidencia. Es persistencia de negocio, distinta de los logs de consola. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L144

```
144 |     h.request = r;
```

En h guarda request. Toma el dato de r después de las validaciones de este método. r.

L145

```
145 |     h.previousStatus = previous;
```

En h guarda estado previo. La transición efectiva proviene de previous y debe estar autorizada por las validaciones previas. previous.

L146

```
146 |     h.newStatus = next;
```

En h guarda estado resultante. La transición efectiva proviene de next y debe estar autorizada por las validaciones previas. next.

L147

```
147 |     h.actorType = actor == null ? "SYSTEM" : "OPERATOR";
```

En h guarda tipo de actor SYSTEM/DRIVER/OPERATOR. Toma el dato de actor == null ? "SYSTEM" : "OPERATOR" después de las validaciones de este método. detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L148

```
148 |     h.actorSubject = actor;
```

En h guarda sub del usuario que produjo el evento. Toma el dato de actor después de las validaciones de este método. actor.

L149

```
149 |     h.reason = reason;
```

En h guarda motivo legible. Toma el dato de reason después de las validaciones de este método. motivo legible.

L150

```
150 |     h.occurredAt = now();
```

En h guarda instante del evento. Toma el dato de now() después de las validaciones de este método.

L151

```
151 |     history.save(h);
```

Solicita persistir la entidad en el contexto JPA.

L152 a L165

```
152 |     org.springframework.transaction.support.TransactionSynchronizationManager
153 |         .registerSynchronization(
154 |             new org.springframework.transaction.support.TransactionSynchronization() {
155 |               @Override
156 |               public void afterCommit() {
157 |                 org.slf4j.LoggerFactory.getLogger(AssignmentStore.class)
158 |                     .info(
159 |                         "Request state changed requestId={} from={} to={} reason={}",
160 |                         r.id,
161 |                         previous,
162 |                         next,
163 |                         reason);
164 |               }
165 |             });
```

Registra un callback del ciclo transaccional; no ejecuta el log antes del commit. Implementa callback afterCommit para anunciar solo cambios confirmados. Se calcula requestId (UUID de solicitud) para reutilizar ese resultado en los pasos siguientes. Datos implicados: identificador UUID del registro

L166

```
166 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L168

```
168 |   private Action action(UUID assignment, String kind, String payload) {
```

Crea una acción pendiente única por asignación y tipo. Si ya existe y no se completó, devuelve la misma; si está completada no crea otra. Tipos y datos: assignment = assignment; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; kind = kind; String es texto; no es por sí mismo una referencia de entidad; payload = contenido normalizado para reintentos; String es texto; no es por sí mismo una referencia de entidad.

L169

```
169 |     UUID id = UUID.randomUUID();
```

Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro. Se calcula id (identificador UUID del registro) para reutilizar ese resultado en los pasos siguientes. genera un nuevo identificador UUID

L170 a L176

```
170 |     jdbc.update(
171 |         "insert into pending_actions(id,assignment_id,kind,status,created_at,updated_at) values
      (?,?,?,'PENDING',?,?) on conflict(assignment_id,kind) do nothing",
172 |         id,
173 |         assignment,
174 |         kind,
175 |         ts(now()),
176 |         ts(now()));
```

Inserta en pending_actions. DO NOTHING evita duplicar la intención/fila. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L177 a L185

```
177 |     return jdbc
178 |         .query(
179 |             "select id from pending_actions where assignment_id=? and kind=? and
      status<>'COMPLETED'",
180 |             (rs, n) -> new Action(rs.getObject(1, UUID.class), assignment, kind, payload),
181 |             assignment,
182 |             kind)
183 |         .stream()
184 |         .findFirst()
185 |         .orElse(null);
```

Consulta pending_actions con el filtro y proyección escritos. Resultado que recibe el llamador: inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente; usa un valor alternativo cuando no hay resultado; ejecuta una consulta SQL y transforma filas mediante el mapper.

L178: ejecuta una consulta SQL y transforma filas mediante el mapper.

L183: inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente.

L185: usa un valor alternativo cuando no hay resultado.

L186

```
186 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L188

```
188 |   public Action propose(UUID requestId) {
```

Inicia la decisión local de seleccionar candidato y persistir una intención RESERVE. Devuelve null cuando todavía no puede proponer. Tipos y datos: requestId = UUID de solicitud; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L189

```
189 |     AssistanceRequest r = lock(requestId);
```

Se calcula r (r) para reutilizar ese resultado en los pasos siguientes.

L190 a L194

```
190 |     if (r.closedAt != null
191 |         || r.resourceReleasePending
192 |         || r.evaluationLeaseUntil.isAfter(now())
193 |         || r.status != Status.PENDING_EVALUATION
194 |         || !r.searchDeadline.isAfter(now())) return null;
```

No propone si la solicitud está cerrada, queda liberación pendiente, otra evaluación conserva su lease, el estado no admite propuestas o venció la búsqueda. Esta condición decide si se entra en la rama siguiente. compara si el instante es estrictamente posterior al otro; PENDING_EVALUATION: solicitud pendiente de evaluación; Datos implicados: fecha de cierre; null indica abierto, liberación remota pendiente que impide reutilización, límite de trabajo de evaluación en curso, estado del objeto; su enum o CHECK define valores permitidos, vencimiento del intento; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue; comprueba presencia antes de usar el dato.

L191: Datos implicados: liberación remota pendiente que impide reutilización; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L192: compara si el instante es estrictamente posterior al otro; Datos implicados: límite de trabajo de evaluación en curso; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L193: PENDING_EVALUATION: solicitud pendiente de evaluación; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L194: compara si el instante es estrictamente posterior al otro; Datos implicados: vencimiento del intento; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L195 a L199

```
195 |     if (jdbc.queryForObject(
196 |             "select count(*) from assignments where request_id=? and (status in
      ('RESERVING','PROPOSED','ACCEPTED'))",
197 |             Long.class,
198 |             requestId)
199 |         > 0) return null;
```

Si ya hay una asignación en reserva, propuesta o aceptación, impide crear otra activa para la misma solicitud. Esta condición decide si se entra en la rama siguiente. ejecuta SQL parametrizado y espera un resultado escalar o una fila.

L200 a L205

```
200 |     if (jdbc.queryForObject(
201 |             "select count(*) from assignments where request_id=? and search_attempt_number=? and
      expires_at is not null",
202 |             Long.class,
203 |             requestId,
204 |             r.searchAttemptNumber)
205 |         >= rule(r, "search", "max-offers-per-attempt")) return null;
```

Cuenta ofertas del intento que llegaron a tener vencimiento; si alcanzó max-offers-per-attempt no publica otra. Esta condición decide si se entra en la rama siguiente. ejecuta SQL parametrizado y espera un resultado escalar o una fila; Datos implicados: número de búsqueda de 1 a 3.

L206

```
206 |     var maybe = evaluations.findFirstByRequestIdOrderByEvaluatedAtDescIdDesc(requestId);
```

Solo intenta proponer usando la evaluación más reciente y de resultado VIABLE; ausencia o resultado distinto no produce reserva. Se calcula maybe (maybe) para reutilizar ese resultado en los pasos siguientes.

L207

```
207 |     if (maybe.isEmpty() || !"VIABLE".equals(maybe.get().result)) return null;
```

La rama condicional retorna sin trabajo si no aplica. Esta condición decide si se entra en la rama siguiente. compara contenido/identidad lógica, no referencias Java; comprueba ausencia de elementos o valor; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L208

```
208 |     var e = maybe.get();
```

Se calcula e (e) para reutilizar ese resultado en los pasos siguientes.

L209

```
209 |     if (e.evaluatedAt.isBefore(now().minusSeconds(300))) return null;
```

Descarta una evaluación de más de 300 segundos antes de emitir una propuesta; no significa reevaluar cada aceptación. Esta condición decide si se entra en la rama siguiente. calcula un instante anterior restando segundos; compara si el instante es estrictamente anterior al otro; Datos implicados: instante de evaluación.

L210 a L217

```
210 |     Set<UUID> tried =
211 |         new HashSet<>(
212 |             jdbc.queryForList(
213 |                 "select tow_truck_id from assignments where request_id=? and
      search_attempt_number=? and (expires_at is not null or created_at>?)",
214 |                 UUID.class,
215 |                 requestId,
216 |                 r.searchAttemptNumber,
217 |                 ts(now().minusSeconds(rule(r, "search", "evaluation-interval-seconds")))));
```

Construye HashSet<> en memoria y conserva la referencia en tried. Los argumentos del constructor determinan los datos iniciales de este objeto. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L218 a L223

```
218 |     Set<UUID> rejectedOperators =
219 |         new HashSet<>(
220 |             jdbc.queryForList(
221 |                 "select distinct operator_id from assignments where request_id=? and
      response_kind='REJECT'",
222 |                 UUID.class,
223 |                 requestId));
```

Construye HashSet<> en memoria y conserva la referencia en rejectedOperators. Los argumentos del constructor determinan los datos iniciales de este objeto. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L224 a L233

```
224 |     var choice =
225 |         candidates.findByEvaluationId(e.id).stream()
226 |             .filter(
227 |                 c ->
228 |                     c.eligible
229 |                         && c.rankingPosition != null
230 |                         && !tried.contains(c.towTruckId)
231 |                         && !rejectedOperators.contains(c.operatorId))
232 |             .sorted(Comparator.comparingInt(c -> c.rankingPosition))
233 |             .findFirst();
```

Elige el candidato elegible de mejor ranking que no tenga grúa ya intentada en esta ronda ni operador que rechazó en cualquier intento. Si no existe, no emite propuesta. Se calcula choice (choice) para reutilizar ese resultado en los pasos siguientes. verifica pertenencia al conjunto o lista; inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente; retiene solo elementos que cumplen la condición; Datos implicados: identificador UUID del registro, aptitud completa del candidato/compromiso, posición entre elegibles, comenzando en 1, UUID de grúa, UUID del perfil operador; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue; comprueba presencia antes de usar el dato

L226: retiene solo elementos que cumplen la condición.

L229: Datos implicados: posición entre elegibles, comenzando en 1; && exige todas las condiciones y deja de evaluar al encontrar una falsa; comprueba presencia antes de usar el dato.

L230: verifica pertenencia al conjunto o lista; Datos implicados: UUID de grúa; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L231: verifica pertenencia al conjunto o lista; Datos implicados: UUID del perfil operador; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L232: Datos implicados: posición entre elegibles, comenzando en 1.

L234

```
234 |     if (choice.isEmpty()) return null;
```

La rama condicional retorna sin trabajo si no aplica. Esta condición decide si se entra en la rama siguiente. comprueba ausencia de elementos o valor.

L235

```
235 |     var c = choice.get();
```

Se calcula c (c) para reutilizar ese resultado en los pasos siguientes.

L236 a L240

```
236 |     if (c.truckLocationUpdatedAt.isBefore(now().minusSeconds(300))
237 |         || c.approachRouteObtainedAt.isBefore(now().minusSeconds(300))
238 |         || c.transportRouteObtainedAt.isBefore(now().minusSeconds(300))
239 |         || Duration.between(now(), r.searchDeadline).getSeconds()
240 |             < rule(r, "search", "minimum-offer-window-seconds")) return null;
```

Comprueba frescura de ubicación y de ambas rutas al emitir, y que reste la ventana mínima de respuesta. Si alguna condición falla, espera otra evaluación. Esta condición decide si se entra en la rama siguiente. calcula un instante anterior restando segundos; compara si el instante es estrictamente anterior al otro; Datos implicados: fecha de posición usada al evaluar, fecha de ruta de aproximación, fecha de ruta de traslado, vencimiento del intento; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L237: calcula un instante anterior restando segundos; compara si el instante es estrictamente anterior al otro; Datos implicados: fecha de ruta de aproximación; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L238: calcula un instante anterior restando segundos; compara si el instante es estrictamente anterior al otro; Datos implicados: fecha de ruta de traslado; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L239: Datos implicados: vencimiento del intento; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L241

```
241 |     // Preserve the planned duration while moving its reference instant to reservation time.
```

Documentación para quien lee el fuente: Preserve the planned duration while moving its reference instant to reservation time. No es una instrucción ejecutable y no impone la regla por sí sola.

L242 a L253

```
242 |     Instant end =
243 |         now()
244 |             .plusSeconds(
245 |                 Math.min(
246 |                         rule(r, "fleet", "hold-seconds"),
247 |                         Duration.between(now(), r.searchDeadline).getSeconds())
248 |                     + rule(r, "execution", "dispatch-deadline-seconds")
249 |                     + c.adjustedApproachSeconds
250 |                     + e.loadingSeconds
251 |                     + e.adjustedTransportSeconds
252 |                     + e.unloadingSeconds
253 |                     + e.operationalMarginSeconds);
```

Calcula fin estimado desde la reserva: suma ventana de respuesta, despacho, aproximación, carga, traslado, descarga y margen. Esa fecha se usa para exigir habilitación suficientemente larga. Se calcula end (end) para reutilizar ese resultado en los pasos siguientes. calcula un instante futuro sumando segundos, sin modificar el instante original; elige el menor valor para respetar el límite más restrictivo; Datos implicados: vencimiento del intento, aproximación ajustada, carga prevista, duración ajustada y redondeada hacia arriba, descarga prevista, contingencia de planificación

L244: calcula un instante futuro sumando segundos, sin modificar el instante original.

L254

```
254 |     var body = mapper.createObjectNode();
```

Se calcula body (body) para reutilizar ese resultado en los pasos siguientes. crea un objeto JSON vacío que se completará como evidencia

L255

```
255 |     body.put("holdSeconds", rule(r, "fleet", "hold-seconds"));
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Añade o reemplaza una propiedad en el objeto JSON o mapa.

L256

```
256 |     body.put("minimumOfferSeconds", rule(r, "search", "minimum-offer-window-seconds"));
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Añade o reemplaza una propiedad en el objeto JSON o mapa.

L257

```
257 |     body.put("requestId", requestId.toString());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Representa el valor como texto; en un nodo JSON produce JSON serializado; añade o reemplaza una propiedad en el objeto JSON o mapa.

L258

```
258 |     body.put("towTruckId", c.towTruckId.toString());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Representa el valor como texto; en un nodo JSON produce JSON serializado; añade o reemplaza una propiedad en el objeto JSON o mapa; Datos implicados: UUID de grúa.

L259

```
259 |     body.put("operatorId", c.operatorId.toString());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Representa el valor como texto; en un nodo JSON produce JSON serializado; añade o reemplaza una propiedad en el objeto JSON o mapa; Datos implicados: UUID del perfil operador.

L260

```
260 |     body.put("vehicleType", r.declaredVehicleType.name());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Añade o reemplaza una propiedad en el objeto JSON o mapa; Datos implicados: copia del tipo de vehículo al solicitar.

L261

```
261 |     body.put("weightKg", r.declaredWeightKg);
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Añade o reemplaza una propiedad en el objeto JSON o mapa; Datos implicados: peso declarado en kg.

L262

```
262 |     body.put("nonRollable", r.mobilityCondition == Mobility.NON_ROLLABLE);
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Añade o reemplaza una propiedad en el objeto JSON o mapa; Datos implicados: condición ROLLABLE/NON_ROLLABLE.

L263

```
263 |     body.put("searchDeadline", r.searchDeadline.toString());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Representa el valor como texto; en un nodo JSON produce JSON serializado; añade o reemplaza una propiedad en el objeto JSON o mapa; Datos implicados: vencimiento del intento.

L264

```
264 |     body.put("estimatedServiceEndAt", end.toString());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa. Representa el valor como texto; en un nodo JSON produce JSON serializado; añade o reemplaza una propiedad en el objeto JSON o mapa.

L265

```
265 |     UUID id = UUID.randomUUID();
```

Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro. Se calcula id (identificador UUID del registro) para reutilizar ese resultado en los pasos siguientes. genera un nuevo identificador UUID

L266 a L276

```
266 |     jdbc.update(
267 |         "insert into assignments(id,request_id,candidate_evaluation_id,tow_truck_id,operator_id,o
      perator_subject,status,created_at,search_attempt_number,reserve_payload) values
      (?,?,?,?,?,?,'RESERVING',?,?,?)",
268 |         id,
269 |         requestId,
270 |         c.id,
271 |         c.towTruckId,
272 |         c.operatorId,
273 |         c.operatorSubject,
274 |         ts(now()),
275 |         r.searchAttemptNumber,
276 |         body.toString());
```

Inserta en assignments. Representa el valor como texto; en un nodo JSON produce JSON serializado; ejecuta escritura SQL parametrizada y devuelve número de filas afectadas; Datos implicados: identificador UUID del registro, UUID de grúa, UUID del perfil operador, sub del operador destinatario, número de búsqueda de 1 a 3.

L277

```
277 |     return action(id, "RESERVE", body.toString());
```

Persiste/recupera intención RESERVE con UUID y payload del intento para que el coordinador haga HTTP. Resultado que recibe el llamador: representa el valor como texto; en un nodo JSON produce JSON serializado.

L278

```
278 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L280

```
280 |   public Action respond(UUID id, String subject, String key, boolean accept, String reason) {
```

Campos: id: identificador UUID del registro; reason: motivo legible. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; subject = subject; String es texto; no es por sí mismo una referencia de entidad; key = key; String es texto; no es por sí mismo una referencia de entidad; accept = accept; boolean es verdadero/falso, sin null; reason = motivo legible; String es texto; no es por sí mismo una referencia de entidad.

L281 a L282

```
281 |     if (key == null || key.isBlank() || key.length() > 100)
282 |       throw new ResponseStatusException(HttpStatus.BAD_REQUEST);
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. comprueba texto vacío o solo espacios; interrumpe la operación con un estado HTTP explícito; 400 indica entrada inválida; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L283

```
283 |     View initial = get(id);
```

Se calcula initial (initial) para reutilizar ese resultado en los pasos siguientes.

L284

```
284 |     if (!initial.operatorSubject().equals(subject)) throw missing();
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. compara contenido/identidad lógica, no referencias Java; ! invierte el booleano de la expresión que sigue.

L285

```
285 |     AssistanceRequest r = lock(initial.requestId());
```

Se calcula r (r) para reutilizar ese resultado en los pasos siguientes.

L286

```
286 |     View a = get(id);
```

Se calcula a (a) para reutilizar ese resultado en los pasos siguientes.

L287 a L288

```
287 |     String kind = accept ? "ACCEPT" : "REJECT",
288 |         fingerprint = RequestService.digest(kind + ":" + (reason == null ? "" : reason.strip()));
```

Distingue ACCEPT/REJECT y hashea tipo más motivo normalizado para idempotencia de respuesta. Se calcula kind (kind) para reutilizar ese resultado en los pasos siguientes. calcula una huella de contenido usada para idempotencia; detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario

L289 a L292

```
289 |     var previous =
290 |         jdbc.queryForMap(
291 |             "select response_key,response_fingerprint,response_kind from assignments where id=?",
292 |             id);
```

Consulta assignments con el filtro y proyección escritos. Se calcula previous (previous) para reutilizar ese resultado en los pasos siguientes.

L293

```
293 |     if (previous.get("response_key") != null) {
```

Esta condición decide si se entra en la rama siguiente. comprueba presencia antes de usar el dato.

L294 a L296

```
294 |       require(
295 |           key.equals(previous.get("response_key"))
296 |               && fingerprint.equals(previous.get("response_fingerprint")));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara contenido/identidad lógica, no referencias Java; && exige todas las condiciones y deja de evaluar al encontrar una falsa

L296: compara contenido/identidad lógica, no referencias Java; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L297

```
297 |       return pending(id);
```

Consulta la acción remota pendiente de la asignación para reanudar el trabajo durable sin crear una intención duplicada.

L298

```
298 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L299 a L303

```
299 |     require(
300 |         r.closedAt == null
301 |             && a.status().equals("PROPOSED")
302 |             && a.expiresAt().isAfter(now())
303 |             && r.searchDeadline.isAfter(now()));
```

Exige propuesta PROPOSED, solicitud abierta y vencimientos de propuesta/búsqueda todavía futuros antes de registrar una respuesta. Alcanzar el instante límite ya no permite aceptar. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara si el instante es estrictamente posterior al otro; compara contenido/identidad lógica, no referencias Java; Datos implicados: fecha de cierre; null indica abierto, vencimiento del intento; && exige todas las condiciones y deja de evaluar al encontrar una falsa; detecta un dato ausente

L301: compara contenido/identidad lógica, no referencias Java; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L302: compara si el instante es estrictamente posterior al otro; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L303: compara si el instante es estrictamente posterior al otro; Datos implicados: vencimiento del intento; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L304 a L310

```
304 |     jdbc.update(
305 |         "update assignments set response_key=?,response_fingerprint=?,response_kind=?,reason=?
      where id=?",
306 |         key,
307 |         fingerprint,
308 |         kind,
309 |         reason,
310 |         id);
```

Actualiza assignments para las filas del filtro. Se calcula response_key (response_key) para reutilizar ese resultado en los pasos siguientes. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L311

```
311 |     // An active proposal retains its admitted route evidence until expiresAt; Fleet locks
      location
```

Documentación para quien lee el fuente: An active proposal retains its admitted route evidence until expiresAt; Fleet locks location No es una instrucción ejecutable y no impone la regla por sí sola.

L312

```
312 |     // while HELD.
```

Documentación para quien lee el fuente: while HELD. No es una instrucción ejecutable y no impone la regla por sí sola.

L313

```
313 |     // Dispatch independently revalidates routes. Acceptance still requires Fleet confirmation
```

Documentación para quien lee el fuente: Dispatch independently revalidates routes. Acceptance still requires Fleet confirmation No es una instrucción ejecutable y no impone la regla por sí sola.

L314

```
314 |     // before its deadline.
```

Documentación para quien lee el fuente: before its deadline. No es una instrucción ejecutable y no impone la regla por sí sola.

L315

```
315 |     if (accept) return action(id, "CONFIRM", null);
```

Persiste la intención CONFIRM; no acepta definitivamente ni recalcula ORS aquí. Solo la respuesta confirmada de Fleet fija ASSIGNED. Esta condición decide si se entra en la rama siguiente. .

L316

```
316 |     jdbc.update("update assignments set status='REJECTED',closed_at=? where id=?", ts(now()),
      id);
```

Actualiza assignments para las filas del filtro. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L317

```
317 |     r.resourceReleasePending = true;
```

En r guarda liberación remota pendiente que impide reutilización. Es una bandera durable de trabajo pendiente: después de reiniciar el sistema debe terminar esta recuperación antes de comprometer recursos incompatibles. true.

L318

```
318 |     event(r, Status.PENDING_EVALUATION, "OPERATOR_REJECTED", subject);
```

PENDING_EVALUATION: solicitud pendiente de evaluación.

L319

```
319 |     return action(id, "RELEASE", null);
```

Persiste/reutiliza la intención RELEASE para liberar la reserva en Fleet. null indica que esta acción no necesita payload de reserva; el HTTP lo ejecuta el coordinador.

L320

```
320 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L322

```
322 |   public Action pending(UUID id) {
```

Busca trabajo durable sin completar; no presupone que Fleet no lo haya aplicado. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L323 a L331

```
323 |     return jdbc
324 |         .query(
325 |             "select p.id,p.kind,a.reserve_payload from pending_actions p join assignments a on
      a.id=p.assignment_id where a.id=? and p.status<>'COMPLETED' order by p.created_at limit 1",
326 |             (rs, n) ->
327 |                 new Action(rs.getObject(1, UUID.class), id, rs.getString(2), rs.getString(3)),
328 |             id)
329 |         .stream()
330 |         .findFirst()
331 |         .orElse(null);
```

Consulta pending_actions, assignments con el filtro y proyección escritos. Resultado que recibe el llamador: inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente; usa un valor alternativo cuando no hay resultado; ejecuta una consulta SQL y transforma filas mediante el mapper.

L324: ejecuta una consulta SQL y transforma filas mediante el mapper.

L329: inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente.

L331: usa un valor alternativo cuando no hay resultado.

L332

```
332 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L334

```
334 |   public List<Action> pendingForRequest(UUID requestId) {
```

Recupera acciones de todas las asignaciones de una solicitud. Campos: requestId: UUID de solicitud. Tipos y datos: requestId = UUID de solicitud; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L335 a L343

```
335 |     return jdbc.query(
336 |         "select p.id,p.assignment_id,p.kind,a.reserve_payload from pending_actions p join
      assignments a on a.id=p.assignment_id where a.request_id=? and p.status<>'COMPLETED' order by
      p.created_at",
337 |         (rs, n) ->
338 |             new Action(
339 |                 rs.getObject(1, UUID.class),
340 |                 rs.getObject(2, UUID.class),
341 |                 rs.getString(3),
342 |                 rs.getString(4)),
343 |         requestId);
```

Consulta pending_actions, assignments con el filtro y proyección escritos. Resultado que recibe el llamador: ejecuta una consulta SQL y transforma filas mediante el mapper.

L344

```
344 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L346

```
346 |   public boolean claim(Action action) {
```

Adquiere por SQL una ventana de 30 s para que otro worker no ejecute simultáneamente la misma acción.

L347 a L352

```
347 |     return jdbc.update(
348 |             "update pending_actions set next_attempt_at=? where id=? and status<>'COMPLETED' and
      next_attempt_at<=?",
349 |             ts(now().plusSeconds(30)),
350 |             action.id(),
351 |             ts(now()))
352 |         == 1;
```

Actualiza pending_actions para las filas del filtro. Resultado que recibe el llamador: calcula un instante futuro sumando segundos, sin modificar el instante original; ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L353

```
353 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L355

```
355 |   public static int retryDelay(int failures) {
```

Escalona la recuperación a 5, 15, 30 y luego 60 segundos. Tipos y datos: failures = failures; int es entero de 32 bits para límites y contadores.

L356

```
356 |     return failures <= 0 ? 5 : failures == 1 ? 15 : failures == 2 ? 30 : 60;
```

Resultado que recibe el llamador: el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L357

```
357 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L359

```
359 |   public void failed(Action action) {
```

Reprograma con backoff estándar sin mínimo adicional del proveedor.

L360

```
360 |     defer(action, 0);
```

Pospone la recuperación; respeta la espera mínima del proveedor y no descarta la intención.

L361

```
361 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L363

```
363 |   public void defer(Action action, int minimum) {
```

Pospone la recuperación; respeta la espera mínima del proveedor y no descarta la intención. Tipos y datos: minimum = minimum; int es entero de 32 bits para límites y contadores.

L364 a L366

```
364 |     Integer failures =
365 |         jdbc.queryForObject(
366 |             "select failure_count from pending_actions where id=?", Integer.class, action.id());
```

Consulta pending_actions con el filtro y proyección escritos. Se calcula failures (failures) para reutilizar ese resultado en los pasos siguientes. ejecuta SQL parametrizado y espera un resultado escalar o una fila

L367

```
367 |     int delay = Math.max(minimum, retryDelay(failures));
```

Usa el mayor de backoff local y espera externa; conserva intención e incrementa fallos. Se calcula delay (delay) para reutilizar ese resultado en los pasos siguientes. elige el mayor valor, por ejemplo para evitar esperas negativas

L368 a L372

```
368 |     jdbc.update(
369 |         "update pending_actions set
      status='RETRYING',failure_count=failure_count+1,updated_at=?,next_attempt_at=? where id=? and
      status<>'COMPLETED'",
370 |         ts(now()),
371 |         ts(now().plusSeconds(delay)),
372 |         action.id());
```

Actualiza pending_actions para las filas del filtro. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. calcula un instante futuro sumando segundos, sin modificar el instante original Calcula un instante futuro sumando segundos, sin modificar el instante original; ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L373

```
373 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L375

```
375 |   public List<Action> dueActions() {
```

Selecciona hasta cien acciones cuyo próximo intento ya venció.

L376 a L384

```
376 |     return jdbc.query(
377 |         "select p.id,p.assignment_id,p.kind,a.reserve_payload from pending_actions p join
      assignments a on a.id=p.assignment_id where p.status<>'COMPLETED' and p.next_attempt_at<=? order
      by p.next_attempt_at,p.id limit 100",
378 |         (rs, n) ->
379 |             new Action(
380 |                 rs.getObject(1, UUID.class),
381 |                 rs.getObject(2, UUID.class),
382 |                 rs.getString(3),
383 |                 rs.getString(4)),
384 |         ts(now()));
```

Consulta pending_actions, assignments con el filtro y proyección escritos. Resultado que recibe el llamador: ejecuta una consulta SQL y transforma filas mediante el mapper.

L385

```
385 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L387

```
387 |   public Action completed(Action action, JsonNode remote) {
```

Procesa respuesta remota durable de RESERVE/CONFIRM/RELEASE y produce siguiente compensación si hace falta. Tipos y datos: remote = remote; JsonNode es árbol JSON navegable usado para snapshots y respuestas externas.

L388

```
388 |     View initial = get(action.assignmentId());
```

Localiza solicitud, bloquea vehículo/solicitud en orden y vuelve a leer asignación bajo ese bloqueo. Se calcula initial (initial) para reutilizar ese resultado en los pasos siguientes.

L389

```
389 |     AssistanceRequest r = lock(initial.requestId());
```

Se calcula r (r) para reutilizar ese resultado en los pasos siguientes.

L390

```
390 |     View a = get(initial.id());
```

Se calcula a (a) para reutilizar ese resultado en los pasos siguientes.

L391 a L393

```
391 |     String actionState =
392 |         jdbc.queryForObject(
393 |             "select status from pending_actions where id=?", String.class, action.id());
```

Consulta pending_actions con el filtro y proyección escritos. Se calcula actionState (actionState) para reutilizar ese resultado en los pasos siguientes. ejecuta SQL parametrizado y espera un resultado escalar o una fila

L394

```
394 |     if ("COMPLETED".equals(actionState)) return pending(a.id());
```

Esta condición decide si se entra en la rama siguiente. compara contenido/identidad lógica, no referencias Java.

L395

```
395 |     String status = remote.path("status").asText();
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a texto; Ruta JSON consultada: status

L396

```
396 |     require(Set.of("HELD", "CONFIRMED", "RELEASED", "EXPIRED").contains(status));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. verifica pertenencia al conjunto o lista; crea un conjunto no modificable y sin duplicados

L397 a L400

```
397 |     jdbc.update(
398 |         "update pending_actions set status='COMPLETED',updated_at=? where id=?",
399 |         ts(now()),
400 |         action.id());
```

Actualiza pending_actions para las filas del filtro. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L401

```
401 |     if (action.kind().equals("RESERVE")) {
```

Esta condición decide si se entra en la rama siguiente. compara contenido/identidad lógica, no referencias Java.

L402 a L408

```
402 |       if (!remote.path("expiresAt").isMissingNode()
403 |           && !remote.path("expiresAt").isNull()
404 |           && (status.equals("HELD") || remote.path("createdHold").asBoolean()))
405 |         jdbc.update(
406 |             "update assignments set expires_at=? where id=?",
407 |             ts(Instant.parse(remote.path("expiresAt").asText())),
408 |             a.id());
```

Actualiza assignments para las filas del filtro. Esta condición decide si se entra en la rama siguiente. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a texto; convierte el nodo a booleano; compara contenido/identidad lógica, no referencias Java; ejecuta escritura SQL parametrizada y devuelve número de filas afectadas; Ruta JSON consultada: expiresAt.expiresAt.createdHold.expiresAt; && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L403: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; Ruta JSON consultada: expiresAt; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L404: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a booleano; compara contenido/identidad lógica, no referencias Java; Ruta JSON consultada: createdHold; && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L409

```
409 |       if (r.closedAt != null) {
```

Esta condición decide si se entra en la rama siguiente. Datos implicados: fecha de cierre; null indica abierto; comprueba presencia antes de usar el dato.

L410

```
410 |         r.resourceReleasePending = true;
```

En r guarda liberación remota pendiente que impide reutilización. Es una bandera durable de trabajo pendiente: después de reiniciar el sistema debe terminar esta recuperación antes de comprometer recursos incompatibles. true.

L411

```
411 |         return action(a.id(), "RELEASE", null);
```

Programa liberación durable de esta asignación en Fleet; no realiza esa llamada dentro de la transacción actual.

L412

```
412 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L413

```
413 |       if (status.equals("HELD")) {
```

Esta condición decide si se entra en la rama siguiente. compara contenido/identidad lógica, no referencias Java.

L414

```
414 |         Instant expiry = Instant.parse(remote.path("expiresAt").asText());
```

Se calcula expiry (expiry) para reutilizar ese resultado en los pasos siguientes. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a texto; Ruta JSON consultada: expiresAt

L415 a L416

```
415 |         if (Duration.between(now(), expiry).getSeconds()
416 |             < rule(r, "search", "minimum-offer-window-seconds")) {
```

Esta condición decide si se entra en la rama siguiente. .

L417 a L418

```
417 |           jdbc.update(
418 |               "update assignments set status='EXPIRED',closed_at=? where id=?", ts(now()),
      a.id());
```

Actualiza assignments para las filas del filtro. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L419

```
419 |           r.resourceReleasePending = true;
```

En r guarda liberación remota pendiente que impide reutilización. Es una bandera durable de trabajo pendiente: después de reiniciar el sistema debe terminar esta recuperación antes de comprometer recursos incompatibles. true.

L420

```
420 |           event(r, Status.PENDING_EVALUATION, "PROPOSAL_WINDOW_TOO_SHORT", null);
```

PENDING_EVALUATION: solicitud pendiente de evaluación.

L421

```
421 |           return action(a.id(), "RELEASE", null);
```

Programa liberación durable de esta asignación en Fleet; no realiza esa llamada dentro de la transacción actual.

L422

```
422 |         }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L423 a L424

```
423 |         jdbc.update(
424 |             "update assignments set status='PROPOSED',expires_at=? where id=?", ts(expiry),
      a.id());
```

Actualiza assignments para las filas del filtro. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L425

```
425 |         event(r, Status.AWAITING_ACCEPTANCE, "PROPOSAL_CREATED", null);
```

AWAITING_ACCEPTANCE: alternativa del ciclo de vida definido en el enum.

L426

```
426 |       } else {
```

Se ejecuta esta rama solo cuando la condición if anterior resultó falsa. Las actualizaciones de ambas ramas son alternativas, no acumulativas.

L427 a L428

```
427 |         jdbc.update(
428 |             "update assignments set status='EXPIRED',closed_at=? where id=?", ts(now()), a.id());
```

Actualiza assignments para las filas del filtro. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L429

```
429 |         event(r, Status.PENDING_EVALUATION, "RESERVATION_EXPIRED", null);
```

PENDING_EVALUATION: solicitud pendiente de evaluación.

L430

```
430 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L431

```
431 |     } else if (action.kind().equals("CONFIRM")) {
```

Compara contenido/identidad lógica, no referencias Java.

L432

```
432 |       if (status.equals("CONFIRMED")) {
```

Esta condición decide si se entra en la rama siguiente. compara contenido/identidad lógica, no referencias Java.

L433

```
433 |         require(r.closedAt == null);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. Datos implicados: fecha de cierre; null indica abierto; detecta un dato ausente

L434

```
434 |         Instant confirmed = Instant.parse(remote.path("confirmedAt").asText());
```

Se calcula confirmed (confirmed) para reutilizar ese resultado en los pasos siguientes. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a texto; Ruta JSON consultada: confirmedAt

L435 a L438

```
435 |         jdbc.update(
436 |             "update assignments set status='ACCEPTED',accepted_at=? where id=?",
437 |             ts(confirmed),
438 |             a.id());
```

Actualiza assignments para las filas del filtro. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L439

```
439 |         if (r.firstAssignedAt == null) r.firstAssignedAt = confirmed;
```

Aplica la confirmación durable: marca ACCEPTED con el instante recibido de Fleet, fija primera aceptación solo si no existía y pasa solicitud a ASSIGNED. Nunca restituye cancelación al reintentar. Esta condición decide si se entra en la rama siguiente. Datos implicados: primera aceptación que impide cancelar incluso durante reemplazos; detecta un dato ausente.

L440

```
440 |         event(r, Status.ASSIGNED, "OPERATOR_ACCEPTED", a.operatorSubject());
```

ASSIGNED: solicitud aceptada/asignada.

L441

```
441 |       } else {
```

Se ejecuta esta rama solo cuando la condición if anterior resultó falsa. Las actualizaciones de ambas ramas son alternativas, no acumulativas.

L442 a L443

```
442 |         jdbc.update(
443 |             "update assignments set status='EXPIRED',closed_at=? where id=?", ts(now()), a.id());
```

Actualiza assignments para las filas del filtro. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L444

```
444 |         r.resourceReleasePending = true;
```

En r guarda liberación remota pendiente que impide reutilización. Es una bandera durable de trabajo pendiente: después de reiniciar el sistema debe terminar esta recuperación antes de comprometer recursos incompatibles. true.

L445

```
445 |         event(r, Status.PENDING_EVALUATION, "CONFIRMATION_NOT_VALID", null);
```

PENDING_EVALUATION: solicitud pendiente de evaluación.

L446

```
446 |         return action(a.id(), "RELEASE", null);
```

Programa liberación durable de esta asignación en Fleet; no realiza esa llamada dentro de la transacción actual.

L447

```
447 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L448

```
448 |     } else {
```

Se ejecuta esta rama solo cuando la condición if anterior resultó falsa. Las actualizaciones de ambas ramas son alternativas, no acumulativas.

L449

```
449 |       require(status.equals("RELEASED") || status.equals("EXPIRED"));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara contenido/identidad lógica, no referencias Java; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera

L450

```
450 |       r.resourceReleasePending = false;
```

En r guarda liberación remota pendiente que impide reutilización. Es una bandera durable de trabajo pendiente: después de reiniciar el sistema debe terminar esta recuperación antes de comprometer recursos incompatibles. false.

L451

```
451 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L452

```
452 |     return null;
```

Devuelve ausencia de resultado/trabajo al llamador y termina esta ejecución del método. No equivale a completar el servicio ni a borrar el registro: la condición previa explica por qué no hay una acción que continuar ahora.

L453

```
453 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L455

```
455 |   public void reserveRejected(Action action) {
```

Cierra el intento de reserva rechazado explícitamente y vuelve a evaluar si la solicitud sigue abierta.

L456

```
456 |     View a = get(action.assignmentId());
```

Se calcula a (a) para reutilizar ese resultado en los pasos siguientes.

L457

```
457 |     AssistanceRequest r = lock(a.requestId());
```

Se calcula r (r) para reutilizar ese resultado en los pasos siguientes.

L458 a L461

```
458 |     jdbc.update(
459 |         "update assignments set status='EXPIRED',closed_at=?,reason='RESOURCE_UNAVAILABLE' where
      id=?",
460 |         ts(now()),
461 |         a.id());
```

Actualiza assignments para las filas del filtro. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L462 a L465

```
462 |     jdbc.update(
463 |         "update pending_actions set status='COMPLETED',updated_at=? where id=?",
464 |         ts(now()),
465 |         action.id());
```

Actualiza pending_actions para las filas del filtro. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L466

```
466 |     if (r.closedAt == null) event(r, Status.PENDING_EVALUATION, "RESOURCE_UNAVAILABLE", null);
```

Esta condición decide si se entra en la rama siguiente. PENDING_EVALUATION: solicitud pendiente de evaluación; Datos implicados: fecha de cierre; null indica abierto; detecta un dato ausente.

L467

```
467 |     else r.resourceReleasePending = false;
```

Cambios: r.resourceReleasePending recibe false. Datos implicados: liberación remota pendiente que impide reutilización.

L468

```
468 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L469

```
469 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 15, 29, 31, 39, 56, 64, 68, 72, 77, 81, 85, 98, 104, 110, 117, 136, 167, 187, 279, 321, 333, 345, 354, 358, 362, 374, 386, 454: separadores sin instrucciones.

9. Ejecución y automatización

El despacho revalida rutas; luego el operador declara etapas en orden. Los reemplazos automáticos son anteriores al despacho.

AutomationJobs (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/AutomationJobs.java

Objetivo y funcionalidad

Dispara recuperación y avance periódico; reúne consultas de estado a Fleet y aísla fallos por solicitud. No actúa sobre etapas posteriores al despacho.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import org.springframework.scheduling.annotation.*;
```

Importa tipos: org.springframework.scheduling.annotation.*. Tareas periódicas y scheduler. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
```

Importa tipos: org.springframework.boot.autoconfigure.condition.ConditionalOnProperty. Arranque/autoconfiguración y condiciones de activación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import com.auxiruta.assistance.routing.FleetClient;
```

Importa tipos: com.auxiruta.assistance.routing.FleetClient. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | @Component
```

Registra componente inyectable.

L10

```
10 | @EnableScheduling
```

Habilita tareas programadas.

L11 a L15

```
11 | @ConditionalOnProperty(
12 |     name = "auxiruta.automation.enabled",
13 |     havingValue = "true",
14 |     matchIfMissing = true)
15 | public class AutomationJobs {
```

Activa bean solo si cumple propiedad configurada. Declara clase AutomationJobs. Dispara recuperación y avance periódico; reúne consultas de estado a Fleet y aísla fallos por solicitud. No actúa sobre etapas posteriores al despacho.

L16

```
16 |   private final AutomationStore store;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L17

```
17 |   private final EvaluationService evaluations;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L18

```
18 |   private final AssignmentCoordinator assignments;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L19

```
19 |   private final FleetClient fleet;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L21 a L25

```
21 |   public AutomationJobs(
22 |       AutomationStore store,
23 |       EvaluationService evaluations,
24 |       AssignmentCoordinator assignments,
25 |       FleetClient fleet) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L26

```
26 |     this.store = store;
```

En this guarda componente que conserva decisiones en transacciones locales. Toma el dato de store después de las validaciones de este método. componente que conserva decisiones en transacciones locales.

L27

```
27 |     this.evaluations = evaluations;
```

En this guarda acceso a evaluaciones persistidas. Toma el dato de evaluations después de las validaciones de este método. acceso a evaluaciones persistidas.

L28

```
28 |     this.assignments = assignments;
```

En this guarda assignments. Toma el dato de assignments después de las validaciones de este método. assignments.

L29

```
29 |     this.fleet = fleet;
```

En this guarda cliente técnico que consulta o modifica reservas en Fleet. Toma el dato de fleet después de las validaciones de este método. cliente técnico que consulta o modifica reservas en Fleet.

L30

```
30 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L32

```
32 |   @org.springframework.context.annotation.Bean
```

Registra el objeto retornado en Spring.

L33

```
33 |   public org.springframework.scheduling.concurrent.ThreadPoolTaskScheduler taskScheduler() {
```

Crea el scheduler que ejecutará los métodos Scheduled; las líneas siguientes fijan hilos y nombres para permitir trabajos periódicos sin bloquear las peticiones HTTP.

L34

```
34 |     var scheduler = new org.springframework.scheduling.concurrent.ThreadPoolTaskScheduler();
```

Construye org.springframework.scheduling.concurrent.ThreadPoolTaskScheduler en memoria y conserva la referencia en scheduler. Los argumentos del constructor determinan los datos iniciales de este objeto. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L35

```
35 |     scheduler.setPoolSize(2);
```

Dos hilos separan recuperación y avance.

L36

```
36 |     scheduler.setThreadNamePrefix("automation-");
```

Prefija los nombres de hilos del scheduler para reconocer en logs qué trabajo proviene de automatización.

L37

```
37 |     return scheduler;
```

Entrega scheduler al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: scheduler.

L38

```
38 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L40 a L42

```
40 |   @org.springframework.context.event.EventListener(
41 |       org.springframework.boot.context.event.ApplicationReadyEvent.class)
42 |   public void resume() {
```

Al arrancar, vuelve a programar trabajo abierto sin reiniciar el recorrido despachado.

L43

```
43 |     store.resumeAfterRestart();
```

Restablece leases de evaluación y agenda trabajo abierto tras reinicio, sin reiniciar el proceso ya despachado.

L44

```
44 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L46

```
46 |   @Scheduled(fixedDelay = 1000, initialDelay = 1000)
```

Tarea periódica: fixedDelay se mide desde fin de ejecución previa. El scheduler invoca el método sin una petición de Postman; fixedDelay espera desde que termina la ejecución anterior.

L47

```
47 |   public void recover() {
```

Trabajo programado de recuperación: procesa acciones remotas pendientes sin requerir que un usuario llame un endpoint.

L48

```
48 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L49

```
49 |       assignments.recoverDue();
```

Busca acciones cuya próxima ejecución ya venció y reintenta el trabajo de recuperación. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L50

```
50 |     } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L51

```
51 |       log("recovery", e);
```

Registra el contexto indicado y la excepción para diagnosticar este trabajo automático. La recuperación no se declara exitosa por escribir el log y las demás solicitudes pueden continuar.

L52

```
52 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L53

```
53 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L55

```
55 |   @Scheduled(fixedDelay = 1000, initialDelay = 2000)
```

Tarea periódica: fixedDelay se mide desde fin de ejecución previa. El scheduler invoca el método sin una petición de Postman; fixedDelay espera desde que termina la ejecución anterior.

L56

```
56 |   public void process() {
```

Tarea que avanza búsquedas, propone recursos y reúne reconciliaciones de Fleet.

L57

```
57 |     List<AutomationStore.Work> checks = new ArrayList<>();
```

Construye ArrayList<> en memoria y conserva la referencia en checks. Los argumentos del constructor determinan los datos iniciales de este objeto. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L58 a L59

```
58 |     for (UUID id : store.due())
59 |       try {
```

Campos: id: identificador UUID del registro. El bucle repite el bloque por cada elemento; el nombre a la izquierda de : es la variable local y a la derecha está la colección. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L60

```
60 |         var work = store.advance(id);
```

Se calcula work (work) para reutilizar ese resultado en los pasos siguientes.

L61

```
61 |         if (work == null) continue;
```

La rama condicional descarta este candidato y sigue con el próximo. Esta condición decide si se entra en la rama siguiente. detecta un dato ausente.

L62

```
62 |         if (work.evaluate()) {
```

Esta condición decide si se entra en la rama siguiente. .

L63

```
63 |           evaluations.evaluate(id);
```

Calcula y persiste evaluación con cobertura, Fleet y rutas ORS; puede dejar pendiente por un fallo temporal. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L64

```
64 |           assignments.propose(id);
```

Selecciona candidato evaluado e inicia reserva/propuesta si siguen vigentes las condiciones. Los argumentos transportan el identificador y los datos validados por este endpoint o trabajo.

L65

```
65 |         } else checks.add(work);
```

Si no corresponde reevaluar candidatos, agrega el trabajo a la lista de comprobaciones del compromiso existente; luego se consulta Fleet en lote.

L66

```
66 |       } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L67

```
67 |         log("request=" + id, e);
```

Registra el contexto indicado y la excepción para diagnosticar este trabajo automático. La recuperación no se declara exitosa por escribir el log y las demás solicitudes pueden continuar.

L68

```
68 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L69 a L70

```
69 |     if (!checks.isEmpty())
70 |       try {
```

Esta condición decide si se entra en la rama siguiente. comprueba ausencia de elementos o valor; ! invierte el booleano de la expresión que sigue.

L71 a L72

```
71 |         var states =
72 |             fleet.statuses(checks.stream().map(AutomationStore.Work::assignmentId).toList());
```

Transforma cada elemento al valor/tipo de salida. Materializa la secuencia en lista. Se calcula states (states) para reutilizar ese resultado en los pasos siguientes. inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente; transforma cada elemento al tipo/valor de salida; materializa el resultado de la secuencia en una lista

L73 a L82

```
73 |         if (states != null)
74 |           for (var work : checks)
75 |             states.stream()
76 |                 .filter(
77 |                     s ->
78 |                         work.assignmentId()
79 |                             .toString()
80 |                             .equals(s.path("assignmentAttemptId").asText()))
81 |                 .findFirst()
82 |                 .ifPresent(s -> store.reconcile(work, s));
```

Cruza respuesta por UUID; ausencia de respuesta no prueba liberación. Esta condición decide si se entra en la rama siguiente. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a texto; compara contenido/identidad lógica, no referencias Java; inicia una secuencia de procesamiento de elementos, sin modificar la colección fuente; retiene solo elementos que cumplen la condición; representa el valor como texto; en un nodo JSON produce JSON serializado; Ruta JSON consultada: assignmentAttemptId; comprueba presencia antes de usar el dato.

L76: retiene solo elementos que cumplen la condición.

L79: representa el valor como texto; en un nodo JSON produce JSON serializado.

L80: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a texto; compara contenido/identidad lógica, no referencias Java; Ruta JSON consultada: assignmentAttemptId.

L83

```
83 |       } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L84

```
84 |         log("fleet-reconciliation", e);
```

Registra el contexto indicado y la excepción para diagnosticar este trabajo automático. La recuperación no se declara exitosa por escribir el log y las demás solicitudes pueden continuar.

L85

```
85 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L86

```
86 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L88

```
88 |   private void log(String task, Exception e) {
```

Tipos y datos: task = task; String es texto; no es por sí mismo una referencia de entidad.

L89 a L90

```
89 |     org.slf4j.LoggerFactory.getLogger(getClass())
90 |         .warn("Automatic task deferred task={} type={}", task, e.getClass().getSimpleName());
```

Registra los campos indicados, sin token ni secreto. Se calcula task (task) para reutilizar ese resultado en los pasos siguientes.

L91

```
91 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L92

```
92 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 8, 20, 31, 39, 45, 54, 87: separadores sin instrucciones.

AutomationStore (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/AutomationStore.java`

Objetivo y funcionalidad

Máquina de decisiones persistentes: plazos, propuestas vencidas, reemplazos y reconciliación. No realiza HTTP dentro de sus transacciones.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.sql.Timestamp;
```

Importa tipos: java.sql.Timestamp. Timestamp y ResultSet para puentear fechas/filas de JDBC con objetos Java. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.springframework.jdbc.core.JdbcTemplate;
```

Importa tipos: org.springframework.jdbc.core.JdbcTemplate. JdbcTemplate para SQL parametrizado, resultados y acciones durables. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | import com.fasterxml.jackson.databind.*;
```

Importa tipos: com.fasterxml.jackson.databind.*. Lectura/escritura JSON, nodos y conversión a DTO. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12 a L13

```
12 | /** Durable planning and local decisions. No network calls in these transactions. */
13 | @Service
```

Documentación para quien lee el fuente: Durable planning and local decisions. No network calls in these transactions. */ @Service No es una instrucción ejecutable y no impone la regla por sí sola.

L14

```
14 | @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L15

```
15 | public class AutomationStore {
```

Declara clase AutomationStore. Máquina de decisiones persistentes: plazos, propuestas vencidas, reemplazos y reconciliación. No realiza HTTP dentro de sus transacciones.

L16

```
16 |   public record Work(UUID requestId, UUID assignmentId, boolean evaluate) {}
```

Declara record inmutable Work. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: requestId: UUID de solicitud. Máquina de decisiones persistentes: plazos, propuestas vencidas, reemplazos y reconciliación. No realiza HTTP dentro de sus transacciones. Tipos y datos: requestId = UUID de solicitud; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; assignmentId = assignmentId; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; evaluate = evaluate; boolean es verdadero/falso, sin null.

L18

```
18 |   private final JdbcTemplate jdbc;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: jdbc = ejecutor SQL parametrizado; JdbcTemplate es ejecutor de SQL parametrizado de Spring dentro de transacciones locales.

L19

```
19 |   private final RequestRepository requests;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L20

```
20 |   private final HistoryRepository history;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L21

```
21 |   private final EvaluationRepository evaluations;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L22

```
22 |   private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L23

```
23 |   private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L25 a L31

```
25 |   public AutomationStore(
26 |       JdbcTemplate jdbc,
27 |       RequestRepository requests,
28 |       HistoryRepository history,
29 |       EvaluationRepository evaluations,
30 |       ObjectMapper mapper,
31 |       Clock clock) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada. Tipos y datos: jdbc = ejecutor SQL parametrizado; JdbcTemplate es ejecutor de SQL parametrizado de Spring dentro de transacciones locales; mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java; clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L32

```
32 |     this.jdbc = jdbc;
```

En this guarda ejecutor SQL parametrizado. Toma el dato de jdbc después de las validaciones de este método. ejecutor SQL parametrizado.

L33

```
33 |     this.requests = requests;
```

En this guarda repositorio de solicitudes. Toma el dato de requests después de las validaciones de este método. repositorio de solicitudes.

L34

```
34 |     this.history = history;
```

En this guarda historial persistente de cambios. Toma el dato de history después de las validaciones de este método. historial persistente de cambios.

L35

```
35 |     this.evaluations = evaluations;
```

En this guarda acceso a evaluaciones persistidas. Toma el dato de evaluations después de las validaciones de este método. acceso a evaluaciones persistidas.

L36

```
36 |     this.mapper = mapper;
```

En this guarda conversor JSON de Jackson. Toma el dato de mapper después de las validaciones de este método. conversor JSON de Jackson.

L37

```
37 |     this.clock = clock;
```

En this guarda reloj usado para evaluar plazos. Toma el dato de clock después de las validaciones de este método. reloj usado para evaluar plazos.

L38

```
38 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L40

```
40 |   private Instant now() {
```

Obtiene el instante del reloj inyectado; truncar a microsegundos mantiene consistencia con PostgreSQL.

L41

```
41 |     return clock.instant().truncatedTo(java.time.temporal.ChronoUnit.MICROS);
```

Resultado que recibe el llamador: lee el instante del reloj inyectado.

L42

```
42 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L44

```
44 |   private static Timestamp ts(Instant t) {
```

Convierte Instant a Timestamp JDBC; la variante nullable preserva null. Tipos y datos: t = t; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo.

L45

```
45 |     return Timestamp.from(t);
```

Adapta un Instant Java a Timestamp para un parámetro JDBC; conserva el instante y no convierte a una fecha local arbitraria.

L46

```
46 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L48

```
48 |   private int rule(AssistanceRequest r, String group, String key) {
```

Tipos y datos: group = group; String es texto; no es por sí mismo una referencia de entidad; key = key; String es texto; no es por sí mismo una referencia de entidad.

L49

```
49 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L50

```
50 |       return mapper.readTree(r.configurationSnapshot).path(group).path(key).asInt();
```

Resultado que recibe el llamador: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a entero; analiza texto JSON para consultar su estructura; Datos implicados: JSON de parámetros vigente al crear.

L51

```
51 |     } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L52

```
52 |       throw new IllegalStateException("Invalid rules");
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L53

```
53 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L54

```
54 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L56

```
56 |   private AssistanceRequest lock(UUID id) {
```

Adquiere bloqueos en orden para serializar operaciones competidoras. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L57

```
57 |     UUID vehicle = requests.vehicleId(id).orElseThrow();
```

Si no encuentra resultado lanza la excepción indicada. Se calcula vehicle (vehicle) para reutilizar ese resultado en los pasos siguientes. si no existe valor, lanza la excepción en vez de seguir con null

L58

```
58 |     jdbc.queryForObject("select id from vehicles where id=? for update", UUID.class, vehicle);
```

Consulta vehicles con el filtro y proyección escritos. Bloquea las filas durante la transacción. Se calcula id (identificador UUID del registro) para reutilizar ese resultado en los pasos siguientes. Ejecuta SQL parametrizado y espera un resultado escalar o una fila.

L59

```
59 |     return requests.lockById(id).orElseThrow();
```

Si no encuentra resultado lanza la excepción indicada. Resultado que recibe el llamador: si no existe valor, lanza la excepción en vez de seguir con null.

L60

```
60 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L62

```
62 |   public void resumeAfterRestart() {
```

Restablece leases de evaluación y agenda trabajo abierto tras reinicio, sin reiniciar el proceso ya despachado.

L63 a L65

```
63 |     jdbc.update(
64 |         "update assistance_requests set
     evaluation_lease_until='1970-01-01T00:00:00Z',next_evaluation_at=? where closed_at is null and
     status in
     ('REQUESTED','PENDING_EVALUATION','AWAITING_ACCEPTANCE','SEARCHING_ALTERNATIVE','ASSIGNED')",
65 |         ts(now()));
```

Actualiza assistance_requests para las filas del filtro. Se calcula evaluation_lease_until (evaluation_lease_until) para reutilizar ese resultado en los pasos siguientes. el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L66

```
66 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L68

```
68 |   public List<UUID> due() {
```

Selecciona trabajo abierto y vencido para procesar por lotes.

L69 a L72

```
69 |     return jdbc.queryForList(
70 |         "select id from assistance_requests where closed_at is null and status in
     ('REQUESTED','PENDING_EVALUATION','AWAITING_ACCEPTANCE','SEARCHING_ALTERNATIVE','ASSIGNED') and
     next_evaluation_at<=? order by created_at,id limit 100",
71 |         UUID.class,
72 |         ts(now()));
```

Consulta assistance_requests con el filtro y proyección escritos.

L73

```
73 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L75

```
75 |   private void event(AssistanceRequest r, Status next, String reason) {
```

Persiste el cambio de estado con actor/motivo y difiere el log hasta confirmar la transacción. Campos: reason: motivo legible. Tipos y datos: reason = motivo legible; String es texto; no es por sí mismo una referencia de entidad.

L76

```
76 |     Status old = r.status;
```

Se calcula old (old) para reutilizar ese resultado en los pasos siguientes. Datos implicados: estado del objeto; su enum o CHECK define valores permitidos

L77

```
77 |     r.status = next;
```

En r guarda estado del objeto; su enum o CHECK define valores permitidos. La transición efectiva proviene de next y debe estar autorizada por las validaciones previas. next.

L78

```
78 |     r.updatedAt = now();
```

En r guarda fecha de modificación. Marca el instante de esta operación para ordenar y auditar su evolución.

L79

```
79 |     RequestStatusHistory h = new RequestStatusHistory();
```

Construye RequestStatusHistory en memoria y conserva la referencia en h. Auditoría de cambios de solicitud con actor, motivo, fecha, idempotencia y evidencia. Es persistencia de negocio, distinta de los logs de consola. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L80

```
80 |     h.request = r;
```

En h guarda request. Toma el dato de r después de las validaciones de este método. r.

L81

```
81 |     h.previousStatus = old;
```

En h guarda estado previo. La transición efectiva proviene de old y debe estar autorizada por las validaciones previas. old.

L82

```
82 |     h.newStatus = next;
```

En h guarda estado resultante. La transición efectiva proviene de next y debe estar autorizada por las validaciones previas. next.

L83

```
83 |     h.actorType = "SYSTEM";
```

En h guarda tipo de actor SYSTEM/DRIVER/OPERATOR. Toma el dato de "SYSTEM" después de las validaciones de este método.

L84

```
84 |     h.reason = reason;
```

En h guarda motivo legible. Toma el dato de reason después de las validaciones de este método. motivo legible.

L85

```
85 |     h.occurredAt = now();
```

En h guarda instante del evento. Toma el dato de now() después de las validaciones de este método.

L86

```
86 |     history.save(h);
```

Solicita persistir la entidad en el contexto JPA.

L87 a L100

```
87 |     org.springframework.transaction.support.TransactionSynchronizationManager
88 |         .registerSynchronization(
89 |             new org.springframework.transaction.support.TransactionSynchronization() {
90 |               @Override
91 |               public void afterCommit() {
92 |                 org.slf4j.LoggerFactory.getLogger(AutomationStore.class)
93 |                     .info(
94 |                         "Automatic transition requestId={} from={} to={} reason={}",
95 |                         r.id,
96 |                         old,
97 |                         next,
98 |                         reason);
99 |               }
100 |             });
```

Registra un callback del ciclo transaccional; no ejecuta el log antes del commit. Implementa callback afterCommit para anunciar solo cambios confirmados. Se calcula requestId (UUID de solicitud) para reutilizar ese resultado en los pasos siguientes. Datos implicados: identificador UUID del registro

L101

```
101 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L103

```
103 |   private long pending(UUID id, String kind) {
```

Busca trabajo durable sin completar; no presupone que Fleet no lo haya aplicado. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; kind = kind; String es texto; no es por sí mismo una referencia de entidad.

L104 a L108

```
104 |     return jdbc.queryForObject(
105 |         "select count(*) from pending_actions p join assignments a on a.id=p.assignment_id where
      a.request_id=? and p.status<>'COMPLETED'"
106 |             + (kind == null ? "" : " and p.kind=?"),
107 |         Long.class,
108 |         kind == null ? new Object[] {id} : new Object[] {id, kind});
```

Consulta pending_actions, assignments contando coincidencias. Resultado que recibe el llamador: ejecuta SQL parametrizado y espera un resultado escalar o una fila; detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L109

```
109 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L111

```
111 |   private void release(UUID assignment) {
```

Tipos y datos: assignment = assignment; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L112 a L117

```
112 |     jdbc.update(
113 |         "insert into pending_actions(id,assignment_id,kind,status,created_at,updated_at) values
      (?,?,'RELEASE','PENDING',?,?) on conflict(assignment_id,kind) do nothing",
114 |         UUID.randomUUID(),
115 |         assignment,
116 |         ts(now()),
117 |         ts(now()));
```

Genera ID nuevo en la aplicación. Inserta en pending_actions. DO NOTHING evita duplicar la intención/fila. Genera un nuevo identificador UUID; ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L114: genera un nuevo identificador UUID.

L118

```
118 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L120

```
120 |   private void close(AssistanceRequest r, String reason) {
```

Campos: reason: motivo legible. Tipos y datos: reason = motivo legible; String es texto; no es por sí mismo una referencia de entidad.

L121 a L124

```
121 |     for (var row :
122 |         jdbc.queryForList(
123 |             "select id,status,expires_at from assignments where request_id=? and status in
      ('RESERVING','PROPOSED')",
124 |             r.id)) {
```

Consulta assignments con el filtro y proyección escritos. El bucle repite el bloque por cada elemento; el nombre a la izquierda de : es la variable local y a la derecha está la colección. Datos implicados: identificador UUID del registro; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L125

```
125 |       UUID id = (UUID) row.get("id");
```

Campos: id: identificador UUID del registro. Se calcula id (identificador UUID del registro) para reutilizar ese resultado en los pasos siguientes.

L126

```
126 |       Timestamp expires = (Timestamp) row.get("expires_at");
```

Se calcula expires (expires) para reutilizar ese resultado en los pasos siguientes.

L127 a L128

```
127 |       String state =
128 |           expires != null && !expires.toInstant().isAfter(now()) ? "EXPIRED" : "CANCELLED";
```

Se calcula state (state) para reutilizar ese resultado en los pasos siguientes. compara si el instante es estrictamente posterior al otro; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue; comprueba presencia antes de usar el dato; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario

L129

```
129 |       jdbc.update("update assignments set status=?,closed_at=? where id=?", state, ts(now()),
      id);
```

Actualiza assignments para las filas del filtro. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L130

```
130 |       r.resourceReleasePending = true;
```

En r guarda liberación remota pendiente que impide reutilización. Es una bandera durable de trabajo pendiente: después de reiniciar el sistema debe terminar esta recuperación antes de comprometer recursos incompatibles. true.

L131 a L135

```
131 |       if (jdbc.queryForObject(
132 |               "select count(*) from pending_actions where assignment_id=? and kind='RESERVE' and
      status<>'COMPLETED'",
133 |               Long.class,
134 |               id)
135 |           == 0) release(id);
```

Consulta pending_actions contando coincidencias. Esta condición decide si se entra en la rama siguiente. ejecuta SQL parametrizado y espera un resultado escalar o una fila.

L136

```
136 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L137

```
137 |     r.replacementPending = false;
```

En r guarda necesidad de iniciar reemplazo después de liberar. Es una bandera durable de trabajo pendiente: después de reiniciar el sistema debe terminar esta recuperación antes de comprometer recursos incompatibles. false.

L138

```
138 |     r.closedAt = now();
```

En r guarda fecha de cierre; null indica abierto. Toma el dato de now() después de las validaciones de este método.

L139

```
139 |     r.closureReasonCode = reason;
```

En r guarda causa codificada del cierre. Toma el dato de reason después de las validaciones de este método. motivo legible.

L140

```
140 |     event(r, Status.UNFULFILLED, reason);
```

UNFULFILLED: alternativa del ciclo de vida definido en el enum.

L141

```
141 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L143

```
143 |   private String deadlineReason(AssistanceRequest r) {
```

Distingue falta de candidatos de una dependencia que no pudo resolverse.

L144

```
144 |     if (pending(r.id, null) > 0) return "DEPENDENCY_TIMEOUT";
```

Esta condición decide si se entra en la rama siguiente. Datos implicados: identificador UUID del registro.

L145 a L154

```
145 |     return evaluations
146 |         .findFirstByRequestIdOrderByEvaluatedAtDescIdDesc(r.id)
147 |         .map(
148 |             e ->
149 |                 "PENDING_DEPENDENCY".equals(e.result)
150 |                     ? "DEPENDENCY_TIMEOUT"
151 |                     : "NO_AVAILABLE_RESOURCE".equals(e.reasonCode)
152 |                         ? "NO_AVAILABLE_RESOURCE"
153 |                         : "NO_VIABLE_CANDIDATE")
154 |         .orElse("DEPENDENCY_TIMEOUT");
```

Transforma cada elemento al valor/tipo de salida. Recupera evaluación más reciente por fecha e ID. Resultado que recibe el llamador: compara contenido/identidad lógica, no referencias Java; transforma cada elemento al tipo/valor de salida; usa un valor alternativo cuando no hay resultado; Datos implicados: identificador UUID del registro, resultado de evaluación, distinto del estado de solicitud, causa codificada; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L146: Datos implicados: identificador UUID del registro.

L147: transforma cada elemento al tipo/valor de salida.

L151: compara contenido/identidad lógica, no referencias Java; Datos implicados: causa codificada.

L154: usa un valor alternativo cuando no hay resultado.

L155

```
155 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L157

```
157 |   public Work advance(UUID id) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L158

```
158 |     AssistanceRequest r = lock(id);
```

Serializa cambio mediante bloqueo de vehículo/solicitud antes de validar precondiciones. Se calcula r (r) para reutilizar ese resultado en los pasos siguientes.

L159

```
159 |     Instant now = now();
```

Se calcula now (instante actual obtenido del reloj inyectado) para reutilizar ese resultado en los pasos siguientes.

L160 a L168

```
160 |     if (r.closedAt != null
161 |         || !Set.of(
162 |                 Status.REQUESTED,
163 |                 Status.PENDING_EVALUATION,
164 |                 Status.AWAITING_ACCEPTANCE,
165 |                 Status.SEARCHING_ALTERNATIVE,
166 |                 Status.ASSIGNED)
167 |             .contains(r.status)
168 |         || r.nextEvaluationAt.isAfter(now)) return null;
```

La rama condicional retorna sin trabajo si no aplica. Esta condición decide si se entra en la rama siguiente. compara si el instante es estrictamente posterior al otro; verifica pertenencia al conjunto o lista; crea un conjunto no modificable y sin duplicados; REQUESTED: alternativa del ciclo de vida definido en el enum; PENDING_EVALUATION: solicitud pendiente de evaluación; AWAITING_ACCEPTANCE: alternativa del ciclo de vida definido en el enum; SEARCHING_ALTERNATIVE: alternativa del ciclo de vida definido en el enum; ASSIGNED: solicitud aceptada/asignada; Datos implicados: fecha de cierre; null indica abierto, estado del objeto; su enum o CHECK define valores permitidos, próximo trabajo de evaluación; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue; comprueba presencia antes de usar el dato.

L161: crea un conjunto no modificable y sin duplicados; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L167: verifica pertenencia al conjunto o lista; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos.

L168: compara si el instante es estrictamente posterior al otro; Datos implicados: próximo trabajo de evaluación; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L169

```
169 |     r.nextEvaluationAt = now.plusSeconds(5);
```

En r guarda próximo trabajo de evaluación. Toma el dato de now.plusSeconds(5) después de las validaciones de este método. calcula un instante futuro sumando segundos, sin modificar el instante original.

L170

```
170 |     // A remote confirmation may already have committed: never infer nonacceptance from elapsed
```

Documentación para quien lee el fuente: A remote confirmation may already have committed: never infer nonacceptance from elapsed No es una instrucción ejecutable y no impone la regla por sí sola.

L171

```
171 |     // time.
```

Documentación para quien lee el fuente: time. No es una instrucción ejecutable y no impone la regla por sí sola.

L172

```
172 |     if (pending(id, "CONFIRM") > 0) return null;
```

No cierra ni reemplaza mientras existe confirmación sin resolver; el paso del tiempo no prueba que el operador no haya aceptado. Esta condición decide si se entra en la rama siguiente. .

L173 a L175

```
173 |     var accepted =
174 |         jdbc.queryForList(
175 |             "select id,accepted_at from assignments where request_id=? and status='ACCEPTED'",
      id);
```

Consulta assignments con el filtro y proyección escritos. Se calcula accepted (accepted) para reutilizar ese resultado en los pasos siguientes.

L176

```
176 |     if (r.status == Status.ASSIGNED) {
```

Esta condición decide si se entra en la rama siguiente. ASSIGNED: solicitud aceptada/asignada; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos.

L177

```
177 |       if (accepted.isEmpty()) return null;
```

La rama condicional retorna sin trabajo si no aplica. Esta condición decide si se entra en la rama siguiente. comprueba ausencia de elementos o valor.

L178

```
178 |       var a = accepted.get(0);
```

Se calcula a (a) para reutilizar ese resultado en los pasos siguientes.

L179

```
179 |       UUID assignment = (UUID) a.get("id");
```

Se calcula assignment (assignment) para reutilizar ese resultado en los pasos siguientes.

L180 a L183

```
180 |       Instant deadline =
181 |           ((Timestamp) a.get("accepted_at"))
182 |               .toInstant()
183 |               .plusSeconds(rule(r, "execution", "dispatch-deadline-seconds"));
```

Se calcula deadline (deadline) para reutilizar ese resultado en los pasos siguientes. calcula un instante futuro sumando segundos, sin modificar el instante original

L183: calcula un instante futuro sumando segundos, sin modificar el instante original.

L184

```
184 |       if (!deadline.isAfter(now)) {
```

Esta condición decide si se entra en la rama siguiente. compara si el instante es estrictamente posterior al otro; ! invierte el booleano de la expresión que sigue.

L185

```
185 |         replace(r, assignment, "DISPATCH_TIMEOUT");
```

Cierra la asignación previa y crea intención de liberación antes de buscar reemplazo.

L186

```
186 |         return null;
```

Devuelve ausencia de resultado/trabajo al llamador y termina esta ejecución del método. No equivale a completar el servicio ni a borrar el registro: la condición previa explica por qué no hay una acción que continuar ahora.

L187

```
187 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L188

```
188 |       if (!r.nextReconciliationAt.isAfter(now)) {
```

Esta condición decide si se entra en la rama siguiente. compara si el instante es estrictamente posterior al otro; Datos implicados: próxima revisión del compromiso; ! invierte el booleano de la expresión que sigue.

L189 a L190

```
189 |         r.nextReconciliationAt =
190 |             now.plusSeconds(rule(r, "fleet", "reconciliation-interval-seconds"));
```

En r guarda próxima revisión del compromiso. Toma el dato de now.plusSeconds(rule(r, "fleet", "reconciliation-interval-seconds")) después de las validaciones de este método. calcula un instante futuro sumando segundos, sin modificar el instante original.

L191 a L192

```
191 |         r.nextEvaluationAt =
192 |             deadline.isBefore(r.nextReconciliationAt) ? deadline : r.nextReconciliationAt;
```

En r guarda próximo trabajo de evaluación. Toma el dato de deadline.isBefore(r.nextReconciliationAt) ? deadline : r.nextReconciliationAt después de las validaciones de este método. compara si el instante es estrictamente anterior al otro; Datos implicados: próxima revisión del compromiso; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L193

```
193 |         return new Work(id, assignment, false);
```

Devuelve trabajo de reconciliación del compromiso existente. false evita iniciar una evaluación de candidatos libres para una solicitud ya asignada.

L194

```
194 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L195 a L196

```
195 |       r.nextEvaluationAt =
196 |           deadline.isBefore(r.nextReconciliationAt) ? deadline : r.nextReconciliationAt;
```

En r guarda próximo trabajo de evaluación. Toma el dato de deadline.isBefore(r.nextReconciliationAt) ? deadline : r.nextReconciliationAt después de las validaciones de este método. compara si el instante es estrictamente anterior al otro; Datos implicados: próxima revisión del compromiso; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L197

```
197 |       return null;
```

Devuelve ausencia de resultado/trabajo al llamador y termina esta ejecución del método. No equivale a completar el servicio ni a borrar el registro: la condición previa explica por qué no hay una acción que continuar ahora.

L198

```
198 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L199 a L202

```
199 |     if (r.status == Status.SEARCHING_ALTERNATIVE
200 |         && r.replacementPending
201 |         && !r.resourceReleasePending
202 |         && pending(id, null) == 0) {
```

Esta condición decide si se entra en la rama siguiente. SEARCHING_ALTERNATIVE: alternativa del ciclo de vida definido en el enum; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos, necesidad de iniciar reemplazo después de liberar, liberación remota pendiente que impide reutilización; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L200: Datos implicados: necesidad de iniciar reemplazo después de liberar; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L201: Datos implicados: liberación remota pendiente que impide reutilización; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L202: && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L203

```
203 |       if (r.searchAttemptNumber >= 1 + rule(r, "search", "max-reassignments")) {
```

Cierra al agotar un intento inicial más los dos reemplazos admitidos. Se evalúa después de liberar los recursos previos. Esta condición decide si se entra en la rama siguiente. Datos implicados: número de búsqueda de 1 a 3.

L204

```
204 |         close(r, "REASSIGNMENT_LIMIT_REACHED");
```

Cierra sin cumplir la solicitud porque alcanzó el cupo de reemplazos; no inicia otra búsqueda fuera del compromiso del MVP.

L205

```
205 |         return null;
```

Devuelve ausencia de resultado/trabajo al llamador y termina esta ejecución del método. No equivale a completar el servicio ni a borrar el registro: la condición previa explica por qué no hay una acción que continuar ahora.

L206

```
206 |       }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L207

```
207 |       r.replacementPending = false;
```

En r guarda necesidad de iniciar reemplazo después de liberar. Es una bandera durable de trabajo pendiente: después de reiniciar el sistema debe terminar esta recuperación antes de comprometer recursos incompatibles. false.

L208

```
208 |       r.searchAttemptNumber++;
```

Inicia el siguiente intento después de la liberación: incrementa contador y concede un nuevo plazo de búsqueda; conserva firstAssignedAt. Datos implicados: número de búsqueda de 1 a 3.

L209

```
209 |       r.searchDeadline = now.plusSeconds(rule(r, "search", "attempt-timeout-seconds"));
```

En r guarda vencimiento del intento. Toma el dato de now.plusSeconds(rule(r, "search", "attempt-timeout-seconds")) después de las validaciones de este método. calcula un instante futuro sumando segundos, sin modificar el instante original.

L210

```
210 |       event(r, Status.PENDING_EVALUATION, "REPLACEMENT_SEARCH_STARTED");
```

PENDING_EVALUATION: solicitud pendiente de evaluación.

L211

```
211 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L212 a L214

```
212 |     if (r.status == Status.SEARCHING_ALTERNATIVE
213 |         && r.replacementPending
214 |         && r.resourceReleasePending) return null;
```

Suspende el reemplazo hasta confirmar la liberación remota para no comprometer dos recursos. Esta condición decide si se entra en la rama siguiente. SEARCHING_ALTERNATIVE: alternativa del ciclo de vida definido en el enum; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos, necesidad de iniciar reemplazo después de liberar, liberación remota pendiente que impide reutilización; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L213: Datos implicados: necesidad de iniciar reemplazo después de liberar; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L214: Datos implicados: liberación remota pendiente que impide reutilización; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L215

```
215 |     if (!r.searchDeadline.isAfter(now)) {
```

Esta condición decide si se entra en la rama siguiente. compara si el instante es estrictamente posterior al otro; Datos implicados: vencimiento del intento; ! invierte el booleano de la expresión que sigue.

L216

```
216 |       close(r, deadlineReason(r));
```

Cierra la búsqueda al vencer su ventana y calcula un motivo acorde al estado de la solicitud.

L217

```
217 |       return null;
```

Devuelve ausencia de resultado/trabajo al llamador y termina esta ejecución del método. No equivale a completar el servicio ni a borrar el registro: la condición previa explica por qué no hay una acción que continuar ahora.

L218

```
218 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L219

```
219 |     if (r.resourceReleasePending || pending(id, null) > 0) return null;
```

La rama condicional retorna sin trabajo si no aplica. Esta condición decide si se entra en la rama siguiente. Datos implicados: liberación remota pendiente que impide reutilización; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L220 a L222

```
220 |     var proposals =
221 |         jdbc.queryForList(
222 |             "select id,expires_at from assignments where request_id=? and status='PROPOSED'",
      id);
```

Consulta assignments con el filtro y proyección escritos. Se calcula proposals (proposals) para reutilizar ese resultado en los pasos siguientes.

L223

```
223 |     if (!proposals.isEmpty()) {
```

Esta condición decide si se entra en la rama siguiente. comprueba ausencia de elementos o valor; ! invierte el booleano de la expresión que sigue.

L224

```
224 |       var a = proposals.get(0);
```

Se calcula a (a) para reutilizar ese resultado en los pasos siguientes.

L225

```
225 |       Instant expires = ((Timestamp) a.get("expires_at")).toInstant();
```

Se calcula expires (expires) para reutilizar ese resultado en los pasos siguientes.

L226

```
226 |       if (!expires.isAfter(now)) {
```

Esta condición decide si se entra en la rama siguiente. compara si el instante es estrictamente posterior al otro; ! invierte el booleano de la expresión que sigue.

L227

```
227 |         UUID assignment = (UUID) a.get("id");
```

Se calcula assignment (assignment) para reutilizar ese resultado en los pasos siguientes.

L228 a L229

```
228 |         jdbc.update(
229 |             "update assignments set status='EXPIRED',closed_at=? where id=?", ts(now),
      assignment);
```

Actualiza assignments para las filas del filtro. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L230

```
230 |         r.resourceReleasePending = true;
```

En r guarda liberación remota pendiente que impide reutilización. Es una bandera durable de trabajo pendiente: después de reiniciar el sistema debe terminar esta recuperación antes de comprometer recursos incompatibles. true.

L231

```
231 |         release(assignment);
```

Registra el trabajo de liberación para la asignación. La liberación remota puede completarse después y debe reconciliarse antes de reutilizar recursos.

L232

```
232 |         event(r, Status.SEARCHING_ALTERNATIVE, "PROPOSAL_EXPIRED");
```

SEARCHING_ALTERNATIVE: alternativa del ciclo de vida definido en el enum.

L233

```
233 |       } else r.nextEvaluationAt = expires.isBefore(r.searchDeadline) ? expires :
      r.searchDeadline;
```

Cambios: r.nextEvaluationAt recibe expires.isBefore(r.searchDeadline) ? expires : r.searchDeadline. Compara si el instante es estrictamente anterior al otro; Datos implicados: próximo trabajo de evaluación, vencimiento del intento; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L234

```
234 |       return null;
```

Devuelve ausencia de resultado/trabajo al llamador y termina esta ejecución del método. No equivale a completar el servicio ni a borrar el registro: la condición previa explica por qué no hay una acción que continuar ahora.

L235

```
235 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L236 a L241

```
236 |     if (jdbc.queryForObject(
237 |             "select count(*) from assignments where request_id=? and search_attempt_number=? and
      expires_at is not null",
238 |             Long.class,
239 |             id,
240 |             r.searchAttemptNumber)
241 |         >= rule(r, "search", "max-offers-per-attempt")) {
```

Consulta assignments contando coincidencias. IS NOT NULL filtra filas que ya tienen valor en la columna indicada. Esta condición decide si se entra en la rama siguiente. ejecuta SQL parametrizado y espera un resultado escalar o una fila; Datos implicados: número de búsqueda de 1 a 3.

L242

```
242 |       close(r, "OFFER_LIMIT_REACHED");
```

Cierra sin cumplir por agotar el cupo de ofertas del intento. Evita seguir proponiendo indefinidamente.

L243

```
243 |       return null;
```

Devuelve ausencia de resultado/trabajo al llamador y termina esta ejecución del método. No equivale a completar el servicio ni a borrar el registro: la condición previa explica por qué no hay una acción que continuar ahora.

L244

```
244 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L245 a L246

```
245 |     if (r.status == Status.SEARCHING_ALTERNATIVE)
246 |       event(r, Status.PENDING_EVALUATION, "SEARCH_RESUMED");
```

Esta condición decide si se entra en la rama siguiente. SEARCHING_ALTERNATIVE: alternativa del ciclo de vida definido en el enum; PENDING_EVALUATION: solicitud pendiente de evaluación; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos.

L247

```
247 |     r.nextEvaluationAt = now.plusSeconds(rule(r, "search", "evaluation-interval-seconds"));
```

En r guarda próximo trabajo de evaluación. Toma el dato de now.plusSeconds(rule(r, "search", "evaluation-interval-seconds")) después de las validaciones de este método. calcula un instante futuro sumando segundos, sin modificar el instante original.

L248

```
248 |     return new Work(id, null, true);
```

Devuelve trabajo de nueva evaluación: aún no hay asignación elegida y true solicita buscar/evaluar candidatos.

L249

```
249 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L251

```
251 |   private void replace(AssistanceRequest r, UUID assignment, String reason) {
```

Cierra la asignación previa y crea intención de liberación antes de buscar reemplazo. Campos: reason: motivo legible. Tipos y datos: assignment = assignment; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; reason = motivo legible; String es texto; no es por sí mismo una referencia de entidad.

L252 a L256

```
252 |     jdbc.update(
253 |         "update assignments set status='CLOSED',closed_at=?,reason=? where id=?",
254 |         ts(now()),
255 |         reason,
256 |         assignment);
```

Actualiza assignments para las filas del filtro. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L257

```
257 |     r.resourceReleasePending = true;
```

En r guarda liberación remota pendiente que impide reutilización. Es una bandera durable de trabajo pendiente: después de reiniciar el sistema debe terminar esta recuperación antes de comprometer recursos incompatibles. true.

L258

```
258 |     r.replacementPending = true;
```

En r guarda necesidad de iniciar reemplazo después de liberar. Es una bandera durable de trabajo pendiente: después de reiniciar el sistema debe terminar esta recuperación antes de comprometer recursos incompatibles. true.

L259

```
259 |     r.nextEvaluationAt = now();
```

En r guarda próximo trabajo de evaluación. Toma el dato de now() después de las validaciones de este método.

L260

```
260 |     release(assignment);
```

Registra el trabajo de liberación para la asignación. La liberación remota puede completarse después y debe reconciliarse antes de reutilizar recursos.

L261

```
261 |     event(r, Status.SEARCHING_ALTERNATIVE, reason);
```

SEARCHING_ALTERNATIVE: alternativa del ciclo de vida definido en el enum.

L262

```
262 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L264

```
264 |   public void reconcile(Work work, JsonNode remote) {
```

Compara compromiso local con Fleet; solo reemplaza ante evidencia válida, no por ausencia o fallo de infraestructura. Tipos y datos: remote = remote; JsonNode es árbol JSON navegable usado para snapshots y respuestas externas.

L265

```
265 |     AssistanceRequest r = lock(work.requestId());
```

Se calcula r (r) para reutilizar ese resultado en los pasos siguientes.

L266

```
266 |     if (r.status != Status.ASSIGNED || r.closedAt != null || pending(r.id, "CONFIRM") > 0)
      return;
```

Reconciliación limitada a solicitudes asignadas abiertas; no sustituye un servicio que ya comenzó su ejecución. Esta condición decide si se entra en la rama siguiente. ASSIGNED: solicitud aceptada/asignada; Datos implicados: estado del objeto; su enum o CHECK define valores permitidos, fecha de cierre; null indica abierto, identificador UUID del registro; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; comprueba presencia antes de usar el dato.

L267 a L272

```
267 |     if (jdbc.queryForObject(
268 |             "select count(*) from assignments where id=? and request_id=? and status='ACCEPTED'",
269 |             Long.class,
270 |             work.assignmentId(),
271 |             r.id)
272 |         == 0) return;
```

Consulta assignments contando coincidencias. Esta condición decide si se entra en la rama siguiente. ejecuta SQL parametrizado y espera un resultado escalar o una fila; Datos implicados: identificador UUID del registro.

L273 a L275

```
273 |     if (remote == null
274 |         || !work.assignmentId().toString().equals(remote.path("assignmentAttemptId").asText()))
275 |       return;
```

Esta condición decide si se entra en la rama siguiente. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a texto; compara contenido/identidad lógica, no referencias Java; representa el valor como texto; en un nodo JSON produce JSON serializado; Ruta JSON consultada: assignmentAttemptId; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue; detecta un dato ausente.

L274: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a texto; compara contenido/identidad lógica, no referencias Java; representa el valor como texto; en un nodo JSON produce JSON serializado; Ruta JSON consultada: assignmentAttemptId; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L276

```
276 |     String status = remote.path("status").asText();
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a texto; Ruta JSON consultada: status

L277 a L283

```
277 |     if (status.equals("MISSING"))
278 |       return; // Infrastructure inconsistency is not authority to free a commitment.
279 |     if ((status.equals("CONFIRMED")
280 |             && remote.path("resource").has("eligible")
281 |             && !remote.path("resource").path("eligible").asBoolean())
282 |         || Set.of("RELEASED", "EXPIRED").contains(status))
283 |       replace(r, work.assignmentId(), "RESOURCE_NOT_ELIGIBLE");
```

La ausencia de registro remoto es inconsistencia de infraestructura; no autoriza a liberar unilateralmente el compromiso. Esta condición decide si se entra en la rama siguiente. navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a booleano; compara contenido/identidad lógica, no referencias Java; verifica pertenencia al conjunto o lista; crea un conjunto no modificable y sin duplicados; Ruta JSON consultada: resource.resource.eligible; && exige todas las condiciones y deja de evaluar al encontrar una falsa; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; ! invierte el booleano de la expresión que sigue.

L280: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; Ruta JSON consultada: resource; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L281: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a booleano; Ruta JSON consultada: resource.eligible; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L282: verifica pertenencia al conjunto o lista; crea un conjunto no modificable y sin duplicados; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera.

L284

```
284 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L285

```
285 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 11, 17, 24, 39, 43, 47, 55, 61, 67, 74, 102, 110, 119, 142, 156, 250, 263: separadores sin instrucciones.

ExecutionController (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/ExecutionController.java`

Objetivo y funcionalidad

Acciones declaradas por el operador: despacho, llegada, carga, inicio y entrega. Cada paso requiere clave de idempotencia.

L1

```
  1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import jakarta.validation.Valid;
```

Importa tipos: jakarta.validation.Valid. Restricciones de DTO y activación de validación de entradas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import jakarta.validation.constraints.*;
```

Importa tipos: jakarta.validation.constraints.*. Restricciones de DTO y activación de validación de entradas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import org.springframework.web.bind.annotation.*;
```

Importa tipos: org.springframework.web.bind.annotation.*. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.security.core.annotation.AuthenticationPrincipal;
```

Importa tipos: org.springframework.security.core.annotation.AuthenticationPrincipal. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.springframework.security.oauth2.jwt.Jwt;
```

Importa tipos: org.springframework.security.oauth2.jwt.Jwt. Validación JWT, Authentication, autorización y cadenas de seguridad. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | import io.swagger.v3.oas.annotations.Operation;
```

Importa tipos: io.swagger.v3.oas.annotations.Operation. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import io.swagger.v3.oas.annotations.responses.*;
```

Importa tipos: io.swagger.v3.oas.annotations.responses.*. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | import io.swagger.v3.oas.annotations.security.SecurityRequirement;
```

Importa tipos: io.swagger.v3.oas.annotations.security.SecurityRequirement. Metadatos OpenAPI de rutas, respuestas y autenticación. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12

```
12 | import static com.auxiruta.assistance.requests.ExecutionStore.Step;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.ExecutionStore.Step. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L14

```
14 | @RestController
```

Expone HTTP y serializa JSON.

L15

```
15 | @RequestMapping("/api/v1/assistance-requests")
```

Ruta /api/v1/assistance-requests combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L16

```
16 | @SecurityRequirement(name = "bearerAuth")
```

Documenta autenticación; no concede permisos.

L17 a L27

```
17 | @ApiResponses({
18 |   @ApiResponse(responseCode = "400", description = "Entrada invalida"),
19 |   @ApiResponse(responseCode = "401", description = "JWT ausente o invalido"),
20 |   @ApiResponse(responseCode = "403", description = "Rol insuficiente"),
21 |   @ApiResponse(responseCode = "404", description = "Recurso inexistente o ajeno"),
22 |   @ApiResponse(responseCode = "409", description = "Conflicto de estado, plazo o idempotencia"),
23 |   @ApiResponse(
24 |       responseCode = "503",
25 |       description = "Dependencia temporalmente no disponible; consultar estado antes de
     reintentar")
26 | })
27 | public class ExecutionController {
```

Documenta código HTTP. Declara clase ExecutionController. Acciones declaradas por el operador: despacho, llegada, carga, inicio y entrega. Cada paso requiere clave de idempotencia.

L28

```
28 |   public record Completion(@NotBlank @Size(min = 10, max = 500) String note) {
```

Exige texto no vacío ni solo espacios. Limita caracteres/elementos según min/max. Declara record inmutable Completion. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Se calcula max (max) para reutilizar ese resultado en los pasos siguientes. Acciones declaradas por el operador: despacho, llegada, carga, inicio y entrega. Cada paso requiere clave de idempotencia. Tipos y datos: note = note; String es texto; no es por sí mismo una referencia de entidad.

L29

```
29 |     public Completion {
```

Constructor compacto de un record: valida o normaliza los parámetros antes de que Java los guarde en campos finales. Evita crear un DTO que incumpla esta condición local.

L30

```
30 |       if (note != null) note = note.strip();
```

Esta condición decide si se entra en la rama siguiente. comprueba presencia antes de usar el dato.

L31

```
31 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L32

```
32 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L34

```
34 |   private final ExecutionService service;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L36

```
36 |   public ExecutionController(ExecutionService service) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L37

```
37 |     this.service = service;
```

En this guarda service. Toma el dato de service después de las validaciones de este método. service.

L38

```
38 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L40

```
40 |   @PostMapping("/{id}/dispatch")
```

Ruta /{id}/dispatch combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L41

```
41 |   @Operation(summary = "OPERATOR asignado: revalidar compromiso y rutas y declarar salida")
```

Documenta en OpenAPI: OPERATOR asignado: revalidar compromiso y rutas y declarar salida. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L42 a L45

```
42 |   public RequestDtos.HistoryView dispatch(
43 |       @PathVariable UUID id,
44 |       @AuthenticationPrincipal Jwt jwt,
45 |       @RequestHeader("Idempotency-Key") String key) {
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Lee header indicado, normalmente Idempotency-Key. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias; key = key; String es texto; no es por sí mismo una referencia de entidad.

L43: Dato del contrato: id: identificador UUID del registro.

L46

```
46 |     return service.perform(id, jwt.getSubject(), key, Step.DISPATCH, null);
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Resultado que recibe el llamador: obtiene sub, identidad estable del usuario autenticado.

L47

```
47 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L49

```
49 |   @PostMapping("/{id}/arrival")
```

Ruta /{id}/arrival combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L50

```
50 |   @Operation(summary = "OPERATOR asignado: registrar llegada al origen después del despacho")
```

Documenta en OpenAPI: OPERATOR asignado: registrar llegada al origen después del despacho. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L51 a L54

```
51 |   public RequestDtos.HistoryView arrival(
52 |       @PathVariable UUID id,
53 |       @AuthenticationPrincipal Jwt jwt,
54 |       @RequestHeader("Idempotency-Key") String key) {
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Lee header indicado, normalmente Idempotency-Key. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias; key = key; String es texto; no es por sí mismo una referencia de entidad.

L52: Dato del contrato: id: identificador UUID del registro.

L55

```
55 |     return service.perform(id, jwt.getSubject(), key, Step.ARRIVAL, null);
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Resultado que recibe el llamador: obtiene sub, identidad estable del usuario autenticado.

L56

```
56 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L58

```
58 |   @PostMapping("/{id}/loading")
```

Ruta /{id}/loading combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L59

```
59 |   @Operation(summary = "OPERATOR asignado: declarar vehículo cargado después de la llegada")
```

Documenta en OpenAPI: OPERATOR asignado: declarar vehículo cargado después de la llegada. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L60 a L63

```
60 |   public RequestDtos.HistoryView loading(
61 |       @PathVariable UUID id,
62 |       @AuthenticationPrincipal Jwt jwt,
63 |       @RequestHeader("Idempotency-Key") String key) {
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Lee header indicado, normalmente Idempotency-Key. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias; key = key; String es texto; no es por sí mismo una referencia de entidad.

L61: Dato del contrato: id: identificador UUID del registro.

L64

```
64 |     return service.perform(id, jwt.getSubject(), key, Step.LOADING, null);
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Resultado que recibe el llamador: obtiene sub, identidad estable del usuario autenticado.

L65

```
65 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L67

```
67 |   @PostMapping("/{id}/transport-start")
```

Ruta /{id}/transport-start combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L68

```
68 |   @Operation(summary = "OPERATOR asignado: iniciar traslado después de cargar el vehículo")
```

Documenta en OpenAPI: OPERATOR asignado: iniciar traslado después de cargar el vehículo. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L69 a L72

```
69 |   public RequestDtos.HistoryView start(
70 |       @PathVariable UUID id,
71 |       @AuthenticationPrincipal Jwt jwt,
72 |       @RequestHeader("Idempotency-Key") String key) {
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Lee header indicado, normalmente Idempotency-Key. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias; key = key; String es texto; no es por sí mismo una referencia de entidad.

L70: Dato del contrato: id: identificador UUID del registro.

L73

```
73 |     return service.perform(id, jwt.getSubject(), key, Step.TRANSPORT_START, null);
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Resultado que recibe el llamador: obtiene sub, identidad estable del usuario autenticado.

L74

```
74 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L76

```
76 |   @PostMapping("/{id}/completion")
```

Ruta /{id}/completion combinada con el prefijo de clase cuando corresponda. El método HTTP y la ruta seleccionan el handler; no basta conocer la URL: la cadena de seguridad valida rol/token y el servicio verifica propiedad/estado.

L77

```
77 |   @Operation(summary = "OPERATOR asignado: registrar entrega con nota y liberar recursos")
```

Documenta en OpenAPI: OPERATOR asignado: registrar entrega con nota y liberar recursos. Es información para consumidores; ejecutar la acción requiere el método HTTP, rol y estado implementados en el controlador/servicio.

L78 a L82

```
78 |   public RequestDtos.HistoryView complete(
79 |       @PathVariable UUID id,
80 |       @AuthenticationPrincipal Jwt jwt,
81 |       @RequestHeader("Idempotency-Key") String key,
82 |       @Valid @RequestBody Completion body) {
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Lee header indicado, normalmente Idempotency-Key. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; jwt = jwt; Jwt es token validado que contiene identidad, emisor, roles y audiencias; key = key; String es texto; no es por sí mismo una referencia de entidad.

L79: Dato del contrato: id: identificador UUID del registro.

L83

```
83 |     return service.perform(id, jwt.getSubject(), key, Step.COMPLETION, body.note());
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Resultado que recibe el llamador: obtiene sub, identidad estable del usuario autenticado.

L84

```
84 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L85

```
85 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 13, 33, 35, 39, 48, 57, 66, 75: separadores sin instrucciones.

ExecutionService (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/ExecutionService.java`

Objetivo y funcionalidad

Coordina preparación, revalidación de despacho, commit y recuperación de liberación sin mantener bloqueos durante HTTP.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import static com.auxiruta.assistance.requests.ExecutionStore.Step;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.ExecutionStore.Step. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | @Service
```

Registra lógica de aplicación como bean. El escaneo de Spring crea este componente e inyecta sus dependencias; no hay que construirlo manualmente en el controlador.

L8

```
8 | public class ExecutionService {
```

Declara clase ExecutionService. Coordina preparación, revalidación de despacho, commit y recuperación de liberación sin mantener bloqueos durante HTTP.

L9

```
9 |   private final ExecutionStore store;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L10

```
10 |   private final DispatchValidator dispatch;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L11

```
11 |   private final AssignmentCoordinator assignments;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L13 a L14

```
13 |   public ExecutionService(
14 |       ExecutionStore store, DispatchValidator dispatch, AssignmentCoordinator assignments) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L15

```
15 |     this.store = store;
```

En this guarda componente que conserva decisiones en transacciones locales. Toma el dato de store después de las validaciones de este método. componente que conserva decisiones en transacciones locales.

L16

```
16 |     this.dispatch = dispatch;
```

En this guarda dispatch. Toma el dato de dispatch después de las validaciones de este método. dispatch.

L17

```
17 |     this.assignments = assignments;
```

En this guarda assignments. Toma el dato de assignments después de las validaciones de este método. assignments.

L18

```
18 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L20 a L21

```
20 |   public RequestDtos.HistoryView perform(
21 |       UUID id, String subject, String key, Step step, String note) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; subject = subject; String es texto; no es por sí mismo una referencia de entidad; key = key; String es texto; no es por sí mismo una referencia de entidad; note = note; String es texto; no es por sí mismo una referencia de entidad.

L21: Dato del contrato: id: identificador UUID del registro.

L22

```
22 |     var prepared = store.prepare(id, subject, key, step, note);
```

Prepara paso bajo transacción; previous no nulo indica repetición ya registrada. Se calcula prepared (prepared) para reutilizar ese resultado en los pasos siguientes.

L23

```
23 |     RequestDtos.HistoryView result = prepared.previous();
```

Se calcula result (resultado de evaluación, distinto del estado de solicitud) para reutilizar ese resultado en los pasos siguientes.

L24

```
24 |     if (result == null) {
```

Esta condición decide si se entra en la rama siguiente. detecta un dato ausente.

L25

```
25 |       var evidence = step == Step.DISPATCH ? dispatch.validate(prepared.input()) : null;
```

Se calcula evidence (JSON de datos usados para justificar la decisión) para reutilizar ese resultado en los pasos siguientes. el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario

L26

```
26 |       result = store.commit(prepared.input(), subject, key, step, note, evidence);
```

Revalida el snapshot de preparación bajo transacción y confirma el paso con actor, clave y evidencia. Evita aplicar el resultado de HTTP si la solicitud cambió mientras tanto.

L27

```
27 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L28

```
28 |     if (step == Step.COMPLETION) assignments.recover(id);
```

Reintenta liberar incluso al repetir entrega; no duplica COMPLETED. Esta condición decide si se entra en la rama siguiente. .

L29

```
29 |     return result;
```

Entrega result al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método. Resultado que recibe el llamador: resultado de evaluación, distinto del estado de solicitud.

L30

```
30 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L31

```
31 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 6, 12, 19: separadores sin instrucciones.

ExecutionStore (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/ExecutionStore.java`

Objetivo y funcionalidad

Transacciones locales de ejecución. Prepara y revalida versión/asignación antes de cambiar el estado. Al completar, guarda intención durable de liberar recursos.

L1

```
1 | package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo. Esta separación permite distinguir las responsabilidades de assistance-service sin compartir sus entidades con otro módulo.

L3

```
3 | import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L4

```
4 | import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L5

```
5 | import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L6

```
6 | import java.sql.Timestamp;
```

Importa tipos: java.sql.Timestamp. Timestamp y ResultSet para puentear fechas/filas de JDBC con objetos Java. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L7

```
7 | import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L8

```
8 | import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L9

```
9 | import org.springframework.jdbc.core.JdbcTemplate;
```

Importa tipos: org.springframework.jdbc.core.JdbcTemplate. JdbcTemplate para SQL parametrizado, resultados y acciones durables. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L10

```
10 | import org.springframework.http.HttpStatus;
```

Importa tipos: org.springframework.http.HttpStatus. Estados, headers, tipos de contenido y respuestas HTTP. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L11

```
11 | import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L12

```
12 | import com.fasterxml.jackson.databind.ObjectMapper;
```

Importa tipos: com.fasterxml.jackson.databind.ObjectMapper. Lectura/escritura JSON, nodos y conversión a DTO. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L13

```
13 | import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida. La importación habilita nombres cortos en el fuente; no ejecuta ni instancia la clase importada.

L15 a L16

```
15 | /** Local transactions only; no remote calls are performed while holding these locks. */
16 | @Service
```

Documentación para quien lee el fuente: Local transactions only; no remote calls are performed while holding these locks. */ @Service No es una instrucción ejecutable y no impone la regla por sí sola.

L17

```
17 | @Transactional
```

Delimita transacción local; readOnly=true declara consulta. Spring abre/reutiliza una transacción al entrar por el proxy y confirma al terminar correctamente. Un fallo no comprobado normalmente revierte cambios locales; no revierte HTTP ya realizado en Fleet.

L18

```
18 | public class ExecutionStore {
```

Declara clase ExecutionStore. Transacciones locales de ejecución. Prepara y revalida versión/asignación antes de cambiar el estado. Al completar, guarda intención durable de liberar recursos.

L19

```
19 |   public enum Step {
```

Declara enumeración cerrada Step. Transacciones locales de ejecución. Prepara y revalida versión/asignación antes de cambiar el estado. Al completar, guarda intención durable de liberar recursos.

L20 a L24

```
20 |     DISPATCH(Status.ASSIGNED, Status.DISPATCHED, "dispatched_at"),
21 |     ARRIVAL(Status.DISPATCHED, Status.ARRIVED, "arrived_at"),
22 |     LOADING(Status.ARRIVED, Status.VEHICLE_LOADED, "loaded_at"),
23 |     TRANSPORT_START(Status.VEHICLE_LOADED, Status.IN_TRANSIT, "transport_started_at"),
24 |     COMPLETION(Status.IN_TRANSIT, Status.COMPLETED, "completed_at");
```

Mapea ASSIGNED a DISPATCHED y fecha dispatched_at. Mapea DISPATCHED a ARRIVED y fecha arrived_at. Mapea ARRIVED a VEHICLE_LOADED y fecha loaded_at. Mapea VEHICLE_LOADED a IN_TRANSIT y fecha transport_started_at. Mapea IN_TRANSIT a COMPLETED y fecha completed_at. ASSIGNED: solicitud aceptada/asignada; DISPATCHED: grúa despachada; ARRIVED: llegada declarada; VEHICLE_LOADED: vehículo cargado; IN_TRANSIT: traslado iniciado; COMPLETED: servicio finalizado.

L25

```
25 |     final Status previous, next;
```

Cada paso conserva predecesor, sucesor y columna temporal segura.

L26

```
26 |     final String column;
```

Tipos y datos: column = column; String es texto; no es por sí mismo una referencia de entidad.

L28

```
28 |     Step(Status previous, Status next, String column) {
```

Tipos y datos: column = column; String es texto; no es por sí mismo una referencia de entidad.

L29

```
29 |       this.previous = previous;
```

En this guarda previous. Toma el dato de previous después de las validaciones de este método. previous.

L30

```
30 |       this.next = next;
```

En this guarda next. Toma el dato de next después de las validaciones de este método. next.

L31

```
31 |       this.column = column;
```

En this guarda column. Toma el dato de column después de las validaciones de este método. column.

L32

```
32 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L34

```
34 |     public boolean admits(Status state) {
```

Solo admite exactamente el estado predecesor; no permite saltar etapas.

L35

```
35 |       return previous == state;
```

Compara constantes enum para exigir el estado predecesor exacto de esta etapa; por ejemplo, llegada solo después de despacho.

L36

```
36 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L37

```
37 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L39 a L52

```
39 |   public record Input(
40 |       UUID requestId,
41 |       long version,
42 |       UUID assignmentId,
43 |       UUID truckId,
44 |       UUID operatorId,
45 |       String subject,
46 |       Instant acceptedAt,
47 |       String rules,
48 |       BigDecimal originLat,
49 |       BigDecimal originLon,
50 |       BigDecimal destinationLat,
51 |       BigDecimal destinationLon,
52 |       String mobility) {}
```

Declara record inmutable Input. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: requestId: UUID de solicitud; version: versión para control optimista de concurrencia; operatorId: UUID del perfil operador; acceptedAt: instante de aceptación efectiva. Transacciones locales de ejecución. Prepara y revalida versión/asignación antes de cambiar el estado. Al completar, guarda intención durable de liberar recursos. Tipos y datos: requestId = UUID de solicitud; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; version = versión para control optimista de concurrencia; long es entero de 64 bits para cantidades y duraciones mayores; assignmentId = assignmentId; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; truckId = truckId; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; operatorId = UUID del perfil operador; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; subject = subject; String es texto; no es por sí mismo una referencia de entidad; acceptedAt = instante de aceptación efectiva; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; rules = configuración o snapshot de reglas que rige esta operación; String es texto; no es por sí mismo una referencia de entidad; originLat = originLat; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; originLon = originLon; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; destinationLat = destinationLat; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; destinationLon = destinationLon; BigDecimal es decimal de precisión explícita para pesos, coordenadas y cálculos; mobility = mobility; String es texto; no es por sí mismo una referencia de entidad.

L40: Dato del contrato: requestId: UUID de solicitud.

L41: Dato del contrato: version: versión para control optimista de concurrencia.

L44: Dato del contrato: operatorId: UUID del perfil operador.

L46: Dato del contrato: acceptedAt: instante de aceptación efectiva.

L54

```
54 |   public record Preparation(Input input, RequestDtos.HistoryView previous) {}
```

Declara record inmutable Preparation. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Transacciones locales de ejecución. Prepara y revalida versión/asignación antes de cambiar el estado. Al completar, guarda intención durable de liberar recursos.

L56

```
56 |   private final JdbcTemplate jdbc;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: jdbc = ejecutor SQL parametrizado; JdbcTemplate es ejecutor de SQL parametrizado de Spring dentro de transacciones locales.

L57

```
57 |   private final RequestRepository requests;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L58

```
58 |   private final HistoryRepository history;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L59

```
59 |   private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos.

L60

```
60 |   private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias. Tipos y datos: mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L62 a L67

```
62 |   public ExecutionStore(
63 |       JdbcTemplate jdbc,
64 |       RequestRepository requests,
65 |       HistoryRepository history,
66 |       Clock clock,
67 |       ObjectMapper mapper) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada. Tipos y datos: jdbc = ejecutor SQL parametrizado; JdbcTemplate es ejecutor de SQL parametrizado de Spring dentro de transacciones locales; clock = reloj usado para evaluar plazos; Clock es fuente de tiempo inyectable; se sustituye en tests para controlar vencimientos; mapper = conversor JSON de Jackson; ObjectMapper es conversor Jackson entre JSON y objetos/árboles Java.

L68

```
68 |     this.jdbc = jdbc;
```

En this guarda ejecutor SQL parametrizado. Toma el dato de jdbc después de las validaciones de este método. ejecutor SQL parametrizado.

L69

```
69 |     this.requests = requests;
```

En this guarda repositorio de solicitudes. Toma el dato de requests después de las validaciones de este método. repositorio de solicitudes.

L70

```
70 |     this.history = history;
```

En this guarda historial persistente de cambios. Toma el dato de history después de las validaciones de este método. historial persistente de cambios.

L71

```
71 |     this.clock = clock;
```

En this guarda reloj usado para evaluar plazos. Toma el dato de clock después de las validaciones de este método. reloj usado para evaluar plazos.

L72

```
72 |     this.mapper = mapper;
```

En this guarda conversor JSON de Jackson. Toma el dato de mapper después de las validaciones de este método. conversor JSON de Jackson.

L73

```
73 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L75

```
75 |   private Instant now() {
```

Obtiene el instante del reloj inyectado; truncar a microsegundos mantiene consistencia con PostgreSQL.

L76

```
76 |     return clock.instant().truncatedTo(java.time.temporal.ChronoUnit.MICROS);
```

Resultado que recibe el llamador: lee el instante del reloj inyectado.

L77

```
77 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L79

```
79 |   private static ResponseStatusException missing() {
```

Construye 404; puede representar recurso inexistente o ajeno para no revelar su existencia.

L80

```
80 |     return new ResponseStatusException(HttpStatus.NOT_FOUND);
```

Resultado que recibe el llamador: interrumpe la operación con un estado HTTP explícito; 404 oculta inexistencia o pertenencia ajena.

L81

```
81 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L83

```
83 |   private static void require(boolean value) {
```

Expresa una precondición; si resulta falsa se aborta con el estado HTTP indicado. Tipos y datos: value = value; boolean es verdadero/falso, sin null.

L84

```
84 |     if (!value) throw new ResponseStatusException(HttpStatus.CONFLICT);
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. interrumpe la operación con un estado HTTP explícito; 409 comunica que el estado/regla actual impide la acción; ! invierte el booleano de la expresión que sigue.

L85

```
85 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L87

```
87 |   static String fingerprint(Step step, String note) {
```

Calcula una huella del contenido para distinguir repetición idéntica de reutilización conflictiva de una clave. Tipos y datos: note = note; String es texto; no es por sí mismo una referencia de entidad.

L88

```
88 |     return RequestService.digest(step.name() + ":" + (note == null ? "" : note.strip()));
```

Resultado que recibe el llamador: calcula una huella de contenido usada para idempotencia; detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L89

```
89 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L91

```
91 |   private RequestDtos.HistoryView view(RequestStatusHistory h) {
```

Transforma entidad/datos internos en DTO; no cambia la base por realizar esta proyección.

L92 a L102

```
92 |     return new RequestDtos.HistoryView(
93 |         h.id,
94 |         h.request.getId(),
95 |         h.previousStatus,
96 |         h.newStatus,
97 |         h.actorType,
98 |         h.actorSubject,
99 |         h.reason,
100 |         h.details,
101 |         h.operationKey,
102 |         h.occurredAt);
```

Resultado que recibe el llamador: Datos implicados: identificador UUID del registro, estado previo, estado resultante, tipo de actor SYSTEM/DRIVER/OPERATOR, sub del usuario que produjo el evento, motivo legible, evidencia adicional, clave durable de acción, instante del evento.

L103

```
103 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L105

```
105 |   private AssistanceRequest lock(UUID id) {
```

Adquiere bloqueos en orden para serializar operaciones competidoras. Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo.

L106

```
106 |     UUID vehicle = requests.vehicleId(id).orElseThrow(ExecutionStore::missing);
```

Si no encuentra resultado lanza la excepción indicada. Se calcula vehicle (vehicle) para reutilizar ese resultado en los pasos siguientes. si no existe valor, lanza la excepción en vez de seguir con null

L107

```
107 |     jdbc.queryForObject("select id from vehicles where id=? for update", UUID.class, vehicle);
```

Consulta vehicles con el filtro y proyección escritos. Bloquea las filas durante la transacción. Se calcula id (identificador UUID del registro) para reutilizar ese resultado en los pasos siguientes. Ejecuta SQL parametrizado y espera un resultado escalar o una fila.

L108

```
108 |     return requests.lockById(id).orElseThrow(ExecutionStore::missing);
```

Si no encuentra resultado lanza la excepción indicada. Resultado que recibe el llamador: si no existe valor, lanza la excepción en vez de seguir con null.

L109

```
109 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L111

```
111 |   public Preparation prepare(UUID id, String subject, String key, Step step, String note) {
```

Campos: id: identificador UUID del registro. Tipos y datos: id = identificador UUID del registro; UUID es identificador de 128 bits para recursos; se transmite como texto y PostgreSQL dispone de un tipo nativo; subject = subject; String es texto; no es por sí mismo una referencia de entidad; key = key; String es texto; no es por sí mismo una referencia de entidad; note = note; String es texto; no es por sí mismo una referencia de entidad.

L112 a L113

```
112 |     if (key == null || key.isBlank() || key.length() > 100)
113 |       throw new ResponseStatusException(HttpStatus.BAD_REQUEST);
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. comprueba texto vacío o solo espacios; interrumpe la operación con un estado HTTP explícito; 400 indica entrada inválida; || acepta cualquiera de las alternativas y deja de evaluar al encontrar una verdadera; detecta un dato ausente.

L114

```
114 |     AssistanceRequest r = lock(id);
```

Serializa cambio mediante bloqueo de vehículo/solicitud antes de validar precondiciones. Se calcula r (r) para reutilizar ese resultado en los pasos siguientes.

L115 a L120

```
115 |     if (jdbc.queryForObject(
116 |             "select count(*) from assignments where request_id=? and operator_subject=?",
117 |             Long.class,
118 |             id,
119 |             subject)
120 |         == 0) throw missing();
```

Consulta assignments contando coincidencias. La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. ejecuta SQL parametrizado y espera un resultado escalar o una fila.

L121

```
121 |     var old = history.findByRequestIdAndOperationKey(id, key);
```

Busca resultado anterior para repetir sin duplicar. Se calcula old (old) para reutilizar ese resultado en los pasos siguientes.

L122

```
122 |     if (old.isPresent()) {
```

Esta condición decide si se entra en la rama siguiente. .

L123

```
123 |       var h = old.get();
```

Se calcula h (h) para reutilizar ese resultado en los pasos siguientes.

L124 a L125

```
124 |       require(
125 |           subject.equals(h.actorSubject) && fingerprint(step,
      note).equals(h.requestFingerprint));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara contenido/identidad lógica, no referencias Java; Datos implicados: sub del usuario que produjo el evento, SHA-256 del contenido asociado a la clave; && exige todas las condiciones y deja de evaluar al encontrar una falsa

L126

```
126 |       return new Preparation(null, view(h));
```

Informa que existe un evento anterior para la misma operación. No prepara una nueva ejecución: devuelve el historial persistido y evita repetir efectos.

L127

```
127 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L128 a L146

```
128 |     var assignments =
129 |         jdbc.query(
130 |             "select id,tow_truck_id,operator_id,operator_subject,accepted_at from assignments
      where request_id=? and status='ACCEPTED'",
131 |             (rs, n) ->
132 |                 new Input(
133 |                     id,
134 |                     r.version,
135 |                     rs.getObject("id", UUID.class),
136 |                     rs.getObject("tow_truck_id", UUID.class),
137 |                     rs.getObject("operator_id", UUID.class),
138 |                     rs.getString("operator_subject"),
139 |                     rs.getTimestamp("accepted_at").toInstant(),
140 |                     r.configurationSnapshot,
141 |                     r.originLatitude,
142 |                     r.originLongitude,
143 |                     r.destinationLatitude,
144 |                     r.destinationLongitude,
145 |                     r.mobilityCondition.name()),
146 |             id);
```

Consulta assignments con el filtro y proyección escritos. Se calcula assignments (assignments) para reutilizar ese resultado en los pasos siguientes. ejecuta una consulta SQL y transforma filas mediante el mapper; Datos implicados: versión para control optimista de concurrencia, JSON de parámetros vigente al crear, latitud de recogida, longitud de recogida, latitud de entrega, longitud de entrega, condición ROLLABLE/NON_ROLLABLE

L147

```
147 |     if (assignments.isEmpty()) throw new ResponseStatusException(HttpStatus.CONFLICT);
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. comprueba ausencia de elementos o valor; interrumpe la operación con un estado HTTP explícito; 409 comunica que el estado/regla actual impide la acción.

L148

```
148 |     var input = assignments.get(0);
```

Se calcula input (snapshot de entrada que evita trabajar con una entidad mutable fuera de transacción) para reutilizar ese resultado en los pasos siguientes.

L149

```
149 |     if (!input.subject().equals(subject)) throw missing();
```

La rama condicional aborta ante la condición inválida. Esta condición decide si se entra en la rama siguiente. compara contenido/identidad lógica, no referencias Java; ! invierte el booleano de la expresión que sigue.

L150

```
150 |     require(step.admits(r.status) && r.closedAt == null && !r.resourceReleasePending);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. Datos implicados: estado del objeto; su enum o CHECK define valores permitidos, fecha de cierre; null indica abierto, liberación remota pendiente que impide reutilización; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue; detecta un dato ausente

L151 a L152

```
151 |     if (step == Step.DISPATCH)
152 |       require(dispatchOpen(input.acceptedAt(), now(), dispatchSeconds(input.rules())));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Esta condición decide si se entra en la rama siguiente. Datos implicados: instante de aceptación efectiva, configuración o snapshot de reglas que rige esta operación. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. Datos implicados: instante de aceptación efectiva, configuración o snapshot de reglas que rige esta operación

L153

```
153 |     return new Preparation(input, null);
```

Devuelve datos preparados para operación nueva, sin evento previo de idempotencia.

L154

```
154 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L156

```
156 |   static boolean dispatchOpen(Instant acceptedAt, Instant now, int seconds) {
```

Exige ahora mayor o igual a aceptación y estrictamente anterior al vencimiento de despacho. Campos: acceptedAt: instante de aceptación efectiva. Tipos y datos: acceptedAt = instante de aceptación efectiva; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; now = instante actual obtenido del reloj inyectado; Instant es instante UTC comparable para plazos y auditoría, sin depender de la zona del equipo; seconds = seconds; int es entero de 32 bits para límites y contadores.

L157

```
157 |     return !now.isBefore(acceptedAt) && now.isBefore(acceptedAt.plusSeconds(seconds));
```

Resultado que recibe el llamador: calcula un instante futuro sumando segundos, sin modificar el instante original; compara si el instante es estrictamente anterior al otro; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L158

```
158 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L160

```
160 |   private int dispatchSeconds(String rules) {
```

Lee el plazo del snapshot de la solicitud, no de una configuración posterior. Tipos y datos: rules = configuración o snapshot de reglas que rige esta operación; String es texto; no es por sí mismo una referencia de entidad.

L161

```
161 |     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L162

```
162 |       return mapper.readTree(rules).path("execution").path("dispatch-deadline-seconds").asInt();
```

Resultado que recibe el llamador: navega claves de un JSON; path devuelve un nodo ausente si no encuentra la clave; convierte el nodo a entero; analiza texto JSON para consultar su estructura; Ruta JSON consultada: execution.dispatch-deadline-seconds; Plazo exclusivo para declarar salida..

L163

```
163 |     } catch (Exception e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. El tipo de excepción distingue un rechazo de negocio de un problema técnico. Lanzar nuevamente conserva el fallo; traducirlo a 503 permite reintentar sin afirmar rechazo definitivo.

L164

```
164 |       throw new IllegalStateException("Invalid rule snapshot");
```

La ejecución normal se interrumpe aquí; el manejador de excepciones transforma el fallo en respuesta cuando corresponde.

L165

```
165 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L166

```
166 |   }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L168 a L174

```
168 |   public RequestDtos.HistoryView commit(
169 |       Input expected,
170 |       String subject,
171 |       String key,
172 |       Step step,
173 |       String note,
174 |       DispatchValidator.Evidence evidence) {
```

Vuelve a validar el estado y aplica transición e historial atómicamente; para despacho comprueba evidencia aún vigente. Tipos y datos: subject = subject; String es texto; no es por sí mismo una referencia de entidad; key = key; String es texto; no es por sí mismo una referencia de entidad; note = note; String es texto; no es por sí mismo una referencia de entidad.

L175

```
175 |     Preparation preparation = prepare(expected.requestId(), subject, key, step, note);
```

Se calcula preparation (preparation) para reutilizar ese resultado en los pasos siguientes.

L176

```
176 |     if (preparation.previous() != null) return preparation.previous();
```

Esta condición decide si se entra en la rama siguiente. comprueba presencia antes de usar el dato.

L177

```
177 |     Input current = preparation.input();
```

Se calcula current (current) para reutilizar ese resultado en los pasos siguientes.

L178 a L180

```
178 |     require(
179 |         current.version() == expected.version()
180 |             && current.assignmentId().equals(expected.assignmentId()));
```

Tras el HTTP vuelve a comprobar versión y asignación. Si cambiaron, aborta en vez de aplicar evidencia a otro estado. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. compara contenido/identidad lógica, no referencias Java; && exige todas las condiciones y deja de evaluar al encontrar una falsa

L180: compara contenido/identidad lógica, no referencias Java; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L181

```
181 |     AssistanceRequest r = requests.findById(expected.requestId()).orElseThrow();
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Se calcula r (r) para reutilizar ese resultado en los pasos siguientes. si no existe valor, lanza la excepción en vez de seguir con null; busca por clave primaria; el resultado puede faltar

L182

```
182 |     Instant at = now();
```

Se calcula at (at) para reutilizar ese resultado en los pasos siguientes.

L183 a L187

```
183 |     if (step == Step.DISPATCH)
184 |       require(
185 |           evidence != null
186 |               && !at.isAfter(evidence.validUntil())
187 |               &&
      evidence.qualificationUntil().isAfter(at.plusSeconds(evidence.plannedSeconds())));
```

Antes de guardar despacho exige evidencia aún vigente y habilitación estrictamente posterior al fin planificado. Esta condición decide si se entra en la rama siguiente. calcula un instante futuro sumando segundos, sin modificar el instante original; compara si el instante es estrictamente posterior al otro; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue; comprueba presencia antes de usar el dato. Es una precondición del método: si no se cumple, require lanza el error y evita avanzar. calcula un instante futuro sumando segundos, sin modificar el instante original; compara si el instante es estrictamente posterior al otro; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue; comprueba presencia antes de usar el dato

L186: compara si el instante es estrictamente posterior al otro; && exige todas las condiciones y deja de evaluar al encontrar una falsa; ! invierte el booleano de la expresión que sigue.

L187: calcula un instante futuro sumando segundos, sin modificar el instante original; compara si el instante es estrictamente posterior al otro; && exige todas las condiciones y deja de evaluar al encontrar una falsa.

L188

```
188 |     RequestStatusHistory h = new RequestStatusHistory();
```

Construye RequestStatusHistory en memoria y conserva la referencia en h. Auditoría de cambios de solicitud con actor, motivo, fecha, idempotencia y evidencia. Es persistencia de negocio, distinta de los logs de consola. Crear el objeto no ejecuta automáticamente un INSERT; la persistencia ocurre al guardar o sincronizar el contexto cuando corresponde.

L189

```
189 |     h.request = r;
```

En h guarda request. Toma el dato de r después de las validaciones de este método. r.

L190

```
190 |     h.previousStatus = r.status;
```

En h guarda estado previo. La transición efectiva proviene de r.status y debe estar autorizada por las validaciones previas. Datos implicados: estado del objeto; su enum o CHECK define valores permitidos.

L191

```
191 |     h.newStatus = step.next;
```

En h guarda estado resultante. La transición efectiva proviene de step.next y debe estar autorizada por las validaciones previas.

L192

```
192 |     h.actorType = "OPERATOR";
```

En h guarda tipo de actor SYSTEM/DRIVER/OPERATOR. Toma el dato de "OPERATOR" después de las validaciones de este método.

L193

```
193 |     h.actorSubject = subject;
```

En h guarda sub del usuario que produjo el evento. Toma el dato de subject después de las validaciones de este método. subject.

L194

```
194 |     h.reason = step == Step.COMPLETION ? note.strip() : step.name();
```

En h guarda motivo legible. Toma el dato de step == Step.COMPLETION ? note.strip() : step.name() después de las validaciones de este método. el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L195

```
195 |     h.details = evidence == null ? null : evidence.json();
```

En h guarda evidencia adicional. Toma el dato de evidence == null ? null : evidence.json() después de las validaciones de este método. detecta un dato ausente; el ternario elige la expresión posterior a ? si la condición es verdadera, y la posterior a : en caso contrario.

L196

```
196 |     h.operationKey = key;
```

En h guarda clave durable de acción. Toma el dato de key después de las validaciones de este método. key.

L197

```
197 |     h.requestFingerprint = fingerprint(step, note);
```

En h guarda SHA-256 del contenido asociado a la clave. Conserva evidencia estable de la operación para comparar reintentos o reconstruir las reglas utilizadas, aunque cambie la configuración futura.

L198

```
198 |     h.occurredAt = at;
```

En h guarda instante del evento. Toma el dato de at después de las validaciones de este método. at.

L199

```
199 |     r.status = step.next;
```

En r guarda estado del objeto; su enum o CHECK define valores permitidos. La transición efectiva proviene de step.next y debe estar autorizada por las validaciones previas.

L200

```
200 |     r.updatedAt = at;
```

En r guarda fecha de modificación. Marca el instante de esta operación para ordenar y auditar su evolución. at.

L201

```
201 |     // Column comes exclusively from the enum, never from client input.
```

Documentación para quien lee el fuente: Column comes exclusively from the enum, never from client input. No es una instrucción ejecutable y no impone la regla por sí sola.

L202 a L205

```
202 |     jdbc.update(
203 |         "update assignments set " + step.column + "=? where id=?",
204 |         Timestamp.from(at),
205 |         current.assignmentId());
```

Actualiza la fecha de la etapa usando una columna tomada del enum cerrado Step; el cliente no puede inyectar nombres SQL. Se calcula id (identificador UUID del registro) para reutilizar ese resultado en los pasos siguientes. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L206

```
206 |     if (step == Step.COMPLETION) {
```

Esta condición decide si se entra en la rama siguiente. .

L207

```
207 |       r.closedAt = at;
```

En r guarda fecha de cierre; null indica abierto. Toma el dato de at después de las validaciones de este método. at.

L208

```
208 |       r.closureReasonCode = "SERVICE_COMPLETED";
```

En r guarda causa codificada del cierre. Toma el dato de "SERVICE_COMPLETED" después de las validaciones de este método.

L209

```
209 |       r.resourceReleasePending = true;
```

En r guarda liberación remota pendiente que impide reutilización. Es una bandera durable de trabajo pendiente: después de reiniciar el sistema debe terminar esta recuperación antes de comprometer recursos incompatibles. true.

L210 a L214

```
210 |       jdbc.update(
211 |           "update assignments set status='CLOSED',closed_at=?,completion_note=? where id=?",
212 |           Timestamp.from(at),
213 |           note.strip(),
214 |           current.assignmentId());
```

Actualiza assignments para las filas del filtro. Se calcula status (estado del objeto; su enum o CHECK define valores permitidos) para reutilizar ese resultado en los pasos siguientes. Ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L215 a L220

```
215 |       jdbc.update(
216 |           "insert into pending_actions(id,assignment_id,kind,status,created_at,updated_at) values
       (?,?,'RELEASE','PENDING',?,?) on conflict(assignment_id,kind) do nothing",
217 |           UUID.randomUUID(),
218 |           current.assignmentId(),
219 |           Timestamp.from(at),
220 |           Timestamp.from(at));
```

Genera ID nuevo en la aplicación. Inserta en pending_actions. DO NOTHING evita duplicar la intención/fila. Genera un nuevo identificador UUID; ejecuta escritura SQL parametrizada y devuelve número de filas afectadas.

L217: genera un nuevo identificador UUID.

L221

```
221 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L222

```
222 |     history.saveAndFlush(h);
```

Persiste y fuerza sincronización SQL para detectar restricciones antes de retornar; commit ocurre al terminar transacción. Guarda y sincroniza SQL para detectar conflictos antes de retornar; no confirma por sí solo la transacción.

L223 a L235

```
223 |     org.springframework.transaction.support.TransactionSynchronizationManager
224 |         .registerSynchronization(
225 |             new org.springframework.transaction.support.TransactionSynchronization() {
226 |               @Override
227 |               public void afterCommit() {
228 |                 org.slf4j.LoggerFactory.getLogger(ExecutionStore.class)
229 |                     .info(
230 |                         "Request execution requestId={} from={} to={}",
231 |                         r.id,
232 |                         h.previousStatus,
233 |                         h.newStatus);
234 |               }
235 |             });
```

Registra un callback del ciclo transaccional; no ejecuta el log antes del commit. Implementa callback afterCommit para anunciar solo cambios confirmados. Se calcula requestId (UUID de solicitud) para reutilizar ese resultado en los pasos siguientes. Datos implicados: identificador UUID del registro, estado previo, estado resultante

L236

```
236 |     return view(h);
```

Convierte el registro ya encontrado/validado a su DTO de respuesta. La proyección evita exponer asociaciones JPA y no realiza una nueva transición; el guardado o la consulta sucedió antes.

L237

```
237 |     }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

L238

```
238 | }
```

Delimitación de la estructura Java: } termina el bloque abierto (método, clase o rama);) cierra argumentos y ; finaliza la instrucción. Esta línea no realiza una transición ni una escritura adicional.

Líneas vacías 2, 14, 27, 33, 38, 53, 55, 61, 74, 78, 82, 86, 90, 104, 110, 155, 159, 167: separadores sin instrucciones.

10. Pruebas y utilidades de demostración

Las pruebas no se empaquetan en app.jar. Integración requiere contenedores y puede reiniciarlos; los seeds demo dejan datos persistentes.

BackendClient (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/BackendClient.java`

Objetivo y funcionalidad

Verificación de api-gateway. Infraestructura de pruebas: HTTP, PKCE, tokens en memoria, comandos Docker e identidades temporales. No se empaqueta en el JAR productivo.

Métodos/casos localizables: encode, form, json, claims, initialize, login, call, direct, admin, secondDriver, cleanup, id, key.

No se ejecuta sola: otras pruebas utilizan sus métodos de apoyo.

BackendIntegrationTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/BackendIntegrationTest.java`

Objetivo y funcionalidad

Verificación de api-gateway. Keycloak/Gateway/BD reales: seguridad, flota, vehículos, solicitudes, propiedad, concurrencia y persistencia tras reinicio; rutas reales bajo indicador optativo.

Métodos/casos localizables: setup, finish, api, a, o, d, dk, identityAndSecurityRegression, fleetAdministrationRegression, vehiclesRequestsAndCancellation, requestSnapshotAndHistorySurviveRestart, realRouteAndFleetEvaluation, routingBudgetState.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dauxiruta.integration=true"
       "-Dtest=BackendIntegrationTest"
```

Requiere Docker y entorno local.

Añadir "-Dauxiruta.realRouting=true" para exigir rutas reales.

DeliveryArtifacts (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/DeliveryArtifacts.java`

Objetivo y funcionalidad

Verificación de api-gateway. Define el plan de 47 pasos, scripts/capturas Postman y entorno sin secretos. Es soporte de entrega, no controlador de negocio.

Métodos/casos localizables: requestBody, write, event, initScript, testScript.

No se ejecuta sola: otras pruebas utilizan sus métodos de apoyo.

DeliveryArtifactsTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/DeliveryArtifactsTest.java`

Objetivo y funcionalidad

Verificación de api-gateway. Compara colección y entorno publicados con la definición Java; exporta solo bajo indicador explícito.

Métodos/casos localizables: committedCollectionMatchesExecutableJavaPlan.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dtest=DeliveryArtifactsTest"
```

No requiere Docker: dependencias controladas.

DeliveryIntegrationTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/DeliveryIntegrationTest.java`

Objetivo y funcionalidad

Verificación de api-gateway. Comprueba contratos OpenAPI y ejecuta por Java las llamadas de la colección. No ejecuta el sandbox JavaScript de Postman.

Métodos/casos localizables: login, openApiMatchesControllersAndExport, executePublishedCollectionWithJava, resolve, cleanupOwnFixtures.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dauxiruta.delivery=true"
      "-Dtest=DeliveryIntegrationTest"
```

Requiere Docker y entorno local.

DemoDataSetupTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/DemoDataSetupTest.java`

Objetivo y funcionalidad

Verificación de api-gateway. Prepara operador principal y habilitación; incorpora preparación persistente de privacidad. Requiere recurso libre.

Métodos/casos localizables: prepareOperator, prepare.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dauxiruta.seed=true" "-Dtest=DemoDataSetupTest"
```

Requiere Docker y entorno local.

Conserva datos de demostración al finalizar.

KeycloakScopeSetupTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/KeycloakScopeSetupTest.java`

Objetivo y funcionalidad

Verificación de api-gateway. Reparación optativa e idempotente del scope basic en instalaciones antiguas de Keycloak.

Métodos/casos localizables: ensureBasicScopeWithoutDeletingRealm.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dauxiruta.repairIdentity=true"
      "-Dtest=KeycloakScopeSetupTest"
```

Requiere Docker y entorno local.

Phase7IntegrationTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/Phase7IntegrationTest.java`

Objetivo y funcionalidad

Verificación de api-gateway. Reservas exclusivas, carrera aceptación/cancelación, rechazo, recuperación de confirmación y evidencia conservada durante oferta.

Métodos/casos localizables: initializeClient, resources, request, proposal, exclusiveReservationAndIdempotence, acceptanceAndCancellationRace, acceptsDurablyAndBlocksAdministration, rejectionReleasesAndIsIdempotent, lostConfirmationRecoveredAfterRestart, sql, internal, cleanupFixtures, fixtureSql, activeProposalKeepsItsRouteEvidenceUntilAcceptance.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dauxiruta.phase7=true"
      "-Dtest=Phase7IntegrationTest"
```

Requiere Docker y entorno local.

Phase8IntegrationTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/Phase8IntegrationTest.java`

Objetivo y funcionalidad

Verificación de api-gateway. Recorrido hasta COMPLETED, pasos inválidos, despacho vencido y entrega durante caída de Fleet.

Métodos/casos localizables: initializeClient, removeTemporaryIdentity, setup, path, action, toTransit, completesServiceAndPreservesIdempotentHistory, expiredDispatchIsRejected, completionKeepsPendingReleaseAcrossFleetOutage, sql, container, startFleet, cleanup.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dauxiruta.phase8=true"
       "-Dtest=Phase8IntegrationTest"
```

Requiere Docker y entorno local.

Phase9IntegrationTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/Phase9IntegrationTest.java`

Objetivo y funcionalidad

Verificación de api-gateway. Perfil demo: búsqueda, vencimientos, cupos, reemplazos, exclusión por rechazo y recuperación tras reinicios.

Métodos/casos localizables: initializeClient, resources, request, current, ready, await, active, proposal, accept, sql, uuid, availabilityTriggersSearchAndOfferExpiryPreservesAttempt, emptySearchClosesAutomatically, twoReassignmentsThenCloseWithoutRestoringCancellation, confirmationIsRecoveredAfterRestartAndDeadline, cancelledUncertainReserveIsCompensatedAutomatically, releaseSurvivesRestartWithoutRepeatingCompletion, fiveExpiredOffersExhaustTheSearchWithoutChangingDeadline, startFleet, cleanup, rejectingOperatorIsExcludedInNextAttempt.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dauxiruta.phase9=true"
       "-Dtest=Phase9IntegrationTest"
```

Requiere Docker y entorno local.

Arrancar con perfil docker,demo y restaurar docker al terminar.

PrivacyDemoSetupTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/PrivacyDemoSetupTest.java`

Objetivo y funcionalidad

Verificación de api-gateway. Crea/reutiliza cuentas secundarias y vehículos demo; comprueba 200 propios, 404 cruzados y listado aislado. Deja datos persistentes.

Métodos/casos localizables: prepareAndVerifyPrivacy, prepare, ensureIdentity, ensureOperator, ensureVehicle, findVehicle.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dauxiruta.privacySeed=true"
       "-Dtest=PrivacyDemoSetupTest"
```

Requiere Docker y entorno local.

Conserva datos de demostración al finalizar.

SecurityConfigurationTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/config/SecurityConfigurationTest.java`

Objetivo y funcionalidad

Verificación de assistance-service. Acceso sin token, rol permitido, rol incorrecto y separación de rutas internas.

Métodos/casos localizables: publicRoute, internalRoute, noTokenRejected, authorizedRoleAccepted, wrongRoleRejected, humanCannotCallInternal.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=SecurityConfigurationTest"
```

No requiere Docker: dependencias controladas.

AssignmentCoordinatorTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/AssignmentCoordinatorTest.java`

Objetivo y funcionalidad

Verificación de assistance-service. Respuesta perdida, compensación, recuperación con payload original, claim y Retry-After.

Métodos/casos localizables: allowClaims, lostConfirmationResponseRemainsDurableWithoutImmediateRetry, cancellationCompensatesTheSameReservationAttempt, recoveryReusesPersistedAttemptAndPayload, anotherWorkerCannotExecuteAnAlreadyClaimedAction, retryAfterHonorsSecondsAndHttpDate.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=AssignmentCoordinatorTest"
```

No requiere Docker: dependencias controladas.

AutomationConfigurationTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/AutomationConfigurationTest.java`

Objetivo y funcionalidad

Verificación de assistance-service. Tiempos demo y backoff, preservando cuotas y planificación.

Métodos/casos localizables: demoChangesOnlyWaitsNotTravelPlanningOrQuotas, recoveryBackoffRemainsBoundedAndNeverDiscards.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=AutomationConfigurationTest"
```

No requiere Docker: dependencias controladas.

AutomationStoreTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/AutomationStoreTest.java`

Objetivo y funcionalidad

Verificación de assistance-service. Cierres, incertidumbre de confirmación, liberación previa al reemplazo y cupos.

Métodos/casos localizables: setup, finish, pendingConfirmationPreventsDeadlineClosure, deadlineClosesWithoutExtendingSearch, dispatchExpiryWaitsForReleaseBeforeStartingNewDeadline, expiredOfferDuringReplacementDoesNotConsumeAnotherReassignment, thirdAcceptedAttemptCannotStartFourthSearch, fiveCreatedOffersCloseTheAttempt, executionStatesAreNeverAutomaticallyReassigned.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=AutomationStoreTest"
```

No requiere Docker: dependencias controladas.

DispatchValidatorTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/DispatchValidatorTest.java`

Objetivo y funcionalidad

Verificación de assistance-service. Compromiso, cobertura, rutas, habilitación y tolerancia a la edad de ubicación después de aceptar.

Métodos/casos localizables: setup, resource, validResourceProducesAuditableEvidence, acceptedLocationDoesNotExpireForDispatch, qualificationMustOutlivePlannedEndStrictly, unconfirmedResourceCannotDispatch, quotaIsTemporaryFailure, confirmedMissingRouteIsDomainConflict, malformedFleetResponseIsTechnicalFailure.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=DispatchValidatorTest"
```

No requiere Docker: dependencias controladas.

EvaluationServiceTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/EvaluationServiceTest.java`

Objetivo y funcionalidad

Verificación de assistance-service. Cobertura, redondeo de duración, límites de recorridos, ranking y vigencia de habilitación.

Métodos/casos localizables: setup, distanceLimitsAreInclusiveAndNotRounded, adjustedSecondsRoundUp, transportDurationBoundary, coverageFailureAvoidsExternalCalls, dependencyFailureDoesNotReject, missingRouteIsARecordedDomainRejection, rankingUsesDurationThenFreeTimeThenLexicalUuid, qualificationMustExtendStrictlyBeyondPlannedEnd, expiredOrClosedRequestIsNotEvaluated, cancellationRaceDiscardsResultAtCommit.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=EvaluationServiceTest"
```

No requiere Docker: dependencias controladas.

EvaluationStoreTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/EvaluationStoreTest.java`

Objetivo y funcionalidad

Verificación de assistance-service. Descartar una evaluación incompatible con una cancelación o modificación concurrente.

Métodos/casos localizables: concurrentCancellationCannotBeOverwritten.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=EvaluationStoreTest"
```

No requiere Docker: dependencias controladas.

ExecutionRulesTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/ExecutionRulesTest.java`

Objetivo y funcionalidad

Verificación de assistance-service. Predecesor de cada paso, plazo exclusivo de despacho y huella de operación.

Métodos/casos localizables: everyActionAcceptsOnlyItsPredecessor, dispatchDeadlineIsExclusive, sameKeyCannotRepresentAnotherStepOrNote.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=ExecutionRulesTest"
```

No requiere Docker: dependencias controladas.

ExecutionServiceTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/ExecutionServiceTest.java`

Objetivo y funcionalidad

Verificación de assistance-service. Fallo de despacho sin transición y repetición de entrega sin duplicar el evento.

Métodos/casos localizables: replayedCompletionRetriesReleaseWithoutAnotherTransition, failedDispatchDoesNotChangeState.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=ExecutionServiceTest"
```

No requiere Docker: dependencias controladas.

RequestRulesTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/RequestRulesTest.java`

Objetivo y funcionalidad

Verificación de assistance-service. Patentes, fingerprints, snapshot, exclusividad de cancelación y copia de datos del vehículo.

Métodos/casos localizables: fingerprintNormalizesDecimalScale, changedBodyHasDifferentFingerprint, previousAcceptanceBlocksCancellationEvenDuringAlternativeSearch, terminalAndExecutionStatesCannotBeCancelled, onlyPreAcceptanceStatesCanBeCancelled, plateNormalizedAndInvalidRejected, snapshotIncludesAllParametersAndCorrectDeadline, creationUsesInjectedClockAndCopiesVehicleData.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=RequestRulesTest"
```

No requiere Docker: dependencias controladas.

OpenRouteServiceClientTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/routing/OpenRouteServiceClientTest.java`

Objetivo y funcionalidad

Verificación de assistance-service. Servidor HTTP Java local: caché, parsing, errores ORS, cuota, snap y clave ausente; no consume ORS real.

Métodos/casos localizables: setup, stop, parsesMetricRouteAndCachesSameRevision, noRouteIsDomainFailure, unknownServerFailureIsNotDomainRejection, quotaRespectsRetryAfterWithoutImmediateRetry, dailyQuotaDiffersFromInvalidCredentials, localBudgetBlocksNetwork, malformedSuccessRemainsPending, excessiveSnapIsRejected, missingKeyDoesNotConsumeQuota.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=OpenRouteServiceClientTest"
```

No requiere Docker: dependencias controladas.

RoutingBudgetTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/routing/RoutingBudgetTest.java`

Objetivo y funcionalidad

Verificación de assistance-service. Presupuesto por minuto/día y bloqueo impuesto por el proveedor con JDBC simulado.

Métodos/casos localizables: setup, consumesBudgetBeforeNetworkCall, minuteLimitPreventsReservation, dailyLimitPreventsReservation, providerBackoffPreventsReservation.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=RoutingBudgetTest"
```

No requiere Docker: dependencias controladas.

TokenSupportTest (auxiruta-commons)

`Ruta: auxiruta-commons/src/test/java/com/auxiruta/commons/security/TokenSupportTest.java`

Objetivo y funcionalidad

Verificación de auxiruta-commons. JWT válido/inválido, emisor, audiencias, roles humanos/técnicos y propiedad.

Métodos/casos localizables: correctAudienceAndIssuerAccepted, wrongIssuerRejected, wrongAudienceRejected, expiredRejected, adminCannotImpersonateDriver, driverAllowed, technicalClientSeparated, wrongAuthorizedPartyRejected, unknownRolesIgnored, ownerAllowedAndForeignResourceHidden, missingAudienceRejectedWithoutException, missingExpiryOrSubjectRejected.

Ejecución desde raíz

```
.\auxiruta-commons\mvnw.cmd -f auxiruta-commons/pom.xml test "-Dtest=TokenSupportTest"
```

No requiere Docker: dependencias controladas.

CandidateServiceTest (fleet-service)

`Ruta: fleet-service/src/test/java/com/auxiruta/fleet/catalog/CandidateServiceTest.java`

Objetivo y funcionalidad

Verificación de fleet-service. Edad inclusiva de ubicación, capacidad, movilidad, habilitación, cobertura y máximo de candidatos.

Métodos/casos localizables: locationAgeInclusive, incompatibleCapacityAndMobilityExcluded, expiredQualificationAndOutsideCoverageExcluded, atMostFiveCandidates.

Ejecución desde raíz

```
.\fleet-service\mvnw.cmd -f fleet-service/pom.xml test "-Dtest=CandidateServiceTest"
```

No requiere Docker: dependencias controladas.

FleetServiceTest (fleet-service)

`Ruta: fleet-service/src/test/java/com/auxiruta/fleet/catalog/FleetServiceTest.java`

Objetivo y funcionalidad

Verificación de fleet-service. Disponibilidad idempotente, habilitación, reloj, perfil propio y bloqueo de ubicación de recursos ocupados.

Métodos/casos localizables: reservations, repeatedAvailabilityDoesNotResetFreeTime, invalidQualificationDoesNotWrite, locationTimestampComesFromClockWithDatabasePrecision, missingOwnProfileIsHidden, replayReturnsOriginalEventWithoutReapplyingOldState, committedTruckCannotChangeLocationButReleasedTruckCan.

Ejecución desde raíz

```
.\fleet-service\mvnw.cmd -f fleet-service/pom.xml test "-Dtest=FleetServiceTest"
```

No requiere Docker: dependencias controladas.

ReservationServiceTest (fleet-service)

`Ruta: fleet-service/src/test/java/com/auxiruta/fleet/catalog/ReservationServiceTest.java`

Objetivo y funcionalidad

Verificación de fleet-service. Reserva/confirmación/liberación, límites de tiempo, idempotencia, compatibilidad demo y ubicación antigua tras aceptar.

Métodos/casos localizables: setup, holdExpiresAtSixtySeconds, shortSearchWindowCreatesDurableTerminalAttempt, activeCommitmentExcludesNewAttempt, confirmationAtExactExpiryIsRejected, confirmedReservationDoesNotExpire, invalidQualificationReleasesHold, releaseIsIdempotent, confirmationCommitsBeforeDeadline, demoWindowComesFromTheRequestSnapshot, legacyPayloadWithoutTimingFieldsRemainsIdempotent, confirmedLocationDoesNotExpire, oldLocationCannotBeConfirmed.

Ejecución desde raíz

```
.\fleet-service\mvnw.cmd -f fleet-service/pom.xml test "-Dtest=ReservationServiceTest"
```

No requiere Docker: dependencias controladas.

SecurityConfigurationTest (fleet-service)

`Ruta: fleet-service/src/test/java/com/auxiruta/fleet/config/SecurityConfigurationTest.java`

Objetivo y funcionalidad

Verificación de fleet-service. Acceso sin token, rol permitido, rol incorrecto y separación de rutas internas.

Métodos/casos localizables: publicRoute, internalRoute, noTokenRejected, authorizedRoleAccepted, wrongRoleRejected, humanCannotCallInternal, technicalClientAcceptedInternally.

Ejecución desde raíz

```
.\fleet-service\mvnw.cmd -f fleet-service/pom.xml test "-Dtest=SecurityConfigurationTest"
```

No requiere Docker: dependencias controladas.

11. Maven Wrapper: herramientas distribuidas

Maven Wrapper es código de terceros distribuido con cada módulo. Se explica uso/configuración, sin reproducir implementaciones repetidas de descarga/plataforma como lógica propia.

api-gateway/.mvn/wrapper/maven-wrapper.properties

`Ruta: api-gateway/.mvn/wrapper/maven-wrapper.properties`

Objetivo y funcionalidad

Fija la distribución Maven descargada por los lanzadores de api-gateway. Determina la herramienta reproducible de compilación, no la versión del backend.

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 3 líneas de herramienta distribuida.

L1

```
wrapperVersion=3.3.4
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

L2

```
distributionType=only-script
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

L3

```
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.11/apache-maven-3
       .9.11-bin.zip
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

api-gateway/mvnw

`Ruta: api-gateway/mvnw`

Objetivo y funcionalidad

Lanza Maven para api-gateway en Unix/Linux sin instalación global de Maven: localiza Java, descarga la distribución si falta y propaga argumentos y código de salida.

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 295 líneas de herramienta distribuida.

Uso: .\api-gateway\mvnw.cmd -f api-gateway/pom.xml verify. install de Commons publica el JAR en .m2.

api-gateway/mvnw.cmd

`Ruta: api-gateway/mvnw.cmd`

Objetivo y funcionalidad

Lanza Maven para api-gateway en Windows sin instalación global de Maven: localiza Java, descarga la distribución si falta y propaga argumentos y código de salida.

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 189 líneas de herramienta distribuida.

Uso: .\api-gateway\mvnw.cmd -f api-gateway/pom.xml verify. install de Commons publica el JAR en .m2.

assistance-service/.mvn/wrapper/maven-wrapper.properties

`Ruta: assistance-service/.mvn/wrapper/maven-wrapper.properties`

Objetivo y funcionalidad

Fija la distribución Maven descargada por los lanzadores de assistance-service. Determina la herramienta reproducible de compilación, no la versión del backend.

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 3 líneas de herramienta distribuida.

L1

```
wrapperVersion=3.3.4
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

L2

```
distributionType=only-script
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

L3

```
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.11/apache-maven-3
       .9.11-bin.zip
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

assistance-service/mvnw

`Ruta: assistance-service/mvnw`

Objetivo y funcionalidad

Lanza Maven para assistance-service en Unix/Linux sin instalación global de Maven: localiza Java, descarga la distribución si falta y propaga argumentos y código de salida.

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 295 líneas de herramienta distribuida.

Uso: .\assistance-service\mvnw.cmd -f assistance-service/pom.xml verify. install de Commons publica el JAR en .m2.

assistance-service/mvnw.cmd

`Ruta: assistance-service/mvnw.cmd`

Objetivo y funcionalidad

Lanza Maven para assistance-service en Windows sin instalación global de Maven: localiza Java, descarga la distribución si falta y propaga argumentos y código de salida.

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 189 líneas de herramienta distribuida.

Uso: .\assistance-service\mvnw.cmd -f assistance-service/pom.xml verify. install de Commons publica el JAR en .m2.

auxiruta-commons/.mvn/wrapper/maven-wrapper.properties

`Ruta: auxiruta-commons/.mvn/wrapper/maven-wrapper.properties`

Objetivo y funcionalidad

Fija la distribución Maven descargada por los lanzadores de auxiruta-commons. Determina la herramienta reproducible de compilación, no la versión del backend.

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 3 líneas de herramienta distribuida.

L1

```
wrapperVersion=3.3.4
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

L2

```
distributionType=only-script
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

L3

```
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.11/apache-maven-3
       .9.11-bin.zip
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

auxiruta-commons/mvnw

`Ruta: auxiruta-commons/mvnw`

Objetivo y funcionalidad

Lanza Maven para auxiruta-commons en Unix/Linux sin instalación global de Maven: localiza Java, descarga la distribución si falta y propaga argumentos y código de salida.

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 295 líneas de herramienta distribuida.

Uso: .\auxiruta-commons\mvnw.cmd -f auxiruta-commons/pom.xml verify. install de Commons publica el JAR en .m2.

auxiruta-commons/mvnw.cmd

`Ruta: auxiruta-commons/mvnw.cmd`

Objetivo y funcionalidad

Lanza Maven para auxiruta-commons en Windows sin instalación global de Maven: localiza Java, descarga la distribución si falta y propaga argumentos y código de salida.

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 189 líneas de herramienta distribuida.

Uso: .\auxiruta-commons\mvnw.cmd -f auxiruta-commons/pom.xml verify. install de Commons publica el JAR en .m2.

fleet-service/.mvn/wrapper/maven-wrapper.properties

`Ruta: fleet-service/.mvn/wrapper/maven-wrapper.properties`

Objetivo y funcionalidad

Fija la distribución Maven descargada por los lanzadores de fleet-service. Determina la herramienta reproducible de compilación, no la versión del backend.

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 3 líneas de herramienta distribuida.

L1

```
wrapperVersion=3.3.4
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

L2

```
distributionType=only-script
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

L3

```
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.11/apache-maven-3
       .9.11-bin.zip
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

fleet-service/mvnw

`Ruta: fleet-service/mvnw`

Objetivo y funcionalidad

Lanza Maven para fleet-service en Unix/Linux sin instalación global de Maven: localiza Java, descarga la distribución si falta y propaga argumentos y código de salida.

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 295 líneas de herramienta distribuida.

Uso: .\fleet-service\mvnw.cmd -f fleet-service/pom.xml verify. install de Commons publica el JAR en .m2.

fleet-service/mvnw.cmd

`Ruta: fleet-service/mvnw.cmd`

Objetivo y funcionalidad

Lanza Maven para fleet-service en Windows sin instalación global de Maven: localiza Java, descarga la distribución si falta y propaga argumentos y código de salida.

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 189 líneas de herramienta distribuida.

Uso: .\fleet-service\mvnw.cmd -f fleet-service/pom.xml verify. install de Commons publica el JAR en .m2.

12. JAR: binarios y contenido empaquetado

Los binarios presentes se inspeccionan sin recompilar. No se afirma que correspondan a cambios de fuente posteriores. La explicación línea por línea de Java está en las secciones de fuente.

api-gateway/target/app.jar

`Ruta: api-gateway/target/app.jar`

Objetivo y funcionalidad

Ejecutable de api-gateway: reúne bytecode, recursos y dependencias en BOOT-INF. El contenedor lo inicia con java -jar; no contiene los fuentes de los tests.

Tamaño: 39.41 MiB; 216 entradas; 5 clases del proyecto; 74 dependencias anidadas.

Ejecutable Spring Boot: BOOT-INF/classes contiene clases/recursos; BOOT-INF/lib las dependencias. El launcher inicia la Start-Class. Usar java -jar con entorno y dependencias configurados.

Manifest real

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.4.2
Build-Jdk-Spec: 21
Implementation-Title: api-gateway
Implementation-Version: 1.0.0
Main-Class: org.springframework.boot.loader.launch.JarLauncher
Start-Class: com.auxiruta.gateway.GatewayApplication
Spring-Boot-Version: 3.5.16
Spring-Boot-Classes: BOOT-INF/classes/
Spring-Boot-Lib: BOOT-INF/lib/
Spring-Boot-Classpath-Index: BOOT-INF/classpath.idx
Spring-Boot-Layers-Index: BOOT-INF/layers.idx
```

Main-Class identifica el launcher; Start-Class la aplicación delegada; Build-Jdk-Spec la versión de Java de construcción, cuando esos campos están presentes.

Clases del proyecto incluidas

`BOOT-INF/classes/com/auxiruta/gateway/config/GatewayRoutes.class`

`BOOT-INF/classes/com/auxiruta/gateway/config/RequestLoggingFilter.class`

`BOOT-INF/classes/com/auxiruta/gateway/config/SecurityConfiguration.class`

`BOOT-INF/classes/com/auxiruta/gateway/config/SecurityErrors.class`

`BOOT-INF/classes/com/auxiruta/gateway/GatewayApplication.class`

Dependencias empaquetadas

`spring-boot-3.5.16.jar | spring-boot-autoconfigure-3.5.16.jar | logback-classic-1.5.34.jar | logback-core-1.5.34.jar | log4j-to-slf4j-2.24.3.jar`

`log4j-api-2.24.3.jar | jul-to-slf4j-2.0.18.jar | jakarta.annotation-api-2.1.1.jar | snakeyaml-2.4.jar | spring-security-config-6.5.11.jar`

`spring-aop-6.2.19.jar | spring-beans-6.2.19.jar | spring-context-6.2.19.jar | spring-security-core-6.5.11.jar | spring-security-crypto-6.5.11.jar`

`spring-expression-6.2.19.jar | micrometer-observation-1.15.12.jar | micrometer-commons-1.15.12.jar | spring-security-oauth2-resource-server-6.5.11.jar | spring-security-oauth2-core-6.5.11.jar`

`spring-security-oauth2-jose-6.5.11.jar | nimbus-jose-jwt-9.37.4.jar | jcip-annotations-1.0-1.jar | slf4j-api-2.0.18.jar | android-json-0.0.20131108.vaadin1.jar`

`spring-core-6.2.19.jar | spring-jcl-6.2.19.jar | spring-security-web-6.5.11.jar | spring-cloud-starter-gateway-server-webflux-4.3.5.jar | spring-cloud-starter-4.3.3.jar`

`spring-cloud-context-4.3.3.jar | spring-cloud-commons-4.3.3.jar | bcprov-jdk18on-1.80.2.jar | spring-cloud-gateway-server-webflux-4.3.5.jar | spring-cloud-gateway-server-4.3.5.jar`

`tomcat-embed-el-10.1.55.jar | hibernate-validator-8.0.3.Final.jar | jakarta.validation-api-3.0.2.jar | jboss-logging-3.6.3.Final.jar | classmate-1.7.3.jar`

`reactor-extra-3.5.5.jar | jackson-databind-2.21.4.jar | jackson-annotations-2.21.jar | jackson-core-2.21.4.jar | jackson-datatype-jdk8-2.21.4.jar`

jackson-datatype-jsr310-2.21.4.jar | jackson-module-parameter-names-2.21.4.jar | reactor-netty-http-1.2.18.jar | netty-codec-http-4.1.135.Final.jar | netty-common-4.1.135.Final.jar

netty-buffer-4.1.135.Final.jar | netty-transport-4.1.135.Final.jar | netty-codec-4.1.135.Final.jar | netty-handler-4.1.135.Final.jar | netty-codec-http2-4.1.135.Final.jar

netty-resolver-dns-4.1.135.Final.jar | netty-resolver-4.1.135.Final.jar | netty-codec-dns-4.1.135.Final.jar | netty-resolver-dns-native-macos-4.1.135.Final-osx-x86_64.jar | netty-resolver-dns-classes-macos-4.1.135.Final.jar

netty-transport-native-epoll-4.1.135.Final-linux-x86_64.jar | netty-transport-native-unix-common-4.1.135.Final.jar | netty-transport-classes-epoll-4.1.135.Final.jar | reactor-netty-core-1.2.18.jar | netty-handler-proxy-4.1.135.Final.jar

netty-codec-socks-4.1.135.Final.jar | spring-webflux-6.2.19.jar | reactor-core-3.7.19.jar | reactive-streams-1.0.4.jar | spring-boot-properties-migrator-3.5.16.jar

spring-boot-configuration-metadata-3.5.16.jar | auxiruta-commons-1.0.0.jar | spring-web-6.2.19.jar | spring-boot-jarmode-tools-3.5.16.jar

api-gateway/target/app.jar.original

Ruta: api-gateway/target/app.jar.original

Objetivo y funcionalidad

Conserva el JAR de api-gateway anterior al reempaquetado Boot: clases y recursos propios, sin el ensamblado ejecutable completo con dependencias.

Tamaño: 0.01 MiB; 18 entradas; 5 clases del proyecto; 0 dependencias anidadas.

JAR previo al reempaquetado Boot: clases/recursos sin ensamblado completo de dependencias; Docker usa app.jar.

Manifest real

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.4.2
Build-Jdk-Spec: 21
Implementation-Title: api-gateway
Implementation-Version: 1.0.0
```

Main-Class identifica el launcher; Start-Class la aplicación delegada; Build-Jdk-Spec la versión de Java de construcción, cuando esos campos están presentes.

Clases del proyecto incluidas

com/auxiruta/gateway/config/GatewayRoutes.class

com/auxiruta/gateway/config/RequestLoggingFilter.class

com/auxiruta/gateway/config/SecurityConfiguration.class

com/auxiruta/gateway/config/SecurityErrors.class

com/auxiruta/gateway/GatewayApplication.class

assistance-service/target/app.jar

Ruta: assistance-service/target/app.jar

Objetivo y funcionalidad

Ejecutable de assistance-service: reúne bytecode, recursos y dependencias en BOOT-INF. El contenedor lo inicia con java -jar; no contiene los fuentes de los tests.

Tamaño: 61.03 MiB; 311 entradas; 78 clases del proyecto; 86 dependencias anidadas.

Ejecutable Spring Boot: BOOT-INF/classes contiene clases/recursos; BOOT-INF/lib las dependencias. El launcher inicia la Start-Class. Usar java -jar con entorno y dependencias configurados.

Manifest real

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.4.2
Build-Jdk-Spec: 21
Implementation-Title: assistance-service
Implementation-Version: 1.0.0
Main-Class: org.springframework.boot.loader.launch.JarLauncher
Start-Class: com.auxiruta.assistance.AssistanceApplication
Spring-Boot-Version: 3.5.16
Spring-Boot-Classes: BOOT-INF/classes/
Spring-Boot-Lib: BOOT-INF/lib/
Spring-Boot-Classpath-Index: BOOT-INF/classpath.idx
Spring-Boot-Layers-Index: BOOT-INF/layers.idx
```

Main-Class identifica el launcher; Start-Class la aplicación delegada; Build-Jdk-Spec la versión de Java de construcción, cuando esos campos están presentes.

Clases del proyecto incluidas

BOOT-INF/classes/com/auxiruta/assistance/AssistanceApplication.class

BOOT-INF/classes/com/auxiruta/assistance/config/ClockConfiguration.class

BOOT-INF/classes/com/auxiruta/assistance/config/OpenApiConfiguration.class

BOOT-INF/classes/com/auxiruta/assistance/config/RequestLoggingFilter.class

BOOT-INF/classes/com/auxiruta/assistance/config/SecurityConfiguration.class

BOOT-INF/classes/com/auxiruta/assistance/config/SecurityErrors.class

BOOT-INF/classes/com/auxiruta/assistance/exceptions/ApiExceptionHandler.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AssignmentController$Rejection.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AssignmentController.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AssignmentCoordinator.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AssignmentStore$1.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AssignmentStore$Action.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AssignmentStore$View.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AssignmentStore.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AssistanceRequest.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AutomationJobs.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AutomationStore$1.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AutomationStore$Work.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AutomationStore.class

BOOT-INF/classes/com/auxiruta/assistance/requests/CandidateEvaluation.class

BOOT-INF/classes/com/auxiruta/assistance/requests/CandidateEvaluationRepository.class

BOOT-INF/classes/com/auxiruta/assistance/requests/DispatchValidator$Evidence.class

BOOT-INF/classes/com/auxiruta/assistance/requests/DispatchValidator.class

BOOT-INF/classes/com/auxiruta/assistance/requests/DomainTypes$Mobility.class

BOOT-INF/classes/com/auxiruta/assistance/requests/DomainTypes$Registration.class

BOOT-INF/classes/com/auxiruta/assistance/requests/DomainTypes$Status.class

BOOT-INF/classes/com/auxiruta/assistance/requests/DomainTypes$VehicleType.class

BOOT-INF/classes/com/auxiruta/assistance/requests/DomainTypes.class

BOOT-INF/classes/com/auxiruta/assistance/requests/Evaluation.class

BOOT-INF/classes/com/auxiruta/assistance/requests/EvaluationReadService$CandidateView.class

BOOT-INF/classes/com/auxiruta/assistance/requests/EvaluationReadService$EvaluationView.class

BOOT-INF/classes/com/auxiruta/assistance/requests/EvaluationReadService.class

BOOT-INF/classes/com/auxiruta/assistance/requests/EvaluationRepository.class

BOOT-INF/classes/com/auxiruta/assistance/requests/EvaluationService.class

BOOT-INF/classes/com/auxiruta/assistance/requests/EvaluationStore$1.class

BOOT-INF/classes/com/auxiruta/assistance/requests/EvaluationStore$Input.class

BOOT-INF/classes/com/auxiruta/assistance/requests/EvaluationStore.class

BOOT-INF/classes/com/auxiruta/assistance/requests/ExecutionController$Completion.class

BOOT-INF/classes/com/auxiruta/assistance/requests/ExecutionController.class

BOOT-INF/classes/com/auxiruta/assistance/requests/ExecutionService.class

BOOT-INF/classes/com/auxiruta/assistance/requests/ExecutionStore$1.class

BOOT-INF/classes/com/auxiruta/assistance/requests/ExecutionStore$Input.class

BOOT-INF/classes/com/auxiruta/assistance/requests/ExecutionStore$Preparation.class

BOOT-INF/classes/com/auxiruta/assistance/requests/ExecutionStore$Step.class

BOOT-INF/classes/com/auxiruta/assistance/requests/ExecutionStore.class

BOOT-INF/classes/com/auxiruta/assistance/requests/HistoryRepository.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestController.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$Cancellation.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$Created.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$CreateRequest.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$CreateVehicle.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$EditVehicle.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$HistoryView.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$PageView.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$RegistrationChange.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$RequestView.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$VehicleView.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestRepository.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestService$1.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestService.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestStatusHistory.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RulesConfiguration$1.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RulesConfiguration.class

BOOT-INF/classes/com/auxiruta/assistance/requests/Vehicle.class

BOOT-INF/classes/com/auxiruta/assistance/requests/VehicleRepository.class

BOOT-INF/classes/com/auxiruta/assistance/routing/FleetClient$1.class

BOOT-INF/classes/com/auxiruta/assistance/routing/FleetClient$2.class

BOOT-INF/classes/com/auxiruta/assistance/routing/FleetClient$Candidate.class

BOOT-INF/classes/com/auxiruta/assistance/routing/FleetClient$Eligibility.class

BOOT-INF/classes/com/auxiruta/assistance/routing/FleetClient.class

BOOT-INF/classes/com/auxiruta/assistance/routing/OpenRouteServiceClient.class

BOOT-INF/classes/com/auxiruta/assistance/routing/RouteProvider$Point.class

BOOT-INF/classes/com/auxiruta/assistance/routing/RouteProvider$Route.class

BOOT-INF/classes/com/auxiruta/assistance/routing/RouteProvider.class

BOOT-INF/classes/com/auxiruta/assistance/routing/RoutingBudget.class

BOOT-INF/classes/com/auxiruta/assistance/routing/RoutingFailure$Reason.class

BOOT-INF/classes/com/auxiruta/assistance/routing/RoutingFailure.class

Dependencias empaquetadas

spring-boot-3.5.16.jar | spring-boot-autoconfigure-3.5.16.jar | logback-classic-1.5.34.jar | logback-core-1.5.34.jar | log4j-to-slf4j-2.24.3.jar

log4j-api-2.24.3.jar | jul-to-slf4j-2.0.18.jar | jakarta.annotation-api-2.1.1.jar | snakeyaml-2.4.jar | spring-security-config-6.5.11.jar

spring-aop-6.2.19.jar | spring-beans-6.2.19.jar | spring-context-6.2.19.jar | spring-security-core-6.5.11.jar | spring-security-crypto-6.5.11.jar

spring-expression-6.2.19.jar | micrometer-observation-1.15.12.jar | micrometer-commons-1.15.12.jar | spring-security-oauth2-resource-server-6.5.11.jar | spring-security-oauth2-core-6.5.11.jar

spring-security-oauth2-jose-6.5.11.jar | nimbus-jose-jwt-9.37.4.jar | jcip-annotations-1.0-1.jar | jackson-databind-2.21.4.jar | jackson-annotations-2.21.jar

jackson-core-2.21.4.jar | jackson-datatype-jdk8-2.21.4.jar | jackson-datatype-jsr310-2.21.4.jar | jackson-module-parameter-names-2.21.4.jar | tomcat-embed-core-10.1.55.jar

tomcat-embed-websocket-10.1.55.jar | spring-web-6.2.19.jar | spring-webmvc-6.2.19.jar | HikariCP-6.3.3.jar | spring-jdbc-6.2.19.jar

hibernate-core-6.6.53.Final.jar | jakarta.persistence-api-3.1.0.jar | jakarta.transaction-api-2.0.1.jar | jboss-logging-3.6.3.Final.jar | hibernate-commons-annotations-7.0.3.Final.jar

jandex-3.2.0.jar | classmate-1.7.3.jar | byte-buddy-1.17.8.jar | jaxb-runtime-4.0.9.jar | jaxb-core-4.0.9.jar

angus-activation-2.0.3.jar | txw2-4.0.9.jar | istack-commons-runtime-4.1.2.jar | jakarta.inject-api-2.0.1.jar | antlr4-runtime-4.13.2.jar

spring-data-jpa-3.5.13.jar | spring-data-commons-3.5.13.jar | spring-orm-6.2.19.jar | spring-tx-6.2.19.jar | slf4j-api-2.0.18.jar

spring-aspects-6.2.19.jar | aspectjweaver-1.9.25.1.jar | tomcat-embed-el-10.1.55.jar | hibernate-validator-8.0.3.Final.jar | jakarta.validation-api-3.0.2.jar

spring-security-oauth2-client-6.5.11.jar | oauth2-oidc-sdk-9.43.6.jar | content-type-2.2.jar | lang-tag-1.7.jar | jakarta.xml.bind-api-4.0.5.jar

jakarta.activation-api-2.1.4.jar | json-smart-2.5.2.jar | accessors-smart-2.5.2.jar | asm-9.7.1.jar | spring-core-6.2.19.jar

spring-jcl-6.2.19.jar | spring-security-web-6.5.11.jar | postgresql-42.7.11.jar | springdoc-openapi-starter-webmvc-ui-2.10.0.jar | springdoc-openapi-starter-webmvc-api-2.10.0.jar

springdoc-openapi-starter-common-2.10.0.jar | commons-lang3-3.17.0.jar | swagger-core-jakarta-2.2.55.jar | swagger-annotations-jakarta-2.2.55.jar | swagger-models-jakarta-2.2.55.jar

jackson-dataformat-yaml-2.21.4.jar | swagger-ui-5.34.0.jar | webjars-locator-lite-1.1.3.jar | jspecify-1.0.0.jar | auxiruta-commons-1.0.0.jar

spring-boot-jarmode-tools-3.5.16.jar

assistance-service/target/app.jar.original

Ruta: assistance-service/target/app.jar.original

Objetivo y funcionalidad

Conserva el JAR de assistance-service anterior al reempaquetado Boot: clases y recursos propios, sin el ensamblado ejecutable completo con dependencias.

Tamaño: 0.15 MiB; 102 entradas; 78 clases del proyecto; 0 dependencias anidadas.

JAR previo al reempaquetado Boot: clases/recursos sin ensamblado completo de dependencias; Docker usa app.jar.

Manifest real

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.4.2
Build-Jdk-Spec: 21
Implementation-Title: assistance-service
Implementation-Version: 1.0.0
```

Main-Class identifica el launcher; Start-Class la aplicación delegada; Build-Jdk-Spec la versión de Java de construcción, cuando esos campos están presentes.

Clases del proyecto incluidas

com/auxiruta/assistance/AssistanceApplication.class

com/auxiruta/assistance/config/ClockConfiguration.class

com/auxiruta/assistance/config/OpenApiConfiguration.class

com/auxiruta/assistance/config/RequestLoggingFilter.class

com/auxiruta/assistance/config/SecurityConfiguration.class

com/auxiruta/assistance/config/SecurityErrors.class

com/auxiruta/assistance/exceptions/ApiExceptionHandler.class

com/auxiruta/assistance/requests/AssignmentController$Rejection.class

com/auxiruta/assistance/requests/AssignmentController.class

com/auxiruta/assistance/requests/AssignmentCoordinator.class

com/auxiruta/assistance/requests/AssignmentStore$1.class

com/auxiruta/assistance/requests/AssignmentStore$Action.class

com/auxiruta/assistance/requests/AssignmentStore$View.class

com/auxiruta/assistance/requests/AssignmentStore.class

com/auxiruta/assistance/requests/AssistanceRequest.class

com/auxiruta/assistance/requests/AutomationJobs.class

com/auxiruta/assistance/requests/AutomationStore$1.class

com/auxiruta/assistance/requests/AutomationStore$Work.class

com/auxiruta/assistance/requests/AutomationStore.class

com/auxiruta/assistance/requests/CandidateEvaluation.class

com/auxiruta/assistance/requests/CandidateEvaluationRepository.class

com/auxiruta/assistance/requests/DispatchValidator$Evidence.class

com/auxiruta/assistance/requests/DispatchValidator.class

com/auxiruta/assistance/requests/DomainTypes$Mobility.class

com/auxiruta/assistance/requests/DomainTypes$Registration.class

com/auxiruta/assistance/requests/DomainTypes$Status.class

com/auxiruta/assistance/requests/DomainTypes$VehicleType.class

com/auxiruta/assistance/requests/DomainTypes.class

com/auxiruta/assistance/requests/Evaluation.class

com/auxiruta/assistance/requests/EvaluationReadService$CandidateView.class

com/auxiruta/assistance/requests/EvaluationReadService$EvaluationView.class

com/auxiruta/assistance/requests/EvaluationReadService.class

com/auxiruta/assistance/requests/EvaluationRepository.class

com/auxiruta/assistance/requests/EvaluationService.class

com/auxiruta/assistance/requests/EvaluationStore$1.class

com/auxiruta/assistance/requests/EvaluationStore$Input.class

com/auxiruta/assistance/requests/EvaluationStore.class

com/auxiruta/assistance/requests/ExecutionController$Completion.class

com/auxiruta/assistance/requests/ExecutionController.class

com/auxiruta/assistance/requests/ExecutionService.class

com/auxiruta/assistance/requests/ExecutionStore$1.class

com/auxiruta/assistance/requests/ExecutionStore$Input.class

com/auxiruta/assistance/requests/ExecutionStore$Preparation.class

com/auxiruta/assistance/requests/ExecutionStore$Step.class

com/auxiruta/assistance/requests/ExecutionStore.class

com/auxiruta/assistance/requests/HistoryRepository.class

com/auxiruta/assistance/requests/RequestController.class

com/auxiruta/assistance/requests/RequestDtos$Cancellation.class

com/auxiruta/assistance/requests/RequestDtos$Created.class

com/auxiruta/assistance/requests/RequestDtos$CreateRequest.class

com/auxiruta/assistance/requests/RequestDtos$CreateVehicle.class

com/auxiruta/assistance/requests/RequestDtos$EditVehicle.class

com/auxiruta/assistance/requests/RequestDtos$HistoryView.class

com/auxiruta/assistance/requests/RequestDtos$PageView.class

com/auxiruta/assistance/requests/RequestDtos$RegistrationChange.class

com/auxiruta/assistance/requests/RequestDtos$RequestView.class

com/auxiruta/assistance/requests/RequestDtos$VehicleView.class

com/auxiruta/assistance/requests/RequestDtos.class

com/auxiruta/assistance/requests/RequestRepository.class

com/auxiruta/assistance/requests/RequestService$1.class

com/auxiruta/assistance/requests/RequestService.class

com/auxiruta/assistance/requests/RequestStatusHistory.class

com/auxiruta/assistance/requests/RulesConfiguration$1.class

com/auxiruta/assistance/requests/RulesConfiguration.class

com/auxiruta/assistance/requests/Vehicle.class

com/auxiruta/assistance/requests/VehicleRepository.class

com/auxiruta/assistance/routing/FleetClient$1.class

com/auxiruta/assistance/routing/FleetClient$2.class

com/auxiruta/assistance/routing/FleetClient$Candidate.class

com/auxiruta/assistance/routing/FleetClient$Eligibility.class

com/auxiruta/assistance/routing/FleetClient.class

com/auxiruta/assistance/routing/OpenRouteServiceClient.class

com/auxiruta/assistance/routing/RouteProvider$Point.class

com/auxiruta/assistance/routing/RouteProvider$Route.class

com/auxiruta/assistance/routing/RouteProvider.class

com/auxiruta/assistance/routing/RoutingBudget.class

com/auxiruta/assistance/routing/RoutingFailure$Reason.class

com/auxiruta/assistance/routing/RoutingFailure.class

auxiruta-commons/target/auxiruta-commons-1.0.0.jar

Ruta: auxiruta-commons/target/auxiruta-commons-1.0.0.jar

Objetivo y funcionalidad

Distribuye clases compiladas de seguridad, errores, reloj y geografía. Los servicios incorporan esta biblioteca mediante Maven; no arranca como servidor.

Tamaño: 0.01 MiB; 17 entradas; 4 clases del proyecto; 0 dependencias anidadas.

Biblioteca regular instalada en .m2; no se inicia como servidor.

Manifest real

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.4.2
Build-Jdk-Spec: 21
Implementation-Title: auxiruta-commons
Implementation-Version: 1.0.0
```

Main-Class identifica el launcher; Start-Class la aplicación delegada; Build-Jdk-Spec la versión de Java de construcción, cuando esos campos están presentes.

Clases del proyecto incluidas

com/auxiruta/commons/errors/Problems.class

com/auxiruta/commons/GeoMath.class

com/auxiruta/commons/security/TokenSupport.class

com/auxiruta/commons/time/UtcClock.class

fleet-service/target/app.jar

Ruta: fleet-service/target/app.jar

Objetivo y funcionalidad

Ejecutable de fleet-service: reúne bytecode, recursos y dependencias en BOOT-INF. El contenedor lo inicia con java -jar; no contiene los fuentes de los tests.

Tamaño: 59.43 MiB; 272 entradas; 51 clases del proyecto; 79 dependencias anidadas.

Ejecutable Spring Boot: BOOT-INF/classes contiene clases/recursos; BOOT-INF/lib las dependencias. El launcher inicia la Start-Class. Usar java -jar con entorno y dependencias configurados.

Manifest real

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.4.2
Build-Jdk-Spec: 21
Implementation-Title: fleet-service
Implementation-Version: 1.0.0
Main-Class: org.springframework.boot.loader.launch.JarLauncher
Start-Class: com.auxiruta.fleet.FleetApplication
Spring-Boot-Version: 3.5.16
Spring-Boot-Classes: BOOT-INF/classes/
Spring-Boot-Lib: BOOT-INF/lib/
Spring-Boot-Classpath-Index: BOOT-INF/classpath.idx
Spring-Boot-Layers-Index: BOOT-INF/layers.idx
```

Main-Class identifica el launcher; Start-Class la aplicación delegada; Build-Jdk-Spec la versión de Java de construcción, cuando esos campos están presentes.

Clases del proyecto incluidas

BOOT-INF/classes/com/auxiruta/fleet/catalog/CandidateController.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/CandidateService$Candidate.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/CandidateService.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetController$AdministrationRules.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetController.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$AvailabilityChange.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$CreateOperator.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$CreateTruck.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$EditOperator.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$EditTruck.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$LinkOperator.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$Location.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$OperatorView.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$PageView.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$QualificationChange.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$RegistrationChange.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$StatusChange.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$StatusView.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$TruckView.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetService$1.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetService.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetTypes$Availability.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetTypes$Operational.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetTypes$Qualification.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetTypes$Registration.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetTypes$VehicleType.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetTypes.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/Operator.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/OperatorRepository.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationController$StatusQueries.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationController.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationJobs.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationRepository.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationService$Eligibility.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationService$Input.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationService$StatusQuery.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationService$View.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationService.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ResourceReservation.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/TowTruck.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/TowTruckRepository.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/TruckStatusHistory.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/TruckStatusHistoryRepository.class

BOOT-INF/classes/com/auxiruta/fleet/config/ClockConfiguration.class

BOOT-INF/classes/com/auxiruta/fleet/config/OpenApiConfiguration.class

BOOT-INF/classes/com/auxiruta/fleet/config/RequestLoggingFilter.class

BOOT-INF/classes/com/auxiruta/fleet/config/SecurityConfiguration.class

BOOT-INF/classes/com/auxiruta/fleet/config/SecurityErrors.class

BOOT-INF/classes/com/auxiruta/fleet/exceptions/ApiExceptionHandler.class

BOOT-INF/classes/com/auxiruta/fleet/FleetApplication.class

Dependencias empaquetadas

spring-boot-3.5.16.jar | spring-boot-autoconfigure-3.5.16.jar | logback-classic-1.5.34.jar | logback-core-1.5.34.jar | log4j-to-slf4j-2.24.3.jar

log4j-api-2.24.3.jar | jul-to-slf4j-2.0.18.jar | jakarta.annotation-api-2.1.1.jar | snakeyaml-2.4.jar | spring-security-config-6.5.11.jar

spring-aop-6.2.19.jar | spring-beans-6.2.19.jar | spring-context-6.2.19.jar | spring-security-core-6.5.11.jar | spring-security-crypto-6.5.11.jar

spring-expression-6.2.19.jar | micrometer-observation-1.15.12.jar | micrometer-commons-1.15.12.jar | spring-security-oauth2-resource-server-6.5.11.jar | spring-security-oauth2-core-6.5.11.jar

spring-security-oauth2-jose-6.5.11.jar | nimbus-jose-jwt-9.37.4.jar | jcip-annotations-1.0-1.jar | jackson-databind-2.21.4.jar | jackson-annotations-2.21.jar

jackson-core-2.21.4.jar | jackson-datatype-jdk8-2.21.4.jar | jackson-datatype-jsr310-2.21.4.jar | jackson-module-parameter-names-2.21.4.jar | tomcat-embed-core-10.1.55.jar

tomcat-embed-websocket-10.1.55.jar | spring-web-6.2.19.jar | spring-webmvc-6.2.19.jar | HikariCP-6.3.3.jar | spring-jdbc-6.2.19.jar

hibernate-core-6.6.53.Final.jar | jakarta.persistence-api-3.1.0.jar | jakarta.transaction-api-2.0.1.jar | jboss-logging-3.6.3.Final.jar | hibernate-commons-annotations-7.0.3.Final.jar

jandex-3.2.0.jar | classmate-1.7.3.jar | byte-buddy-1.17.8.jar | jaxb-runtime-4.0.9.jar | jaxb-core-4.0.9.jar

angus-activation-2.0.3.jar | txw2-4.0.9.jar | istack-commons-runtime-4.1.2.jar | jakarta.inject-api-2.0.1.jar | antlr4-runtime-4.13.2.jar

spring-data-jpa-3.5.13.jar | spring-data-commons-3.5.13.jar | spring-orm-6.2.19.jar | spring-tx-6.2.19.jar | slf4j-api-2.0.18.jar

spring-aspects-6.2.19.jar | aspectjweaver-1.9.25.1.jar | tomcat-embed-el-10.1.55.jar | hibernate-validator-8.0.3.Final.jar | jakarta.validation-api-3.0.2.jar

jakarta.xml.bind-api-4.0.5.jar | jakarta.activation-api-2.1.4.jar | spring-core-6.2.19.jar | spring-jcl-6.2.19.jar | spring-security-web-6.5.11.jar

postgresql-42.7.11.jar | springdoc-openapi-starter-webmvc-ui-2.10.0.jar | springdoc-openapi-starter-webmvc-api-2.10.0.jar | springdoc-openapi-starter-common-2.10.0.jar | commons-lang3-3.17.0.jar

swagger-core-jakarta-2.2.55.jar | swagger-annotations-jakarta-2.2.55.jar | swagger-models-jakarta-2.2.55.jar | jackson-dataformat-yaml-2.21.4.jar | swagger-ui-5.34.0.jar

webjars-locator-lite-1.1.3.jar | jspecify-1.0.0.jar | auxiruta-commons-1.0.0.jar | spring-boot-jarmode-tools-3.5.16.jar

fleet-service/target/app.jar.original

Ruta: fleet-service/target/app.jar.original

Objetivo y funcionalidad

Conserva el JAR de fleet-service anterior al reempaquetado Boot: clases y recursos propios, sin el ensamblado ejecutable completo con dependencias.

Tamaño: 0.08 MiB; 70 entradas; 51 clases del proyecto; 0 dependencias anidadas.

JAR previo al reempaquetado Boot: clases/recursos sin ensamblado completo de dependencias; Docker usa app.jar.

Manifest real

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.4.2
Build-Jdk-Spec: 21
Implementation-Title: fleet-service
Implementation-Version: 1.0.0
```

Main-Class identifica el launcher; Start-Class la aplicación delegada; Build-Jdk-Spec la versión de Java de construcción, cuando esos campos están presentes.

Clases del proyecto incluidas

com/auxiruta/fleet/catalog/CandidateController.class

com/auxiruta/fleet/catalog/CandidateService$Candidate.class

com/auxiruta/fleet/catalog/CandidateService.class

com/auxiruta/fleet/catalog/FleetController$AdministrationRules.class

com/auxiruta/fleet/catalog/FleetController.class

com/auxiruta/fleet/catalog/FleetDtos$AvailabilityChange.class

com/auxiruta/fleet/catalog/FleetDtos$CreateOperator.class

com/auxiruta/fleet/catalog/FleetDtos$CreateTruck.class

com/auxiruta/fleet/catalog/FleetDtos$EditOperator.class

com/auxiruta/fleet/catalog/FleetDtos$EditTruck.class

com/auxiruta/fleet/catalog/FleetDtos$LinkOperator.class

com/auxiruta/fleet/catalog/FleetDtos$Location.class

com/auxiruta/fleet/catalog/FleetDtos$OperatorView.class

com/auxiruta/fleet/catalog/FleetDtos$PageView.class

com/auxiruta/fleet/catalog/FleetDtos$QualificationChange.class

com/auxiruta/fleet/catalog/FleetDtos$RegistrationChange.class

com/auxiruta/fleet/catalog/FleetDtos$StatusChange.class

com/auxiruta/fleet/catalog/FleetDtos$StatusView.class

com/auxiruta/fleet/catalog/FleetDtos$TruckView.class

com/auxiruta/fleet/catalog/FleetDtos.class

com/auxiruta/fleet/catalog/FleetService$1.class

com/auxiruta/fleet/catalog/FleetService.class

com/auxiruta/fleet/catalog/FleetTypes$Availability.class

com/auxiruta/fleet/catalog/FleetTypes$Operational.class

com/auxiruta/fleet/catalog/FleetTypes$Qualification.class

com/auxiruta/fleet/catalog/FleetTypes$Registration.class

com/auxiruta/fleet/catalog/FleetTypes$VehicleType.class

com/auxiruta/fleet/catalog/FleetTypes.class

com/auxiruta/fleet/catalog/Operator.class

com/auxiruta/fleet/catalog/OperatorRepository.class

com/auxiruta/fleet/catalog/ReservationController$StatusQueries.class

com/auxiruta/fleet/catalog/ReservationController.class

com/auxiruta/fleet/catalog/ReservationJobs.class

com/auxiruta/fleet/catalog/ReservationRepository.class

com/auxiruta/fleet/catalog/ReservationService$Eligibility.class

com/auxiruta/fleet/catalog/ReservationService$Input.class

com/auxiruta/fleet/catalog/ReservationService$StatusQuery.class

com/auxiruta/fleet/catalog/ReservationService$View.class

com/auxiruta/fleet/catalog/ReservationService.class

com/auxiruta/fleet/catalog/ResourceReservation.class

com/auxiruta/fleet/catalog/TowTruck.class

com/auxiruta/fleet/catalog/TowTruckRepository.class

com/auxiruta/fleet/catalog/TruckStatusHistory.class

com/auxiruta/fleet/catalog/TruckStatusHistoryRepository.class

com/auxiruta/fleet/config/ClockConfiguration.class

com/auxiruta/fleet/config/OpenApiConfiguration.class

com/auxiruta/fleet/config/RequestLoggingFilter.class

com/auxiruta/fleet/config/SecurityConfiguration.class

com/auxiruta/fleet/config/SecurityErrors.class

com/auxiruta/fleet/exceptions/ApiExceptionHandler.class

com/auxiruta/fleet/FleetApplication.class

13. Glosario de lectura

Entidad / DTO

Entidad: objeto JPA persistente. DTO: contrato de entrada/salida que no representa una tabla.

Controller / Service / Repository / Store

Controller traduce HTTP; Service aplica reglas; Repository consulta; Store delimita operaciones transaccionales en este diseño.

JWT / sub / aud / azp

JWT firmado: sub identifica usuario, aud destinatario, azp cliente que lo obtuvo. No son contraseñas.

Rol / propiedad

Rol habilita tipo de acción; propietario/asignatario limita el recurso sobre el que puede actuar.

401 / 403 / 404 / 409 / 503

Autenticación / permiso / inexistente o ajeno / conflicto / dependencia temporal.

Snapshot

Copia de datos/reglas usados al decidir, conservada aunque el origen cambie.

Idempotencia / fingerprint

La clave identifica intención; la huella impide reutilizarla con otro contenido.

Bloqueo pesimista / optimista

Uno serializa acceso mediante lock; otro detecta cambios con @Version.

Transacción / compensación

SQL local no engloba ambos servicios. Las intenciones durables y compensaciones recuperan operaciones HTTP inciertas.

HELD / CONFIRMED

Retención temporal de propuesta / compromiso tras aceptación. Distintos del estado de solicitud y del registro de grúa.

Caché / cuota

Caché reutiliza evidencia vigente; cuota limita llamadas reales. Rutas en memoria, presupuesto persistido.

Perfil demo / usuario demo

El perfil reduce tiempos; la cuenta es identidad. No se eliminan permisos ni dependencia ORS.

JAR / original

Bytecode y recursos; el ejecutable Boot añade dependencias/launcher. original es el artefacto anterior al reempaquetado.

Referencias técnicas de consulta

`https://docs.docker.com/reference/compose-file/extension/`

`https://docs.docker.com/reference/compose-file/fragments/`

`https://docs.docker.com/engine/logging/drivers/json-file/`

Cobertura: 101 archivos propios, 6662 líneas no vacías comentadas, 28 archivos de pruebas/soporte y 7 binarios. Los restantes son Maven Wrapper.